

FANSHAWE

Process Engineering

Martin Volkening

2026-09-08

Table of contents

Introduction	1
Course schedule	2
Labs	3
Assignments	3
Quizzes	4
Tests	4
Other Grade Considerations	4
How to succeed	4
Cheating and Plagiarism	4
Use of generative AI	4
FOL	6
Accessibility	6
Fanshawe's Recording Policy	6
1 Types of Manufacturing Processes	7
1.1 Learning Objectives	7
1.2 Introduction to Manufacturing Processes	7
1.3 Process Selection Framework	8
1.4 Casting Processes	9
1.4.1 Sand Casting	10
1.4.2 Die Casting	11
1.4.3 Investment Casting (Lost-Wax)	11
1.4.4 Centrifugal Casting	12
1.4.5 Permanent Mold Casting	12
1.4.6 Shell Molding	12
1.5 Molding Processes	12
1.5.1 Injection Molding	13
1.5.2 Blow Molding	14
1.5.3 Rotational Molding	15
1.5.4 Thermoforming	15
1.5.5 Powder Metallurgy	15
1.6 Forming Processes	15
1.6.1 Forging	16

1.6.2	Stamping	16
1.6.3	Bending	17
1.6.4	Shearing	17
1.6.5	Rolling	17
1.6.6	Extrusion (Metal)	17
1.7	Machining Processes	17
1.7.1	Turning	18
1.7.2	Milling	18
1.7.3	Drilling	18
1.7.4	Grinding	18
1.7.5	Electrical Discharge Machining (EDM)	19
1.8	Joining Processes	19
1.8.1	Welding	19
1.8.2	Brazing and Soldering	20
1.8.3	Adhesive Bonding	20
1.8.4	Mechanical Fastening	20
1.9	Surface Treatment and Coating	20
1.9.1	Electroplating	21
1.9.2	Anodizing	21
1.9.3	Powder Coating	21
1.9.4	Galvanizing	21
1.10	Additive Manufacturing (3D Printing)	21
1.10.1	FDM (Fused Deposition Modeling)	22
1.10.2	SLA (Stereolithography)	22
1.10.3	SLS (Selective Laser Sintering)	22
1.10.4	Metal 3D Printing (DMLS/SLM)	22
1.11	Industry Applications	22
1.12	Sustainability in Manufacturing	23
1.13	Review Questions	24
1.14	Summary	25
1.14.1	Key Takeaways	26
1.15	References	26
2	Process and Design	27
2.1	Learning Objectives	27
2.2	Introduction to Process Engineering	27
2.3	Types of Manufacturing Automation	28
2.3.1	Fixed Automation	28
2.3.2	Programmable Automation	29
2.3.3	Flexible Automation	30
2.3.4	Automation Advantages and Disadvantages	31
2.4	Project Development Phases	31
2.4.1	Phase 1: Concept Development	32
2.4.2	Phase 2: Front-End Engineering Design (FEED)	32
2.4.3	Phase 3: Detailed Engineering	33
2.4.4	Phase 4-7: Construction, Commissioning, and Start-up	34

2.5	Process Documentation	34
2.5.1	Process Flow Diagrams (PFDs)	34
2.5.2	Piping & Instrumentation Diagrams (P&IDs)	35
2.6	Design for Manufacturing (DFM) and Assembly (DFA)	36
2.6.1	DFM Principles	36
2.6.2	DFA Analysis	37
2.7	HAZOP Studies	38
2.7.1	HAZOP Methodology	38
2.7.2	HAZOP Example	38
2.8	Cost Estimation	39
2.9	Case Study: Automotive Paint Line Project	40
2.10	Review Questions	41
2.11	Summary	43
2.11.1	Key Takeaways	43
2.12	References	43
3	Industrial Robotics	45
3.1	Learning Objectives	45
3.2	Introduction to Industrial Robotics	45
3.2.1	Why Use Robots?	46
3.2.2	Robot Market Growth	46
3.2.3	Industries Using Robots	46
3.3	Robot Configurations	47
3.3.1	Common Robot Types	47
3.3.2	Detailed Configuration Comparison	47
3.3.3	Degrees of Freedom	48
3.4	Robot Specifications	48
3.4.1	Key Specifications	48
3.4.2	Work Envelope	49
3.4.3	Payload Considerations	49
3.4.4	Repeatability vs Accuracy	50
3.5	Robot System Components	51
3.5.1	System Architecture	51
3.5.2	End Effectors (EOAT)	51
3.5.3	Gripper Selection Example	52
3.6	Coordinate Systems and Motion	52
3.6.1	Coordinate Frames	53
3.6.2	Motion Types	53
3.7	Robot Applications	54
3.7.1	Automotive Applications	54
3.7.2	Food & Beverage Applications	54
3.7.3	Aerospace/Defense Applications	54
3.8	Robot Safety	55
3.8.1	Hazard Categories	55
3.8.2	Safety Standards	55
3.8.3	Safeguarding Methods	56

3.9	Collaborative Robots (Cobots)	56
3.9.1	Cobot Characteristics	56
3.9.2	Collaborative Operation Modes (ISO/TS 15066)	56
3.9.3	Force and Pressure Limits	57
3.10	Robot Programming Basics	57
3.10.1	Programming Methods	57
3.10.2	Basic Program Structure	57
3.10.3	Key Programming Concepts	58
3.11	Video Resources	59
3.11.1	Introduction to Industrial Robots	59
3.11.2	Collaborative Robots Explained	59
3.12	Summary	59
3.13	Review Questions	59
3.14	References	69
4	Plant Layout and Facility Design	71
4.1	Learning Objectives	71
4.2	Types of Facility Layouts	72
4.2.1	Visualizing the Layout Spectrum	72
4.3	Process Layout (Functional Layout)	72
4.3.1	Characteristics of Process Layouts	73
4.3.2	Real-World Examples	74
4.4	Product Layout (Assembly Line)	74
4.4.1	Characteristics of Product Layouts	74
4.5	Hybrid Layout (Cellular Manufacturing)	75
4.5.1	Group Technology Concept	75
4.6	Fixed-Position Layout	76
4.6.1	Examples of Fixed-Position Layouts	76
4.7	Designing Process Layouts	76
4.7.1	Step 1: Gather Information	76
4.7.2	Step 2: Develop Block Plans	77
4.7.3	Interactive Example: Comparing Two Layouts	77
4.7.4	Step 3: Develop Detailed Layout	78
4.8	Designing Product Layouts (Line Balancing)	78
4.8.1	Line Balancing Steps	78
4.8.2	Key Formulas	78
4.8.3	Interactive Example: Pizza Assembly Line	79
4.8.4	Line Balancing Calculation	79
4.9	Warehouse Layout Design	81
4.9.1	Key Principles	81
4.10	Office Layout Design	81
4.10.1	Two Main Approaches	81
4.10.2	Office Ergonomics Checklist	82
4.10.3	The Future of Office Design	82
4.11	Summary: Choosing the Right Layout	83
4.11.1	Key Takeaways	83

4.12 Industry Case Studies	83
4.12.1 Case Study 1: Food Processing Plant Layout	83
4.12.2 Case Study 2: Aerospace Manufacturing Facility	85
4.12.3 Case Study 3: Automotive Supplier - Hybrid Cell Layout	86
4.13 Ergonomics in Manufacturing	86
4.13.1 The NIOSH Lifting Equation	87
4.13.2 Lifting Index Calculation	87
4.13.3 NIOSH Lifting Example	87
4.13.4 Ergonomic Workstation Design	89
4.14 Modern Layout Trends and Industry 4.0	90
4.14.1 Flexible Manufacturing Layouts	90
4.14.2 Industry 4.0 Layout Considerations	90
4.14.3 AGV and AMR Traffic Planning	91
4.15 Layout Simulation Software	91
4.15.1 Common Layout Planning Software	91
4.15.2 Simulation Benefits	92
4.16 Practice Problems	92
4.17 Review Questions	93
4.18 Chapter Summary	95
4.19 References	96
5 Simulation Modeling with AnyLogic	97
5.1 Learning Objectives	97
5.2 Introduction to Simulation Modeling	97
5.2.1 What is Simulation?	98
5.2.2 Why AnyLogic?	99
5.3 The Three Simulation Paradigms	99
5.3.1 Discrete-Event Simulation (DES)	99
5.3.2 Agent-Based Modeling (ABM)	99
5.3.3 System Dynamics (SD)	100
5.3.4 Choosing the Right Paradigm	100
5.4 The AnyLogic Interface and Libraries	100
5.4.1 The AnyLogic Workspace	100
5.4.2 Essential Libraries for Manufacturing	101
5.4.3 The Process Modeling Library	102
5.5 Production Line Simulation	102
5.5.1 Case Study: Automotive Door Panel Assembly Line	102
5.5.2 Step 1: Calculate Required Takt Time	103
5.5.3 Step 2: Define Process Parameters	103
5.5.4 Step 3: Build the Simulation Model	103
5.5.5 Step 4: Simulate and Analyze Results	103
5.5.6 Interpreting the Results	106
5.5.7 Step 5: Test Improvement Scenarios	107
5.6 Warehouse Simulation: Picking and Order Fulfillment	108
5.6.1 The Warehouse Challenge	108
5.6.2 Case Study: E-Commerce Distribution Center	109

5.6.3	Picking Strategies Compared	109
5.6.4	Simulating Picker Movement	109
5.6.5	Autonomous Mobile Robots (AMRs) in Warehouses	111
5.7	Built-in AnyLogic Examples	112
5.7.1	Manufacturing Examples	112
5.7.2	Warehouse and Logistics Examples	113
5.8	Building Your First Model: Step-by-Step Tutorial	113
5.8.1	Tutorial: Three-Station Production Line	113
5.9	Advanced Topics	116
5.9.1	3D Visualization	116
5.9.2	Integration with Real Data	116
5.9.3	Optimization Experiments	117
5.10	Key Takeaways	118
5.11	Practice Problems	119
5.11.1	Problem 1: Takt Time Calculation	119
5.11.2	Problem 2: Simulation Model Design	119
5.11.3	Problem 3: Warehouse Picking Analysis	119
5.11.4	Problem 4: AMR Fleet Sizing	119
5.12	Further Resources	120
5.12.1	AnyLogic Official Resources	120
5.12.2	Academic Resources	120
5.12.3	Industry Case Studies	120
6	Integrated Manufacturing Systems	121
6.1	Learning Objectives	121
6.2	Material Handling Methods & Systems	121
6.3	Fundamentals of Production Lines	122
6.3.1	Methods of Work Transport	123
6.3.2	Work Cycle Types	124
6.4	Key Time Metrics	124
6.4.1	Takt Time vs. Cycle Time	124
6.4.2	Manufacturing Cycle Time and Efficiency	126
6.5	Manual Assembly Lines	127
6.5.1	Line Balancing	127
6.5.2	Line Balancing Example: Electronics Assembly	128
6.6	Automated Production Lines	130
6.6.1	Impact of Station Failures	130
6.6.2	Buffer Sizing	132
6.7	Cellular Manufacturing	133
6.7.1	Part Families and Machine Cells	133
6.8	Flexible Manufacturing Systems (FMS)	134
6.8.1	FMS Components	134
6.9	Modern Manufacturing Concepts	135
6.9.1	SMED (Single-Minute Exchange of Die)	135
6.9.2	AGV and AMR Systems	136
6.9.3	Digital Twin	136

6.10 Industry Applications	137
6.11 Review Questions	137
6.12 Summary	139
6.12.1 Key Takeaways	139
6.13 References	140
7 Lean Manufacturing	141
7.1 Learning Objectives	141
7.2 Introduction to Lean Manufacturing	141
7.2.1 Definition of Lean	142
7.3 Evolution of Manufacturing Systems	142
7.3.1 Comparison of Manufacturing Systems	142
7.4 The Toyota Production System (TPS)	143
7.4.1 Historical Context	144
7.4.2 Two Pillars of TPS	145
7.5 The 3 M's of Lean	145
7.6 The Seven Types of Waste (MUDA)	146
7.6.1 Detailed Breakdown of the Seven Wastes	147
7.7 The 5S Methodology	148
7.7.1 Visual Factory	149
7.8 Just-in-Time (JIT) Production	150
7.8.1 The JIT Philosophy	150
7.8.2 Push vs. Pull Systems	151
7.8.3 Kanban System	151
7.9 Key Time Metrics	152
7.9.1 Definitions	152
7.9.2 Formulas and Calculations	153
7.9.3 Interactive Example: Calculating Takt Time	153
7.10 Setup Reduction (SMED)	154
7.10.1 The SMED Concept	154
7.10.2 SMED Methodology	154
7.11 Kaizen: Continuous Improvement	155
7.11.1 Kaizen Principles	156
7.11.2 Key Kaizen Concepts	156
7.12 Value Stream Mapping (VSM)	157
7.12.1 VSM Symbols	157
7.13 Summary	157
7.14 Review Questions	157
7.15 References	158
8 Ergonomics in Automated Manufacturing	159
8.1 Learning Objectives	159
8.2 What is Ergonomics?	159
8.2.1 Why Ergonomics Matters in Manufacturing	161
8.3 The Seven Ergonomic Risk Factors	161
8.3.1 Risk Factor 1: Awkward Postures	162

8.3.2	Risk Factor 2: Repetition	163
8.3.3	Risk Factor 3: Force	164
8.4	Musculoskeletal Disorders (MSDs)	165
8.4.1	The MSD Development Cycle	166
8.5	Preventing MSDs: Best Practices	166
8.5.1	Proper Lifting Technique	167
8.6	Ergonomic Design Principles	168
8.6.1	Anthropometry: Designing for Human Dimensions	170
8.6.2	Work Surface Height Guidelines	171
8.7	Ergonomic Assessment Tools	171
8.7.1	REBA: Rapid Entire Body Assessment	172
8.7.2	REBA Scoring Tables	172
8.7.3	REBA Employee Assessment Worksheet	173
8.7.4	REBA Example Calculation: Step-by-Step Walkthrough	178
8.7.5	REBA Example Calculation	182
8.7.6	NIOSH Lifting Equation	184
8.8	AI-Powered Ergonomic Assessment	186
8.8.1	Benefits of AI Ergonomic Assessment	188
8.9	Ergonomics in Automated Manufacturing	188
8.9.1	Human-Automation Interaction Points	189
8.9.2	Collaborative Robot Ergonomics	189
8.10	Employer Responsibilities	190
8.11	Case Study: Ergonomics in Automated Assembly	191
8.12	Summary	192
8.13	Review Questions	193
8.14	Additional Resources	195
8.14.1	Videos	195
8.14.2	Online Tools	195
8.15	References	195
9	Machine Guarding and Functional Safety	197
9.1	Learning Objectives	197
9.2	Introduction: The Safety Hierarchy	197
9.3	Physical Guarding and Interlocks	199
9.3.1	Fixed Guards	199
9.3.2	Interlocked Guards	200
9.3.3	Stop Categories	200
9.3.4	Trapped Key Systems (Lock-Out/Tag-Out)	200
9.4	Presence-Sensing Devices	201
9.4.1	Light Curtains	202
9.4.2	Laser Area Scanners	203
9.4.3	Pressure-Sensitive Safety Mats	204
9.5	Control-Actuated Safety Devices	205
9.5.1	Two-Hand Controls	205
9.5.2	Enable/Deadman Switches	206
9.6	Collaborative Robotics (Cobots)	206

9.6.1	Collaborative Operation Methods	207
9.6.2	Power and Force Limiting (PFL)	207
9.7	Safety Distance Calculation	208
9.7.1	The Safety Distance Formula	208
9.7.2	Standard Values for K	209
9.7.3	Penetration Depth (C)	209
9.7.4	Example Calculation	210
9.8	Emergency Stop (E-Stop) Systems	211
9.8.1	E-Stop Pull Cords	212
9.9	Functional Safety Standards	212
9.9.1	Safety Integrity Levels (SIL) - IEC 62061	212
9.9.2	Performance Levels (PL) - ISO 13849-1	213
9.10	Risk Assessment Process	214
9.10.1	Risk Estimation	214
9.11	Summary	215
9.12	Review Questions	216
9.13	References	217
10	Statistical Process Control (SPC)	219
10.1	Learning Objectives	219
10.2	Introduction to SPC	219
10.2.1	What is Statistical Process Control?	219
10.2.2	Brief History of SPC	220
10.2.3	Why SPC Matters	221
10.3	Understanding Variation	221
10.3.1	Common Cause vs. Special Cause Variation	222
10.3.2	The Two Mistakes in Process Control	222
10.3.3	When is a Process “In Control”?	223
10.4	Control Charts: The Foundation of SPC	224
10.4.1	What is a Control Chart?	224
10.4.2	Control Chart Components	225
10.4.3	Why ± 3 Sigma?	225
10.4.4	Choosing the Right Control Chart	226
10.5	Variable Control Charts	227
10.5.1	X-bar and R Chart	227
10.5.2	Control Chart Constants	230
10.5.3	Individual and Moving Range (I-MR) Chart	231
10.6	Attribute Control Charts	232
10.6.1	p Chart (Proportion Defective)	232
10.6.2	c Chart (Count of Defects)	233
10.7	Interpreting Control Charts	234
10.7.1	Out-of-Control Signals (Western Electric Rules)	234
10.7.2	Additional Patterns to Watch For	235
10.8	Process Capability	236
10.8.1	What is Process Capability?	236
10.8.2	Capability Indices	236

10.8.3 Cp vs. Cpk: Why Both Matter	237
10.8.4 Capability Example Calculation	237
10.8.5 Capability Index Interpretation	238
10.9 Responding to Out-of-Control Signals	239
10.9.1 The Response Process	239
10.9.2 Investigation Checklist (5 Ms + E)	240
10.10 SPC Implementation	240
10.10.1 Steps to Implement SPC	240
10.10.2 Common SPC Mistakes	240
10.11 Summary	241
10.12 Review Questions	241
10.12.1 Conceptual Questions	241
10.12.2 Calculation Questions	242
10.12.3 Application Questions	243
10.13 References	245
11 Measurement Systems Analysis	247
11.1 Learning Objectives	247
11.2 Introduction to Measurement Systems Analysis	247
11.2.1 Why MSA Matters	248
11.2.2 The Cost of Poor Measurement	248
11.2.3 MSA in Industry Standards	248
11.3 Components of Measurement Variation	249
11.3.1 The Measurement System Model	249
11.3.2 Accuracy vs. Precision	249
11.4 Accuracy Studies	250
11.4.1 Bias	250
11.4.2 Linearity	251
11.4.3 Stability	252
11.4.4 Stability Control Chart	253
11.5 Precision Studies: Gauge R&R	254
11.5.1 Repeatability vs. Reproducibility	254
11.5.2 Gauge R&R Study Design	254
11.5.3 Range Method (Short Form)	255
11.5.4 ANOVA Method (Full Study)	256
11.5.5 Calculating %GRR and Acceptance Criteria	260
11.5.6 Gauge R&R Acceptance Criteria Summary	261
11.5.7 Visual Analysis of Gauge R&R	262
11.6 Attribute Measurement Systems	262
11.6.1 Kappa Statistic	263
11.6.2 Attribute Agreement Study Example	264
11.6.3 Calculating Kappa	266
11.6.4 Attribute MSA Best Practices	267
11.7 Measurement System Resolution	267
11.7.1 Rule of Ten	267
11.7.2 Resolution Impact on Gauge R&R	268

11.8	Common Sources of Measurement Error	269
11.9	MSA and SPC Relationship	269
11.9.1	Effect on Control Charts	270
11.9.2	Effect on Capability Studies	270
11.10	Conducting an MSA Study: Step-by-Step	272
11.10.1	Data Collection Sheet Template	272
11.11	Video Resources	273
11.11.1	Understanding Gauge R&R	273
11.11.2	MSA in Practice	273
11.12	Summary	273
11.13	Review Questions	273
11.14	References	280
12	Working Effectively with AI	283
12.1	Learning Objectives	283
12.2	Introduction: AI in the Modern Workplace	283
12.2.1	The Education Paradox	284
12.2.2	AI Impact by Industry	284
12.3	Prompt Engineering Fundamentals	285
12.3.1	The Seven Rules of Effective Prompts	286
12.4	Prompt Quality Comparison: Bad, Medium, and Good	286
12.4.1	Example 1: Motor Selection	286
12.4.2	Example 2: Troubleshooting a PLC System	286
12.4.3	Example 3: Safety Analysis	287
12.5	Anatomy of an Excellent Engineering Prompt	287
12.6	Interactive Exercise: Rate These Prompts	288
12.6.1	Exercise 1: Pump Selection	288
12.6.2	Exercise 2: Identify the Quality Level	289
12.7	The Four Steps of Effective AI Use	289
12.7.1	Step 1: Define the Task	289
12.7.2	Step 2: Prompt with Context	290
12.7.3	Step 3: Iterate and Refine	290
12.7.4	Step 4: Validate and Apply	290
12.8	AI Tools and Their Use Cases	291
12.8.1	Chat Interfaces (Claude.ai, ChatGPT, Gemini)	292
12.8.2	NotebookLM (Google)	293
12.8.3	Claude Code (Command Line Tool)	293
12.8.4	Browser Extensions (Claude Extension, etc.)	294
12.8.5	Choosing the Right Tool	294
12.9	The Iterative Approach: Multi-Turn AI Collaboration	295
12.9.1	The Iteration Cycle	295
12.9.2	Worked Example: Designing a Conveyor Control System	295
12.10	Practice Topics for Iterative AI Sessions	297
12.10.1	Beginner Level	297
12.10.2	Intermediate Level	297
12.10.3	Advanced Level	297

12.11	Critical Evaluation of AI Outputs	298
12.11.1	Red Flags in AI Responses	298
12.11.2	Verification Checklist	298
12.12	When NOT to Use AI	299
12.12.1	AI Excels At:	299
12.12.2	AI Struggles With:	299
12.13	Ethical Considerations	300
12.13.1	Key Principles	300
12.13.2	Industry-Specific Considerations	300
12.14	Hands-On Lab Exercise	301
12.14.1	Objective	301
12.14.2	Scenario	301
12.14.3	Instructions	301
12.14.4	Deliverables	302
12.15	Summary	302
12.15.1	Key Takeaways	302
12.16	Review Questions	303
12.17	References and Further Reading	303
12.18	Glossary	304
13	AI Tools for Manufacturing Engineering	305
13.1	Chapter Overview	305
13.2	Week 1 — AI Foundations for Manufacturing Engineers	306
13.2.1	Learning Objectives	306
13.2.2	The AI Landscape	307
13.2.3	How LLMs Actually Work — Enough to Use Them Well	311
13.2.4	Why LLMs Fail	312
13.2.5	Intellectual Property Risk in Industrial Settings	313
13.2.6	Prompt Engineering — The R-C-T-C-F Pattern	315
13.2.7	Self-Check Questions — Week 1	316
13.2.8	Further Reading — Week 1	317
13.3	Week 2 — Manufacturing Data and Statistical Thinking	317
13.3.1	Learning Objectives	317
13.3.2	Where Plant Data Comes From	318
13.3.3	Why Industrial Data Is Messy	320
13.3.4	Statistical Process Control	323
13.3.5	Where SPC Fails	327
13.3.6	Self-Check Questions — Week 2	328
13.3.7	Further Reading — Week 2	328
13.4	Week 3 — Machine Learning and Predictive Maintenance	328
13.4.1	Learning Objectives	328
13.4.2	The Economics of Maintenance	329
13.4.3	The NASA C-MAPSS Dataset	333
13.4.4	The Machine Learning Problem: From Data to Prediction	334
13.4.5	The Fatal Mistake: Row-Wise Train/Test Splitting	336
13.4.6	Random Forest — Intuition and Why It Works	336

13.4.7	Evaluation — Measuring Whether a Model Is Useful . . .	339
13.4.8	Self-Check Questions — Week 3	340
13.4.9	Further Reading — Week 3	340
13.5	Week 4 — Feature Engineering and Model Building	341
13.5.1	Learning Objectives	341
13.5.2	Feature Engineering for Time-Series Data	342
13.5.3	Random Forest Internals	344
13.5.4	Evaluation Done Right	345
13.5.5	Self-Check Questions — Week 4	346
13.5.6	Further Reading — Week 4	346
13.6	Week 5 — Deploying AI in a Manufacturing Context	347
13.6.1	Learning Objectives	347
13.6.2	The OT/IT Boundary	348
13.6.3	Dashboards and Operators	349
13.6.4	Model Drift and the Production AI Lifecycle	350
13.6.5	Self-Check Questions — Week 5	353
13.6.6	Further Reading — Week 5	353
13.7	Week 6 — Ethics, the Broader AI Landscape, and the Road Ahead	354
13.7.1	Learning Objectives	354
13.7.2	AI Ethics in Manufacturing	354
13.7.3	The Broader AI Landscape — Where This Is Going . . .	357
13.7.4	Self-Check Questions — Week 6	358
13.7.5	Further Reading — Week 6	359
13.8	Key Takeaways	359
14	Total Productive Maintenance (TPM)	361
14.1	Learning Objectives	361
14.2	Introduction to TPM	361
14.2.1	What is Total Productive Maintenance?	361
14.2.2	History of TPM	362
14.2.3	The Goals of TPM	362
14.2.4	Traditional vs. TPM Approach	363
14.3	The Eight Pillars of TPM	364
14.4	Overall Equipment Effectiveness (OEE)	365
14.4.1	What is OEE?	365
14.4.2	OEE Component Formulas	365
14.4.3	OEE Calculation Example	366
14.4.4	OEE Benchmarks	368
14.5	The Six Big Losses	368
14.5.1	Typical Loss Distribution	370
14.6	Autonomous Maintenance	370
14.6.1	The Seven Steps of Autonomous Maintenance	371
14.6.2	The AM Tag System	371
14.6.3	Example: AM Cleaning and Inspection Checklist	372
14.7	Planned Maintenance	372
14.7.1	Types of Maintenance	373

14.7.2	Predictive Maintenance Technologies	373
14.7.3	The P-F Curve	374
14.8	Focused Improvement (Kobetsu Kaizen)	375
14.8.1	The Kaizen Process	375
14.8.2	Loss Elimination Tools	377
14.9	TPM Implementation	381
14.9.1	The 12 Steps of TPM Implementation	381
14.9.2	Success Factors	382
14.10	Case Study: TPM Implementation	382
14.10.1	Background	382
14.10.2	Problem Analysis	382
14.10.3	Improvements Implemented	383
14.10.4	Results After 18 Months	383
14.11	Summary	385
14.12	Review Questions	385
14.12.1	Conceptual Questions	385
14.12.2	Calculation Questions	386
14.12.3	Application Questions	387
14.13	References	389
15	Troubleshooting and Root Cause Analysis	391
15.1	Learning Objectives	391
15.2	Introduction to Troubleshooting	391
15.2.1	The Cost of Downtime	391
15.2.2	Reactive vs. Proactive Problem Solving	392
15.3	Systematic Troubleshooting Methodology	393
15.3.1	The DMAIC Troubleshooting Framework	393
15.3.2	Step 1: Define the Problem	393
15.3.3	Problem Statement Template	394
15.3.4	Step 2: Gather Data and Observations	394
15.3.5	The “Change Analysis” Approach	394
15.4	Understanding Root Cause vs. Symptoms	395
15.4.1	The Causation Hierarchy	395
15.4.2	Why Finding Root Cause Matters	396
15.5	The 5 Whys Technique	396
15.5.1	How It Works	396
15.5.2	5 Whys Example: Production Line Stoppage	397
15.5.3	5 Whys Template	397
15.5.4	Multiple Branches in 5 Whys	397
15.6	Ishikawa (Fishbone) Diagram	398
15.6.1	The 6M Categories	399
15.6.2	Constructing a Fishbone Diagram	399
15.6.3	Fishbone Diagram Best Practices	399
15.6.4	Video: How to Create a Fishbone Diagram	400
15.7	Fault Tree Analysis (FTA)	400
15.7.1	FTA Symbols	400

15.7.2	FTA Example: Pump Fails to Deliver Flow	401
15.7.3	Calculating Probability with FTA	401
15.8	Pareto Analysis	402
15.8.1	Creating a Pareto Chart	402
15.8.2	Interpreting Pareto Analysis	403
15.8.3	Stratified Pareto Analysis	403
15.9	Is/Is Not Analysis	404
15.9.1	Is/Is Not Matrix	404
15.9.2	Key Insights from Is/Is Not Analysis	404
15.10	The 8D Problem-Solving Methodology	404
15.10.1	Detailed 8D Steps	405
15.10.2	D3: Containment Actions	405
15.11	Corrective Action Implementation	406
15.11.1	Types of Corrective Actions	406
15.11.2	SMART Corrective Actions	406
15.12	Verification and Validation	407
15.12.1	Verification vs. Validation	407
15.12.2	Effectiveness Monitoring	407
15.13	Case Study: Automotive Welding Defects	408
15.13.1	The Problem	408
15.13.2	Initial Problem Definition (5W2H)	408
15.13.3	5 Whys Analysis	408
15.13.4	Fishbone Diagram Findings	409
15.13.5	8D Report Summary	409
15.14	Documentation and Knowledge Management	409
15.14.1	Key Documentation Elements	410
15.14.2	Creating a Troubleshooting Knowledge Base	410
15.15	Troubleshooting Tools Quick Reference	410
15.16	Video Resources	411
15.16.1	Understanding 5 Whys	411
15.16.2	8D Problem Solving	411
15.17	Summary	411
15.18	Review Questions	411
15.19	References	417
16	Industrial Automation Fundamentals	419
16.1	Learning Objectives	419
16.2	Introduction to Industrial Automation	419
16.2.1	Why Automate?	420
16.2.2	Automation in Key Industries	420
16.3	Types of Automation	420
16.3.1	Automation Classification	421
16.3.2	The Automation Pyramid	422
16.4	Programmable Logic Controllers (PLCs)	423
16.4.1	What is a PLC?	423
16.4.2	PLC Hardware Components	423

16.4.3	The PLC Scan Cycle	423
16.4.4	PLC Programming Languages (IEC 61131-3)	424
16.4.5	Ladder Logic Example	425
16.5	Industrial Sensors	426
16.5.1	Sensor Classification	426
16.5.2	Sensor Wiring: PNP vs NPN	427
16.6	Actuators and Motor Control	427
16.6.1	Types of Industrial Actuators	427
16.6.2	Variable Frequency Drives (VFDs)	427
16.6.3	VFD Benefits	429
16.6.4	Servo Systems	429
16.7	Industrial Communication Networks	430
16.7.1	Network Architecture	430
16.7.2	Common Industrial Protocols	430
16.8	Human-Machine Interface (HMI) and SCADA	431
16.8.1	HMI Overview	431
16.8.2	HMI Design Best Practices	432
16.8.3	SCADA Systems	432
16.9	Safety in Automation	433
16.9.1	Safety System Architecture	433
16.9.2	Safety Integrity Levels (SIL)	433
16.10	Industry 4.0 and IIoT	434
16.10.1	Industry 4.0 Technologies	434
16.10.2	IIoT Architecture	434
16.10.3	Predictive Maintenance Example	434
16.11	Video Resources	436
16.11.1	Introduction to PLCs	436
16.11.2	Industrial Networks Explained	436
16.12	Summary	436
16.13	Review Questions	437
16.14	References	443
17	Quality Management Systems	445
17.1	Learning Objectives	445
17.2	Introduction to Quality Management Systems	445
17.2.1	Why Implement a QMS?	446
17.2.2	The Evolution of Quality Management	446
17.2.3	Quality Management Principles	446
17.3	ISO 9001:2015 Structure	447
17.3.1	The Ten Clauses	448
17.3.2	The Process Approach and PDCA	448
17.3.3	Risk-Based Thinking	449
17.4	Industry-Specific Quality Standards	450
17.4.1	Comparison of Major Standards	450
17.4.2	IATF 16949:2016 (Automotive)	450
17.4.3	Automotive Core Tools	451

17.4.4 AS9100D (Aerospace/Defense)	457
17.4.5 ISO 22000:2018 (Food Safety)	458
17.5 Documentation Requirements	458
17.5.1 Documentation Hierarchy	459
17.5.2 Document Control Requirements	459
17.5.3 Documented Information Required by ISO 9001:2015	459
17.6 Internal Auditing	460
17.6.1 The Audit Process	461
17.6.2 Audit Checklist Example	461
17.6.3 Types of Audit Findings	461
17.6.4 Auditor Competence	461
17.7 Corrective and Preventive Action (CAPA)	462
17.7.1 The CAPA Process	462
17.7.2 CAPA Effectiveness Criteria	467
17.8 Management Review	469
17.8.1 Management Review Inputs	469
17.8.2 Management Review Outputs	469
17.8.3 Sample Management Review Dashboard	470
17.9 Continuous Improvement	470
17.9.1 Improvement Methodologies	470
17.9.2 Key Performance Indicators (KPIs)	471
17.9.3 Cost of Quality (COQ)	471
17.10 Integration with Quality Tools	473
17.10.1 Quality Tools Integration Map	473
17.10.2 Linking Tools to QMS Requirements	474
17.11 Certification Process	474
17.11.1 The Certification Journey	474
17.11.2 Certification Audit Types	474
17.12 Video Resources	475
17.12.1 ISO 9001:2015 Overview	475
17.12.2 Internal Auditing Best Practices	475
17.13 Summary	475
17.14 Review Questions	475
17.15 References	483
18 Process Failure Mode and Effects Analysis (PFMEA)	485
18.1 Learning Objectives	485
18.2 Introduction to PFMEA	485
18.2.1 What is PFMEA?	485
18.2.2 Brief History of FMEA	486
18.2.3 DFMEA vs. PFMEA	486
18.2.4 Why PFMEA Matters	488
18.3 PFMEA Fundamentals	488
18.3.1 Core Objectives	488
18.3.2 The PFMEA Team	489
18.3.3 Key Terminology	490

18.3.4	The PFMEA Form	490
18.4	The PFMEA Methodology: 10 Steps	490
18.4.1	Step 1: Define Scope and Process Boundaries	491
18.4.2	Step 2: Process Mapping and Identifying Functions	491
18.4.3	Step 3: Identifying Potential Failure Modes	492
18.4.4	Step 4: Analyzing Failure Effects	493
18.4.5	Step 5: Determining Severity Ratings	493
18.4.6	Step 6: Identifying Potential Causes	494
18.4.7	Step 7: Rating Occurrence	494
18.4.8	Step 8: Current Controls and Detection Ratings	495
18.4.9	Step 9: Calculating RPN and Prioritization	496
18.4.10	Limitations of RPN	497
18.4.11	Action Priority (AP) Method — AIAG-VDA Alternative	498
18.4.12	Step 10: Recommended Actions and Follow-Up	498
18.5	Complete PFMEA Example: Injection Molding	499
18.6	Industry-Specific PFMEA Examples	500
18.6.1	Example: Welding Process	500
18.6.2	Example: Heat Treatment Process	501
18.6.3	Example: Food Processing (Pasteurization)	501
18.7	Integration with Other Quality Tools	502
18.7.1	From PFMEA to Control Plan	503
18.8	Common Pitfalls and Best Practices	503
18.8.1	Common Pitfalls to Avoid	503
18.8.2	Best Practices	504
18.9	Case Study: Preventing a Major Quality Issue	504
18.9.1	Background	504
18.9.2	PFMEA Analysis	504
18.9.3	Actions Taken	505
18.9.4	Results	505
18.10	Summary	506
18.11	Review Questions	506
18.11.1	Conceptual Questions	506
18.11.2	Calculation Questions	507
18.11.3	Application Questions	508
18.11.4	Critical Thinking Questions	509
18.12	References	510
19	Labs	513
19.1	How to use this chapter	513
19.2	Lab 1 — Logic Thinking for Robot Control: LightBot & Rocksi	516
19.2.1	Learning outcomes	517
19.2.2	Prep work	517
19.2.3	Materials & equipment	517
19.2.4	Procedure	517
19.2.5	Deliverables & submission	518
19.2.6	Rubric	518

19.2.7	Instructor notes	518
19.3	Lab 2 — Engineering Design Process Documentation	519
19.3.1	Learning outcomes	520
19.3.2	Prep work	520
19.3.3	Materials & equipment	520
19.3.4	Procedure	520
19.3.5	Deliverables & submission	521
19.3.6	Rubric	521
19.3.7	Instructor notes	522
19.4	Lab 3 — Manufacturing Facility Design & Simulation (AnyLogic)	522
19.4.1	Learning outcomes	522
19.4.2	Prep work	523
19.4.3	Materials & equipment	523
19.4.4	Procedure	523
19.4.5	Deliverables & submission	524
19.4.6	Rubric	524
19.4.7	Instructor notes	524
19.5	Lab 4 — Lean: Value Stream Mapping, Takt and Pull	525
19.5.1	Learning outcomes	525
19.5.2	Prep work	525
19.5.3	Materials & equipment	526
19.5.4	Procedure	526
19.5.5	Deliverables & submission	527
19.5.6	Rubric	527
19.5.7	Instructor notes	528
19.6	Lab 5 — Ergonomics in Automated Manufacturing (REBA & Inseer)	528
19.6.1	Learning outcomes	528
19.6.2	Prep work	529
19.6.3	Materials & equipment	529
19.6.4	Procedure	529
19.6.5	Deliverables & submission	529
19.6.6	Rubric	529
19.6.7	Instructor notes	530
19.7	Lab 6 — SPC and Gauge R&R: the Virtual Caliper	530
19.7.1	Learning outcomes	531
19.7.2	Prep work	531
19.7.3	Materials & equipment	531
19.7.4	Procedure	531
19.7.5	Deliverables & submission	537
19.7.6	Rubric	537
19.7.7	Instructor notes	538
19.8	Machine-learning module: Labs 7 to 12	538
19.9	Lab 7 — Machine Learning, Week 1: AI Literacy and Prompt Engineering	540
19.9.1	Learning outcomes	540

19.9.2	Prep work	541
19.9.3	Materials & equipment	541
19.9.4	Procedure	541
19.9.5	Deliverables & submission	542
19.9.6	Rubric	542
19.9.7	Instructor notes	543
19.10	Lab 8 — Machine Learning, Week 2: Manufacturing Data and SPC	543
19.10.1	Learning outcomes	543
19.10.2	Prep work	544
19.10.3	Materials & equipment	544
19.10.4	Procedure	544
19.10.5	Deliverables & submission	545
19.10.6	Rubric	545
19.10.7	Instructor notes	546
19.11	Lab 9 — Machine Learning, Week 3: The C-MAPSS Dataset	546
19.11.1	Learning outcomes	546
19.11.2	Prep work	546
19.11.3	Materials & equipment	546
19.11.4	Procedure	547
19.11.5	Deliverables & submission	547
19.11.6	Rubric	548
19.11.7	Instructor notes	548
19.12	Lab 10 — Machine Learning, Week 4: Building the Predictive-Maintenance Model	548
19.12.1	Learning outcomes	548
19.12.2	Prep work	548
19.12.3	Materials & equipment	549
19.12.4	Procedure	549
19.12.5	Deliverables & submission	550
19.12.6	Rubric	550
19.12.7	Instructor notes	550
19.13	Lab 11 — Machine Learning, Week 5: Deploying the App	550
19.13.1	Learning outcomes	551
19.13.2	Prep work	551
19.13.3	Materials & equipment	551
19.13.4	Procedure	551
19.13.5	Deliverables & submission	552
19.13.6	Rubric	552
19.13.7	Instructor notes	552
19.14	Lab 12 — Machine Learning, Week 6: Capstone Presentations	552
19.14.1	Learning outcomes	553
19.14.2	Prep work	553
19.14.3	Materials & equipment	553
19.14.4	Procedure	553
19.14.5	Deliverables & submission	554

19.14.6 Rubric	554
19.14.7 Instructor notes	556
19.15 Lab 13 — TPM/OEE, Root Cause and a New Dataset	556
19.15.1 Learning outcomes	557
19.15.2 Prep work	557
19.15.3 Materials & equipment	557
19.15.4 Procedure	558
19.15.5 Deliverables & submission	558
19.15.6 Rubric	559
19.15.7 Instructor notes	559
19.16 Lab 14 — Robotic Work-Cell Design in RoboDK for Web	560
19.16.1 Learning outcomes	560
19.16.2 Prep work	560
19.16.3 Materials & equipment	560
19.16.4 Procedure	561
19.16.5 Deliverables & submission	561
19.16.6 Rubric	561
19.16.7 Instructor notes	562
19.17 Teacher preparation	562
19.17.1 Per-lab prep matrix	563
19.17.2 Consolidated shopping list (buy / restock each term)	566
19.17.3 Software, licences & accounts — set up before the term	567
19.17.4 Room bookings	567
19.17.5 Appendix: instrumentation-trainer lab stream	568

Introduction

Why are we studying this?

- Defining and identifying manufacturing processes is critical to understanding the role that automation plays and how to program an industrial process line.
- What are some examples of a manufacturing process?
- What do we have or use that is manufactured?
- What is the biggest manufacturing change that you have experienced so far?
- Processes have changed throughout the centuries and have been driven by the four Industrial Revolutions.
- First Industrial Revolution – Coal in 1765
 - This transformed society from an agricultural base to an industrial base. Processes became mechanized and rather than hand made, products were manufactured. The discovery of coal and the development of the steam engine and metal forging transformed the way goods were created.
- Second Industrial Revolution – Gas in 1870 *The discovery of oil and electricity fueled the second industrial revolution. The discovery of oil led to the invention of the combustion engine and new steel and chemical based products entered the manufacturing streams. Technological advances in communication were driven by the invention of the telegraph and later the telephone. Modes of transportation developed with the invention of the airplane and automobile. These discoveries helped to drive the advent of mass production which forever changed how items were produced.
- Third Industrial Revolution – Electronics and Nuclear Energy in 1969 *The third industrial revolution is dominated by the development of electronics and the beginning of the age of computers.* Technological developments were centered around four pillars of science: telecommunications, biotechnology, information technology and energy engineering.

- Fourth Industrial Revolution – Internet and AI
 - The fourth industrial revolution describes the current era of digitalization, automation, and advanced manufacturing technology. It's characterized by the convergence of emerging technologies, such as robotics, the Internet of Things (IoT), and 3D printing.

Check out these videos:

The Four Industrial Revolutions: <https://www.youtube.com/watch?v=kpW9JcWxKq0>

and

Industry 4.0 Explained: <https://www.youtube.com/watch?v=okXk4Bnz2Lc>

What is a Fifth Industrial Revolution going to look like?

The Fifth Industrial Revolution: https://www.youtube.com/watch?v=sGI2FFm_8tw

Further Reading Whitepaper 'What is Manufacturing' by Intel

Grade items and their weights are maintained in FOL and are the single source of truth. The course schedule below shows what is assessed and when; for current weights and marks, check the FOL grade book.

Course schedule

The term is **14 weeks**. Every lab runs in the week of its number (Lab 1 in week 1, Lab 14 in week 14). Each lab's prerequisite lecture material is taught in the same week or earlier.

Wk	Lecture	Lab
1	Ch 1 Types of Manufacturing	Lab 1 — LightBot & Rocksi
2	Ch 2 Process & Design · Ch 8 Industrial Robotics	Lab 2 — Design Documentation
3	Ch 4 Plant Layout · Ch 16 AnyLogic Simulation	Lab 3 — Facility Design & Simulation
4	Ch 3 Integrated Mfg Systems · Ch 5 Lean	Lab 4 — Lean VSM, Takt and Pull
5	Ch 7 Ergonomics · Ch 6 Guarding & Safety pt 1	Lab 5 — Ergonomics

Wk	Lecture	Lab
6	Ch 10 SPC · Ch 11 MSA	Lab 6 — SPC & Gauge R&R
7	Midterm Test (in lecture time) · Ch 20 Working with AI	Lab 7 — ML Week 1
8	Ch 21 Weeks 1–2	Lab 8 — ML Week 2
9	Ch 21 Week 3 · Ch 12 TPM	Lab 9 — ML Week 3
10	Ch 21 Week 4 · Ch 15 Troubleshooting & RCA	Lab 10 — ML Week 4
11	Ch 21 Week 5 · Ch 6 Safety pt 2 · Ch 9 Automation	Lab 11 — ML Week 5 · Lab 14 released
12	Ch 21 Week 6 · Ch 13 Quality Systems	Lab 12 — ML Capstone
13	Ch 14 PFMEA	Lab 13 — TPM/OEE & RCA
14	Final Test · review	Lab 14 — RoboDK work cell due

The midterm is written during lecture time in week 7, so the week-7 lab period runs as normal. Lab 14 is a **take-home** lab: it is released in week 11 and is due at the end of week 14, and it requires a short screen recording.

This grid is a reconstruction from the course plan — confirm it against what is actually taught before relying on the week-by-week lecture pairings.

Labs

- Late penalty is 25% per lab session.
- Labs 1–6 must be completed before Week 7.
- Lab 14 is a take-home lab, released in week 11 and due at the end of week 14; it requires a short screen recording. All other labs are due at the end of the scheduled lab period.
- Bring your lab kit and PPE to every lab session.

Assignments

- 6 Assignments.
- All are FOL quizzes, with at least one week open.

Quizzes

- 6 Lecture Quizzes and 6 Lab Quizzes.
- Quizzes are given randomly throughout the term as a “pop quiz”.
- Quizzes are given at the start of the lab session (10 minute log in window) or at the end of the lecture.
- **You must be in attendance to write the quizzes.**

Tests

- During in-person lecture time (90 minutes). One midterm test in week 7, one final test in week 14.
- Both tests are cumulative and online, in-person.
- Closed-book with handwritten cheat sheet; details will be posted on FOL closer to the date.
- Kit calculators allowed.

Other Grade Considerations

In order to pass, two conditions must be met: * Overall passing grade (50%). * Attendance and submission of the mandatory labs (labs 1 through 12). Lab 13 and Lab 14 are not mandatory to pass, but are graded.

How to succeed

- Come to class and lab sessions Participate
- Complete examples given in class
- Do the homework and assigned reading
- Attend office hours

Cheating and Plagiarism

Students must write their reports and assignments in their **own words**. Whenever students take an idea or a passage from another author, they must acknowledge their debt both by using quotation marks where appropriate and by proper referencing such as footnotes or citations. This includes non-published sources, such as work completed in previous years or a classmate's work.

Use of generative AI

Fanshawe policy A136 permits generative AI in this course only where the instructor has given prior consent. This section is that consent, and it is specific:

for each assessment below, AI use is either **required**, **permitted with disclosure**, or **not permitted**. If an assessment is not listed, treat it as **not permitted** and ask.

Where AI is **permitted or required**, you must still be able to explain every line you submit. If you are asked “why does this line do that?” and cannot answer, that work does not count as yours — regardless of whether it runs correctly.

Where AI use is **permitted with disclosure**, add a short note naming the tool and what you used it for.

Assessment	AI use	Why
Lab 1 — LightBot & Rocksi	Not permitted	The outcome <i>is</i> your own logic; an AI solving the puzzle removes the lab
Lab 2 — Design Documentation	Not permitted	Already stated in the lab
Lab 3 — AnyLogic Simulation	Not permitted	Already stated in the lab
Lab 4 — Lean VSM	Permitted with disclosure	Not for the map or the takt/PCE calculations
Lab 5 — Ergonomics	Permitted with disclosure	Inseer is itself an AI tool and is required — that is a lab instrument, not a generative AI writing your report
Lab 6 — SPC & Gauge R&R	Not permitted for readings and calculations; permitted for checking your written reflection	You are the measurement system being studied
Labs 7–12 — ML module	Required	You are assessed on directing an LLM well, not on writing Python
Lab 13 — TPM/OEE & RCA	Required for the classifier task; permitted with disclosure elsewhere	Same skill, unfamiliar dataset
Lab 14 — RoboDK work cell	Permitted with disclosure	The station, program and cycle times must be your own
Assignments 1–6	Permitted with disclosure	

Assessment	AI use	Why
Pop quizzes, midterm, final	Not permitted	Closed-book, in-person

FOL

- Lectures will be posted after class, as well as supplementary material.
- Assignments and lab manuals will be posted ahead of time. You will always have at least one week's notice.
- All announcements will be made through the FOL course.

Accessibility

Please contact me if you require material in an alternate format or if any other arrangements can make this course more accessible to you.

Consider contacting Accessibility Services for specifics on accommodations that you may be eligible for. M-Fr 8:30-4:30 F2010, 519-452-4282, accessibility@fanshawec.ca

Fanshawe's Recording Policy

It is forbidden for students to record without written permission from the instructor (and) as an approved accommodation. Recording without permission or sharing a recording will result in disciplinary action.

If a class is being recorded with permission: it will be announced, and alternate means to participation will be made available to protect students' privacy.

Chapter 1

Types of Manufacturing Processes

1.1 Learning Objectives

By the end of this chapter, you will be able to:

1. Identify and classify the seven main types of manufacturing processes
 2. Select appropriate manufacturing processes based on material, volume, and precision requirements
 3. Compare cost, speed, and quality trade-offs between process types
 4. Recognize manufacturing processes used in automotive, food, and defense industries
 5. Apply process selection criteria to real-world manufacturing scenarios
-

1.2 Introduction to Manufacturing Processes

Manufacturing is defined as the conversion of raw materials into finished goods on a large scale using man and machine. **Manufacturing processes** are the specific methods used for this conversion. Based on product requirements, different types of manufacturing processes are selected to achieve the required output with optimal cost, quality, and efficiency.

Seven Main Categories of Manufacturing Processes

Each category contains multiple specific processes for different applications

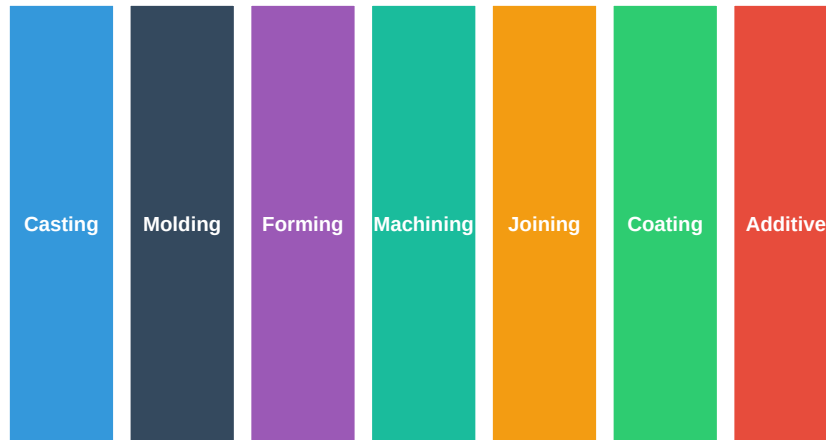


Figure 1.1: Overview of Manufacturing Process Categories

Why do we need different manufacturing processes?

Different products require different manufacturing approaches based on:

- **Material type:** Metals, plastics, ceramics, composites
- **Production volume:** One-off prototypes vs. millions of units
- **Geometric complexity:** Simple shapes vs. intricate details
- **Precision requirements:** Loose tolerances vs. micron-level accuracy
- **Cost constraints:** Budget limitations affect process selection
- **Lead time:** How quickly parts are needed

No single process is optimal for all situations!

A good video summarizing 6 of the 7 manufacturing processes is linked below (Coating/Plating is not covered):

Manufacturing Processes Overview: https://www.youtube.com/watch?v=Um_g8sQ_p3Y

1.3 Process Selection Framework

Before examining each process type, it's important to understand how engineers select the right manufacturing process. The following visualization shows the key factors:



Table 1.1: Manufacturing Process Comparison Summary

Process Type	Typical Materials	Production Volume	Typical Tolerance	Tooling Cost	Usability
Casting Molding Forming Machining Joining	Metals, alloys	Low to High	±0.5-2mm	Medium-High	Low
	Plastics, rubber	Medium to Very High	±0.1-0.5mm	High	Very Low
	Metals, sheet	Medium to Very High	±0.1-1mm	Medium-High	Low
	All materials	Low to Medium	±0.01-0.1mm	Low	High
	All materials	All volumes	Varies	Low	Medium
Coating Additive	All materials	All volumes	N/A	Low	Low
	Plastics, metals, ceramics	Prototype to Low	±0.1-0.3mm	None	High

1.4 Casting Processes

Casting is a process of pouring liquid metal into a mold containing the hollow shape of the desired outcome. It uses sprue, gates, and runners to direct the molten metal flow.

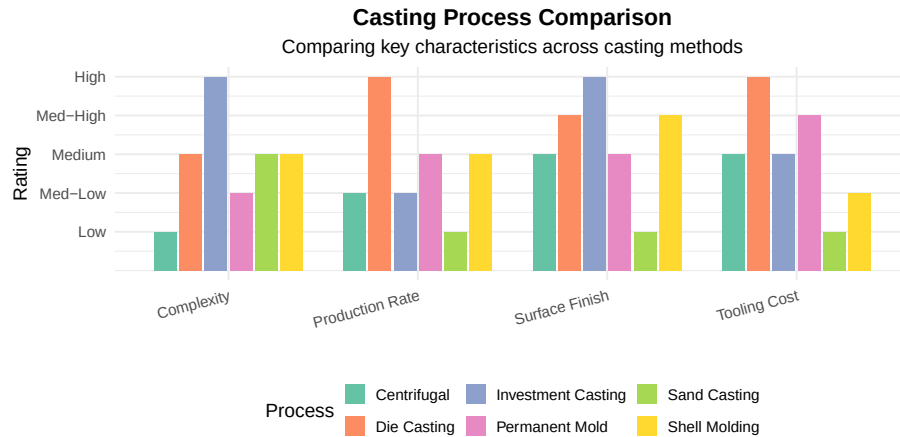


Figure 1.3: Comparison of Casting Processes

1.4.1 Sand Casting

Sand casting is a metal casting process that uses sand as the mold material. This process has a low production rate as the sand mold must be destroyed to remove the part. However, it is economical for low-volume production.

Process Steps:

1. Create a pattern (usually wood or metal)
2. Pack sand around the pattern to form the mold
3. Remove the pattern, leaving a cavity
4. Pour molten metal into the cavity
5. Allow solidification
6. Break the sand mold to remove the casting

Advantages: Low tooling cost, can produce very large parts, suitable for all metals

Disadvantages: Poor surface finish, loose tolerances, slow cycle time

Examples: Engine blocks, machine tool bases, cylinder heads, pump housings, large gears

Discussion: Why is sand casting still widely used despite its limitations?

Sand casting remains popular because:

1. **Low startup cost** - A wood pattern costs hundreds vs. steel dies costing tens of thousands
2. **Flexibility** - Easy to modify patterns for design changes
3. **Size capability** - Can cast parts weighing several tons
4. **Material versatility** - Works with nearly any metal

5. **Short lead time** - Parts can be produced in days vs. weeks for die casting

For prototypes and low-volume production (< 500 parts), sand casting is often the most economical choice even with its lower quality.

1.4.2 Die Casting

Die casting is used for producing metal parts where high precision is required. Molten metal is forced under high pressure into reusable metal dies, enabling high-volume production.

Die Casting Process: <https://www.youtube.com/watch?v=0oibUY8KUQM>
Process:

1. Steel molds (dies) are machined to the part shape
2. Dies are mounted in a die casting machine
3. Molten metal is injected under high pressure (10-175 MPa)
4. Metal solidifies rapidly due to die cooling
5. Part is ejected and the cycle repeats

Advantages: Excellent dimensional accuracy, smooth surface finish, high production rates (up to 400 shots/hour)

Disadvantages: High die cost (\$50,000-\$500,000), limited to non-ferrous metals, porosity issues

Examples: Engine components, transmission housings, electronic enclosures, power tool housings

1.4.3 Investment Casting (Lost-Wax)

Investment casting produces complex, precision parts by creating a wax pattern that is “invested” (surrounded) with ceramic slurry, then melted out to leave a mold cavity.

Investment Casting Process: <https://www.youtube.com/watch?v=QtxVdC7pBQM>

Advantages: Excellent surface finish, tight tolerances ($\pm 0.1\%$), complex geometries, any castable metal

Disadvantages: Expensive, slow cycle time, limited part size

Examples: Turbine blades, aerospace components, medical implants, jewelry, firearm components

Why do aerospace companies pay premium prices for investment casting?

Investment casting is essential for aerospace because:

1. **Complex internal passages** - Turbine blades have intricate cooling channels

2. **Superior metallurgy** - Controlled solidification produces better grain structure
3. **Near-net shape** - Minimizes machining of expensive superalloys
4. **Weight savings** - Can create hollow structures impossible with other methods
5. **Material options** - Works with titanium, Inconel, and other aerospace alloys

A single turbine blade might cost \$1,000-\$5,000, but there's no other process that can produce the required geometry and performance.

1.4.4 Centrifugal Casting

Centrifugal casting spins molten metal in a rotating die, using centrifugal force to distribute the metal and float impurities to the inner surface.

Applications: Pipes, tubes, cylinder liners, bushings, rings

1.4.5 Permanent Mold Casting

Uses reusable metal molds with gravity or low-pressure filling. Good for medium-volume production of aluminum and copper alloy parts.

Applications: Pistons, gear housings, wheels, pipe fittings

1.4.6 Shell Molding

Uses resin-coated sand that forms a thin, strong shell around a heated pattern. Better dimensional accuracy than sand casting.

Applications: Gear housings, cylinder heads, connecting rods

1.5 Molding Processes

Molding uses a rigid frame to shape hot liquid or ductile raw material. It is predominantly used for plastics and rubber products.

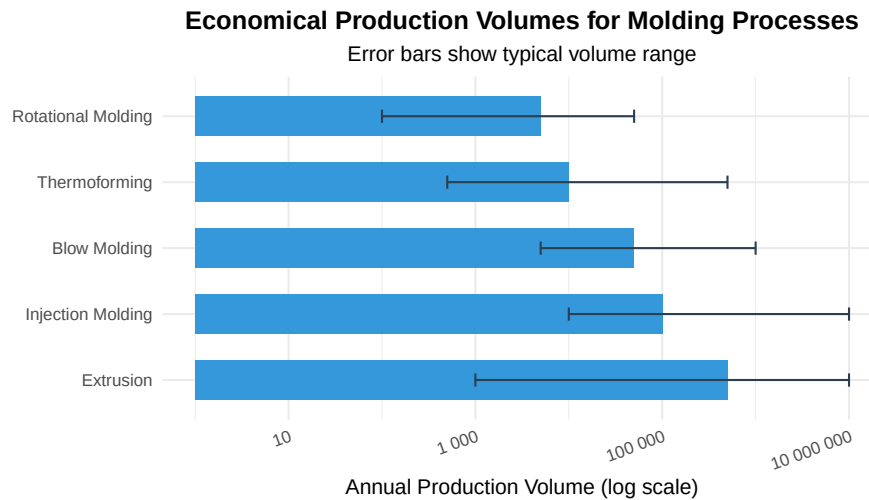


Figure 1.4: Molding Process Selection by Production Volume

1.5.1 Injection Molding

Injection molding is the most common process for manufacturing plastic parts. Plastic pellets are melted and injected under high pressure into a mold cavity.

Injection Molding Process: <https://www.youtube.com/watch?v=RMjtmr3CqA>

Process Cycle:

1. **Clamping** - Mold halves are clamped together
2. **Injection** - Molten plastic is injected into the cavity
3. **Cooling** - Part solidifies in the mold
4. **Ejection** - Part is pushed out by ejector pins

Cycle times: 10-60 seconds typical, as fast as 2 seconds for small parts

Examples: Automotive dashboards, phone cases, medical devices, food containers, toys

Live cell — try different volumes, mold cost, or cycle time.

```
#| label: injection-molding-cost
# Injection Molding Cost Analysis Example
# Comparing costs at different production volumes

mold_cost <- 50000 # Typical mold cost in dollars
material_cost_per_kg <- 2.50 # ABS plastic
part_weight_kg <- 0.1
cycle_time_sec <- 30
```

```

machine_rate_per_hour <- 75 # Machine + operator cost

# Calculate cost per part at different volumes
volumes <- c(1000, 10000, 50000, 100000, 500000)

cost_analysis <- data.frame(
  Volume = volumes,
  Mold_Per_Part = mold_cost / volumes,
  Material = part_weight_kg * material_cost_per_kg,
  Processing = (cycle_time_sec / 3600) * machine_rate_per_hour
)

cost_analysis$Total_Per_Part <- cost_analysis$Mold_Per_Part +
                                cost_analysis$Material +
                                cost_analysis$Processing

cat("Injection Molding Cost Breakdown:\n")
cat("=====\n")
for(i in 1:nrow(cost_analysis)) {
  cat(sprintf("At %s parts: %.2f per part (Tooling: %.2f, Material: %.2f, Processing: %.2f)\n",
    format(cost_analysis$Volume[i], big.mark = " "),
    cost_analysis$Total_Per_Part[i],
    cost_analysis$Mold_Per_Part[i],
    cost_analysis$Material[i],
    cost_analysis$Processing[i]))
}

```

Discussion: When does injection molding become cost-effective vs. 3D printing?

Break-even analysis:

- 3D printing: ~\$5-20 per part (no tooling, slow production)
- Injection molding: \$50,000 tooling + \$0.50-2 per part

Break-even point: Approximately 5,000-10,000 parts

Rule of thumb: - < 500 parts: 3D printing or machining - 500-5,000 parts: Consider soft tooling (aluminum molds) - > 5,000 parts: Steel injection molds become economical

1.5.2 Blow Molding

Blow molding creates hollow plastic parts by inflating a heated plastic tube (parison) inside a mold cavity.

Types: - **Extrusion blow molding** - Parison extruded, then blown - **Injection blow molding** - Preform injection molded, then blown - **Stretch blow molding** - Preform stretched and blown (PET bottles)

Examples: Bottles, containers, fuel tanks, automotive ducts, toys

1.5.3 Rotational Molding

Creates hollow parts by rotating a mold on two axes while heating, causing plastic powder to melt and coat the interior surface.

Advantages: Low tooling cost, no internal stresses, large hollow parts

Examples: Kayaks, playground equipment, tanks, coolers

1.5.4 Thermoforming

Heats a plastic sheet until pliable, then forms it over a mold using vacuum, pressure, or mechanical force.

Examples: Packaging (blister packs), disposable cups, refrigerator liners

1.5.5 Powder Metallurgy

Though involving metal, powder metallurgy uses molding principles. Metal powders are compacted in dies and sintered (heated below melting point) to fuse particles.

Advantages: Near-net shape, complex geometries, controlled porosity

Examples: Self-lubricating bearings, gears, filters, magnets

1.6 Forming Processes

Forming uses mechanical forces (compression, tension, shear) to shape material without removing any material. This preserves material grain structure and can improve mechanical properties.

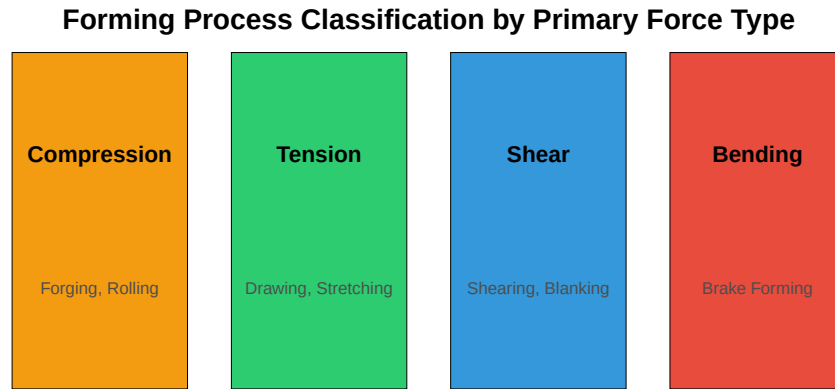


Figure 1.5: Types of Forming Forces

1.6.1 Forging

Forging uses compressive forces from hammers or presses to shape heated metal. Forged parts have superior strength due to aligned grain structure.

Forging Process: <https://www.youtube.com/watch?v=o4vQJnPwgiA>

Types of Forging: - **Open-die forging** - Simple shapes, large parts - **Closed-die (impression) forging** - Complex shapes, high volume - **Cold forging** - Room temperature, excellent finish - **Hot forging** - Heated material, easier deformation

Examples: Crankshafts, connecting rods, gears, aircraft structural components, hand tools

Why are aircraft landing gear forgings so expensive?

Aircraft landing gear forgings can cost \$100,000+ because:

1. **Material** - High-strength steel or titanium alloys
2. **Size** - Large parts require massive presses (50,000+ tons)
3. **Certification** - Extensive testing and documentation
4. **Grain flow** - Must be optimized for fatigue resistance
5. **NDT requirements** - 100% ultrasonic and magnetic particle inspection
6. **Low volume** - Only a few hundred per aircraft type per year

The forged grain structure provides 20-40% better fatigue life than equivalent castings or machined parts.

1.6.2 Stamping

Stamping uses dies and presses to form, cut, or shape sheet metal. Modern stamping lines can produce 1,000+ parts per hour.

Operations include: - **Blanking** - Cutting flat shapes from sheet - **Piercing** - Punching holes - **Drawing** - Forming cup shapes - **Bending** - Creating angles - **Coining** - Fine detail forming

Examples: Automotive body panels, appliance housings, electronic enclosures

1.6.3 Bending

Bending uses press brakes to create angular shapes in sheet metal.

Key formula:

$$\text{Bend Allowance} = \frac{\pi \times A \times (R + K \times T)}{180}$$

Where: A = bend angle, R = inside radius, T = material thickness, K = K-factor (typically 0.3-0.5)

1.6.4 Shearing

Shearing cuts sheet metal without chip formation using opposed cutting edges.

Types: Blanking, piercing, trimming, notching, slitting

1.6.5 Rolling

Reduces material thickness by passing through rotating rolls. Used for sheet, plate, and structural shapes.

1.6.6 Extrusion (Metal)

Forces material through a die to create constant cross-section profiles.

Examples: Aluminum window frames, heat sinks, structural shapes

1.7 Machining Processes

Machining removes material using cutting tools to achieve precise dimensions and surface finishes. It is the most versatile manufacturing process but typically has higher per-part costs.

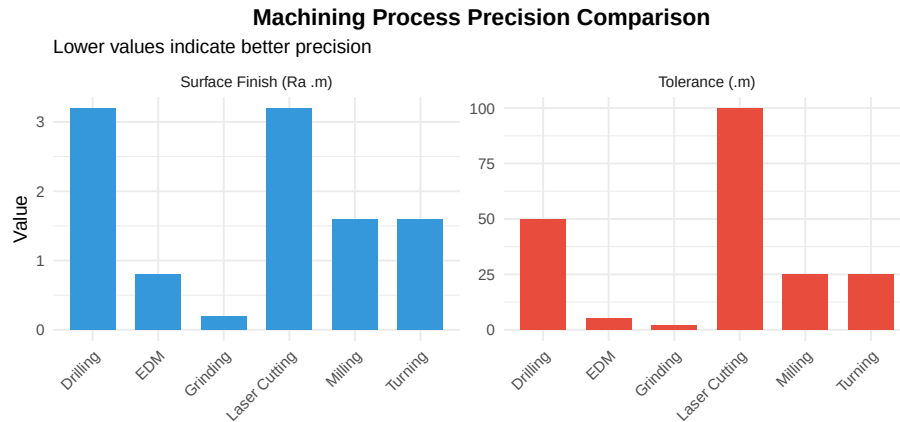


Figure 1.6: Machining Process Capabilities

1.7.1 Turning

Rotates the workpiece while a cutting tool moves to create cylindrical shapes.

Performed on lathes or turning centers

Operations: Facing, turning, boring, threading, grooving, parting

1.7.2 Milling

Rotates a cutting tool while moving the workpiece to remove material. The most versatile machining process.

Types: - **Peripheral milling** - Cutter axis parallel to surface - **Face milling** - Cutter axis perpendicular to surface - **End milling** - Creates slots, pockets, contours

1.7.3 Drilling

Creates cylindrical holes using rotating drill bits.

Related operations: Reaming (sizing), boring (enlarging), tapping (threading)

1.7.4 Grinding

Uses abrasive wheels for precision finishing. Achieves the finest tolerances and surface finishes.

Types: Surface grinding, cylindrical grinding, centerless grinding

1.7.5 Electrical Discharge Machining (EDM)

Uses electrical sparks to erode conductive materials. Essential for hardened materials and complex geometries.

Applications: Injection mold cavities, dies, aerospace components

When should you use EDM instead of conventional machining?

Choose EDM when:

- 1. **Hardened materials** - Can machine 60+ HRC steels that would destroy cutting tools
- 2. **Complex internal features** - Wire EDM cuts intricate shapes
- 3. **Tight corners** - Can achieve sharp internal corners impossible with rotary tools
- 4. **Fragile parts** - No cutting forces (non-contact process)
- 5. **Thin walls** - No deflection from tool pressure

Limitations: - Slow material removal (grams/hour vs. kg/hour for milling) - Only works on conductive materials - Recast layer may need removal - Higher cost per part

1.8 Joining Processes

Joining combines two or more components into a single assembly. Methods range from permanent (welding) to removable (fasteners).

Table 1.2: Joining Process Comparison

Process	Joint Strength	Dissimilar Metals	Automation	Typical Applications
Arc Welding	Excellent	Limited	Medium	Structural steel, pressure vessels
Resistance Welding	Excellent	Limited	High	Auto body panels, wire mesh
Laser Welding	Excellent	Limited	High	Electronics, medical devices
Brazing	Good	Yes	Medium	Carbide tooling, HVAC
Soldering	Fair	Yes	High	Electronics, plumbing
Adhesive Bonding	Good	Yes	High	Composites, glass, plastics
Mechanical Fastening	Excellent	Yes	High	All industries

1.8.1 Welding

Melts and fuses base materials together, often with added filler metal.

Common Types: - **MIG (GMAW)** - Gas Metal Arc Welding - fast, easy automation - **TIG (GTAW)** - Gas Tungsten Arc Welding - precise, clean welds

- **Stick (SMAW)** - Shielded Metal Arc Welding - versatile, field use - **Resistance** - Uses electrical resistance heating - **Laser/Electron Beam** - High precision, deep penetration

1.8.2 Brazing and Soldering

Joins metals using a filler metal with a lower melting point than the base materials. The base materials do not melt.

- **Brazing** - Filler melts above 450°C (840°F)
- **Soldering** - Filler melts below 450°C

1.8.3 Adhesive Bonding

Uses chemical adhesives to join materials. Essential for composites, glass, and dissimilar materials.

Advantages: Distributes stress, joins dissimilar materials, seals against fluids

Disadvantages: Surface preparation critical, limited temperature resistance, aging concerns

1.8.4 Mechanical Fastening

Uses bolts, screws, rivets, or clips. Allows disassembly for maintenance.

1.9 Surface Treatment and Coating

Coating and surface treatment processes protect parts from corrosion, wear, or environmental damage, and can improve appearance or electrical properties.

Table 1.3: Surface Treatment and Coating Processes

Process	Typical Thickness	Base Material	Primary Benefit	Indus
Electroplating	1-50 m	Conductive metals	Corrosion, appearance	Autom
Anodizing	5-100 m	Aluminum	Corrosion, hardness	Aerosp
Powder Coating	50-150 m	Any metal	Corrosion, appearance	Applia
Painting	25-75 m	Any	Appearance	All
Hot Dip Galvanizing	45-85 m	Steel	Corrosion	Const
PVD/CVD	1-10 m	Any	Wear, hardness	Toolin
Thermal Spray	100-500 m	Any	Wear, thermal barrier	Aerosp

1.9.1 Electroplating

Deposits metal coating using electrical current. Common coatings include chrome, nickel, zinc, gold, and silver.

1.9.2 Anodizing

Electrochemically grows an oxide layer on aluminum. Provides corrosion resistance and can be dyed various colors.

1.9.3 Powder Coating

Applies dry powder electrostatically, then cures with heat. Produces thick, durable, attractive finishes.

1.9.4 Galvanizing

Coats steel with zinc for corrosion protection. Hot-dip galvanizing provides excellent outdoor durability.

1.10 Additive Manufacturing (3D Printing)

Additive manufacturing builds parts layer-by-layer from digital models. It's the opposite of machining (subtractive manufacturing).

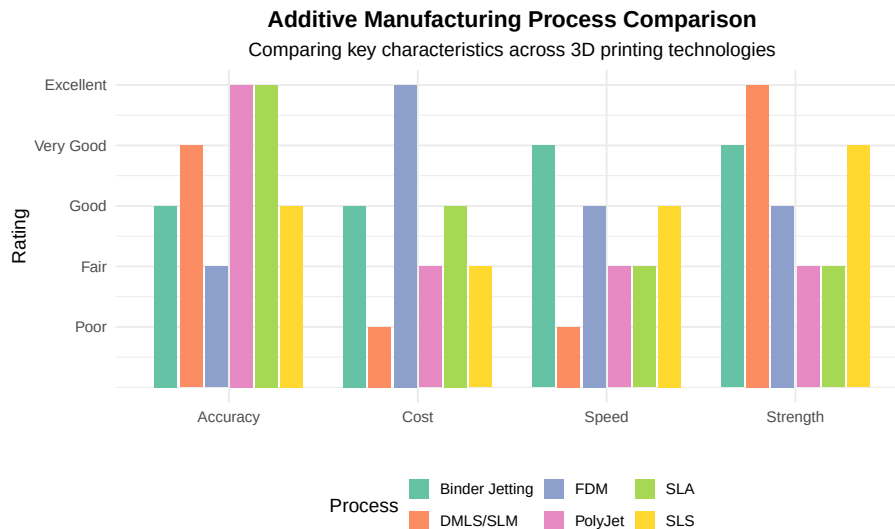


Figure 1.7: Additive Manufacturing Process Comparison

How 3D Printing Works: <https://www.youtube.com/watch?v=Vx0Z6LplaMU>

1.10.1 FDM (Fused Deposition Modeling)

Extrudes thermoplastic filament through a heated nozzle. Most common desktop 3D printing technology.

Materials: PLA, ABS, PETG, Nylon, TPU

1.10.2 SLA (Stereolithography)

Uses UV laser to cure liquid photopolymer resin. Excellent surface finish and detail.

1.10.3 SLS (Selective Laser Sintering)

Fuses powder material with a laser. Produces strong functional parts without support structures.

1.10.4 Metal 3D Printing (DMLS/SLM)

Fuses metal powder with high-power lasers. Used for aerospace, medical, and tooling applications.

When does metal 3D printing make sense vs. traditional manufacturing?

Metal 3D printing is advantageous when:

1. **Complex internal features** - Conformal cooling channels in molds
2. **Topology optimization** - Organic shapes that minimize weight
3. **Consolidation** - Combining multiple parts into one
4. **Low volume** - Too few parts to justify tooling
5. **Custom/one-off** - Medical implants, prototypes

Traditional manufacturing is better when: - High volumes (> 100-1000 parts depending on complexity) - Simple geometries - Large parts (3D print beds are limited) - Maximum strength required (forging still superior)

Cost comparison: - Simple bracket: \$5 machined vs. \$50+ 3D printed - Complex aerospace part: \$10,000 machined vs. \$3,000 3D printed

1.11 Industry Applications

Table 1.4: Manufacturing Process Applications by Industry

Process	Automotive	Food Industry	Defense
---------	------------	---------------	---------

Die Casting	Engine blocks, transmission cases	Equipment housings	Aircraft structural
Injection Molding	Dashboard, interior trim, connectors	Packaging, containers, caps	Connectors, housing
Forging	Crankshafts, connecting rods, gears	Mixer blades, pump impellers	Landing gear, turb
Stamping	Body panels, structural components	Cans, trays, lids	Aircraft skins, miss
Machining	Engine components, precision parts	Valves, fittings	Precision weapons c
Welding	Body structure, exhaust systems	Tanks, piping, conveyors	Aircraft structures,
Powder Coating	Wheels, chassis components	Equipment frames	Military vehicles, ec
3D Printing	Prototypes, custom fixtures	Custom tooling, fixtures	Turbine blades, bra

1.12 Sustainability in Manufacturing

Modern manufacturing must consider environmental impact. Here’s how different processes compare:

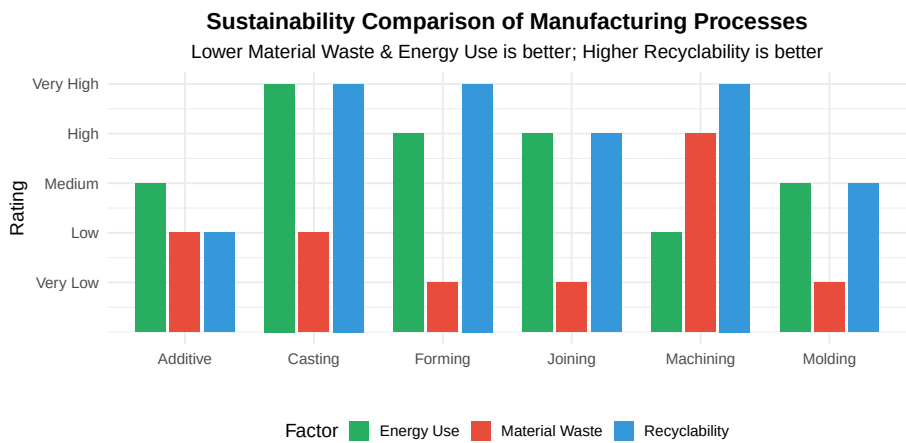


Figure 1.8: Environmental Considerations by Process Type

Discussion: How can manufacturers reduce environmental impact?

Strategies for sustainable manufacturing:

- 1. **Near-net-shape processes** - Casting, forging, and molding reduce machining waste
- 2. **Closed-loop recycling** - Reprocess scrap material on-site
- 3. **Energy-efficient equipment** - Modern machines use 30-50% less energy
- 4. **Additive manufacturing** - Only uses material where needed
- 5. **Water-based coatings** - Replace solvent-based paints
- 6. **Lean manufacturing** - Reduces overproduction and waste
- 7. **Life cycle assessment** - Consider total environmental impact

Example: Aluminum die casting recycles 95%+ of sprues and runners, while machining from billet may waste 80% of material.

1.13 Review Questions

Question 1: Process Identification

A company needs to produce 50,000 aluminum housings per year. Each housing is a complex shape with internal fins for heat dissipation. The surface must be smooth and dimensions must be within $\pm 0.3\text{mm}$. Which process would you recommend?

Answer: Die Casting is the best choice because: - High volume (50,000/year) justifies tooling cost - Aluminum is ideal for die casting - Complex internal features are possible - $\pm 0.3\text{mm}$ tolerance is achievable - Good surface finish without secondary operations

Alternative: If volumes were lower ($< 5,000$), investment casting might be considered.

Question 2: Cost Comparison

Calculate the break-even point between 3D printing and injection molding for a small plastic part: - 3D printing: \$8 per part (no tooling) - Injection molding: \$25,000 mold + \$0.50 per part

Answer: Let n = number of parts

Break-even: $\$8n = \$25,000 + \$0.50n$

$\$7.50n = \$25,000$

$n = 3,333$ parts

Below 3,333 parts: 3D printing is cheaper **Above 3,333 parts:** Injection molding is cheaper

At 10,000 parts: - 3D printing: \$80,000 - Injection molding: \$30,000

Question 3: Material Selection

Match each material/product to the most appropriate manufacturing process:

- a) Stainless steel surgical instrument
- b) Large plastic kayak
- c) Titanium jet engine bracket
- d) Aluminum beverage can

Answers:

- a) **Investment casting** or **machining** - Complex shapes, high precision, excellent surface finish
- b) **Rotational molding** - Large hollow plastic part, low tooling cost
- c) **Additive manufacturing (DMLS)** or **forging** - Complex geometry, lightweight, strength critical
- d) **Deep drawing (stamping)** - Very high volume, thin-wall cylinder

Question 4: Process Selection Scenario

You need to manufacture a new product with these requirements: -

Quantity: 200 units for initial market test - Material: ABS plastic - Size: 150mm × 100mm × 50mm - Tolerance: ±0.5mm - Surface: Smooth, will be painted

What process would you recommend and why?

Answer: 3D Printing (FDM or SLA) would be most appropriate because:

1. **Low quantity (200)** - Injection molding tooling (\$30,000+) not justified
2. **Market test phase** - Design may change based on feedback
3. **Quick turnaround** - Parts in days vs. weeks for tooling
4. **Tolerances achievable** - ±0.5mm is within FDM capabilities
5. **Post-processing acceptable** - Parts will be painted anyway

Cost comparison: - FDM: $200 \times \$15 = \$3,000$ - Injection molding: $\$30,000 + (200 \times \$1) = \$30,200$

If the product succeeds and volume increases to 10,000+, transition to injection molding.

Question 5: Hybrid Manufacturing

What is hybrid manufacturing and when is it used?

Answer: Hybrid manufacturing combines additive and subtractive processes in a single machine or workflow.

Common combinations: - 3D print near-net shape → CNC machine critical surfaces - Cast blank → Machine precision features - Forge rough shape → Machine final dimensions

Benefits: - Reduced material waste vs. pure machining - Better precision than pure additive - Faster than pure machining for complex parts

Example: Aerospace companies 3D print titanium brackets, then machine mounting surfaces to achieve tight tolerances. This can reduce material waste by 90% compared to machining from billet.

1.14 Summary

Table 1.5: Manufacturing Processes Summary

Process Category	Key Principle	Best For
Casting	Pour liquid into mold	Complex metal shapes, medium-high volume
Molding	Shape heated/softened material	Plastic parts, very high volume
Forming	Deform solid with force	Strong metal parts, high volume
Machining	Remove material with tools	Precision parts, any material
Joining	Combine multiple parts	Assembly of components
Coating	Apply protective layer	Corrosion/wear protection
Additive	Build layer by layer	Prototypes, complex geometry

1.14.1 Key Takeaways

1. **No single process is best for everything** - Selection depends on material, volume, complexity, and cost constraints
2. **Volume drives economics** - High tooling costs require high volumes to amortize
3. **Consider the complete supply chain** - Secondary operations, assembly, and finishing affect total cost
4. **Sustainability matters** - Modern manufacturing must minimize waste and energy use
5. **Hybrid approaches are growing** - Combining processes optimizes cost and performance

1.15 References

1. Groover, M.P. (2020). *Fundamentals of Modern Manufacturing* (7th ed.). Wiley.
2. Kalpakjian, S., & Schmid, S.R. (2020). *Manufacturing Engineering and Technology* (8th ed.). Pearson.
3. DeGarmo, E.P., Black, J.T., & Kohser, R.A. (2019). *DeGarmo's Materials and Processes in Manufacturing* (13th ed.). Wiley.
4. Afzal, M. "Types of Manufacturing Processes." *Engineering Articles*.

Chapter 2

Process and Design

2.1 Learning Objectives

By the end of this chapter, you will be able to:

1. Describe the stages of process plant project development from concept to operation
 2. Differentiate between fixed, programmable, and flexible automation systems
 3. Interpret Process Flow Diagrams (PFDs) and Piping & Instrumentation Diagrams (P&IDs)
 4. Explain the purpose and content of Front-End Engineering Design (FEED)
 5. Apply Design for Manufacturing (DFM) and Design for Assembly (DFA) principles
 6. Understand the role of HAZOP studies in process safety
 7. Identify key engineering deliverables for each project phase
-

2.2 Introduction to Process Engineering

Process engineering is a specialized branch of engineering that focuses on the design, optimization, and implementation of manufacturing processes and production systems. Process engineers work at the intersection of design and production, ensuring that products can be manufactured efficiently, safely, and economically.

What is the role of a process engineer?

Process engineers are responsible for:

- **Designing production processes** - Selecting equipment, defining sequences, optimizing flow
- **Improving existing processes** - Reducing waste, increasing throughput, improving quality
- **Scaling up from lab to production** - Taking R&D concepts to full-scale manufacturing
- **Ensuring safety and compliance** - Meeting regulatory and environmental requirements
- **Supporting production** - Troubleshooting issues, training operators
- **Cost optimization** - Reducing manufacturing costs while maintaining quality

2.3 Types of Manufacturing Automation

Automation is essential for achieving high productivity, consistent quality, and competitive manufacturing costs. There are three fundamental types of automation, each suited to different production scenarios.

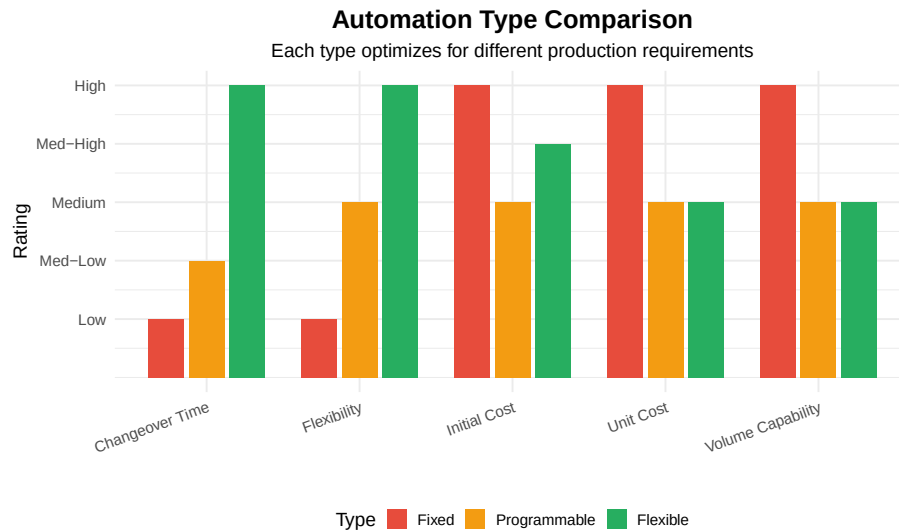


Figure 2.1: Comparison of Automation Types

2.3.1 Fixed Automation

Fixed automation (also called “hard automation”) uses specialized equipment designed for a specific product. The sequence of operations is determined by

the equipment configuration and cannot be easily changed.

Characteristics: - Very high production rates (thousands per hour) - Low per-unit cost at high volumes - High initial capital investment - Inflexible - dedicated to one product

Examples: - Automotive transfer lines for engine blocks - Beverage bottling lines - Chemical processing plants - Paper manufacturing

Live cell — try different equipment costs, production volumes, or operating costs.

```
#| label: fixed-automation-economics
# Fixed Automation Economic Analysis
# When does fixed automation make sense?

# Example: Automotive transfer line for cylinder heads
fixed_automation_cost <- 5000000 # Initial equipment cost
annual_production <- 500000      # Parts per year
fixed_cost_per_part <- 2.50      # Operating cost per part
equipment_life_years <- 10

# Calculate total cost per part including amortization
annual_equipment_cost <- fixed_automation_cost / equipment_life_years
total_annual_cost <- annual_equipment_cost + (fixed_cost_per_part * annual_production)
cost_per_part <- total_annual_cost / annual_production

cat("Fixed Automation Analysis:\n")
cat("=====\n")
cat(sprintf("Equipment cost: %s\n", format(fixed_automation_cost, big.mark = " ")))
cat(sprintf("Annual production: %s parts\n", format(annual_production, big.mark = " ")))
cat(sprintf("Operating cost per part: %.2f\n", fixed_cost_per_part))
cat(sprintf("Amortized equipment cost per part: %.2f\n", annual_equipment_cost/annual_production))
cat(sprintf("Total cost per part: %.2f\n", cost_per_part))
```

2.3.2 Programmable Automation

Programmable automation allows the production equipment to be reprogrammed for different products. It's suited for batch production where products change periodically.

Characteristics: - Medium production rates - Significant changeover time between products - Lower initial cost than fixed automation - Suitable for batch sizes of 100-10,000

Examples: - CNC machining centers - Industrial robots - PLC-controlled assembly machines - Batch chemical reactors

Discussion: When should you choose programmable over fixed automation?

Choose programmable automation when:

1. **Product variety** - You make multiple products on the same equipment
2. **Volume uncertainty** - Demand may shift between products
3. **Product evolution** - Frequent design changes expected
4. **Lower volumes** - Not enough volume to justify dedicated equipment
5. **Capital constraints** - Can't afford multiple dedicated lines

Example scenario: A machine shop that makes 20 different part numbers for aerospace customers would use CNC machines (programmable) rather than dedicated transfer lines (fixed) because: - Volumes per part number are low (100-1,000/year) - New part designs are introduced regularly - Flexibility to respond to customer changes is critical

2.3.3 Flexible Automation

Flexible automation can produce a variety of parts with minimal changeover time. It combines the flexibility of programmable automation with the productivity approaching fixed automation.

Characteristics: - Medium-high production rates - Very fast changeover (minutes, not hours) - High initial investment - Suitable for medium variety, medium volume

Examples: - Flexible Manufacturing Systems (FMS) - Robotic assembly cells - Automated guided vehicle (AGV) systems - Modern automotive final assembly lines

Table 2.1: Automation Type Selection Guide

Production Characteristic	Fixed Automation	Programmable Automation
Annual Volume	> 1,000,000 units	1,000 - 100,000 units
Product Variety	Single product	Several products in batches
Changeover Frequency	Rarely (years)	Weekly to monthly
Initial Investment	Very High (\$5M+)	Medium (\$500K-\$2M)
Operating Cost	Lowest per unit	Medium per unit
Best Industries	Automotive, Beverage, Chemical	Job shops, Aerospace, Medical

List industries where you can find various automated process systems

By automation type:

Fixed Automation: - Oil refineries - Chemical plants - Food processing (high-volume) - Beverage bottling - Automotive powertrain - Steel mills - Paper/pulp mills

Programmable Automation: - Aerospace manufacturing - Medical device production - Pharmaceutical (batch) - Job shops - Tool & die making - Defense contractors

Flexible Automation: - Electronics assembly - Automotive final assembly - Warehouse/distribution - Semiconductor fabrication - Consumer appliances - Furniture manufacturing

2.3.4 Automation Advantages and Disadvantages

Table 2.2: Advantages and Disadvantages of Manufacturing Automation

Category	Point
Advantages	Lowered Operating Costs - Reduced labor per unit
Advantages	Improved Worker Safety - Removes humans from hazardous operations
Advantages	Reduced Factory Lead Times - Faster, more predictable production
Advantages	Increased Production Output - 24/7 operation possible
Advantages	Smaller Environmental Footprint - Optimized resource usage
Advantages	Increased Productivity and Efficiency - Consistent cycle times
Advantages	Increased System Versatility - Modern systems adapt to changes
Disadvantages	Higher Start-up Cost - Significant capital investment required
Disadvantages	Higher Cost of Maintenance - Specialized technicians needed
Disadvantages	Obsolescence/Depreciation - Technology changes rapidly
Disadvantages	Workforce Displacement - Fewer direct labor jobs
Disadvantages	Not Economical for Small Scale - Minimum volumes required

2.4 Project Development Phases

Manufacturing projects follow a structured development process from initial concept to full operation. Understanding these phases is essential for process engineers.

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

Project Development Phases with Stage–Gate Reviews

Stage–Gate Process: Each gate requires approval before proceeding

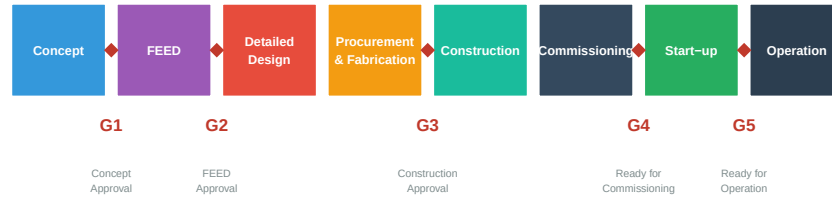


Figure 2.2: Project Development Phases and Gate Reviews

2.4.1 Phase 1: Concept Development

The initial phase defines the project scope, evaluates feasibility, and develops preliminary cost estimates.

Key Deliverables: - Project charter and business case - Preliminary process flow diagrams - Order-of-magnitude cost estimate ($\pm 30\text{-}50\%$) - Initial schedule - Risk assessment

2.4.2 Phase 2: Front-End Engineering Design (FEED)

FEED (also called FEL - Front-End Loading) develops the project definition to a level sufficient for final investment decision.

FEED Process Overview: <https://www.youtube.com/watch?v=gSkoGFLDxF0>

FEED Deliverables include:

- Project organization chart
- Defined project scope
- Process Flow Diagrams (PFDs)
- Piping & Instrumentation Diagrams (P&IDs)
- Equipment datasheets and specifications
- Major equipment list
- 2D and 3D preliminary models
- Equipment layout and installation plan
- HAZOP and safety studies
- Automation strategy
- Budget estimate ($\pm 10\text{-}15\%$)
- Project timeline

- Fixed-bid quote capability

What are the steps of Process System Design and Engineering?

1. **Process Development**
 - Develop PFDs & P&IDs
 - Process simulation
 - Heat and mass balance
2. **Equipment Engineering**
 - 2D & 3D models
 - Equipment layout and installation plan
 - Skid design
 - Mechanical & structural design
3. **Systems Engineering**
 - Piping design
 - Electrical design
 - Instrumentation design
 - Control system architecture
4. **Installation Planning**
 - Lifts & installation planning
 - Construction sequencing
 - Tie-in planning

2.4.3 Phase 3: Detailed Engineering

Detailed engineering produces all documents needed for procurement and construction.

Table 2.3: Engineering Disciplines and Deliverables

Discipline	Key Deliverables	Systems Provided
Process	P&IDs, datasheets, process packages, operating procedures	Process units, utility
Mechanical	Vessel drawings, heat exchanger specs, rotating equipment specs	Pressure vessels, tank
Piping	Isometrics, pipe specs, stress analysis, 3D models	Process piping, utility
Electrical	Single-line diagrams, load lists, cable schedules, lighting	Power distribution, m
I&C	Logic diagrams, instrument index, control narratives, HMI specs	DCS/PLC, safety sys
Civil/Structural	Foundation drawings, structural steel, buildings, roads	Foundations, structur

Automation and Controls Engineering needs to include which tasks?

Control System Design: - Automation architecture design - Control panel fabrication drawings - PLC/DCS programming - HMI/SCADA design - Safety system design (SIS)

Documentation: - Control narratives - Logic diagrams (cause & effect) - Instrument datasheets - Loop diagrams - Cable schedules

Integration: - Controls integration services - Network architecture - Cybersecurity - Historian configuration - MES integration

2.4.4 Phase 4-7: Construction, Commissioning, and Start-up

Table 2.4: Later Project Phases

Phase	Key Activities	
Procurement & Fabrication	Purchase equipment, fabricate skids, manufacture custom items	I
Construction	Civil work, install equipment, piping, electrical, instrumentation	S
Commissioning	System checkout, loop testing, safety system validation	V
Start-up	Initial operation, performance testing, operator training	O

What needs to be planned for Testing & Commissioning?

Pre-Commissioning: - Mechanical completion checks - Piping pressure tests
- Electrical continuity tests - Instrument calibration

Commissioning: - Loop checking - Control system function tests - Safety system (SIS) validation - Utility systems start-up - Dry run testing

Performance Testing: - Factory Acceptance Tests (FAT) - Site Acceptance Tests (SAT) - Performance guarantee tests - Reliability testing

2.5 Process Documentation

2.5.1 Process Flow Diagrams (PFDs)

PFDs show the overall process flow, major equipment, and key operating conditions. They are the “big picture” view of the process.

Table 2.5: Process Flow Diagram (PFD) Content

Element	What's Shown	What's NOT Shown
Major Equipment	Reactors, vessels, heat exchangers, pumps, compressors	Pipe sizes, valve
Process Streams	Main process flow lines with arrows	Minor bypasses,
Operating Conditions	Temperature, pressure, flow rates at key points	All intermediate
Control Loops	Major control loops only	Local instrument
Utility Streams	Steam, cooling water, nitrogen (simplified)	Detailed utility c
Stream Data	Stream numbers with heat/mass balance table	All intermediate

2.5.2 Piping & Instrumentation Diagrams (P&IDs)

P&IDs are the detailed “roadmap” of the process, showing all equipment, piping, valves, and instrumentation.

How to Read P&IDs: <https://www.youtube.com/watch?v=j4EOTerfyTY>

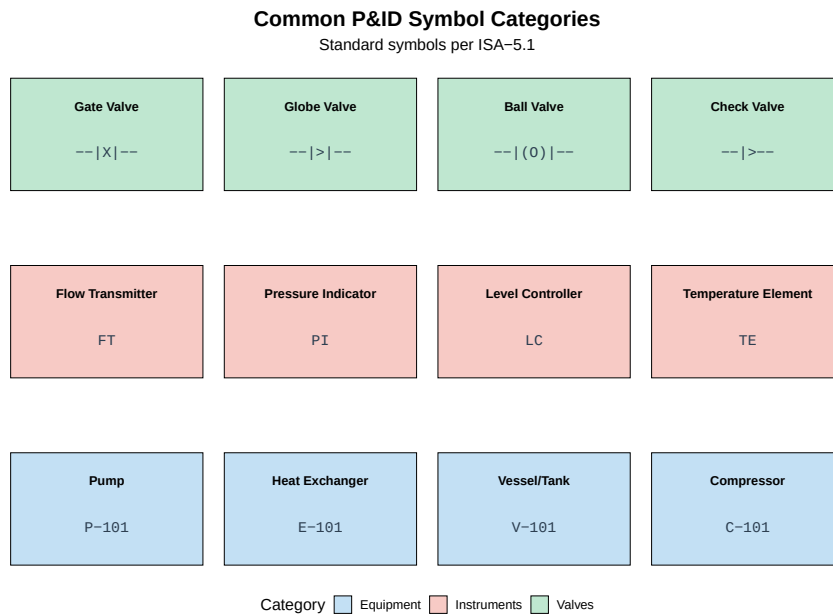


Figure 2.3: Common P&ID Symbols (Simplified)

Table 2.6: ISA Instrument Tag Structure

First Letter	Measured Variable	Modifier Letters	Example Tag
F	Flow	T = Transmitter	FT-101 (Flow Transmitter)
L	Level	C = Controller	LC-201 (Level Controller)
P	Pressure	I = Indicator	PI-301 (Pressure Indicator)
T	Temperature	E = Element/Sensor	TE-401 (Temperature Element)
A	Analysis	V = Valve	AT-501 (Analyzer Transmitter)
S	Speed	A = Alarm	SI-601 (Speed Indicator)

Quick Quiz: Decode these instrument tags

Decode the following P&ID tags:

1. **FIC-101** = ?
2. **TT-205** = ?

- 3. **PSV-301** = ?
- 4. **LCV-402** = ?

Answers:

- 1. **FIC-101** = Flow Indicating Controller (loop 101)
- 2. **TT-205** = Temperature Transmitter (loop 205)
- 3. **PSV-301** = Pressure Safety Valve (loop 301)
- 4. **LCV-402** = Level Control Valve (loop 402)

2.6 Design for Manufacturing (DFM) and Assembly (DFA)

DFM and **DFA** are methodologies that consider manufacturing and assembly constraints during product design, reducing cost and improving quality.

2.6.1 DFM Principles

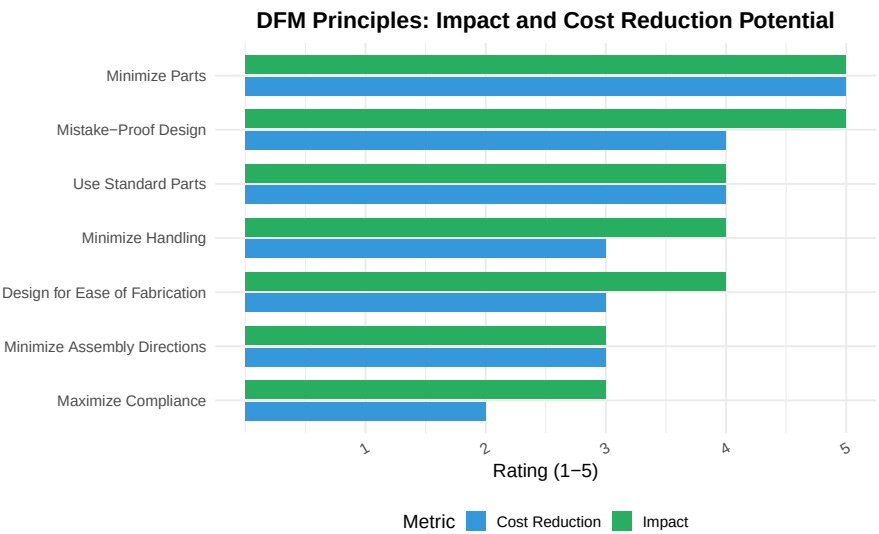


Figure 2.4: DFM Guidelines

Table 2.7: Design for Manufacturing Guidelines

DFM Guideline	Why It Matters
---------------	----------------

2.6. DESIGN FOR MANUFACTURING (DFM) AND ASSEMBLY (DFA) 37

Minimize total number of parts	Fewer parts = fewer chances for failure, less inventory, lower cost	Combine
Use standard components	Standard parts are cheaper, faster to procure, easier to replace	Use M6
Design for ease of fabrication	Complex features increase manufacturing cost and defect rates	Avoid un
Design for ease of assembly	Reduces assembly time and errors	Use snap
Minimize assembly directions	Multiple directions require repositioning, increasing time	All parts
Maximize compliance in assembly	Tight tolerances increase cost; allow self-alignment where possible	Use tape
Design for ease of service	Products will need maintenance; design for technician access	Locate s

2.6.2 DFA Analysis

DFA systematically evaluates assembly operations to identify improvement opportunities.

This one is live — try changing the part list or handling/insertion times and re-running.

```
#| label: dfa-example
# DFA Analysis Example: Simple Assembly
# Calculate assembly efficiency using Boothroyd-Dewhurst method

# Assembly data (simplified)
parts <- data.frame(
  Part = c("Base", "PCB", "Cover", "Screw 1", "Screw 2", "Screw 3", "Screw 4", "Label"),
  Handling_Time = c(2.5, 3.0, 2.5, 3.5, 3.5, 3.5, 3.5, 4.0), # seconds
  Insertion_Time = c(3.0, 4.0, 3.5, 6.0, 6.0, 6.0, 6.0, 2.0), # seconds
  Theoretically_Necessary = c(TRUE, TRUE, TRUE, FALSE, FALSE, FALSE, FALSE, FALSE)
)

# Calculations
total_parts <- nrow(parts)
total_time <- sum(parts$Handling_Time + parts$Insertion_Time)
min_parts <- sum(parts$Theoretically_Necessary)
ideal_time <- min_parts * 3 # 3 seconds is theoretical minimum per part

efficiency <- (ideal_time / total_time) * 100

cat("DFA Analysis Results:\n")
cat("=====\n")
cat(sprintf("Total parts: %d\n", total_parts))
cat(sprintf("Theoretically minimum parts: %d\n", min_parts))
cat(sprintf("Total assembly time: %.1f seconds\n", total_time))
cat(sprintf("Ideal assembly time: %.1f seconds\n", ideal_time))
cat(sprintf("Design Efficiency: %.1f%%\n", efficiency))
cat("\nOpportunity: Combine 4 screws into snap-fit design\n")
```

Case Study: Redesigning for DFM/DFA

Original Design: - 15 parts - 4 different screw types - Assembly time: 180 seconds - Multiple assembly directions - Special tools required

Redesigned using DFM/DFA: - 8 parts (47% reduction) - 1 standard screw type - Assembly time: 75 seconds (58% reduction) - Single assembly direction (top-down) - No special tools

Cost Impact: - Part cost reduced 35% - Assembly labor reduced 58% - Quality defects reduced 60% - Total product cost reduced 40%

2.7 HAZOP Studies

HAZOP (Hazard and Operability Study) is a systematic method to identify hazards and operability problems in process plants.

HAZOP Study Overview: <https://www.youtube.com/watch?v=6AqtX8oCpKI&t>

HAZOP Study Overview: <https://www.youtube.com/watch?v=ukn0RRt0ReA&list=PLeTiXOrLeXhI5IZKv3GeOUMOYpz5mG36S>

2.7.1 HAZOP Methodology

Table 2.8: HAZOP Guide Words and Examples

Guide Word	Meaning	Example with FLOW
NO/NOT	Complete negation of design intent	No flow - pump failure, blocked line
MORE	Quantitative increase	More flow - control valve fails open
LESS	Quantitative decrease	Less flow - partial blockage, pump wear
AS WELL AS	Qualitative modification/addition	Flow + contamination - upstream leak
PART OF	Qualitative decrease	Only part of intended flow path active
REVERSE	Logical opposite of intent	Reverse flow - check valve failure
OTHER THAN	Complete substitution	Wrong material flowing

2.7.2 HAZOP Example

Node	Parameter	Guide Word	Deviation	Cause
------	-----------	------------	-----------	-------

Feed Pump P-101 to Reactor R-101	Flow	No	No Flow	Pump failure, blocked suction
Feed Pump P-101 to Reactor R-101	Flow	More	More Flow	Control valve fails open
Feed Pump P-101 to Reactor R-101	Pressure	More	High Pressure	Downstream blockage
Feed Pump P-101 to Reactor R-101	Temperature	Less	Low Temperature	Cooling water leak into feed

Discussion: Why is HAZOP done during FEED, not later?

HAZOP should be completed during FEED because:

1. **Cost of changes** - Changes during FEED cost 1x; during detailed design cost 10x; during construction cost 100x
2. **Design influence** - P&IDs are still being developed and can be modified
3. **Equipment selection** - Safety requirements may affect equipment sizing
4. **Budget accuracy** - Safety systems must be included in project budget
5. **Regulatory approval** - Many jurisdictions require HAZOP before construction permits

Best practice: Conduct preliminary HAZOP early in FEED, then detailed HAZOP when P&IDs are 60-80% complete.

2.8 Cost Estimation

Accurate cost estimation is critical for project approval and budget management.

Table 2.10: Cost Estimate Classification (AACE International)

AACE Class	Project Phase	Engineering Complete	Accuracy Range	Purpose
Class 5	Concept Screening	0-2%	-50% to +100%	Screening alternatives
Class 4	Concept Study	1-15%	-30% to +50%	Feasibility
Class 3	FEED/Budget	10-40%	-20% to +30%	Authorization
Class 2	Detailed Design	30-75%	-15% to +20%	Control
Class 1	Construction	65-100%	-10% to +15%	Bid/Tender

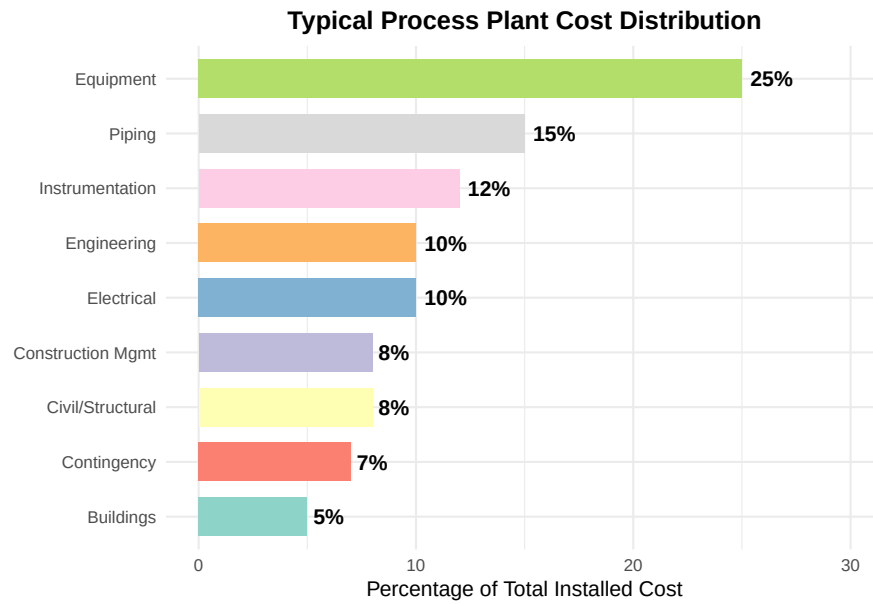


Figure 2.5: Typical Process Plant Cost Breakdown

2.9 Case Study: Automotive Paint Line Project

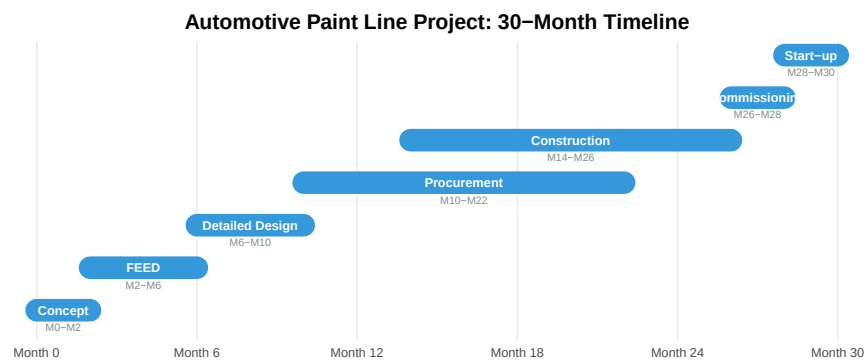


Figure 2.6: Automotive Paint Line Project Timeline

Case Study Details: Paint Line Engineering Challenges

Project Overview: - New automotive paint line for SUV body shop - Capacity: 60 jobs per hour (JPH) - Process: E-coat, primer, basecoat, clearcoat

Key Engineering Challenges:

- 1. Environmental Control** - Temperature: $\pm 1^{\circ}\text{C}$ throughout paint booth - Humidity: 65% $\pm 5\%$ RH - Air cleanliness: Class 10,000
 - 2. Automation Requirements** - 48 paint robots (8 per zone) - Conveyor system with 2m pitch - Zone tracking and recipe management
 - 3. Safety Systems** - Explosion-proof electrical classification - Fire suppression throughout - VOC monitoring and control
 - 4. Quality Systems** - Inline thickness measurement - Automated visual inspection - Recipe management and traceability
- Project Outcome:** - On-time completion - Under budget by 3% - Quality metrics exceeded targets - 99.2% first-pass yield achieved
-

2.10 Review Questions

Question 1: Automation Selection

A medical device company needs to produce 5,000 units per year of a surgical instrument. The design is mature and not expected to change. Volumes could grow to 50,000 units if successful. Which automation type should they initially choose?

Answer: Programmable automation (CNC machining, robotic cells)

Reasoning: - Current volume (5,000/year) doesn't justify fixed automation - Flexibility allows for minor design refinements - Can scale up by adding shifts or machines - Lower initial investment preserves capital - If volume grows to 50,000, can justify dedicated equipment later

Question 2: P&ID Interpretation

Decode the following instrument tags and explain their function:

- a) FIC-201
- b) PSV-305
- c) TT-401
- d) LCV-150

Answers:

- a) **FIC-201** - Flow Indicating Controller (loop 201)
 - Measures flow, displays value, and controls a valve
- b) **PSV-305** - Pressure Safety Valve (loop 305)
 - Relieves pressure if it exceeds setpoint (safety device)
- c) **TT-401** - Temperature Transmitter (loop 401)
 - Converts temperature measurement to 4-20mA signal

- d) **LCV-150** - Level Control Valve (loop 150)
- Valve that is manipulated by a level controller

Question 3: Project Phases

Why is FEED often considered the most critical project phase?

Answer: FEED is critical because:

1. **Defines scope** - All subsequent work based on FEED deliverables
2. **Locks in 80% of cost** - Design decisions determine equipment and construction costs
3. **Lowest cost to change** - Changes cost 10-100x more in later phases
4. **Authorization basis** - Management approves project based on FEED estimate
5. **Risk identification** - HAZOP and risk studies identify safety requirements
6. **Contractor selection** - FEED package used for competitive bidding

Poor FEED = project overruns, scope changes, and budget problems.

Question 4: DFM Application

A product currently uses 12 screws of 3 different sizes for assembly. Apply DFM principles to improve this design.

Answer: Apply these DFM principles:

1. **Minimize parts:** Can some screws be eliminated by using integral snap-fits?
2. **Standardize:** Use one screw size instead of three
 - Reduces inventory complexity
 - Single tool for assembly
 - Lower procurement cost
3. **Design for ease:** Consider alternatives:
 - Snap-fit connections (no tools)
 - Twist-lock features
 - Press-fit pins

Possible redesign: - Reduce from 12 screws to 4 screws (for serviceability) - All same size (M3 $\times$ 8) - Add 4 snap-fits for initial positioning - Result: 67% fewer fasteners, single tool, faster assembly

Question 5: HAZOP Analysis

For a reactor feed pump, apply the HAZOP guide word “REVERSE” to the parameter “FLOW”. What could cause this? What are the consequences? What safeguards would you recommend?

Answer:

Deviation: Reverse flow through feed pump

Causes: - Pump stops while discharge valve open - Check valve fails (stuck open or missing) - Parallel pump creates backflow - Reactor pressure exceeds pump shutoff head

Consequences: - Reactor contents backflow into feed tank - Contamination of feed material - Possible chemical reaction in feed system - Pump damage (running backward)

Safeguards: - Check valve on pump discharge - Automatic pump discharge block valve (closes on pump stop) - Low flow alarm and trip - Reactor pressure monitoring - Backflow-preventing orifice

2.11 Summary

Table 2.11: Chapter 2 Summary

Topic	Key Takeaway
Automation Types	Fixed for high volume single product; Programmable for batch; Flexible for variety
Project Phases	Concept → FEED → Detailed Design → Construction → Commissioning → Start-up
PFDs vs P&IDs	PFD = big picture (equipment, flows); P&ID = detailed (all instruments, valves, pip
DFM/DFA	Design for easy manufacturing and assembly reduces cost 20-40%
HAZOP	Systematic hazard identification using guide words; do it during FEED
Cost Estimation	Estimate accuracy improves with engineering completion; AACE Class 1-5

2.11.1 Key Takeaways

1. **Automation selection** depends on production volume, variety, and changeover requirements
2. **FEED phase** defines 80% of project cost with only 10-15% of engineering effort
3. **P&IDs** are the critical document connecting design to construction
4. **DFM/DFA** should be applied early in design to maximize impact
5. **HAZOP** identifies hazards systematically and must be done before construction
6. **Cost estimates** improve in accuracy as engineering progresses

2.12 References

1. Towler, G., & Sinnott, R. (2021). *Chemical Engineering Design* (7th ed.). Elsevier.

2. AACE International. (2020). *Cost Estimate Classification System*.
3. ISA-5.1. (2022). *Instrumentation Symbols and Identification*.
4. Boothroyd, G., Dewhurst, P., & Knight, W.A. (2011). *Product Design for Manufacture and Assembly* (3rd ed.). CRC Press.
5. CCPS. (2008). *Guidelines for Hazard Evaluation Procedures* (3rd ed.). Wiley.

Chapter 3

Industrial Robotics

3.1 Learning Objectives

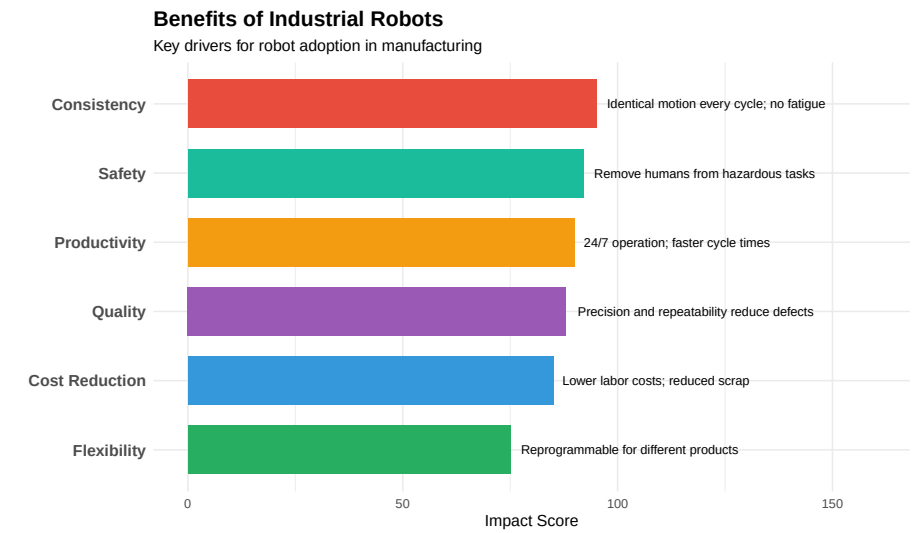
After completing this chapter, you will be able to:

1. Define industrial robots and explain their role in modern manufacturing
 2. Identify the main types of robot configurations and their applications
 3. Describe robot specifications including payload, reach, repeatability, and speed
 4. Explain the components of a robotic system (manipulator, controller, teach pendant, end effector)
 5. Understand robot coordinate systems and motion types
 6. Identify common robot applications in automotive, food, and defense industries
 7. Apply robot safety standards and safeguarding methods
 8. Describe collaborative robot (cobot) technology and applications
 9. Understand basic robot programming concepts
-

3.2 Introduction to Industrial Robotics

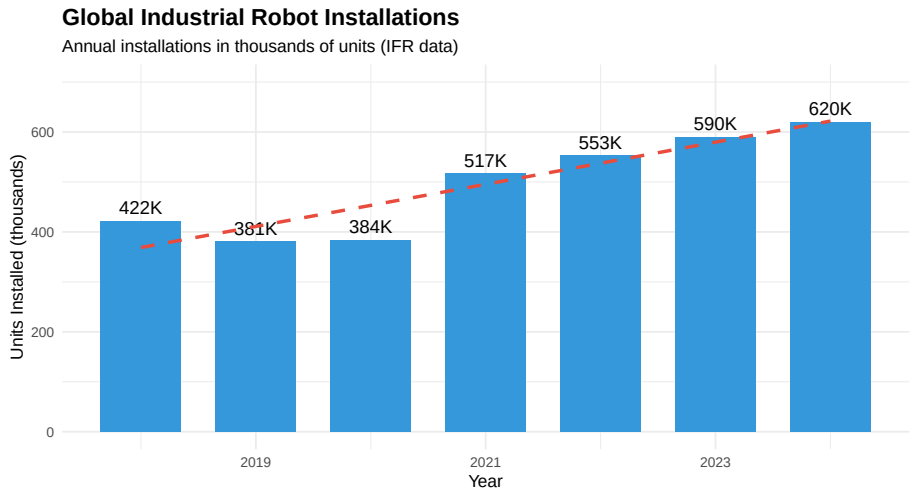
An **industrial robot** is defined by ISO 8373 as “an automatically controlled, reprogrammable, multipurpose manipulator, programmable in three or more axes, which can be either fixed in place or mobile for use in industrial automation applications.”

3.2.1 Why Use Robots?



3.2.2 Robot Market Growth

``geom_smooth()`` using formula = 'y ~ x'`



3.2.3 Industries Using Robots

Table 3.1: Industrial Robot Usage by Industry Sector

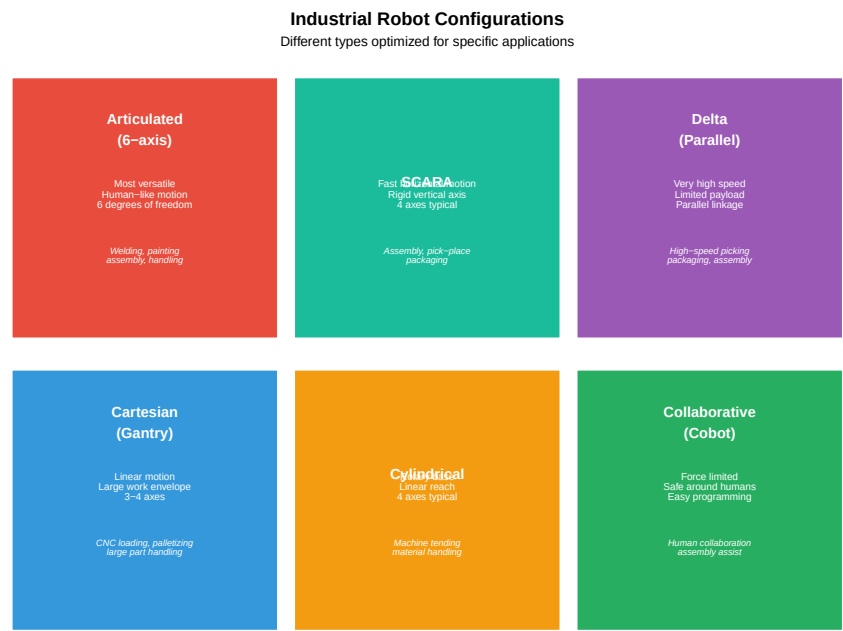
Industry	Market Share (%)	Key Applications	Growth Rate (%)
----------	------------------	------------------	-----------------

Automotive	30	Welding, painting, assembly, material handling	Stable
Electrical/Electronics	25	Assembly, testing, packaging, dispensing	High
Metal & Machinery	12	Machine tending, welding, cutting	Moderate
Food & Beverage	8	Palletizing, packaging, pick-and-place	High
Plastics & Chemicals	5	Injection molding, packaging, assembly	Moderate
Aerospace/Defense	4	Drilling, fastening, inspection, composite layup	High

3.3 Robot Configurations

Industrial robots come in various mechanical configurations, each suited to different applications.

3.3.1 Common Robot Types



3.3.2 Detailed Configuration Comparison

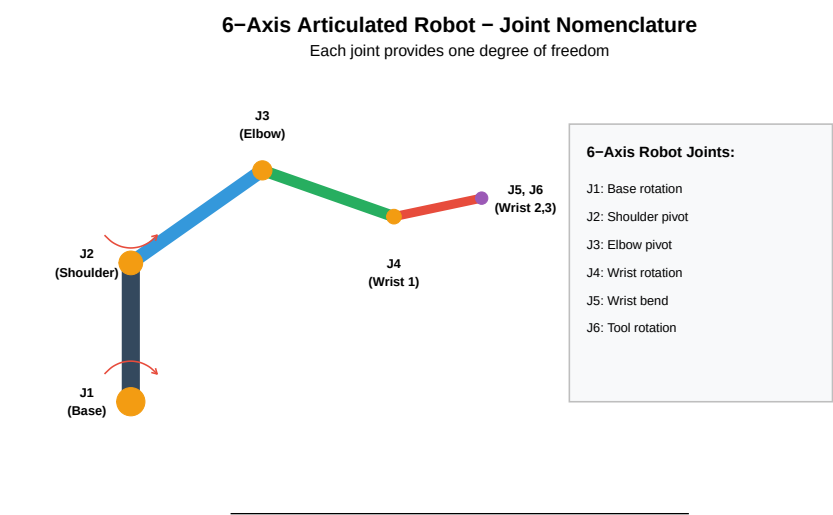
Table 3.2: Robot Configuration Comparison

Configuration	Axes	Payload Range	Reach	Repeatability	Speed	Cost
6-Axis Articulated	6	3-2300 kg	0.5-4 m	±0.02-0.1 mm	Moderate-Fast	\$\$-\$\$\$\$

SCARA	4	1-20 kg	0.2-1.2 m	± 0.01 - 0.05 mm	Very Fast
Delta/Parallel	3-4	0.5-12 kg	0.5-1.5 m	± 0.05 - 0.1 mm	Extremely Fast
Cartesian/Gantry	3-6	5-1000+ kg	Custom (meters)	± 0.05 - 0.5 mm	Moderate

3.3.3 Degrees of Freedom

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



3.4 Robot Specifications

Understanding robot specifications is essential for proper selection and application.

3.4.1 Key Specifications

Table 3.3: Key Robot Specifications

Specification	Definition	Typical Values
Payload	Maximum mass the robot can handle at full speed	3-2300 kg (varies with speed)
Reach	Maximum distance from base to tool center point	500-4000 mm for articulated robots
Repeatability	Ability to return to same position repeatedly	± 0.01 to ± 0.1 mm
Accuracy	Ability to reach a commanded position exactly	± 0.1 to ± 1.0 mm
Maximum Speed	Maximum velocity at tool center point or joint speeds	1-12 m/s TCP; 100-200 deg/s joints
Degrees of Freedom	Number of independent axes of motion	4-7 axes
Mounting	How robot is installed (floor, ceiling, wall, angle)	Floor (most common), Ceiling, Wall, Angle

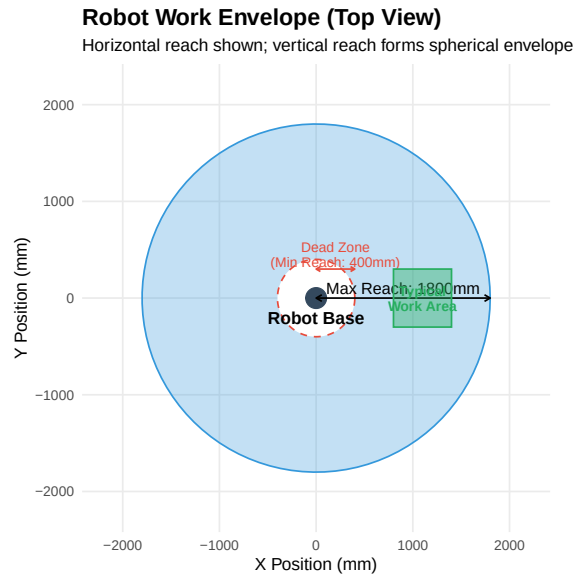
IP Rating

Ingress Protection rating for dust/water resistance

IP40 (standard) to IP67 (harsh)

3.4.2 Work Envelope

The **work envelope** (or workspace) is the three-dimensional space the robot can reach.

**3.4.3 Payload Considerations**

Live cell — try different payload, safety factor, or robot capacity values.

```
#| label: payload-calc
# Payload Calculation Example
robot_max_payload <- 10 # kg
gripper_weight <- 2.5 # kg
part_weight <- 5.0 # kg
safety_factor <- 1.2 # 20% safety margin

total_load <- gripper_weight + part_weight
required_capacity <- total_load * safety_factor
available_capacity <- robot_max_payload - gripper_weight

cat("Payload Analysis:\n")
cat("      \n")
cat("Robot maximum payload:", robot_max_payload, "kg\n")
cat("Gripper weight:", gripper_weight, "kg\n")
cat("Part weight:", part_weight, "kg\n")
```

```

cat("Total load:", total_load, "kg\n")
cat("With safety factor (", safety_factor, "x):", required_capacity, "kg\n")
cat("      \n")
cat("Available capacity for part:", available_capacity, "kg\n")

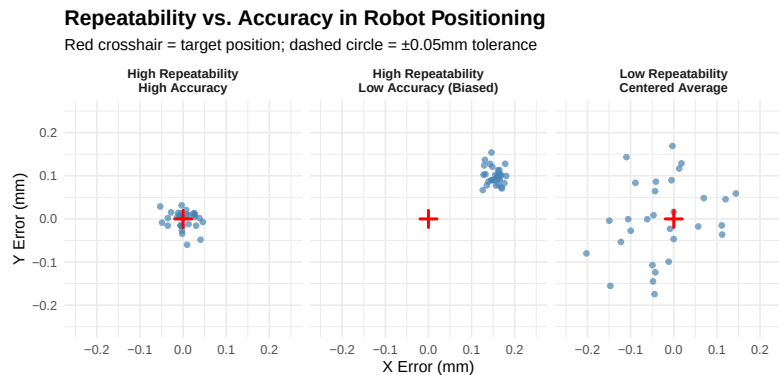
if(required_capacity <= robot_max_payload) {
  cat("Result: ACCEPTABLE - Robot can handle this load\n")
} else {
  cat("Result: EXCEEDED - Select larger robot or lighter gripper\n")
}

# Note about moment load
cat("\nNote: Also verify moment load (payload × distance from flange)\n")
cat("Moment at wrist:", part_weight, "kg ×", 0.3, "m =", part_weight * 0.3, "kg·m\n")

```

3.4.4 Repeatability vs Accuracy

Warning in annotate("circle", x0 = 0, y0 = 0, r = 0.05, color = "red", linetype = "dashed"): Ignoring unknown parameters: `x0`, `y0`, and `r`



Understanding the Difference

Repeatability: - Ability to return to the SAME taught position - More important for most applications - Quoted spec (e.g., $\pm 0.05\text{mm}$) is typically 3 - Achieved through consistent mechanical design and servo control

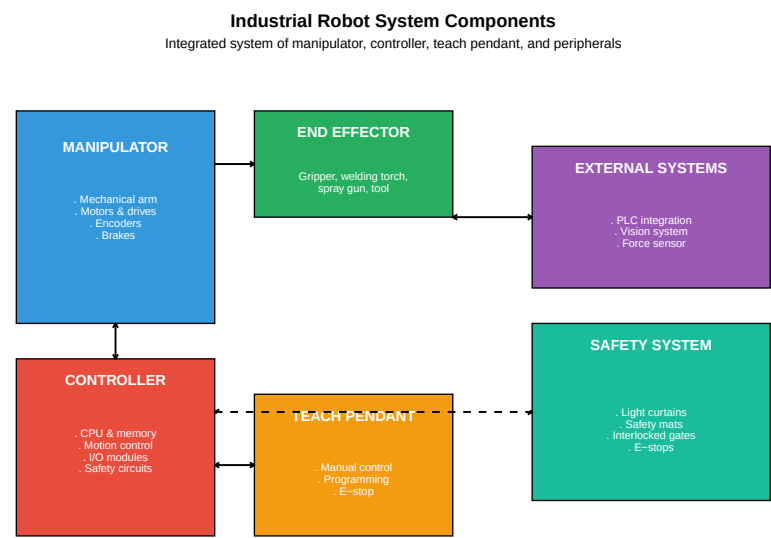
Accuracy: - Ability to reach a COMMANDED position - Important for offline programming - Affected by mechanical tolerances, deflection, calibration - Can be improved through calibration

Why Repeatability Matters More: In most applications, the robot is taught positions by jogging to them. As long as it returns to those positions consistently (repeatability), accuracy doesn't matter. Accuracy only matters when positions are calculated (offline programming) rather than taught.

3.5 Robot System Components

A complete robotic system consists of several integrated components.

3.5.1 System Architecture



3.5.2 End Effectors (EOAT)

End of Arm Tooling (EOAT) is the device attached to the robot’s wrist to interact with parts.

Table 3.4: Common End Effector Types

EOAT Type	Operating Principle	Applications
Mechanical Gripper	Fingers actuated by pneumatic, electric, or hydraulic	General part handling; assembly
Vacuum Gripper	Suction cups powered by venturi or vacuum pump	Flat surfaces; sheet material; bo
Magnetic Gripper	Electromagnetic or permanent magnet	Ferrous metal parts; steel sheets
Welding Torch	MIG, TIG, spot welding equipment	Automotive body, frames, comp
Spray Gun	Paint, adhesive, sealant dispensing	Automotive painting; sealing
Deburring Tool	Rotary tool for edge finishing	Casting, machining finishing
Force/Torque Sensor	Measures forces and torques at tool	Assembly; polishing; insertion
Vision Camera	2D or 3D imaging for guidance	Part location; inspection; guidan

3.5.3 Gripper Selection Example

Live cell — try a different part mass, acceleration, or friction coefficient.

```
#| label: gripper-selection
# Gripper Force Calculation
part_mass <- 3.0          # kg
acceleration <- 15        # m/s2 (robot acceleration)
gravity <- 9.81           # m/s2
safety_factor <- 2.0      # Minimum safety factor
friction_coefficient <- 0.3 # Steel on rubber

# Calculate forces
weight_force <- part_mass * gravity
inertia_force <- part_mass * acceleration
total_force <- sqrt(weight_force^2 + inertia_force^2)

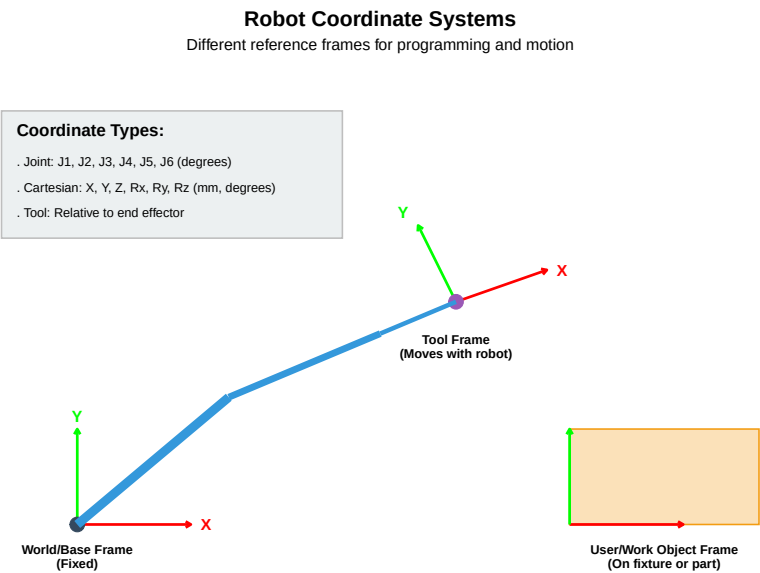
# Required grip force (friction gripper)
required_grip <- total_force / friction_coefficient * safety_factor

cat("Gripper Force Calculation:\n")
cat("      \n")
cat("Part mass:", part_mass, "kg\n")
cat("Weight force:", round(weight_force, 1), "N\n")
cat("Inertia force:", round(inertia_force, 1), "N\n")
cat("Combined force:", round(total_force, 1), "N\n")
cat("Friction coefficient:", friction_coefficient, "\n")
cat("Safety factor:", safety_factor, "\n")
cat("      \n")
cat("Required grip force:", round(required_grip, 1), "N\n")
cat("\nSelect gripper with grip force ", ceiling(required_grip/10)*10, "N\n")
```

3.6 Coordinate Systems and Motion

Understanding coordinate systems is essential for robot programming.

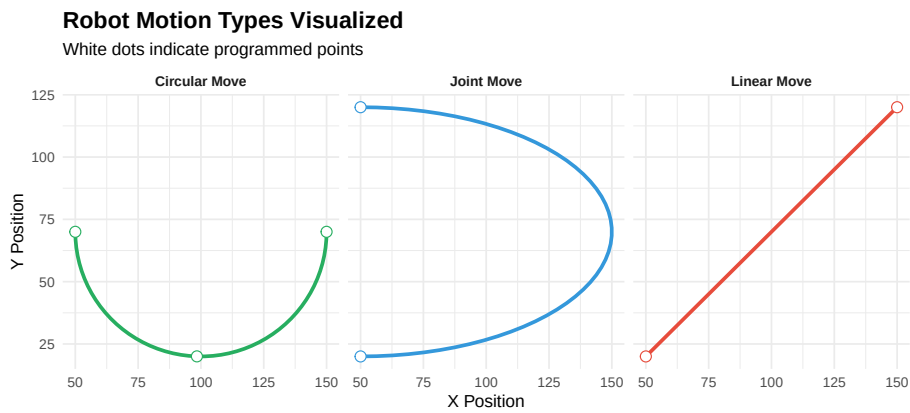
3.6.1 Coordinate Frames



3.6.2 Motion Types

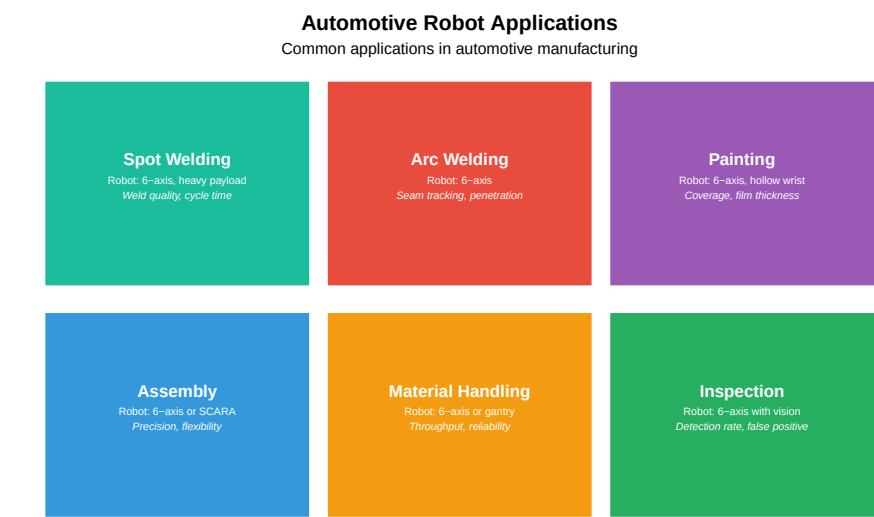
Table 3.5: Robot Motion Types

Motion Type	Path Characteristic	Speed Control
Joint Move (MoveJ)	Unpredictable path; each joint moves at constant speed	Joint velocity (% or deg/s)
Linear Move (MoveL)	Straight line from start to end; TCP follows linear path	TCP velocity (mm/s)
Circular Move (MoveC)	Arc or circle; defined by start, via, and end points	TCP velocity (mm/s)



3.7 Robot Applications

3.7.1 Automotive Applications



3.7.2 Food & Beverage Applications

Table 3.6: Food \& Beverage Robot Applications

Application	Description	Robot Type
Palletizing	Stack cases/bags onto pallets in patterns	4-axis palletizer, 6-axis
Case Packing	Load products into cases/cartons	Delta, SCARA, 6-axis
Pick and Place	Transfer products between conveyors/stations	Delta (high speed), SCARA
Cutting/Portioning	Protein portioning, cake cutting	6-axis with vision
Decorating	Applying icing, toppings, decorations	6-axis, delta
Quality Inspection	Vision-based defect detection, grading	6-axis with vision

3.7.3 Aerospace/Defense Applications

Table 3.7: Aerospace/Defense Robot Applications

Application	Description	Accuracy Required
Drilling & Fastening	Precision hole drilling; rivet/bolt installation	±0.05mm position

Composite Layup	Automated fiber placement (AFP); tape laying	±0.25mm fiber placement	Complex
NDT/Inspection	Ultrasonic, X-ray, visual inspection of structures	Full coverage	Large pa
Surface Treatment	Painting, coating, surface prep for bonding	Consistent thickness	Large en
Assembly	Large structure assembly; wing-to-fuselage	±0.5mm	Heavy p

3.8 Robot Safety

Robot safety is critical to protect workers from potential hazards.

3.8.1 Hazard Categories

Table 3.8: Robot Hazard Categories and Mitigations

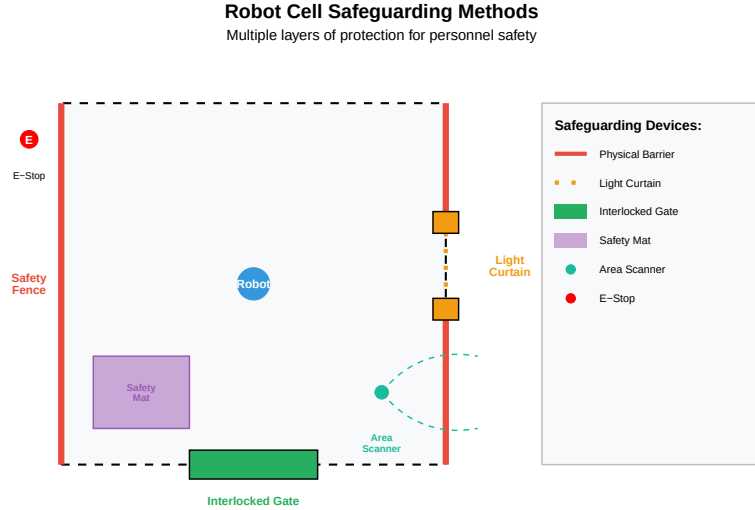
Hazard	Description	Risk_Factors
Impact	Robot strikes person with arm or tooling	Speed, mass, sharp edges, unexpected motion
Crushing	Person caught between robot and fixed object	Limited escape space, high forces
Trapping	Body part caught in articulating joints or mechanisms	Pinch points at joints, insufficient clearance
Ejection	Part or tool flies off due to grip failure or breakage	Grip force, centrifugal force, tool condition
Other	Electrical, thermal, noise, radiation hazards	Voltage, heat, welding arc, laser

3.8.2 Safety Standards

Table 3.9: Key Robot Safety Standards

Standard	Scope	Key Requirements
ISO 10218-1:2011	Robot manufacturer requirements	Stop functions, speed limiting, singularity protection
ISO 10218-2:2011	Robot system integrator requirements	Risk assessment, safeguarding, layout, validation
ISO/TS 15066:2016	Collaborative robot safety	Force/pressure limits, speed/separation monitoring
ANSI/RIA R15.06	US equivalent to ISO 10218	Safeguarding devices, risk assessment
ANSI/RIA R15.08	Industrial mobile robots	Navigation safety, personnel detection

3.8.3 Safeguarding Methods



3.9 Collaborative Robots (Cobots)

Collaborative robots are designed to work safely alongside humans without traditional guarding.

3.9.1 Cobot Characteristics

Table 3.10: Traditional Robot vs. Collaborative Robot Comparison

	Traditional Industrial Robot	Collaborative Robot
Safety Approach	Safeguarded cell; humans excluded	Inherently safe; force/speed limited
Payload	Up to 2300 kg	Typically 3-35 kg
Speed	Very high (>2 m/s)	Limited (<1.5 m/s typically)
Reach	Up to 4 m	500-1300 mm typical
Programming	Specialized programmers	Intuitive; hand guiding
Guarding	Required (fences, curtains)	Often not required
Investment	High (\$50K-500K+ with cell)	Lower (\$25K-75K typical)
Applications	High-volume, dedicated tasks	Flexible, shared workspace

3.9.2 Collaborative Operation Modes (ISO/TS 15066)

Table 3.11: ISO/TS 15066 Collabora

Collaboration Mode	Description
Safety-Rated Monitored Stop	Robot stops when human enters collaborative zone; resumes when clear
Hand Guiding	Operator physically guides robot; robot follows input forces
Speed and Separation Monitoring	Robot adjusts speed based on distance to human; stops if too close
Power and Force Limiting	Robot limits force/pressure on contact to safe levels

3.9.3 Force and Pressure Limits

ISO/TS 15066 specifies maximum contact forces and pressures for different body areas:

Table 3.12: ISO/TS 15066 Biomechanical Limits (Selected Values)

Body Area	Max Pressure (N/cm ²)	Quasi-Static Pressure	Max Force (N)	Quasi-Static Force
Skull/Forehead	130	130	130	130
Face	65	65	65	65
Neck (front/back)	145	145	150	150
Chest	140	140	140	140
Abdomen	110	110	110	110
Hand/Finger	280	280	140	140
Arm	190	190	150	150
Leg	220	220	220	220

3.10 Robot Programming Basics

3.10.1 Programming Methods

Table 3.13: Robot Programmi

Method	Description	Advantages
Online/Teach Pendant	Use teach pendant to jog robot to positions and record	Direct; prec
Lead-Through/Hand Guiding	Physically move robot arm to desired positions	Intuitive; fa
Offline Programming	Create programs on PC without robot; download later	No producti
Simulation-Based	Program and test in virtual environment; transfer to real robot	Risk-free tes

3.10.2 Basic Program Structure

Example Robot Program (ABB RAPID-style pseudocode):

```

PROGRAM Main()

    ! Initialize
    HomePosition()
    GripperOpen()

    ! Main cycle
    WHILE running DO

        ! Move to pick position
        MoveJ pApproach, v500, z50, tool1
        MoveL pPick, v100, fine, tool1

        ! Pick part
        GripperClose()
        WaitTime 0.2

        ! Move to place position
        MoveL pApproach, v200, z50, tool1
        MoveJ pPlaceApproach, v500, z50, tool1
        MoveL pPlace, v100, fine, tool1

        ! Place part
        GripperOpen()
        WaitTime 0.2

        ! Return
        MoveL pPlaceApproach, v200, z50, tool1

        ! Increment counter
        PartCount := PartCount + 1

    ENDWHILE

ENDPROGRAM

```

3.10.3 Key Programming Concepts

Table 3.14: Key Robot Programming Concepts

Concept	Description	Example
Position/Target	Stored robot position (joint angles or Cartesian coordinates)	pHome, pPick
Motion Instruction	Command to move robot (MoveJ, MoveL, MoveC)	MoveL pTarget

Speed Data	Velocity parameter (mm/s for TCP, % for joints)	v100 = 100 mm/s, v50
Zone Data	Corner path blending (fine = stop at point, z10 = 10mm blend)	fine = stop, z5 = 5mm
Tool Data	Definition of tool center point relative to flange	tool1 with TCP offset
Work Object	Definition of work coordinate system for part positions	wobj_fixture with base
I/O Commands	Control external devices (SetDO, WaitDI, etc.)	SetDO doGripper, 1; V

3.11 Video Resources

3.11.1 Introduction to Industrial Robots

3.11.2 Collaborative Robots Explained

3.12 Summary

Industrial robotics is a fundamental technology in modern manufacturing:

1. **Robot types** include articulated (most versatile), SCARA (fast assembly), delta (high-speed picking), and cartesian (large work areas)
 2. **Key specifications** include payload, reach, repeatability, speed, and degrees of freedom
 3. **System components** work together: manipulator, controller, teach pendant, end effector, and safety system
 4. **Coordinate systems** (world, tool, user) enable flexible programming and positioning
 5. **Applications** span automotive, food & beverage, and aerospace with industry-specific requirements
 6. **Safety** requires comprehensive risk assessment and appropriate safeguarding
 7. **Collaborative robots** enable human-robot cooperation with force/speed limiting
 8. **Programming** can be online (teach pendant), lead-through, or offline
-

3.13 Review Questions

Question 1: Compare articulated (6-axis) robots, SCARA robots, and delta robots. For each, describe the typical applications and key advantages.

Answer:

Robot Type	Configuration	Key Advantages	Typical Applications
6-Axis Articulated	6 revolute joints; human-like arm	Maximum flexibility; can reach any orientation; large payload range	Welding, painting, assembly, material handling, machine tending
SCARA	4 axes; selective compliance	Very fast horizontal motion; rigid vertical; compact; precise	Pick-and-place, assembly, screw driving, packaging
Delta (Parallel)	3-4 axes; parallel linkage	Extremely fast (150+ picks/min); low moving mass	High-speed picking, packaging, sorting, light assembly

Detailed Comparison:

6-Axis Articulated: - Most versatile - can position tool at any angle - Wide payload range (3 kg to 2300+ kg) - Can reach around obstacles - Complex programming possible - *Best for:* Tasks requiring complex orientations, welding seams, painting

SCARA: - Inherently rigid in Z (vertical) - good for insertion - Very fast in X-Y plane - Lower cost than 6-axis - Compact footprint - *Best for:* Assembly operations, circuit board population, packaging

Delta: - Highest speed of any configuration - Low inertia (motors at base, not moving) - Limited payload (typically <15 kg) - Requires overhead mounting - *Best for:* High-speed sorting, packaging lines, food handling

Question 2: A robot has a maximum payload of 15 kg. The gripper weighs 4 kg and the part weighs 8 kg. Is this robot suitable? What other factors should be considered?

Answer:

Initial Assessment:

Total load = Gripper weight + Part weight

Total load = 4 kg + 8 kg = 12 kg

Robot capacity: 15 kg

Load: 12 kg

Remaining margin: 3 kg (20%)

Consideration with Safety Factor:

Typical safety factor: 1.2 - 1.5

Required capacity = 12 kg $\times$ 1.25 = 15 kg

This equals the robot's maximum capacity - marginal!

Additional Factors to Consider:

1. Moment Load (Torque at Wrist)

- Distance from flange to center of gravity matters
- If part is held 300mm from flange: $\text{Moment} = 8 \text{ kg} \times 0.3 \text{ m} = 2.4 \text{ kg} \cdot \text{m}$
- Robot specs include moment limits - must verify

2. Inertia

- Fast motion requires acceleration
- Higher mass + longer distance = higher inertia
- May need to derate payload for high-speed applications

3. Payload Derating with Reach

- Many robots derate payload at extended reach
- 15 kg at 500mm may become 10 kg at 1500mm
- Check payload diagram in robot specifications

4. Orientation

- Payload may vary with wrist orientation
- Horizontal reach vs. vertical lift differ

5. Acceleration Requirements

- Higher acceleration = higher effective load
- May need to reduce speed/acceleration

Recommendation: The 15 kg robot is marginal for this application. Consider:
 - Select next size up (20 kg) for adequate margin - Use lighter gripper (3 kg or less)
 - Reduce part handling distance from flange - Limit acceleration/speed if using this robot

Question 3: Explain the difference between repeatability and accuracy. Why is repeatability often more important than accuracy for robot applications?

Answer:

Definitions:

Repeatability: - Ability to return to the **same taught position** multiple times - Measures consistency/precision of motion - Quoted as $\pm$ value (e.g., $\pm 0.05 \text{ mm}$ at 3) - Affected by: mechanical wear, servo control, thermal effects

Accuracy: - Ability to reach a **commanded position** exactly - Measures how close actual position is to theoretical position - Typically worse than repeatability (e.g., $\pm 0.5 \text{ mm}$) - Affected by: manufacturing tolerances, calibration, deflection, backlash

Why Repeatability is Usually More Important:

1. Teaching Method

- Most robots are programmed by teaching (jogging to positions)

- Robot doesn't need to know where it "should" be - just return to where it was taught
- As long as it returns consistently (repeatability), accuracy doesn't matter

2. Process Requirements

- Welding: Need to follow same seam every time → repeatability
- Assembly: Need to insert part in same hole every time → repeatability
- Pick-place: Need to pick from same fixture position → repeatability

3. Compensation Possible

- Poor accuracy can be compensated by adjusting taught points
- Poor repeatability cannot be compensated

When Accuracy Matters:

1. Offline Programming

- Positions calculated from CAD data, not taught
- Robot must go where commanded, first time
- Common in aerospace (large parts, tight access)

2. Multi-Robot Systems

- Robots must agree on positions
- Calibration to common frame requires accuracy

3. Frequent Program Changes

- If programs are frequently changed without re-teaching
- Offline modifications need accurate execution

Bottom Line: For most industrial applications, ± 0.05 mm repeatability is far more valuable than ± 0.5 mm accuracy because taught positions are used.

Question 4: Describe the four collaborative operation modes defined in ISO/TS 15066. Give an example application for each.

Answer:

1. Safety-Rated Monitored Stop

Description: Robot operates at normal speed when workspace is clear. When a person enters the collaborative zone, the robot stops and holds position. Robot resumes automatically when person exits.

Technical Requirements: - Safety-rated sensors (light curtains, scanners) - Safety-rated monitored stop function - Clearly defined collaborative zone

Example Application: Machine tending cell where operator occasionally loads raw material. Robot works at full speed during machining cycle but stops safely when operator enters to load/unload.

2. Hand Guiding

Description: Operator physically contacts the robot through a hand guiding device and manually moves it. Robot follows operator's force input.

Technical Requirements: - Hand guiding device (force/torque sensing) - Emergency stop accessible at guiding device - Speed reduction while guiding - Safe torque control

Example Application: Teaching a paint path on an aerospace component. Operator guides robot along surface contour while robot records positions. Also used for "collaborative finishing" where operator guides robot holding a polishing tool.

3. Speed and Separation Monitoring

Description: Robot continuously monitors distance to the nearest person. Speed adjusts based on separation: full speed when far, reduced speed when closer, stop if minimum distance breached.

Technical Requirements: - Safety-rated distance monitoring (usually area scanners) - Safety-rated speed control - Multiple speed/distance zones defined - Real-time separation calculation

Example Application: Shared packaging area where robot palletizes cases while operator moves around the cell. Robot slows when operator approaches, stops if too close, and resumes at appropriate speed as operator moves away.

4. Power and Force Limiting

Description: Robot is designed so that forces and pressures from any contact are below injury thresholds. Contact is allowed because it cannot cause harm.

Technical Requirements: - Inherently safe design (compliant joints, low mass, rounded surfaces) - Force/torque sensing or current monitoring - Verified force limits per ISO/TS 15066 body area tables - May include padding/soft covers

Example Application: Assembly assist where robot holds a heavy part while human worker installs fasteners. Robot and human are in direct contact, with robot providing position support and human providing dexterity. Force is limited so accidental contact causes no injury.

Question 5: Calculate the required gripper force for a robot picking a 2 kg part with a friction coefficient of 0.4. The robot accelerates at 10 m/s². Use a safety factor of 2.0.

Answer: (live — change the numbers to check your own version)

```
#| label: q5-answer
# Given values
mass <- 2.0           # kg
```

```

acceleration <- 10      # m/s2
gravity <- 9.81         # m/s2
friction_coef <- 0.4    # Coefficient of friction
safety_factor <- 2.0    # Safety factor

# Calculate forces
# Weight force (vertical)
F_gravity <- mass * gravity

# Inertia force (from acceleration)
F_inertia <- mass * acceleration

# Combined force (vector sum for worst case)
# Worst case: acceleration horizontal while holding against gravity
F_combined <- sqrt(F_gravity2 + F_inertia2)

# Required friction force to hold part
# F_friction =      × F_normal
# F_normal = Grip force
# F_friction must overcome F_combined

# Without safety factor:
F_grip_min <- F_combined / friction_coef

# With safety factor:
F_grip_required <- F_grip_min * safety_factor

cat("Gripper Force Calculation:\n")
cat("      \n\n")
cat("Input Parameters:\n")
cat("  Part mass:", mass, "kg\n")
cat("  Robot acceleration:", acceleration, "m/s2\n")
cat("  Friction coefficient:", friction_coef, "\n")
cat("  Safety factor:", safety_factor, "\n\n")

cat("Force Analysis:\n")
cat("  Gravity force: Fg = m × g =", mass, "×", gravity, "=",
    round(F_gravity, 1), "N\n")
cat("  Inertia force: Fi = m × a =", mass, "×", acceleration, "=",
    round(F_inertia, 1), "N\n")
cat("  Combined force: Fc = √(Fg2 + Fi2) =", round(F_combined, 1), "N\n\n")

cat("Grip Force Calculation:\n")
cat("  F_friction =      × F_grip F_combined\n")
cat("  F_grip F_combined /      =", round(F_combined, 1), "/", friction_coef,

```



```

    "=", round(F_grip_min, 1), "N\n")
cat("  With safety factor:", round(F_grip_min, 1), "x", safety_factor,
    "=", round(F_grip_required, 1), "N\n\n")

cat("                                \n")
cat("REQUIRED GRIPPER FORCE:", ceiling(F_grip_required), "N minimum\n")
cat("Select gripper rated for ", ceiling(F_grip_required/10)*10, "N\n")

```

Question 6: List and describe five safety devices used to protect personnel around industrial robots.

Answer:

1. Physical Barriers (Safety Fencing)

Description: Fixed guards or fencing that physically prevent access to the robot work envelope.

Characteristics: - Rigid construction (steel, aluminum, polycarbonate) - Must withstand expected impact forces - Minimum height (typically 1.8 m) - Properly anchored to floor - May include interlocked gates for authorized entry

Application: Primary safeguard for most traditional robot cells.

2. Light Curtains (AOPD - Active Opto-electronic Protective Device)

Description: Transmitter and receiver create a sensing field of infrared beams. Breaking any beam triggers a stop.

Characteristics: - Type 2 or Type 4 (safety-rated) - Resolution (14mm for finger, 40mm for hand/arm) - Muting capability for material transfer - Very fast response (<20ms)

Application: Access points where frequent entry needed; allows material flow while protecting personnel.

3. Safety Mats (Pressure-Sensitive Mats)

Description: Floor mats that detect weight/pressure and trigger stop when stepped on.

Characteristics: - Placed in hazardous zones - Withstand industrial environment (oil, debris) - Fast response - Self-monitoring for failures

Application: Areas where overhead detection isn't practical; around robot bases.

4. Area Scanners (Safety Laser Scanners)

Description: Laser scanner detects objects/personnel entering defined zones. Multiple warning and stop zones configurable.

Characteristics: - 270° field of view typical - Multiple programmable zones - Warning zone (slow robot) and stop zone (stop robot) - Type 3 safety rating - Detects at floor level

Application: Open cells requiring flexibility; mobile robot protection; speed/separation monitoring.

5. Interlocked Gates/Doors

Description: Access gates with safety switches that stop robot when opened.

Characteristics: - Mechanical or magnetic interlock switches - Trapped key systems for lockout - May include guard locking (prevent opening while robot moving) - Escape release from inside - Meets ISO 14119

Application: Authorized entry points for maintenance, setup, troubleshooting.

Additional Devices:

- **E-Stop (Emergency Stop):** Palm-operated buttons to immediately stop robot
- **Two-Hand Controls:** Require both hands on buttons to initiate motion
- **Enabling Devices:** 3-position switches held by personnel in restricted area
- **Presence-Sensing Devices:** Capacitive, radar, or vision-based detection

Question 7: Explain the difference between MoveJ (Joint Move) and MoveL (Linear Move). When would you use each?

Answer:

MoveJ (Joint Move)

How It Works: - Each joint moves from start to end angle independently - All joints start and stop together (coordinated) - Path of TCP (Tool Center Point) is unpredictable - curved - Robot calculates joint interpolation, not TCP path

Characteristics: - Fastest point-to-point motion - No singularity issues during move - Path may vary depending on starting configuration - Speed specified in % of max or joint velocity (°/s)

When to Use: - Moving between distant points through free space - Approach movements before precise positioning - Home position moves - Any time when path doesn't matter, only endpoints - Escaping from near-singularity positions

MoveL (Linear Move)

How It Works: - TCP follows a straight line from start to end - Robot calculates joint positions to maintain linear TCP path - Orientation is interpolated linearly as well - All axes coordinated to produce straight line

Characteristics: - Guaranteed straight path - Consistent, predictable motion - Slower than MoveJ for same endpoint distance - Can encounter singularities during move - Speed specified in mm/s at TCP

When to Use: - Process paths (welding, sealing, cutting) - Insertion/extraction operations - Approach to pick/place positions - Any time path must be straight - When moving close to obstacles

Practical Example:

Imagine picking a part from a fixture:

! Move from home to near pick position - path doesn't matter

MoveJ pApproach, v1000, z50, tool1 ! Joint move - fast

! Move down to pick - must be straight to avoid collision

MoveL pPick, v100, fine, tool1 ! Linear move - controlled

! Pick part, then retract straight up

GripperClose()

MoveL pApproach, v200, z10, tool1 ! Linear move - straight up

! Move to place area - through open space

MoveJ pPlaceApproach, v1000, z50, tool1 ! Joint move - fast

Key Decision: - If path matters → MoveL - If only destination matters → MoveJ

Question 8: A manufacturing engineer needs to select a robot for a palletizing application. The requirements are: 25 kg payload, 2.2 m horizontal reach, 10 cycles per minute, floor-mounted. Recommend a robot type and justify your selection.

Answer:

Application Analysis:

Requirement	Value	Implication
Payload	25 kg	Medium-heavy; rules out delta and most SCARA
Reach	2.2 m	Large work envelope needed
Cycle rate	10/min	6 seconds/cycle; moderate speed

Requirement	Value	Implication
Mounting	Floor	Typical industrial installation
Motion type	Palletizing	Mainly vertical and horizontal; some rotation

Robot Type Evaluation:

1. 4-Axis Palletizing Robot (RECOMMENDED) - Specifically designed for palletizing - 4 axes: base rotation, arm, elbow, wrist - High payload capability (25-700+ kg available) - Long reach available (up to 3.2m) - Fast for palletizing motions - Lower cost than 6-axis - Examples: FANUC M-410iC, KUKA KR QUANTEC PA, ABB IRB 460

Verdict: Excellent match for this application

2. 6-Axis Articulated Robot - Maximum flexibility - Can handle complex pallet patterns - Higher cost than 4-axis for same payload - May be over-specified for simple palletizing - Examples: FANUC M-710iC/50, ABB IRB 6700

Verdict: Capable but potentially over-specified and more expensive

3. Cartesian/Gantry Robot - Can cover very large areas - Simple motion control - Higher installation complexity - Less common for palletizing applications

Verdict: Possible but not ideal for this payload/cycle rate

Recommended Selection:

FANUC M-410iC/185 or similar 4-axis palletizer - Payload: 185 kg (well above 25 kg requirement with margin for gripper) - Reach: 3.143 m (exceeds 2.2 m requirement) - Repeatability: ± 0.5 mm - Axes: 4

Justification: 1. **Purpose-built:** Designed specifically for palletizing 2. **Adequate payload:** 185 kg allows for heavy gripper + multiple parts 3. **Reach:** 3.1 m provides margin for future needs 4. **Speed:** Optimized kinematics for palletizing motion profile 5. **Reliability:** Proven in thousands of palletizing applications 6. **Cost:** Lower than equivalent 6-axis robot 7. **Simplicity:** 4 axes easier to program and maintain than 6

Additional Recommendations: - Vacuum or mechanical gripper depending on product - Vision system for product detection if variety exists - Safety scanner for operator access during pallet changes - Interface to conveyor for product delivery

3.14 References

1. Siciliano, B., & Khatib, O. (Eds.). (2016). *Springer Handbook of Robotics* (2nd ed.). Springer.
2. ISO 10218-1:2011. *Robots and Robotic Devices - Safety Requirements for Industrial Robots - Part 1: Robots*.
3. ISO 10218-2:2011. *Robots and Robotic Devices - Safety Requirements for Industrial Robots - Part 2: Robot Systems and Integration*.
4. ISO/TS 15066:2016. *Robots and Robotic Devices - Collaborative Robots*.
5. International Federation of Robotics. (2023). *World Robotics Report*. IFR.
6. Niku, S.B. (2020). *Introduction to Robotics: Analysis, Control, Applications* (3rd ed.). Wiley.
7. Craig, J.J. (2018). *Introduction to Robotics: Mechanics and Control* (4th ed.). Pearson.
8. FANUC Corporation. (2023). *Robot Technical Manuals*. FANUC.
9. ABB Robotics. (2023). *Technical Reference Manual - RAPID Instructions*. ABB.
10. Robotic Industries Association. (2023). *ANSI/RIA R15.06-2012: Industrial Robots and Robot Systems - Safety Requirements*. RIA.

Chapter 4

Plant Layout and Facility Design

4.1 Learning Objectives

By the end of this chapter, you will be able to:

- Identify and describe the four basic types of facility layouts
- Compare the advantages and disadvantages of each layout type
- Apply the steps involved in designing a process layout
- Calculate cycle time, theoretical minimum stations, and line efficiency for product layouts
- Understand the principles of warehouse and office layout design

Why is Plant Layout Important?

Plant layout directly impacts:

- **Productivity:** Efficient layouts minimize wasted movement and time
 - **Cost:** Poor layouts increase material handling costs by 20-50%
 - **Safety:** Well-designed layouts reduce accidents and injuries
 - **Flexibility:** The right layout allows adaptation to changing demands
 - **Employee Morale:** Good layouts improve working conditions
-

4.2 Types of Facility Layouts

There are **four basic layout types**, each suited to different production requirements:

Table 4.1: Overview of Facility Layout Types

Layout Type	Also Known As	Best For	Ex
Process Layout	Functional Layout, Job Shop	High variety, low volume	Ho
Product Layout	Flow Line, Assembly Line	Low variety, high volume	Au
Hybrid/Cellular Layout	Group Technology, Cell Layout	Medium variety, medium volume	El
Fixed-Position Layout	Project Layout	Very large or immobile products	Sh

4.2.1 Visualizing the Layout Spectrum

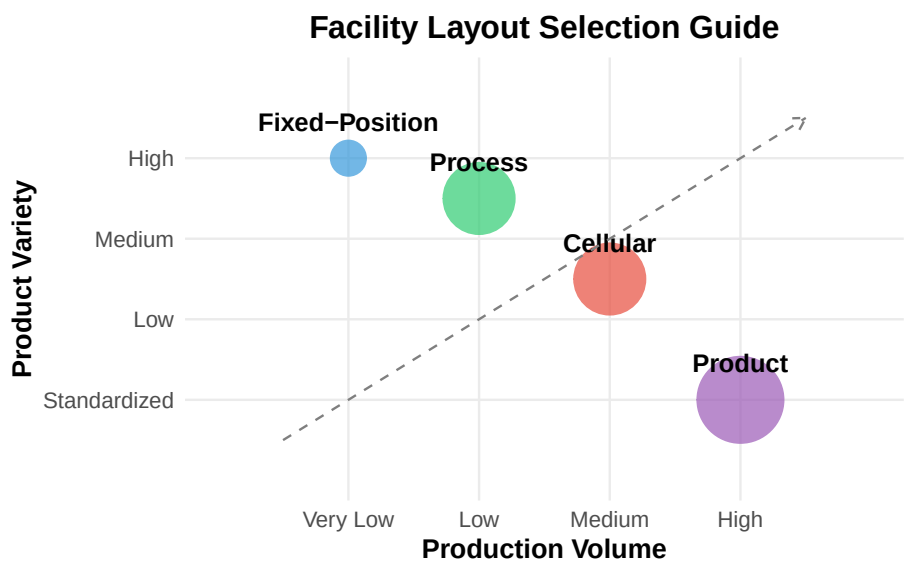


Figure 4.1: Product Variety vs. Production Volume for Different Layouts

4.3 Process Layout (Functional Layout)

In a **process layout**, similar resources (machines, equipment, workers with similar skills) are grouped together. Work flows between departments based on the specific requirements of each job.

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

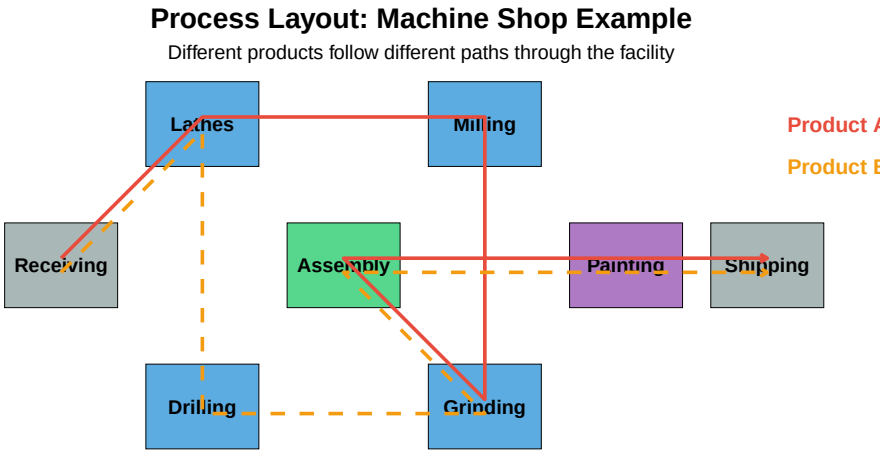


Figure 4.2: Example Process Layout - Machine Shop

4.3.1 Characteristics of Process Layouts

Table 4.2: Process Layout Characteristics

Characteristic	Description
Equipment	General-purpose machines
Labor	Skilled workers required
Flexibility	High - can handle variety
Processing Speed	Slower due to varied routing
Material Handling	Higher costs (complex paths)
Scheduling	Complex - different routes
Space Requirements	Higher - WIP inventory storage

Discussion Question: Can you identify a process layout in your daily life?

Think about: - A hospital emergency room (patients routed based on their needs)
- A university campus (students move between buildings for different

classes) - A grocery store (customers choose their own path) - A machine shop (parts routed based on required operations)

What makes these process layouts? Similar functions are grouped together, and “products” (patients, students, customers) follow different paths based on their individual needs.

4.3.2 Real-World Examples

4.4 Product Layout (Assembly Line)

In a **product layout**, equipment and workstations are arranged in a line according to the sequence of operations needed to produce the product. Every unit follows the same path.

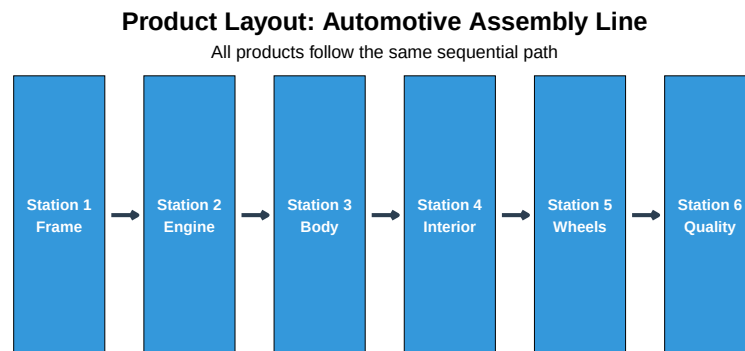


Figure 4.3: Product Layout - Assembly Line

4.4.1 Characteristics of Product Layouts

Table 4.3: Product Layout: Advantages vs. Disadvantages

Advantage	Disadvantage
High production rate	High initial investment
Low unit cost	Low flexibility
Low material handling	Line stops affect all
Simple scheduling	Monotonous work
Less WIP inventory	Requires balanced workloads

Quick Quiz: Product Layout

Question: A car wash where vehicles move through a series of cleaning stations is an example of which layout type?

Answer: Product Layout - all cars follow the same path through the same sequence of operations.

4.5 Hybrid Layout (Cellular Manufacturing)

Hybrid layouts combine the flexibility of process layouts with the efficiency of product layouts. The most common type is the **cellular layout** using Group Technology.

4.5.1 Group Technology Concept

Group Technology (GT) identifies parts with similar characteristics and groups them into **part families**. Machines are then arranged into **cells** to process these families.

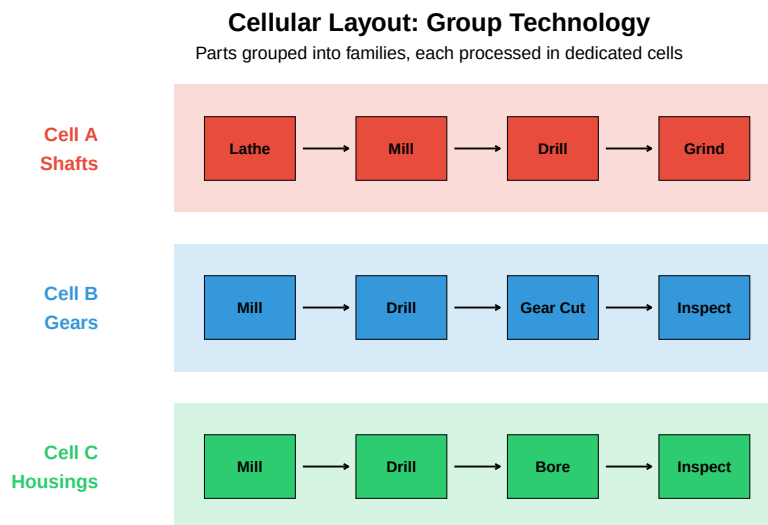


Figure 4.4: Cellular Layout with Manufacturing Cells

Discussion Question: Benefits of Cellular Manufacturing

What are the main benefits of cellular manufacturing over traditional process layouts?

1. **Reduced material handling** - Parts stay within the cell
2. **Shorter lead times** - No waiting between departments

3. **Less WIP inventory** - Smaller batches, faster flow
 4. **Improved quality** - Workers responsible for complete part
 5. **Team accountability** - Cell teams own their output
-

4.6 Fixed-Position Layout

In a **fixed-position layout**, the product remains stationary while workers, equipment, and materials are brought to it. This is used when the product is too large or heavy to move.

4.6.1 Examples of Fixed-Position Layouts

- **Shipbuilding** - Ships built in dry docks
- **Aircraft manufacturing** - Large aircraft assembled in hangars
- **Construction** - Buildings, bridges, dams
- **Surgery** - Patient remains stationary

Challenge Question: Managing a Fixed-Position Layout

What are the unique challenges of managing a fixed-position layout?

Think about: - Scheduling deliveries of materials and equipment - Coordinating multiple contractors/teams - Limited space around the product - Weather and environmental factors (outdoor projects) - Ensuring safety with many workers in one area

4.7 Designing Process Layouts

Designing an effective process layout involves **three main steps**:

4.7.1 Step 1: Gather Information

You need to determine: - **Space requirements** for each department - **Available space** in the facility - **Closeness relationships** between departments

4.7.1.1 From-To Matrix (Load Matrix)

A From-To matrix shows the number of trips or loads between departments:

Table 4.4: From-To Matrix: Loads per Day Between Departments

	Receiving	Machining	Assembly	Painting	Shipping
Receiving	-	100	50	0	0

Machining	0	-	200	50	0
Assembly	0	0	-	100	60
Painting	0	0	0	-	150
Shipping	0	0	0	0	-

4.7.2 Step 2: Develop Block Plans

Use the **Load-Distance Model** to evaluate layout alternatives:

$$\text{Total Cost} = \sum_{i=1}^n \sum_{j=1}^n L_{ij} \times D_{ij} \times C$$

Where: - L_{ij} = Number of loads between departments i and j - D_{ij} = Distance between departments i and j - C = Cost per unit distance

4.7.3 Interactive Example: Comparing Two Layouts

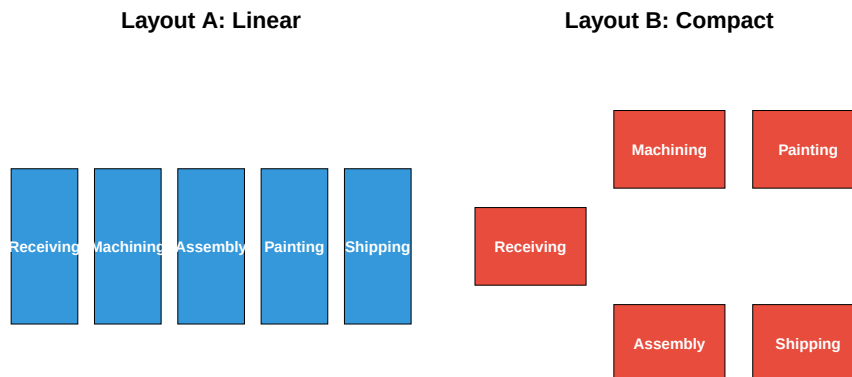


Figure 4.5: Comparing Two Process Layout Alternatives

Live cell — try different from-to volumes or layout distances.

```
#| label: load-distance-calc
# Calculate Load-Distance scores for both layouts
# From-To data: Receiving->Machining=100, Machining->Assembly=200,
#               Assembly->Painting=100, Painting->Shipping=150

# Layout A distances (linear): all adjacent = 1 unit
layout_A_score <- 100*1 + 200*1 + 100*1 + 150*1
cat("Layout A Total Load-Distance:", layout_A_score, "\n")
```

```
# Layout B distances (compact): most departments closer together
# Receiving to Machining = 1.12 (diagonal)
# Machining to Assembly = 1
# Assembly to Painting = 1.12 (diagonal)
# Painting to Shipping = 1
layout_B_score <- 100*1.12 + 200*1 + 100*1.12 + 150*1
cat("Layout B Total Load-Distance:", round(layout_B_score, 1), "\n")

cat("\nLayout A is better by:", round(layout_B_score - layout_A_score, 1), "units")
```

Why does the linear layout win in this case?

In this example, the linear layout (A) wins because the process flow is essentially sequential (Receiving → Machining → Assembly → Painting → Shipping).

When flow is **sequential**, a **product layout** (linear arrangement) is more efficient. Process layouts are better when there's **varied routing** between departments.

4.7.4 Step 3: Develop Detailed Layout

Once the block plan is selected: - Determine exact sizes and shapes of departments - Plan aisle locations and widths - Consider utilities, safety exits, and accessibility - Use CAD software or 3D modeling for visualization

4.8 Designing Product Layouts (Line Balancing)

The key challenge in product layout design is **line balancing** - assigning tasks to workstations to minimize idle time while meeting production requirements.

4.8.1 Line Balancing Steps

1. Identify tasks and precedence relationships
2. Calculate required cycle time
3. Calculate theoretical minimum number of stations
4. Assign tasks to stations
5. Calculate efficiency

4.8.2 Key Formulas

Cycle Time:

$$C = \frac{\text{Available Production Time}}{\text{Desired Output}}$$

Theoretical Minimum Stations:

$$TM = \frac{\sum t_i}{C}$$

Line Efficiency:

$$\text{Efficiency} = \frac{\sum t_i}{n \times C} \times 100\%$$

Where: - C = Cycle time - $\sum t_i$ = Total task time - n = Number of workstations

4.8.3 Interactive Example: Pizza Assembly Line

Table 4.5: Pizza Assembly Tasks

Task	Description	Time (sec)	Predecessors
A	Roll dough	40	-
B	Spread sauce	25	A
C	Add cheese	20	B
D	Add toppings	35	B
E	Season	15	C, D
F	Box pizza	30	E

Precedence Diagram: Pizza Assembly

Arrows show required sequence of operations

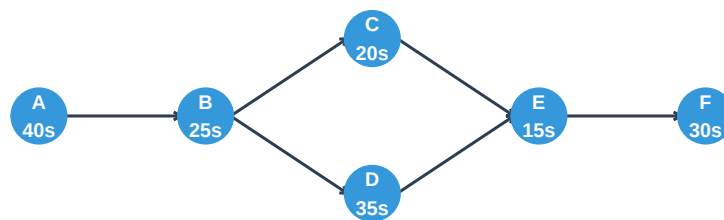


Figure 4.6: Precedence Diagram for Pizza Assembly

4.8.4 Line Balancing Calculation

This one is live — edit the numbers and re-run it to see how the results change.

```

#| label: line-balance-calc
# Given data
total_task_time <- 40 + 25 + 20 + 35 + 15 + 30 # seconds
desired_output <- 60 # pizzas per hour
available_time <- 3600 # seconds per hour

# Step 1: Calculate Cycle Time
cycle_time <- available_time / desired_output
cat("Cycle Time:", cycle_time, "seconds per pizza\n")

# Step 2: Calculate Theoretical Minimum Stations
TM <- total_task_time / cycle_time
cat("Theoretical Minimum Stations:", TM, "=", ceiling(TM), "stations\n")

# Step 3: Assign tasks to stations (one possible solution)
cat("\n--- Workstation Assignments ---\n")
cat("Station 1: A (40s) + B (25s) = 65s > 60s ... EXCEEDS!\n")
cat("Let's try again:\n")
cat("Station 1: A (40s) = 40s [Idle: 20s]\n")
cat("Station 2: B (25s) + D (35s) = 60s [Idle: 0s]\n")
cat("Station 3: C (20s) + E (15s) + F (30s) = 65s > 60s... EXCEEDS!\n")
cat("Revised:\n")
cat("Station 3: C (20s) + E (15s) = 35s [Idle: 25s]\n")
cat("Station 4: F (30s) = 30s [Idle: 30s]\n")

# With 4 stations
n_stations <- 4
efficiency <- (total_task_time / (n_stations * cycle_time)) * 100
cat("\n--- Results ---\n")
cat("Number of Stations:", n_stations, "\n")
cat("Line Efficiency:", round(efficiency, 1), "%\n")
cat("Balance Delay (Idle Time):", round(100 - efficiency, 1), "%\n")

```

Try It Yourself: Can you find a better balance?

Challenge: Can you assign tasks to only 3 stations while respecting the precedence constraints?

Hint: Consider these assignments: - Station 1: A (40s) + ? - Station 2: B (25s) + ? + ? - Station 3: ? + ?

Remember: Total time per station cannot exceed 60 seconds!

Solution: It's actually possible with careful planning: - Station 1: A (40s) = 40s - Station 2: B (25s) + C (20s) + E (15s) = 60s (but E needs C AND D complete!)

So 3 stations is NOT feasible due to precedence constraints. The minimum is 4

stations for this problem.

4.9 Warehouse Layout Design

Warehouse layouts are a special case of process layouts focused on **minimizing material handling**.

4.9.1 Key Principles

1. **Place high-volume items near the dock**
2. **Group items that are often picked together**
3. **Use ABC analysis** for storage assignment:
 - **A items** (20% of SKUs, 80% of picks): Prime locations
 - **B items** (30% of SKUs, 15% of picks): Secondary locations
 - **C items** (50% of SKUs, 5% of picks): Remote locations

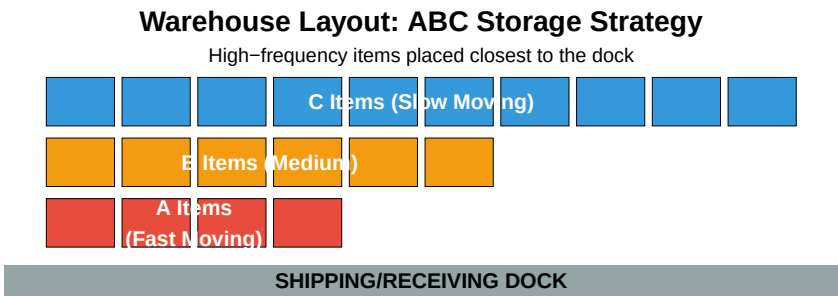


Figure 4.7: Warehouse Layout: ABC Storage Strategy

4.10 Office Layout Design

Office layouts balance **communication needs** with **privacy requirements**.

4.10.1 Two Main Approaches

Table 4.6: Office Layout Comparison

Aspect	Open Plan	Traditional/Closed
Space	Shared workspaces	Private offices
Walls	Partitions/dividers	Permanent walls
Flexibility	High - easy to reconfigure	Low - expensive to change
Communication	Excellent	Limited
Privacy	Low	High
Cost	Lower	Higher

4.10.2 Office Ergonomics Checklist

Click to expand: Office Ergonomics Best Practices

Posture: - Support the small of the back - Keep shoulders relaxed - Maintain neutral wrist position - Feet flat on floor or footrest

Lighting: - Position monitor to reduce glare - Adjust brightness for screen vs. paper work - Use task lighting when needed

Workstation: - Monitor at arm's length - Top of screen at or below eye level - Keyboard and mouse at elbow height

Work Habits: - Take breaks every 30-60 minutes - Vary tasks to prevent repetitive strain - Use keyboard shortcuts to reduce mouse use

4.10.3 The Future of Office Design

4.11 Summary: Choosing the Right Layout

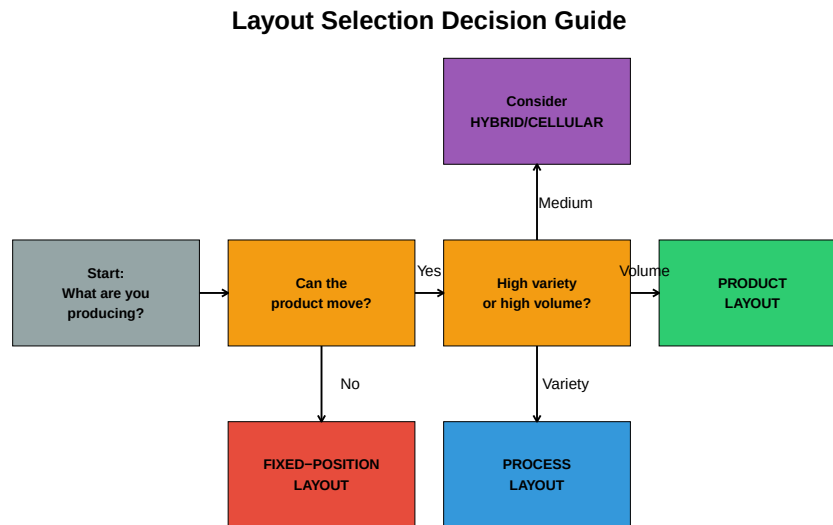


Figure 4.8: Layout Selection Decision Guide

4.11.1 Key Takeaways

1. **Process layouts** offer flexibility for varied products but have higher material handling costs
2. **Product layouts** are efficient for high-volume, standardized products
3. **Cellular layouts** provide a middle ground with benefits of both
4. **Fixed-position layouts** are necessary when products cannot be moved
5. **Line balancing** is critical for efficient product layouts
6. **Warehouse layouts** should minimize travel distance for high-frequency items

4.12 Industry Case Studies

4.12.1 Case Study 1: Food Processing Plant Layout

Food processing facilities have unique layout requirements due to food safety, hygiene, and regulatory compliance.

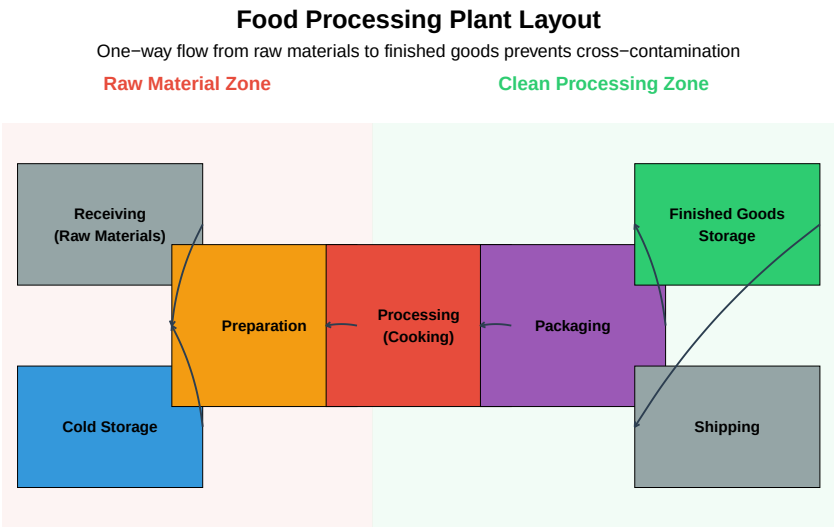


Figure 4.9: Food Processing Plant Layout Example

Key Design Principles for Food Processing:

Table 4.7: Food Processing Layout Design Principles

Principle	Description	Regulation
Linear Flow	Raw materials → Processing → Packaging (no backtracking)	FSMA
Zone Separation	Physical barriers between raw and ready-to-eat areas	HACCP
Cleanability	Smooth, non-porous surfaces; rounded corners; proper drainage	3-A Star
Temperature Control	Maintain cold chain; separate refrigerated zones	FDA 21
Sanitary Design	Handwashing stations, foot baths, air curtains at zone transitions	USDA F
Allergen Control	Dedicated lines or thorough cleaning between allergen products	FALCPA

Discussion Question: Cross-Contamination Prevention

Question: A food processing plant produces both peanut butter products and tree nut-free products. What layout considerations are essential?

Key Considerations:

1. **Physical Separation:** Completely separate production lines with physical barriers (walls, not just distance)
2. **Air Handling:** Separate HVAC systems to prevent airborne allergen transfer

3. **Personnel Flow:** Dedicated workers for each line, or strict gowning/cleaning procedures when crossing zones
4. **Scheduling:** If shared equipment is unavoidable, schedule allergen products last before deep cleaning
5. **Traffic Patterns:** Forklift and cart paths should not cross between allergen zones
6. **Verification:** Air sampling and surface swabs to verify effectiveness

4.12.2 Case Study 2: Aerospace Manufacturing Facility

Aerospace facilities often use **fixed-position layouts** combined with **process layouts** for component fabrication.

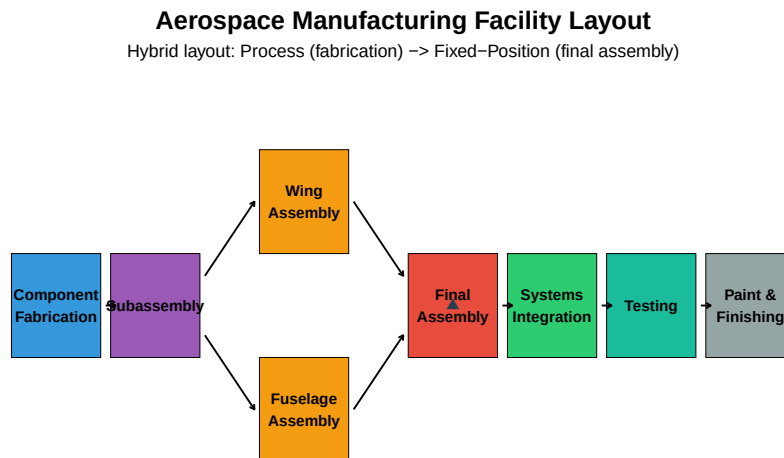


Figure 4.10: Aerospace Assembly Facility Layout

Aerospace Layout Characteristics:

- **Clean rooms** for precision components (hydraulics, avionics)
- **High-bay areas** for large assembly operations
- **Overhead cranes** for moving heavy components
- **Flow-line final assembly** (aircraft moves through stations) for high-volume commercial aircraft
- **Fixed-position** for military aircraft or low-volume production
- **Strict FOD (Foreign Object Debris)** control throughout

Video: Boeing 737 Assembly

Watch how Boeing uses a moving assembly line for high-volume aircraft production:

4.12.3 Case Study 3: Automotive Supplier - Hybrid Cell Layout

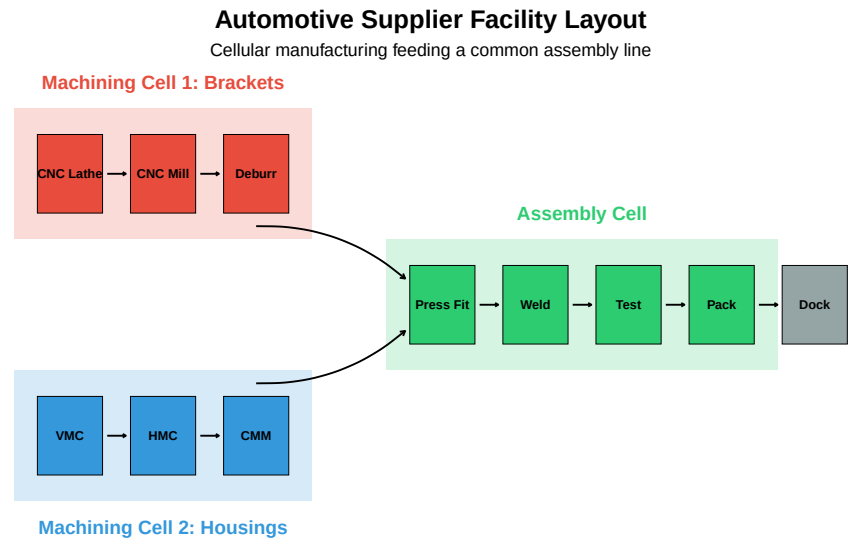


Figure 4.11: Automotive Supplier Facility with Cellular Layout

JIT Delivery Requirements:

Automotive suppliers must often deliver directly to the assembly line in **sequence** (JIS - Just-In-Sequence):

Table 4.8: JIT/JIS Requirements and Layout Implications

Requirement	Typical Standard	Layout Impact
Delivery Window	±30 minutes of scheduled time	Dedicated shipping lane, staging area n
Sequence Accuracy	100% (0 sequence errors)	Pick-to-light systems, visual managemen
Quality Level	0 PPM target (Six Sigma)	Inline inspection, mistake-proofing (pol
Packaging	Returnable containers to line side	Container cleaning/storage area needed
Traceability	Full lot/serial traceability required	Barcode/RFID scanning stations throu

4.13 Ergonomics in Manufacturing

Ergonomics (human factors engineering) is critical for plant layout design to ensure worker safety, reduce injuries, and improve productivity.

4.13.1 The NIOSH Lifting Equation

The National Institute for Occupational Safety and Health (NIOSH) developed an equation to evaluate manual lifting tasks:

$$RWL = LC \times HM \times VM \times DM \times AM \times FM \times CM$$

Where **RWL** = Recommended Weight Limit (in pounds or kg)

Table 4.9: NIOSH Lifting Equation Factors

Factor	Name	Formula	Description
LC	Load Constant	51 lb (23 kg)	Maximum weight under ideal conditions
HM	Horizontal Multiplier	10/H	H = horizontal distance from spine to load
VM	Vertical Multiplier	$1 - 0.0075 \times (V - 30)$	V = vertical height of hands at start (inches)
DM	Distance Multiplier	$0.82 + 1.8/D$	D = vertical travel distance (inches)
AM	Asymmetric Multiplier	$1 - 0.0032A$	A = angle of asymmetry (degrees)
FM	Frequency Multiplier	Table lookup	Based on lift frequency and duration
CM	Coupling Multiplier	Table lookup	Based on grip quality (good/fair/poor)

4.13.2 Lifting Index Calculation

$$LI = \frac{\text{Actual Weight}}{\text{RWL}}$$

Table 4.10: Lifting Index Interpretation

Lifting Index	Risk Level	Action Required
LI 1.0	Low	Task is acceptable for most workers
$1.0 < LI \leq 3.0$	Moderate	Task poses risk; engineering controls recommended
$LI > 3.0$	High	Task is unacceptable; immediate redesign required

4.13.3 NIOSH Lifting Example

Also live — try changing the box weight, distances, or lift frequency below and re-running.

```
#| label: niosh-example-plant
# Example: Worker lifts 35 lb boxes from pallet to conveyor
# Conditions:
# - Horizontal distance (H) = 15 inches
# - Vertical height at start (V) = 10 inches
# - Vertical travel distance (D) = 20 inches
# - Asymmetry angle (A) = 30 degrees
```

```

# - Frequency = 4 lifts per minute, 2 hours
# - Coupling = Fair (handles present)

LC <- 51 # Load constant (lb)

# Calculate multipliers
HM <- 10 / 15 # Horizontal multiplier
VM <- 1 - 0.0075 * abs(10 - 30) # Vertical multiplier
DM <- 0.82 + (1.8 / 20) # Distance multiplier
AM <- 1 - 0.0032 * 30 # Asymmetric multiplier
FM <- 0.72 # From NIOSH table: 4 lifts/min, 1-2 hours
CM <- 0.95 # Fair coupling

cat("Multiplier Values:\n")
cat("  HM (Horizontal):", round(HM, 3), "\n")
cat("  VM (Vertical):", round(VM, 3), "\n")
cat("  DM (Distance):", round(DM, 3), "\n")
cat("  AM (Asymmetry):", round(AM, 3), "\n")
cat("  FM (Frequency):", FM, "\n")
cat("  CM (Coupling):", CM, "\n")

# Calculate RWL
RWL <- LC * HM * VM * DM * AM * FM * CM
cat("\nRecommended Weight Limit (RWL):", round(RWL, 1), "lb\n")

# Calculate Lifting Index
actual_weight <- 35
LI <- actual_weight / RWL
cat("Lifting Index (LI):", round(LI, 2), "\n")

if (LI <= 1.0) {
  cat("\nRisk Level: LOW - Task is acceptable\n")
} else if (LI <= 3.0) {
  cat("\nRisk Level: MODERATE - Consider engineering controls\n")
} else {
  cat("\nRisk Level: HIGH - Immediate redesign required\n")
}

```

Discussion Question: Reducing the Lifting Index

Based on the example above, what layout or workstation changes could reduce the Lifting Index?

Solutions (in order of effectiveness):

1. **Reduce horizontal distance (H):** Move conveyor closer to pallet position

- If $H = 10''$: HM improves from 0.67 to 1.0 $\rightarrow$ RWL increases by 50%
- 2. **Raise the starting height (V):** Use elevated pallet positions or pallet lifters
 - If $V = 30''$ (optimal): VM improves from 0.85 to 1.0 $\rightarrow$ RWL increases by 18%
- 3. **Eliminate asymmetry (A):** Reposition conveyor directly in front of pallet
 - If $A = 0^\circ$: AM improves from 0.90 to 1.0 $\rightarrow$ RWL increases by 11%
- 4. **Reduce frequency:** Add a second worker or use automation
 - If 1 lift/min: FM improves from 0.72 to 0.94 $\rightarrow$ RWL increases by 31%
- 5. **Improve coupling:** Use boxes with good handles
 - If good coupling: CM improves from 0.95 to 1.0 $\rightarrow$ RWL increases by 5%

Best solution: Implement a scissor lift for the pallet (adjustable height) and move the conveyor closer. This addresses the largest multiplier reductions.

4.13.4 Ergonomic Workstation Design

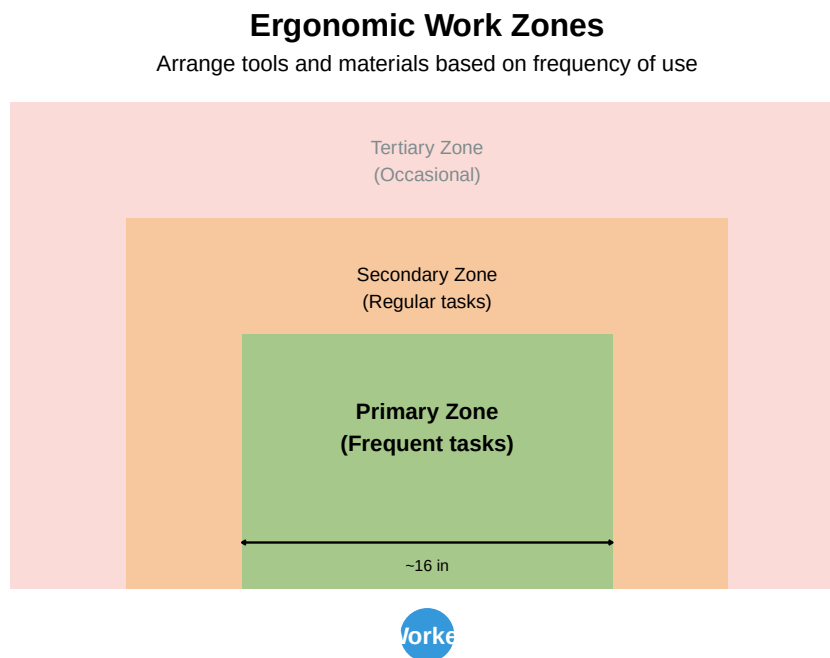


Figure 4.12: Ergonomic Work Zones for Standing Operations

Layout Implications:

- **Primary zone:** Most frequently used tools and parts
- **Secondary zone:** Regularly used items, occasional reaches
- **Tertiary zone:** Rarely needed items, should trigger redesign if frequent use required

4.14 Modern Layout Trends and Industry 4.0

4.14.1 Flexible Manufacturing Layouts

Modern facilities must adapt quickly to changing product demands. Key trends include:

Table 4.11: Modern Flexible Layout Trends

Trend	Description	B
Modular Workstations	Pre-built workstation modules that can be relocated in hours	R
Reconfigurable Cells	Cells with standardized machine interfaces for quick changeover	Sw
Mobile Equipment	Equipment on wheels or air bearings for rapid repositioning	El
Flexible Utilities	Overhead utility drops and floor trenches allow equipment mobility	N
Digital Layout Planning	Digital twins enable virtual layout testing before physical changes	R

4.14.2 Industry 4.0 Layout Considerations

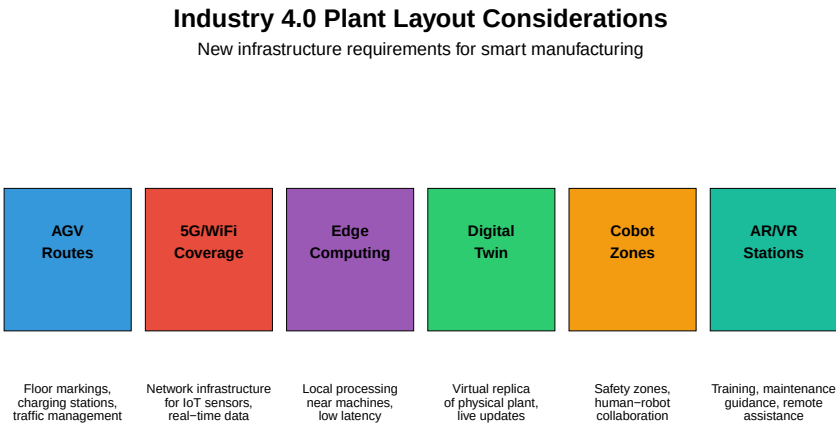


Figure 4.13: Industry 4.0 Enabled Plant Layout Elements

4.14.3 AGV and AMR Traffic Planning

Automated Guided Vehicles (AGVs) and Autonomous Mobile Robots (AMRs) require careful layout planning:

Table 4.12: AGV/AMR Layout Planning Guidelines

Consideration	Design Guidelines
Route Design	Unidirectional loops preferred; minimize intersections; avoid steep grades
Charging Locations	Opportunity charging at pickup/dropoff points; dedicated charging bays for breaks
Traffic Intersections	Yield zones, traffic signals, or elevation changes at crossings
Floor Requirements	Smooth, level surfaces; magnetic tape or QR codes for navigation; no oil/water
Safety Zones	Minimum 1m clearance from pedestrian paths; light curtains at crossings
Staging Areas	Buffer areas for queue management; waiting positions clear of traffic

Video: AGV Systems in Modern Warehouses

4.15 Layout Simulation Software

Modern plant layout design relies heavily on simulation software to test alternatives before physical implementation.

4.15.1 Common Layout Planning Software

Table 4.13: Plant Layout Planning Software Comparison

Software	Primary Use	Key Features
AutoCAD Factory Design Suite	2D/3D CAD layout design	Extensive equipment libraries, interference checking, etc.
FlexSim	3D discrete event simulation	Drag-and-drop modeling, VR capability, real-time 3D visualization
Arena	Process simulation	Statistical analysis, what-if scenarios, bottleneck identification
Plant Simulation (Siemens)	Digital factory simulation	Full Siemens integration, AGV simulation, energy analysis
SketchUp + Extensions	Quick 3D visualization	Low cost, easy learning curve, walkthrough capability

4.15.2 Simulation Benefits

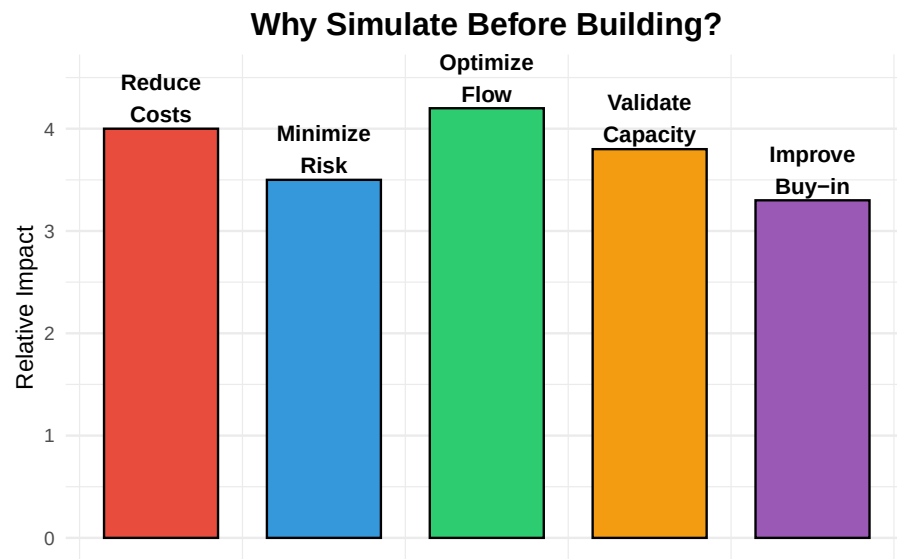


Figure 4.14: Benefits of Layout Simulation

ROI of Simulation:

- **10-20x** return on simulation investment typical
- **30-50%** reduction in layout change costs
- **Weeks to months** saved in ramp-up time
- **Stakeholder alignment** - visual communication of concepts

Video: Plant Simulation Demo

4.16 Practice Problems

Problem 1: Layout Selection

Scenario: A company manufactures custom kitchen cabinets. Each order is unique, with different dimensions, materials, and finishes. Production volume is about 50 units per month.

Question: Which layout type would you recommend and why?

Answer: A **process layout** would be most appropriate because: - High product variety (custom orders) - Low production volume - Different operations needed for each order - Flexibility is more important than efficiency

Problem 2: Line Balancing

Given: - Total task time: 240 seconds - Desired output: 30 units per hour - Available time: 3600 seconds per hour

Calculate: 1. Cycle time 2. Theoretical minimum number of stations 3. If you use 3 stations, what is the efficiency?

Solution: 1. Cycle time = $3600 / 30 = 120$ seconds 2. TM = $240 / 120 = 2$ stations 3. Efficiency = $(240 / (3 \times 120)) \times 100 = 66.7\%$

Problem 3: Warehouse Layout

Scenario: A warehouse has the following items: - Item X: 500 picks/day - Item Y: 100 picks/day - Item Z: 50 picks/day

Question: How should these items be positioned relative to the shipping dock?

Answer: Using ABC analysis: - Item X (high frequency): **Closest to dock** (A zone) - Item Y (medium frequency): **Middle distance** (B zone) - Item Z (low frequency): **Farthest from dock** (C zone)

4.17 Review Questions

Question 1: What distinguishes a cellular layout from both process and product layouts?

Answer: A cellular layout combines characteristics of both:

- **From process layout:** Equipment variety within each cell, flexibility to handle part variations
- **From product layout:** Sequential flow within the cell, dedicated equipment for part families
- **Unique characteristics:**
 - Parts grouped into families using Group Technology
 - Cells are self-contained mini-factories
 - Workers often cross-trained to operate multiple machines
 - Reduced material handling compared to process layout
 - More flexibility than pure product layout

Question 2: Calculate the Lifting Index for the following scenario

Scenario: A worker lifts 25 kg boxes from floor level ($V=0$) to a shelf at waist height. The horizontal distance is 40 cm, vertical travel is 75 cm, no asymmetry, frequency is 2 lifts per minute for 4 hours, and coupling is poor (no handles).

Given multipliers: - LC = 23 kg - HM = 25/H (H in cm) - VM = 1 - 0.003|V - 75| (V in cm) - DM = 0.82 + 4.5/D (D in cm) - AM = 1.0 (no asymmetry) - FM = 0.65 (from tables) - CM = 0.90 (poor coupling)

Solution:

$$HM = 25/40 = 0.625$$

$$VM = 1 - 0.003|0 - 75| = 1 - 0.225 = 0.775$$

$$DM = 0.82 + 4.5/75 = 0.82 + 0.06 = 0.88$$

$$AM = 1.0$$

$$FM = 0.65$$

$$CM = 0.90$$

$$RWL = 23 \times 0.625 \times 0.775 \times 0.88 \times 1.0 \times 0.65 \times 0.90$$

$$RWL = 23 \times 0.625 \times 0.775 \times 0.88 \times 0.65 \times 0.90$$

$$RWL = 5.7 \text{ kg}$$

$$LI = 25/5.7 = 4.4$$

Risk Level: HIGH (LI > 3.0) - Immediate redesign required

Recommended actions: Raise starting height (use pallet lifter), reduce horizontal distance, add handles to boxes, reduce frequency.

Question 3: List three Industry 4.0 technologies that impact plant layout design

Answer: (Any three of the following)

1. **AGVs/AMRs:** Require route planning, charging stations, floor markings, safety zones
2. **Collaborative robots (cobots):** Need safety zones, but smaller footprint than traditional robot cells
3. **Digital twins:** Enable virtual layout testing, reduce physical prototyping
4. **IoT sensors:** Require network infrastructure (5G/WiFi), edge computing locations
5. **AR/VR systems:** Training stations, remote assistance capabilities
6. **Flexible automation:** Modular equipment, quick-connect utilities

Question 4: What are the key differences between food processing plant layouts and general manufacturing?

Answer:

Aspect	Food Processing	General Manufacturing
Flow direction	Strictly one-way (raw → finished)	Can have backtracking
Zone separation	Physical barriers between raw/clean	Logical separation often sufficient

Aspect	Food Processing	General Manufacturing
Surfaces	Sanitary design, smooth, cleanable	Durability focused
Temperature	Multiple temperature zones (cold chain)	Generally ambient
Regulatory	HACCP, FSMA, FDA oversight	OSHA, industry standards
Allergen control	Dedicated lines or validated cleaning	Not typically a concern
Traceability	Lot-level required by regulation	Often batch-level sufficient

Question 5: A warehouse has three product categories with the following pick frequencies. Design the storage zones.

Data: - Category A: 1200 picks/day, 500 SKUs - Category B: 600 picks/day, 1500 SKUs - Category C: 200 picks/day, 3000 SKUs

Solution:

Using ABC analysis based on pick frequency: - **Category A** (60% of picks, 10% of SKUs): Prime locations closest to shipping dock - **Category B** (30% of picks, 30% of SKUs): Secondary locations, moderate distance - **Category C** (10% of picks, 60% of SKUs): Remote locations, maximum distance from dock

Additional considerations: - Within each zone, use cube utilization for dense storage of slow movers - Consider pick path optimization (serpentine vs. return routing) - High-frequency items at ergonomic pick heights (waist level) - Reserve space near dock for cross-docking high-velocity items

4.18 Chapter Summary

Table 4.14: Chapter 4 Summary: Plant Layout and Fa

Topic	Key Concepts
Layout Types	Fixed-position, Process, Product, Hybrid/Cellular - selected based on volume and va
Process Layout Design	From-To matrix, Load-Distance model, Block plans, Closeness relationships
Product Layout (Line Balancing)	Cycle time, Theoretical minimum stations, Precedence diagrams, Line efficiency
Cellular Manufacturing	Group Technology, Part families, Self-contained cells, Reduced material handling
Warehouse Design	ABC analysis, High-frequency items near dock, Pick path optimization
Ergonomics	NIOSH lifting equation, RWL, Lifting Index, Work zones, Workstation design

Modern Trends Simulation	Flexible layouts, Industry 4.0, AGV/AMR planning, Digital twins FlexSim, Arena, Plant Simulation - virtual testing before physical change
---	--

4.19 References

- Reid, R. D., & Sanders, N. R. (2019). *Operations Management: An Integrated Approach* (7th ed.). Wiley. Chapter 10.
- Groover, M. P. (2020). *Automation, Production Systems, and Computer-Integrated Manufacturing* (5th ed.). Pearson.
- Waters, T. R., Putz-Anderson, V., & Garg, A. (1994). *Applications Manual for the Revised NIOSH Lifting Equation*. NIOSH Publication No. 94-110.
- Tompkins, J. A., White, J. A., Bozer, Y. A., & Tanchoco, J. M. A. (2010). *Facilities Planning* (4th ed.). Wiley.
- FDA. (2021). *Current Good Manufacturing Practice, Hazard Analysis, and Risk-Based Preventive Controls for Human Food*. 21 CFR Part 117.
- OSHA. (2021). *Ergonomics: The Study of Work*. OSHA Publication 3125.

Chapter 5

Simulation Modeling with AnyLogic

5.1 Learning Objectives

After completing this chapter, you will be able to:

1. Explain what simulation modeling is and why it is essential for manufacturing system design
 2. Describe the three simulation paradigms available in AnyLogic (discrete-event, agent-based, and system dynamics)
 3. Identify the key components and libraries in AnyLogic for manufacturing simulation
 4. Build a basic production line simulation using the Process Modeling Library
 5. Model warehouse picking and order fulfillment operations
 6. Analyze simulation results to optimize throughput, reduce bottlenecks, and improve efficiency
 7. Apply simulation to validate design decisions before physical implementation
 8. Use built-in AnyLogic examples to accelerate model development
-

5.2 Introduction to Simulation Modeling

Before You Read: Imagine you are designing a new automotive assembly line that will cost \$50 million to build. How would you test whether your design

will meet production targets *before* spending any money on construction?

In manufacturing, making the wrong design decision can be incredibly expensive. A misplaced workstation, an undersized conveyor, or an incorrect staffing level can result in millions of dollars in lost productivity. **Simulation modeling** allows engineers to build virtual representations of manufacturing systems and test them under various conditions—all without spending a single dollar on physical equipment.

5.2.1 What is Simulation?

Simulation is the process of creating a computer model that mimics the behavior of a real-world system over time. Unlike static calculations or spreadsheet analysis, simulation captures the *dynamic* nature of manufacturing systems:

- Products arrive at random intervals
- Machines break down unexpectedly
- Workers take breaks
- Quality defects occur randomly
- Rush orders disrupt schedules

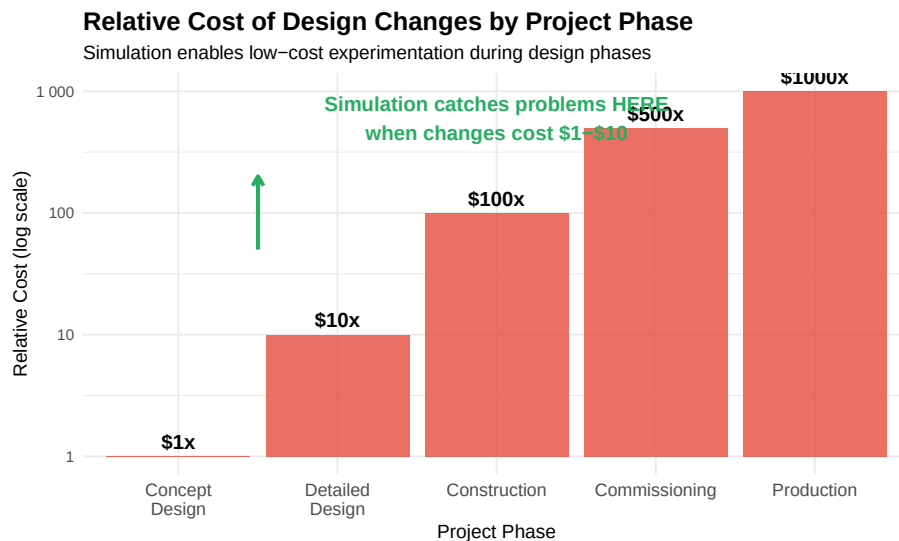


Figure 5.1: The value of simulation in reducing project risk

Industry Application: Mercedes-Benz engineers used AnyLogic simulation to optimize their Sprinter van production line, achieving a **5% efficiency gain** by identifying and eliminating bottlenecks before any physical changes were made. This saved millions in potential rework costs.

5.2.2 Why AnyLogic?

AnyLogic is one of the world's leading simulation platforms, used by companies including Tesla, BMW, Amazon, Ford, and General Electric. It stands out from other simulation software for several key reasons:

Table 5.1: AnyLogic compared to typical simulation software

Feature	AnyLogic	Other Software
Multi-method modeling	Yes - all three paradigms	Usually single method
Manufacturing libraries	Material Handling Library	Varies
3D visualization	Built-in 3D	Often requires plugins
Cloud-based simulation	AnyLogic Cloud	Limited
Academic version	Free for students	Varies
Agent-based modeling	Industry-leading	Limited support

5.3 The Three Simulation Paradigms

AnyLogic is unique because it supports **three different simulation approaches** within a single platform. Understanding when to use each paradigm is essential for effective modeling.

5.3.1 Discrete-Event Simulation (DES)

Discrete-event simulation models systems as a sequence of events that occur at specific points in time. Between events, nothing changes. This is the most common approach for manufacturing simulation.

Key Concept: Discrete-Event Simulation In DES, entities (products, orders, customers) flow through a series of processes (workstations, queues, resources). The simulation “jumps” from event to event, making it computationally efficient.

Best for: - Assembly lines and production systems - Warehouse operations and logistics - Service processes and queuing systems - Call centers and order processing

5.3.2 Agent-Based Modeling (ABM)

Agent-based modeling simulates systems by defining individual agents (workers, robots, vehicles) that have their own behaviors, goals, and decision rules. Agents interact with each other and their environment, and complex system behavior *emerges* from these interactions.

Discrete-Event Simulation: Entities Flow Through Processes

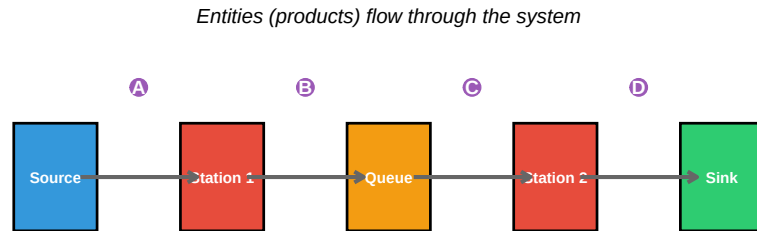


Figure 5.2: Discrete-event simulation flow diagram

Best for: - Autonomous mobile robots (AMRs) and AGVs - Pedestrian and crowd simulation - Market and consumer behavior - Complex adaptive systems

Industry Application: Tesla uses agent-based simulation in AnyLogic to optimize the deployment of autonomous mobile robots in their manufacturing facilities. Each robot is modeled as an agent with its own task allocation logic, collision avoidance, and charging behavior.

5.3.3 System Dynamics (SD)

System dynamics models systems using stocks, flows, and feedback loops. It focuses on aggregate behavior over time rather than individual entities.

Best for: - Strategic planning and capacity modeling - Supply chain dynamics - Policy analysis - Long-term trends and forecasting

5.3.4 Choosing the Right Paradigm

Warning: The ``label.size`` argument of ``geom_label()`` is deprecated as of ggplot2 3.5.0. Please use the ``linewidth`` argument instead.

Quick Check: A company wants to simulate their new automated warehouse with 50 mobile robots that must navigate, avoid collisions, and coordinate pick-ups. Which paradigm would you recommend and why?

5.4 The AnyLogic Interface and Libraries

5.4.1 The AnyLogic Workspace

When you open AnyLogic, you'll see several key areas:

Selecting the Appropriate Simulation Paradigm

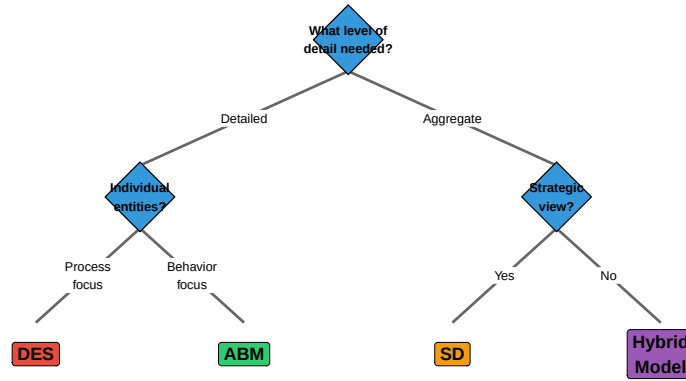


Figure 5.3: Decision framework for selecting simulation paradigm

1. **Model Browser** (left panel) - Shows all elements in your model
2. **Graphical Editor** (center) - Where you build your model visually
3. **Palette** (right panel) - Contains libraries and elements you can drag into your model
4. **Properties** (bottom) - Configure selected elements

5.4.2 Essential Libraries for Manufacturing

AnyLogic includes specialized libraries that dramatically accelerate model development:

Table 5.2: Key AnyLogic libraries for manufacturing simulation

Library	Primary Use	Key Elements
Process Modeling Library	Discrete-event simulation of any process	Source, Queue, Delay, Service, Sink
Material Handling Library	Conveyors, cranes, AGVs, forklifts	Conveyor, Station, Transporter, Crane
Pedestrian Library	Worker movement and ergonomics	PedSource, PedService, PedSink
Road Traffic Library	Truck logistics and yard management	CarSource, Road, Intersection
Rail Library	Rail-based transport systems	RailNetwork, Train, RailYard
Fluid Library	Bulk material and liquid processing	Tank, Pipe, Valve, FlowRate

5.4.3 The Process Modeling Library

The **Process Modeling Library** is the foundation for most manufacturing simulations. It provides building blocks for creating flowchart-based models:

Process Modeling Library Elements

Drag and connect these elements to build your simulation



Figure 5.4: Key Process Modeling Library elements

Hands-On Activity: Open AnyLogic and create a new model. Drag a Source, Queue, Delay, and Sink from the Process Modeling Library. Connect them in sequence. Run the simulation and observe entities flowing through.

5.5 Production Line Simulation

Now let's apply these concepts to design and analyze an automotive production line. We'll walk through a complete example using AnyLogic's built-in examples as a starting point.

5.5.1 Case Study: Automotive Door Panel Assembly Line

The Scenario: You are designing a door panel assembly line for an automotive supplier. The line must:

- Process **480 door panels per 8-hour shift** (1 panel per minute)
- Include 5 workstations: stamping, welding, painting, assembly, and inspection
- Meet quality target of **< 2% defect rate**
- Maintain **85% overall equipment effectiveness (OEE)**

Your task is to simulate this line to verify it meets these requirements before construction begins.

5.5.2 Step 1: Calculate Required Takt Time

Before building the simulation, we need to calculate the **takt time**—the maximum time allowed per unit to meet demand.

$$\text{Takt Time} = \frac{\text{Available Production Time}}{\text{Customer Demand}} \quad (5.1)$$

For our example:

$$\text{Takt Time} = \frac{8 \text{ hours} \times 60 \text{ min/hour}}{480 \text{ panels}} = 1 \text{ minute/panel}$$

This means each workstation must complete its operation in **less than 1 minute** to avoid becoming a bottleneck.

5.5.3 Step 2: Define Process Parameters

Table 5.3: Process parameters for door panel assembly line

Station	Mean Time (sec)	Std Dev (sec)	Defect Rate (%)	Availability (%)
Stamping	45	5	0.5	95
Welding	52	8	0.8	92
Painting	48	6	0.6	90
Assembly	55	7	0.7	94
Inspection	30	4	0.0	98

5.5.4 Step 3: Build the Simulation Model

In AnyLogic, we would create this model by:

1. Dragging a **Source** block to generate door panels at the required rate
2. Adding **Service** blocks for each workstation (with appropriate process times)
3. Connecting **Queue** blocks between stations to handle waiting
4. Adding **SelectOutput** blocks after each station to route defective parts
5. Placing a **Sink** at the end to collect finished panels

5.5.5 Step 4: Simulate and Analyze Results

Let's simulate what the output might look like when we run this model:

Production Line Model Structure in AnyLogic

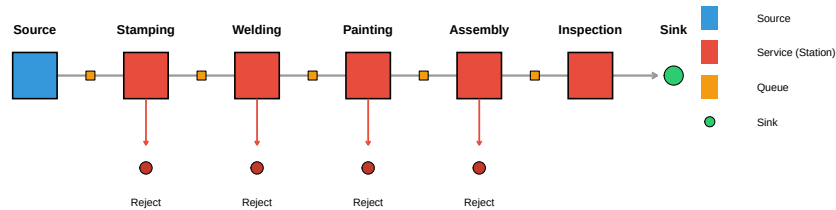


Figure 5.5: AnyLogic production line model structure

```
# Simulate production line results
set.seed(123)

# Generate hourly production data
hours <- 1:8
good_parts <- c(56, 58, 55, 57, 59, 54, 58, 57)
defects <- c(1, 2, 2, 1, 1, 3, 1, 2)
target <- rep(60, 8)

production_df <- data.frame(
  Hour = hours,
  Good = good_parts,
  Defects = defects,
  Target = target
)

# Generate queue length data over time
time_points <- seq(0, 480, by = 5)
queue_data <- data.frame(
  Time = rep(time_points, 5),
  Queue = rep(c("Q1", "Q2", "Q3", "Q4", "Q5"), each = length(time_points)),
  Length = c(
    pmax(0, rnorm(length(time_points), 2, 1)), # Q1
    pmax(0, rnorm(length(time_points), 5, 2)), # Q2 - bottleneck
    pmax(0, rnorm(length(time_points), 3, 1.5)), # Q3
    pmax(0, rnorm(length(time_points), 4, 1.8)), # Q4
    pmax(0, rnorm(length(time_points), 1, 0.5)) # Q5
  )
)

# Create multi-panel plot
p1 <- ggplot(production_df, aes(x = Hour)) +
```



```

geom_col(aes(y = Good), fill = "#2ECC71", alpha = 0.8) +
geom_col(aes(y = -Defects), fill = "#E74C3C", alpha = 0.8) +
geom_line(aes(y = Target), color = "#3498DB", linewidth = 1.2, linetype = "dashed") +
geom_hline(yintercept = 0, color = "black") +
annotate("text", x = 8.5, y = 60, label = "Target", color = "#3498DB", fontface = "bold") +
labs(
  title = "Hourly Production Output",
  subtitle = "Green = Good parts, Red = Defects, Dashed = Target",
  x = "Hour",
  y = "Units"
) +
theme_minimal()

p2 <- ggplot(queue_data, aes(x = Time, y = Length, color = Queue)) +
  geom_line(alpha = 0.7) +
  geom_hline(yintercept = 5, linetype = "dashed", color = "red") +
  annotate("text", x = 450, y = 6, label = "Warning\nThreshold",
    color = "red", size = 3 * diagram_text_scale) +
  scale_color_brewer(palette = "Set1") +
  labs(
    title = "Queue Lengths Over Shift",
    subtitle = "Q2 (after Stamping) shows potential bottleneck",
    x = "Time (minutes)",
    y = "Queue Length"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

gridExtra::grid.arrange(p1, p2, nrow = 2)

```

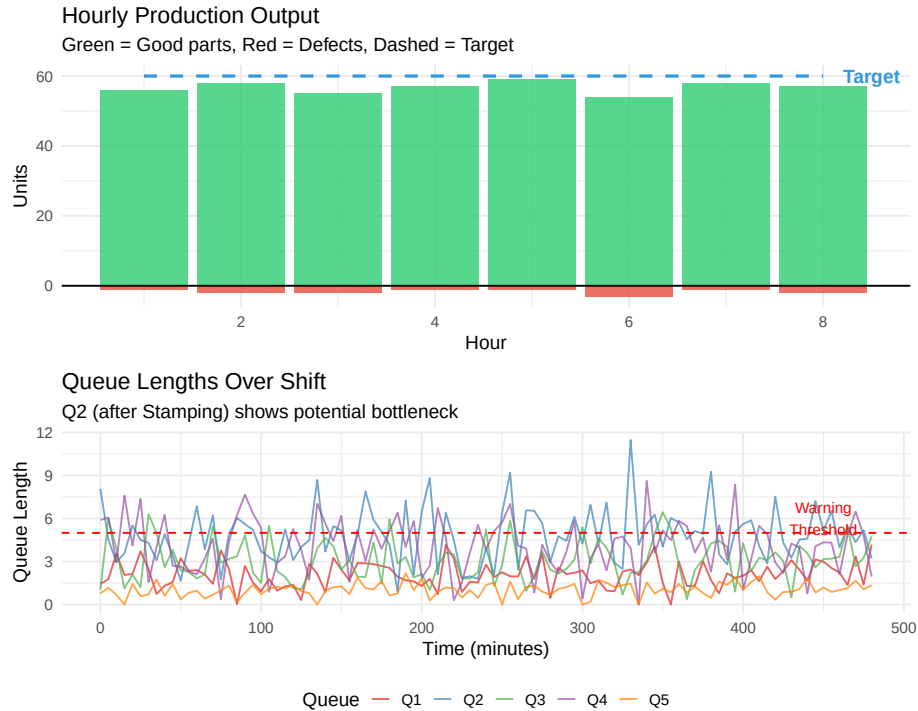


Figure 5.6: Simulated production line performance over 8-hour shift

5.5.6 Interpreting the Results

From our simulation, we can observe:

1. **Production Target:** The line produces approximately 454 good parts per shift ($454/480 = 94.6\%$ of target)
2. **Defect Rate:** 13 defects / 467 total = 2.8% (exceeds our 2% target)
3. **Bottleneck Identified:** Queue Q2 (after Stamping) consistently has the longest queue, indicating the **Welding station** is the bottleneck

⚠ Warning

Critical Finding: The Welding station with a mean process time of 52 seconds and 8% variability is creating a bottleneck. Even though 52 seconds is under the 60-second takt time, the variability combined with 92% availability causes cumulative delays.

5.5.7 Step 5: Test Improvement Scenarios

The power of simulation is testing “what-if” scenarios. Let’s compare different improvement options:

```
# Create scenario comparison data
scenarios <- data.frame(
  Scenario = factor(c("Current State", "Add Buffer", "Reduce Variability",
    "Add Parallel Station", "Combined Approach"),
    levels = c("Current State", "Add Buffer", "Reduce Variability",
      "Add Parallel Station", "Combined Approach")),
  Throughput = c(454, 462, 471, 485, 498),
  Investment = c(0, 15, 50, 200, 180),
  ROI = c(0, 53, 34, 15.5, 24.4)
)

# Throughput comparison
ggplot(scenarios, aes(x = Scenario, y = Throughput, fill = Scenario)) +
  geom_col() +
  geom_hline(yintercept = 480, linetype = "dashed", color = "red", linewidth = 1) +
  geom_text(aes(label = Throughput), vjust = -0.5, fontface = "bold") +
  annotate("text", x = 5, y = 485, label = "Target: 480", color = "red") +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Throughput by Improvement Scenario",
    subtitle = "Simulation results for 8-hour shift",
    x = "",
    y = "Parts per Shift"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    axis.text.x = element_text(angle = 30, hjust = 1)
  )
```

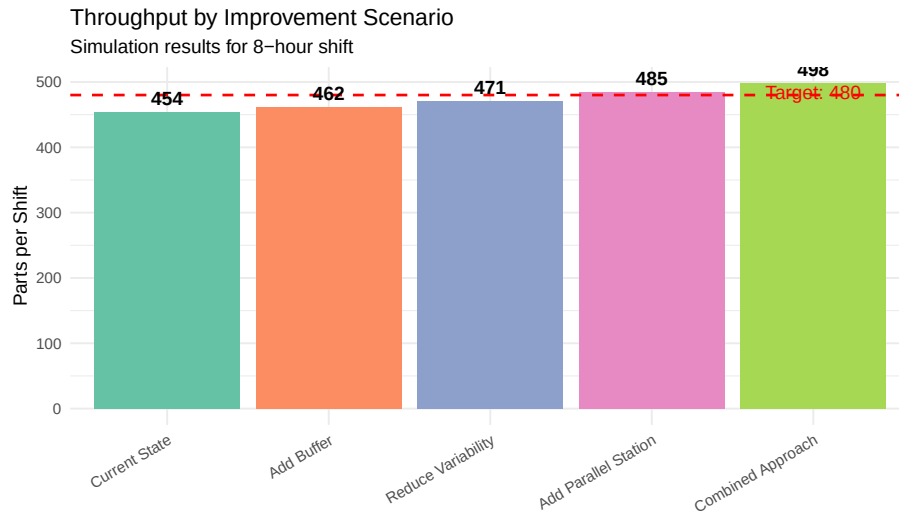


Figure 5.7: Comparison of improvement scenarios

Group Discussion: Based on these results, which improvement option would you recommend? Consider not just throughput but also implementation cost, timeline, and risk. Discuss the trade-offs with your team.

5.6 Warehouse Simulation: Picking and Order Fulfillment

Modern warehouses face immense pressure to fulfill orders quickly and accurately. Simulation helps optimize picking strategies, staffing levels, and automation investments.

5.6.1 The Warehouse Challenge

Before You Read: If you had to fill 1,000 customer orders per day from a warehouse with 50,000 different products (SKUs), how would you organize the picking process? What factors would affect efficiency?

Warehouse operations involve complex interactions between:

- **Order arrivals** (often unpredictable)
- **Picker movement** (walking is typically 50% of picker time)
- **Storage locations** (where products are stored affects travel time)
- **Picking strategies** (single-order vs. batch picking)

- **Material handling equipment** (carts, forklifts, conveyor systems)
- **Packing and shipping** (labor-intensive final steps)

5.6.2 Case Study: E-Commerce Distribution Center

The Scenario: An e-commerce company is designing a new distribution center to handle:

- **13,000 order lines per day** (an order line = one product in one order)
- **750 picking cartons** processed daily
- Peak periods with **3x normal order volume**
- Delivery promise of **same-day shipping** for orders before 2 PM

They need to decide: What picking algorithm will minimize fulfillment time while meeting labor cost targets?

5.6.3 Picking Strategies Compared

AnyLogic allows us to simulate and compare different picking strategies:

Table 5.4: Warehouse picking strategies

Strategy	Description	Best For
Single-Order Picking	One picker completes one order at a time	Small operations, high accuracy
Batch Picking	Picker collects items for multiple orders simultaneously	High-volume with clustered orders
Zone Picking	Warehouse divided into zones; each picker handles one zone	Very large warehouses
Wave Picking	Orders grouped into waves released at intervals	Managing workload throughout the day
Cluster Picking	Multiple orders picked into compartmentalized cart	Medium volume, diverse product range

5.6.4 Simulating Picker Movement

One of the key factors in warehouse efficiency is picker travel distance. AnyLogic can model this using the **Pedestrian Library** combined with the **Process Modeling Library**.

```
# Simulate picker travel data for different storage strategies
set.seed(456)

# Random storage vs. velocity-based storage
n_picks <- 500
random_storage <- rnorm(n_picks, mean = 180, sd = 40) # feet per pick
velocity_storage <- rnorm(n_picks, mean = 95, sd = 25) # feet per pick
abc_storage <- rnorm(n_picks, mean = 120, sd = 30) # feet per pick

travel_data <- data.frame(
  Strategy = rep(c("Random Storage", "ABC Analysis", "Velocity-Based"), each = n_picks),
  Distance = c(random_storage, abc_storage, velocity_storage)
```

```

)

ggplot(travel_data, aes(x = Strategy, y = Distance, fill = Strategy)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = mean(velocity_storage), linetype = "dashed", color = "#2ECC71") +
  stat_summary(fun = mean, geom = "point", shape = 18, size = 4, color = "red") +
  annotate("text", x = 3.35, y = mean(velocity_storage),
           label = paste0("Best: ", round(mean(velocity_storage)), " ft"),
           color = "#2ECC71", size = 4 * diagram_text_scale, fontface = "bold") +
  scale_fill_manual(values = c("#E74C3C", "#F39C12", "#2ECC71")) +
  labs(
    title = "Picker Travel Distance by Storage Strategy",
    subtitle = "Red diamond = mean; Velocity-based storage reduces travel by 47%",
    x = "Storage Assignment Strategy",
    y = "Travel Distance per Pick (feet)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

```

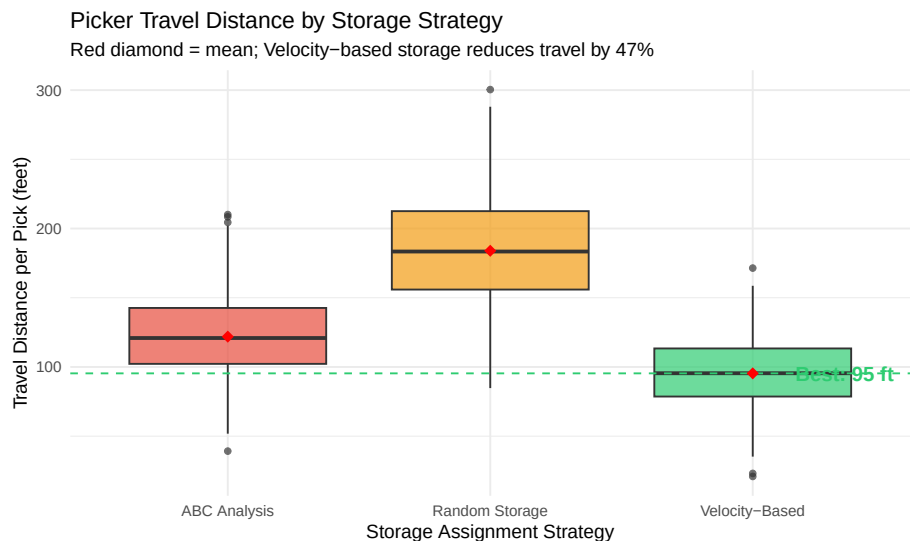


Figure 5.8: Impact of storage assignment on picker travel distance

Industry Application: Kuehne+Nagel used AnyLogic simulation to design a new warehouse processing 13,000 order lines per day. By testing different picking algorithms in simulation, they identified the optimal approach before building the physical facility, avoiding costly redesigns.

5.6.5 Autonomous Mobile Robots (AMRs) in Warehouses

Modern warehouses increasingly use **autonomous mobile robots** to reduce picker travel time. Instead of workers walking to products, robots bring shelving units to stationary pick stations.

```
# Simulate AMR performance
amr_counts <- c(5, 10, 15, 20, 25, 30, 35, 40, 45, 50)
fulfillment_times <- c(45, 28, 18, 13, 10.5, 9, 8.2, 7.8, 7.6, 7.5)
utilization <- c(98, 95, 88, 78, 68, 58, 52, 48, 45, 43)

amr_data <- data.frame(
  AMRs = amr_counts,
  Time = fulfillment_times,
  Utilization = utilization
)

# Dual y-axis plot
coeff <- 0.5

ggplot(amr_data, aes(x = AMRs)) +
  geom_line(aes(y = Time), color = "#3498DB", linewidth = 1.5) +
  geom_point(aes(y = Time), color = "#3498DB", size = 3) +
  geom_line(aes(y = Utilization * coeff), color = "#E74C3C", linewidth = 1.5, linetype = "dashed") +
  geom_point(aes(y = Utilization * coeff), color = "#E74C3C", size = 3) +
  geom_vline(xintercept = 25, linetype = "dotted", color = "#2ECC71", linewidth = 1.2) +
  annotate("text", x = 27, y = 40, label = "Optimal\nPoint", color = "#2ECC71", fontface = "bold")
  scale_y_continuous(
    name = "Fulfillment Time (minutes)",
    sec.axis = sec_axis(~ . / coeff, name = "Robot Utilization (%)")
  ) +
  labs(
    title = "Finding the Optimal Number of Autonomous Mobile Robots",
    subtitle = "Blue = Fulfillment time (lower is better); Red = Robot utilization (higher is bet",
    x = "Number of AMRs"
  ) +
  theme_minimal() +
  theme(
    axis.title.y.left = element_text(color = "#3498DB"),
    axis.title.y.right = element_text(color = "#E74C3C")
  )
)
```

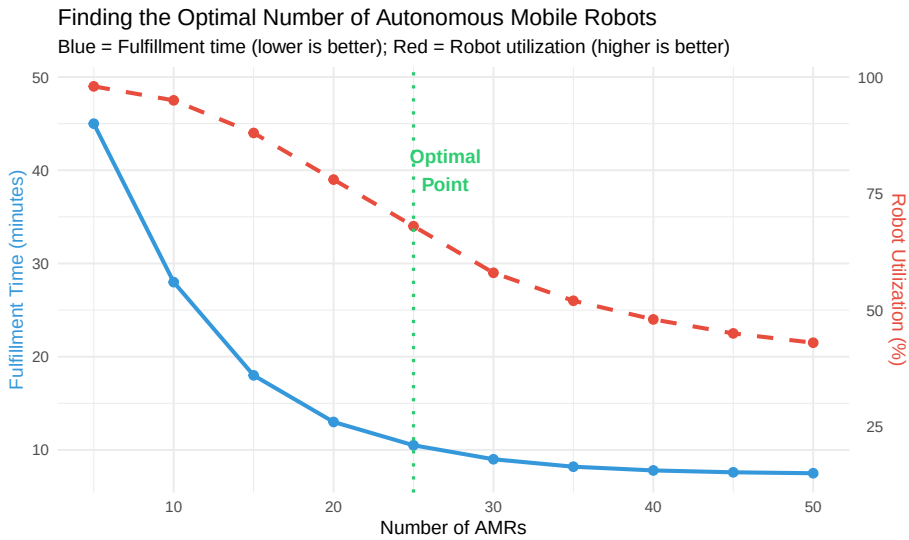


Figure 5.9: Simulation results: order fulfillment time vs. number of AMRs

This simulation reveals a classic trade-off: adding more robots reduces order fulfillment time but also decreases utilization (robots spend more time idle). The **optimal point** balances these factors based on business requirements.

Quick Check: 1. Why does adding more robots eventually show diminishing returns? 2. If robots cost \$50,000 each and labor costs \$25/hour, how would you calculate the breakeven point? 3. What other factors should be considered beyond raw cost?

5.7 Built-in AnyLogic Examples

AnyLogic includes numerous pre-built example models that serve as excellent learning resources and starting points for your own projects. Here are the key examples relevant to manufacturing and warehouse operations:

5.7.1 Manufacturing Examples

Table 5.5: Key manufacturing example models in AnyLogic

Example Name	Location	What You'll Learn
Job Shop	Help > Example Models > Manufacturing	Routing jobs through a shop
Assembly Line	Help > Example Models > Manufacturing	Line balancing

Flexible Manufacturing System	Help > Example Models > Manufacturing	Flexible routing, machine s
Conveyor System	Help > Example Models > Material Handling	Conveyor design, merging,
AGV System	Help > Example Models > Material Handling	Vehicle dispatching, path p

5.7.2 Warehouse and Logistics Examples

Table 5.6: Key warehouse example models in AnyLogic

Example Name	Location	What You’ll Learn
Warehouse	Help > Example Models > Logistics	Storage assignment, inventory management
Distribution Center	Help > Example Models > Logistics	Receiving, put-away, picking, shipping in
Order Picking	Help > Example Models > Logistics	Picking route optimization, batch vs. sin
AS/RS System	Help > Example Models > Material Handling	Automated storage retrieval, crane sequ
Cross-Docking	Help > Example Models > Logistics	Minimal storage, direct transfer operati

Hands-On Activity: Open AnyLogic and navigate to Help > Example Models. Open the “Assembly Line” example and run it. Then: 1. Identify the bottleneck station (highest queue length) 2. Modify the process time for that station 3. Observe how throughput changes 4. Document your findings

5.8 Building Your First Model: Step-by-Step Tutorial

Let’s walk through building a simple but complete production line model from scratch.

5.8.1 Tutorial: Three-Station Production Line

Objective: Model a production line with three workstations and analyze throughput.

5.8.1.1 Step 1: Create a New Model

1. Open AnyLogic
2. File > New > Model
3. Name it “ProductionLine_Tutorial”
4. Select “Discrete Event” as the model type

5.8.1.2 Step 2: Add the Process Flow

From the **Process Modeling Library**, drag these elements onto the canvas in order:

1. **Source** - Generates parts entering the system
2. **Queue** - Holds parts waiting for Station 1
3. **Delay** - Represents Station 1 processing time
4. **Queue** - Holds parts waiting for Station 2
5. **Service** - Represents Station 2 (requires a worker resource)
6. **Queue** - Holds parts waiting for Station 3
7. **Delay** - Represents Station 3 processing time
8. **Sink** - Removes completed parts from the system

5.8.1.3 Step 3: Connect the Elements

Click on the output port of each element and drag to the input port of the next element. The flow should look like:

Source → Queue → Delay → Queue → Service → Queue → Delay → Sink

5.8.1.4 Step 4: Configure Parameters

Click each element and set these properties:

Source: - Arrivals defined by: Inter-arrival time - Inter-arrival time: `exponential(1)` (1 part per minute on average)

Delays (Stations 1 and 3): - Delay time: `triangular(0.5, 0.8, 1.2)` (minutes)

Service (Station 2): - Delay time: `triangular(0.6, 0.9, 1.5)` (minutes) - Requires a Resource Pool (add one and set capacity to 1)

Queues: - Maximum capacity: 10

5.8.1.5 Step 5: Add Statistics Collection

Add these statistics to track performance:

1. **TimeMeasureStart** - After Source
2. **TimeMeasureEnd** - Before Sink
3. **Dataset** - To log throughput over time

5.8.1.6 Step 6: Run and Analyze

```
# Simulate expected results
set.seed(789)
time <- seq(0, 480, by = 1)

# Generate cumulative throughput
throughput <- cumsum(rpois(length(time), lambda = 0.85))

# Generate queue lengths
```

```

q1 <- pmax(0, rnorm(length(time), 1.5, 0.8))
q2 <- pmax(0, rnorm(length(time), 3.2, 1.2)) # Bottleneck
q3 <- pmax(0, rnorm(length(time), 0.8, 0.5))

tutorial_data <- data.frame(
  Time = time,
  Throughput = throughput,
  Q1 = q1,
  Q2 = q2,
  Q3 = q3
)

p1 <- ggplot(tutorial_data, aes(x = Time, y = Throughput)) +
  geom_line(color = "#2ECC71", linewidth = 1) +
  geom_hline(yintercept = 400, linetype = "dashed", color = "red") +
  annotate("text", x = 50, y = 420, label = "Target: 400", color = "red") +
  labs(title = "Cumulative Throughput", x = "Time (min)", y = "Parts Completed") +
  theme_minimal()

tutorial_long <- tutorial_data %>%
  tidyr::pivot_longer(cols = c(Q1, Q2, Q3), names_to = "Queue", values_to = "Length")

p2 <- ggplot(tutorial_long, aes(x = Time, y = Length, color = Queue)) +
  geom_line(alpha = 0.6) +
  scale_color_manual(values = c("#3498DB", "#E74C3C", "#F39C12"),
                    labels = c("Before Station 1", "Before Station 2", "Before Station 3")) +
  labs(title = "Queue Lengths", x = "Time (min)", y = "Parts Waiting") +
  theme_minimal() +
  theme(legend.position = "bottom")

gridExtra::grid.arrange(p1, p2, ncol = 2)

```

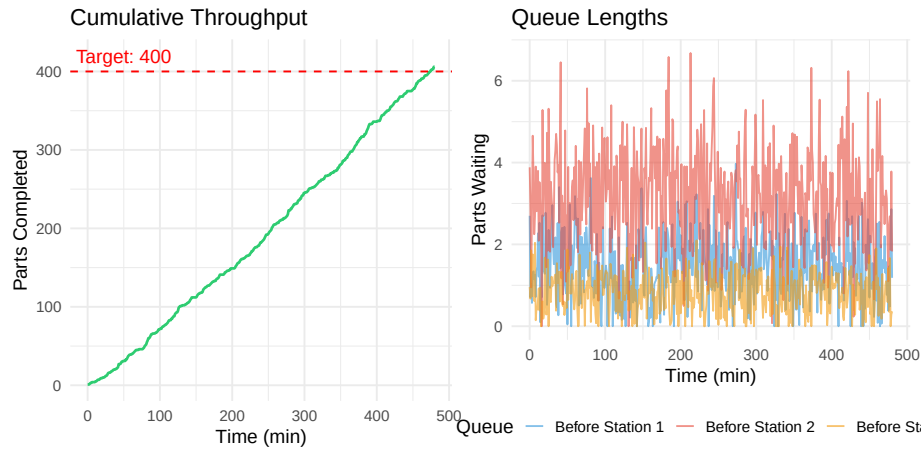


Figure 5.10: Expected results from tutorial model

Verify Your Model: After running, you should observe: - Approximately 400-420 parts completed in 480 minutes - Queue 2 (before Station 2) has the longest average length - The system is stable (queues don't grow without bound)

5.9 Advanced Topics

5.9.1 3D Visualization

AnyLogic includes powerful 3D visualization capabilities that help communicate simulation results to stakeholders who may not understand abstract flowcharts.

Benefits of 3D visualization: - Easier validation with plant managers (“Does this look like our facility?”) - Executive presentations and funding requests - Training material for new employees - Virtual commissioning before physical construction

5.9.2 Integration with Real Data

Production simulations become more valuable when connected to real data:

- **Historical data import:** Load actual order data, machine logs, or production records
- **Real-time dashboards:** Connect to databases for live monitoring
- **Digital twin operation:** Mirror physical operations in real-time

5.9.3 Optimization Experiments

AnyLogic includes an **Optimization** experiment type that automatically searches for the best parameter values:

```
# Simulate optimization process
iterations <- 1:50
staff_levels <- c(
  rep(5, 5), rep(7, 5), rep(6, 5), rep(8, 5), rep(7, 5),
  rep(6, 10), rep(7, 5), rep(7, 5), rep(7, 5)
)
throughput <- c(
  rnorm(5, 380, 10), rnorm(5, 445, 8), rnorm(5, 420, 9), rnorm(5, 450, 7), rnorm(5, 448, 7),
  rnorm(10, 425, 8), rnorm(5, 450, 6), rnorm(5, 452, 5), rnorm(5, 455, 4)
)

opt_data <- data.frame(
  Iteration = iterations,
  Staff = staff_levels,
  Throughput = throughput
)

opt_data$Best <- cummax(opt_data$Throughput)

ggplot(opt_data, aes(x = Iteration)) +
  geom_point(aes(y = Throughput), alpha = 0.5, color = "#3498DB") +
  geom_line(aes(y = Best), color = "#E74C3C", linewidth = 1.2) +
  geom_hline(yintercept = 455, linetype = "dashed", color = "#2ECC71") +
  annotate("text", x = 45, y = 460, label = "Optimal: 7 staff, 455 parts",
    color = "#2ECC71", fontface = "bold") +
  labs(
    title = "Optimization Experiment: Finding Optimal Staffing",
    subtitle = "Blue points = individual runs; Red line = best found so far",
    x = "Iteration",
    y = "Throughput (parts/shift)"
  ) +
  theme_minimal()
```

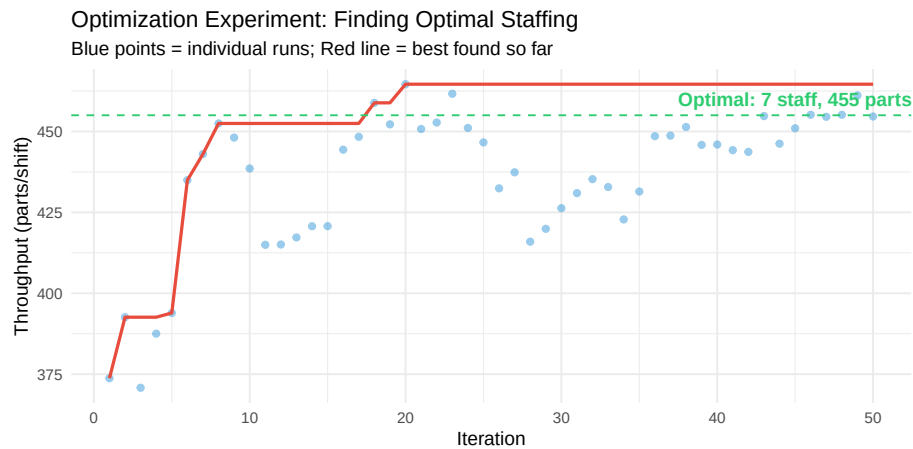


Figure 5.11: Optimization search finding the best staffing level

5.10 Key Takeaways

What We Learned:

1. **Simulation enables risk-free experimentation** - Test design changes in virtual environments before spending money on physical implementation
2. **AnyLogic supports three paradigms** - Discrete-event (for process flows), agent-based (for autonomous entities), and system dynamics (for strategic planning)
3. **Manufacturing simulation identifies bottlenecks** - Queue analysis reveals which stations limit throughput
4. **Warehouse simulation optimizes picking** - Compare strategies like batch picking, zone picking, and AMR deployment before implementation
5. **Built-in examples accelerate learning** - Start from AnyLogic's example models to understand best practices
6. **The value is in the “what-if”** - Simulation's true power is comparing scenarios to find optimal solutions

Key Equations: - Takt Time: Equation Equation 5.1

Before Moving On: - Can you explain when to use discrete-event vs. agent-based simulation? - Can you build a basic three-station production line in AnyLogic? - Can you identify how simulation could help in your workplace or industry?

5.11 Practice Problems

5.11.1 Problem 1: Takt Time Calculation

A refrigerator assembly plant operates two 8-hour shifts per day with a 30-minute lunch break per shift. Customer demand is 720 units per day.

- Calculate the takt time in seconds
- If the bottleneck station has a cycle time of 62 seconds, will they meet demand? Why or why not?
- Suggest two approaches to resolve the issue

5.11.2 Problem 2: Simulation Model Design

You are asked to simulate a paint shop with: - 3 spray booths operating in parallel - Conveyor system bringing parts from preparation area - Curing oven with 30-minute cycle time - Quality inspection with 5% rejection rate

- Sketch the simulation model structure
- List which AnyLogic libraries you would use
- Identify potential bottlenecks to investigate

5.11.3 Problem 3: Warehouse Picking Analysis

A warehouse uses single-order picking with 5 pickers. Each picker walks an average of 3 miles per 8-hour shift. The company is considering batch picking with specialized carts.

- If batch picking reduces travel by 35%, how many miles would be saved per picker per shift?
- If workers walk at 3 mph and pick items at a rate of 2 per minute while at a location, what percentage of time is currently spent walking?
- What information would you need to simulate this scenario in AnyLogic?

5.11.4 Problem 4: AMR Fleet Sizing

Using the graph in Figure 5.9 as a reference:

- If the company requires fulfillment time under 12 minutes, what is the minimum number of AMRs needed?
- What trade-off occurs when moving from 25 to 50 AMRs?
- If each AMR costs \$50,000 and operates for 5 years, what is the daily cost per AMR?

Challenge Problem: Design a simulation experiment to determine the optimal number of pickers and AMRs for a warehouse with variable order volumes.

The solution should: - Handle daily order volumes ranging from 5,000 to 20,000 lines - Meet a 2-hour fulfillment target - Minimize total labor and equipment costs

5.12 Further Resources

5.12.1 AnyLogic Official Resources

- **AnyLogic University:** Free online courses covering all simulation paradigms
- **Example Models:** Help > Example Models in AnyLogic software
- **Cloud Platform:** cloud.anylogic.com for running simulations online

5.12.2 Academic Resources

- Borshchev, A. (2013). *The Big Book of Simulation Modeling*. AnyLogic North America.
- Law, A.M. (2015). *Simulation Modeling and Analysis* (5th ed.). McGraw-Hill.

5.12.3 Industry Case Studies

- Mercedes-Benz Sprinter production optimization
- Kuehne+Nagel warehouse picking algorithm selection
- Tesla autonomous mobile robot deployment

Tip

Getting Started: Download the free Personal Learning Edition of AnyLogic from anylogic.com. It includes all features and example models—the only limitation is model size.

Chapter 6

Integrated Manufacturing Systems

6.1 Learning Objectives

By the end of this chapter, you will be able to:

1. Compare different types of production facility layouts and material handling methods
 2. Calculate Manufacturing Cycle Time (MCT) and Manufacturing Cycle Efficiency (MCE)
 3. Analyze production line performance including the impact of station failures
 4. Apply line balancing techniques to optimize workstation assignments
 5. Explain the concepts of takt time, cycle time, and their relationship
 6. Describe Flexible Manufacturing Systems (FMS) and their applications
 7. Understand modern concepts including digital twins and Industry 4.0
-

6.2 Material Handling Methods & Systems

There are four common types of production facility layouts, each with distinct features and material handling requirements:

Table 6.1: Material Handling Methods by Plant Layout

Layout Type	Product Characteristics	Material Handling
Fixed-Position	Large, heavy, or immobile products	Cranes, hoists, fork lifts bring materials
Process (Job Shop)	High variety, low volume, custom work	Fork lifts, AGVs, carts move WIP
Cellular	Part families, medium variety/volume	Conveyors within cells, carts between cells
Product (Flow Line)	Standardized products, high volume	Conveyors, automated transfer systems

Discussion: How do you choose between layout types?

Selection criteria:

Factor	Favors Process Layout	Favors Product Layout
Volume	Low (< 1,000/year)	High (> 10,000/year)
Variety	High (many products)	Low (1-2 products)
Routing	Variable by product	Fixed, same for all
Equipment	General-purpose	Specialized
Worker skills	High, varied	Specific, repetitive
WIP inventory	High	Low
Lead time	Long	Short

When in doubt: If volume and variety are moderate, consider cellular manufacturing as a compromise.

6.3 Fundamentals of Production Lines

A **production line** consists of workstations arranged so that the product moves from one station to the next, with a portion of the total work performed at each location.

Warning: Using ``size`` aesthetic for lines was deprecated in ggplot2 3.4.0. i Please use ``linewidth`` instead.

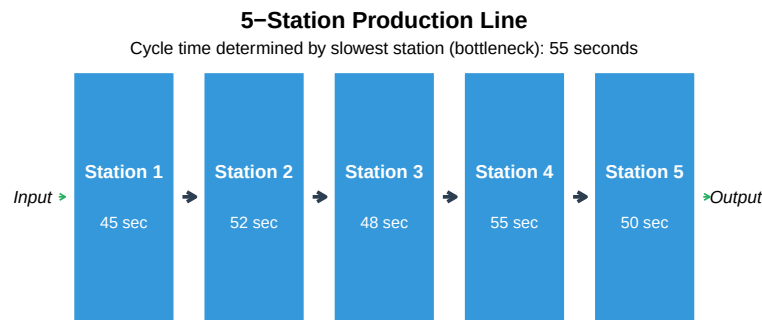


Figure 6.1: Production Line Concept

6.3.1 Methods of Work Transport

Table 6.2: Work Transport Methods

Method	Description	Advantages
Manual	Workers pass parts by hand or using simple carts	Low cost, flexible, easy to change
Continuous	Conveyor moves at constant speed; work done while moving	Smooth flow, no transfer time
Synchronous (Intermittent)	All units move simultaneously in discrete steps	Fixed cycle time, easy to control
Asynchronous	Units move independently when released by worker	Absorbs variation, no blocking

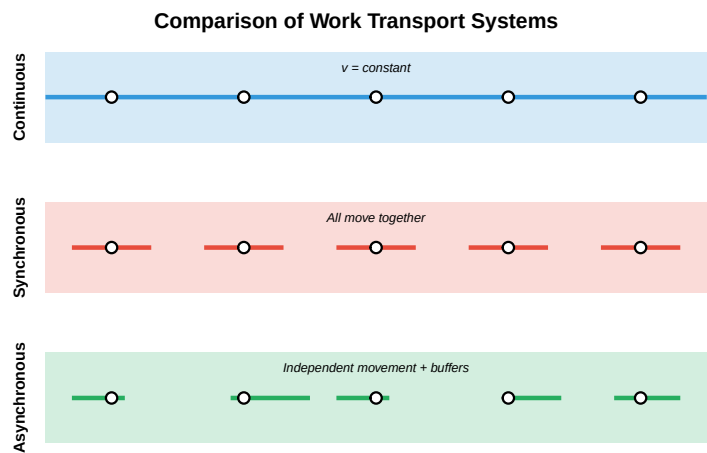


Figure 6.2: Work Transport System Types

What causes blocking and starving in production lines?

Starving: A station is ready to work but has no parts to process. - **Causes:** Upstream station is slower, upstream breakdown, material shortage

Blocking: A station finishes but cannot release the part. - **Causes:** Downstream station still working, downstream breakdown, no buffer space

Solutions: 1. **Buffers** - Storage between stations absorbs variation 2. **Line balancing** - Equalize work content across stations 3. **Parallel stations** - Add capacity at bottlenecks 4. **Preventive maintenance** - Reduce breakdowns

6.3.2 Work Cycle Types

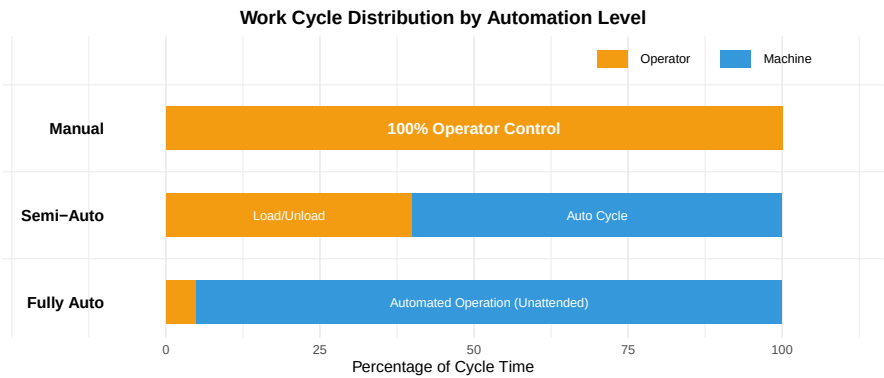


Figure 6.3: Manual, Semi-Automated, and Fully Automated Machine Cycles

6.4 Key Time Metrics

Understanding time metrics is essential for analyzing and improving production systems.

Table 6.3: Key Manufacturing

Metric	Definition	Formula
Takt Time	Available production time ÷ Customer demand	$T_{\text{takt}} = \frac{\text{text}}$
Cycle Time	Time between units coming off the line	$T_c = \frac{60}{R_p}$
Lead Time	Total time from order to delivery	Order processing + Manuf
Throughput Time	Time from raw material to finished goods	Processing + Waiting + M

6.4.1 Takt Time vs. Cycle Time

Live cell — try different demand, shift length, or downtime figures.

```

#| label: takt-vs-cycle
# Takt Time Calculation Example
# Automotive assembly plant

# Given data
shift_length_min <- 480      # 8 hours = 480 minutes
breaks_min <- 30            # Two 15-minute breaks
planned_downtime_min <- 20  # Changeover, meetings
customer_demand <- 60       # Cars per shift

# Calculate available time
available_time <- shift_length_min - breaks_min - planned_downtime_min
cat("Available Production Time:", available_time, "minutes per shift\n")

# Calculate Takt Time
takt_time <- available_time / customer_demand
cat("Takt Time:", round(takt_time, 2), "minutes per car\n")
cat("          =", round(takt_time * 60, 1), "seconds per car\n\n")

# Compare to actual cycle time
actual_cycle_time <- 7.5 # minutes (current performance)
cat("Actual Cycle Time:", actual_cycle_time, "minutes per car\n")

if (actual_cycle_time <= takt_time) {
  cat("Status: MEETING DEMAND - Cycle time is within takt time\n")
  cat("Capacity margin:", round((1 - actual_cycle_time/takt_time) * 100, 1), "%\n")
} else {
  cat("Status: CANNOT MEET DEMAND - Cycle time exceeds takt time\n")
  cat("Shortfall:", round((actual_cycle_time/takt_time - 1) * 100, 1), "% over capacity\n")
}

```

Discussion: What happens when cycle time exceeds takt time?

When Cycle Time > Takt Time, you cannot meet customer demand.

Options to fix this:

1. **Reduce cycle time:**
 - Kaizen improvements at bottleneck
 - Better tools/fixtures
 - Automation of slow tasks
2. **Increase available time:**
 - Add overtime
 - Add shifts
 - Reduce breaks (carefully!)
3. **Add capacity:**
 - Parallel stations at bottleneck

- Additional production lines
 - Outsourcing
4. **Manage demand:**
- Negotiate delivery schedules
 - Build inventory in advance
 - Redirect to other facilities

Key insight: Takt time is the “heartbeat” of lean manufacturing - everything should be synchronized to it.

6.4.2 Manufacturing Cycle Time and Efficiency

This one is live — edit the time breakdown and re-run it.

```
#| label: mct-mce
# Manufacturing Cycle Time (MCT) and Efficiency (MCE) Example

# Given production data
production_time_min <- 480 # Total time equipment ran
units_produced <- 60      # Good units produced

# MCT Calculation
MCT <- production_time_min / units_produced
cat("Manufacturing Cycle Time (MCT):", MCT, "minutes per unit\n\n")

# MCE Calculation requires understanding of value-added time
# Typical breakdown of production time:
value_added_time <- 3.5 # Actual processing time (minutes)
inspection_time <- 0.5  # Quality checks
wait_time <- 2.5        # Queuing between operations
move_time <- 1.5        # Transport between stations

total_time_per_unit <- value_added_time + inspection_time + wait_time + move_time
cat("Time breakdown per unit:\n")
cat("  Value-added (processing):", value_added_time, "min\n")
cat("  Inspection:", inspection_time, "min\n")
cat("  Waiting:", wait_time, "min\n")
cat("  Moving:", move_time, "min\n")
cat("  Total:", total_time_per_unit, "min\n\n")

# MCE Calculation
MCE <- (value_added_time / total_time_per_unit) * 100
cat("Manufacturing Cycle Efficiency (MCE):", round(MCE, 1), "%\n")
cat("\nInterpretation: Only", round(MCE, 1), "% of time adds value.\n")
cat("Opportunity:", round(100 - MCE, 1), "% of time is waste that could be reduced.\n")
```

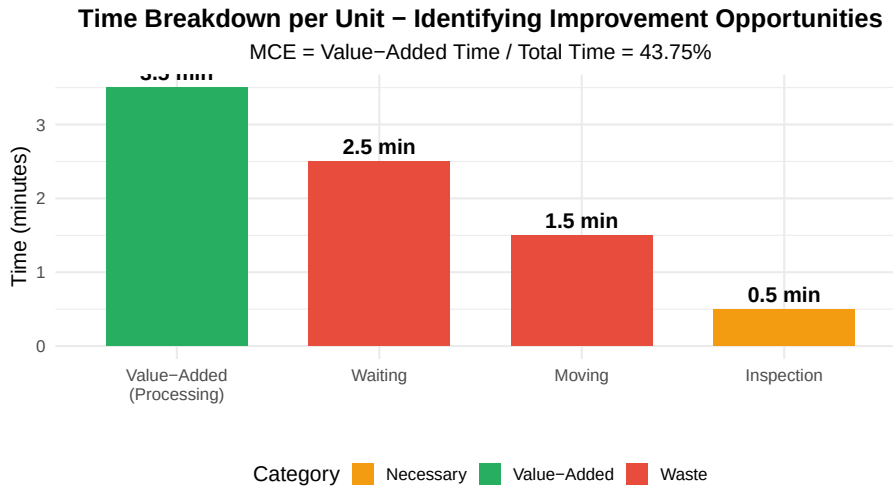


Figure 6.4: Manufacturing Cycle Efficiency - Where Does Time Go?

6.5 Manual Assembly Lines

Manual assembly lines use human workers at each station. The efficiency depends on how well work is balanced across stations.

6.5.1 Line Balancing

Line balancing assigns tasks to workstations to minimize idle time while meeting production rate requirements.

Key formulas:

$$T_c = \frac{\text{Available Time}}{\text{Required Output}} = \frac{60}{R_p}$$

$$TM = \frac{\sum t_i}{T_c} = \frac{T_{wc}}{T_c}$$

$$E_b = \frac{T_{wc}}{n \times T_c} \times 100\%$$

Where: - T_c = Cycle time (time available per unit) - R_p = Production rate (units per hour) - TM = Theoretical minimum stations - T_{wc} = Total work content (sum of all task times) - n = Actual number of stations - E_b = Line efficiency (balance efficiency)

6.5.2 Line Balancing Example: Electronics Assembly

Live cell — try different task times or a different desired output rate.

```
#| label: line-balance-example
# Line Balancing Example: Tablet Computer Assembly

# Task data
tasks <- data.frame(
  Task = c("A", "B", "C", "D", "E", "F", "G", "H"),
  Description = c("Install battery", "Mount display", "Connect cables",
    "Install motherboard", "Add memory", "Install camera",
    "Attach case back", "Final test"),
  Time_sec = c(30, 45, 20, 55, 15, 25, 35, 40),
  Predecessors = c("-", "A", "A", "B,C", "D", "D", "E,F", "G")
)

# Display task data
cat("Assembly Tasks:\n")
print(tasks[, c("Task", "Description", "Time_sec", "Predecessors")])

# Calculate metrics
total_work_content <- sum(tasks$Time_sec)
cat("\nTotal Work Content:", total_work_content, "seconds\n")

# Production requirements
desired_output <- 45 # units per hour
available_time <- 3600 # seconds per hour

# Step 1: Calculate cycle time
cycle_time <- available_time / desired_output
cat("\nStep 1 - Cycle Time:")
cat("\n Required output:", desired_output, "units/hour")
cat("\n Cycle time = 3600 /", desired_output, "=", cycle_time, "seconds\n")

# Step 2: Calculate theoretical minimum stations
TM <- total_work_content / cycle_time
cat("\nStep 2 - Theoretical Minimum Stations:")
cat("\n TM =", total_work_content, "/", cycle_time, "=", round(TM, 2))
cat("\n Minimum stations required:", ceiling(TM), "\n")

# Step 3: Assign tasks to stations (using largest task time rule)
cat("\nStep 3 - Station Assignments (respecting precedence):\n")
cat(" Station 1: A (30s) + C (20s) + B (45s) = 95s > 80s... EXCEEDS\n")
cat(" Let's try: A (30s) + C (20s) = 50s [30s idle]\n")
cat(" Station 2: B (45s) + E (15s) = 60s... wait, E needs D\n")
```



```

cat("  Station 2: B (45s) + D (55s) = 100s > 80s... EXCEEDS\n")
cat("  Station 2: B (45s) = 45s [35s idle]\n")
cat("  Station 3: D (55s) = 55s [25s idle]\n")
cat("  Station 4: E (15s) + F (25s) + G (35s) = 75s [5s idle]\n")
cat("  Station 5: H (40s) = 40s [40s idle]\n")

n_stations <- 5
efficiency <- (total_work_content / (n_stations * cycle_time)) * 100
balance_delay <- 100 - efficiency

cat("\nStep 4 - Results:")
cat("\n  Number of stations:", n_stations)
cat("\n  Line efficiency:", round(efficiency, 1), "%")
cat("\n  Balance delay (idle time):", round(balance_delay, 1), "%")
cat("\n\n  This is poor balance - consider redesigning tasks or adding parallel stations.")

```

Precedence Diagram: Tablet Assembly

Arrows indicate task sequence requirements

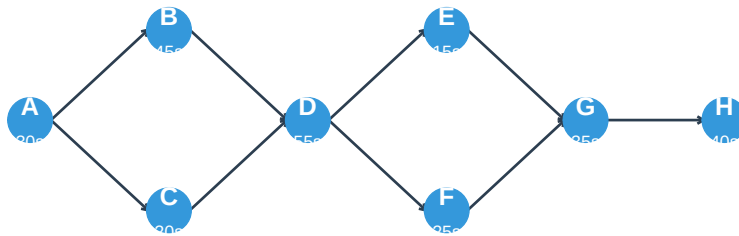


Figure 6.5: Precedence Diagram for Tablet Assembly

Try It: Can you find a better balance?

Challenge: Reassign tasks to improve efficiency above 80%.

Hints: - Can task A be split into two smaller tasks? - Could parallel workstations handle the bottleneck task D? - What if we allow 85 seconds per station (slower line, fewer stations)?

One solution with parallel stations: - Station 1: A + C = 50s - Station 2a: B = 45s (parallel) - Station 2b: B = 45s (parallel) - Station 3: D = 55s - Station 4: E + F = 40s - Station 5: G + H = 75s

With parallelism at Station 2, effective cycle time stays at 55s (Station 3 bot-

tleneck), but we can now achieve higher throughput.

6.6 Automated Production Lines

Automated lines reduce human intervention using mechanized transfer systems and automated workstations. They're used for high-volume production with well-defined work content.

6.6.1 Impact of Station Failures

The performance of automated lines is degraded by station failures. Even small failure probabilities can significantly reduce output.

Actual Cycle Time:

$$T_p = T_c + pT$$

Actual Production Rate:

$$R_p = \frac{60}{T_p} = \frac{60}{T_c + pT}$$

Where: - T_c = Ideal cycle time (minutes) - p = Probability of station failure per cycle - T = Average downtime per failure (minutes)

Live cell — try adding a scenario or changing the number of stations.

```
#| label: downtime-analysis
# Automated Line Performance Analysis

# Line parameters
ideal_cycle_time <- 1.0      # minutes per part (ideal)
n_stations <- 10             # Number of stations

# Scenario comparison
scenarios <- data.frame(
  Scenario = c("World Class", "Good", "Average", "Poor"),
  Failure_Prob = c(0.005, 0.02, 0.05, 0.10), # Per station per cycle
  Repair_Time = c(2, 5, 10, 15) # Minutes average
)

# Calculate line performance for each scenario
cat("Automated Line Performance Analysis\n")
cat("=====\n")
cat("Ideal cycle time:", ideal_cycle_time, "min/part\n")
cat("Number of stations:", n_stations, "\n")
```

```
cat("Ideal production rate:", 60/ideal_cycle_time, "parts/hour\n\n")

for(i in 1:nrow(scenarios)) {
  p <- scenarios$Failure_Prob[i]
  T <- scenarios$Repair_Time[i]

  # Line failure probability (any station)
  p_line <- 1 - (1 - p)^n_stations

  # Actual cycle time
  Tp <- ideal_cycle_time + p_line * T

  # Actual production rate
  Rp <- 60 / Tp

  # Efficiency
  efficiency <- (ideal_cycle_time / Tp) * 100

  cat(sprintf("%s Performance:\n", scenarios$Scenario[i]))
  cat(sprintf("  Station failure prob: %.1f%%\n", p * 100))
  cat(sprintf("  Line failure prob: %.1f%%\n", p_line * 100))
  cat(sprintf("  Average repair time: %d min\n", T))
  cat(sprintf("  Actual cycle time: %.2f min\n", Tp))
  cat(sprintf("  Actual production: %.1f parts/hour\n", Rp))
  cat(sprintf("  Line efficiency: %.1f%%\n\n", efficiency))
}
```

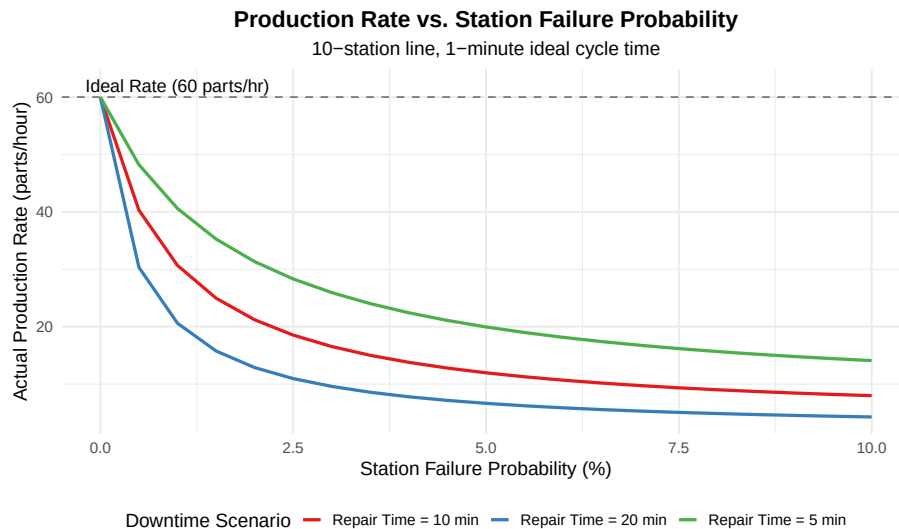


Figure 6.6: Impact of Station Failures on Production Rate

Discussion: Why is preventive maintenance critical for automated lines?

The math shows why: - At 5% failure probability with 10-minute repairs, you lose 25% of capacity - This costs more than the maintenance to prevent it

Cost comparison (example): - Lost production: 15 parts/hour $\times$ \$50/part $\times$ 8 hours = \$6,000/day - Preventive maintenance: \$500/day - **ROI of PM: 12:1**

Best practices: 1. Track failure data by station 2. Prioritize PM on high-failure stations 3. Stock critical spare parts 4. Train operators on quick changeover/repair 5. Consider redundant stations for bottlenecks

6.6.2 Buffer Sizing

Buffers between stations decouple operations and improve overall line availability.

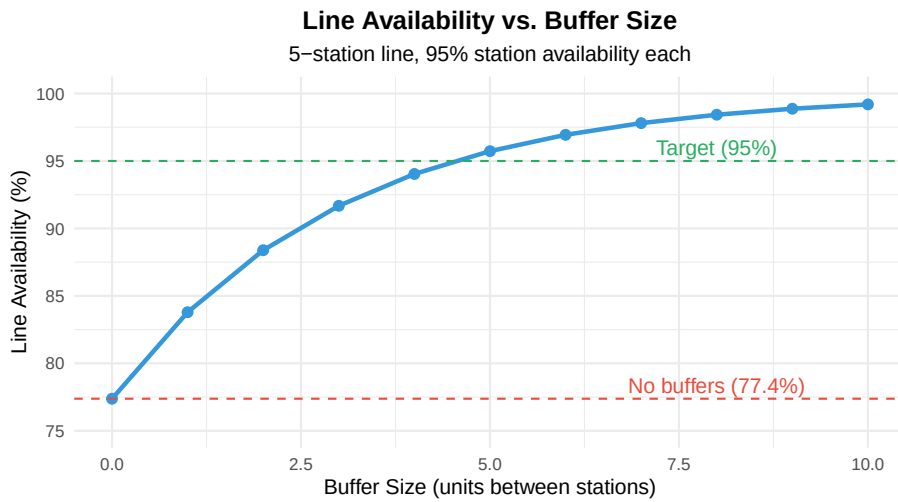


Figure 6.7: Effect of Buffers on Line Availability

6.7 Cellular Manufacturing

Cellular manufacturing applies Group Technology to organize production into cells that specialize in “families” of similar parts.

6.7.1 Part Families and Machine Cells

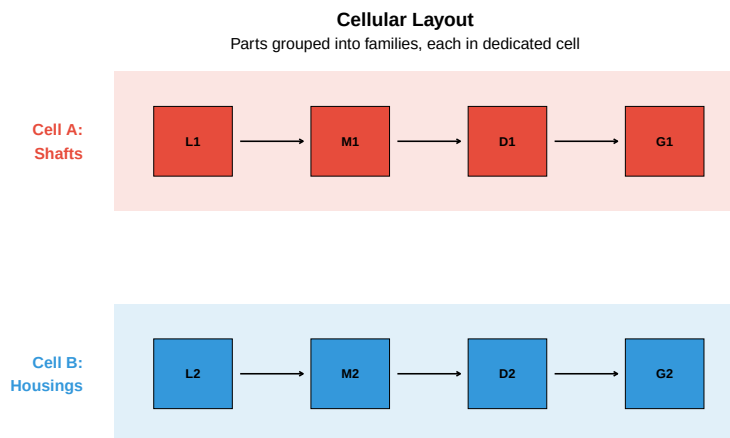


Figure 6.8: Cellular Manufacturing: From Process Layout to Cells

Table 6.4: Cellular Manufacturing Benefits vs. Traditional Process Layout

Metric	Traditional (Process)	Cellular	Typical Improvement
Setup time	High - frequent changeovers	Low - dedicated to family	50-90% reduction
Work-in-process	High - batches waiting	Low - continuous flow	50-90% reduction
Lead time	Long - complex routing	Short - U-cell flow	50-80% reduction
Quality defects	Higher - multiple handoffs	Lower - cell ownership	30-50% reduction
Material handling	High - long distances	Low - within cell	40-70% reduction
Floor space	More - large aisles	Less - compact cells	20-50% reduction
Worker skills	Specialized by machine	Cross-trained on cell	Increased flexibility

6.8 Flexible Manufacturing Systems (FMS)

An **FMS** is a highly automated cell consisting of CNC machines interconnected by automated material handling, all controlled by a central computer.

6.8.1 FMS Components

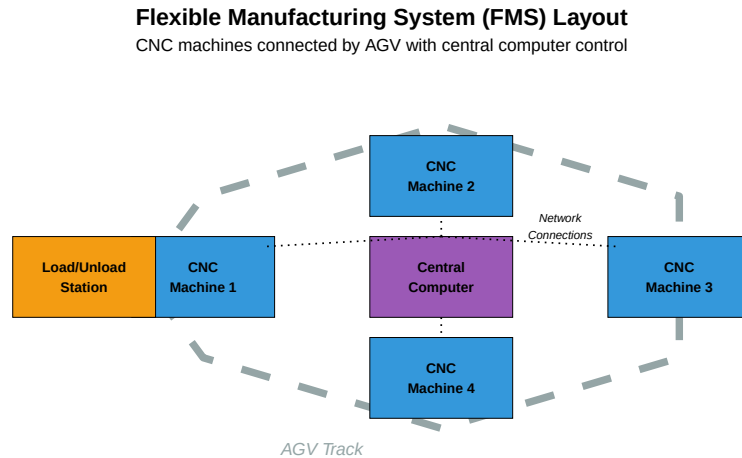


Figure 6.9: Flexible Manufacturing System Components

Table 6.5: FMS Characteristics and Applications

Characteristic	FMS Capability
----------------	----------------

Production Volume	Medium (2,000 - 100,000/year per part type)
Part Variety	Medium (4-100 different part types)
Batch Size	Small to medium (1-500 per batch)
Setup Time	Minimal (automatic pallet/fixture changes)
Typical Parts	Prismatic parts (housings, brackets, covers)
Investment	\$5M - \$50M+ depending on size
Best Suited For	Aerospace, automotive components, job shops with repeat parts

6.9 Modern Manufacturing Concepts

6.9.1 SMED (Single-Minute Exchange of Die)

SMED is a lean manufacturing technique to reduce changeover time to under 10 minutes (“single digit minutes”).

Table 6.6: SMED Methodology

Step	Description	Example
1. Separate	Distinguish internal setup (machine stopped) from external setup (machine running)	Pre-staging
2. Convert	Convert as much internal setup to external setup as possible	Use quick change clamps
3. Streamline	Reduce time for remaining internal and external activities	Eliminate unnecessary steps

Case Study: Injection Molding Changeover

Before SMED: - Changeover time: 90 minutes - Machine stopped for entire process - One operator working alone

After SMED analysis:

Activity	Before	Type	After	Change
Find tools	10 min	Internal	0	External - tools pre-staged
Remove old mold	15 min	Internal	5 min	Quick-release clamps
Get new mold	10 min	Internal	0	External - mold pre-staged at machine
Install new mold	20 min	Internal	8 min	Standardized mold heights

Activity	Before	Type	After	Change
Connect utilities	10 min	Internal	4 min	Quick-connect fittings
Adjust settings	15 min	Internal	3 min	Pre-programmed recipes
First article	10 min	Internal	5 min	Improved documentation

Results: - Total changeover: 25 minutes (72% reduction) - Additional capacity: +15 changeovers/week possible - Smaller batch sizes now economical

6.9.2 AGV and AMR Systems

Table 6.7: AGV vs. AMR Comparison

Feature	AGV (Automated Guided Vehicle)	AMR (Autonomous Mobile Robot)
Navigation	Fixed path (wires, magnets, tape)	Dynamic (SLAM, vision, sensors)
Flexibility	Low - follows predetermined routes	High - calculates optimal routes
Infrastructure	Requires installation (floor modifications)	Minimal (maps environment)
Cost	Lower vehicle cost, higher infrastructure	Higher vehicle cost, lower infrastructure
Path Changes	Requires physical modification	Software update only
Obstacle Handling	Stops and waits	Navigates around obstacles
Best Application	High-volume, repetitive routes	Variable routes, dynamic environment

6.9.3 Digital Twin

A **digital twin** is a virtual representation of a physical manufacturing system that is updated in real-time with sensor data.

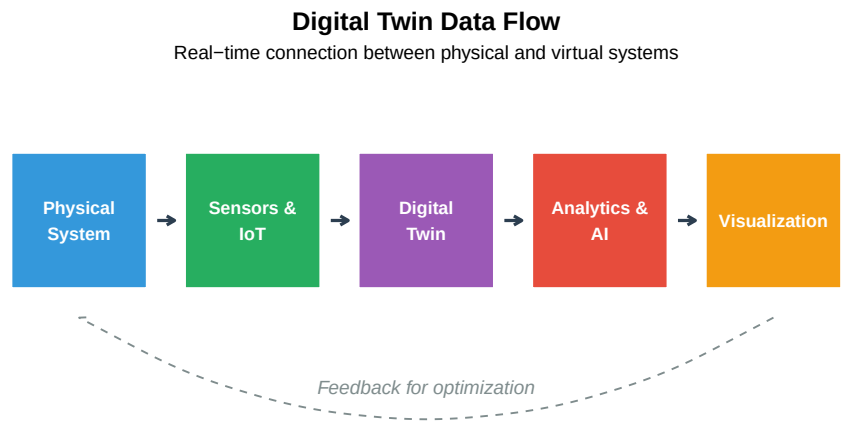


Figure 6.10: Digital Twin Concept

Digital Twin Applications: - Predictive maintenance (predict failures before they occur) - Process optimization (simulate changes before implementation) - Training (virtual commissioning and operator training) - Quality prediction (correlate parameters with outcomes)

6.10 Industry Applications

Table 6.8: Manufacturing System Applications by Industry

System Type	Automotive	Food Processing
Transfer Line	Engine block machining, cylinder head lines	Beverage bottling, canning lines
Manual Assembly	Final vehicle assembly, interior trim	Packaging, quality inspection
FMS	Transmission components, brake parts	Multi-product bakery equipment
Cellular	Subassemblies, wiring harnesses	Specialty product lines
AGV/AMR	Body shop material delivery, parts sequencing	Ingredient delivery, packaging transport

6.11 Review Questions

Question 1: Takt Time Calculation

A plant operates 8-hour shifts with 30 minutes of breaks. Daily demand is 400 units. Calculate the takt time.

Solution:

Available time = 8 hours $\times$ 60 min - 30 min = 450 minutes

Takt time = 450 min / 400 units = **1.125 minutes per unit** = 67.5 seconds

If actual cycle time is 60 seconds, the plant has $(67.5 - 60)/67.5 = 11\%$ **capacity margin**.

Question 2: Line Efficiency

A 6-station assembly line has a cycle time of 2 minutes. The task times at each station are: 1.8, 1.9, 2.0, 1.7, 1.6, and 1.5 minutes. Calculate the line efficiency.

Solution:

Total work content = 1.8 + 1.9 + 2.0 + 1.7 + 1.6 + 1.5 = 10.5 minutes

Line efficiency = $(10.5) / (6 \times 2.0) \times 100\% = 10.5/12 \times 100\% = 87.5\%$

Balance delay (idle time) = $100\% - 87.5\% = 12.5\%$

Question 3: Downtime Impact

An automated line has 8 stations, each with 3% failure probability per cycle and 8-minute average repair time. The ideal cycle time is 0.5 minutes. What is the actual production rate?

Solution:

Line failure probability = $1 - (1 - 0.03)^8 = 1 - 0.97^8 = 1 - 0.784 = 21.6\%$

Actual cycle time = $0.5 + (0.216 \times 8) = 0.5 + 1.73 = 2.23$ **minutes**

Actual production rate = $60 / 2.23 = 26.9$ **parts/hour**

(vs. ideal of 120 parts/hour - only 22% of ideal capacity!)

This shows why reliability is critical in automated systems.

Question 4: FMS Selection

A company makes 15 different part numbers with annual volumes of 500-5,000 each. Parts are machined from aluminum and require milling, drilling, and tapping. Which system is most appropriate: dedicated transfer line, FMS, or CNC job shop?

Answer: FMS is most appropriate because:

- **Volume range (500-5,000)** is ideal for FMS
- **Part variety (15 types)** requires flexibility
- **Similar operations** (milling, drilling, tapping) can share equipment
- **Material (aluminum)** machines well on CNC

Transfer line would require too much volume per part. Job shop would have high setup times and low efficiency for these volumes.

Question 5: SMED Application

A stamping press has a 45-minute die change. Internal activities are 35 minutes, external are 10 minutes but currently done with machine stopped. How much can SMED reduce this?

Answer:

Step 1 - Separate: Do the 10 minutes of external work while press runs. - New changeover: 35 minutes (30% reduction)

Step 2 - Convert: Analyze the 35 minutes: - If 15 minutes can be converted to external (pre-staging, etc.): New changeover = 20 minutes

Step 3 - Streamline: Quick-release clamps, standardization might save another 5 minutes. - Final changeover: **15 minutes (67% reduction)**

This enables smaller batch sizes and more frequent changeovers.

6.12 Summary

Table 6.9: Chapter 3 Summary

Topic	Key Points
Production Lines	Manual, continuous, synchronous, or asynchronous transfer; each has trade-offs
Time Metrics	Takt time = demand pace; Cycle time = actual pace; MCE measures value-added %
Line Balancing	Assign tasks to minimize idle time; Efficiency = work content / (stations × cycle time)
Automated Lines	Reliability is critical; Small failure probabilities cause large capacity losses
Cellular Manufacturing	Group Technology groups part families; Reduces setup, WIP, lead time 50-90%
FMS	Automated cells for medium variety/volume; Computer-controlled flexibility
Modern Concepts	SMED for quick changeover; AGV/AMR for material handling; Digital twins for opt

6.12.1 Key Takeaways

1. **Match system to product** - Volume and variety determine optimal manufacturing system
2. **Takt time sets the pace** - Everything should be designed to meet customer demand rate
3. **Balance matters** - Unbalanced lines waste capacity through idle time
4. **Reliability is critical** - Small failures compound to large losses in automated systems

5. **Flexibility has value** - FMS and cellular systems enable quick response to change
 6. **Technology evolves** - Digital twins and AMRs are transforming manufacturing
-

6.13 References

1. Groover, M.P. (2020). *Automation, Production Systems, and Computer-Integrated Manufacturing* (5th ed.). Pearson.
2. Liker, J.K. (2004). *The Toyota Way*. McGraw-Hill.
3. Shingo, S. (1985). *A Revolution in Manufacturing: The SMED System*. Productivity Press.
4. Black, J.T. (1991). *The Design of the Factory with a Future*. McGraw-Hill.

Chapter 7

Lean Manufacturing

7.1 Learning Objectives

By the end of this chapter, you will be able to:

- Define Lean Manufacturing and explain its origins
- Identify and describe the 3 M's of Lean (Muda, Mura, Muri)
- Recognize and eliminate the seven types of waste
- Apply the 5S methodology to workplace organization
- Understand Just-in-Time (JIT) principles and Kanban systems
- Calculate Takt time and apply line balancing concepts
- Implement setup reduction techniques (SMED)

“Perfection is not attainable. But if we chase perfection, we can catch excellence.” — Vince Lombardi

7.2 Introduction to Lean Manufacturing

Lean Manufacturing is the systematic elimination of waste. The term can be traced to Jim Womack, Daniel Jones, and Daniel Roos' book *The Machine that Changed the World* (1991), which examined the Toyota Production System.

What does “Lean” actually mean?

The term “Lean” refers to cutting “fat” from production activities — eliminating anything that doesn't add value to the customer. Think of it like a lean athlete: no excess weight, maximum efficiency, peak performance.

7.2.1 Definition of Lean

Table 7.1: Lean Manufacturing Targets (Womack, Jones, Roos 1990)

Metric	Lean Target
Human effort in factory	Half the hours
Defects in finished product	Half the defects
Engineering effort	One-third the hours
Factory space for same output	Half the space
In-process inventories	A tenth or less

7.3 Evolution of Manufacturing Systems

Manufacturing has evolved through three major paradigms. Understanding this evolution helps us appreciate why Lean emerged as the dominant philosophy.

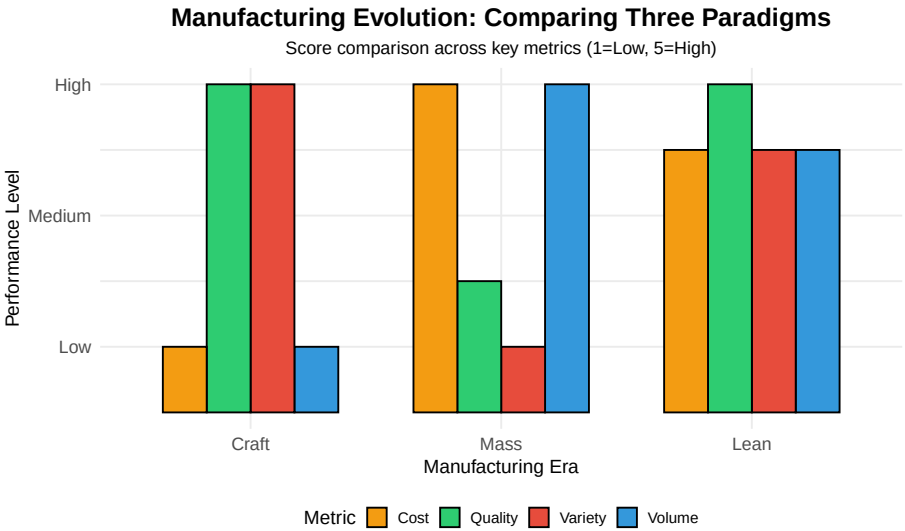


Figure 7.1: Evolution of Manufacturing Systems

7.3.1 Comparison of Manufacturing Systems

Table 7.2: Evolution of Manufacturing Systems

Characteristic	Craft Manufacturing	Mass Manufacturing	Lean Manufacturing
Time Period	Late 1800s	1920s (Ford)	1970s (Toyota)
Process	Built on blocks, workers walk around	Assembly line	Cellular manufacturing
Workforce	Craftsmen with pride	Low skilled, simplistic jobs	Highly skilled, multi-tasking
Components	Hand-crafted, hand-fitted	Interchangeable parts	Standardized parts
Quality	Excellent	Lower	Excellent
Cost	Very expensive	Affordable	Low
Volume	Few produced	Billions - identical	Highly variable
Flexibility	Very high	Very low	High

Discussion: Why did Mass Manufacturing dominate for so long?

Mass manufacturing dominated because:

1. **Economies of scale** - Higher volume = lower unit cost
2. **Predictable demand** - Post-war boom created stable markets
3. **Limited competition** - Few global competitors
4. **Consumer acceptance** - “Any color as long as it’s black” (Henry Ford)

What changed? - Global competition (especially from Japan) - Customers demanded variety and quality - Product lifecycles shortened - Technology enabled flexibility

7.4 The Toyota Production System (TPS)

The most influential lean manufacturing model is the **Toyota Production System (TPS)**, developed by Taiichi Ohno and Shigeo Shingo at Toyota.

7.4.1 Historical Context

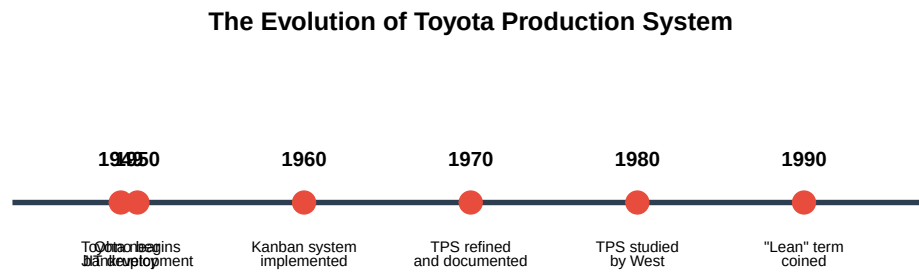


Figure 7.2: Toyota's Journey to Lean Excellence

In 1949, Toyota was on the brink of bankruptcy. Ford's car production was at least **8 times more efficient** than Toyota's.

Kiichiro Toyoda's Challenge: "To achieve the same rate of production as the United States in three years."

Taiichi Ohno accepted this challenge. Inspired by American supermarkets, he developed the Just-in-Time method.

7.4.2 Two Pillars of TPS

The House of Toyota Production System

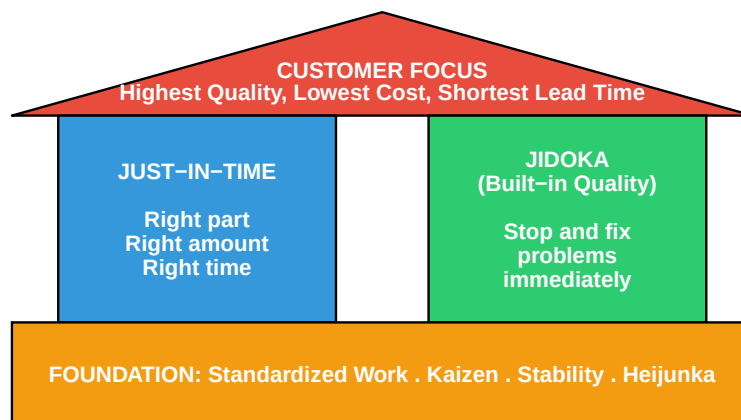


Figure 7.3: The Two Pillars of Toyota Production System

- 1. Just-in-Time (JIT):** Produce only what is needed, when it is needed, in the amount needed.
 - 2. Jidoka (Built-in Quality):** Automation with a human touch — stop production immediately when a defect is detected.
-

7.5 The 3 M's of Lean

Lean manufacturing focuses on eliminating three types of inefficiency, known as the **3 M's**:

The 3 M's of Lean



Figure 7.4: The 3 M's of Lean Manufacturing

Table 7.3: The 3 M's Explained

Japanese	English	Description
Muda (ムダ)	Waste	Activities that consume resources without adding value
Mura (ムラ)	Inconsistency/Unevenness	Variation and lack of uniformity in processes
Muri (ムリ)	Overburden/Unreasonableness	Overburdening people or equipment beyond capacity

7.6 The Seven Types of Waste (MUDA)

Shigeo Shingo identified seven categories of waste common to manufacturing. These are activities that add cost without adding value.

“Waste is everything that is not absolutely essential.” — Hiroyuki Hirano

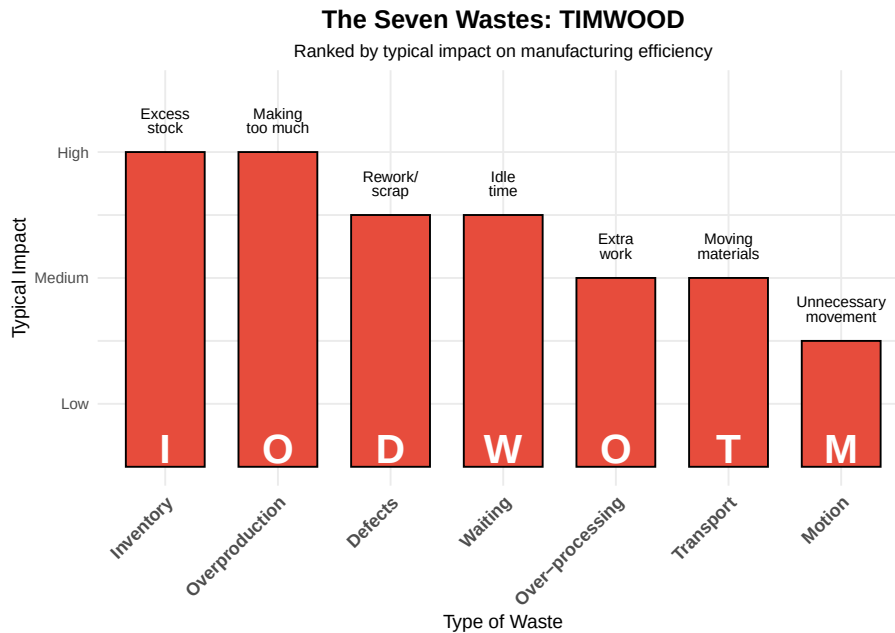


Figure 7.5: The Seven Types of Waste (TIMWOOD)

7.6.1 Detailed Breakdown of the Seven Wastes

Table 7.4: The Seven Wastes: Detailed

#	Waste Type	Description	Examples
1	**Overproduction**	Producing more than needed, earlier than needed	Building to forecast
2	**Inventory**	Excess raw materials, WIP, or finished goods	Safety stock, excess stock
3	**Waiting**	Idle time waiting for materials, information, or equipment	Machine down, waiting for materials
4	**Transportation**	Unnecessary movement of materials between processes	Long distances, multiple moves
5	**Over-processing**	Doing more work than the customer requires	Tighter tolerances, unnecessary steps
6	**Motion**	Unnecessary movement of people	Reaching, bending, carrying
7	**Defects**	Products that don't meet specifications	Scrap, rework, returns

Memory Aid: TIMWOOD

Use **TIMWOOD** to remember the seven wastes:

- **T**ransportation
- **I**nventory
- **M**otion
- **W**aiting

- **Overproduction**
- **Over-processing**
- **Defects**

Some people also use **DOWNTIME** (adding “Non-utilized talent” as an 8th waste).

Interactive Exercise: Identify the Waste

Scenario 1: A worker walks 50 feet to get a tool, uses it for 30 seconds, then walks back.

Answer: This is **Motion** waste. The solution would be to place tools at the point of use (5S).

Scenario 2: A machine produces 1000 parts, but only 800 are needed this week.

Answer: This is **Overproduction** waste. The solution would be to implement pull production based on actual demand.

Scenario 3: Parts sit in a queue for 3 days before being processed at the next station.

Answer: This is both **Inventory** and **Waiting** waste. The solution would be to balance the line and implement one-piece flow.

7.7 The 5S Methodology

5S is a workplace organization methodology that creates a clean, efficient, and safe working environment.

The 5S Methodology for Workplace Organization

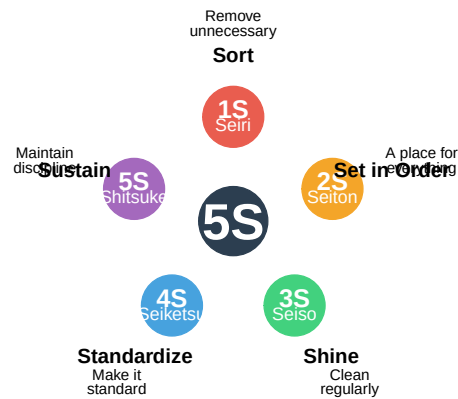


Figure 7.6: The 5S Methodology

Table 7.5: The 5S Steps Explained

Step	Japanese	English	Action	Key Question
1S	**Seiri**	Sort	Remove unnecessary items from the workplace	Do we need this?
2S	**Seiton**	Set in Order	Organize items so they are easy to find and use	Where does this belong?
3S	**Seiso**	Shine	Clean the workplace regularly	Is everything clean?
4S	**Seiketsu**	Standardize	Create standards for maintaining 1S-3S	How do we keep it this way?
5S	**Shitsuke**	Sustain	Build discipline to maintain standards	Are we following our standards?

7.7.1 Visual Factory

“The ability to understand the status of a production area in **5 minutes or less** by simple observation without use of computers or speaking to anyone.”

A well-implemented 5S creates a **visual factory** where problems are immediately apparent.

7.8 Just-in-Time (JIT) Production

7.8.1 The JIT Philosophy

“Deliver the right material, in the exact quantity, with perfect quality, in the right place, just before it is needed.” — Ohno and Shingo

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

Just-in-Time: Delivering Value to the Customer



Figure 7.7: Just-in-Time: The Right Everything

7.8.2 Push vs. Pull Systems

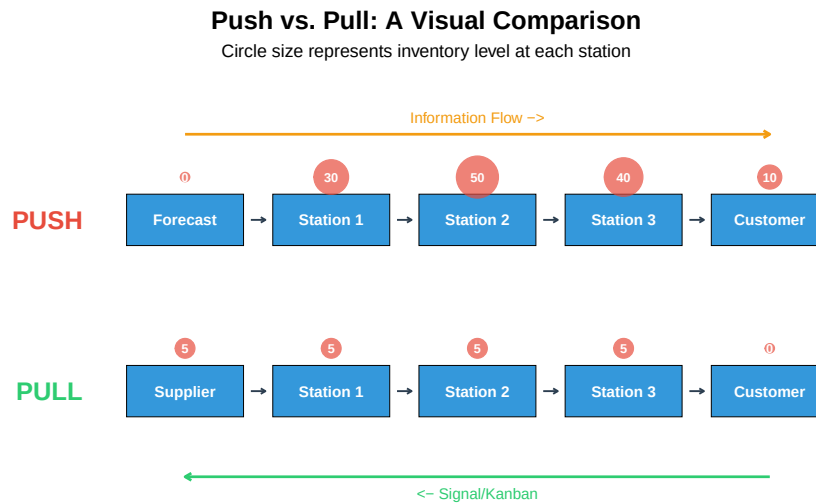


Figure 7.8: Push vs. Pull Production Systems

Table 7.6: Push vs. Pull System Comparison

Aspect	Push System	Pull System
Production trigger	Forecast/Schedule	Actual demand/Kanban signal
Information flow	Forward (upstream to downstream)	Backward (downstream to upstream)
Inventory levels	High (buffers at each station)	Low (minimal buffers)
Flexibility	Low	High
Problem visibility	Problems hidden by inventory	Problems immediately visible
Customer responsiveness	Slow	Fast

7.8.3 Kanban System

Kanban () is Japanese for “signboard” or “billboard.” It’s a scheduling system for lean and JIT production.

Sample Production Kanban Card

PRODUCTION KANBAN

Part Number:	3278784		
Part Name:	Securing Bracket		
From Process:	CNC Turning (CNC-		
To Process:	CNC Machining (CNC-M7)		
Container Qty:	12	Container Type:	A1

Figure 7.9: Example Kanban Card

7.9 Key Time Metrics

Understanding time metrics is essential for lean manufacturing analysis and improvement.

7.9.1 Definitions

Understanding Time Metrics in Manufacturing

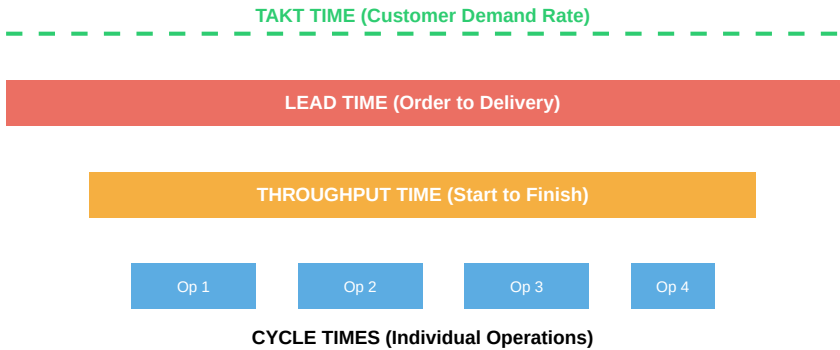


Figure 7.10: Relationship Between Time Metrics

7.9.2 Formulas and Calculations

Takt Time — The pace of production to meet customer demand:

$$\text{Takt Time} = \frac{\text{Available Production Time}}{\text{Customer Demand}}$$

Cycle Time — Time to complete one unit at a workstation:

$$\text{Cycle Time} = \frac{\text{Net Operating Time}}{\text{Units Produced}}$$

Throughput Time — Total time from start to finish:

$$\text{Throughput Time} = \text{Processing} + \text{Inspection} + \text{Queue} + \text{Move Time}$$

7.9.3 Interactive Example: Calculating Takt Time

Live cell — try different shift length, breaks, or customer demand.

```
#| label: takt-calculation
# Given data
available_time_per_day <- 8 * 60 # 8 hours = 480 minutes
breaks_and_meetings <- 60      # 60 minutes for breaks/meetings
customer_demand <- 240         # 240 units per day

# Calculate net available time
net_available_time <- available_time_per_day - breaks_and_meetings
cat("Net Available Time:", net_available_time, "minutes/day\n")

# Calculate Takt Time
takt_time <- net_available_time / customer_demand
cat("Takt Time:", takt_time, "minutes per unit\n")
cat("Takt Time:", takt_time * 60, "seconds per unit\n")

# Interpretation
cat("\n--- Interpretation ---\n")
cat("To meet customer demand, we must produce 1 unit every",
    takt_time, "minutes (", takt_time * 60, "seconds).\n")
```

Practice Problem: Calculate Your Takt Time

Given: - Shift length: 10 hours - Two 15-minute breaks and one 30-minute lunch - Customer requires 300 units per shift

Calculate: 1. Net available production time 2. Takt time in minutes 3. Takt time in seconds

Solution:

Net time = $(10 \times 60) - (15 + 15 + 30) = 600 - 60 = 540$ minutes

Takt time = $540 / 300 = 1.8$ minutes = 108 seconds per unit

7.10 Setup Reduction (SMED)

SMED (Single Minute Exchange of Dies) is a methodology developed by Shigeo Shingo to dramatically reduce changeover time.

7.10.1 The SMED Concept

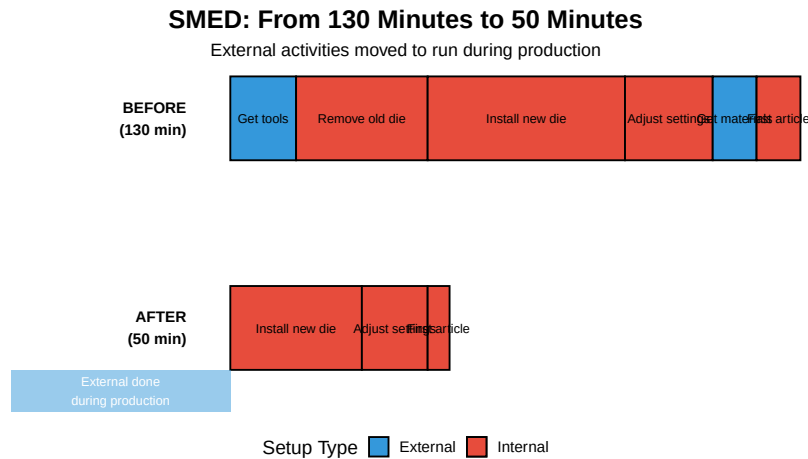


Figure 7.11: SMED: Internal vs External Setup

7.10.2 SMED Methodology

Table 7.7: The Four Steps of

Step	Action	Description
1	**Document current process**	Video the entire changeover, identify all steps
2	**Separate internal and external**	Internal = machine must be stopped; External = c
3	**Convert internal to external**	Find ways to perform 'internal' tasks while machin
4	**Streamline all operations**	Reduce time for remaining internal tasks through

Case Study: Press Setup Reduction

Problem: Presses were idle more than 50% of the time due to setup. - Average production run: 1-2 days - Setup time: 1-2 days

Root Cause Analysis: - Using standard bolt fasteners with open-end wrenches - No pre-staging of tools or dies - Sequential operations that could be parallel

Solutions Implemented:

Change	Time Saved
Socket sets + power tools	10%
Quick-release clamps	50%
Pre-staging materials	20%
Parallel operations	20%

Result: Setup time reduced from 1-2 days to 4-6 hours, with a target of under 1 hour.

7.11 Kaizen: Continuous Improvement

Kaizen () means “change for the better” — the philosophy of continuous, incremental improvement.

7.11.1 Kaizen Principles

Plan–Do–Check–Act: The Engine of Kaizen



Figure 7.12: The Kaizen Continuous Improvement Cycle

7.11.2 Key Kaizen Concepts

- **Everyone participates** — from CEO to frontline workers
 - **Small, incremental changes** — not major overhauls
 - **Focus on process** — blame the process, not the person
 - **Data-driven decisions** — measure before and after
 - **Standardize improvements** — lock in gains
-

7.12 Value Stream Mapping (VSM)

Value Stream Mapping is a lean tool for analyzing the flow of materials and information required to bring a product to the customer.

7.12.1 VSM Symbols

Table 7.8: Key Metrics in Value Stream Maps

Metric	Description	Typical Location
Process Time (PT)	Time to actually process one unit	In process box
Lead Time (LT)	Total time from raw material to finished good	Timeline at bottom
Changeover Time (C/O)	Time to switch between products	In process box
Uptime	Percentage of time equipment is available	In process box
Takt Time	Required pace to meet demand	Noted on map
WIP Inventory	Units waiting between processes	Triangle symbol between processes

7.13 Summary

Table 7.9: Lean Manufacturing: Key Concepts Summary

Concept	Definition
Lean Manufacturing	Systematic elimination of waste
3 M's	Muda (waste), Mura (inconsistency), Muri (overburden)
7 Wastes	TIMWOOD: Transport, Inventory, Motion, Waiting, Overproduction, Over-processing
5S	Sort, Set in Order, Shine, Standardize, Sustain
JIT	Right part, right quantity, right quality, right place, right time
Kanban	Visual signal system for pull production
SMED	Single Minute Exchange of Dies - rapid changeover
Kaizen	Continuous incremental improvement

7.14 Review Questions

Question 1: What are the seven types of waste, and which is often considered the most impactful?

The seven types of waste (TIMWOOD) are: 1. **T**ransportation 2. **I**nventory 3. **M**otion 4. **W**aiting 5. **O**verproduction 6. **O**ver-processing 7. **D**efects

Inventory is often considered the most impactful because it hides other problems, ties up capital, and can become obsolete.

Question 2: Explain the difference between Push and Pull production systems.

Push System: - Production based on forecasts - Materials pushed through the system - Results in high inventory - Problems hidden by buffer stock

Pull System: - Production triggered by actual demand - Materials pulled by downstream operations - Minimal inventory - Problems immediately visible

Question 3: Calculate the Takt time for a facility with 450 minutes of available time and demand of 150 units.

$$\text{Takt Time} = \frac{450 \text{ minutes}}{150 \text{ units}} = 3 \text{ minutes per unit}$$

This means we need to produce one unit every 3 minutes (180 seconds) to meet customer demand.

Question 4: What is the difference between internal and external setup in SMED?

Internal Setup: Activities that can ONLY be performed when the machine is stopped (e.g., physically removing the old die).

External Setup: Activities that CAN be performed while the machine is still running (e.g., getting tools, pre-staging the next die, preparing materials).

The key to SMED is converting as many internal activities to external as possible.

7.15 References

- Womack, J., Jones, D., & Roos, D. (1990). *The Machine that Changed the World*.
- Ohno, T. (1988). *Toyota Production System: Beyond Large-Scale Production*.
- Shingo, S. (1985). *A Revolution in Manufacturing: The SMED System*.
- Liker, J. (2004). *The Toyota Way*.
- Wang, H.P., Chang, T.C., & Wysk, R.A. (2007). *Computer Aided Manufacturing*, Chapter 18.

Chapter 8

Ergonomics in Automated Manufacturing

8.1 Learning Objectives

By the end of this chapter, you will be able to:

- Define ergonomics and explain its importance in manufacturing
- Identify the seven major ergonomic risk factors
- Recognize common musculoskeletal disorders (MSDs) and their causes
- Apply ergonomic design principles to workstation design
- Conduct a REBA (Rapid Entire Body Assessment) analysis
- Understand the role of AI and automation in ergonomic assessment
- Design workstations that optimize human performance and safety

“Ergonomics is the science of fitting the job to the worker, not forcing the worker to fit the job.” — International Ergonomics Association

8.2 What is Ergonomics?

Ergonomics (from Greek *ergon* = work, *nomos* = laws) is the scientific discipline focused on understanding the interactions between humans and other elements of a system, with the goal of optimizing human well-being and overall system performance.

The Three Domains of Ergonomics
Ergonomics considers the whole human experience at work

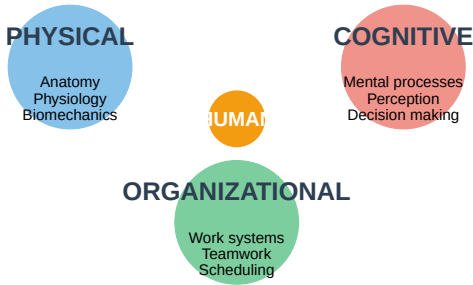


Figure 8.1: The Three Domains of Ergonomics

Table 8.1: The Three

Domain	Focus	Key Topics
Physical Ergonomics	Human body’s response to physical stress	Posture, material han
Cognitive Ergonomics	Mental processes and human-system interaction	Mental workload, dec
Organizational Ergonomics	Optimization of sociotechnical systems	Communication, team

8.2.1 Why Ergonomics Matters in Manufacturing

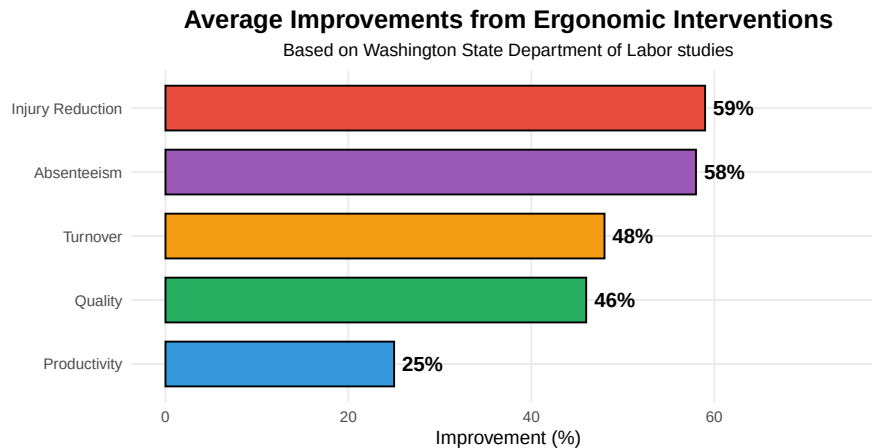


Figure 8.2: The Business Case for Ergonomics

The Cost of Ignoring Ergonomics

Direct Costs: - Workers' compensation claims - Medical expenses - Legal fees
- Increased insurance premiums

Indirect Costs (often 4-10x direct costs): - Lost productivity - Training replacement workers - Reduced quality - Overtime to cover absent workers - Administrative time - Low morale

Statistics: - MSDs account for **33%** of all workplace injuries - Average MSD claim costs **\$15,000-\$20,000** - Lost workdays due to MSDs average **12 days** per case - Carpal tunnel surgery costs approximately **\$30,000** per case (including lost time)

8.3 The Seven Ergonomic Risk Factors

The **Initial Ergonomic Risk Assessment (INERA)** identifies seven primary risk factors that contribute to musculoskeletal disorders. Understanding these factors is the first step in prevention.

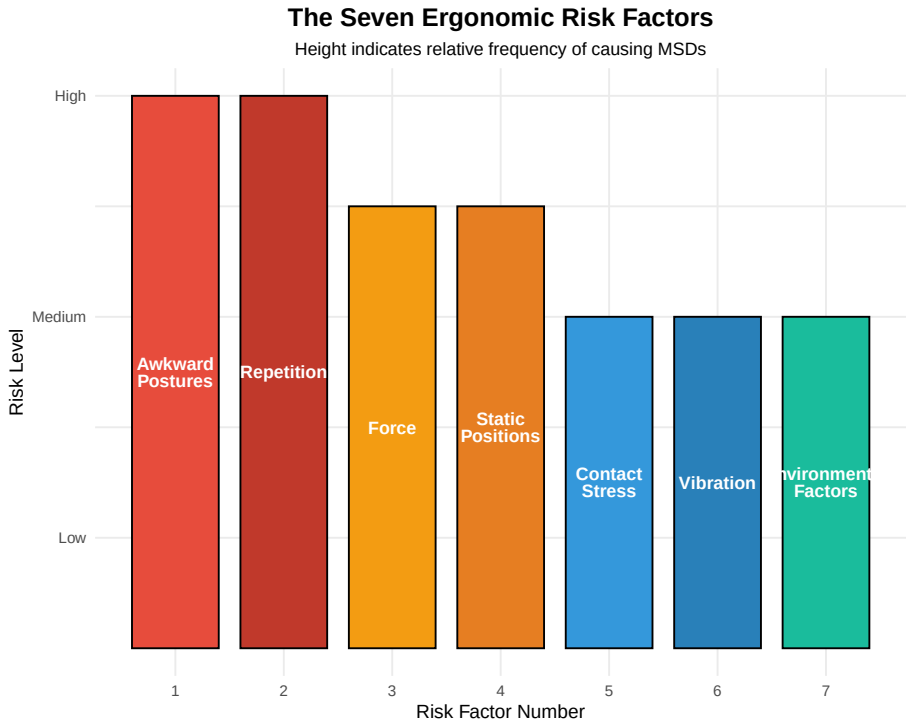


Figure 8.3: The Seven Ergonomic Risk Factors

Table 8.2: The Seven Ergonomic Risk Factors

#	Risk Factor	Definition
1	**Awkward Postures**	Positions that deviate from neutral body alignment
2	**Repetition**	Performing the same motion repeatedly
3	**Force**	Amount of physical effort required to do a task
4	**Static Positions**	Maintaining the same position for extended periods
5	**Contact Stress**	Pressure from hard surfaces or edges on body tissues
6	**Vibration**	Oscillating movements transferred to the body
7	**Environmental Factors**	Temperature, lighting, noise affecting work performance

8.3.1 Risk Factor 1: Awkward Postures

Neutral posture is the position where joints are naturally aligned, minimizing stress on muscles, tendons, and skeletal system. Any deviation from neutral increases injury risk.

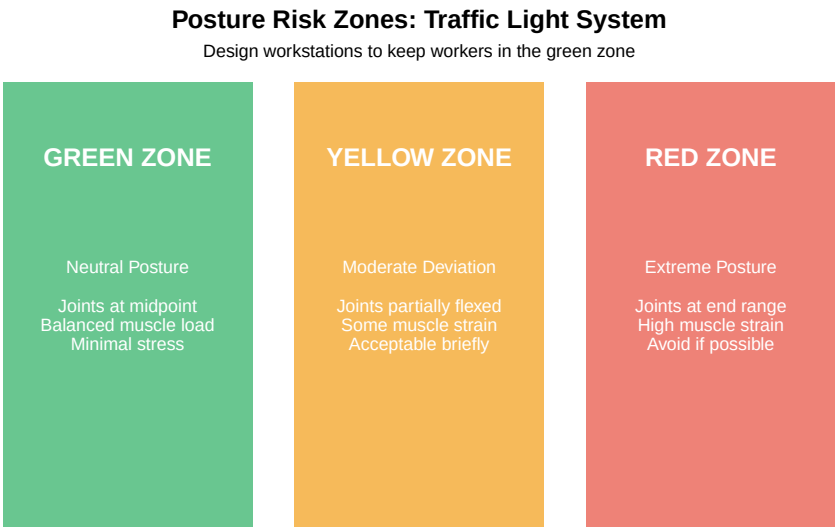


Figure 8.4: Posture Risk Zones

Table 8.3: Neutral vs. Awkward Postures by Body Part

Body Part	Neutral Position	Awkward Position	Risk Level
Head/Neck	Balanced on spine, looking straight ahead	Bent forward >20°, tilted, twisted	N
Shoulders	Relaxed, arms at sides	Raised, reaching overhead, behind body	SI
Elbows	Close to body, bent 90-120°	Fully extended or tightly bent	T
Wrists	Straight, in line with forearm	Bent up/down, twisted side-to-side	C
Back	Natural S-curve maintained	Bent forward, twisted, arched	L
Hips/Knees	Thighs parallel to floor, feet flat	Squatting, kneeling, twisted	K

8.3.2 Risk Factor 2: Repetition

Repetitive motion is one of the most significant risk factors in manufacturing. The risk increases with:

- **Frequency:** How often the motion occurs
- **Duration:** How long the task is performed
- **Recovery time:** Rest between repetitions

Table 8.4: Repetition Risk Thresholds

Cycle Time	Repetitions/Hour	Risk Level	Action Required
> 30 seconds	< 120	Low	Monitor

15-30 seconds	120-240	Moderate	Investigate and consider changes
< 15 seconds	> 240	High	Immediate intervention needed

8.3.3 Risk Factor 3: Force

The amount of **physical effort** required significantly impacts injury risk. Force requirements depend on:

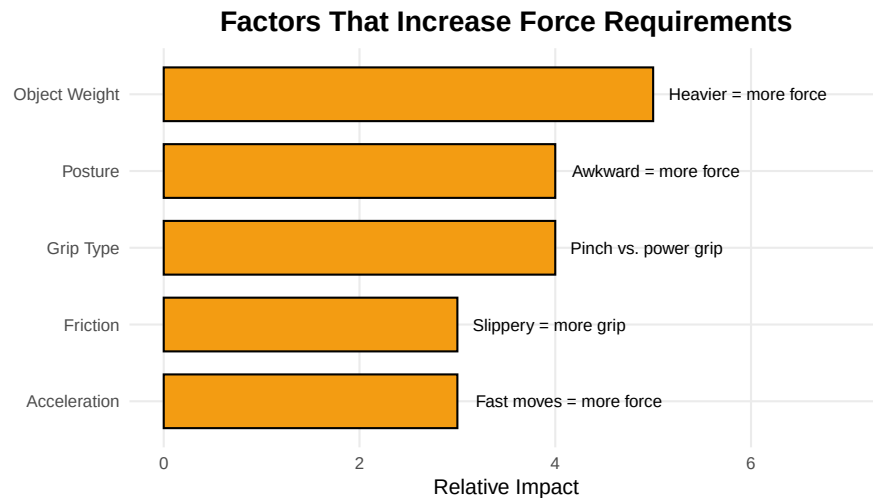


Figure 8.5: Factors Affecting Force Requirements

Interactive Exercise: Identify the Risk Factors

Scenario: An assembly line worker performs the following task 400 times per shift:

1. Reaches overhead to grab a part from a bin
2. Holds the part while inserting 4 screws with a power screwdriver
3. Places the completed assembly on a conveyor
4. The workstation has poor lighting and the floor is concrete

Identify all risk factors present:

Answer: 1. **Awkward Posture** - Reaching overhead 2. **Repetition** - 400 cycles per shift 3. **Force** - Gripping parts, controlling screwdriver 4. **Static Position** - Holding part during assembly 5. **Vibration** - Power screwdriver 6. **Contact Stress** - Standing on concrete (feet) 7. **Environmental** - Poor lighting

This task has ALL SEVEN risk factors present!

8.4 Musculoskeletal Disorders (MSDs)

Musculoskeletal Disorders (MSDs) are injuries and disorders affecting muscles, nerves, tendons, ligaments, joints, cartilage, and spinal discs. They are the most common and costly occupational health problem.

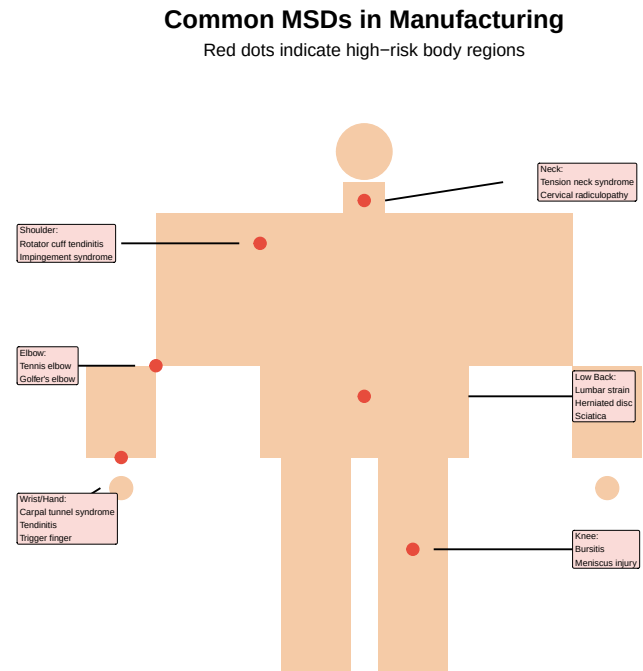


Figure 8.6: Common MSDs by Body Region

Table 8.5: Common Musculoskeletal Disorders in Manufacturing

Disorder	Description	Causes
Carpal Tunnel Syndrome	Compression of median nerve in the wrist	Repetitive wrist motions, forceful exertions
Tendinitis	Inflammation of tendons from overuse	Repetitive motions, awkward postures
Low Back Pain	Strain or injury to muscles/discs of lower back	Heavy lifting, bending, twisting
Rotator Cuff Injury	Damage to shoulder tendons	Overhead work, repetitive motions
Epicondylitis	Inflammation of elbow tendons (tennis/golfer's elbow)	Repetitive arm/wrist motions
Trigger Finger	Finger gets stuck in bent position	Repetitive gripping, hand tools

8.4.1 The MSD Development Cycle

MSDs typically develop gradually through a predictable cycle:

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

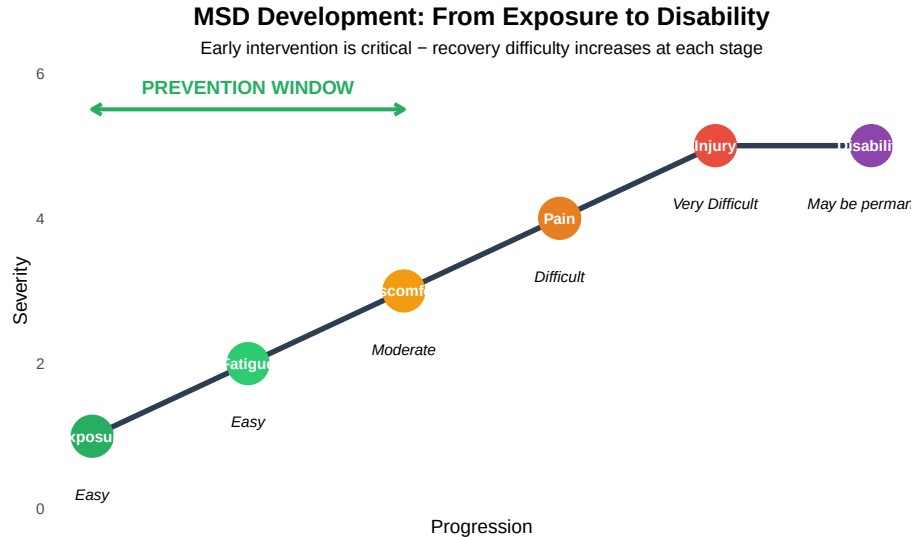


Figure 8.7: The MSD Development Cycle

Warning Signs: When to Take Action

Early Warning Signs (Act Immediately): - Persistent fatigue in specific body parts - Discomfort during or after work - Tightness or stiffness - Minor aching or soreness

Serious Warning Signs (Seek Medical Attention): - Persistent pain that doesn't go away with rest - Numbness or tingling - Loss of strength or grip - Swelling or inflammation - Reduced range of motion - Pain that wakes you at night

Key Message: Report symptoms early! The earlier the intervention, the better the outcome.

8.5 Preventing MSDs: Best Practices

Prevention is far more effective and less costly than treatment. Here are evidence-based strategies for MSD prevention:

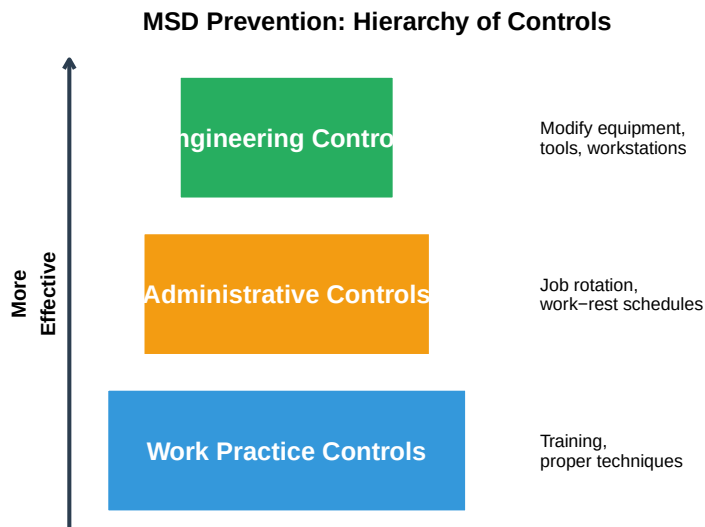


Figure 8.8: MSD Prevention Hierarchy

Table 8.6: MSD Prevention Strategies

Strategy	Engineering Solutions	Worker Actions
Maintain Neutral Posture	Adjustable workstations, proper tool selection	Awareness of body position, s
Reduce Repetition	Automation, job rotation between stations	Vary tasks when possible
Minimize Force	Mechanical assists, better handles, lighter parts	Use tools properly, ask for he
Avoid Static Positions	Sit-stand workstations, adjustable fixtures	Shift weight, change position
Eliminate Contact Stress	Padding, rounded edges, anti-fatigue mats	Use padding, wear appropriat
Control Vibration	Vibration-dampening tools, isolation mounts	Limit exposure time, grip too
Optimize Environment	Proper lighting, climate control, noise reduction	Report issues, use available c
Allow Recovery	Rest break areas, micro-break reminders	Take breaks, report fatigue

8.5.1 Proper Lifting Technique

Improper lifting is a leading cause of back injuries. The NIOSH lifting equation provides guidelines, but proper technique is fundamental.

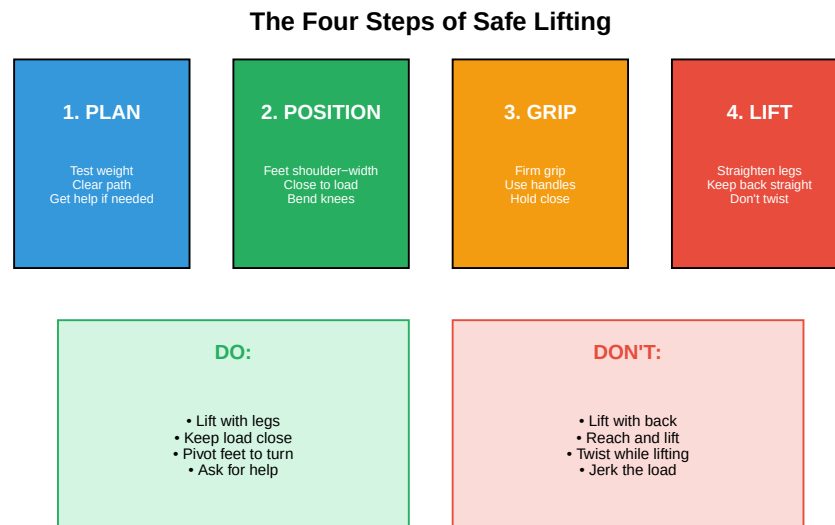


Figure 8.9: Proper Lifting Technique

8.6 Ergonomic Design Principles

Effective ergonomic design follows established principles that account for human capabilities and limitations.

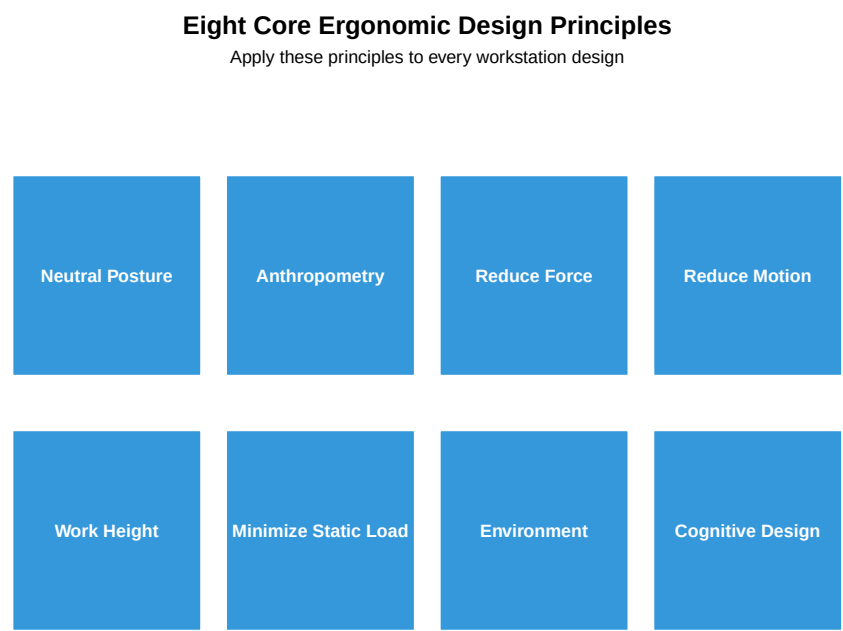


Figure 8.10: Ergonomic Design Principles

Table 8.7: Ergonomic Design Principles in Detail

Principle	Description	Implementation
1. Neutral Posture	Design to promote natural, balanced body positions	Adjustable workstation
2. Accommodate Anthropometry	Accommodate different body sizes (5th-95th percentile)	Adjustable equipment
3. Reduce Excessive Force	Minimize force requirements for all tasks	Mechanical assists, bearings
4. Reduce Excessive Motion	Reduce extreme motions, frequencies, and durations	Bring work closer, eliminate unnecessary motions
5. Optimize Work Height	Match work surface height to task requirements	Precision work higher, general work lower
6. Minimize Static Load	Avoid holding same position for extended periods	Support arms, provide rest breaks
7. Environmental Conditions	Optimize lighting, temperature, noise, air quality	Task lighting, climate control
8. Cognitive Support	Design interfaces for easy understanding and use	Clear displays, logical layouts

8.6.1 Anthropometry: Designing for Human Dimensions

Anthropometry is the scientific study of human body measurements. Workstations must accommodate the range of workers who will use them.

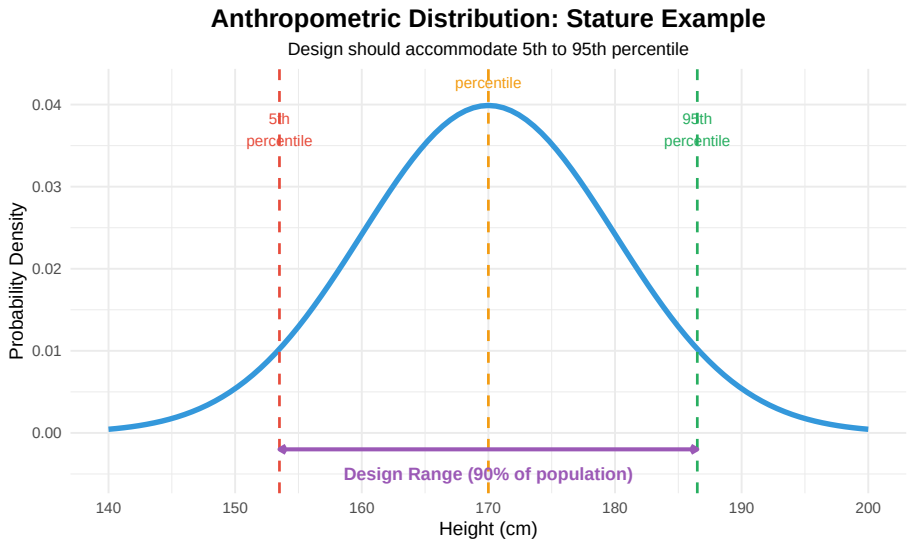


Figure 8.11: Anthropometric Design Considerations

Table 8.8: Key Anthropometric Measurements for Worksta

Measurement	5th % Female (cm)	50th % Combined (cm)	95th
Stature (Standing Height)	150		170
Eye Height (Standing)	138		158
Shoulder Height	121		140
Elbow Height	93		104
Knuckle Height	64		74
Sitting Height	79		87
Seated Eye Height	68		76
Seated Elbow Height	18		24
Thigh Clearance	12		15
Forward Reach	64		73

8.6.2 Work Surface Height Guidelines

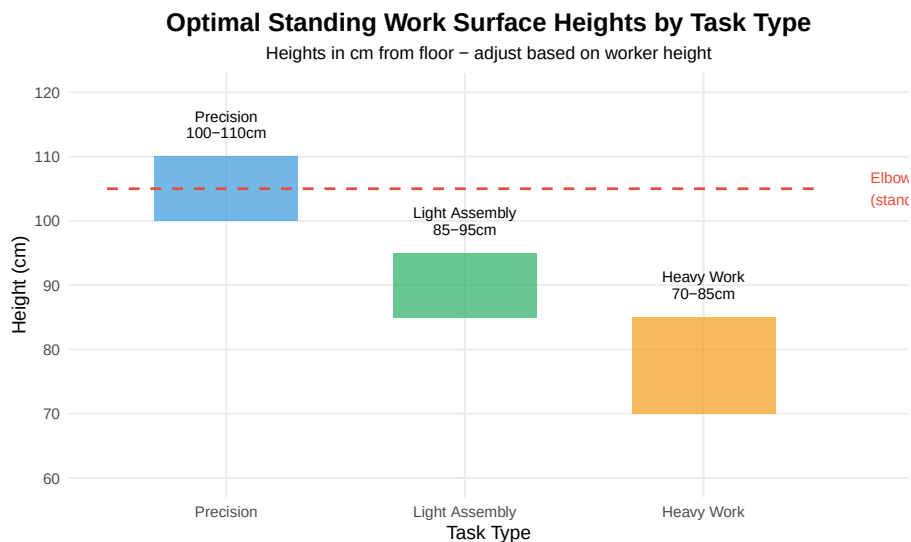


Figure 8.12: Recommended Work Surface Heights

Discussion: Why Different Heights for Different Tasks?

Precision Work (100-110 cm): - Requires close visual inspection - Hands need to be at or above eye level - Examples: Electronics assembly, inspection, watchmaking

Light Assembly (85-95 cm): - General manipulation tasks - Hands at about elbow height - Examples: Packaging, general assembly, sorting

Heavy Work (70-85 cm): - Requires pushing, pressing, or lifting - Lower height allows use of body weight - Examples: Packing boxes, pressing operations, heavy assembly

Key Principle: The work surface height should allow the worker to maintain **neutral posture** while performing the specific task.

8.7 Ergonomic Assessment Tools

Several standardized tools help assess ergonomic risk and guide interventions. Understanding these tools is essential for the process engineer.

Table 8.9: Common Ergonomic Assessment Tools

Tool	Full Name	Focus Area	Best Used For
RULA	Rapid Upper Limb Assessment	Upper body postures	Sedentary/computer work
REBA	Rapid Entire Body Assessment	Whole body postures	Manufacturing tasks
NIOSH Lifting Equation	NIOSH Lifting Equation	Manual lifting tasks	Lifting, lowering, pushing, pulling
OCRA	Occupational Repetitive Action	Repetitive upper limb tasks	Assembly line work
EAWS	Ergonomic Assessment Worksheet	Comprehensive workstation	Complete work environment
Snook Tables	Snook Psychophysical Tables	Manual material handling	Push, pull, lift, carry

8.7.1 REBA: Rapid Entire Body Assessment

REBA is one of the most widely used ergonomic assessment tools in manufacturing. It evaluates the whole body posture and provides a risk score.

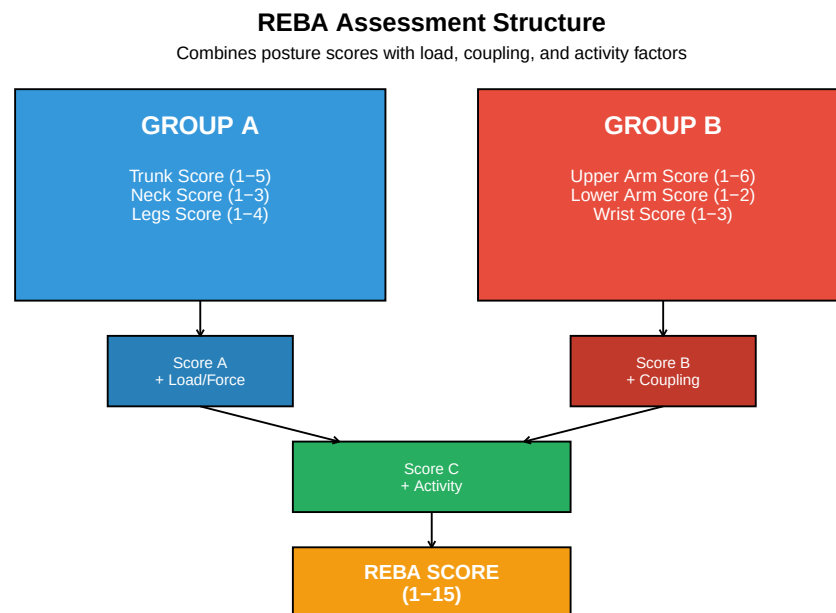


Figure 8.13: REBA Assessment Framework

8.7.2 REBA Scoring Tables

Trunk Posture Scoring:

Table 8.10: REBA Trunk Posture Scores

Position	Score	Adjustment
Upright (0°)	1	—
0-20° flexion/extension	2	+1 if twisted
20-60° flexion, >20° extension	3	+1 if twisted
>60° flexion	4	+1 if twisted or side-bending

Neck Posture Scoring:

Table 8.11: REBA Neck Posture Scores

Position	Score	Adjustment
0-20° flexion	1	+1 if twisted or side-bending
>20° flexion or extension	2	+1 if twisted or side-bending

REBA Action Levels:

Table 8.12: REBA Action Levels

REBA Score	Risk Level	Action
1	Negligible	None necessary
2-3	Low	May be necessary
4-7	Medium	Necessary
8-10	High	Necessary soon
11-15	Very High	Necessary NOW

8.7.3 REBA Employee Assessment Worksheet

The REBA Employee Assessment Worksheet provides a systematic approach to evaluating whole-body posture risk. This worksheet is based on the original work by Hignett and McAtamney (2000) and is widely used in manufacturing ergonomic assessments.

8.7.3.1 Group A: Neck, Trunk, and Leg Analysis**Step 1: Neck Posture Score**

Table 8.13: REBA Neck Posture Scoring

Position	Base Score	Adjustment	Max Score
0-20° flexion	1	+1 if twisted or side-bending	2

>20° flexion	2	+1 if twisted or side-bending	3
In extension	2	+1 if twisted or side-bending	3

Step 2: Trunk Posture Score

Table 8.14: REBA Trunk Posture Scoring

Position	Base Score	Adjustment	Max Score
Upright (0°)	1	—	1
0-20° flexion or extension	2	+1 if twisted or side-bending	3
20-60° flexion or >20° extension	3	+1 if twisted or side-bending	4
>60° flexion	4	+1 if twisted or side-bending	5

Step 3: Legs Posture Score

Table 8.15: REBA Legs Posture Scoring

Position	Base Score	Knee Adjustment
Bilateral weight bearing, walking, or sitting	1	+1 if knees 30-60° flexi
Unilateral weight bearing, featherweight, or unstable posture	2	+1 if knees 30-60° flexi

Step 4: Look Up Table A Score

Table 8.16: REBA Table A: Posture Score A (Trunk × Neck × Legs)

Trunk Score	Neck Score	Legs = 1	Legs = 2	Legs = 3	Legs = 4
1	1	1	1	3	4
1	2	2	2	3	4
1	3	2	3	4	5
2	1	2	3	4	5
2	2	3	4	5	6
2	3	4	5	6	7
3	1	3	4	5	6
3	2	4	5	6	7
3	3	5	6	7	8
4	1	4	5	6	7
4	2	5	6	7	8
4	3	6	7	8	9
5	1	6	7	8	9
5	2	7	8	9	9

5	3	8	9	9	9
---	---	---	---	---	---

Step 5: Add Load/Force Score

Table 8.17: REBA Load/Force Scoring

Load/Force	Score	Shock Adjustment
< 5 kg (< 11 lbs)	0	—
5-10 kg (11-22 lbs)	1	+1 if shock or rapid build-up of force
> 10 kg (> 22 lbs)	2	+1 if shock or rapid build-up of force

$$\text{Score A} = \text{Table A Score} + \text{Load/Force Score}$$

8.7.3.2 Group B: Arm and Wrist Analysis**Step 6: Upper Arm Posture Score**

Table 8.18: REBA Upper Arm Posture Scoring

Position	Base Score	Adjustments
20° extension to 20° flexion	1	+1 if shoulder raised; +1 if arm abducted; -1 if leaning/suppo
>20° extension	2	+1 if shoulder raised; +1 if arm abducted; -1 if leaning/suppo
20-45° flexion	2	+1 if shoulder raised; +1 if arm abducted; -1 if leaning/suppo
45-90° flexion	3	+1 if shoulder raised; +1 if arm abducted; -1 if leaning/suppo
>90° flexion	4	+1 if shoulder raised; +1 if arm abducted; -1 if leaning/suppo

Step 7: Lower Arm Posture Score

Table 8.19: REBA Lower Arm Posture Scoring

Position	Score
60-100° flexion (optimal range)	1
< 60° or > 100° flexion	2

Step 8: Wrist Posture Score

Table 8.20: REBA Wrist Posture Scoring

Position	Base Score	Adjustment	Max Score
----------	------------	------------	-----------

0-15° flexion or extension (neutral)	1	+1 if wrist is twisted or deviated	2
>15° flexion or extension	2	+1 if wrist is twisted or deviated	3

Step 9: Look Up Table B ScoreTable 8.21: REBA Table B: Posture Score B (Upper Arm $\times$ Lower Arm $\times$ Wrist)

Upper Arm Score	Lower Arm Score	Wrist = 1	Wrist = 2	Wrist = 3
1	1	1	1	2
1	2	2	2	3
2	1	1	2	3
2	2	2	3	4
3	1	3	4	5
3	2	4	5	5
4	1	4	5	6
4	2	5	6	7
5	1	6	7	8
5	2	7	8	8
6	1	7	8	9
6	2	8	9	9

Step 10: Add Coupling Score

Table 8.22: REBA Coupling Score

Coupling Quality	Description
Good	Well-fitting handle and mid-range power grip
Fair	Hand hold acceptable but not ideal; coupling acceptable via other body
Poor	Hand hold not acceptable although possible; no handles
Unacceptable	Awkward, unsafe grip; no handles; coupling unacceptable using other bo

$$\text{Score B} = \text{Table B Score} + \text{Coupling Score}$$

8.7.3.3 Calculating the Final REBA Score**Step 11: Look Up Table C Score**

Table 8.23: REBA Table C: Combined Score (Score A $\times$ Score B)

Score A	B=1	B=2	B=3	B=4	B=5	B=6	B=7	B=8	B=9	B=10	B=11	B=12
1	1	1	1	2	3	3	4	5	6	7	7	7
2	1	2	2	3	4	4	5	6	6	7	7	8
3	1	2	3	3	4	5	6	7	7	8	8	8
4	2	3	3	4	5	6	7	8	8	9	9	9
5	3	4	4	5	6	7	8	8	9	9	9	10
6	3	4	5	6	7	8	8	9	9	10	10	10
7	4	5	6	7	8	8	9	9	10	10	11	11
8	5	6	7	8	8	9	9	10	10	11	11	11
9	6	6	7	8	9	9	10	10	10	11	11	12
10	7	7	8	9	9	10	10	11	11	11	12	12
11	7	7	8	9	9	10	11	11	11	12	12	12
12	7	8	8	9	9	10	11	11	12	12	12	12

Step 12: Add Activity Score

Table 8.24: REBA Activity Score (Add +1 for each condition present)

Activity Condition	Score
One or more body parts are static (held > 1 minute)	+1
Repeated small range actions (> 4 $\times$ per minute, excluding walking)	+1
Action causes rapid large range changes in postures or unstable base	+1

REBA Score = Table C Score + Activity Score

8.7.3.4 REBA Score Interpretation

Table 8.25: REBA Action Level Guidelines

REBA Score	Risk Level	Action Required	Priority
1	Negligible	None necessary	Continue monitoring
2-3	Low	May be necessary	Investigate further when convenient
4-7	Medium	Necessary	Implement changes to reduce risk
8-10	High	Necessary soon	Investigate and implement changes soon
11-15	Very High	Necessary NOW	Stop task and make immediate changes

8.7.4 REBA Example Calculation: Step-by-Step Walk-through

Let's work through a complete REBA assessment for a manufacturing worker performing a packaging task.

Scenario: A worker stands at a packaging station, bending forward to pick up items from a low bin and placing them into boxes on a conveyor. The task is performed continuously for 4-hour shifts.

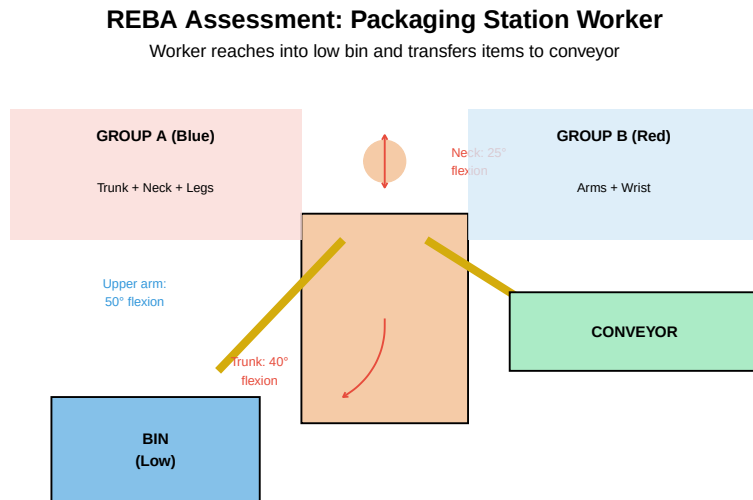


Figure 8.14: REBA Assessment Scenario: Packaging Worker

8.7.4.1 Step-by-Step Assessment

Live cell — try different posture, load, or coupling scores.

```
#| label: reba-step-by-step
# =====
# REBA Assessment: Packaging Station Worker
# =====
# Worker bends forward to pick items from low bin and place on conveyor
# Items weigh approximately 7 kg each
# Task performed continuously with repetitive motions

cat("===== GROUP A: TRUNK, NECK, LEGS =====\n\n")

# Step 1: Neck Score
# Worker looks down into bin - approximately 25° flexion
neck_base <- 2      # >20° flexion
```

```

neck_twist <- 0      # No twist or side-bending
neck_score <- neck_base + neck_twist
cat("Step 1 - Neck Score:", neck_score, "\n")
cat("  Position: >20° flexion (looking down into bin)\n")
cat("  Base score: 2, Twist adjustment: 0\n\n")

# Step 2: Trunk Score
# Worker bends forward approximately 40° to reach bin
trunk_base <- 3      # 20-60° flexion
trunk_twist <- 1     # Slight twist when reaching
trunk_score <- trunk_base + trunk_twist
cat("Step 2 - Trunk Score:", trunk_score, "\n")
cat("  Position: 40° forward flexion with slight twist\n")
cat("  Base score: 3, Twist adjustment: +1\n\n")

# Step 3: Legs Score
# Worker stands with weight distributed evenly
legs_base <- 1       # Bilateral weight bearing
legs_knee <- 0       # Knees not flexed significantly
legs_score <- legs_base + legs_knee
cat("Step 3 - Legs Score:", legs_score, "\n")
cat("  Position: Standing, bilateral weight bearing\n")
cat("  Base score: 1, Knee adjustment: 0\n\n")

# Step 4: Table A Lookup
# Trunk = 4, Neck = 2, Legs = 1
# From Table A: Score = 5
table_a_score <- 5
cat("Step 4 - Table A Score:", table_a_score, "\n")
cat("  Lookup: Trunk=4, Neck=2, Legs=1 → Table A = 5\n\n")

# Step 5: Load/Force Score
# Items weigh approximately 7 kg
load_force <- 1      # 5-10 kg
shock_force <- 0     # No shock/rapid buildup
load_score <- load_force + shock_force
cat("Step 5 - Load/Force Score:", load_score, "\n")
cat("  Load: 7 kg (in 5-10 kg range) = 1\n\n")

# SCORE A CALCULATION
score_a <- table_a_score + load_score
cat(">>> SCORE A = Table A + Load/Force =", table_a_score, "+", load_score, "=", score_a, "\n\n")

cat("===== GROUP B: UPPER ARM, LOWER ARM, WRIST =====\n\n")

```

```

# Step 6: Upper Arm Score
# Arms reaching forward/down approximately 50° flexion
upper_arm_base <- 3 # 45-90° flexion
shoulder_raised <- 0 # Not raised
arm_abducted <- 0 # Not abducted
arm_support <- 0 # Not supported
upper_arm_score <- upper_arm_base + shoulder_raised + arm_abducted + arm_support
cat("Step 6 - Upper Arm Score:", upper_arm_score, "\n")
cat(" Position: 50° flexion reaching into bin\n")
cat(" Base score: 3, No adjustments needed\n\n")

# Step 7: Lower Arm Score
# Elbow bent approximately 70° (in optimal range)
lower_arm_score <- 1 # 60-100° flexion
cat("Step 7 - Lower Arm Score:", lower_arm_score, "\n")
cat(" Position: 70° elbow flexion (optimal range)\n\n")

# Step 8: Wrist Score
# Wrist in moderate flexion to grasp items
wrist_base <- 2 # >15° flexion
wrist_twist <- 0 # No significant twist
wrist_score <- wrist_base + wrist_twist
cat("Step 8 - Wrist Score:", wrist_score, "\n")
cat(" Position: ~20° wrist flexion when grasping\n")
cat(" Base score: 2, Twist adjustment: 0\n\n")

# Step 9: Table B Lookup
# Upper Arm = 3, Lower Arm = 1, Wrist = 2
# From Table B: Score = 4
table_b_score <- 4
cat("Step 9 - Table B Score:", table_b_score, "\n")
cat(" Lookup: Upper Arm=3, Lower Arm=1, Wrist=2 → Table B = 4\n\n")

# Step 10: Coupling Score
# Worker grasps boxes - acceptable grip but not ideal handles
coupling_score <- 1 # Fair coupling
cat("Step 10 - Coupling Score:", coupling_score, "\n")
cat(" Coupling: Fair - hand hold acceptable but not ideal\n\n")

# SCORE B CALCULATION
score_b <- table_b_score + coupling_score
cat(">>> SCORE B = Table B + Coupling =", table_b_score, "+", coupling_score, "=", score_b, "\n\n")

cat("===== FINAL REBA SCORE CALCULATION =====\n\n")

```

```

# Step 11: Table C Lookup
# Score A = 6, Score B = 5
# From Table C: Score = 7
table_c_score <- 7
cat("Step 11 - Table C Score:", table_c_score, "\n")
cat("  Lookup: Score A=6, Score B=5 → Table C = 7\n\n")

# Step 12: Activity Score
# Check all three conditions:
static_posture <- 0 # Body parts not held >1 minute
repeated_actions <- 1 # Repeated small range actions (continuous picking)
rapid_changes <- 0 # No rapid large range changes
activity_score <- static_posture + repeated_actions + rapid_changes
cat("Step 12 - Activity Score:", activity_score, "\n")
cat("  Static posture >1 min: No (+0)\n")
cat("  Repeated actions >4×/min: Yes (+1)\n")
cat("  Rapid large changes: No (+0)\n\n")

# FINAL REBA SCORE
reba_final <- table_c_score + activity_score
cat("                                \n")
cat("      FINAL REBA SCORE:", reba_final, "                                \n")
cat("                                \n\n")

# Interpretation
cat("INTERPRETATION:\n")
cat("                                \n")
if (reba_final == 1) {
  cat("Risk Level: NEGLIGIBLE\n")
  cat("Action: None necessary\n")
} else if (reba_final <= 3) {
  cat("Risk Level: LOW\n")
  cat("Action: May be necessary\n")
} else if (reba_final <= 7) {
  cat("Risk Level: MEDIUM\n")
  cat("Action: Necessary\n")
} else if (reba_final <= 10) {
  cat("Risk Level: HIGH\n")
  cat("Action: Necessary soon\n")
} else {
  cat("Risk Level: VERY HIGH\n")
  cat("Action: Necessary NOW - Stop task immediately\n")
}
cat("                                \n")

```

8.7.4.2 Recommendations for This Assessment

Based on the REBA score of **8 (HIGH risk)**, the following interventions should be implemented soon:

Table 8.26: Recommended Interventions Based on REBA Assessment

Priority	Intervention	Target Risk Factor	Expected Impact
1	Raise bin height	Trunk flexion	Reduce trunk score from 4 to 2
2	Add tilt mechanism	Neck flexion	Reduce neck score from 2 to 1
3	Reduce load weight	Load/force	Reduce load score from 1 to 0
4	Implement job rotation	Repetition	Reduce activity score

8.7.5 REBA Example Calculation

Live cell — plug in your own posture scores and re-run.

```
#| label: reba-example
# REBA Assessment Example: Assembly Worker

# Group A Scores (Trunk, Neck, Legs)
trunk_score <- 3      # 20-60° forward bend
neck_score <- 2       # >20° flexion
legs_score <- 1        # Bilateral weight bearing

# Look up Table A score (trunk + neck + legs)
# For this combination: Score A = 4
table_a_score <- 4

# Load/Force Score
load_force <- 1        # 5-10 kg intermittent

# Score A with load
score_a <- table_a_score + load_force
cat("Score A (posture + load):", score_a, "\n")

# Group B Scores (Upper arm, Lower arm, Wrist)
upper_arm_score <- 3   # 45-90° flexion
lower_arm_score <- 2   # <60° or >100° flexion
wrist_score <- 2       # >15° flexion/extension

# Look up Table B score
# For this combination: Score B = 5
table_b_score <- 5

# Coupling Score
```

```

coupling <- 1          # Acceptable grip

# Score B with coupling
score_b <- table_b_score + coupling
cat("Score B (posture + coupling):", score_b, "\n")

# Look up Table C (Score A × Score B)
# For Score A = 5 and Score B = 6: Score C = 8
score_c <- 8

# Activity Score
activity <- 1          # One or more body parts static >1 min

# Final REBA Score
reba_final <- score_c + activity
cat("\n=== FINAL REBA SCORE:", reba_final, "===\n")

# Interpretation
cat("\nInterpretation:\n")
if (reba_final <= 1) {
  cat("Risk Level: Negligible - No action necessary\n")
} else if (reba_final <= 3) {
  cat("Risk Level: Low - Action may be necessary\n")
} else if (reba_final <= 7) {
  cat("Risk Level: Medium - Action necessary\n")
} else if (reba_final <= 10) {
  cat("Risk Level: High - Action necessary soon\n")
} else {
  cat("Risk Level: Very High - Action necessary NOW\n")
}

```

Practice Problem: REBA Assessment

Scenario: A worker is performing a task with the following postures:

- **Trunk:** 30° forward flexion, twisted
- **Neck:** 15° flexion
- **Legs:** Weight on one leg
- **Upper Arm:** 60° flexion, shoulder raised
- **Lower Arm:** 80° flexion
- **Wrist:** Neutral
- **Load:** 8 kg carried
- **Coupling:** Fair grip (handles exist but not ideal)
- **Activity:** Repeated small range actions

Calculate the REBA score and determine the action level.

Solution:

Trunk: 3 + 1 (twisted) = 4
 Neck: 1 (0-20° flexion)
 Legs: 2 (weight on one leg)
 Table A = 5
 Load/Force = 1 (5-10kg)
 Score A = 6

Upper Arm: 3 + 1 (raised) = 4
 Lower Arm: 1 (60-100°)
 Wrist: 1 (neutral)
 Table B = 4
 Coupling = 1 (fair)
 Score B = 5

Table C (Score A=6, Score B=5) = 8
 Activity = +1 (repeated actions)
 REBA Score = 9

Risk Level: HIGH - Action necessary soon

8.7.6 NIOSH Lifting Equation

The **NIOSH Lifting Equation** calculates a **Recommended Weight Limit (RWL)** for lifting tasks.

$$RWL = LC \times HM \times VM \times DM \times AM \times FM \times CM$$

Where:

Table 8.27: NIOSH Lifting Equation Factors

Factor	Name	Value	Description
LC	Load Constant	23 kg (51 lb)	Maximum weight under ideal c
HM	Horizontal Multiplier	25/H	H = horizontal distance from r
VM	Vertical Multiplier	$1 - 0.003\sqrt{V-75}$	V = vertical height of hands at
DM	Distance Multiplier	$0.82 + 4.5/D$	D = vertical travel distance (c
AM	Asymmetric Multiplier	$1 - 0.0032A$	A = angle of asymmetry (degre
FM	Frequency Multiplier	Table lookup	Based on lift frequency and du
CM	Coupling Multiplier	Table lookup	Based on hand-load coupling q

Lifting Index (LI) indicates risk level:

$$LI = \frac{\text{Actual Weight}}{\text{RWL}}$$

Table 8.28: Lifting Index Interpretation

Lifting Index	Risk	Action
LI 1.0	Low	Task acceptable for most workers
1.0 < LI 2.0	Moderate	Some workers may be at risk - consider changes
2.0 < LI 3.0	High	Many workers at risk - changes recommended
LI > 3.0	Very High	Unacceptable - immediate changes required

Live cell — change the distances, angle, or frequency and re-run.

```
#| label: niosh-example
# NIOSH Lifting Equation Example

# Task parameters
H <- 40    # Horizontal distance (cm)
V <- 30    # Vertical height at start (cm)
D <- 60    # Vertical travel distance (cm)
A <- 30    # Asymmetry angle (degrees)
F <- 2     # Lifts per minute
duration <- 2 # Hours of lifting

# Constants
LC <- 23   # Load constant (kg)

# Calculate multipliers
HM <- 25 / H
VM <- 1 - 0.003 * abs(V - 75)
DM <- 0.82 + 4.5 / D
AM <- 1 - 0.0032 * A

# Frequency multiplier (from table, for 2 lifts/min, 2 hours, V<75cm)
FM <- 0.84

# Coupling multiplier (assume "fair" coupling)
CM <- 0.95

cat("Multipliers:\n")
cat("HM =", round(HM, 3), "\n")
cat("VM =", round(VM, 3), "\n")
cat("DM =", round(DM, 3), "\n")
cat("AM =", round(AM, 3), "\n")
```

```

cat("FM =", FM, "\n")
cat("CM =", CM, "\n\n")

# Calculate RWL
RWL <- LC * HM * VM * DM * AM * FM * CM
cat("Recommended Weight Limit (RWL):", round(RWL, 1), "kg\n\n")

# Calculate Lifting Index for actual load
actual_weight <- 15 # kg
LI <- actual_weight / RWL
cat("Actual Weight:", actual_weight, "kg\n")
cat("Lifting Index (LI):", round(LI, 2), "\n\n")

# Interpretation
if (LI <= 1.0) {
  cat("Risk Level: LOW - Task acceptable\n")
} else if (LI <= 2.0) {
  cat("Risk Level: MODERATE - Consider changes\n")
} else if (LI <= 3.0) {
  cat("Risk Level: HIGH - Changes recommended\n")
} else {
  cat("Risk Level: VERY HIGH - Immediate changes required\n")
}

```

8.8 AI-Powered Ergonomic Assessment

Artificial intelligence is revolutionizing ergonomic assessment by enabling **real-time, automated analysis** of worker postures without disrupting operations.

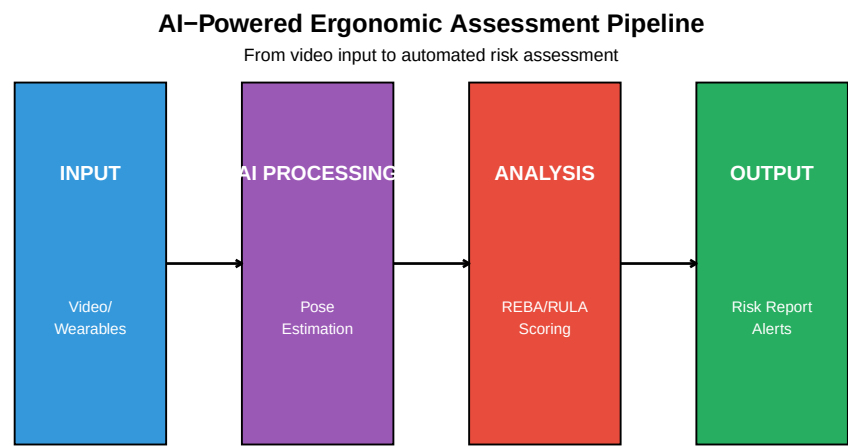


Figure 8.15: AI Ergonomics Assessment Pipeline

Table 8.29: AI-Powered Ergonomic Assessment Tools

Tool	Technology	Key Features
TuMeke Ergonomics	Computer vision + wearables	Real-time REBA scoring, multi-camera support
VelocityEHS	AI video analysis	Automated job analysis, risk prioritization
Protex AI	Real-time video AI	Privacy-preserving analysis, floor-wide monitoring
SoterTask	Wearable sensors + AI	Wearable clips, real-time coaching, haptic feedback
Viso.ai	Custom AI models	Custom pose detection, integration APIs
ErgoPlus Platform	Mobile app + AI	Mobile assessment, improvement tracking

8.8.1 Benefits of AI Ergonomic Assessment

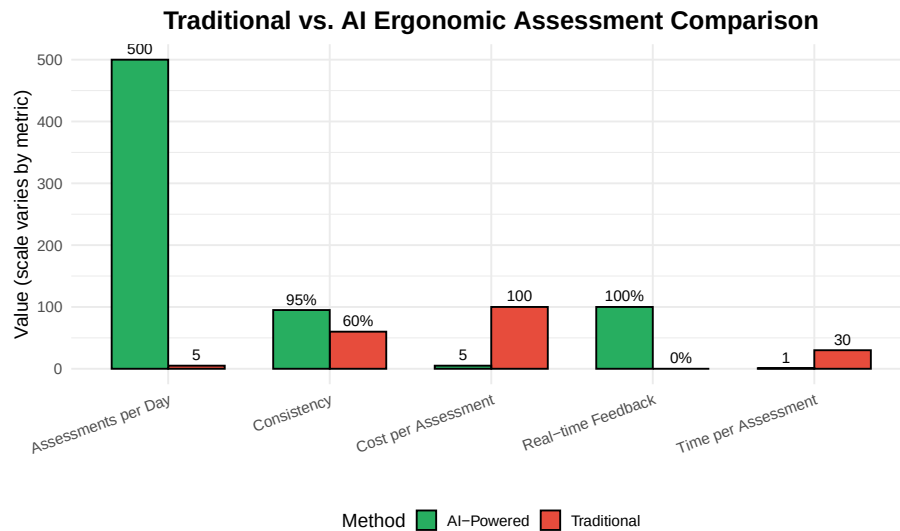


Figure 8.16: Traditional vs. AI Ergonomic Assessment

8.9 Ergonomics in Automated Manufacturing

Industrial automation inherently involves the mechanization of processes traditionally carried out by human operators. In the automation design phase, engineers must strike a balance between mechanization and the necessary human interaction with these systems.

8.9.1 Human-Automation Interaction Points

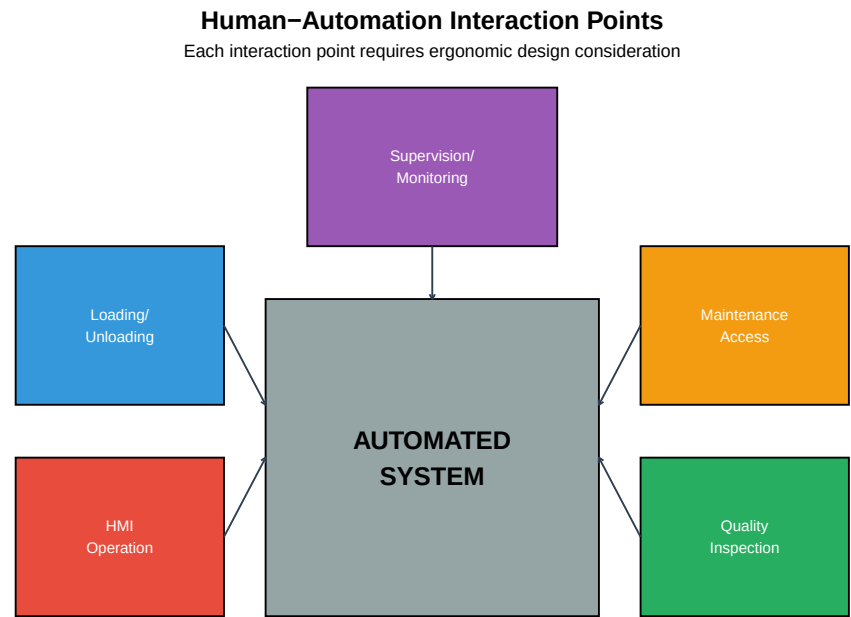


Figure 8.17: Ergonomic Considerations in Automated Systems

Table 8.30: Ergonomic Considerations for Automated Systems

Interaction Point	Ergonomic Challenges	Design Solutions
Loading/Unloading	Repetitive motion, lifting, reaching, awkward postures	Height-adjustable conveyors
HMI Operation	Static posture, visual strain, cognitive load	Adjustable screens, proper
Maintenance	Awkward access, confined spaces, tool use	Adequate access space, tool
Quality Inspection	Static positions, visual strain, fine motor tasks	Adjustable fixtures, magnif
Supervision	Sedentary position, cognitive fatigue, vigilance	Multiple monitors at eye le

8.9.2 Collaborative Robot Ergonomics

Cobots present unique ergonomic opportunities and challenges:

Table 8.31: Ergonomic Considerations for Collaborative Robots

Aspect	Opportunity	Challenge
Task Allocation	Robot handles heavy, repetitive, or precision tasks	Determining optimal task div

Workstation Layout	Shared workspace allows flexible positioning	Ensuring adequate
Pacing	Robot adapts to human pace, not vice versa	Avoiding pressure
Force Assistance	Robot provides powered assist for lifting	Proper force limits
Cognitive Load	Robot handles routine tasks, human focuses on judgment	Maintaining situational awareness

8.10 Employer Responsibilities

Employers have both legal and ethical obligations to protect workers from ergonomic hazards.

Table 8.32: Employer Ergonomics Program Responsibilities

Responsibility	Actions
Management Commitment	Allocate resources, set goals, lead by example
Worker Involvement	Include workers in assessments, design teams, solution development
Training	Ergonomics awareness, proper techniques, symptom recognition
Hazard Identification	Regular assessments, worker surveys, injury analysis
Early Reporting	Non-punitive reporting system, prompt response to symptoms
Hazard Control	Engineering controls, work organization, PPE as last resort
Program Evaluation	Track injuries, measure improvements, adjust programs

OSHA Ergonomics Guidelines

While OSHA does not have a specific ergonomics standard, employers are still responsible under the **General Duty Clause** (Section 5(a)(1)) which requires employers to provide a workplace “free from recognized hazards.”

OSHA’s Recommended Program Elements: 1. Management leadership 2. Worker participation 3. Hazard identification and assessment 4. Hazard prevention and control 5. Education and training 6. Program evaluation

Industry-Specific Guidelines: - Poultry Processing - Nursing Homes - Retail Grocery Stores - Shipyards

Visit: OSHA Ergonomics eTool

8.11 Case Study: Ergonomics in Automated Assembly

Company: Automotive parts manufacturer

Problem: High rate of upper extremity MSDs in assembly department - 15 recordable injuries per year - Lost workdays averaging 20 days per case - Workers' comp costs exceeding \$300,000 annually

Assessment Findings:

Table 8.33: Ergonomic Assessment Findings

Workstation	Primary Issues	REBA Score	Risk Level
Sub-assembly	Overhead reaching, high repetition	9	High
Main line	Forward bending, static standing	8	High
Inspection	Awkward wrist postures, visual strain	7	Medium
Packaging	Lifting, carrying, repetition	10	Very High

Interventions Implemented:

1. **Sub-assembly:**
 - Tilting fixtures to bring work closer
 - Parts bins repositioned to eliminate overhead reaching
 - Cobot added for repetitive insertion tasks
2. **Main line:**
 - Height-adjustable workstations installed
 - Anti-fatigue mats added
 - Job rotation implemented
3. **Inspection:**
 - Magnifying task lights installed
 - Angled fixtures to reduce wrist bending
 - Ergonomic seating provided
4. **Packaging:**
 - Vacuum lift assists installed
 - Conveyor heights adjusted
 - Case erectors automated

Results After 18 Months:

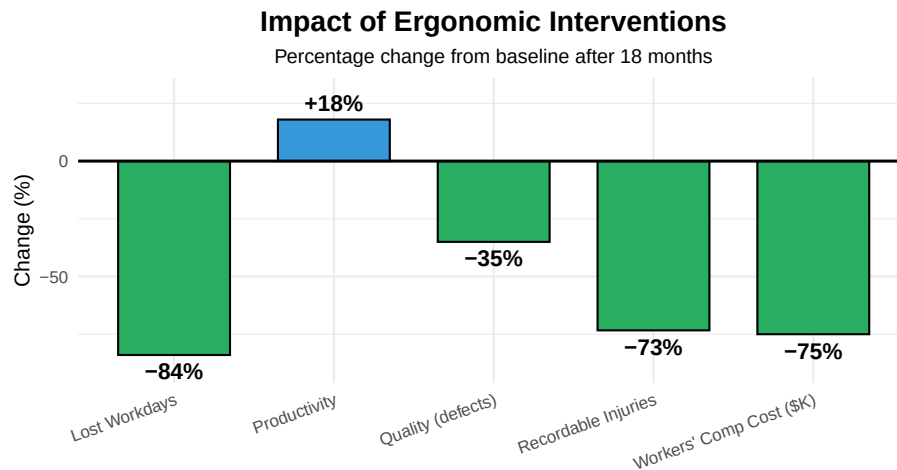


Figure 8.18: Ergonomic Intervention Results

ROI Analysis: (live cell — try different investment or savings figures)

```
#| label: roi-analysis
# Investment
equipment_cost <- 150000 # Lift assists, fixtures, cobots
training_cost <- 15000
total_investment <- equipment_cost + training_cost

# Annual Savings
injury_reduction <- (15 - 4) * 20000 # Avg cost per injury
productivity_gain <- 0.18 * 500000 # 18% improvement on labor cost
quality_improvement <- 0.35 * 50000 # 35% defect reduction

annual_savings <- injury_reduction + productivity_gain + quality_improvement

cat("Total Investment: $", format(total_investment, big.mark = " "), "\n")
cat("Annual Savings: $", format(annual_savings, big.mark = " "), "\n")
cat("Payback Period:", round(total_investment / annual_savings, 1), "years\n")
cat("5-Year ROI:", round((annual_savings * 5 - total_investment) / total_investment * 100, 1), "%\n")
```

8.12 Summary

Table 8.34: Ergonomics in Manufacturing: Key Concepts Summary

Topic	Key Points
Ergonomics Definition	Fitting the job to the worker; physical, cognitive, organizational domains
Risk Factors	Posture, repetition, force, static positions, contact stress, vibration, environment
MSDs	Injuries to muscles, nerves, tendons; develop gradually from exposure to disability
Prevention	Engineering controls most effective; proper lifting, neutral postures, recovery time
Design Principles	Neutral posture, anthropometry, work height optimization, environmental control
Assessment Tools	REBA for whole body, RULA for upper limb, NIOSH for lifting tasks
AI in Ergonomics	Real-time monitoring, automated REBA/RULA scoring, continuous assessment
Employer Role	Management commitment, worker involvement, training, hazard control, evaluation

8.13 Review Questions

Question 1: What are the seven ergonomic risk factors, and which two are most commonly associated with carpal tunnel syndrome?

The seven ergonomic risk factors are: 1. Awkward Postures 2. Repetition 3. Force 4. Static Positions 5. Contact Stress 6. Vibration 7. Environmental Factors

Carpal tunnel syndrome is most commonly associated with: - **Repetition** - Repeated wrist movements compress the median nerve - **Force** - Forceful gripping increases pressure in the carpal tunnel - Also contributing: awkward wrist postures and vibration

Question 2: A worker performs a task 8 times per minute for a 4-hour shift. The cycle time is 7.5 seconds. Is this considered high repetition? Why?

Analysis: - Cycle time: 7.5 seconds (< 15 seconds threshold) - Repetitions per hour: $8 \times 60 = 480$ (> 240 threshold)

Yes, this is HIGH repetition because: 1. Cycle time is less than 15 seconds 2. Repetitions exceed 240 per hour

Risk Level: High - Immediate intervention needed

Recommended actions: - Job rotation with different tasks - Automation of repetitive elements - Micro-breaks every 20-30 minutes - Workstation redesign to reduce cycle elements

Question 3: Explain why ergonomic design should accommodate the 5th to 95th percentile of the population rather than designing for the “average” person.

Designing for the average excludes most people: - The “average” person in ALL dimensions doesn’t exist - A person may be average height but have

long arms or short legs - Designing for only the 50th percentile fits only about 50% of users

The 5th-95th percentile range: - Accommodates 90% of the population - Addresses both small and large users - Provides adjustability rather than fixed dimensions

Application rules: - **Clearances** (overhead, legroom): Design for 95th percentile (largest) - **Reach distances:** Design for 5th percentile (smallest) - **Work surfaces:** Make adjustable to accommodate range

Example: A fixed workstation at 95cm height: - Too high for 5th percentile female (elbow at ~93cm) - Too low for 95th percentile male (elbow at ~117cm) - Adjustable range of 85-115cm accommodates both

Question 4: Calculate the NIOSH Lifting Index for the following task and determine if intervention is needed.

Task Parameters: - Horizontal distance (H): 35 cm - Vertical height (V): 80 cm - Travel distance (D): 50 cm - Asymmetry (A): 0° (symmetric lift) - Frequency: 4 lifts/minute for 1 hour - Coupling: Good (handles) - Actual load: 18 kg

Solution:

$$LC = 23 \text{ kg}$$

$$HM = 25/35 = 0.714$$

$$VM = 1 - 0.003|80-75| = 0.985$$

$$DM = 0.82 + 4.5/50 = 0.91$$

$$AM = 1 - 0.0032(0) = 1.0$$

$$FM = 0.75 \text{ (4/min, 1 hour, } V > 75\text{cm, from table)}$$

$$CM = 1.0 \text{ (good coupling)}$$

$$RWL = 23 \times 0.714 \times 0.985 \times 0.91 \times 1.0 \times 0.75 \times 1.0$$

$$RWL = 11.0 \text{ kg}$$

$$LI = 18 / 11.0 = 1.64$$

Risk Level: MODERATE ($1.0 < LI < 2.0$)

Action: Some workers may be at risk - consider changes

Recommendations: - Reduce horizontal distance (bring load closer) - Reduce lifting frequency - Use mechanical assist for loads >12 kg

Question 5: What are three advantages of AI-powered ergonomic assessment over traditional methods?

1. Scale and Coverage: - Traditional: 5-10 assessments per day by trained ergonomist - AI: Hundreds or thousands of assessments simultaneously - Can monitor entire facility continuously

2. Consistency and Objectivity: - Traditional: Inter-rater reliability issues; different assessors may score differently - AI: Consistent scoring algorithm applied uniformly - Removes subjective interpretation

3. Real-time Feedback: - Traditional: Assessment completed, report generated days/weeks later - AI: Immediate scoring and alerts - Can provide real-time coaching to workers

Additional advantages: - Lower cost per assessment - Non-disruptive (workers not aware of assessment) - Longitudinal tracking of posture patterns - Data for predictive analytics

8.14 Additional Resources

8.14.1 Videos

Ergonomics Fundamentals: - Introduction to Workplace Ergonomics - Safe Manual Handling Techniques

Assessment Tools: - REBA Assessment Tutorial - RULA Assessment Guide - NIOSH Lifting Equation Explained

AI Ergonomics: - TuMeke AI Ergonomics Demo - VelocityEHS Industrial Ergonomics

8.14.2 Online Tools

- ErgoPlus REBA Calculator
- NIOSH Lifting Equation Calculator
- Cornell University Ergonomics Web

8.15 References

- Berlin, C., & Adams, C. (2017). *Production Ergonomics: Designing Work Systems to Support Optimal Human Performance*. Ubiquity Press.
- Hignett, S., & McAtamney, L. (2000). Rapid Entire Body Assessment (REBA). *Applied Ergonomics*, 31(2), 201-205.
- McAtamney, L., & Corlett, E.N. (1993). RULA: A Survey Method for Investigation of Work-Related Upper Limb Disorders. *Applied Ergonomics*, 24(2), 91-99.
- NIOSH. (1994). *Applications Manual for the Revised NIOSH Lifting Equation*. DHHS (NIOSH) Publication No. 94-110.
- OSHA. (2000). *Ergonomics: The Study of Work*. OSHA 3125.

- Waters, T.R., Putz-Anderson, V., & Garg, A. (1993). Revised NIOSH Equation for the Design and Evaluation of Manual Lifting Tasks. *Ergonomics*, 36(7), 749-776.
 - ISO 11228-1:2021. Ergonomics — Manual handling — Part 1: Lifting, lowering and carrying.
 - ISO 11228-3:2007. Ergonomics — Manual handling — Part 3: Handling of low loads at high frequency.
-

Chapter 9

Machine Guarding and Functional Safety

9.1 Learning Objectives

By the end of this chapter, you will be able to:

- Apply the Hierarchy of Controls to manufacturing safety design
- Identify and specify appropriate physical guarding and interlock systems
- Select and position presence-sensing devices (light curtains, area scanners)
- Calculate safety distances for protective device placement
- Understand collaborative robot safety requirements
- Design emergency stop systems according to standards
- Differentiate between safety integrity levels (SIL) and performance levels (PL)

“Safety isn’t expensive, it’s priceless.” — Anonymous

9.2 Introduction: The Safety Hierarchy

In process engineering, safety is not merely a feature added at the end of a project; it is a **fundamental design constraint**. Before deploying safety devices, the engineer must apply the **Hierarchy of Controls** — a systematic approach to eliminating or reducing workplace hazards.

Warning: Using ``size`` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use ``linewidth`` instead.

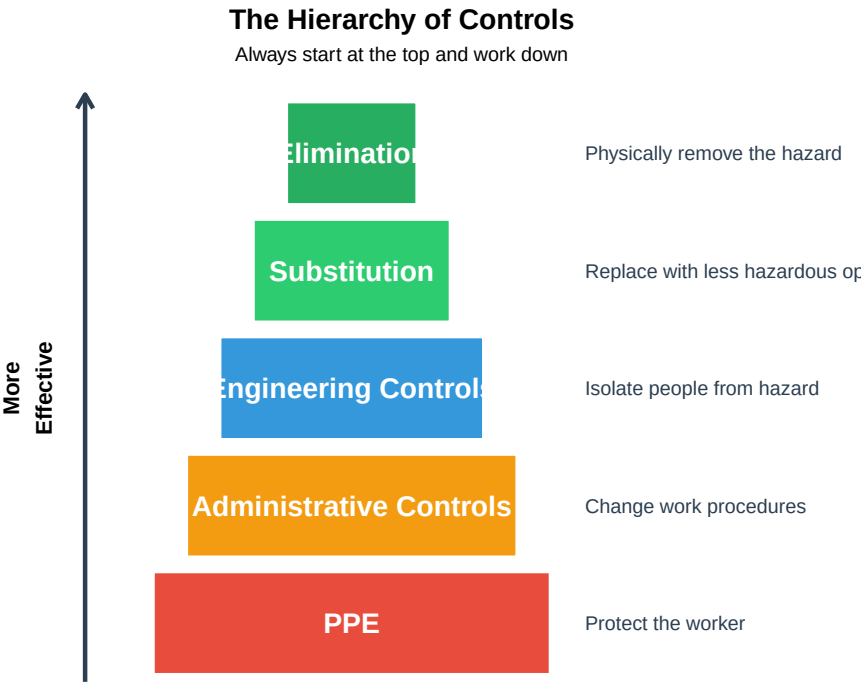


Figure 9.1: The Hierarchy of Controls (Most to Least Effective)

Table 9.1: Hierarchy of Controls: Detailed

Priority	Control Level	Description	Example
1	**Elimination**	Physically remove the hazard entirely	Elimination
2	**Substitution**	Replace with something less hazardous	Use low
3	**Engineering Controls**	Isolate people from the hazard through design	Install r
4	**Administrative Controls**	Change the way people work	Safety p
5	**PPE**	Protect the worker with equipment	Safety g

Discussion: Why not just rely on PPE?

PPE (Personal Protective Equipment) is the least effective control because:

1. **It doesn't eliminate the hazard** — the danger still exists
2. **It depends on human compliance** — workers may forget or choose not to wear it
3. **It can fail** — equipment degrades, doesn't fit properly, or is damaged
4. **It's the last line of defense** — if PPE fails, the worker is exposed

Key Principle: The more we can design out hazards through engineering controls, the less we depend on human behavior to keep people safe.

“If you have to post a warning sign, you’ve already failed at design.”

9.3 Physical Guarding and Interlocks

Physical barriers are the most intuitive form of protection. They create a “hard” separation between the operator and moving parts, ensuring that humans cannot enter the danger zone during machine operation.

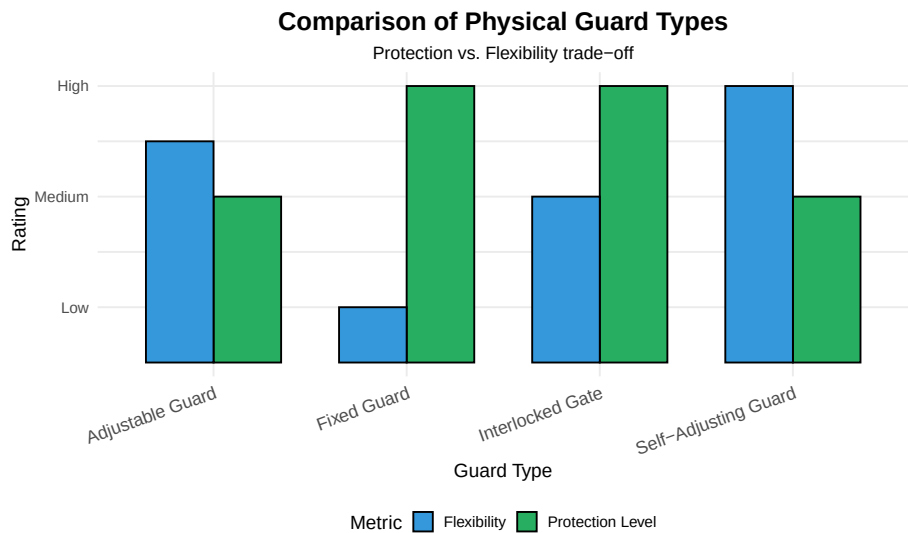


Figure 9.2: Types of Physical Guarding

9.3.1 Fixed Guards

Fixed guards are permanent barriers that require tools to remove. They are the simplest and most reliable form of protection.

Table 9.2: Fixed Guards: Key Characteristics

Characteristic	Description
Construction	Wire mesh, polycarbonate sheets, or solid metal panels
Removal	Requires tools (screws, bolts) to remove
Best Application	Areas that need infrequent access for maintenance only
Advantages	Simple, reliable, low cost, no moving parts to fail

Disadvantages

Impedes visibility, slows maintenance, can be removed and not replaced

9.3.2 Interlocked Guards

Interlocked guards are access points equipped with sensors. When the guard is opened, the safety controller initiates a stop.

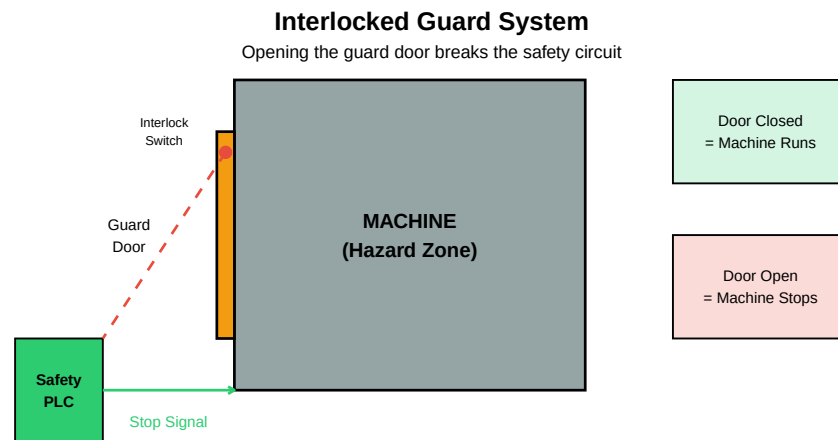


Figure 9.3: Interlocked Guard Operation Principle

9.3.3 Stop Categories

When an interlock is triggered, the machine must stop. The **stop category** determines how this happens:

Table 9.3: Machine Stop Categories (IEC 60204-1)

Category	Type	Method	Application
Category 0	Uncontrolled Stop	Immediate removal of power to actuators	Emergency stop
Category 1	Controlled Stop	Controlled deceleration, then power removed	High-inertia loads
Category 2	Controlled Stop	Controlled stop with power maintained	Process that must be maintained

9.3.4 Trapped Key Systems (Lock-Out/Tag-Out)

Trapped key systems use a mechanical sequence to ensure that a machine cannot be energized unless all access keys are returned to the control panel.

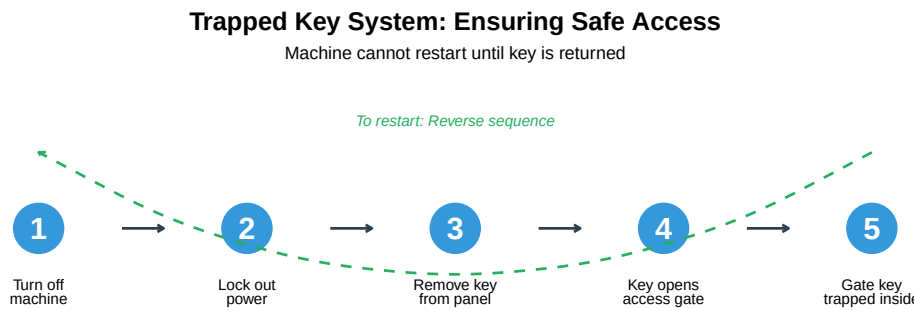


Figure 9.4: Trapped Key System Sequence

Case Study: The Fatal Bypass

Incident: A maintenance worker was killed when a robot arm unexpectedly activated while he was inside a robotic cell.

What Happened: - The guard interlock had been bypassed with a “defeat device” (a magnet taped to fool the sensor) - Workers had bypassed the interlock because it slowed down troubleshooting - The bypass had been in place for months without management knowledge

Root Causes: 1. Interlock caused inconvenience during frequent troubleshooting 2. No monitoring system to detect bypasses 3. Culture that tolerated “workarounds”

Lessons Learned: - Design interlocks that don’t impede normal operations - Use tamper-evident or tamper-resistant devices - Regular safety audits to detect bypasses - Zero-tolerance policy for defeating safety devices

Never bypass a safety device, even temporarily.

9.4 Presence-Sensing Devices

Presence-sensing devices allow for a “fence-less” environment, improving visibility and ergonomics while maintaining high safety levels. They detect when a person enters a hazardous zone and trigger a machine stop.

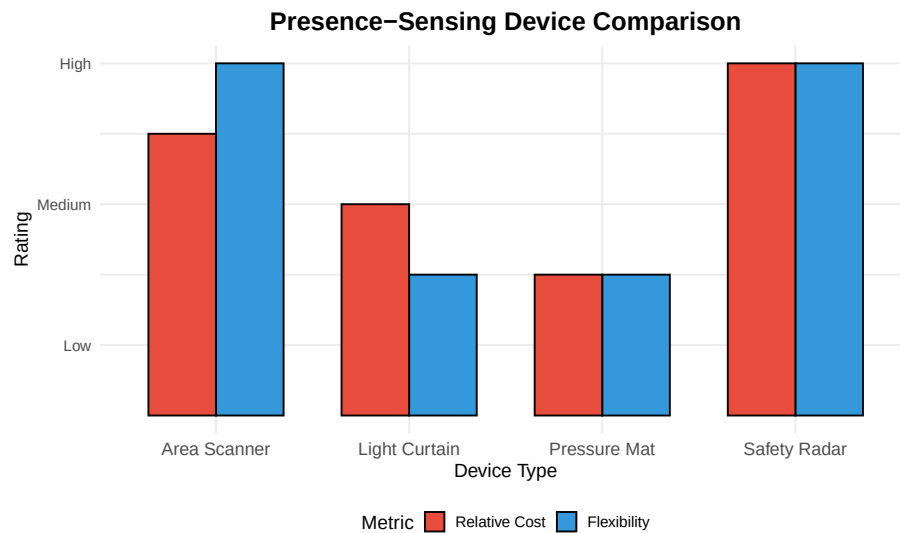


Figure 9.5: Types of Presence-Sensing Devices

9.4.1 Light Curtains

Light curtains use a transmitter and receiver to create an array of infrared beams. If any beam is broken, the safety circuit is interrupted.

Light Curtain: Detection Through Beam Interruption

Any object larger than the resolution will trigger a stop

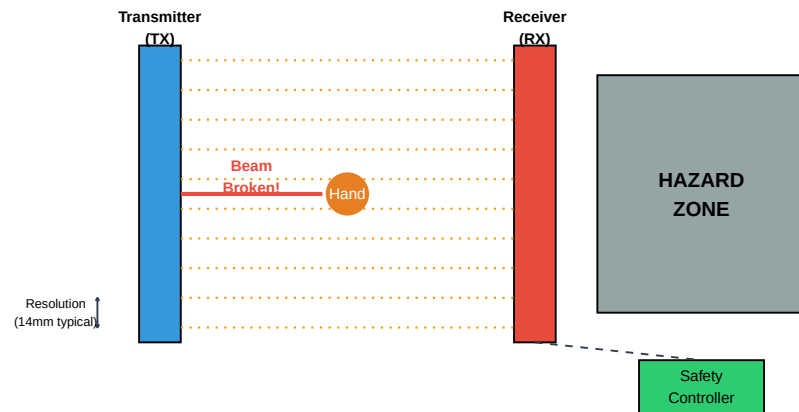


Figure 9.6: Light Curtain Operating Principle

Table 9.4: Light Curtain Specifications by Application

Parameter	Finger Detection (14mm)	Hand Detection (30mm)	Body Detection (40mm)
Resolution	14mm beam spacing	30mm beam spacing	40mm+ beam spacing
Height	150-1800mm	300-1800mm	900-2000mm
Range	0.5-20m	0.5-20m	0.5-30m
Response Time	5-15ms	10-20ms	15-30ms
Application	Point of operation guarding	Perimeter guarding	Area access detection

9.4.2 Laser Area Scanners

Laser area scanners use LiDAR technology to monitor a 2D plane. They are highly flexible and allow for programmable zones.

Laser Area Scanner: Programmable Safety Zones
270° scanning field with configurable warning and safety zones

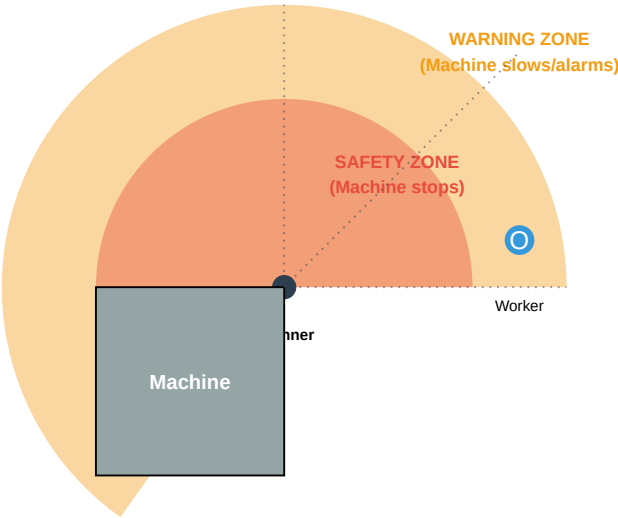


Figure 9.7: Laser Area Scanner Zones

Table 9.5: Light Curtains vs. Area Scanners

Feature	Light Curtains	Area Scanners
---------	----------------	---------------

Detection Type	Vertical/Linear Barrier	Horizontal/Area Coverage
Flexibility	Fixed installation	Programmable zones
Zone Configuration	Single detection plane	Multiple warning + safety zones
Cost	Generally lower (\$1,000-5,000)	Higher (\$3,000-10,000)
Best Use Case	Clear entry/exit points, press brakes	Large open floors, AGV paths, fl
Environmental Limits	Dust/debris can cause false trips	More tolerant of dust, outdoor ra

9.4.3 Pressure-Sensitive Safety Mats

Pressure-sensitive mats detect the presence of a person standing on them. They are used primarily in legacy systems or specific zones where optical sensors may be obscured.

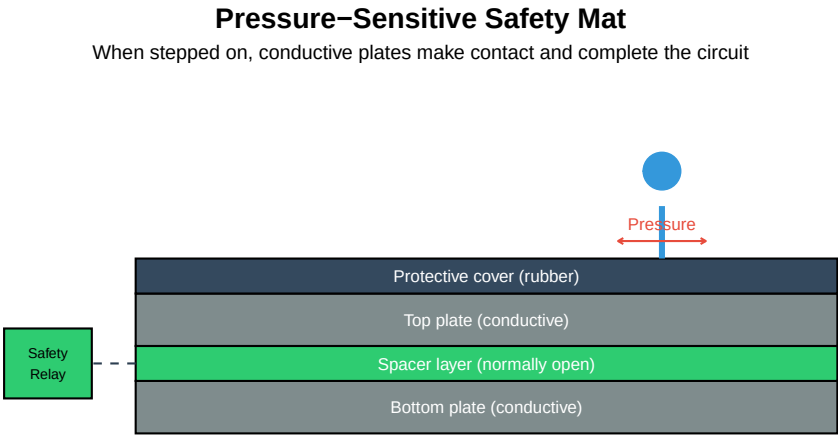


Figure 9.8: Pressure-Sensitive Safety Mat Construction

When to Use Pressure Mats vs. Optical Sensors

Use Pressure Mats When: - Environment has heavy dust, smoke, or steam (blinds optical sensors) - Detection area is well-defined and doesn't change - Workers need to stand in a specific location - Budget is limited

Use Optical Sensors When: - Flexibility in zone configuration is needed - Environment is relatively clean - Detection area is large or needs to change - Higher integrity level is required

Hybrid Approach: Many modern systems use both — area scanners for perimeter protection and pressure mats for specific danger zones near the machine.

9.5 Control-Actuated Safety Devices

These devices ensure that the operator’s body—specifically their hands—is in a predetermined **safe location** before the machine can cycle.

9.5.1 Two-Hand Controls

Two-hand controls are common in pressing or stamping operations. To initiate a cycle, the operator must press two buttons simultaneously.

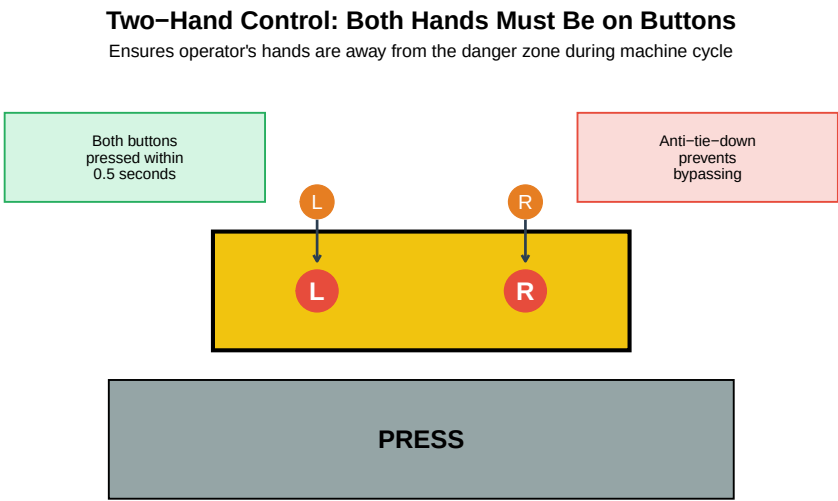


Figure 9.9: Two-Hand Control System

Anti-Tie-Down Requirement: Modern systems require both buttons to be pressed within **0.5 seconds** of each other. This prevents operators from taping one button down to work with one hand.

Table 9.6: Two-Hand Control Requirements

Requirement	Standard Value	Purpose
Synchronous activation	Within 0.5 seconds	Prevents one-hand operation
Continuous pressure	Must be held throughout cycle	Ensures hands stay on controls
Release before restart	Both must be released before next cycle	Prevents automatic cycling
Button spacing	Minimum 260mm (10 inches) apart	Cannot press both with one hand/arm
Guards around buttons	Prevent accidental activation	Prevents hip, elbow, or object activation

9.5.2 Enable/Deadman Switches

Enable switches (also called deadman switches) are used during **Teach Mode** for robotics. The operator must hold a three-position trigger in the **middle** position.

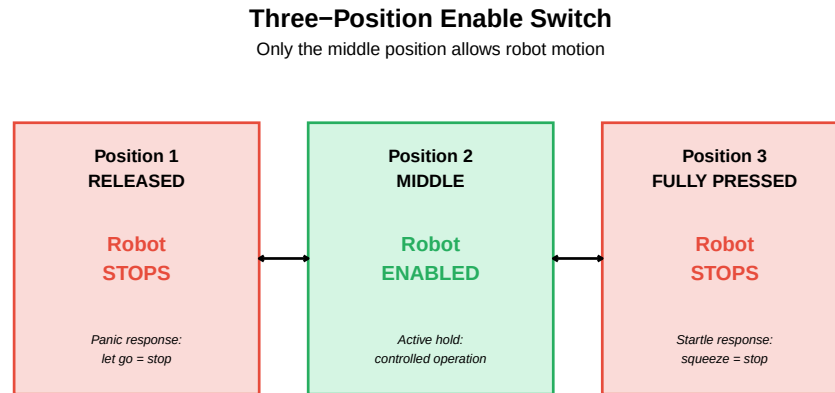


Figure 9.10: Three-Position Enable Switch Operation

Why Three Positions?

The three-position design accounts for **two natural human reactions** to danger:

1. **Panic (Release):** When frightened, people often release what they’re holding — robot stops.
2. **Startle (Squeeze):** When startled, people often grip tighter — robot stops.

Only a deliberate, controlled **middle hold** allows the robot to move. This is fundamental to safe teach pendant operation.

9.6 Collaborative Robotics (Cobots)

Collaborative robots (cobots) represent a shift from “separation” to “collaboration.” They rely on sophisticated internal sensors rather than external cages, allowing humans and robots to work in the same space.

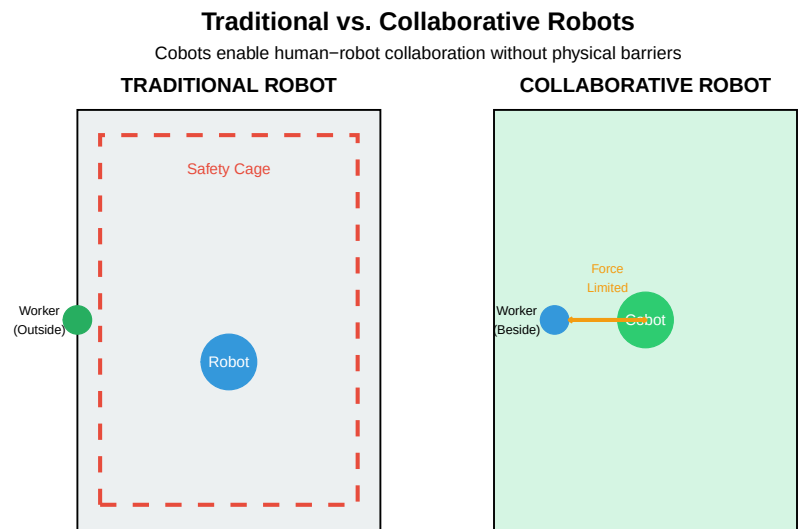


Figure 9.11: Traditional Robot vs. Collaborative Robot

9.6.1 Collaborative Operation Methods

ISO 10218 and ISO/TS 15066 define **four methods** for safe collaborative operation:

Table 9.7: Four Methods of Collaborative Robot Operation

Method	How It Works	Sensors Used
Safety-Rated Monitored Stop	Robot stops when human enters workspace, resumes when clear	Area scanners
Hand Guiding	Operator physically guides robot through motions	Force/torque sensors
Speed and Separation Monitoring	Robot slows or stops based on human proximity	Area scanners
Power and Force Limiting	Robot limits force/power to safe levels on contact	Joint torque sensors

9.6.2 Power and Force Limiting (PFL)

The most common cobot safety method is **Power and Force Limiting**. The robot detects a change in torque in its joints. If it bumps into a human, it stops before a regulated amount of force is exceeded.

Table 9.8: ISO/TS 15066 Biomechanical Limits (Quasi-static contact)

Body Region	Maximum Pressure (N/cm ²)	Maximum Force (N)	Risk Level
Skull/Forehead	130	130	Critical
Face	65	65	Critical
Neck	145	150	Critical
Back/Shoulders	210	210	Moderate
Chest	140	140	Moderate
Abdomen	110	110	Moderate
Hand/Fingers	300	140	Lower

Discussion: Are Cobots Always Safe?

Common Misconception: “Cobots are safe, so we don’t need to do a risk assessment.”

Reality: Cobots are *potentially* safe, but a **risk assessment is always required**.

Factors that can make a cobot unsafe:

1. **End-of-arm tooling** — A soft gripper is different from a sharp blade
2. **Workpiece** — Carrying a heavy or sharp object increases risk
3. **Speed** — Even force-limited robots can cause injury at high speed
4. **Application** — Some tasks bring humans closer to hazards
5. **Environment** — Slippery floors, confined spaces, etc.

Every collaborative application requires a documented risk assessment per ISO 12100.

9.7 Safety Distance Calculation

A safety device cannot be placed arbitrarily. It must be far enough away that the machine comes to a **complete halt** before the human can reach the hazard.

9.7.1 The Safety Distance Formula

The minimum safety distance S is calculated as:

$$S = (K \times T) + C$$

Where:

- K : The hand/body approach speed (standardized values)
- T : The total stopping time of the system (device response + machine stop time)

- C : The “penetration depth” (how far a hand can reach through before being detected)

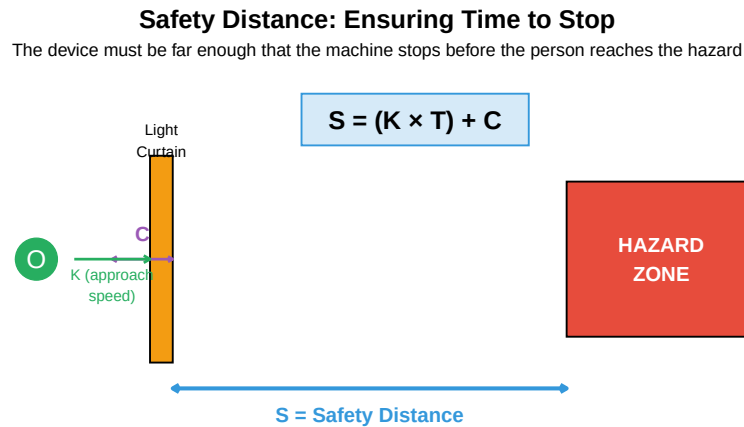


Figure 9.12: Safety Distance Components

9.7.2 Standard Values for K

Table 9.9: Standard Approach Speed Values (ISO 13855)

Approach Type	K Value (mm/s)	When to Use
Hand/Arm Approach (fast)	2000	Close approach to point of operation
Hand/Arm Approach (normal)	1600	General approach to machines
Walking Approach	1600	Area access detection
Standing Reach	0 (static)	Worker already in position

9.7.3 Penetration Depth (C)

The penetration depth depends on the **resolution** of the sensing device:

Table 9.10: Penetration Depth Values Based on Resolution

Resolution (mm)	C Value (mm)	Detection Type
14	0	Finger detection
>14 to 20	80	Finger/Hand
>20 to 30	130	Hand detection
>30 to 40	240	Hand/Arm

>40

850

Body detection

9.7.4 Example Calculation

Live cell — try different approach speed, response time, or resolution values.

```
#| label: safety-calc-example
# Given parameters
K <- 2000          # Approach speed (mm/s) - fast hand approach
T_device <- 0.020  # Light curtain response time (20 ms = 0.020 s)
T_machine <- 0.150 # Machine stopping time (150 ms = 0.150 s)
C <- 0            # Penetration depth for 14mm resolution

# Calculate total stopping time
T_total <- T_device + T_machine
cat("Total stopping time (T):", T_total * 1000, "ms\n")

# Calculate minimum safety distance
S <- (K * T_total) + C
cat("Minimum safety distance (S):", S, "mm\n")
cat("Minimum safety distance (S):", S / 25.4, "inches\n")

# Interpretation
cat("\n--- Interpretation ---\n")
cat("The light curtain must be at least", S, "mm from the hazard.\n")
cat("This ensures the machine stops before a person can reach the danger zone.\n")
```

Practice Problem: Calculate Safety Distance

Given: - Light curtain resolution: 30mm (hand detection) - Light curtain response time: 15ms - Machine stopping time: 200ms - Application: Operator hand approach

Calculate the minimum safety distance.

Solution:

```
K = 1600 mm/s (normal hand approach)
T = 0.015 + 0.200 = 0.215 seconds
C = 130 mm (for 30mm resolution)

S = (K × T) + C
S = (1600 × 0.215) + 130
S = 344 + 130
S = 474 mm (approximately 18.7 inches)
```

The light curtain must be installed at least **474mm** from the nearest hazard point.

9.8 Emergency Stop (E-Stop) Systems

The **Emergency Stop** is the “last resort” — it is not a safety device but a **complementary protective measure**. It must be available to stop the machine when all other measures have failed.

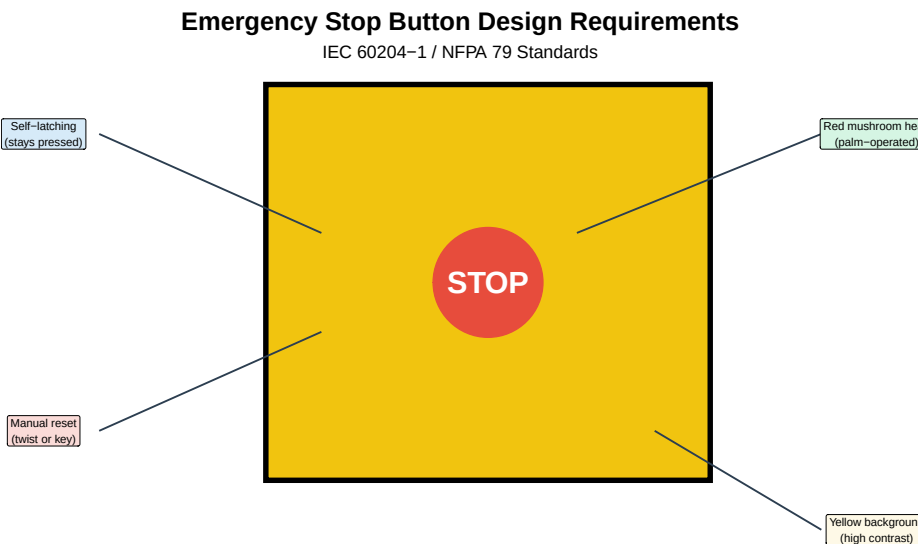


Figure 9.13: E-Stop Design Requirements

Table 9.11: E-Stop Requirements per IEC 60204-1

Requirement	Standard
Color	Red (RAL 3000 or equivalent)
Shape	Mushroom head (palm or fist operated)
Background	Yellow (high contrast)
Operation	Self-latching (stays activated when pressed)
Reset	Manual reset required (twist, pull, or key)
Accessibility	Within easy reach of all operators
Wiring	Normally closed contacts (fail-safe)
Function	Category 0 or Category 1 stop

9.8.1 E-Stop Pull Cords

For long conveyors or machines where an operator cannot quickly reach a fixed button, **pull cords** provide continuous E-Stop coverage.

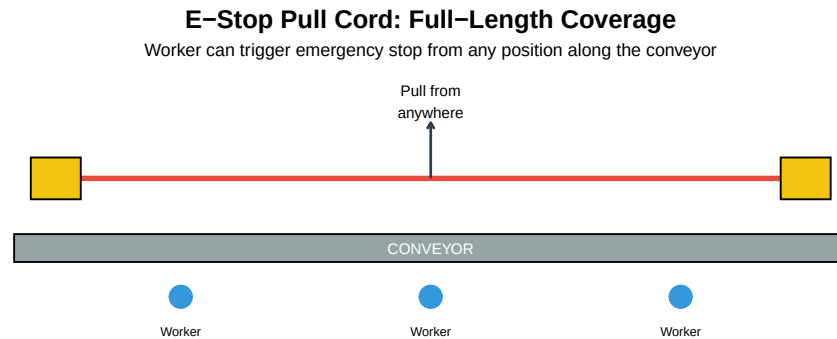


Figure 9.14: E-Stop Pull Cord System

Common E-Stop Mistakes

Mistake 1: Using E-Stop for normal stopping - E-Stops are for emergencies only - Normal stops should use a separate control - Frequent E-Stop use causes wear and desensitizes operators

Mistake 2: E-Stop that doesn't stop everything - All hazardous motion must stop - Auxiliary equipment (conveyors, feeders) often forgotten - Review entire system, not just the primary machine

Mistake 3: Automatic restart after E-Stop reset - Machine must NOT automatically restart when E-Stop is released - Separate "Start" action required - This prevents unexpected motion after reset

Mistake 4: Hidden or blocked E-Stops - E-Stops must be visible and accessible - Guard placement sometimes blocks access - Regular audits needed to ensure accessibility

9.9 Functional Safety Standards

Modern safety systems must meet specific **performance levels** to ensure reliability. Two main frameworks exist:

9.9.1 Safety Integrity Levels (SIL) - IEC 62061

Table 9.12: Safety Integrity Levels (PFH = Probability of Dangerous Failure per Hour)

SIL Level	PFH Range	Approximate Meaning	Typical Application
SIL 1	10 ⁻⁶ to <10 ⁻⁵	Dangerous failure less than once per 11 years	Low-risk tasks
SIL 2	10 ⁻⁵ to <10 ⁻⁴	Dangerous failure less than once per 114 years	Most industrial machinery
SIL 3	10 ⁻⁴ to <10 ⁻³	Dangerous failure less than once per 1,140 years	High-risk processes

9.9.2 Performance Levels (PL) - ISO 13849-1

Table 9.13: Performance Levels per ISO 13849-1

Performance Level	PFH Range	Risk Reduction	Equivalent SIL
PL a	10 ⁻⁶ to <10 ⁻⁵	Lowest	< SIL 1
PL b	3×10 ⁻⁶ to <10 ⁻⁵	Low	SIL 1
PL c	10 ⁻⁵ to <3×10 ⁻⁵	Medium	SIL 1
PL d	10 ⁻⁵ to <10 ⁻⁴	High	SIL 2
PL e	10 ⁻⁴ to <10 ⁻³	Highest	SIL 3

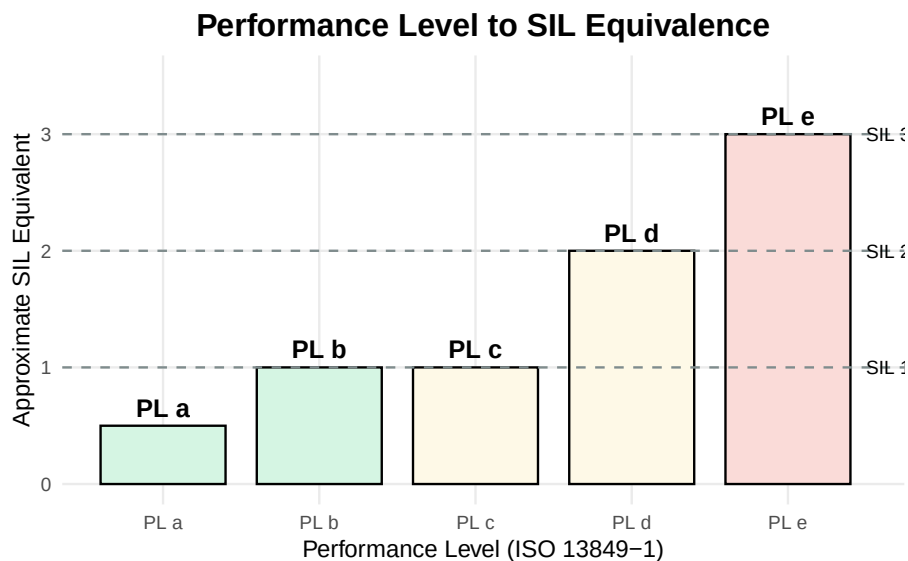


Figure 9.15: Relationship Between SIL and PL

Which Standard to Use: SIL or PL?

Use ISO 13849-1 (Performance Levels) when: - Designing safety functions for standard machinery - Using off-the-shelf safety components - Simpler systems with well-defined safety functions

Use IEC 62061 (SIL) when: - Complex programmable systems (safety PLCs) - Process industries - When customer or regulation specifies SIL

Note: Many modern systems use both standards, as they are harmonized and can be used together. The choice often depends on industry practice and customer requirements.

9.10 Risk Assessment Process

Before selecting any safety device, a **risk assessment** must be performed. This is required by ISO 12100.



Figure 9.16: Risk Assessment Process (ISO 12100)

9.10.1 Risk Estimation

Risk is typically estimated using three factors:

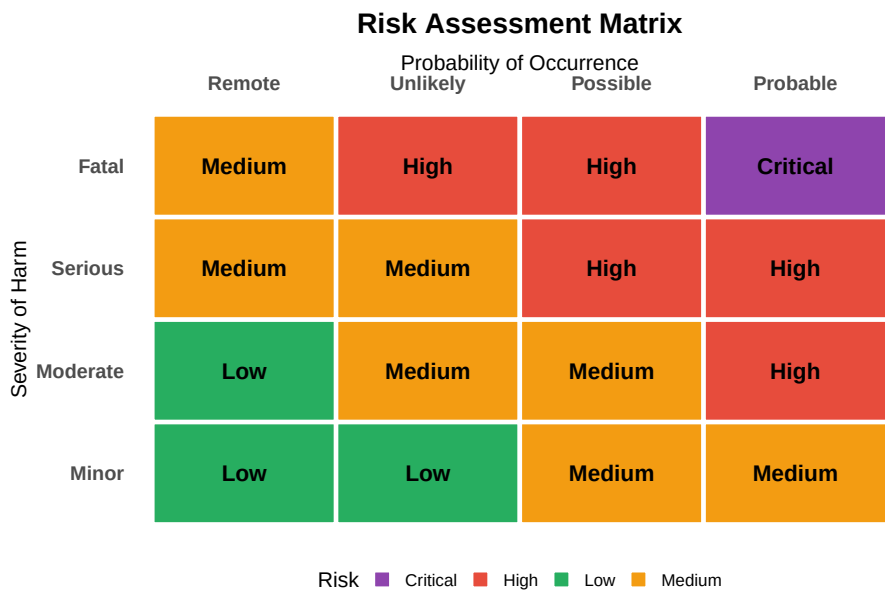


Figure 9.17: Risk Estimation Matrix

9.11 Summary

Table 9.14: Machine Guarding and Safety: Key Concepts Summary

Topic	Key Points	Critic
Hierarchy of Controls	Elimination > Substitution > Engineering > Administrative > PPE	Always
Physical Guarding	Fixed guards, interlocked gates, trapped key systems	Stop c
Presence Sensing	Light curtains, area scanners, pressure mats	Resolu
Control-Actuated Devices	Two-hand controls, enable/deadman switches	Anti-t
Collaborative Robots	PFL, speed/separation monitoring, hand guiding	Risk a
Safety Distance	$S = (K \times T) + C$ formula for device placement	Includ
E-Stop Systems	Red mushroom on yellow, self-latching, manual reset	E-Stop
Functional Safety	SIL (IEC 62061) and PL (ISO 13849-1) standards	Requir

9.12 Review Questions

Question 1: What are the five levels of the Hierarchy of Controls, and why is PPE the least effective?

The five levels are (from most to least effective):

1. **Elimination** — Physically remove the hazard
2. **Substitution** — Replace with something less hazardous
3. **Engineering Controls** — Isolate people from the hazard
4. **Administrative Controls** — Change how people work
5. **PPE** — Protect the worker with equipment

PPE is least effective because: - The hazard still exists - It depends on human compliance - Equipment can fail or be worn incorrectly - It's the last line of defense

Question 2: A light curtain has a response time of 12ms and resolution of 14mm. The machine takes 180ms to stop. Calculate the minimum safety distance for hand approach.

Given: - $K = 2000$ mm/s (hand approach, use higher value for safety) - $T = 0.012 + 0.180 = 0.192$ seconds - $C = 0$ mm (14mm resolution)

Calculation:

$$S = (K \times T) + C$$

$$S = (2000 \times 0.192) + 0$$

$$S = 384 \text{ mm}$$

The light curtain must be installed at least **384mm** (approximately 15 inches) from the hazard.

Question 3: Explain why a three-position enable switch stops the robot in both position 1 (released) and position 3 (fully pressed).

The three-position design accounts for two natural human reactions to danger:

Position 1 (Released): - When frightened, people often release what they're holding (panic response) - Robot stops immediately

Position 3 (Fully Pressed): - When startled, people often grip tighter (startle response) - Robot stops immediately

Position 2 (Middle): - Only a deliberate, controlled hold allows operation - Requires conscious effort to maintain - This ensures the operator is alert and in control

Question 4: What are the four methods of collaborative robot operation according to ISO 10218?

1. **Safety-Rated Monitored Stop**

- Robot stops when human enters workspace
- Resumes when human leaves
- Uses area scanners or light curtains
- 2. **Hand Guiding**
 - Operator physically guides robot
 - Force/torque sensors enable safe teaching
- 3. **Speed and Separation Monitoring**
 - Robot speed varies based on human proximity
 - Uses external sensors to track human position
- 4. **Power and Force Limiting (PFL)**
 - Robot limits contact force to safe levels
 - Built-in joint torque sensors
 - Most common cobot method

Question 5: What are the requirements for an E-Stop button according to IEC 60204-1?

Physical Requirements: - **Color:** Red - **Shape:** Mushroom head (palm or fist operated) - **Background:** Yellow (high contrast)

Functional Requirements: - **Self-latching:** Stays activated when pressed - **Manual reset:** Requires deliberate action to release - **Accessibility:** Within easy reach of all operators - **Wiring:** Normally closed contacts (fail-safe) - **Function:** Category 0 or Category 1 stop - **No automatic restart:** Separate start action required after reset

9.13 References

- ISO 12100:2010 — Safety of machinery — General principles for design — Risk assessment and risk reduction
- ISO 13849-1:2015 — Safety of machinery — Safety-related parts of control systems
- ISO 13855:2010 — Safety of machinery — Positioning of safeguards with respect to approach speeds
- ISO 10218-1/2:2011 — Robots and robotic devices — Safety requirements for industrial robots
- ISO/TS 15066:2016 — Robots and robotic devices — Collaborative robots
- IEC 60204-1:2016 — Safety of machinery — Electrical equipment of machines
- IEC 62061:2021 — Safety of machinery — Functional safety of safety-related control systems
- OSHA 29 CFR 1910.212 — General requirements for all machines
- ANSI/RIA TR R15.306 — Task-based risk assessment methodology

Chapter 10

Statistical Process Control (SPC)

10.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain the purpose and benefits of Statistical Process Control
- Distinguish between common cause and special cause variation
- Select the appropriate control chart for different data types
- Construct and interpret $\bar{X}$ -R, $\bar{X}$ -S, and Individual-MR charts
- Construct and interpret attribute control charts (p, np, c, u)
- Apply control chart interpretation rules (Western Electric rules)
- Calculate and interpret process capability indices (C_p , C_{pk} , P_p , P_{pk})
- Respond appropriately to out-of-control signals

“In God we trust; all others must bring data.” — W. Edwards Deming

10.2 Introduction to SPC

10.2.1 What is Statistical Process Control?

Statistical Process Control (SPC) is a method of quality control that uses statistical methods to monitor and control a process. It helps ensure that the process operates at its full potential to produce conforming product.

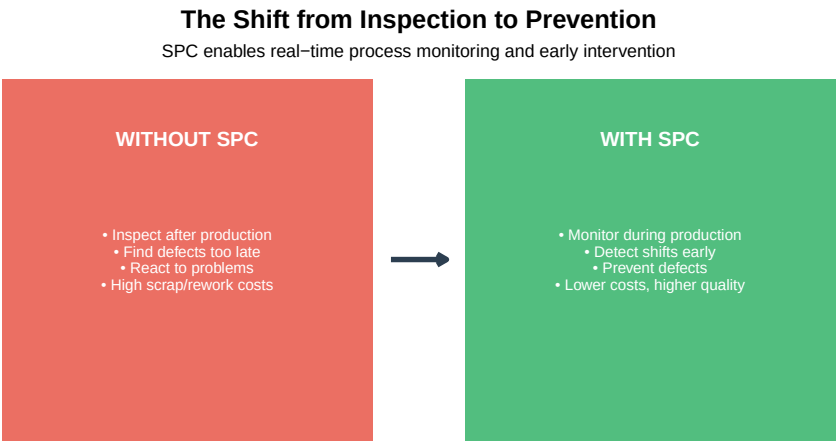


Figure 10.1: The SPC Philosophy: Prevention Through Monitoring

10.2.2 Brief History of SPC

Table 10.1: Evolution of Statistical Process Control

Year	Development	Significance
1920s	Walter Shewhart develops control charts at Bell Labs	Birth of modern quality control
1930s-40s	SPC used in WWII military production	Proved effectiveness in manufacturing
1950s	Deming teaches SPC to Japanese industry	Foundation of Japanese quality management
1980s	US automotive industry adopts SPC	Response to Japanese competition
1990s	SPC becomes requirement in ISO/QS-9000	Industry-wide standardization
2000s+	Real-time SPC, automated data collection	Integration with Industry 4.0

10.2.3 Why SPC Matters

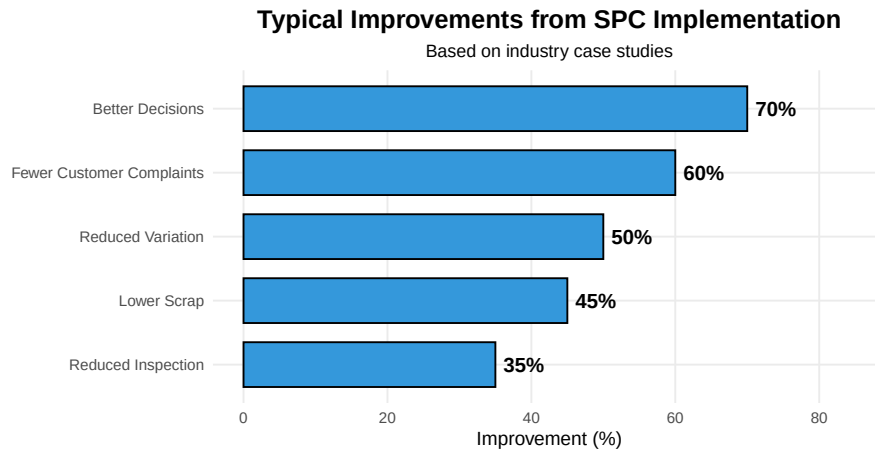


Figure 10.2: Benefits of SPC Implementation

10.3 Understanding Variation

All processes exhibit variation. No two products are ever exactly identical. The key insight of SPC is that variation comes from two fundamentally different sources, and we must respond to them differently.

10.3.1 Common Cause vs. Special Cause Variation

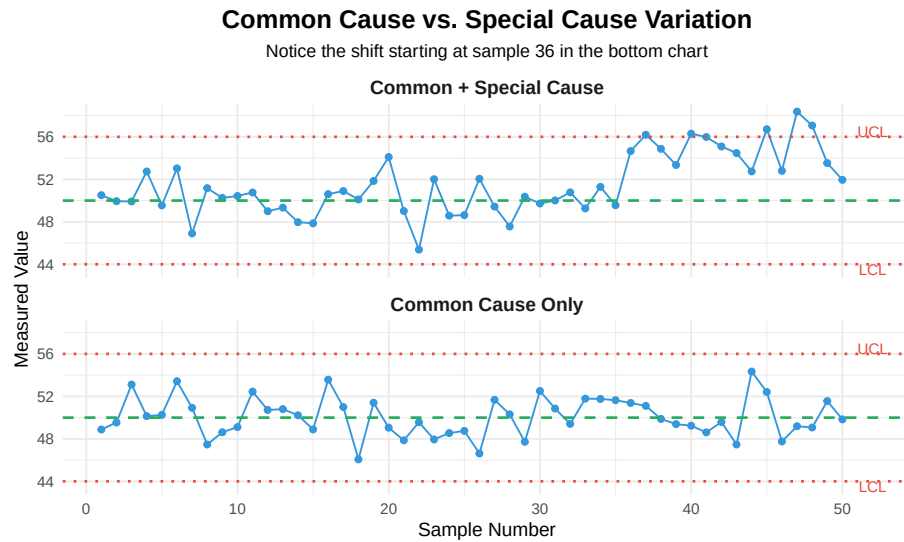


Figure 10.3: Two Types of Variation

Table 10.2: Common Cause vs. Special Cause Variation

Characteristic	Common Cause Variation	Special Cause Variation
Other Names	Random, chance, noise, inherent	Assignable
Source	Built into the process; many small factors	Specific, identifiable
Pattern	Random, stable, predictable pattern	Non-random
Predictability	Statistically predictable within limits	Unpredictable
Examples	Machine vibration, material variation, ambient temperature	Tool breakage, operator error
Action Required	Improve the system (management decision)	Find and eliminate the cause
Who Acts	Management — requires system change	Operators

10.3.2 The Two Mistakes in Process Control

Understanding variation prevents two costly mistakes:

Two Mistakes That Waste Resources and Reduce Quality

Control charts help us avoid both mistakes



Figure 10.4: The Two Fundamental Mistakes

The Funnel Experiment: Why Tampering Makes Things Worse

Deming's Funnel Experiment demonstrates why adjusting a stable process increases variation:

Setup: Drop a ball through a funnel onto a target. The ball lands near (but not exactly on) the target due to common cause variation.

Rule 1 (Correct): Leave the funnel alone. Variation remains constant.

Rule 2 (Tampering): After each drop, move the funnel to compensate for the last error. - Result: Variation DOUBLES!

Rule 3 (More Tampering): Move funnel to compensate relative to the target. - Result: Variation increases without bound!

Lesson: Adjusting a stable process based on individual deviations (common cause) actually makes the process worse. Only adjust when special causes are identified.

Real-World Example: An operator notices a part is 0.02mm below target and adjusts the machine. The next part is 0.03mm above target (the adjustment plus normal variation). They adjust again... and the process oscillates with increasing variation.

10.3.3 When is a Process “In Control”?

A process is **in statistical control** when:

1. Only common cause variation is present
2. All points fall within control limits
3. No non-random patterns exist

4. The process is stable and predictable

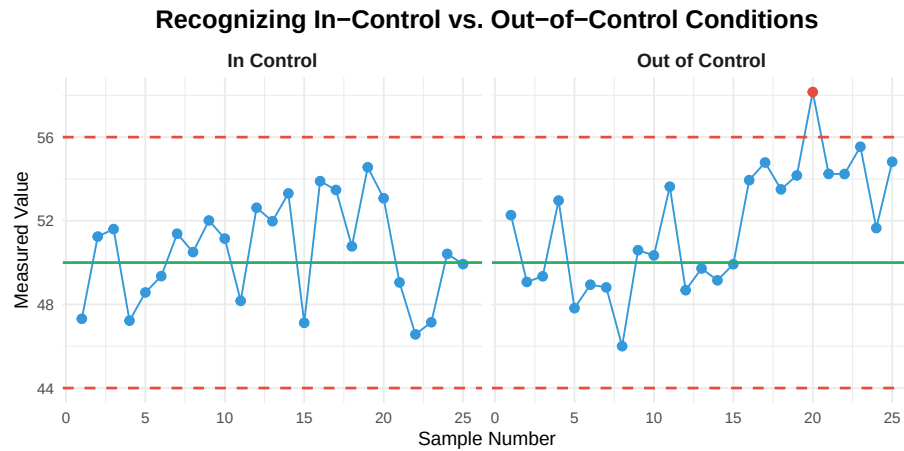


Figure 10.5: In-Control vs. Out-of-Control Process

10.4 Control Charts: The Foundation of SPC

10.4.1 What is a Control Chart?

A **control chart** is a graph that shows process data over time with statistically determined control limits. It provides a visual method for distinguishing between common and special cause variation.

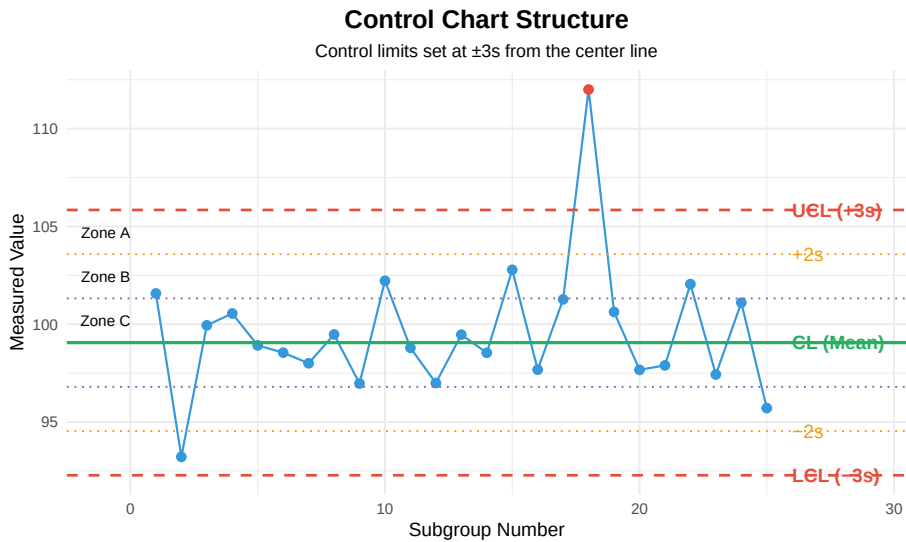


Figure 10.6: Anatomy of a Control Chart

10.4.2 Control Chart Components

Table 10.3: Control Chart Components

Component	Description	Purpose
Center Line (CL)	The process average; target for the process	Shows where process
Upper Control Limit (UCL)	Upper boundary of expected variation (+3)	Signals if process shift
Lower Control Limit (LCL)	Lower boundary of expected variation (-3)	Signals if process shift
Data Points	Individual or subgroup statistics plotted over time	Visual representation
Zones A, B, C	Regions between ± 1 , ± 2 , and ± 3 for pattern analysis	Used for detecting trends

10.4.3 Why ± 3 Sigma?

Control limits are set at ± 3 **standard deviations** from the center line. This is not arbitrary:

- For a normal distribution, 99.73% of data falls within ± 3
- Only 0.27% (about 3 in 1000) will fall outside by chance
- This balances the risk of both mistakes (false alarms vs. missed signals)

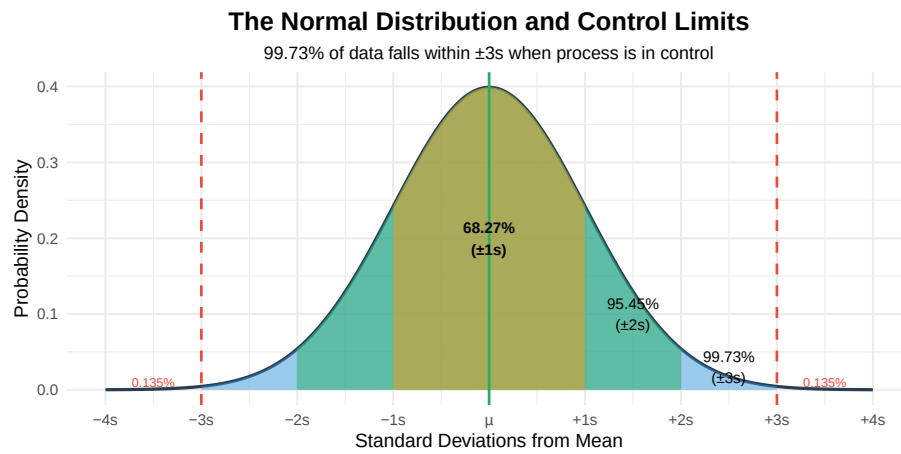


Figure 10.7: Normal Distribution and Control Limits

10.4.4 Choosing the Right Control Chart

The type of control chart depends on the **type of data** being collected:

Selecting the Right Control Chart

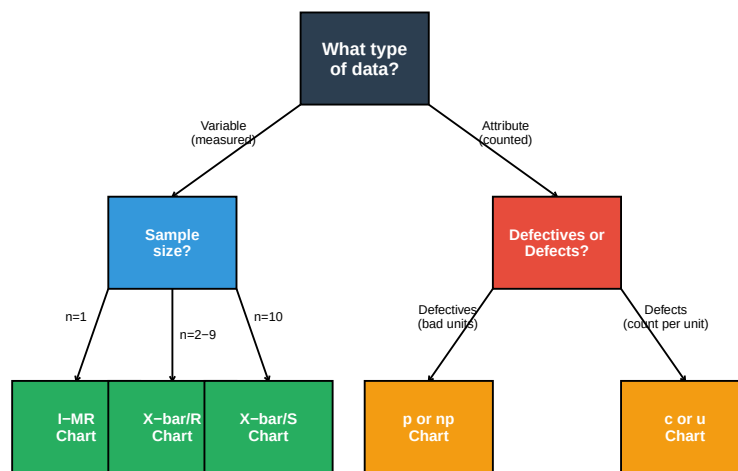


Figure 10.8: Control Chart Selection Flowchart

Table 10.4: Control Chart Summary

Chart Type	Data Type	Sample Size	What It Charts	Typical
X-bar / R	Variable	2-9 per subgroup	Subgroup mean and range	Production
X-bar / S	Variable	10 per subgroup	Subgroup mean and std dev	Large sub
I-MR	Variable	1 (individuals)	Individual values and moving range	Continuo
p chart	Attribute	Variable or constant	Proportion defective	Pass/fail
np chart	Attribute	Constant	Number defective	Pass/fail
c chart	Attribute	Constant	Number of defects	Defects p
u chart	Attribute	Variable	Defects per unit	Defects p

Question: Variable vs. Attribute Data - What's the Difference?

Variable Data (Measured): - Can take any value within a range - Measured on a continuous scale - Examples: length, weight, temperature, voltage, pressure - More information per measurement - Smaller sample sizes needed

Attribute Data (Counted): - Discrete categories (good/bad, pass/fail) - Counted, not measured - Examples: number of defects, number of rejects, yes/no - Less information per observation - Larger sample sizes needed

When to Use Each: - Use variable data when you CAN measure (more powerful) - Use attribute data when you can only classify or count - Sometimes you convert variable to attribute (e.g., pass/fail based on tolerance)

10.5 Variable Control Charts

10.5.1 X-bar and R Chart

The **X-bar and R chart** is the most common control chart for variable data. It tracks both the process average (X-bar) and the process spread (Range).

10.5.1.1 Constructing an X-bar/R Chart

Step 1: Collect data in subgroups (typically 4-5 samples)

```
# Example: Measuring shaft diameter (mm)
# 25 subgroups of 5 measurements each

set.seed(100)
n_subgroups <- 25
n_samples <- 5
```

```
# Generate data
data <- matrix(rnorm(n_subgroups * n_samples, mean = 25.00, sd = 0.05),
              nrow = n_subgroups, ncol = n_samples)

# Add a special cause (shift) at subgroup 18
data[18, ] <- data[18, ] + 0.12

# Calculate statistics for each subgroup
xbar <- apply(data, 1, mean)
R <- apply(data, 1, function(x) max(x) - min(x))

cat("First 5 subgroups:\n")
print(round(data[1:5, ], 4))
cat("\nX-bar values:", round(xbar[1:5], 4))
cat("\nRange values:", round(R[1:5], 4))
```

First 5 subgroups:

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	24.9749	24.9781	24.9776	25.0006	24.9834
[2,]	25.0066	24.9640	24.9131	24.9456	25.0682
[3,]	24.9961	25.0115	25.0089	25.0135	24.9765
[4,]	25.0443	24.9421	25.0949	25.0504	25.0421
[5,]	25.0058	25.0124	24.8864	24.8963	24.9271

X-bar values: 24.9829 24.9795 25.0013 25.0348 24.9456

Range values: 0.0257 0.1551 0.037 0.1528 0.126

Step 2: Calculate control limits

```
# Control chart constants for n=5
A2 <- 0.577
D3 <- 0      # D3 = 0 for n < 7
D4 <- 2.114

# Calculate grand mean and average range
X_double_bar <- mean(xbar)
R_bar <- mean(R)

cat("Grand Mean (X-double-bar):", round(X_double_bar, 4), "mm\n")
cat("Average Range (R-bar):", round(R_bar, 4), "mm\n\n")

# X-bar chart limits
UCL_xbar <- X_double_bar + A2 * R_bar
LCL_xbar <- X_double_bar - A2 * R_bar

cat("X-bar Chart:\n")
```

```

cat("  UCL =", round(UCL_xbar, 4), "mm\n")
cat("  CL  =", round(X_double_bar, 4), "mm\n")
cat("  LCL =", round(LCL_xbar, 4), "mm\n\n")

# R chart limits
UCL_R <- D4 * R_bar
LCL_R <- D3 * R_bar

cat("R Chart:\n")
cat("  UCL =", round(UCL_R, 4), "mm\n")
cat("  CL  =", round(R_bar, 4), "mm\n")
cat("  LCL =", round(LCL_R, 4), "mm\n")

```

Grand Mean (X-double-bar): 25.004 mm

Average Range (R-bar): 0.1023 mm

X-bar Chart:

UCL = 25.0631 mm

CL = 25.004 mm

LCL = 24.945 mm

R Chart:

UCL = 0.2164 mm

CL = 0.1023 mm

LCL = 0 mm

Step 3: Plot the charts

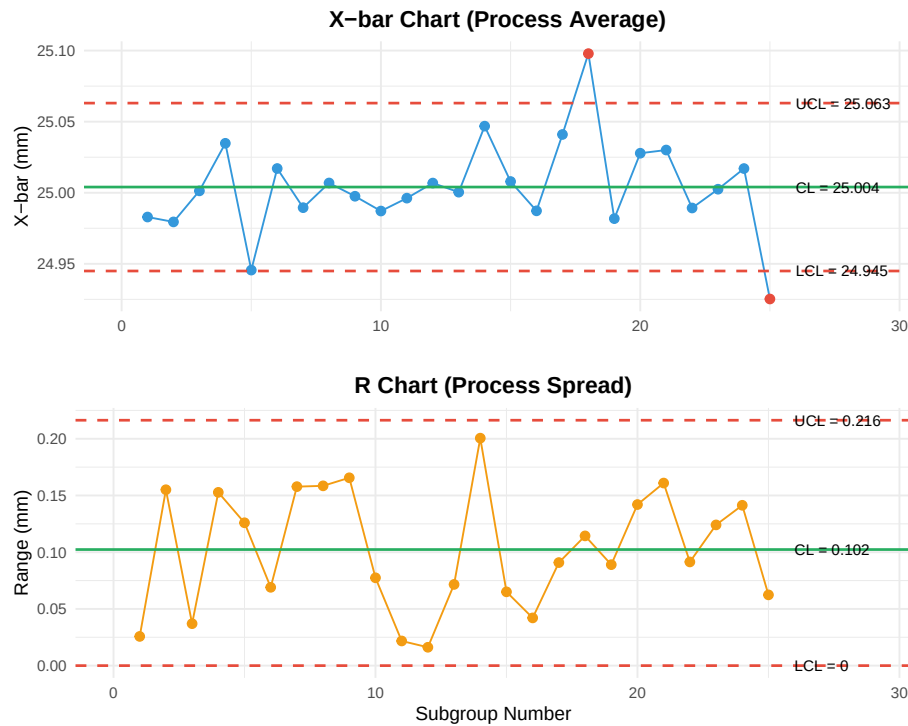


Figure 10.9: X-bar and R Control Charts

Interpretation: Subgroup 18 shows a point above the UCL on the X-bar chart, indicating a special cause. Investigation reveals the cause and corrective action is taken.

10.5.2 Control Chart Constants

Table 10.5: Control Chart Constants for X-bar/R Charts

n	A2	D3	D4	d2
2	1.880	0.000	3.267	1.128
3	1.023	0.000	2.574	1.693
4	0.729	0.000	2.282	2.059
5	0.577	0.000	2.114	2.326
6	0.483	0.000	2.004	2.534
7	0.419	0.076	1.924	2.704
8	0.373	0.136	1.864	2.847
9	0.337	0.184	1.816	2.970

10	0.308	0.223	1.777	3.078
----	-------	-------	-------	-------

10.5.3 Individual and Moving Range (I-MR) Chart

When sample size is $n = 1$ (one measurement per time period), use the **I-MR chart**.

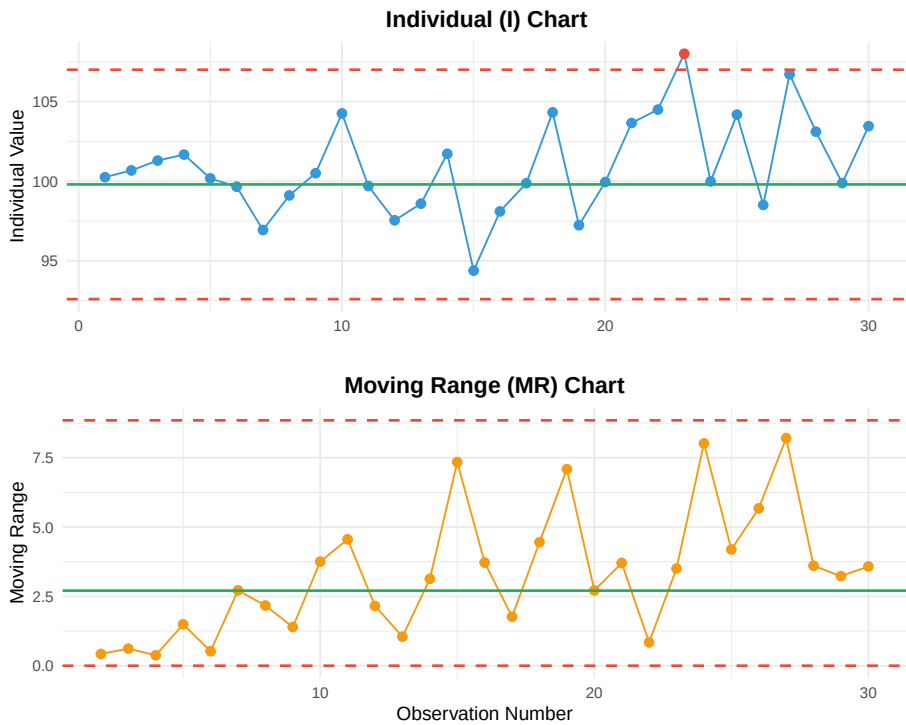


Figure 10.10: Individual and Moving Range Chart

I-MR Chart Formulas:

Chart	Center Line	UCL	LCL
I Chart	$\bar{X}$	$\bar{X} + 2.66\bar{MR}$	$\bar{X} - 2.66\bar{MR}$
MR Chart	$\bar{MR}$	$3.267 \times \bar{MR}$	0

10.6 Attribute Control Charts

10.6.1 p Chart (Proportion Defective)

The **p chart** tracks the proportion of defective units when sample sizes may vary.

```
# Example: Daily inspection data
# Varying sample sizes (different production volumes)

set.seed(300)
days <- 20
sample_sizes <- sample(80:120, days, replace = TRUE)
defectives <- rbinom(days, sample_sizes, prob = 0.05)

# Add a special cause on day 15
defectives[15] <- rbinom(1, sample_sizes[15], prob = 0.12)

# Calculate proportion
p <- defectives / sample_sizes

# Calculate p-bar (weighted average)
p_bar <- sum(defectives) / sum(sample_sizes)

cat("p-bar (average proportion defective):", round(p_bar, 4), "\n\n")

# Calculate control limits (vary with sample size)
UCL_p <- p_bar + 3 * sqrt(p_bar * (1 - p_bar) / sample_sizes)
LCL_p <- pmax(0, p_bar - 3 * sqrt(p_bar * (1 - p_bar) / sample_sizes))

cat("Sample of control limits (varying with n):\n")
cat("Day 1: UCL =", round(UCL_p[1], 4), ", LCL =", round(LCL_p[1], 4), "(n =", sample_sizes[1], ")\n")
cat("Day 5: UCL =", round(UCL_p[5], 4), ", LCL =", round(LCL_p[5], 4), "(n =", sample_sizes[5], ")\n")
```

p-bar (average proportion defective): 0.0493

Sample of control limits (varying with n):

Day 1: UCL = 0.1167 , LCL = 0 (n = 93)

Day 5: UCL = 0.1143 , LCL = 0 (n = 100)

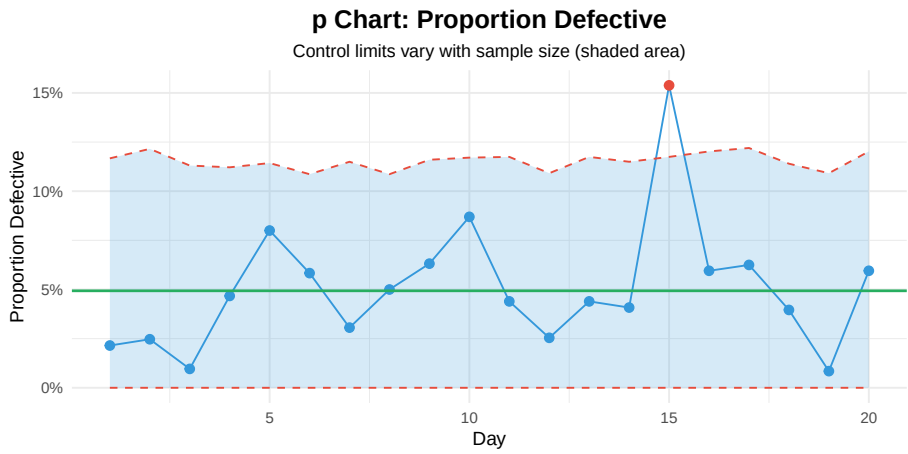


Figure 10.11: p Chart for Proportion Defective

10.6.2 c Chart (Count of Defects)

The **c chart** tracks the number of defects when the sample size (area of opportunity) is constant.

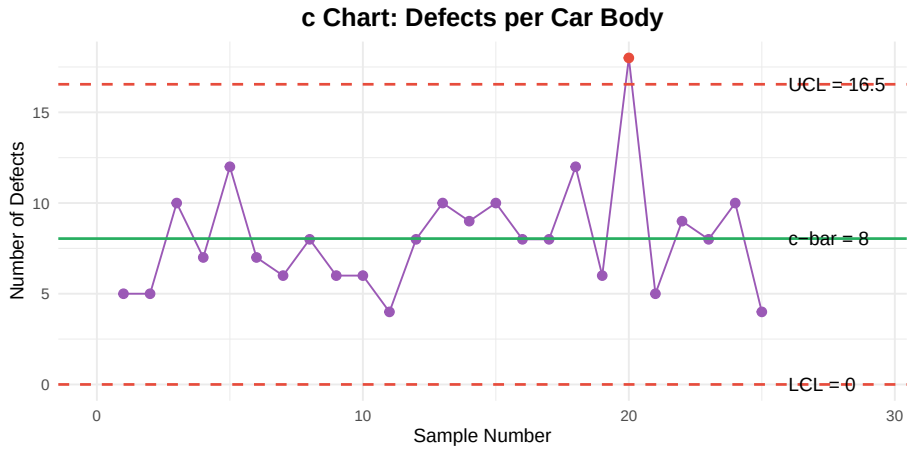


Figure 10.12: c Chart for Defects per Unit

Table 10.6: Attribute Control C

Chart	Center Line	UCL
p chart	$\bar{p} = \frac{\sum d}{\sum n}$	$\bar{p} + 3\sqrt{\frac{\bar{p}(1-\bar{p})}{n}}$

np chart	$\bar{np} = \frac{\sum d}{k}$	$\bar{np} + 3\sqrt{\bar{np}(1-\bar{p})}$
c chart	$\bar{c} = \frac{\sum c}{k}$	$\bar{c} + 3\sqrt{\bar{c}}$
u chart	$\bar{u} = \frac{\sum c}{\sum n}$	$\bar{u} + 3\sqrt{\frac{\bar{u}}{n}}$

10.7 Interpreting Control Charts

10.7.1 Out-of-Control Signals (Western Electric Rules)

A point doesn't have to be outside the control limits to indicate a problem. The **Western Electric Rules** identify non-random patterns:

Table 10.7: Western Electric Rules for Control Chart Interpretation

Rule	Pattern	Description
Rule 1	**Point beyond control limit**	One point above UCL or below LCL
Rule 2	**Zone A: 2 of 3 consecutive points**	2 out of 3 consecutive points in Zone A ($>2\sigma$)
Rule 3	**Zone B: 4 of 5 consecutive points**	4 out of 5 consecutive points in Zone B ($>1\sigma$)
Rule 4	**8 consecutive points on one side**	8 consecutive points above or below the center line

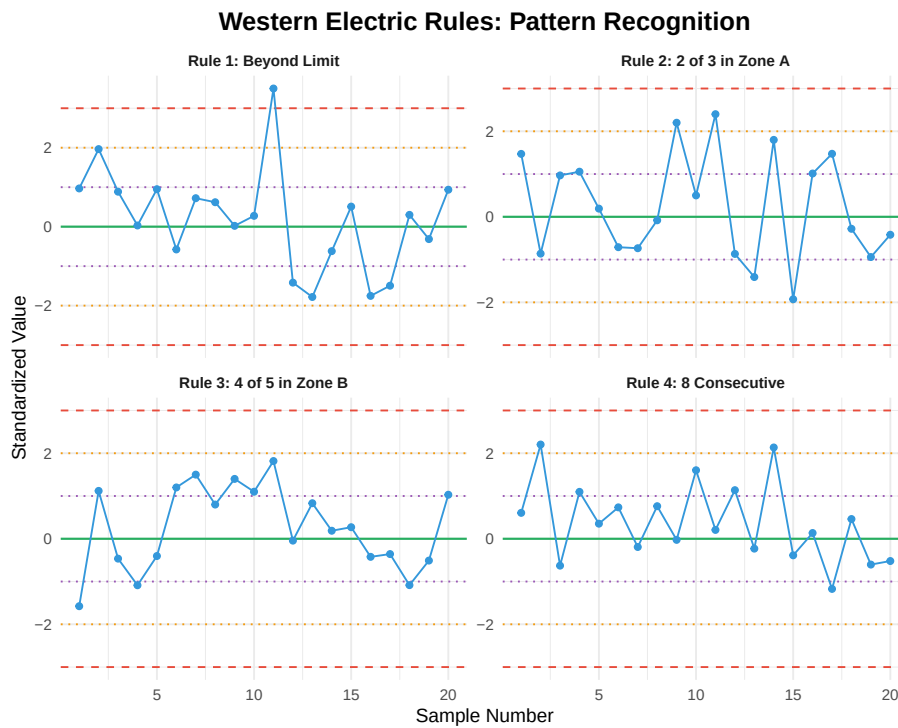


Figure 10.13: Visual Examples of Western Electric Rules

10.7.2 Additional Patterns to Watch For

Table 10.8: Additional Control Chart Patterns

Pattern	Description	Possible Causes
Trend	6 or more consecutive points steadily increasing or decreasing	Tool wear, temperature
Cycle	Repeating high-low pattern	Shift changes, batch effect
Stratification	Points consistently near center line (too little variation)	Incorrect limits, data manipulation
Mixture	Points avoiding center line (bimodal distribution)	Two machines, two operators

Interactive Exercise: Identify the Out-of-Control Condition

Look at this sequence of points (standardized values):

0.5, 0.8, 1.2, 0.9, 1.4, 1.6, 1.3, 1.8, -0.2, 0.3

Questions:

1. Are any points beyond ± 3 ?
2. Is there a Rule 4 violation (8 consecutive on one side)?

3. What pattern do you see?

Answers:

1. No — all points are within ± 3
2. **YES** — Points 1-8 are all positive (above center line)
3. This is a **Rule 4 violation**: 8 consecutive points above the center line indicates a likely process shift upward

Action: Investigate what changed — new material lot? Different operator? Equipment adjustment?

10.8 Process Capability

10.8.1 What is Process Capability?

Process capability compares the process performance to the specification limits. It answers: “Can this process consistently produce parts within tolerance?”

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

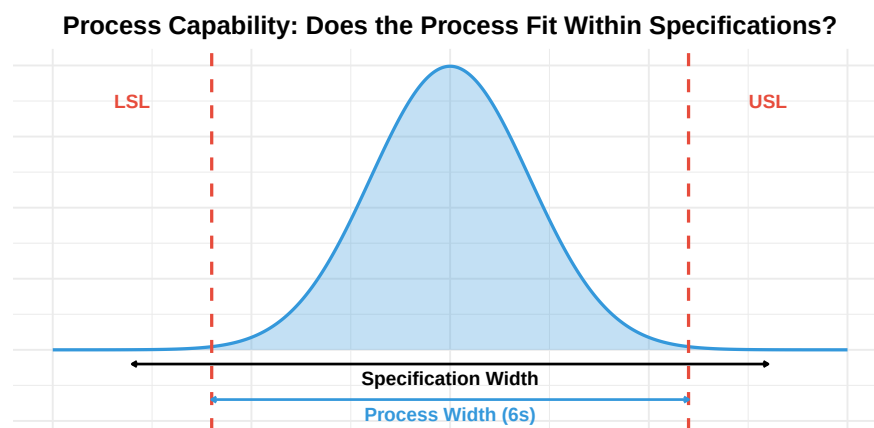


Figure 10.14: Process Capability: Comparing Process to Specifications

10.8.2 Capability Indices

Table 10.9: Process Capability

Index	Formula
-------	---------

Cp	$\frac{USL - LSL}{6\sigma}$	Potential
Cpk	$\min\left(\frac{USL - \bar{X}}{3\sigma}, \frac{\bar{X} - LSL}{3\sigma}\right)$	Actual ca
Pp	$\frac{USL - LSL}{6s}$	Overall p
Ppk	$\min\left(\frac{USL - \bar{X}}{3s}, \frac{\bar{X} - LSL}{3s}\right)$	Overall p

10.8.3 Cp vs. Cpk: Why Both Matter

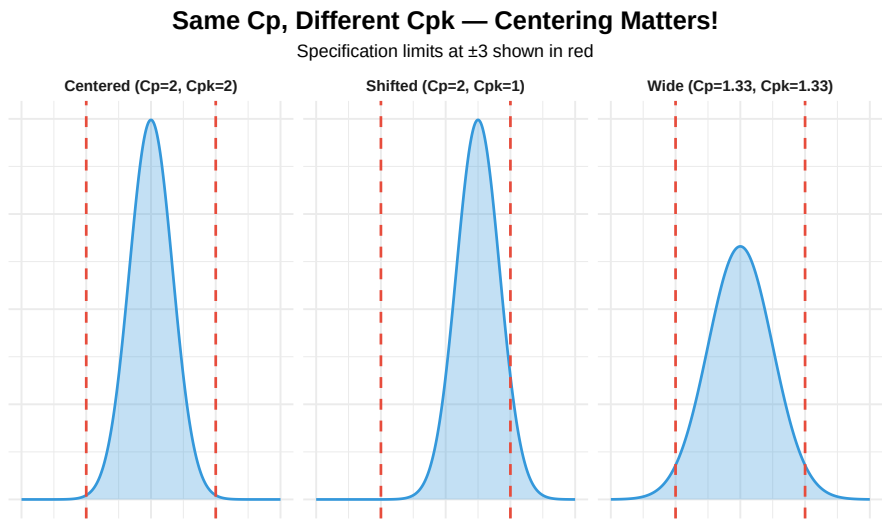


Figure 10.15: Cp vs. Cpk: The Importance of Centering

10.8.4 Capability Example Calculation

```
# Example: Shaft diameter specification 25.00 ± 0.15 mm
# Process data from 25 subgroups of 5

USL <- 25.15
LSL <- 24.85
Target <- 25.00

# Using earlier X-bar/R data
X_bar_cap <- mean(xbar)
R_bar_cap <- mean(R)

# Estimate sigma using R-bar/d2 (d2 = 2.326 for n=5)
d2 <- 2.326
sigma_within <- R_bar_cap / d2
```

```

cat("Process Statistics:\n")
cat("  Mean (X-bar):", round(X_bar_cap, 4), "mm\n")
cat("  Sigma (estimated):", round(sigma_within, 4), "mm\n\n")

# Calculate Cp
Cp <- (USL - LSL) / (6 * sigma_within)
cat("Cp =", round(Cp, 2), "\n")

# Calculate Cpk
Cpu <- (USL - X_bar_cap) / (3 * sigma_within)
Cpl <- (X_bar_cap - LSL) / (3 * sigma_within)
Cpk <- min(Cpu, Cpl)

cat("Cpu =", round(Cpu, 2), "\n")
cat("Cpl =", round(Cpl, 2), "\n")
cat("Cpk =", round(Cpk, 2), "\n\n")

# Interpretation
cat("Interpretation:\n")
if (Cpk >= 1.33) {
  cat("Process is CAPABLE (Cpk >= 1.33)\n")
} else if (Cpk >= 1.00) {
  cat("Process is marginally CAPABLE (1.00 <= Cpk < 1.33)\n")
} else {
  cat("Process is NOT CAPABLE (Cpk < 1.00)\n")
}

```

```

Process Statistics:
  Mean (X-bar): 25.004 mm
  Sigma (estimated): 0.044 mm

```

```

Cp = 1.14
Cpu = 1.11
Cpl = 1.17
Cpk = 1.11

```

```

Interpretation:
Process is marginally CAPABLE (1.00 <= Cpk < 1.33)

```

10.8.5 Capability Index Interpretation

Table 10.10: Capability Index Interpretation Guide

Cpk Value	Sigma Level	PPM Defective	Interpretation
< 1.00	< 3	> 2,700	Not capable — improvement required

1.00 - 1.33	3 - 4	2,700 - 64	Marginally capable — monitor closely	Consumer p
1.33 - 1.67	4 - 5	64 - 0.6	Capable — meets typical requirements	General ma
1.67 - 2.00	5 - 6	0.6 - 0.002	Good capability — automotive/aerospace	Automotive
> 2.00	> 6	< 0.002	Excellent — Six Sigma level	Aerospace, r

Question: When to use Cp/Cpk vs. Pp/Ppk?

Cp/Cpk (Process Capability): - Uses within-subgroup variation (estimated from $R\text{-bar}/d2$) - Represents the POTENTIAL capability if process is in control - Use for ongoing process monitoring - Requires process to be in statistical control

Pp/Ppk (Process Performance): - Uses overall standard deviation (s) of all data - Represents ACTUAL performance including all variation - Use for initial capability studies or when process stability is unknown - Includes both common and special cause variation

Rule of Thumb: - If $Pp \geq Cp$ and $Ppk \geq Cpk \rightarrow$ Process is stable - If $Pp < Cp$ or $Ppk < Cpk \rightarrow$ Process has special cause variation (not in control)

10.9 Responding to Out-of-Control Signals

10.9.1 The Response Process

Out-of-Control Response Procedure

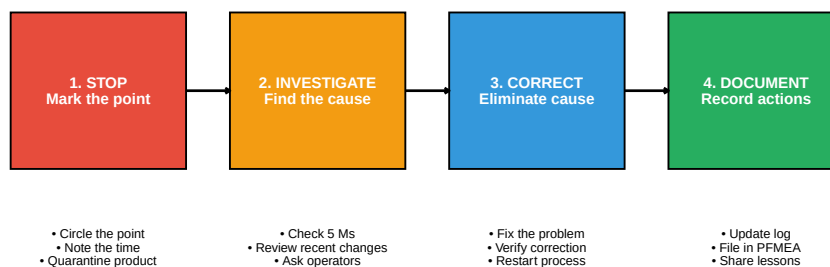


Figure 10.16: Responding to Out-of-Control Conditions

10.9.2 Investigation Checklist (5 Ms + E)

Table 10.11: Investigation Checklist: 5 Ms + E

Category	Questions to Ask
Man	Different operator? New employee? Fatigue? Training issue?
Machine	Equipment malfunction? Tool wear? Maintenance due? Setup change?
Material	New lot? Different supplier? Out-of-spec material?
Method	Procedure changed? Parameter drift? Wrong program?
Measurement	Gauge drift? Calibration due? Different inspector?
Environment	Temperature change? Humidity? Vibration? Contamination?

10.10 SPC Implementation

10.10.1 Steps to Implement SPC

Table 10.12: SPC Implementation Steps

Step	Action	Key Considerations
1	Select the process and characteristic	Start with critical/problem processes
2	Determine measurement system adequacy (MSA)	Gauge R&R must be acceptable (< 10%)
3	Collect initial data (25+ subgroups)	Process must be running normally
4	Calculate trial control limits	Limits based on actual process, not specification
5	Identify and remove special causes	Investigate each out-of-control point
6	Recalculate limits with stable data	These become the standard limits
7	Implement ongoing monitoring	Train operators, establish reaction plan
8	Review and improve continuously	Update limits when process improves

10.10.2 Common SPC Mistakes

Table 10.13: Common SPC Mistakes and Why They Are Wrong

Mistake	Why It's Wrong
Using specification limits as control limits	Control limits are based on process performance, not specification
Changing limits too frequently	Limits should only change when process fundamentally changes
Ignoring the R chart	R chart often signals problems before X-bar chart
Over-adjusting (tampering)	Adjusting for common cause variation increases variation
Not acting on signals	Defeats the purpose of SPC

10.11 Summary

Table 10.14: SPC Key Concepts Summary

Topic	Key Points
Variation	Common cause (inherent) vs. special cause (assignable); respond differently
Control Charts	Graph with UCL/LCL at ± 3 ; distinguishes between variation types
Variable Charts	X-bar/R (n=2-9), X-bar/S (n 10), I-MR (n=1)
Attribute Charts	p (proportion), np (count defectives), c (defects), u (defects/unit)
Interpretation	Western Electric rules: beyond limits, 2 of 3 in Zone A, 4 of 5 in Zone B, 8 consecutive
Capability	Cp (potential), Cpk (actual); 1.33 is capable; 1.67 for automotive

10.12 Review Questions

10.12.1 Conceptual Questions

Question 1: What is the difference between common cause and special cause variation?

Common Cause Variation: - Inherent to the process - Always present - Random and predictable - Many small factors - System-level improvement required - Examples: machine vibration, material variation, ambient temperature

Special Cause Variation: - External to normal process - Intermittent or new - Non-random patterns - Specific assignable cause - Local action to eliminate - Examples: tool breakage, new operator, bad material lot

Key Point: Only act on special causes. Attempting to adjust for common cause variation (tampering) makes the process worse.

Question 2: Why are control limits set at ± 3 sigma rather than ± 2 sigma?

At ± 3 : - 99.73% of data falls within limits (when in control) - Only 0.27% false alarm rate (about 3 per 1000 points) - Good balance between detecting real problems and avoiding false alarms

If we used ± 2 : - 95.45% of data within limits - 4.55% false alarm rate (about 45 per 1000 points!) - Too many false alarms $\rightarrow$ operators ignore the chart

If we used ± 4 : - 99.99% within limits - Almost no false alarms - But we'd miss real problems (low sensitivity)

The ± 3 choice balances: - Risk of false alarm (treating common cause as special) - Risk of missed signal (treating special cause as common)

Question 3: When would you use an I-MR chart instead of an X-bar/R chart?

Use I-MR Chart When:

1. **Sample size is $n = 1$** — only one measurement per time period
2. **Destructive testing** — can't afford multiple samples
3. **Expensive measurement** — time or cost prohibitive
4. **Continuous process** — each reading is independent (batch process)
5. **Long intervals** — measurements far apart in time
6. **Homogeneous batches** — within-batch variation is negligible

Examples: - Daily chemical batch analysis - Weekly calibration check - Each lot incoming inspection (one sample per lot) - Continuous process temperature readings

Caution: I-MR charts are less sensitive than X-bar charts because averaging in subgroups reduces noise.

10.12.2 Calculation Questions

Question 4: Calculate control limits for an X-bar/R chart given the following data.

Given: - 20 subgroups of $n = 4$ measurements - Grand mean (X-double-bar) = 50.25 - Average range (R-bar) = 2.40 - Constants for $n = 4$: $A_2 = 0.729$, $D_3 = 0$, $D_4 = 2.282$

Calculate UCL and LCL for both charts.

Solution:

X-bar Chart:

$$UCL = \bar{\bar{X}} + A_2 \times \bar{R}$$

$$UCL = 50.25 + 0.729 \times 2.40 = 50.25 + 1.75 = 52.00$$

$$LCL = \bar{\bar{X}} - A_2 \times \bar{R}$$

$$LCL = 50.25 - 0.729 \times 2.40 = 50.25 - 1.75 = 48.50$$

$$CL = 50.25$$

R Chart:

$$UCL = D_4 \times \bar{R} = 2.282 \times 2.40 = 5.48$$

$$LCL = D_3 \times \bar{R} = 0 \times 2.40 = 0$$

$$CL = 2.40$$

Question 5: Calculate C_p and C_{pk} for a process with the following data.

Given: - Specification: 100 ± 5 ($USL = 105$, $LSL = 95$) - Process mean = 102
- Process standard deviation (σ) = 1.5

Solution:

Cp (Potential Capability):

$$C_p = (USL - LSL) / (6 \sigma)$$

$$C_p = (105 - 95) / (6 \times 1.5)$$

$$C_p = 10 / 9 = 1.11$$

Cpk (Actual Capability):

$$C_{pu} = (USL - \bar{X}) / (3 \sigma) = (105 - 102) / (3 \times 1.5) = 3 / 4.5 = 0.67$$

$$C_{pl} = (\bar{X} - LSL) / (3 \sigma) = (102 - 95) / (3 \times 1.5) = 7 / 4.5 = 1.56$$

$$C_{pk} = \min(C_{pu}, C_{pl}) = \min(0.67, 1.56) = 0.67$$

Interpretation: - $C_p = 1.11$ suggests process spread could fit within specs IF centered - $C_{pk} = 0.67$ shows process is NOT capable because it's shifted toward USL - Action: Center the process closer to 100 (target)

Question 6: A p chart has $\bar{p} = 0.04$ and sample size $n = 200$. Calculate control limits.

Solution:

Center Line:

$$CL = \bar{p} = 0.04 \text{ (4\% defective)}$$

Upper Control Limit:

$$UCL = \bar{p} + 3 \times \sqrt{\bar{p} \times (1 - \bar{p}) / n}$$

$$UCL = 0.04 + 3 \times \sqrt{(0.04 \times 0.96) / 200}$$

$$UCL = 0.04 + 3 \times \sqrt{0.000192}$$

$$UCL = 0.04 + 3 \times 0.01386$$

$$UCL = 0.04 + 0.0416 = 0.0816 \text{ (8.16\%)}$$

Lower Control Limit:

$$LCL = \bar{p} - 3 \times \sqrt{\bar{p} \times (1 - \bar{p}) / n}$$

$$LCL = 0.04 - 0.0416 = -0.0016$$

Since LCL cannot be negative: $LCL = 0$

Final Limits: - $UCL = 8.16\%$ - $CL = 4.00\%$ - $LCL = 0\%$

10.12.3 Application Questions

Question 7: You observe 8 consecutive points above the center line but all within control limits. What does this indicate?

This is a Rule 4 violation (Western Electric Rules).

Rule: If R chart is out of control, fix that FIRST before interpreting the X-bar chart.

10.13 References

- Montgomery, D.C. (2019). *Introduction to Statistical Quality Control* (8th ed.). Wiley.
 - Wheeler, D.J., & Chambers, D.S. (1992). *Understanding Statistical Process Control* (2nd ed.). SPC Press.
 - AIAG. (2005). *Statistical Process Control (SPC) Reference Manual* (2nd ed.).
 - Western Electric. (1956). *Statistical Quality Control Handbook*. AT&T.
 - Deming, W.E. (1986). *Out of the Crisis*. MIT Press.
 - Shewhart, W.A. (1931). *Economic Control of Quality of Manufactured Product*. Van Nostrand.
 - ASTM E2587. Standard Practice for Use of Control Charts in Statistical Process Control.
-

Chapter 11

Measurement Systems Analysis

11.1 Learning Objectives

After completing this chapter, you will be able to:

1. Explain the importance of measurement system quality in process control
 2. Identify the components of measurement system variation
 3. Define and calculate accuracy, bias, linearity, stability, repeatability, and reproducibility
 4. Conduct and interpret a Gauge R&R study using both the Range and ANOVA methods
 5. Determine measurement system acceptability using %GRR and ndc criteria
 6. Apply attribute measurement system analysis for pass/fail decisions
 7. Recognize common sources of measurement error and implement improvements
 8. Understand the relationship between MSA and SPC effectiveness
-

11.2 Introduction to Measurement Systems Analysis

Measurement Systems Analysis (MSA) is the science of evaluating the quality of measurement processes. Before we can trust our data for process

control, quality decisions, or capability studies, we must first verify that our measurement system is capable of providing reliable results.

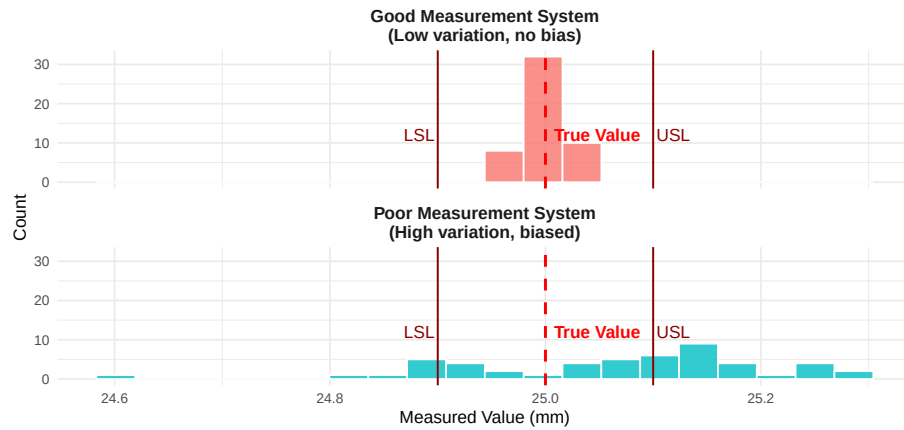
11.2.1 Why MSA Matters

“You can’t improve what you can’t measure - but you also can’t improve what you measure incorrectly.”

Consider this scenario:

The Same Parts Measured by Two Different Systems

Poor measurement systems lead to incorrect accept/reject decisions



11.2.2 The Cost of Poor Measurement

Table 11.1: Business Impact of Poor Measurement System

Impact	Description
False Accepts (Type II Error)	Bad parts passed as good; reach customer
False Rejects (Type I Error)	Good parts rejected as bad; scrapped or reworked
Process Adjustment Errors	Adjusting process based on measurement noise, not real changes
Capability Misassessment	Cp/Cpk calculations are wrong; true capability unknown
Customer Complaints	Inconsistent quality due to measurement-based decisions

11.2.3 MSA in Industry Standards

MSA is required by multiple quality standards:

- **IATF 16949** (Automotive): Requires MSA for all measurement systems referenced in control plans
- **AS9100** (Aerospace): Requires demonstrated measurement capability
- **ISO 9001**: Requires monitoring and measurement resources to be suitable

- **FDA 21 CFR Part 820** (Medical Devices): Requires validated measurement equipment

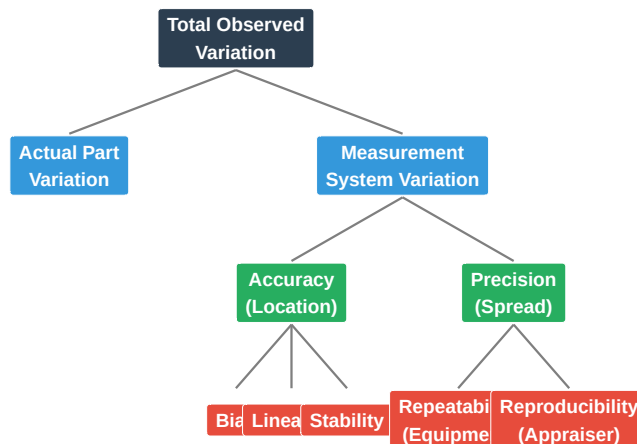
11.3 Components of Measurement Variation

Total measurement variation comes from multiple sources. Understanding these components is essential for improvement.

11.3.1 The Measurement System Model

Components of Measurement System Variation

Total variation = Part variation + Measurement system variation



11.3.2 Accuracy vs. Precision

These are often confused but are fundamentally different concepts:

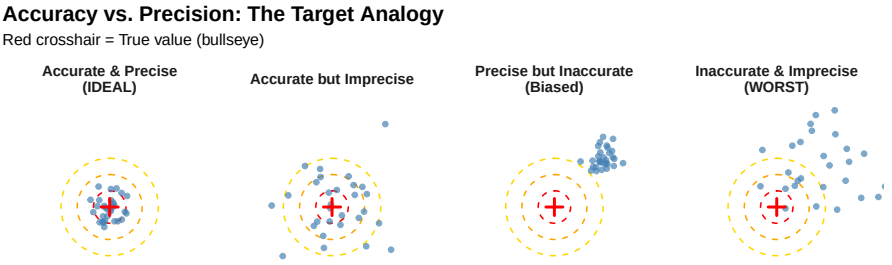


Table 11.2: Accuracy v

	Definition	Affected By
Accuracy	How close measurements are to the true value (average)	Calibration, bias, linearity
Precision	How close measurements are to each other (spread)	Repeatability, reproducibility,

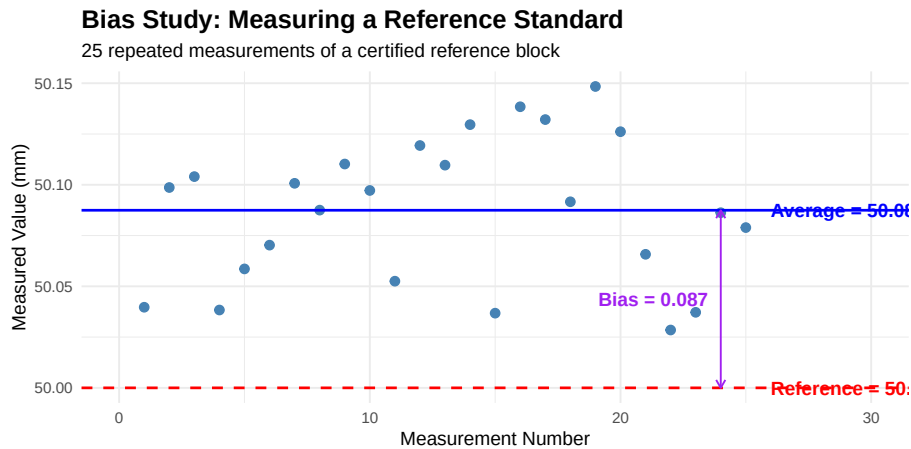
11.4 Accuracy Studies

Accuracy studies evaluate how well the measurement system indicates the true value.

11.4.1 Bias

Bias is the systematic error - the difference between the average of measurements and the true (reference) value.

$$\text{Bias} = \bar{x}_{\text{observed}} - x_{\text{reference}}$$



Live cell — try different reference and measured values.

```
#| label: bias-calculation
# Bias study calculation
reference_value <- 50.000 # Certified reference standard
measurements <- c(50.08, 50.07, 50.09, 50.08, 50.06, 50.10, 50.07, 50.08,
                  50.09, 50.08, 50.07, 50.11, 50.08, 50.09, 50.07)

# Calculate bias
average_measured <- mean(measurements)
bias <- average_measured - reference_value

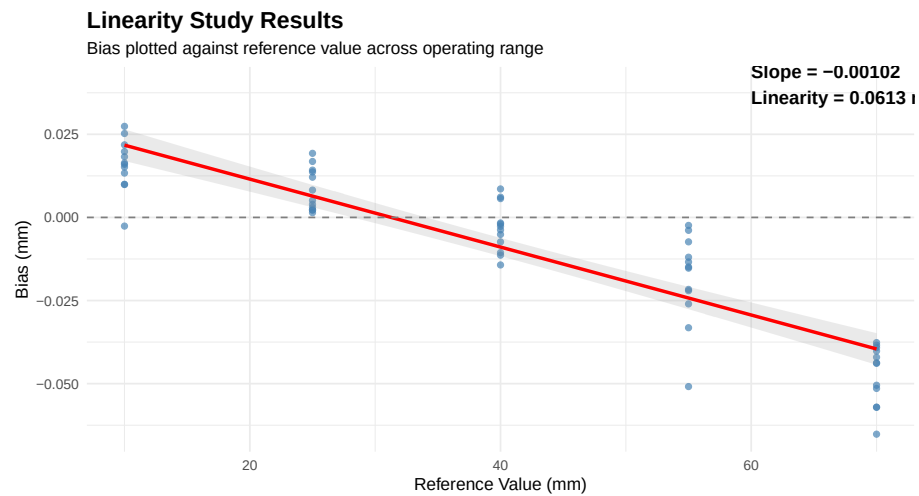
cat("Reference Value:", reference_value, "mm\n")
cat("Average Measured:", round(average_measured, 4), "mm\n")
cat("Bias:", round(bias, 4), "mm\n")
cat("Bias as % of Tolerance (±0.10):", round(abs(bias) / 0.10 * 100, 1), "%\n")

# Statistical test for significance
t_test <- t.test(measurements, mu = reference_value)
cat("\nt-test p-value:", round(t_test$p.value, 4))
if(t_test$p.value < 0.05) {
  cat(" - Bias is statistically significant\n")
} else {
  cat(" - Bias is not statistically significant\n")
}
```

11.4.2 Linearity

Linearity measures whether bias changes across the measurement range. A gauge might be accurate at one end of its range but biased at the other.

```
`geom_smooth()` using formula = 'y ~ x'
```



Live cell — try a different set of reference points and biases.

```
#| label: linearity-calc
# Linearity calculation
reference <- c(10, 25, 40, 55, 70)
avg_bias <- c(0.020, 0.008, -0.002, -0.018, -0.048)

# Fit linear regression
model <- lm(avg_bias ~ reference)

# Calculate linearity
slope <- coef(model)[2]
range_used <- max(reference) - min(reference)
linearity <- abs(slope) * range_used

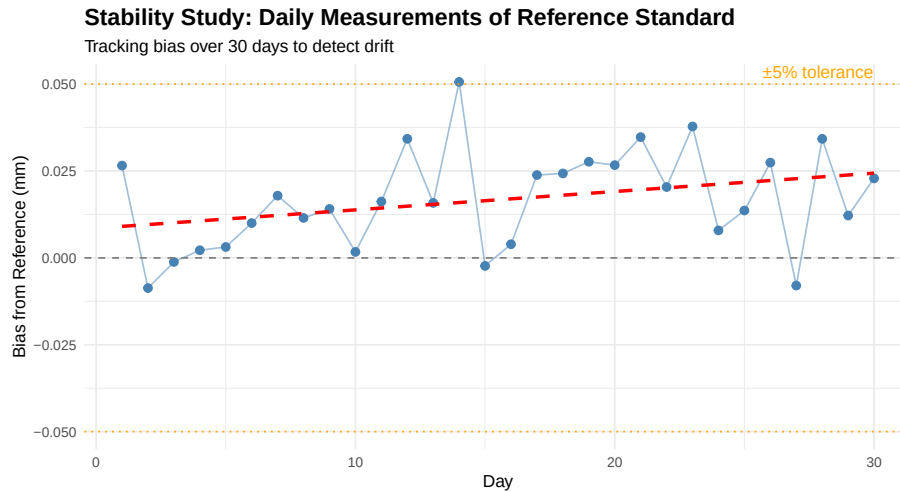
cat("Regression equation: Bias =", round(coef(model)[1], 4), "+",
    round(slope, 5), "× Reference\n")
cat("Linearity (slope × range):", round(linearity, 4), "mm\n")
cat("As % of tolerance (±0.10):", round(linearity / 0.10 * 100, 1), "%\n")

# R-squared
cat("R² =", round(summary(model)$r.squared, 3),
    "- indicates", ifelse(summary(model)$r.squared > 0.7, "significant", "minor"),
    "linearity issue\n")
```

11.4.3 Stability

Stability (also called drift) measures whether the measurement system's accuracy changes over time.

```
`geom_smooth()` using formula = 'y ~ x'
```



11.4.4 Stability Control Chart

For ongoing stability monitoring, use a control chart. Live cell — try different measurements or a different historical sigma.

```
#| label: stability-control-chart
# Stability monitoring with control chart
reference <- 25.000
measurements <- c(25.012, 25.008, 25.015, 25.018, 25.022, 25.019, 25.025,
                  25.028, 25.031, 25.026, 25.033, 25.038, 25.035, 25.041, 25.039)
days <- 1:15

# Calculate control limits (based on historical sigma)
historical_sigma <- 0.010
x_bar <- mean(measurements[1:5]) # Baseline from first 5 days
UCL <- x_bar + 3 * historical_sigma
LCL <- x_bar - 3 * historical_sigma

cat("Baseline Average:", round(x_bar, 4), "\n")
cat("UCL:", round(UCL, 4), "\n")
cat("LCL:", round(LCL, 4), "\n")

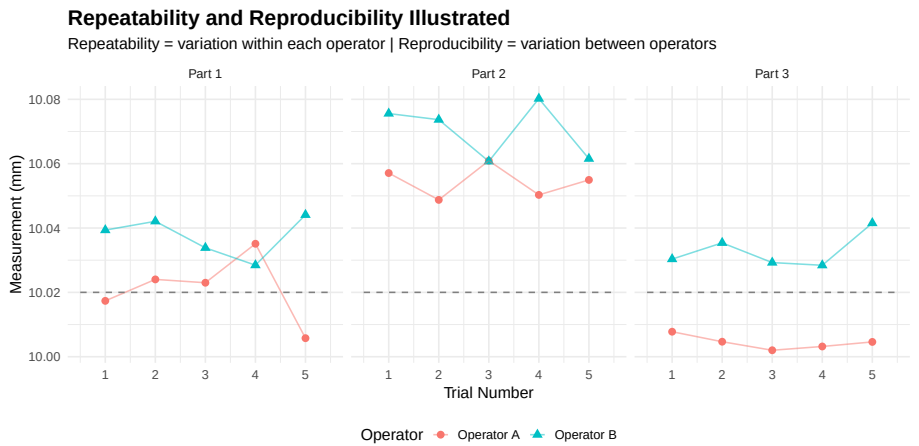
# Check for out-of-control
out_of_control <- which(measurements > UCL | measurements < LCL)
if(length(out_of_control) > 0) {
  cat("\nOut-of-control points on days:", out_of_control, "\n")
  cat("Drift detected - recalibration recommended\n")
} else {
```

```
cat("\nNo out-of-control points - measurement system stable\n")
}
```

11.5 Precision Studies: Gauge R&R

Gauge R&R (Repeatability and Reproducibility) is the most common MSA study. It quantifies precision variation.

11.5.1 Repeatability vs. Reproducibility



Table

Component	Also Known As	Definition
Repeatability (EV)	Equipment Variation, Within-operator variation	Variation when same op
Reproducibility (AV)	Appraiser Variation, Between-operator variation	Variation when different
Gauge R&R (GRR)	Total measurement system variation	Combined repeatability

11.5.2 Gauge R&R Study Design

Table 11.4: Standard Gauge R&R Study Design Parameters

Parameter	Typical_Value	Notes
Number of Parts	10	Minimum 5, 10 recommended for statisti
Number of Operators	3	Minimum 2, 3 is standard for detecting c
Number of Trials	3	Minimum 2, 3 is standard for repeatabili

Total Measurements	$10 \times 3 \times 3 = 90$	More measurements = better precision in estimates
Part Selection	Cover the full process range	Parts should represent typical production variation
Randomization	Randomize measurement order	Blind measurements when possible; prevents bias

11.5.3 Range Method (Short Form)

The Range Method is a quick approximation useful for initial screening.

Live cell — try different measurement data for the two operators.

```
#| label: range-method
# Range Method Gauge R&R Calculation
# Two operators, five parts, two trials each

# Data: Measurements by each operator (2 trials per part)
operator_a <- matrix(c(
  0.85, 0.87, # Part 1
  0.92, 0.91, # Part 2
  0.78, 0.80, # Part 3
  0.95, 0.96, # Part 4
  0.88, 0.85 # Part 5
), ncol = 2, byrow = TRUE)

operator_b <- matrix(c(
  0.86, 0.88, # Part 1
  0.93, 0.92, # Part 2
  0.80, 0.79, # Part 3
  0.94, 0.97, # Part 4
  0.87, 0.88 # Part 5
), ncol = 2, byrow = TRUE)

# Step 1: Calculate ranges for each part-operator combination
range_a <- apply(operator_a, 1, function(x) max(x) - min(x))
range_b <- apply(operator_b, 1, function(x) max(x) - min(x))

# Step 2: Average range
R_bar <- mean(c(range_a, range_b))

# Step 3: Calculate Repeatability (EV)
# d2 for 2 trials = 1.128
d2_trials <- 1.128
EV <- R_bar / d2_trials

# Step 4: Calculate part averages by operator
avg_a <- rowMeans(operator_a)
avg_b <- rowMeans(operator_b)
```

```

# Step 5: Range of operator averages
X_diff <- abs(mean(avg_a) - mean(avg_b))

# d2 for 2 operators = 1.128
d2_operators <- 1.128
n_parts <- 5
n_trials <- 2

# Step 6: Reproducibility (AV)
AV_squared <- (X_diff / d2_operators)^2 - (EV^2 / (n_parts * n_trials))
AV <- sqrt(max(0, AV_squared))

# Step 7: Gauge R&R
GRR <- sqrt(EV^2 + AV^2)

# Step 8: Express as % of tolerance
tolerance <- 0.30 # Example tolerance of ±0.15 = 0.30 total

cat("=== Range Method Gauge R&R Results ===\n\n")
cat("Average Range (R-bar):", round(R_bar, 4), "\n")
cat("Repeatability (EV):", round(EV, 4), "\n")
cat("Reproducibility (AV):", round(AV, 4), "\n")
cat("Gauge R&R (GRR):", round(GRR, 4), "\n\n")
cat("%EV (of tolerance):", round(EV / tolerance * 100 * 5.15, 1), "%\n")
cat("%AV (of tolerance):", round(AV / tolerance * 100 * 5.15, 1), "%\n")
cat("%GRR (of tolerance):", round(GRR / tolerance * 100 * 5.15, 1), "%\n")

```

11.5.4 ANOVA Method (Full Study)

ANOVA (Analysis of Variance) is a statistical technique that partitions the total observed variation in a dataset into distinct components, each attributable to a specific source. In the context of Gauge R&R, ANOVA separates the total measurement variation into contributions from:

- **Parts** (the actual differences between items being measured)
- **Operators** (systematic differences between people taking measurements)
- **Part × Operator interaction** (whether certain operators measure certain parts differently)
- **Repeatability** (random error when the same operator measures the same part multiple times)

11.5.4.1 Why Use ANOVA for Gauge R&R?

The fundamental question in any measurement system analysis is: *“How much of the variation I observe comes from my measurement system versus the ac-*

tual differences between parts?” ANOVA answers this question by statistically decomposing variance—it calculates how much each source contributes to what we observe.

Consider the alternative: if we simply looked at the spread of all measurements, we wouldn’t know whether a wide spread indicates highly variable parts (good—our gauge can detect differences) or a poor measurement system (bad—we’re adding noise). ANOVA untangles these effects mathematically.

11.5.4.2 Why ANOVA is the Preferred Method

While the Range Method provides a quick estimate, ANOVA offers several critical advantages for Gauge R&R studies:

Aspect	Range Method	ANOVA Method
Interaction effects	Cannot detect	Identifies Part $\times$ Operator interactions
Statistical rigor	Approximation using range-based estimates	Exact variance component estimation
Hypothesis testing	Not available	Provides F-tests and p-values for each source
Accuracy	Tends to overestimate GRR	More precise variance estimates
Information richness	Basic EV/AV breakdown	Full decomposition with confidence intervals
Industry acceptance	Screening only	Required for formal MSA studies (AIAG)

The ability to detect **interaction effects** is particularly important. If Operator A consistently measures Part 3 higher than Operator B does (but they agree on other parts), this indicates a training issue or technique difference that the Range Method would miss entirely. ANOVA flags this interaction, enabling targeted corrective action.

11.5.4.3 How ANOVA Works in Gauge R&R

ANOVA uses the **F-statistic** to test whether variation between groups (e.g., between operators) is significantly larger than variation within groups (repeatability). The mathematical basis is:

$$F = \frac{\text{Mean Square Between Groups}}{\text{Mean Square Within Groups}} = \frac{MS_{\text{factor}}}{MS_{\text{error}}}$$

A large F-value indicates that the factor (Part, Operator, or their interaction) contributes meaningfully to the observed variation. The p-value tells us the probability of seeing such an F-value if the factor had no real effect.

From the ANOVA table's Mean Squares, we extract **variance components** (σ^2) for each source, then combine them to calculate the standard metrics: %EV (repeatability), %AV (reproducibility), %GRR, and the number of distinct categories (ndc).

```
# Full Gauge R&R using ANOVA method
# 10 parts, 3 operators, 3 trials

set.seed(42)

# Generate realistic GRR data
n_parts <- 10
n_operators <- 3
n_trials <- 3

# Part true values (most of the variation)
part_effects <- rnorm(n_parts, 0, 0.05)

# Operator effects (some variation)
operator_effects <- c(-0.008, 0.003, 0.005)

# Create dataset
grr_data <- expand.grid(
  Part = 1:n_parts,
  Operator = paste0("Op", 1:n_operators),
  Trial = 1:n_trials
)

# Add measurements
grr_data$Measurement <- 10 +
  part_effects[grr_data$Part] +
  operator_effects[as.numeric(factor(grr_data$Operator))] +
  rnorm(nrow(grr_data), 0, 0.006) # Repeatability error

# Convert to factors
grr_data$Part <- factor(grr_data$Part)
grr_data$Operator <- factor(grr_data$Operator)

# Fit ANOVA model
anova_model <- aov(Measurement ~ Part + Operator + Part:Operator, data = grr_data)

# Extract variance components
anova_table <- anova(anova_model)
print(anova_table)
```

Analysis of Variance Table

Response: Measurement

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Part	9	0.136635	0.0151816	343.6163	< 2.2e-16 ***
Operator	2	0.003430	0.0017152	38.8223	1.52e-11 ***
Part:Operator	18	0.000430	0.0000239	0.5401	0.9263
Residuals	60	0.002651	0.0000442		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
# Extract Mean Squares from ANOVA
MS_part <- anova_table["Part", "Mean Sq"]
MS_operator <- anova_table["Operator", "Mean Sq"]
MS_interaction <- anova_table["Part:Operator", "Mean Sq"]
MS_error <- anova_table["Residuals", "Mean Sq"]

# Calculate variance components
n_p <- 10 # number of parts
n_o <- 3 # number of operators
n_r <- 3 # number of trials

# Repeatability variance
var_repeatability <- MS_error

# Interaction variance
var_interaction <- max(0, (MS_interaction - MS_error) / n_r)

# Operator variance
var_operator <- max(0, (MS_operator - MS_interaction) / (n_p * n_r))

# Part variance
var_part <- max(0, (MS_part - MS_interaction) / (n_o * n_r))

# Reproducibility = Operator + Interaction
var_reproducibility <- var_operator + var_interaction

# Total Gauge R&R
var_GRR <- var_repeatability + var_reproducibility

# Total variation
var_total <- var_part + var_GRR

# Convert to standard deviations
sigma_repeatability <- sqrt(var_repeatability)
sigma_reproducibility <- sqrt(var_reproducibility)
sigma_GRR <- sqrt(var_GRR)
```

```

sigma_part <- sqrt(var_part)
sigma_total <- sqrt(var_total)

# Calculate study variation (5.15 sigma for 99% of distribution)
SV_repeatability <- 5.15 * sigma_repeatability
SV_reproducibility <- 5.15 * sigma_reproducibility
SV_GRR <- 5.15 * sigma_GRR
SV_part <- 5.15 * sigma_part
SV_total <- 5.15 * sigma_total

cat("\n=== ANOVA Gauge R&R Results ===\n\n")
cat("Variance Components:\n")
cat("  Repeatability:", round(var_repeatability, 8), "\n")
cat("  Reproducibility:", round(var_reproducibility, 8), "\n")
cat("  Part-to-Part:", round(var_part, 8), "\n")
cat("  Total:", round(var_total, 8), "\n")

cat("\nStudy Variation (5.15):\n")
cat("  Repeatability:", round(SV_repeatability, 5), "\n")
cat("  Reproducibility:", round(SV_reproducibility, 5), "\n")
cat("  Gauge R&R:", round(SV_GRR, 5), "\n")
cat("  Part-to-Part:", round(SV_part, 5), "\n")
cat("  Total:", round(SV_total, 5), "\n")

```

```
=== ANOVA Gauge R&R Results ===
```

```

Variance Components:
  Repeatability: 4.418e-05
  Reproducibility: 5.638e-05
  Part-to-Part: 0.0016842
  Total: 0.00178476

```

```

Study Variation (5.15):
  Repeatability: 0.03423
  Reproducibility: 0.03867
  Gauge R&R: 0.05164
  Part-to-Part: 0.21135
  Total: 0.21757

```

11.5.5 Calculating %GRR and Acceptance Criteria

```

# Calculate %GRR using both methods
tolerance <- 0.30 # Total tolerance

```

```

# Method 1: % of Tolerance (%GRR_Tolerance)
pct_GRR_tol <- (SV_GRR / tolerance) * 100

# Method 2: % of Total Variation (%GRR_TV)
pct_GRR_tv <- (sigma_GRR / sigma_total) * 100

# Number of Distinct Categories (ndc)
ndc <- 1.41 * (sigma_part / sigma_GRR)

cat("=== Gauge R&R Acceptance Criteria ===\n\n")
cat("%GRR (of Tolerance):", round(pct_GRR_tol, 1), "%\n")
cat("%GRR (of Total Variation):", round(pct_GRR_tv, 1), "%\n")
cat("Number of Distinct Categories (ndc):", round(ndc, 1), "%\n\n")

# Acceptance criteria
cat("AIAG Acceptance Guidelines:\n")
cat("      \n")
if(pct_GRR_tol < 10) {
  cat("%GRR < 10%: ACCEPTABLE - Measurement system is acceptable\n")
} else if(pct_GRR_tol < 30) {
  cat("%GRR 10-30%: MARGINAL - May be acceptable based on application\n")
} else {
  cat("%GRR > 30%: UNACCEPTABLE - Measurement system needs improvement\n")
}

if(ndc >= 5) {
  cat("ndc 5: ACCEPTABLE - Adequate discrimination\n")
} else if(ndc >= 2) {
  cat("ndc 2-4: MARGINAL - Limited discrimination ability\n")
} else {
  cat("ndc < 2: UNACCEPTABLE - Cannot distinguish between parts\n")
}

```

=== Gauge R&R Acceptance Criteria ===

%GRR (of Tolerance): 17.2 %
 %GRR (of Total Variation): 23.7 %
 Number of Distinct Categories (ndc): 5.8

AIAG Acceptance Guidelines:

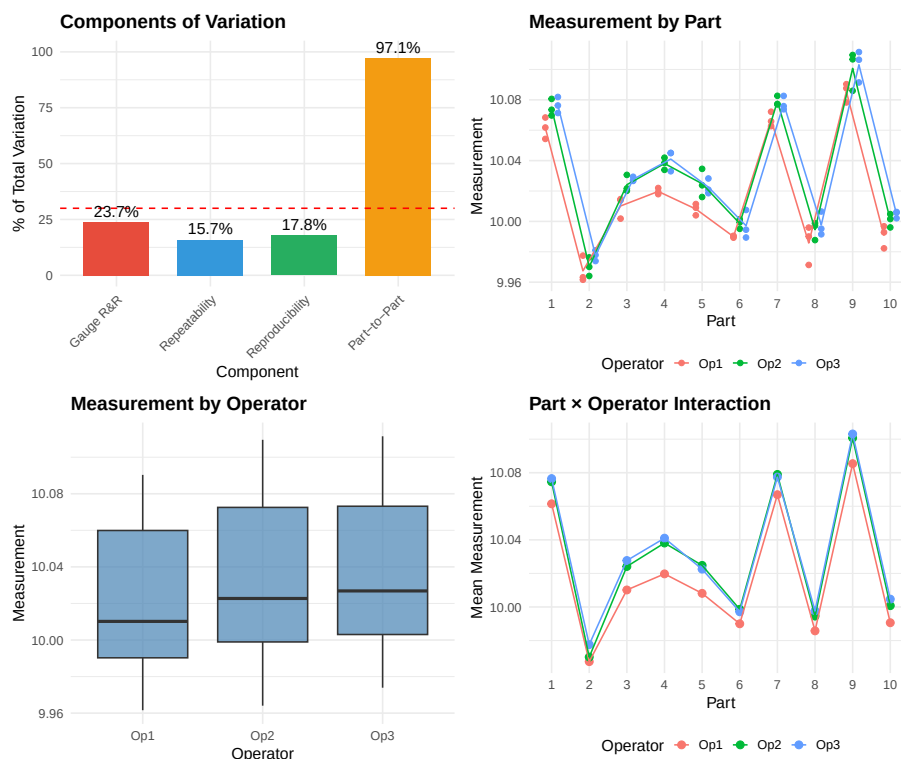
%GRR 10-30%: MARGINAL - May be acceptable based on application
 ndc 5: ACCEPTABLE - Adequate discrimination

11.5.6 Gauge R&R Acceptance Criteria Summary

Table 11.5: AIAG Gauge R&R Acceptance Guidelines

Criterion	Status	Interpretation
%GRR < 10%	Acceptable	Measurement system is acceptable for process control and capability
%GRR 10-30%	Marginal	May be acceptable depending on importance, cost of gauge, repair
%GRR > 30%	Unacceptable	Measurement system is not acceptable; action required
ndc = 5	Acceptable	Gauge can adequately distinguish between parts
ndc = 2-4	Marginal	Limited ability to distinguish; may be OK for simple pass/fail
ndc < 2	Unacceptable	Gauge cannot distinguish between parts; almost useless for control

11.5.7 Visual Analysis of Gauge R&R



11.6 Attribute Measurement Systems

For pass/fail or categorical measurements (visual inspection, go/no-go gauges), we use **Attribute MSA** or **Attribute Agreement Analysis**.

11.6.1 Kappa Statistic

When evaluating attribute measurement systems (pass/fail, accept/reject, visual inspection), you might think that **percent agreement** would be sufficient—if two inspectors agree 90% of the time, isn’t that good enough? The problem is that some agreement happens purely by chance, and percent agreement doesn’t account for this.

11.6.1.1 Why Percent Agreement Can Be Misleading

Consider a visual inspection where 95% of parts are good. If an inspector simply guesses “Accept” for every part (without even looking), they would be “correct” 95% of the time. Two such inspectors would agree 90.5% of the time by pure chance:

$$P_e = P(\text{both say Accept}) + P(\text{both say Reject}) = 0.95^2 + 0.05^2 = 0.905$$

This is why we need **Cohen’s Kappa** (κ)—it measures agreement *beyond what we’d expect from chance alone*.

11.6.1.2 The Kappa Formula

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

Where:

- P_o = Observed proportion of agreement (how often they actually agree)
- P_e = Expected proportion of agreement by chance (how often they’d agree if guessing randomly based on their individual tendencies)
- $1 - P_e$ = Maximum possible agreement beyond chance

Interpreting the formula: The numerator ($P_o - P_e$) represents *actual agreement minus chance agreement*—the “real” agreement attributable to consistent measurement. The denominator ($1 - P_e$) normalizes this to a 0–1 scale by dividing by the maximum possible improvement over chance.

Kappa Value	Meaning
$\kappa = 1$	Perfect agreement (everyone agrees, and not by chance)
$\kappa = 0$	Agreement equals chance—no better than random guessing
$\kappa < 0$	Agreement is <i>worse</i> than chance (systematic disagreement)

11.6.1.3 When to Use Kappa

Use Kappa when evaluating:

- **Inspector vs. Reference:** Does the operator match the known correct answer? (Most important for MSA)
- **Inspector vs. Inspector:** Do two operators classify the same parts the same way?
- **Inspector vs. Self (Repeatability):** Does the same operator give the same answer when re-inspecting the same part?

For a robust attribute MSA, you should calculate Kappa for all three comparisons. High agreement with other operators but low agreement with the reference standard indicates *consistent but incorrect* inspection—a training problem.

Table 11.6: Kappa Statistic Interpretation Guide

Kappa	Interpretation	Action
< 0	Less than chance agreement	Major issues - do not use
0.00 - 0.20	Slight agreement	Major improvement needed
0.21 - 0.40	Fair agreement	Significant improvement needed
0.41 - 0.60	Moderate agreement	Some improvement needed
0.61 - 0.80	Substantial agreement	Acceptable for most applications
0.81 - 1.00	Almost perfect agreement	Excellent - acceptable

11.6.2 Attribute Agreement Study Example

```
# Attribute Agreement Analysis
# 3 operators inspect 30 samples, 2 trials each
# Reference standard (known good/bad) available

set.seed(123)
n_samples <- 30
n_operators <- 3
n_trials <- 2

# Reference classification (10 bad, 20 good)
reference <- c(rep("Reject", 10), rep("Accept", 20))

# Simulate operator decisions (some disagreement)
simulate_decision <- function(ref, accuracy) {
  sapply(ref, function(r) {
    if(runif(1) < accuracy) r else ifelse(r == "Accept", "Reject", "Accept")
  })
}
```



```

# Operators with different accuracy levels
op1_t1 <- simulate_decision(reference, 0.92)
op1_t2 <- simulate_decision(reference, 0.92)
op2_t1 <- simulate_decision(reference, 0.88)
op2_t2 <- simulate_decision(reference, 0.88)
op3_t1 <- simulate_decision(reference, 0.95)
op3_t2 <- simulate_decision(reference, 0.95)

# Calculate agreement metrics

# 1. Within-operator agreement (repeatability)
within_op1 <- mean(op1_t1 == op1_t2)
within_op2 <- mean(op2_t1 == op2_t2)
within_op3 <- mean(op3_t1 == op3_t2)

# 2. Each operator vs. reference
op1_vs_ref <- mean(op1_t1 == reference & op1_t2 == reference)
op2_vs_ref <- mean(op2_t1 == reference & op2_t2 == reference)
op3_vs_ref <- mean(op3_t1 == reference & op3_t2 == reference)

# 3. All operators agree (both trials)
all_agree_t1 <- mean(op1_t1 == op2_t1 & op2_t1 == op3_t1)
all_agree_both <- mean(op1_t1 == op1_t2 & op1_t1 == op2_t1 & op2_t1 == op2_t2 &
                        op2_t1 == op3_t1 & op3_t1 == op3_t2)

cat("=== Attribute Agreement Analysis ===\n\n")
cat("Within-Operator Agreement (Repeatability):\n")
cat("  Operator 1:", round(within_op1 * 100, 1), "%\n")
cat("  Operator 2:", round(within_op2 * 100, 1), "%\n")
cat("  Operator 3:", round(within_op3 * 100, 1), "%\n\n")

cat("Each Operator vs. Reference (both trials correct):\n")
cat("  Operator 1:", round(op1_vs_ref * 100, 1), "%\n")
cat("  Operator 2:", round(op2_vs_ref * 100, 1), "%\n")
cat("  Operator 3:", round(op3_vs_ref * 100, 1), "%\n\n")

cat("All Operators Agreement:\n")
cat("  All agree (one trial):", round(all_agree_t1 * 100, 1), "%\n")
cat("  All agree (both trials):", round(all_agree_both * 100, 1), "%\n")

```

=== Attribute Agreement Analysis ===

Within-Operator Agreement (Repeatability):

Operator 1: 83.3 %

Operator 2: 76.7 %

Operator 3: 93.3 %

Each Operator vs. Reference (both trials correct):

Operator 1: 83.3 %

Operator 2: 73.3 %

Operator 3: 93.3 %

All Operators Agreement:

All agree (one trial): 70 %

All agree (both trials): 53.3 %

11.6.3 Calculating Kappa

```
# Calculate Kappa for Operator 1 vs Reference
# Create confusion matrix for trial 1
actual <- factor(reference, levels = c("Accept", "Reject"))
predicted <- factor(op1_t1, levels = c("Accept", "Reject"))

confusion <- table(Predicted = predicted, Actual = actual)
print(confusion)

# Calculate proportions
n <- sum(confusion)
p_o <- sum(diag(confusion)) / n # Observed agreement

# Expected agreement by chance
p_accept <- sum(confusion[1,]) / n * sum(confusion[,1]) / n
p_reject <- sum(confusion[2,]) / n * sum(confusion[,2]) / n
p_e <- p_accept + p_reject

# Kappa
kappa <- (p_o - p_e) / (1 - p_e)

cat("\nKappa Calculation (Operator 1 vs Reference):\n")
cat("Observed agreement (Po):", round(p_o, 3), "\n")
cat("Expected agreement (Pe):", round(p_e, 3), "\n")
cat("Kappa:", round(kappa, 3), "\n")

# Interpretation
if(kappa >= 0.81) {
  cat("Interpretation: Almost perfect agreement\n")
} else if(kappa >= 0.61) {
  cat("Interpretation: Substantial agreement\n")
} else if(kappa >= 0.41) {
  cat("Interpretation: Moderate agreement\n")
}
```

```

} else {
  cat("Interpretation: Fair or less agreement - needs improvement\n")
}

```

	Actual	
Predicted	Accept	Reject
Accept	17	1
Reject	3	9

Kappa Calculation (Operator 1 vs Reference):

Observed agreement (Po): 0.867

Expected agreement (Pe): 0.533

Kappa: 0.714

Interpretation: Substantial agreement

11.6.4 Attribute MSA Best Practices

Table 11.7: Best Practices for Attribute MSA Studies

Practice	Description
Use boundary samples	Include samples at the accept/reject boundary where decisions are hardest
Include clear accept/reject	Include obvious accept and obvious reject samples as controls
Blind testing	Operators should not know which samples are repeated or reference
Multiple trials	Minimum 2-3 trials per operator to assess within-operator repeatability
Reference standard	Must have known correct answers to assess accuracy, not just agreement
Adequate sample size	Minimum 30-50 samples recommended for statistical validity

11.7 Measurement System Resolution

Resolution (also called discrimination) is the smallest increment a measurement system can detect.

11.7.1 Rule of Ten

The **Rule of Ten** states that measurement resolution should be at least 1/10 of: - The tolerance, OR - The process variation (6)

Whichever is smaller.

Live cell — try different tolerance, process spread, or gauge resolution values.

```

#| label: resolution-check
# Resolution adequacy check

```

```

tolerance <- 0.100 # mm total tolerance
process_6sigma <- 0.080 # 6 sigma of process
gauge_resolution <- 0.010 # mm (smallest increment)

# Check against tolerance
ratio_tolerance <- tolerance / gauge_resolution
# Check against process
ratio_process <- process_6sigma / gauge_resolution

cat("=== Resolution Adequacy Check ===\n\n")
cat("Tolerance:", tolerance, "mm\n")
cat("Process 6 :", process_6sigma, "mm\n")
cat("Gauge Resolution:", gauge_resolution, "mm\n\n")

cat("Tolerance / Resolution:", ratio_tolerance, ":1")
if(ratio_tolerance >= 10) cat(" - ADEQUATE\n") else cat(" - INADEQUATE\n")

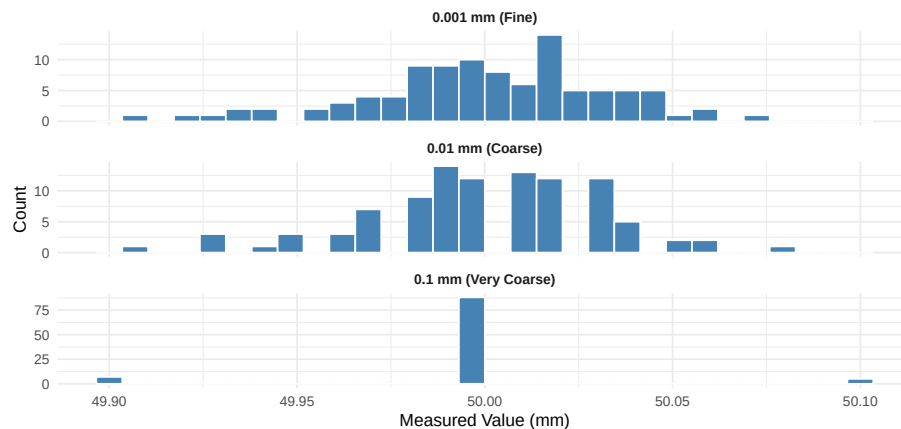
cat("Process 6 / Resolution:", ratio_process, ":1")
if(ratio_process >= 10) cat(" - ADEQUATE\n") else cat(" - INADEQUATE\n")

```

11.7.2 Resolution Impact on Gauge R&R

Effect of Gauge Resolution on Measurement Distribution

Same parts measured with different resolution gauges



11.8 Common Sources of Measurement Error

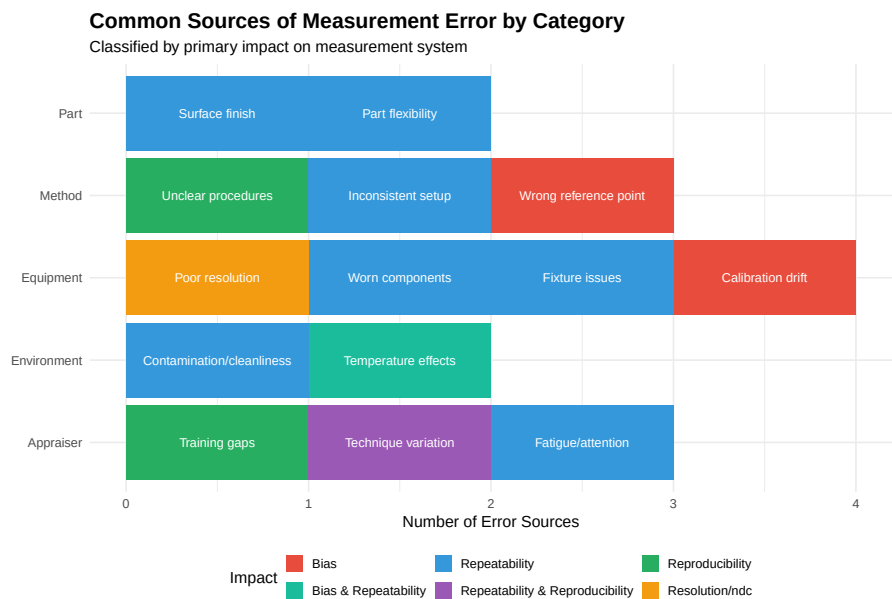


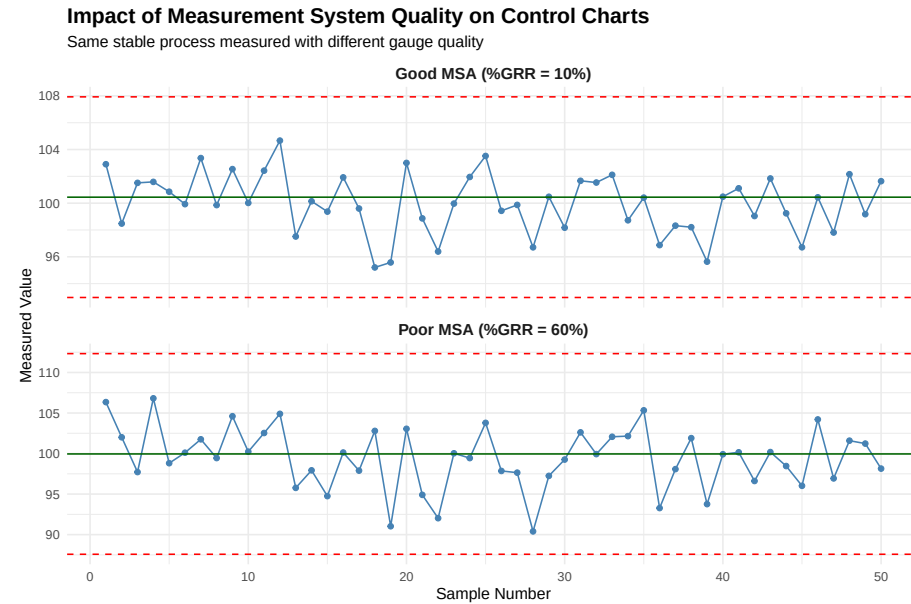
Table 11.8: MSA Issues and Improvement Action

Issue Identified	Likely Causes	Improvement Action
High Repeatability (EV)	Gauge condition, fixture, resolution, part variation during measurement	Maintain
High Reproducibility (AV)	Training, technique, procedure clarity	Standardize
Significant Bias	Calibration, master accuracy, technique	Recalibrate
Poor Linearity	Gauge mechanism, calibration across range	Repair/Replace
Drift/Stability Issues	Environmental effects, gauge wear, master deterioration	Environment Control
Low ndc	Resolution, excessive gauge variation relative to part variation	Higher Resolution

11.9 MSA and SPC Relationship

Poor measurement systems directly impact SPC effectiveness.

11.9.1 Effect on Control Charts



11.9.2 Effect on Capability Studies

Live cell — try different tolerance/sigma values or GRR percentages.

```
#| label: msa-capability-effect
# True process parameters
true_mean <- 100
true_sigma <- 2.0
USL <- 106
LSL <- 94
tolerance <- USL - LSL

# True capability
true_Cp <- tolerance / (6 * true_sigma)
true_Cpk <- min((USL - true_mean), (true_mean - LSL)) / (3 * true_sigma)

cat("=== True Process Capability ===\n")
cat("True :", true_sigma, "\n")
cat("True Cp:", round(true_Cp, 2), "\n")
cat("True Cpk:", round(true_Cpk, 2), "\n\n")

# With good measurement system (10% GRR)
grr_good <- 0.10 * tolerance / 5.15
observed_sigma_good <- sqrt(true_sigma^2 + grr_good^2)
```

```
observed_Cp_good <- tolerance / (6 * observed_sigma_good)
observed_Cpk_good <- min((USL - true_mean), (true_mean - LSL)) / (3 * observed_sigma_good)

cat("=== With Good MSA (10% GRR) ===\n")
cat("Observed  :", round(observed_sigma_good, 3), "\n")
cat("Observed Cp:", round(observed_Cp_good, 2), "\n")
cat("Observed Cpk:", round(observed_Cpk_good, 2), "\n\n")

# With poor measurement system (50% GRR)
grr_poor <- 0.50 * tolerance / 5.15
observed_sigma_poor <- sqrt(true_sigma^2 + grr_poor^2)
observed_Cp_poor <- tolerance / (6 * observed_sigma_poor)
observed_Cpk_poor <- min((USL - true_mean), (true_mean - LSL)) / (3 * observed_sigma_poor)

cat("=== With Poor MSA (50% GRR) ===\n")
cat("Observed  :", round(observed_sigma_poor, 3), "\n")
cat("Observed Cp:", round(observed_Cp_poor, 2), "\n")
cat("Observed Cpk:", round(observed_Cpk_poor, 2), "\n\n")

cat("Poor MSA makes a capable process (Cpk=1.0) appear incapable (Cpk=",
    round(observed_Cpk_poor, 2), ")\n")
```

11.10 Conducting an MSA Study: Step-by-Step

Gauge R&R Study Process Flow

Steps for conducting a complete MSA study



11.10.1 Data Collection Sheet Template

Table 11.9: Gauge R&R Data Collection Sheet (Partial Example)

Part	Operator	Trial	Measurement
1	A	1	_____
1	A	2	_____
1	B	1	_____
1	B	2	_____
1	C	1	_____
1	C	2	_____
2	A	1	_____
2	A	2	_____
2	B	1	_____
2	B	2	_____
2	C	1	_____

2 C

2

11.11 Video Resources

11.11.1 Understanding Gauge R&R

11.11.2 MSA in Practice

11.12 Summary

Measurement Systems Analysis ensures that your data can be trusted for process control and decision-making:

1. **Before SPC, do MSA** - Garbage in, garbage out; verify measurement quality first
2. **Accuracy components** - Bias, linearity, and stability affect location (average)
3. **Precision components** - Repeatability (within operator) and reproducibility (between operators)
4. **Key metrics** - %GRR < 30% and $ndc \geq 5$ for acceptable systems
5. **Resolution matters** - 10:1 ratio vs. tolerance or process variation
6. **Attribute MSA** - Use Kappa for pass/fail decisions
7. **Continuous improvement** - Regular verification and recertification

“The measurement system is part of the process. Fix the measurement system before trying to fix the process.”

11.13 Review Questions

Question 1: A gauge R&R study yields %GRR = 45% and $ndc = 2$. What does this mean and what should be done?

Answer:

This measurement system is **unacceptable**:

- **%GRR = 45%** exceeds the 30% maximum threshold, meaning 45% of the observed variation is due to the measurement system, not the parts
- **$ndc = 2$** indicates the gauge can only distinguish between 2 categories of parts, which is insufficient for process control (minimum 5 required)

Actions required: 1. Investigate root causes of high variation: - Check gauge calibration and condition - Evaluate operator technique and training - Review measurement procedure for ambiguity - Assess fixture and part positioning - Check gauge resolution adequacy

2. Prioritize improvement efforts:

- If repeatability (EV) is high: Focus on gauge condition, fixture, technique consistency
- If reproducibility (AV) is high: Focus on training, procedure clarity, technique standardization

3. Re-run study after improvements

4. If improvements insufficient, consider upgrading to a higher-capability gauge

5. Until resolved, this gauge should not be used for process control or capability studies

Question 2: What is the difference between repeatability and reproducibility? Give an example of each source of error.

Answer:

Repeatability (Equipment Variation - EV): - Variation when the **same operator** measures the **same part** multiple times with the **same gauge** - Also called “within-operator” variation - Represents the inherent precision of the gauge and measurement technique

Examples of repeatability errors: - Gauge resolution limitations - Gauge mechanism play/backlash - Inconsistent part positioning in fixture - Variations in technique even by same operator - Environmental micro-changes during measurement

Reproducibility (Appraiser Variation - AV): - Variation when **different operators** measure the **same parts** - Also called “between-operator” variation - Represents differences in how operators apply the measurement method

Examples of reproducibility errors: - Different technique between operators (pressure applied, angle, etc.) - Different interpretation of measurement procedure - Different reading of analog scales - Training differences - Physical differences (eyesight, hand steadiness)

Question 3: Calculate the bias and determine if it is acceptable. Reference value = 25.000 mm, Tolerance = ± 0.050 mm, Measured values: 25.012, 25.015, 25.008, 25.011, 25.014, 25.009, 25.013, 25.010, 25.012, 25.011

Answer:

Live cell — check your own version by changing the numbers.

```

#| label: q3-answer
reference <- 25.000
tolerance_total <- 0.100 # ±0.050 = 0.100 total
measurements <- c(25.012, 25.015, 25.008, 25.011, 25.014,
                  25.009, 25.013, 25.010, 25.012, 25.011)

# Calculate bias
mean_measured <- mean(measurements)
bias <- mean_measured - reference

cat("Reference value:", reference, "mm\n")
cat("Mean measured:", round(mean_measured, 4), "mm\n")
cat("Bias:", round(bias, 4), "mm\n\n")

# Express as % of tolerance
bias_pct <- abs(bias) / tolerance_total * 100
cat("Bias as % of tolerance:", round(bias_pct, 1), "%\n")

# T-test for significance
t_result <- t.test(measurements, mu = reference)
cat("T-test p-value:", round(t_result$p.value, 6), "\n\n")

# Acceptability
cat("Acceptability Assessment:\n")
if(bias_pct < 10) {
  cat("- Bias < 10% of tolerance: ACCEPTABLE\n")
} else if(bias_pct < 25) {
  cat("- Bias 10-25% of tolerance: MARGINAL\n")
} else {
  cat("- Bias > 25% of tolerance: UNACCEPTABLE\n")
}

if(t_result$p.value < 0.05) {
  cat("- Bias is statistically significant (p < 0.05)\n")
  cat("- Consider recalibration or technique adjustment\n")
}

```

The bias of 0.0115 mm (11.5% of tolerance) is **marginally acceptable** and statistically significant. Recalibration should be considered.

Question 4: Why is it important to conduct MSA before capability studies (C_p , C_{pk})?

Answer:

MSA must precede capability studies because measurement error directly inflates observed variation:

Mathematical Relationship:

$$\sigma_{observed}^2 = \sigma_{process}^2 + \sigma_{measurement}^2$$

Impacts on Capability:

1. **Understated Cp/Cpk:** Observed sigma is always larger than true process sigma, so calculated capability will be lower than actual capability
2. **False Alarms:** A capable process may appear incapable due to measurement noise, leading to unnecessary process adjustments or investment
3. **Missed Issues:** High measurement variation masks actual process variation; you can't see problems through the "fog" of gauge error
4. **Bad Decisions:** Incorrect capability data leads to:
 - Wrong process acceptance decisions
 - Inappropriate control limits
 - Misguided improvement investments
 - Customer quality issues

Example: - True process: Cpk = 1.5 (very capable) - With 50% GRR: Observed Cpk = 0.9 (appears incapable) - Result: Unnecessary process "improvement" efforts

Rule of Thumb: If %GRR > 30%, any capability study is essentially meaningless because too much of the observed variation is measurement noise, not real process variation.

Question 5: A visual inspection has the following results: Kappa vs. reference = 0.72, within-operator agreement = 88%, between-operator agreement = 75%. Evaluate this attribute measurement system.

Answer:

Results Interpretation:

1. **Kappa vs. Reference = 0.72**
 - Falls in "Substantial agreement" range (0.61-0.80)
 - Operators are correctly classifying parts most of the time
 - Acceptable for many applications, but improvement possible
2. **Within-Operator Agreement = 88%**
 - Operators are fairly consistent with themselves
 - 12% of the time, same operator gives different result on same part
 - Should target >90% for critical inspections
3. **Between-Operator Agreement = 75%**
 - Operators disagree on 25% of parts
 - This is a significant reproducibility issue
 - Major source of inconsistency in product quality

Assessment: MARGINAL - Improvement Needed**Recommended Actions:**

1. **Standardize criteria:** Create clear visual standards with boundary samples
 - Photos of accept/reject borderline cases
 - Written descriptions of defect criteria
2. **Training:**
 - Calibration session with all operators and reference samples
 - Identify operators with lower agreement for targeted training
3. **Improve conditions:**
 - Check lighting adequacy and consistency
 - Ensure proper viewing distance and angle
 - Reduce fatigue with appropriate break schedules
4. **Consider automation:** For critical inspections, automated vision systems may provide better consistency
5. **Re-study after improvements** to verify effectiveness

Question 6: What is the “Rule of Ten” for gauge resolution, and how would you apply it?

Answer:

The Rule of Ten: The gauge resolution should be at least **1/10 of the tolerance** or **1/10 of the process variation (6 σ)**, whichever is smaller.

$$\text{Resolution} \leq \frac{\min(\text{Tolerance}, 6\sigma)}{10}$$

Application Example:

Given: - Part tolerance: ± 0.05 mm (total = 0.10 mm) - Process 6 σ : 0.08 mm - Available gauges: 0.01 mm resolution, 0.001 mm resolution

Calculation:

Required resolution $\min(0.10, 0.08) / 10$
 Required resolution $0.08 / 10$
 Required resolution 0.008 mm

Assessment: - 0.01 mm gauge: $0.01 > 0.008 \rightarrow$ **INADEQUATE** - 0.001 mm gauge: $0.001 < 0.008 \rightarrow$ **ADEQUATE**

Why It Matters: 1. If resolution is too coarse, gauge cannot detect small part differences 2. Results in low ndc (number of distinct categories) 3. Control charts will show “stair-step” patterns 4. Capability studies will be inaccurate

Practical Tips: - For SPC applications, 10:1 is minimum; 20:1 is preferred - For capability studies, 10:1 is acceptable - For simple pass/fail gauging, 5:1 may be acceptable

Question 7: Given the following ANOVA table from a Gauge R&R study, calculate the variance components and %GRR.

Source	DF	SS	MS
Part	9	0.2850	0.03167
Operator	2	0.0025	0.00125
Part×Operator	18	0.0090	0.00050
Repeatability	60	0.0180	0.00030
Total	89	0.3145	

Study design: 10 parts, 3 operators, 3 trials

Answer:

Live cell — check your own version by changing the ANOVA table values.

```
#| label: q7-answer
# Given values from ANOVA table
MS_part <- 0.03167
MS_operator <- 0.00125
MS_interaction <- 0.00050
MS_error <- 0.00030

n_parts <- 10
n_operators <- 3
n_trials <- 3

# Variance components
var_repeatability <- MS_error
cat("Var(Repeatability) =", var_repeatability, "\n")

var_interaction <- max(0, (MS_interaction - MS_error) / n_trials)
cat("Var(Interaction) =", round(var_interaction, 6), "\n")

var_operator <- max(0, (MS_operator - MS_interaction) / (n_parts * n_trials))
cat("Var(Operator) =", round(var_operator, 6), "\n")

var_part <- max(0, (MS_part - MS_interaction) / (n_operators * n_trials))
cat("Var(Part) =", round(var_part, 6), "\n\n")

# Reproducibility = Operator + Interaction
var_reproducibility <- var_operator + var_interaction
```

```

cat("Var(Reproducibility) =", round(var_reproducibility, 6), "\n")

# Total Gauge R&R
var_GRR <- var_repeatability + var_reproducibility
cat("Var(GRR) =", round(var_GRR, 6), "\n")

# Total variation
var_total <- var_part + var_GRR
cat("Var(Total) =", round(var_total, 6), "\n\n")

# Calculate %GRR (of total variation)
sigma_GRR <- sqrt(var_GRR)
sigma_total <- sqrt(var_total)
pct_GRR <- (sigma_GRR / sigma_total) * 100

cat("%GRR (of Total Variation) =", round(pct_GRR, 1), "%\n\n")

# ndc
sigma_part <- sqrt(var_part)
ndc <- 1.41 * (sigma_part / sigma_GRR)
cat("ndc =", round(ndc, 1), "\n\n")

if(pct_GRR < 10) {
  cat("Assessment: ACCEPTABLE (< 10%)\n")
} else if(pct_GRR < 30) {
  cat("Assessment: MARGINAL (10-30%)\n")
} else {
  cat("Assessment: UNACCEPTABLE (> 30%)\n")
}

```

Question 8: What actions would you take if a Gauge R&R study showed high repeatability (EV) but low reproducibility (AV)?

Answer:

When **repeatability is high** (main contributor to GRR) but **reproducibility is low**, the measurement variation is coming from the gauge/equipment/technique rather than differences between operators.

Root Cause Investigation - Focus Areas:

1. **Gauge Condition**
 - Worn measuring surfaces or contacts
 - Loose components or mechanism play
 - Damaged or dirty probe/sensor
2. **Gauge Resolution**
 - Resolution may be inadequate for the tolerance

- Check 10:1 rule compliance
- 3. **Fixture/Fixturing**
 - Part not held consistently
 - Fixture wear or damage
 - Part not seating properly
- 4. **Measurement Technique**
 - Inconsistent contact force
 - Varying measurement location on part
 - Speed of measurement varies
- 5. **Part Characteristics**
 - Part surface finish affecting readings
 - Part flexibility/deflection during measurement
 - Part temperature variation
- 6. **Environment**
 - Vibration affecting readings
 - Temperature instability
 - Contamination

Improvement Actions:

Priority	Action
1	Verify gauge calibration and condition; repair/replace if needed
2	Check and improve fixture; ensure consistent part positioning
3	Evaluate and potentially upgrade gauge resolution
4	Standardize technique (force, speed, location)
5	Control environmental factors
6	Re-run study to verify improvement

Note: Low AV in this case is actually good - it means operators are consistent with each other. The problem is the measurement equipment itself.

11.14 References

1. AIAG. (2010). *Measurement Systems Analysis Reference Manual* (4th ed.). Automotive Industry Action Group.
2. Wheeler, D.J. (2006). *EMP III: Evaluating the Measurement Process* (3rd ed.). SPC Press.
3. Montgomery, D.C. (2019). *Introduction to Statistical Quality Control* (8th ed.). Wiley.
4. Burdick, R.K., Borror, C.M., & Montgomery, D.C. (2005). *Design and Analysis of Gauge R&R Studies*. SIAM.

5. ASTM E2782-17. *Standard Guide for Measurement Systems Analysis*.
6. ISO 22514-7:2021. *Statistical Methods in Process Management - Capability and Performance - Part 7: Capability of Measurement Processes*.
7. Wheeler, D.J., & Lyday, R.W. (1989). *Evaluating the Measurement Process* (2nd ed.). SPC Press.
8. Minitab. (2021). *Gage R&R Study (Crossed)*. Minitab Support Documentation.
9. JCGM 100:2008. *Evaluation of Measurement Data - Guide to the Expression of Uncertainty in Measurement* (GUM).
10. Dietrich, E., & Schulze, A. (2011). *Statistical Procedures for Machine and Process Qualification* (6th ed.). Hanser.

Chapter 12

Working Effectively with AI

12.1 Learning Objectives

After completing this chapter, you will be able to:

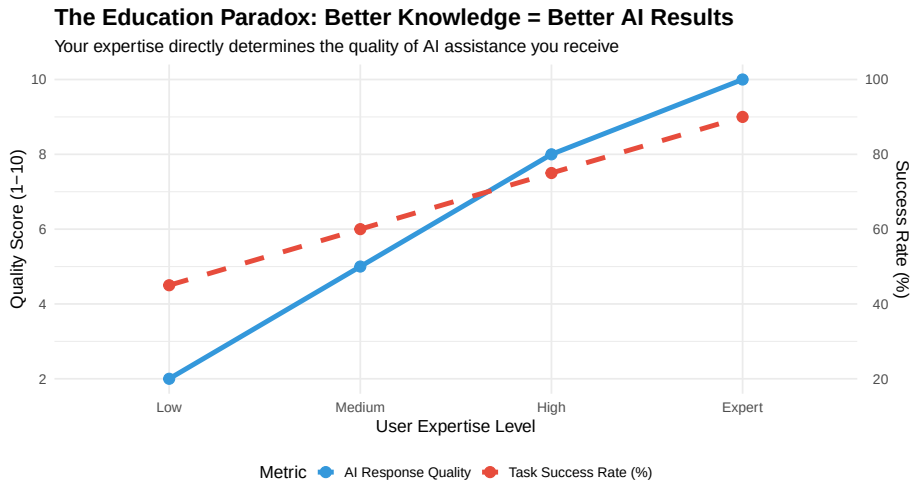
1. Understand AI's role in modern engineering work and its limitations
 2. Apply the four steps of effective AI use: Define, Prompt, Iterate, Validate
 3. Apply prompt engineering principles to get better AI responses
 4. Distinguish between poor, adequate, and excellent prompts
 5. Select appropriate AI tools for different tasks (chat interfaces, NotebookLM, Claude Code, browser extensions)
 6. Use iterative AI collaboration to solve complex engineering problems
 7. Critically evaluate AI-generated content for accuracy and safety
 8. Identify tasks suited for AI assistance versus human expertise
 9. Apply ethical considerations when using AI in engineering contexts
-

12.2 Introduction: AI in the Modern Workplace

Artificial Intelligence has fundamentally changed how engineering work is performed. According to research, AI is predominantly used for **coding and technical work** (34% of professional use), with complex tasks showing potential **12x speedups** compared to traditional methods.

However, there's a critical paradox: **more complex tasks have lower success rates** (61-66% versus 70% for simpler tasks). This means engineers must develop skills to effectively collaborate with AI, not just delegate to it.

12.2.1 The Education Paradox



Key Insight: There is a nearly perfect correlation between prompt sophistication and response quality. You need strong foundational knowledge to get quality AI assistance. “AI will do it for me” is a trap.

Discussion: Why does expertise improve AI results?

The “education paradox” reveals a fundamental truth about AI collaboration:

- **Better questions lead to better answers:** Experts know what to ask and how to frame problems
- **Domain knowledge enables validation:** You can only verify AI output if you understand the domain
- **Iterative refinement requires expertise:** Knowing when to push back or ask follow-up questions
- **Critical evaluation is essential:** AI can sound confident while being wrong

Think about it: If you don’t understand the fundamentals of motor selection, how would you know if an AI recommendation is correct or dangerously undersized?

12.2.2 AI Impact by Industry

Table 12.1: AI Impact Across Engineering Industries

Industry	Common AI Applications	Potential Impact
Automotive	Simulation modeling, control systems, predictive maintenance	10-12x
Food Processing	Process optimization, quality control, equipment diagnostics	8-10x

Defense	System design, modeling, compliance documentation	10-12x
Mining	Equipment diagnostics, predictive maintenance, safety analysis	8-10x
General Manufacturing	Documentation, troubleshooting guides, process analysis	6-8x

Why do automotive and defense require “Critical” human oversight?

Safety-critical applications require mandatory human validation because:

1. **Regulatory requirements:** ISO 26262 (automotive) and MIL-STDs (defense) require documented human review
2. **Liability:** Engineers are legally responsible for designs, not AI tools
3. **Failure consequences:** Errors can cause injury, death, or mission failure
4. **AI limitations:** Current AI cannot reliably reason about edge cases and failure modes
5. **Standards compliance:** AI cannot certify compliance with safety standards

Best practice: Use AI for initial research and drafting, but always validate with human experts and formal analysis methods (FMEA, FTA, HAZOP).

12.3 Prompt Engineering Fundamentals

Prompt engineering is the skill of crafting inputs to AI systems that produce useful, accurate, and relevant outputs. In engineering contexts, this skill is becoming as important as technical writing.

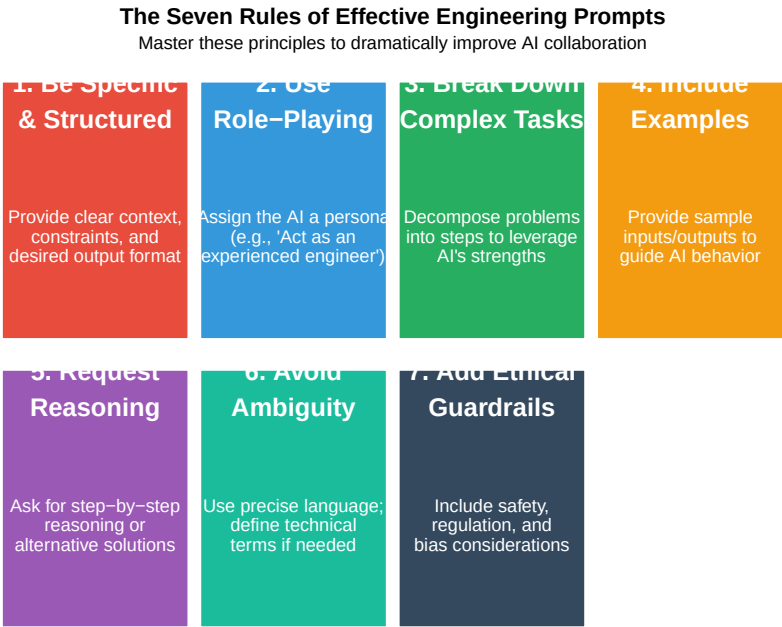
Before you read: What makes a good question?

Think about the difference between these two questions:

1. “Can you help me with my motor problem?”
2. “I have a 3-phase 460V 50HP motor that trips its overload after 15 minutes of running. Ambient temperature is 35°C. What diagnostic steps should I follow?”

Which question would get better help from a human expert? The same principle applies to AI. Before reading about prompt engineering, consider what information you would need to give a colleague to get useful help.

12.3.1 The Seven Rules of Effective Prompts



12.4 Prompt Quality Comparison: Bad, Medium, and Good

The following examples demonstrate the dramatic difference that prompt quality makes. Each example shows the same task approached three ways.

12.4.1 Example 1: Motor Selection

Quality Level	Prompt
Bad	Help me pick a motor.
Medium	I need to select a motor for a conveyor system. The conveyor moves boxes that weigh
Good	Act as a senior electromechanical engineer. I need to select an AC induction motor fo

12.4.2 Example 2: Troubleshooting a PLC System

Quality Level	Prompt
Bad	My PLC isn't working. Fix it.
Medium	I have an Allen-Bradley PLC that's showing a fault. The output for a motor starter isn't turning
Good	Act as an experienced automation technician troubleshooting a control system issue.\n\nSystem:

12.4.3 Example 3: Safety Analysis

Quality Level	Prompt
Bad	Is this machine safe?
Medium	I have a robotic welding cell. What safety features does it need?
Good	Act as a certified machine safety engineer (TUV or equivalent).\n\nI need to perform a risk assess

Warning: For safety-critical applications, AI should NEVER be the final authority. Always validate AI-generated safety analyses with certified professionals and applicable standards.

12.5 Anatomy of an Excellent Engineering Prompt

Let's break down the components of a well-structured prompt:

Anatomy of an Excellent Engineering Prompt

Each component serves a specific purpose in guiding AI response quality

Assignment	"Act as a senior electromechanical engineer specializing in automotive systems"	Give expert perspective for response
Context	"I am designing a regenerative braking system for an electric vehicle"	Establish the problem domain and application
Specification	"Vehicle mass: 2000kg, max speed: 150km/h, battery: 400V nominal"	Provide concrete parameters for calculations
Constraint	"Must comply with ISO 26262 ASIL-B, Budget under \$500/unit"	Define boundaries and requirements
Deliverable	"Provide: 1) Energy calculations, 2) Component selection, 3) Control strategy"	Specify what you need from the AI
Format	"Format as a technical design document with diagrams described"	Ensure output is immediately usable
Standards	"Reference ISO 26262 and SAE J2954 where applicable"	Anchor response in industry requirements

12.6 Interactive Exercise: Rate These Prompts

For each prompt below, identify what’s missing and how you would improve it.

12.6.1 Exercise 1: Pump Selection

Original Prompt: “What pump should I use for water?”

Table 12.5: Analysis: What’s Missing from This Prompt?

Missing Element	Why It’s Important
Flow rate requirement	Cannot size pump without knowing GPM or L/min needed
Head/pressure requirement	Total dynamic head determines pump curve selection
Fluid properties	Temperature, viscosity, solids content affect pump type
Operating environment	Indoor/outdoor, hazardous area, food-grade requirements
Power supply	Single/three phase, voltage determines motor options
Budget/efficiency goals	Premium efficiency vs. standard, capital vs. operating cost

Your Task: Rewrite this prompt to include all necessary information for a food processing application pumping tomato paste at 80°C.

12.6.2 Exercise 2: Identify the Quality Level

Rate each prompt as Bad, Medium, or Good and justify your rating:

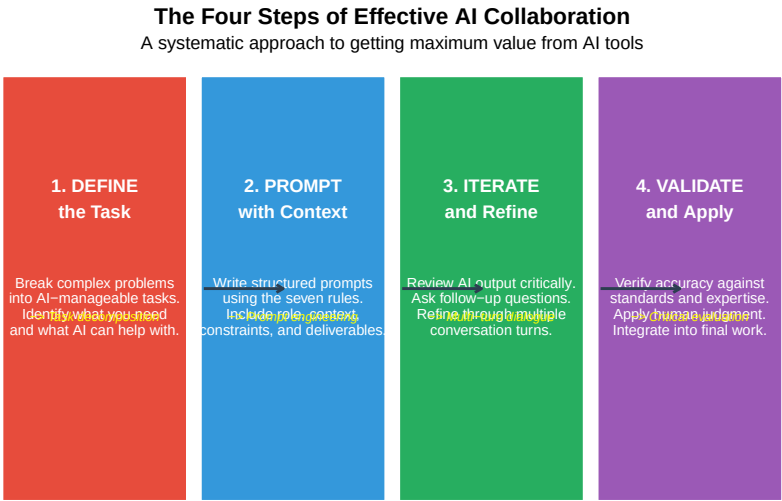
Table 12.6: Exe

#	Prompt
1	How do I wire a motor?
2	Explain the difference between VFD and soft starter for a 50HP motor application in a cement plant with hig
3	Write code for Arduino.
4	Design a sensor network for predictive maintenance on a CNC machine. Include vibration, temperature, and c
5	Help with my project.

12.7 The Four Steps of Effective AI Use

The research shows that effectively working with AI follows a consistent four-step process. Master these steps to maximize the value of AI collaboration.

Warning: Using ``size`` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use ``linewidth`` instead.



12.7.1 Step 1: Define the Task

Before engaging with AI, clearly define what you need:

- **What is the end goal?** A calculation, a design, troubleshooting guidance?

- **What constraints apply?** Budget, standards, materials, timeline?
- **What do you already know?** Your expertise shapes the conversation
- **Can AI actually help with this?** Some tasks require physical inspection or human judgment

Key Insight: The better you understand the problem, the better AI can help. Vague understanding leads to vague prompts and useless responses.

12.7.2 Step 2: Prompt with Context

Apply the seven rules of effective prompting:

1. Assign a role to establish expertise level
2. Provide complete context and background
3. Specify constraints and requirements
4. List clear deliverables
5. Request step-by-step reasoning
6. Define the output format
7. Include relevant standards

12.7.3 Step 3: Iterate and Refine

Rarely is the first response perfect. Plan for 3-6 turns of conversation:

- Review critically: What's useful? What's missing? What's wrong?
- Ask targeted follow-up questions
- Request expansions on specific areas
- Challenge assumptions and ask for alternatives

12.7.4 Step 4: Validate and Apply

AI output is a **starting point**, not a final answer:

- Verify calculations and units
- Cross-check against standards and codes
- Apply your engineering judgment
- Document what came from AI assistance
- Take responsibility for the final work product

Case Study: When validation caught an AI error

Scenario: An engineering student used AI to calculate the required conveyor belt width for a food processing line handling 500 kg/hr of bulk product.

AI Response: Recommended a 300mm belt based on throughput calculations.

Validation Issue: The AI correctly calculated volumetric flow but failed to consider: - Minimum belt width for the discharge chute (400mm required) - Product spread factor for irregular materials - Safety factor for surge capacity

Actual Requirement: 500mm belt per facility standards.

Lesson: AI calculations may be mathematically correct but miss practical engineering constraints. Always validate against physical requirements, codes, and site-specific standards.

Discussion: When should you NOT use AI?

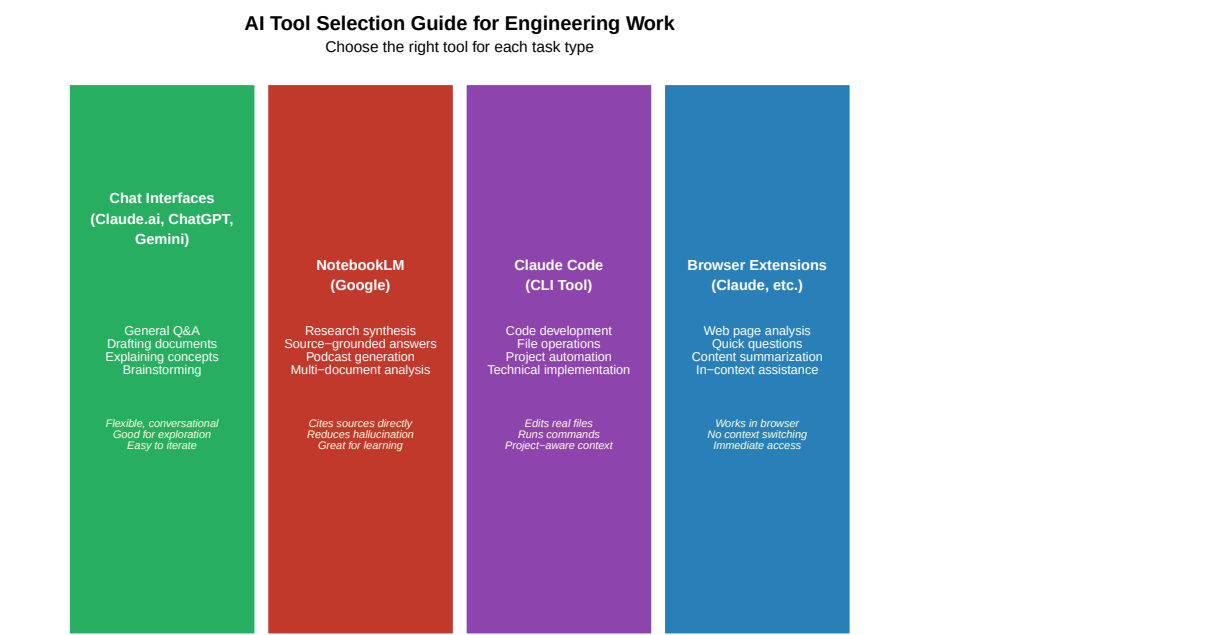
AI is not appropriate when:

1. **Physical inspection is required:** AI cannot see, touch, or measure actual equipment
2. **Real-time safety decisions:** Lockout/tagout procedures, emergency response
3. **Legal certification:** Stamping drawings, signing off on safety assessments
4. **Novel situations:** First-of-a-kind designs with no training data
5. **Classified/proprietary work:** Confidential designs should not be uploaded

Rule of thumb: If a human expert would need to be physically present, AI cannot substitute.

12.8 AI Tools and Their Use Cases

Different AI tools serve different purposes. Understanding when to use each tool maximizes productivity.



12.8.1 Chat Interfaces (Claude.ai, ChatGPT, Gemini)

Web-based conversational AI platforms are the most versatile starting point for AI collaboration.

Table 12.7: Common Engineering Use Cases

Use Case	Description
Technical Explanations	Explain complex concepts, standards, or technical principles in accessible language
Document Drafting	Draft reports, specifications, procedures, or technical memos
Calculation Verification	Double-check hand calculations; request step-by-step solutions
Brainstorming Solutions	Generate alternative approaches to engineering problems
Code Generation	Write PLC ladder logic concepts, Python scripts, or MATLAB code
Standard Interpretation	Interpret requirements from ISO, OSHA, NEC, or other standards

Best Practices for Chat Interfaces:

- Start new conversations for new topics (avoid context pollution)
- Use the “regenerate” feature to see alternative responses
- Upload images of diagrams, schematics, or equipment for visual analysis
- Copy important responses to notes (conversations can be lost)

12.8.2 NotebookLM (Google)

NotebookLM is specifically designed for **source-grounded research**. Upload documents, and the AI answers questions based only on your sources.

Table 12.8: NotebookLM Features for Engineering

Feature	Description
Source-Based Answers	Answers come directly from your uploaded documents, reducing hallucination
Citation Links	Every answer includes clickable citations to the exact source passage
Audio Overview	Generates AI podcast-style audio summaries of your documents for learning
Multi-Document Synthesis	Combines information from multiple PDFs, websites, or text files
Note Generation	Creates structured notes and summaries from source materials

Example NotebookLM Workflow:

1. Upload: ISO 12100, ISO 13849-1, equipment manual PDF
2. Ask: “What risk assessment steps does ISO 12100 require?”
3. Review: AI provides answer with direct citations to document sections
4. Generate: Create an audio overview to reinforce learning

When to Use NotebookLM: When you need answers grounded in specific documents rather than general AI knowledge. Ideal for standards compliance, equipment documentation, and research synthesis.

12.8.3 Claude Code (Command Line Tool)

Claude Code is a **terminal-based AI assistant** that can read, write, and execute code in your actual project environment.

Table 12.9: Claude Code Capabilities for Technical Work

Capability	Description	Example
File Operations	Read, write, and edit files directly in your project	Edit src/motor_control.py to add a new function
Code Development	Generate, refactor, and debug code with full context	Write a Python script to parse CSV data
Command Execution	Run build commands, tests, and system tools	Run pytest and fix any failing tests
Project Context	Understands your entire codebase structure	Find all files that reference the 'config' module
Git Integration	Create commits, branches, and pull requests	Commit changes with descriptive messages

Engineering Applications:

- **PLC Code Analysis:** Review and understand legacy ladder logic or structured text
- **Data Processing:** Create scripts to analyze production data, SPC charts, or sensor logs

- **Documentation Generation:** Auto-generate code documentation or API references
- **Test Automation:** Write unit tests for engineering calculations

Example Session:

```
> claude "Analyze the motor_control.py file and suggest improvements for  
        handling encoder signal noise"
```

Claude Code reads the file, understands the context, and provides specific recommendations with code changes that can be applied directly.

12.8.4 Browser Extensions (Claude Extension, etc.)

Browser extensions provide **instant AI assistance** without leaving your current workflow.

Table 12.10: Browser Extension Use Cases

Use Case	How It Helps	Example
Webpage Summarization	Summarize long technical articles or datasheets instantly	On a 20-page document
Technical Documentation	Ask questions about the documentation page you’re reading	On All About Circuits
Email Composition	Draft professional responses to technical inquiries	‘Help me write an email’
Research Assistance	Explain unfamiliar terms or concepts on any webpage	Highlighting key points
Quick Calculations	Get quick engineering calculations without switching apps	‘Convert 100°C to Fahrenheit’

Best Practices:

- Use for quick queries and summaries
- For complex problems, move to a full chat interface
- Be cautious with confidential information on web pages
- Extensions work best for reading comprehension tasks

12.8.5 Choosing the Right Tool

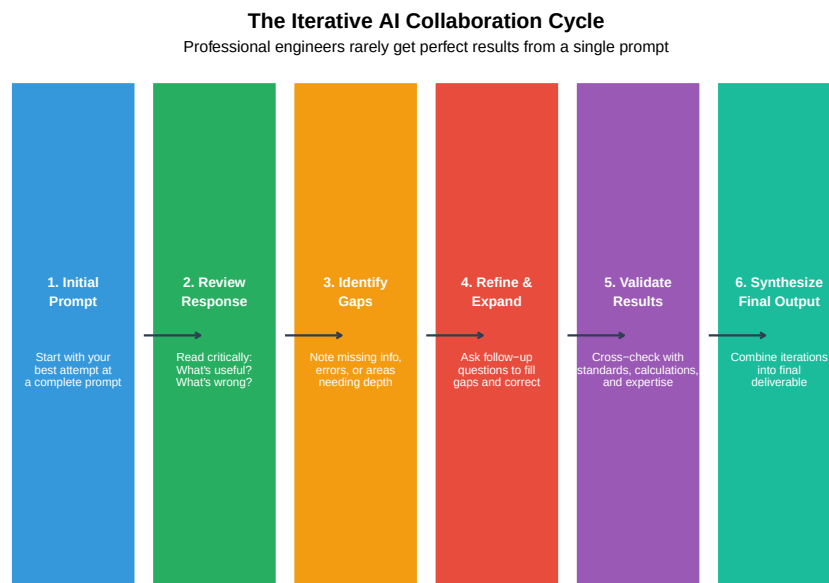
Table 12.11: Tool Selection Quick Reference

Situation	Best Tool	Why
Need to understand a technical concept	Chat Interface	Flexible conversation, can ask follow-up questions
Writing code for a project	Claude Code	Can directly modify files and run code
Researching from specific documents	NotebookLM	Answers grounded in your uploaded documents
Quick question about a webpage	Browser Extension	No context switching, immediate answers
Drafting technical documentation	Chat Interface	Good at structured document generation
Analyzing production data	Claude Code	Can write and execute analysis scripts

12.9 The Iterative Approach: Multi-Turn AI Collaboration

Complex engineering problems are rarely solved in a single prompt. The iterative approach mirrors how you would work with a human expert.

12.9.1 The Iteration Cycle



12.9.2 Worked Example: Designing a Conveyor Control System

Turn 1 - Initial Prompt:

Act as an automation engineer. Design a basic control system for a conveyor belt in a packaging line. The conveyor is 15m long and transports boxes to a palletizing station.

AI Response Summary: Generic overview with basic motor control, start/stop logic, and simple sensors.

Turn 2 - Adding Specificity:

Good start. Now expand with these details:

- Add accumulation zones (3 zones, 5m each) with photo-eye sensors
- Include a reject station at the 10m mark for overweight boxes
- Integrate with an upstream case sealer (need handshake signals)
- Use Allen-Bradley CompactLogix as the controller

Show the I/O list and basic ladder logic structure.

AI Response Summary: Detailed I/O list, zone control logic, sensor placement, and communication requirements.

Turn 3 - Safety and Edge Cases:

Now address:

1. What happens if the downstream palletizer faults?
2. Add E-stop integration per ISO 13849 PLd requirements
3. How do we handle box jams in each zone?
4. What's the startup sequence after E-stop reset?

Include safety relay wiring diagram description.

AI Response Summary: Complete safety integration, fault handling, jam detection, and recovery procedures.

Turn 4 - Final Validation:

Review your complete design and:

1. List any assumptions you made that I should verify
2. Identify potential failure modes you haven't addressed
3. What information would you need to finalize this for implementation?
4. Are there any code standards (ISA, IEC) I should reference?

AI Response Summary: Comprehensive assumption list, remaining questions, and standards references for documentation.

Key Point: This four-turn conversation produced a design that would have required a much longer single prompt to achieve, and the iterative approach allowed for course correction along the way.

Try it yourself: Practice iterative prompting

Your Challenge: Use the four-turn approach to design a temperature monitoring system for a food processing facility.

Turn 1: Define the basic requirements (number of zones, temperature ranges, alerting needs)

Turn 2: Add specifics (sensor types, communication protocol, integration with existing systems)

Turn 3: Address failure modes and safety (what happens if a sensor fails? How are critical temperature excursions handled?)

Turn 4: Request validation questions and identify what you need to verify before implementation

Compare your results: Did the iterative approach give you a more complete design than your initial prompt would have?

12.10 Practice Topics for Iterative AI Sessions

Use these topics to practice the iterative approach. Each topic is designed to require multiple turns to fully develop.

12.10.1 Beginner Level

Table 12.12: Beginner Practice Topics

Topic	Iteration Hint
Motor Starter Selection	Start with application, then add environment, protection requirements
Sensor Selection for Proximity Detection	Begin with what you're detecting, then material, distance, environment
Basic PLC Program for Traffic Light	Define sequence first, then timing, then inputs/outputs, then safety
Belt Conveyor Speed Calculation	Start with throughput requirement, then add physical constraints

12.10.2 Intermediate Level

Table 12.13: Intermediate Practice Topics

Topic	Iteration Hint
VFD Parameter Setup for Pump Application	Define motor and load first, then control requirements, then safety
PFMEA for Automated Assembly Station	Describe process steps, then potential failures, then iterate
Pneumatic Circuit Design with Safety	Start with actuator requirements, then add safety, then valves
HMI Screen Layout for Batch Process	Define operator tasks first, then alarms, then navigation, then safety

12.10.3 Advanced Level

Table 12.14: Advanced Practice Topics

Topic	Iteration Hint
Predictive Maintenance System Architecture	Define assets and failure modes, then sensors, then algorithms
Robot Cell Integration with Vision System	Robot selection first, then vision requirements, then safety
Energy Management System for Manufacturing Plant	Start with loads and usage patterns, then monitoring, then optimization
Complete Machine Safety Assessment	Describe machine function, then hazards, then risk assessment

12.11 Critical Evaluation of AI Outputs

AI can produce confident-sounding responses that are completely wrong. Engineers must develop critical evaluation skills.

12.11.1 Red Flags in AI Responses

Table 12.15: Red Flags in AI-Generated Responses

Red Flag	Example
Calculations Without Units	"The motor needs 150" - 150 what? HP? kW? Nm?
Missing Safety Considerations	Design has no E-stop, no guarding discussion, no lockout procedure
Outdated Standards Referenced	References NEC 2014 when 2023 is current; cites withdrawn IEC standards
Overconfident Language	"This will definitely work" or "This is the only solution"
Generic Without Specifics	"Use appropriate sensors" without specifying type, range, or model
Ignoring Stated Constraints	You specified 24VDC and AI designs for 120VAC

12.11.2 Verification Checklist

Before using any AI-generated engineering content:

Table 12.16: AI Output Verification Checklist

Category	Verification Item	Verified
Calculations		
Calculations	All calculations include units	☐
Calculations	Results are dimensionally consistent	☐
Calculations	Order of magnitude is reasonable	☐
Standards Compliance		
Standards	Referenced standards are current versions	☐
Standards	Applicable regional codes are addressed (NEC, CSA, IEC)	☐
Safety		
Safety	All identified hazards have mitigation	☐
Safety	Failure modes are considered	☐
Safety	Human factors are addressed	☐
Practical Implementation		
Practical	Specified components are available/not obsolete	☐
Practical	Cost and lead time are realistic	☐

12.12 When NOT to Use AI

AI is a tool, not a replacement for engineering judgment. Recognize its limitations.

12.12.1 AI Excels At:

- Generating first drafts and starting points
- Explaining concepts and standards
- Creating documentation templates
- Performing routine calculations (with verification)
- Suggesting troubleshooting approaches
- Comparing options and trade-offs

12.12.2 AI Struggles With:

Table 12.17: Tasks Requiring Human Engineering Judgment

Limitation	Description	Example
Site-Specific Context	AI doesn't know your plant layout, existing equipment, or local practices	Recommendations
Current Conditions	AI cannot inspect, measure, or observe actual equipment conditions	Cannot diagnose
Ethical Judgment	Decisions balancing safety, cost, and risk require human judgment	Choosing between options
Novel Situations	Unprecedented problems may not have training data to draw from	First-of-kind problems
Physical Intuition	Experience-based intuition about what 'feels wrong' in a machine	Recognizing anomalies
Accountability	AI cannot stamp drawings, sign off on safety, or take legal responsibility	A P.Eng. must sign

Quick Check: AI or Human?

For each task below, decide whether AI assistance is appropriate, limited, or inappropriate:

1. **Drafting a maintenance procedure for a conveyor system** → *Appropriate with verification*
2. **Deciding whether to shut down a production line during a safety incident** → *Human only*
3. **Generating a bill of materials from a design specification** → *Appropriate with verification*
4. **Certifying a machine guarding design for compliance** → *Human only (requires P.Eng.)*
5. **Troubleshooting a motor that "sounds funny"** → *Limited - AI can suggest causes, human must inspect*
6. **Explaining the difference between ISO 13849 and IEC 62443** → *Appropriate*

Key insight: AI excels at information synthesis and drafting, but physical observation, legal certification, and real-time safety decisions require human expertise.

12.13 Ethical Considerations

Using AI in engineering work carries professional and ethical responsibilities.

12.13.1 Key Principles

1. **Transparency:** Disclose AI assistance where required by professional standards or employer policy
2. **Verification:** Never submit AI-generated work as final without verification by qualified personnel
3. **Accountability:** You remain responsible for work product regardless of AI assistance
4. **Confidentiality:** Do not input proprietary designs, client data, or trade secrets into public AI systems
5. **Competence:** AI assistance does not substitute for required qualifications or experience

12.13.2 Industry-Specific Considerations

Table 12.18: Industry-Specific AI Ethics Considerations

Industry	Key Ethical/Legal Concern	Guidance
Automotive	Functional safety (ISO 26262) requires traceable human decisions	Document AI use
Food Processing	Food safety regulations (FDA, CFIA) require validated processes	AI can draft, but verify
Defense	ITAR/export controls may prohibit certain AI tool usage	Consult legal counsel
Mining	Worker safety regulations require qualified sign-off	P.Eng. or equivalent

Reflection: Your responsibilities as an AI-assisted engineer

Consider these questions for your own practice:

1. **Disclosure:** Does your employer have a policy on AI tool usage? What about your professional engineering body?
2. **Verification:** What specific steps will you take to verify AI-generated calculations before using them?

3. **Confidentiality:** What types of information from your work should NEVER be input into public AI tools?
4. **Professional development:** How will you continue building expertise rather than becoming dependent on AI for answers?
5. **Liability:** If an AI-assisted design fails, who is responsible? (Hint: It's always the licensed engineer who approved it.)

Remember: AI tools change rapidly, but professional engineering ethics remain constant. Your responsibility for the safety and quality of your work never transfers to a tool.

12.14 Hands-On Lab Exercise

12.14.1 Objective

Practice the complete AI collaboration workflow on a realistic engineering problem.

12.14.2 Scenario

You are tasked with designing an automated inspection station for a food packaging line. The station must:

- Inspect sealed packages for proper seal integrity
- Reject packages with defects
- Track inspection statistics
- Integrate with existing Allen-Bradley PLC system

12.14.3 Instructions

1. **Write your initial prompt** (5 minutes)
 - Apply the seven rules of effective prompts
 - Include role, context, specifications, constraints, and deliverables
2. **Conduct 3-4 iteration turns** (15 minutes)
 - Review each AI response critically
 - Identify gaps and request specific expansions
 - Push for component specifications and safety considerations
3. **Evaluate the final output** (5 minutes)
 - Use the verification checklist
 - Note any red flags
 - Identify what still requires human expertise
4. **Document your process** (5 minutes)
 - What worked well in your prompting approach?
 - What would you do differently next time?

- What human expertise was essential?

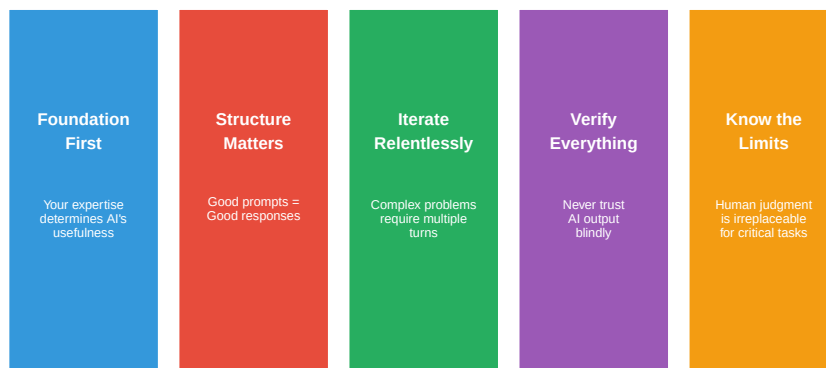
12.14.4 Deliverables

- Initial prompt (graded on structure and completeness)
 - Iteration log showing refinement process
 - Critical evaluation notes
 - Reflection on lessons learned
-

12.15 Summary

Five Principles for Effective AI Collaboration in Engineering

Master these concepts to leverage AI as a powerful professional tool



12.15.1 Key Takeaways

1. **The Education Paradox:** Strong foundational knowledge is required to get quality AI assistance. “AI will do it for me” is a dangerous mindset.
2. **The Four Steps:** Effective AI use follows a consistent process: Define → Prompt → Iterate → Validate. Skipping steps leads to poor results.
3. **Prompt Engineering is a Skill:** Like technical writing, crafting effective prompts requires practice and follows learnable principles.
4. **Choose the Right Tool:** Different AI tools serve different purposes. Chat interfaces for exploration, NotebookLM for source-grounded research, Claude Code for development, and browser extensions for quick assistance.
5. **Iteration is Normal:** Professional engineers should expect 3-6 turns of AI conversation for complex problems.

6. **Critical Evaluation is Essential:** AI produces confident-sounding outputs that may be wrong. Verify everything.
 7. **Human Judgment Remains Central:** AI augments but does not replace engineering expertise, especially for safety-critical decisions.
-

12.16 Review Questions

1. Explain the “education paradox” in AI usage and why it matters for engineering students.
 2. List and describe the four steps of effective AI use. Why is each step important?
 3. List the seven rules of effective engineering prompts and give an example of each.
 4. What are three red flags that should make you question an AI-generated engineering recommendation?
 5. Describe the iterative approach to AI collaboration and why single-prompt interactions are often inadequate.
 6. Compare and contrast the four main types of AI tools (chat interfaces, NotebookLM, Claude Code, browser extensions). When would you use each?
 7. For which types of engineering tasks is AI assistance most appropriate? Least appropriate?
 8. What ethical considerations apply when using AI for engineering work in safety-critical industries?
 9. Transform this bad prompt into a good prompt: “Help me size a pump.”
 10. You need to research a new ISO standard and create a summary for your team. Which AI tool(s) would you use and why?
 11. Why is it important to include standards references in engineering prompts?
 12. How does NotebookLM differ from general chat interfaces like ChatGPT or Claude.ai? When would this difference matter?
-

12.17 References and Further Reading

- Anthropic Economic Index Report (2026) - AI usage patterns in profes-

sional work

- ISO 12100 - Safety of machinery, General principles for design
 - IEC 62443 - Industrial communication networks, Network and system security
 - Professional Engineers Ontario - Use of Artificial Intelligence guidelines
 - OpenAI Prompt Engineering Guide - General prompting techniques
 - IEEE Code of Ethics - Professional responsibilities in technology
-

12.18 Glossary

Prompt Engineering: The practice of designing inputs to AI systems to achieve desired outputs.

Iteration/Multi-turn Conversation: Using multiple exchanges with AI to progressively refine and improve outputs.

Augmentation: Using AI to enhance human capabilities rather than replace them entirely.

Automation: Delegating complete tasks to AI with minimal human intervention.

Hallucination: AI generating plausible-sounding but factually incorrect information.

Context Window: The amount of text an AI can consider at once; important for complex problems requiring extensive background.

Chat Interface: Web-based conversational AI platforms (e.g., Claude.ai, ChatGPT, Gemini) for flexible, general-purpose AI interaction.

NotebookLM: Google's AI tool that provides source-grounded answers based on uploaded documents, with citation links and audio overview generation.

Claude Code: A command-line AI tool that can read, write, and execute code directly in project environments with full file system access.

Browser Extension: AI tools integrated into web browsers for instant assistance without switching applications; useful for webpage analysis and quick queries.

Source-Grounded AI: AI systems that base responses on specific provided documents rather than general training data, reducing hallucination risk.

Chapter 13

AI Tools for Manufacturing Engineering

13.1 Chapter Overview

This chapter runs over six weeks — twelve hours of classroom theory paired with twelve hours of hands-on laboratory work (see Section 19.8). By the end of the six weeks you will have built, trained, and publicly deployed a predictive-maintenance web application using real industrial sensor data. That app will be on your résumé.

The technical path moves from wide to narrow: Week 1 surveys the entire AI landscape and teaches you to use the tools you already have access to (large language models) wisely. Week 2 grounds you in the messy reality of plant data and refreshes your statistical process control. Weeks 3 through 5 build and deploy a machine-learning model for predicting when a turbofan engine will fail — the same class of problem that GE Aviation, Pratt & Whitney, and every major Tier-1 automotive supplier runs on their production lines. Week 6 steps back to examine ethics, the broader landscape, and where this field is heading.

The companion lab handout (Section 19.8) provides the step-by-step exercises, AI-assisted Python workflows, and deliverable instructions for each week.

Tip

A Note on Python in This Course: You are not expected to already know Python, and this course does not teach it as a separate subject. Instead, you will direct an LLM to write, explain, and debug the code for you — increasingly how manufacturing engineers work in industry. Your job is to specify precisely what the code needs to do (the R-C-T-C-F

pattern from Week 1 is built for exactly this), read what the AI produces well enough to catch its mistakes, and ask it to explain any line you don't follow. This is the central skill the course is built around: using AI to (1) automate routine work you already understand, and (2) produce work that is genuinely above your current skill level — punching above your weight — while you stay accountable for whether the result is actually correct.

Before You Start: Think about the last time a machine at your co-op, at home, or on a plant tour broke down unexpectedly. How much warning was there? What would have been different if you had known two weeks in advance that it was going to fail?

13.2 Week 1 — AI Foundations for Manufacturing Engineers

13.2.1 Learning Objectives

	By the end of this week, you will be able to...	Assessment
1.1	Distinguish AI, machine learning, deep learning, and large language models, and give a manufacturing example of each	Self-check Q1–Q2
1.2	Explain in plain language how an LLM generates text and identify three reasons it fails	Self-check Q4
1.3	Determine what plant data is safe to paste into a public LLM and what is not	Self-check Q3
1.4	Write a structured prompt using the R-C-T-C-F pattern that produces useful engineering output	Lab Week 1 Exercise 1

	By the end of this week, you will be able to...	Assessment
1.5	Use an LLM to generate Python code you understand well enough to debug	Lab Week 1 Exercise 4

13.2.2 The AI Landscape

The terms “AI,” “machine learning,” “deep learning,” and “large language model” are used interchangeably in headlines. They are not the same thing. Understanding the actual relationship will save you from choosing the wrong tool — and from being fooled by vendor marketing.

The AI Landscape: A Nested Hierarchy

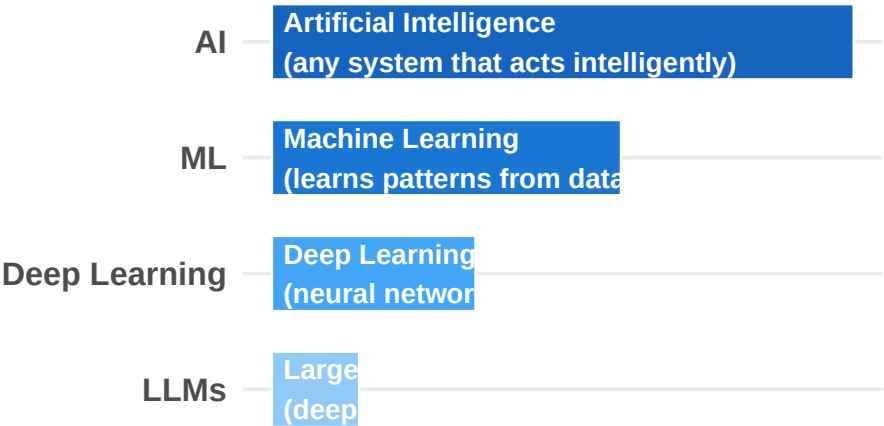


Figure 13.1: The AI landscape is a nested hierarchy. Each inner layer is a specialization of the outer one. Not all AI is machine learning, and not all machine learning is deep learning.

Figure Figure 13.1 illustrates the key insight: every LLM is deep learning, every deep learning system is machine learning, and all machine learning is AI — but the reverse is not true. There are AI systems (like the rule-based expert systems used in 1980s industrial diagnostics) that are not machine learning at all.

13.2.2.1 Machine Learning

Definition: Machine learning is the set of techniques that allow a computer to learn a relationship from data, rather than having a programmer write explicit rules.

Manufacturing example: Your CNC lathe at Précitech has been producing transmission shafts for two years. The QMS contains 50,000 parts with their measured diameters paired with the tool age at the time each part was made. You suspect tool age affects diameter — but how, exactly?

A traditional programmer would guess: “Maybe diameter = $25.000 + 0.0001 \times \text{tool_age}$?” and tune constants by hand for years. Machine learning lets you skip the guessing. You hand the 50,000 examples to a **linear regression** algorithm and it finds the best-fit relationship itself:

$$\text{diameter} \approx 25.0021 + 0.000098 \times \text{tool_age}$$

Give it a new tool age, it predicts the diameter. The algorithm *learned* the slope and intercept from the data.

Technique	Best for	Manufacturing example
Linear regression	Continuous predictions	Predicting diameter from tool age
Logistic regression	Yes/no decisions	Will this part pass inspection?
Decision trees	Mixed numeric/categorical data	Route part to rework vs. scrap
Random forests	Robust tabular predictions	Predicting machine failure (Week 3)
K-means clustering	Grouping similar items	Identifying defect families from inspection data
Support Vector Machines	Classification with clear boundaries	Pass/fail from multiple measurement features

ML is the **right tool** when you have structured tabular data, a clearly defined output to predict, and you want the result to be explainable. It is the **wrong tool** when your inputs are unstructured (images, audio, free text) or when you have very little data.

13.2.2.2 Deep Learning

Definition: Deep learning is machine learning that uses **neural networks** with many layers. The “deep” refers to the number of layers, not philosophical

complexity.

A neural network is inspired (loosely) by neurons in the brain: many simple processing units connected in layers, each taking inputs, multiplying them by learned weights, summing them, and passing the result to the next layer.

Manufacturing example: Automated visual inspection on a CNC turning line. The input is a 1024×768 pixel image — roughly 786,000 numbers. Whether a surface scratch is a defect depends on the *pattern* across those numbers, not any single pixel. This is exactly the problem deep learning excels at. A **convolutional neural network** (CNN) trained on 10,000 labeled images can learn to classify new images with high accuracy. Tools like Roboflow and YOLO make this approachable in an afternoon.

Why didn't deep learning dominate until ~2012? Three things came together: (1) the internet generated enough labeled images to train large networks; (2) GPUs designed for video games turned out to be perfect for the matrix math inside neural networks; (3) researchers solved technical problems that had prevented deep networks from learning effectively. The 2012 ImageNet competition — won by a deep network called AlexNet — is widely considered the start of the modern deep learning era.

13.2.2.3 Large Language Models

Definition: A large language model is a specific kind of deep neural network trained on enormous amounts of text, with the goal of predicting the next word (or token) given everything before it.

That is mechanically what an LLM does: **predict the next token**. Then repeat, using the generated token as part of the input for the next prediction. This is called **autoregressive generation**.

"The cutting tool needs to be" → LLM → "replaced"
 "The cutting tool needs to be replaced" → LLM → "because"
 ... and so on

The model has no plan, no goal, no understanding — just “what token probably comes next, given all the text I’ve seen?” Yet because predicting the next word *well* requires absorbing grammar, facts, the structure of engineering documents, and the way Python code is written, the implicit “knowledge” encoded in billions of parameters is surprisingly broad.

What LLMs are good at:

- Drafting documents in standard formats (emails, reports, SOPs, FMEAs)
- Summarizing long documents
- Translating between languages
- Explaining concepts at a chosen level of detail
- Generating code in common languages (Python, SQL, R, MATLAB)
- Brainstorming options to evaluate

- Restructuring information (table $\rightarrow$ prose, prose $\rightarrow$ table)

What LLMs are bad at:

- Arithmetic — they predict text, not numbers
- Citing real sources — they invent plausible-sounding citations
- Knowing your specific equipment, processes, or customers
- Real-time information (a model with no web access cannot tell you today's stock price)
- Consistency — the same question twice can produce different answers
- Safety-critical work where the cost of error is high

13.2.2.4 Side-by-Side Comparison

Property	Machine Learning	Deep Learning	Large Language Models
Typical input	Tabular numbers	Images, audio, sensor streams	Text
Typical output	Number or category	Category or another image	More text
Training data needed	Hundreds to thousands	Tens of thousands to millions	Trillions of words (already done — you use a pre-trained model)
Can you train it in this course?	Yes (Week 4)	No (follow-on course)	No — you use someone else's
Best at	Sensor predictions, quality, scheduling	Vision, sound, complex pattern recognition	Documents, explanations, code generation, summarization
Explainability	Often good (readable formula or rules)	Poor (millions of parameters)	Very poor (billions of parameters)
Manufacturing examples	Predictive maintenance, yield prediction	Defect detection from images	Drafting SOPs, summarizing reports, generating Python code

Industry Application: At Ford's Dearborn Truck Plant, AI systems monitor thousands of sensor channels from body-welding robots. Anomaly detection (a form of ML) runs continuously on each weld. A separate LLM-based assistant helps maintenance planners draft work orders from natural-language descriptions of symptoms. Both tools are in production — they solve different problems, and neither replaces the engineer who interprets the output.

The big takeaway: **use the simplest tool that solves your problem.** A linear regression you fully understand is often better than a neural network you don't. An LLM is excellent for drafting documents but a terrible choice for predicting tool wear from sensor data — that is a job for ML.

13.2.3 How LLMs Actually Work — Enough to Use Them Well

You do not need to know the mathematics. You do need a mental model of what happens when you type into a chat box.

13.2.3.1 Tokens

LLMs do not see words — they see **tokens**, pieces of words. The word “manufacturing” might be one token or two (“manu” + “facturing”), depending on the model’s tokenizer.

A rough rule: **1 token** $\approx$ **¾ of an English word** ($\approx$ 4 characters). So a 1,000-word document is roughly 1,300 tokens.

You can see this for yourself at the OpenAI Tokenizer (<https://platform.openai.com/tokenizer>). Try tokenizing common engineering terms (“Cpk,” “AIAG,” “PFMEA”) versus natural language. Technical jargon gets chopped into many small pieces; common words become single tokens. This affects model performance on highly specialized text.

13.2.3.2 Context Windows

The **context window** is how much text the model can “see” at once — your entire conversation, including previous messages and its own previous outputs. Modern models have generous windows (hundreds of thousands of tokens, or roughly 500 pages), but free-tier offerings often have smaller limits.

When you paste a 50-page document and ask for a summary, all 50 pages go into the context window. If the document exceeds the window, the oldest content gets silently dropped in some implementations. This is why **focused prompts produce better results than document dumps.**

13.2.3.3 Training Cutoff and Outdated Knowledge

The model was trained on data up to a specific date — its **training cutoff**. After that date it knows nothing. If you ask about a software version released last month, an event from yesterday, or a standard revised last quarter, the model either says it does not know (good) or fabricates something plausible (bad).

Some tools integrate web search to extend the model’s knowledge to real time. Free tiers may or may not include this. Always check.

13.2.3.4 Sampling and Why Answers Vary

LLMs do not deterministically pick the single most-likely next token. They sample from a probability distribution, introducing randomness. Ask the same question twice and you can get different answers — sometimes very different ones. For engineering work: **do not treat one LLM answer as definitive**. Ask the same question two or three different ways and look for consistency.

13.2.4 Why LLMs Fail

Three failure modes every engineer must internalize.

13.2.4.1 Hallucinations

Warning

Critical Concept: A hallucination is when the model produces text that is grammatically correct, confidently stated, and completely false. This is not a bug — it is a direct consequence of how the model works.

The model predicts plausible-sounding text. When asked about something niche or unfamiliar, it still generates text that *sounds* like the right kind of answer, with no internal “do I actually know this?” check.

Classic engineering hallucinations:

- Inventing ASTM, ISO, or SAE standard numbers that do not exist
- Citing papers that were never written, attributed to real researchers
- Stating cutting parameters for alloys with complete confidence and no source
- Describing features of commercial software that the software does not have

You will deliberately trigger a hallucination in Lab Week 1, Exercise 3. That exercise is designed to create a lasting memory — after you have personally caught an LLM confidently lying, you will never paste an LLM citation into a technical memo without checking.

Defence: Verify anything that matters. If the model cites a standard, look it up. If it gives a number, find the source. Use the model to *draft*, then *verify* — never to do final research.

13.2.4.2 Math Errors

LLMs are language models, not calculators. They learn to produce text that *looks like* arithmetic — but they do not compute internally. Simple sums often come out right; multi-step calculations frequently do not.

If you ask an LLM to compute Cpk given a mean, sigma, USL, and LSL, you may get the right answer, the wrong answer, or a wrong answer with a correct-looking setup. **Do not trust LLM arithmetic on safety-critical decisions.** Use the LLM to set up the formula; do the calculation yourself or with a verified script.

Some models can call an external Python interpreter to handle math reliably. If your tool offers this capability, use it.

13.2.4.3 Sycophancy

LLMs are trained to be helpful and agreeable. If you push back on a correct answer, the model often capitulates and gives you a wrong one. If you confidently state something incorrect, the model frequently agrees.

This is dangerous for technical review. Do not ask “is my reasoning here correct?” — that biases the model toward yes. Instead ask “what is wrong with this reasoning?” or “what would a critic say?” You will get genuinely useful pushback.

13.2.5 Intellectual Property Risk in Industrial Settings

Warning

Career Risk: Mishandling intellectual property with AI tools is a fireable offence at most manufacturers. Read this section twice.

The default assumption to adopt: anything you paste into a public LLM (Claude.ai free tier, ChatGPT free tier, Google Gemini, free Copilot) may be:

- Retained by the provider
- Used to improve future model versions
- Accessible to the provider’s employees
- Stored in a jurisdiction with different data protection laws than yours

Even when the provider’s policy says they do not use your data for training, your data has still left your plant’s network. For many things this is acceptable. For some things it is a contract violation and a privacy breach.

What never to paste into a public LLM:

- Customer names, contracts, drawings, or specifications

- Proprietary process parameters — feeds, speeds, coolant concentrations developed over years
- Anything export-controlled (in Canada, the Controlled Goods Program; in the US, ITAR/EAR — defense-adjacent manufacturers are commonly affected; **when in doubt, ask compliance**)
- Employee personal data
- Source code with embedded credentials or secrets
- Drawings or models marked “Confidential” or “Proprietary”
- Anything you would not send to your competitor’s email address

Real cases:

Case	Year	What happened
Samsung	2023	Engineers pasted source code into ChatGPT to debug it. Samsung banned internal LLM use company-wide within a month.
Federal court sanctions	2023–2024	Multiple lawyers filed court briefs containing fabricated case citations generated by ChatGPT. Judges imposed monetary sanctions.
Air Canada chatbot	2024	A BC tribunal ruled Air Canada liable for false information given by its customer-service chatbot. Companies are responsible for what their AI tools say.

Safe alternatives:

Need	Safe approach
Draft a generic SOP template	Abstract your situation; do not paste proprietary procedures
Summarize a long technical document	Redact identifying details, or use an enterprise tool with contractual data-protection guarantees
Debug code that touches sensitive systems	Strip credentials, names, and paths; reproduce the bug in a minimal isolated example

Need	Safe approach
Engineering work with real customer or defense data	Use only your employer’s approved enterprise LLM (Claude Team/Enterprise, ChatGPT Enterprise, Microsoft Copilot for Business), or a locally-hosted model (Ollama, LM Studio)

13.2.6 Prompt Engineering — The R-C-T-C-F Pattern

A prompt is the text you send to the model. A good prompt produces useful output on the first try. A bad prompt produces generic output that requires multiple rounds of refinement, wasting your time and the model’s context window.

The **R-C-T-C-F pattern** is a five-element structure designed for engineering prompts:

Element	Meaning	Example
Role	Who should the LLM act as?	“You are a quality engineer at an ISO 9001 automotive Tier-1 supplier.”
Context	What background does the LLM need?	“We run a CNC lathe producing transmission shafts at 25.000 ± 0.050 mm.”
Task	What do you want it to do?	“Review the following FMEA and identify the three highest-RPN failure modes.”
Constraints	What should it avoid or limit?	“Limit to 200 words. Do not invent risks not in the document. Cite RPN values explicitly.”
Format	How should the output look?	“Output as a numbered list. Each item: failure mode, RPN, current mitigation status.”

Not every prompt needs all five elements, but missing more than two usually produces noticeably worse output. Compare these:

Bad prompt: “explain SPC”

Produces a generic textbook definition that is not tailored to any audience, purpose, or length.

R-C-T-C-F prompt: “You are explaining statistical process control to a new operator on a CNC turning line. They have a high-school math background and have never heard of standard deviation. In under 150 words, explain what SPC is, why we use it, and what they need to do at their station. Use one analogy from daily life. Output as plain prose, no bullets, no headings.”

Produces a focused, audience-appropriate, length-controlled explanation that the operator can actually use.

13.2.6.1 Patterns That Work in Engineering

- **“Show your reasoning before the answer.”** Forces the model to think step-by-step, which reduces errors on multi-part problems.
- **“What might be wrong with this approach?”** Provides a critical second opinion on your own thinking. Avoids sycophancy.
- **“Generate three different versions.”** Options beat a single answer on open-ended problems.
- **“What questions should I have asked but didn’t?”** Surfaces gaps in your problem framing.
- **“Cite your sources, and if you don’t have a source, say so.”** Partial defence against hallucinations. Not foolproof, but it forces the model to flag uncertainty.

13.2.6.2 Patterns That Do Not Work

- Yes/no questions on judgment calls — you get hedged, useless answers
- Pasting massive documents without a focus question — output will be generic
- “What’s the best way to...” — there is no best; give constraints instead
- Asking for specific numerical recommendations on safety-critical decisions — verify everything independently

Lab Preview: In Week 1 Lab (Section 19.8), you will rewrite five real bad prompts from manufacturing engineers using R-C-T-C-F, deliberately trigger and document a hallucination, and use an LLM to generate your Week 2 Python starter code. The code you generate this week becomes your starting point next week — so write the prompt carefully.

13.2.7 Self-Check Questions — Week 1

Work through these in your own words before the Week 2 lecture. If you find yourself looking answers up in these notes rather than recalling them, re-read

the relevant section.

1. Why is “predict the next word” enough to produce text that summarizes a document, drafts code, and explains engineering concepts?
2. A colleague says “we use AI to predict tool wear.” What would you need to ask them to determine whether they mean classical ML, deep learning, or an LLM?
3. Your supervisor pastes a customer’s machined-part drawing into the free tier of ChatGPT to “ask what the tolerances mean.” What three problems do you raise with them, ranked by severity?
4. What is the difference between an LLM making a math error and an LLM hallucinating? Give an engineering example of each.
5. Rewrite this prompt using R-C-T-C-F: “*summarize this report for the boss.*”

13.2.8 Further Reading — Week 1

- Google Cloud. *AI vs. machine learning*. <https://cloud.google.com/learn/artificial-intelligence-vs-machine-learning>
- MIT Sloan. *Machine learning, explained*. <https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained>
- Google Developers. *Machine Learning Crash Course* (free, online). <https://developers.google.com/machine-learning/crash-course>
- 3Blue1Brown. *Neural Networks* video series (start with the first video). <https://www.3blue1brown.com/topics/neural-networks>
- Hugging Face. *LLM Course, Chapter 1* (free, online). <https://huggingface.co/learn/llm-course/chapter1/1>
- OpenAI. *Tokenizer* (interactive demo). <https://platform.openai.com/tokenizer>
- Anthropic. *Prompt Engineering Overview*. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
- Wikipedia. *Hallucination (artificial intelligence)*. [https://en.wikipedia.org/wiki/Hallucination_\(artificial_intelligence\)](https://en.wikipedia.org/wiki/Hallucination_(artificial_intelligence))
- TechCrunch. *Samsung bans use of generative AI tools like ChatGPT*. <https://techcrunch.com/2023/05/02/samsung-bans-use-of-generative-ai-tools-like-chatgpt-after-april-internal-data-leak/>

13.3 Week 2 — Manufacturing Data and Statistical Thinking

13.3.1 Learning Objectives

	By the end of this week, you will be able to...	Assessment
2.1	Name the five levels of the ISA-95 plant data hierarchy and identify which system type belongs at each level	Self-check Q1
2.2	Identify and categorize common data quality problems in an industrial dataset	Lab Week 2 Task 2
2.3	Direct an LLM to construct an X-bar control chart in Python and interpret whether a process is in statistical control	Lab Week 2 Task 3
2.4	Calculate Cpk and explain its meaning to a non-engineer	Self-check Q3
2.5	Articulate four limitations of SPC and explain how machine learning addresses them	Self-check Q4–Q5

Before You Read: In your last co-op or lab, how did data get from a machine to a spreadsheet? How many steps were involved, and at how many steps could something go wrong?

13.3.2 Where Plant Data Comes From

Every modern manufacturing facility generates data from at least five separate IT systems. You will be expected to work with all of them.

13.3.2.1 The ISA-95 Plant Data Hierarchy

This is the **ISA-95 hierarchy** (Figure Figure 13.2). Different rules apply at different levels: you absolutely do not deploy Python scripts on a PLC (Level 1), but you absolutely do deploy them at Level 3 or 4. The boundary between OT and IT is one of the most important distinctions in manufacturing engineering — you will encounter it again in Week 5.

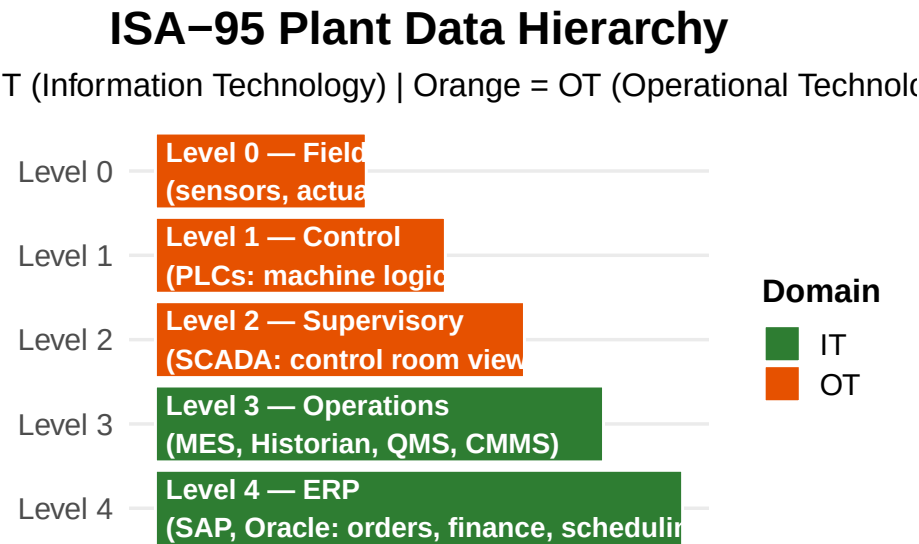


Figure 13.2: The ISA-95 data hierarchy. Levels 0–2 are Operational Technology (OT); Levels 3–4 are Information Technology (IT). Python scripts belong at Level 3 or higher — never on a PLC.

System	Full Name	What It Stores	Typical Sample Rate
PLC	Programmable Logic Controller	Real-time sensor and actuator state	1 ms – 100 ms
SCADA	Supervisory Control and Data Acquisition	Aggregated control-room view	1 s – 10 s
Historian	(AVEVA PI, GE Proficy, Ignition)	Long-term time-series sensor data	1 s – 1 min, compressed
MES	Manufacturing Execution System	Production events, cycle starts, alarms, OEE	Per event
QMS	Quality Management System	Inspection results, dispositions, NCRs	Per part or per sample

System	Full Name	What It Stores	Typical Sample Rate
CMMS	Computerized Maintenance Management System	Work orders, downtime reasons, parts replaced	Per event
ERP	Enterprise Resource Planning (SAP, Oracle)	Orders, materials, finance, scheduling	Per transaction

13.3.2.2 Four Types of Plant Data

The Week 2 lab uses **quality data** (Type 3 below). Know all four:

1. **Time-series / process data** — continuous sensor streams: temperature, pressure, vibration, motor current. Comes from PLCs via SCADA or the historian. High sample rates. *The dominant data type for predictive maintenance (Week 3).*
2. **Batch / event data** — discrete events with timestamps. “Cycle started at 08:00:03.” “Alarm raised at 14:22:11.” Comes from the MES.
3. **Quality / inspection data** — measurements of finished parts: diameters, surface finish, hardness, pass/fail flags. Comes from the QMS. *This is what your Week 2 CSV contains.*
4. **Maintenance data** — work orders, breakdowns, scheduled maintenance, parts replaced. Comes from the CMMS. Often the worst-quality data in the plant — heavily dependent on technicians typing notes correctly.

Industry Application: At a Tier-1 stamping plant in Windsor, the quality team needed to correlate surface defects with press tonnage. The defect data was in the QMS; the tonnage data was in the historian — sampled 10× faster, in UTC, with a different part identifier scheme. Reconciling the two datasets took two days of work before any analysis could begin. This is typical.

13.3.3 Why Industrial Data Is Messy

Five problems appear so consistently in plant data that you should expect all of them until proven otherwise.

The 80/20 rule is real: In real industrial data work, you will spend 80% of your time making the data trustworthy and 20% doing the analysis. The 20% gets the conference talks. The 80% is what makes you employable.

13.3.3.1 Missing Values

Sensors fail. Networks drop. Maintenance happens. People forget to log. The result is gaps in your data. The Week 2 lab dataset has two working days missing entirely due to a simulated network outage.

Response	When to use	Risk
Drop the rows	Few rows missing at random	Lost information
Forward fill (last known value)	Slowly changing variables; brief gaps	Wrong if the variable changed during the gap
Linear interpolation	Smooth continuous variables	Manufactures data that may not exist
Leave as NaN	When honest gaps must stay honest	Requires downstream code to handle NaNs
Investigate and fill from another source	Always preferred	Requires legwork

The right answer for the lab’s two missing days: document it, note it, move on. **Never silently delete data.**

13.3.3.2 Unit Confusion

Warning

Lesson from History: Unit confusion crashed the Mars Climate Orbiter in 1999. NASA assumed Lockheed Martin’s thrust calculations were in newton-seconds; they were in pound-force seconds. The orbiter entered the Martian atmosphere at the wrong angle and burned up. \$327 million lost.

In your plant, the cases are smaller but more frequent. A surface roughness gauge outputs readings in either inches or micrometres depending on a setting. An operator on day shift uses one; the night shift operator uses the other. The CSV column is labelled `surface_roughness_um` — but a fraction of the values are actually in inches. Silent corruption.

How to detect: ask whether the *range* of values is physically plausible. Surface roughness of 0.00001 μm is impossible (atomic-scale). That same number in inches is a reasonable Ra value. Once you spot the range issue, you can find the unit error and convert.

13.3.3.3 Sample Rate Mismatches

A vibration sensor records at 10,000 Hz. The MES logs cycle starts at 1 Hz. To join “vibration during each cycle,” you cannot simply align timestamps —

you must aggregate the vibration data per cycle (mean? RMS? peak?) before joining. This is rarely covered in undergraduate statistics but appears constantly in plant work.

13.3.3.4 Timestamp Problems

A short list of real timestamp problems:

- PLC clock drifts 30 seconds per day; nobody resyncs it
- Historian stores in UTC; MES stores in local time; CSV export converts back without telling you
- Daylight saving creates one duplicate hour each fall and one missing hour each spring
- Timezone-naïve timestamps from different systems merged silently
- Operator-entered timestamps rounded to the nearest 5 minutes

Whenever joining two data sources, plot the timestamp distribution and look for gaps, duplicates, and step discontinuities before doing anything else.

13.3.3.5 Outliers — Three Flavours

Type	What it is	What to do
Sensor error	Physically impossible value (negative time, 250 mm diameter on a 50 mm part)	Remove or flag — this never happened
Real special cause	A real bad part, a real event you need to know about	Keep it. This is the most important data in your dataset.
Tail of common-cause variation	A real measurement that happens to be far from the mean	Keep it. It is part of natural variation.

The hardest case is distinguishing a real special cause from a sensor error. The rule of thumb: if you can find a physical or process explanation, it is special cause. If the value is physically impossible, it is sensor error. If neither, it is natural variation.

The Week 2 lab contains a row with `diameter_mm = 250.123`. The Mazak QT-200 chuck will not grip a 250 mm bar — this must be a sensor error, likely a missing decimal point (true value probably 25.0123). The lab walks you through finding and handling this.

13.3.4 Statistical Process Control

SPC was developed by Walter Shewhart at Bell Labs in the 1920s. A century later it is still the dominant approach to quality monitoring in nearly every manufacturing industry. You will encounter it on day one of your career.

13.3.4.1 The Central Idea: Two Kinds of Variation

Every process produces variation. No two parts are identical. Shewhart's fundamental insight: variation has two causes.

Common-cause variation is the inherent noise of a stable process — slight differences in material, temperature, vibration, operator technique — all small, all unavoidable, all individually unpredictable but collectively predictable. A “stable” process produces *only* common-cause variation.

Special-cause variation is when *something has changed*: a tool broke, coolant ran out, a new batch of material arrived, a temperature controller failed. Each special cause produces a noticeable, often non-random pattern in the data.

The job of SPC is to **detect special-cause variation as quickly as possible** so you can investigate and fix it before it produces bad parts.

13.3.4.2 Control Charts and Control Limits

A **control chart** is a plot of a process metric over time with computed control limits. If a point falls outside the limits, the process is presumed out of statistical control — there is a special cause to investigate.

The two types you need to know:

- **X-bar chart** (with R or S chart): for subgroups. Take n parts at a time (typically $n = 4$ or 5), compute the subgroup mean, plot the means over time. Pair with a range or standard deviation chart.
- **Individuals chart** (I-chart): for one-at-a-time inspection, where you measure every part.

13.3.4.3 Control Limits vs. Specification Limits

This is the most-confused distinction in undergraduate SPC. Get it right:

- **Specification limits (USL, LSL)** come from **engineering or the customer**. They describe the design tolerance. These limits have nothing to do with your data — they were set when the part was designed.
- **Control limits (UCL, LCL)** come from **the data itself**. They describe how the process *actually behaves*. They are computed from the process mean and standard deviation.

A process can be **in control but out of spec** — behaving consistently, but consistently making bad parts. A process can be **out of control but in spec**

Control Chart — Précitech CNC Cell 3 (

Specification limits (engineering tolerance) | Orange = control limits

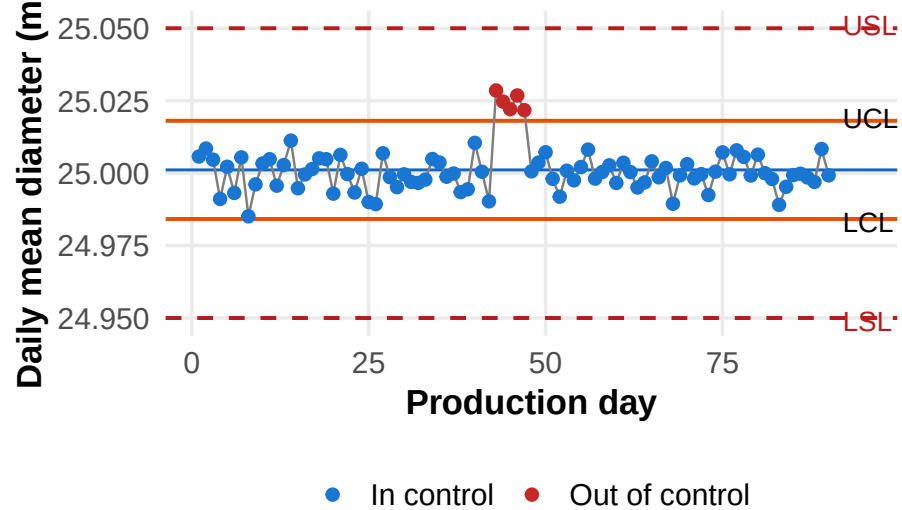


Figure 13.3: Simulated X-bar control chart for daily mean diameter over 90 production days. A special-cause event (bad bar stock) is visible around days 43–47 — the same event you will find in the Week 2 lab dataset.

— most parts are good, but something has changed and you should investigate before parts start failing.

	In spec	Out of spec
In statistical control	Ideal — stable and capable	Consistent but bad — fix the process design
Out of statistical control	Lucky for now — find the special cause before parts go OOS	Emergency — bad parts being made now

13.3.4.4 The X-bar Control Limit Formula

When you plot **subgroup means** rather than individual measurements, the means themselves vary less than individual parts — by a factor of $\sqrt{n}$ (the central limit theorem). The control limits on an X-bar chart are therefore:

$$UCL_{\bar{X}} = \bar{\bar{X}} + 3 \cdot \frac{\sigma}{\sqrt{n}} \quad (13.1)$$

$$LCL_{\bar{X}} = \bar{\bar{X}} - 3 \cdot \frac{\sigma}{\sqrt{n}} \quad (13.2)$$

where σ is the standard deviation of *individual* parts, n is the subgroup size, and $\bar{\bar{X}}$ is the overall process mean.

A common student mistake: computing control limits using the standard deviation of the *daily means* instead of the standard deviation of individual parts. If the daily means have standard deviation $s_{\bar{x}}$, then $s_{\bar{x}} \approx \sigma/\sqrt{n}$ — so using $s_{\bar{x}}$ instead of $\sigma/\sqrt{n}$ effectively applies the $\sqrt{n}$ correction twice. The control limits become too narrow, generating false alarms constantly.

13.3.4.5 Process Capability: C_p and C_{pk}

SPC tells you whether your process is *stable*. **Process capability** tells you whether a stable process is *capable* of meeting specifications.

$$C_p = \frac{USL - LSL}{6\sigma} \quad (13.3)$$

$$C_{pk} = \min \left(\frac{USL - \mu}{3\sigma}, \frac{\mu - LSL}{3\sigma} \right) \quad (13.4)$$

where σ is the standard deviation of *individual* measurements (not the X-bar), and μ is the process mean.

- C_p is the *potential* capability — how good the process could be if perfectly centred between USL and LSL
- C_{pk} is the *actual* capability — accounts for off-centre processes
- $C_{pk} \leq C_p$ always; they are equal only when the process is perfectly centred

Process Capability — Précitech CNC Cell 3 (Si

Red shaded area = predicted fraction of out-of-spec parts

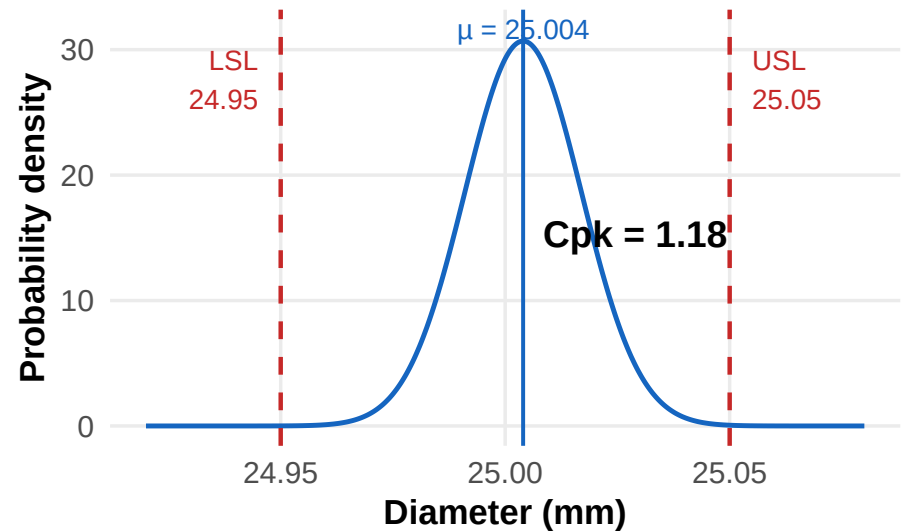


Figure 13.4: Process capability visualization. A C_{pk} of 1.16 means the nearest specification limit is 3.48 from the process mean — better than minimum (1.00) but below the automotive standard (1.33). The shaded tail area under each specification limit represents the expected fraction of non-conforming parts.

Industry C_{pk} targets (your customer’s PPAP requirements are authoritative; these are typical):

Industry	Typical C_{pk} target	Notes
Automotive (general)	1.33	AIAG PPAP common requirement
Automotive (safety-critical: brakes, steering, airbags)	1.67	
Aerospace	1.67 – 2.00	Often combined with 100% inspection
Medical devices	1.67 or higher	Plus FDA regulatory requirements

Industry	Typical C_{pk} target	Notes
General machining (non-critical)	1.00 – 1.33	Typical starting point for new processes

The Week 2 lab dataset gives $C_{pk} \approx 1.16$. That is marginal — a real plant would treat this as “improvement project needed” rather than “ready for sustained production.”

13.3.5 Where SPC Fails

SPC is essential but not sufficient. Four situations expose its limits:

1. **Non-normal data:** The 3-sigma control limits assume an approximately normal distribution. For bounded measurements (geometric tolerances near zero, defect counts), the distribution may be skewed or bimodal. Raw SPC limits will produce false alarms or miss real shifts.
2. **Autocorrelated data:** Tool wear is the perfect example. Today’s part diameter depends on yesterday’s — the tool did not reset overnight. SPC assumes each measurement is independent. When this assumption is violated, control limits become misleading.
3. **Slow drift:** A 0.001 mm/day diameter drift is too slow to trigger a 3-sigma rule in any single sample. By the time SPC detects it, you may have produced months of slowly degrading parts.
4. **Multiple operators or shifts blended together:** Three operators with different setup biases, blended into one chart, makes the common-cause variation appear inflated — hiding real operator-to-operator differences.

13.3.5.1 The Transition to Machine Learning

SPC tells you **when** a process went out of control. It does not tell you **when it will go out of control** or **what will cause it**. Those are predictive questions requiring a different toolset.

Tool	What it tells you	How fast
SPC	Something IS wrong right now	After it happens
Machine learning	Something WILL go wrong, and when	Before it happens

Both have a role. SPC is your safety net. ML is your early-warning system. The best operations deploy both. Week 3 introduces the predictive side.

13.3.6 Self-Check Questions — Week 2

1. A new colleague plots the last 90 daily mean diameters on a control chart, using ± 3 of the daily means as control limits. What is wrong with this, or is it correct? Explain.
2. Your CMM reports surface flatness in mm. The drawing specifies inches. The CSV column header says “mm.” Three days of data have suspiciously small values. What likely happened, and how would you detect it without anyone telling you?
3. A process has $C_{pk} = 0.85$. The control chart shows no out-of-control points. The plant manager declares the process “in control” and refuses your improvement project. What is the technically correct response?
4. Why does tool-wear autocorrelation violate the assumptions of a standard X-bar chart?
5. Name the four levels of the ISA-95 hierarchy and give one example of data that lives at each level.

13.3.7 Further Reading — Week 2

- NIST/SEMATECH. *e-Handbook of Statistical Methods, Chapter 6: Process Monitoring*. <https://www.itl.nist.gov/div898/handbook/pmc/pmc.htm>
 - ASQ. *Statistical Process Control Overview*. <https://asq.org/quality-resources/statistical-process-control>
 - ASQ. *Process Capability — Cp & Cpk Overview*. <https://asq.org/quality-resources/process-capability>
 - ISA. *ISA-95 Standards Overview*. <https://www.isa.org/standards-and-publications/isa-standards/isa-standards-committees/isa95>
 - Walter Shewhart, *Economic Control of Quality of Manufactured Product* (1931). Available at <https://archive.org/details/economiccontrolo0000shew>
-

13.4 Week 3 — Machine Learning and Predictive Maintenance

13.4.1 Learning Objectives

	By the end of this week, you will be able to...	Assessment
3.1	Compare the four maintenance regimes and justify when each makes economic sense	Self-check Q5
3.2	Sketch the P-F curve and explain what the P-F interval means for maintenance scheduling	Self-check Q4
3.3	Describe the structure of the NASA C-MAPSS dataset and explain how RUL is computed from run-to-failure data	Lab Week 3 Task 1
3.4	Explain what a Random Forest is and why it is the default first choice for tabular industrial data	Self-check Q1, Q3
3.5	Identify the engine-wise train/validation split requirement and explain why row-wise splitting causes data leakage	Self-check Q1

Before You Read: Recall from Week 2 that SPC told you *when* a process went out of control — only after it happened. What would it be worth to your plant if you knew the bearing on a CNC spindle was going to fail in three weeks, rather than finding out when it seized mid-shift?

13.4.2 The Economics of Maintenance

Before any technical content, the *why*. Maintenance strategies fall along a spectrum, and the economics drive every business decision.

Regime	What it means	Cost profile	When it makes sense
Reactive (“run to failure”)	Wait until it breaks, then fix it	Low day-to-day cost; catastrophic when failure hits	Non-critical equipment where failure has no downstream impact
Preventive (scheduled)	Replace parts at fixed intervals regardless of condition	Predictable; wastes remaining useful life	Industry default for decades; still appropriate for cheap, fast-to-replace components
Predictive / CBM (condition-based)	Replace based on actual condition, monitored continuously	Higher up-front investment; minimizes waste and surprise	Capital-intensive, downtime-sensitive operations
Prescriptive	System recommends <i>what</i> to do, not just <i>when</i>	Highest technology investment; emerging	Sites with mature digital infrastructure and reliable ground-truth feedback

The shift from preventive to predictive is the central economic story of Industry 4.0.

13.4.2.1 The Financial Case for Predictive Maintenance

Consider a bearing on the spindle of a CNC lathe at Précitech.

Figure 13.5 shows why every plant manager wants predictive maintenance. For a 10-machine cell with one unplanned bearing failure per machine per year, the plant loses roughly CAD \$178,000 annually on this single component type. A predictive model catching 70% of these failures before they become functional failures would save approximately CAD \$125,000 per year — from one component, on one cell.

13.4.2.2 The P-F Curve

The most important concept in maintenance theory:

Different sensor types detect failure at different positions on the P-F curve (Figure 13.6):

Bearing Failure Event by Maintenance Reg

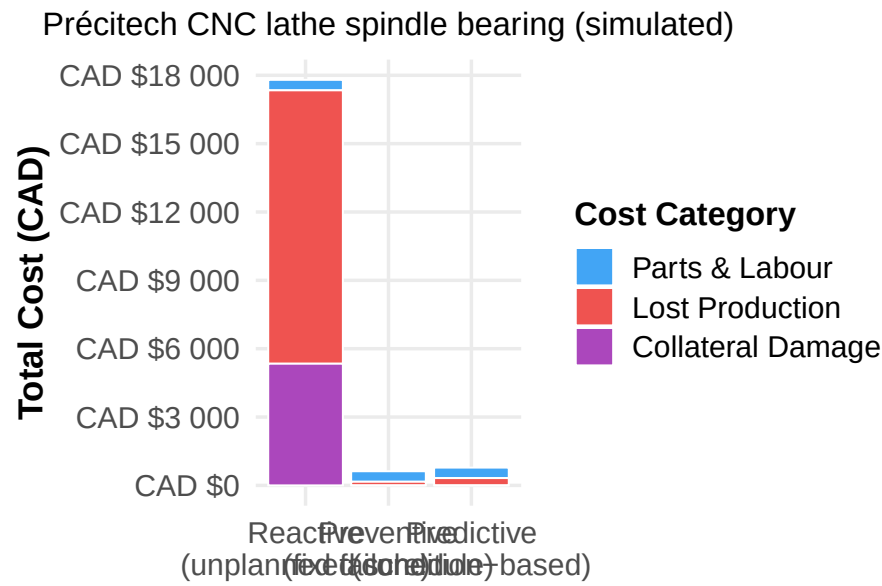


Figure 13.5: Cost comparison for one bearing failure event across three maintenance regimes. Predictive maintenance avoids the bulk of the unplanned downtime cost by catching the failure before it becomes a functional failure.

The P-F Curve

Different sensor types detect failure at different positions on the

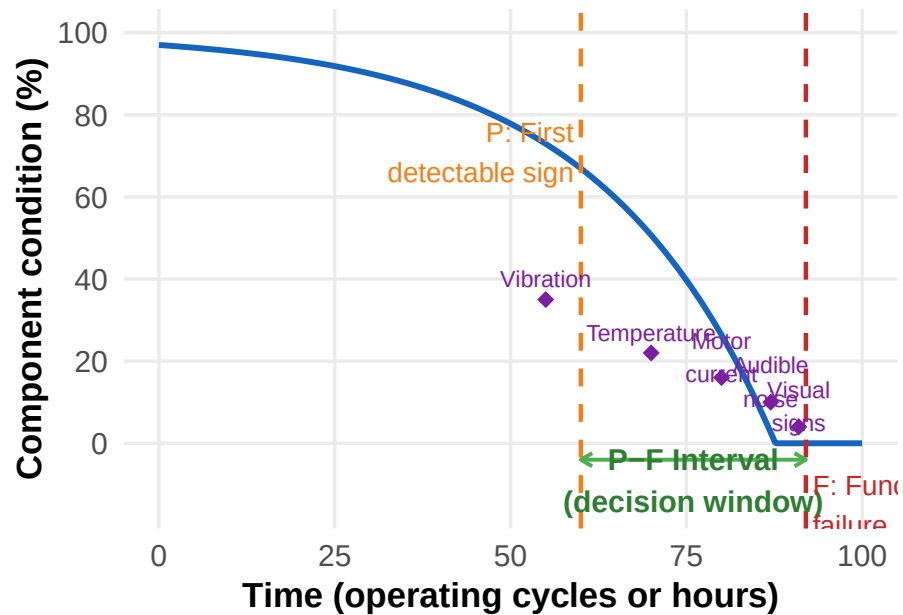


Figure 13.6: The P-F curve: a stylized view of how a machine component's condition degrades from normal operation to functional failure. The P-F interval — the time between when failure becomes detectable (P) and when function is lost (F) — is the maintenance engineer's decision window. Different sensor types detect failure at different positions along the curve.

Sensor type	Typical position on P-F curve	Approximate P-F interval
Vibration analysis	Earliest — often months before functional failure	Months to weeks
Temperature monitoring	Later — bearing temperature rises as wear progresses	Weeks to days
Motor current signature	Even later — current draw increases as friction rises	Days to hours
Audible noise (human ear)	Late	Hours to minutes
Visual smoke or sparks	Functional failure imminent	Minutes

Predictive maintenance models exploit the **earliest** indicators (vibration, subtle sensor drift) to maximize the decision window. This is why the C-MAPSS dataset you will work with uses 21 sensors per cycle — many of them recording subtle thermodynamic shifts that no human operator would notice.

Industry Application: GE Aviation’s OnPoint predictive maintenance program for CFM56 engines (the most common commercial airliner engine in service) uses sensor data from every flight to continuously update engine health models. Airlines receive maintenance recommendations before any symptom is observable on the shop floor. This has reduced unscheduled removals by over 30% on participating fleets.

13.4.3 The NASA C-MAPSS Dataset

Your capstone project uses a real research benchmark — the same dataset used in thousands of academic papers since 2008.

13.4.3.1 What It Represents

C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) is a high-fidelity simulation of a turbofan jet engine — the kind that powers commercial airliners. NASA used it to generate datasets of run-to-failure trajectories: simulated engines run from a healthy state until they fail, with 21 sensors recorded at each operating cycle.

The data comes in four variants (FD001 through FD004) with different combinations of operating conditions and fault modes. **FD001** is the simplest — one operating condition, one fault mode, 100 training engines, 100 test engines. This is what your capstone uses.

13.4.3.2 Data Structure

Each row of the training file represents one operating cycle of one engine:

Column	Meaning	Notes
<code>unit_number</code>	Engine ID (1–100)	Resets for each engine
<code>time_in_cycles</code>	Operating cycle number	Starts at 1 for each engine; ends at failure
<code>op_setting_1</code> to <code>op_setting_3</code>	Operating conditions (altitude, throttle, etc.)	Mostly constant in FD001
<code>sensor_1</code> to <code>sensor_21</code>	Sensor readings (temperatures, pressures, fan speed, fuel flow, bleed enthalpy, etc.)	Some constant; some highly predictive

There is **no RUL column** in the training data. You compute it:

$$\text{RUL} = (\text{max cycle for this engine}) - (\text{current cycle})$$

If engine #1 ran for 192 cycles total, its RUL at cycle 1 is 191, at cycle 100 is 92, and at cycle 192 (the failure cycle) is 0.

Quick Check: If you plot RUL for all 100 training engines stacked together (all rows in the file), what shape would the RUL values form? Think about it before reading ahead — you will verify your answer in Lab Week 3, Task 5.

13.4.4 The Machine Learning Problem: From Data to Prediction

13.4.4.1 Supervised Learning

Your capstone uses **supervised learning** — the most common and commercially important category of machine learning.

Supervised learning requires four things:

1. **Inputs (features):** measurements available at prediction time. For C-MAPSS: 21 sensors and 3 operating settings, plus engineered features you create.
2. **Outputs (labels):** what you want to predict, paired with each input row. For C-MAPSS: RUL.
3. **A model:** a mathematical function that maps inputs to outputs.
4. **A training algorithm:** a procedure for finding the model parameters that best fit many (input, output) pairs.

The supervised learning problem for C-MAPSS is:

Given this engine’s current and recent sensor readings, predict how many more operating cycles it will run before failure.

This is **regression** — predicting a continuous number — not classification.

13.4.4.2 Feature Engineering

A single sensor reading at a single cycle tells you the current state but not the *trend*. Two engines might have the same temperature reading today; one is on its 50th cycle, the other on its 150th. They are at very different points in their degradation trajectory.

Feature engineering creates new variables from raw ones to expose the patterns the model needs to learn:

Feature type	What it captures	Example
Rolling mean (window = 5)	Local average; smooths noise	Average of last 5 sensor readings
Rolling std	Local variability; increases as machine degrades	Variability of last 5 readings
Rolling min / max	Local extremes	Highest temperature in the last 5 cycles
Delta / rate of change	Speed of drift	Today’s reading minus yesterday’s
Domain-specific	Physics-based features	Vibration RMS in a specific frequency band

For C-MAPSS, rolling mean and rolling standard deviation on each informative sensor (window = 5) is the single highest-leverage feature engineering step. It typically reduces model RMSE from approximately 30 to approximately 20 cycles.

A critical implementation detail: rolling features must be computed **per engine**, with the window resetting at the boundary of each engine’s data. If you apply rolling statistics across the full dataset without grouping by engine, the last few cycles of engine #1 will contaminate the rolling window of engine #2’s first few cycles — a subtle but serious bug.

13.4.4.3 Clipping the RUL Label at 125

C-MAPSS engines run from healthy to failure, but “healthy” doesn’t really degrade. An engine with 200 cycles left looks essentially identical to one with 300 cycles left — both are healthy. The model wastes capacity trying to distinguish them.

Standard practice on C-MAPSS research: clip RUL labels at 125. Any engine healthier than 125 cycles is labelled “125 — basically healthy.” The model then focuses all of its capacity on the degradation phase, where the actual predictive signal is.

This simple change typically reduces RMSE by approximately 30% — a clear case where understanding the physics of your problem lets you frame it better for the algorithm.

13.4.5 The Fatal Mistake: Row-Wise Train/Test Splitting

Warning

The Most Common Bug in Industrial ML: Do NOT split rows randomly between training and validation. Do not. This is the single most important technical lesson of the capstone.

Why is row-wise splitting a mistake for this dataset? Because rows from the same engine are not independent. Engine #1’s cycle 50 and cycle 51 are nearly identical — they share the same machine, the same wear state, the same operating history. If cycle 50 is in training and cycle 51 is in validation, the model effectively sees the validation set during training. The validation score becomes meaninglessly optimistic.

The correct approach is **engine-wise splitting**:

Figure 13.7 illustrates the correct split. The validation set contains entire engine histories that the model has never seen during training. This simulates the real deployment situation: a new engine arrives at your plant; your model must predict its RUL without having seen this specific engine before.

How to detect the bug if you accidentally use row-wise splitting: your validation RMSE will be suspiciously low — 5 to 8 cycles instead of the expected 15 to 25. When you see that, immediately check whether any engine ID appears in both training and validation sets. If it does, you have data leakage.

13.4.6 Random Forest — Intuition and Why It Works

Many algorithms can solve the RUL regression problem. Your capstone uses **Random Forest** because it is the strongest first-try algorithm for tabular industrial data — robust, fast, requires almost no tuning, and provides interpretable feature importance rankings.

Engine-Wise Train/Validation Split — C-MAPSS

one engine (entire run-to-failure history). Blue = training. Red =

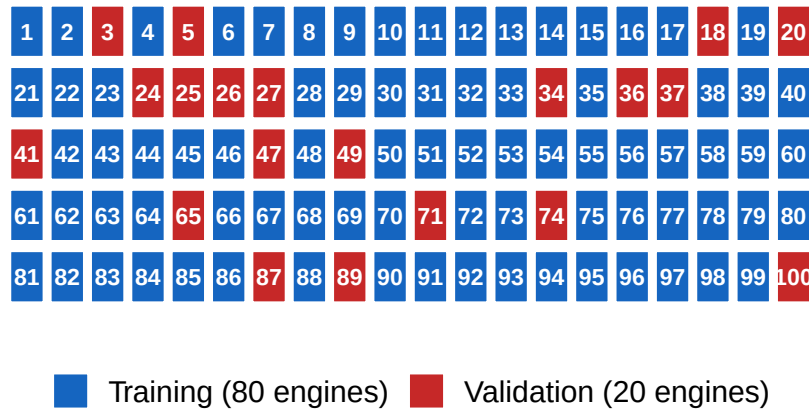


Figure 13.7: Correct engine-wise train/validation split. Entire engine histories go to either training or validation — never split an engine's cycles across the two sets. This simulates the real deployment situation: can your model predict RUL for an engine it has never seen?

13.4.6.1 Step 1: A Single Decision Tree

A **decision tree** is a flowchart of yes/no questions about your features, with a prediction at each leaf node. The questions and their split thresholds are *learned from the data* — the algorithm picks the questions that best separate training rows by their RUL values.

Strengths of a single tree: - Trivially interpretable — you can read the exact rules - Handles mixed numeric and categorical inputs without normalization - Fast to train and predict

Critical weakness: - A deep tree memorizes training data and generalizes poorly to new engines. It learns the noise, not the signal.

13.4.6.2 Step 2: Many Trees, Averaged

Random Forest’s key idea: build many trees, each slightly different, and average their predictions.

Two sources of randomness make each tree unique: 1. **Row bagging:** each tree is trained on a random subset of rows, sampled with replacement 2. **Feature subsets:** at each split point, each tree considers only a random subset of features

Because each tree has a different random view of the data, they make different mistakes. When you average their predictions, the individual errors partially cancel out. The ensemble is far more accurate than any single tree and much less prone to overfitting.

This technique — averaging many slightly-wrong models — is called an **ensemble method**. You will see ensembles everywhere in industrial ML.

Why Random Forest is the default first-try for tabular plant data:

Property	Why it matters in manufacturing
Handles mixed feature scales without normalization	Real plant data has wildly different units (mm, °C, RPM, bar); not normalizing breaks gradient-based methods but not RF
Robust to outliers	One bad sensor reading does not corrupt the whole model
Captures non-linear relationships and interactions	Real processes are non-linear; RF learns this automatically
Provides feature importance	Tells you which sensors matter — directly useful for instrumentation decisions
Fast to train, even on large datasets	You can iterate quickly in a lab session
Almost no hyperparameter tuning needed for a solid baseline	<code>n_estimators=100</code> , <code>max_depth=10</code> gets you 90% of the way there

13.4.7 Evaluation — Measuring Whether a Model Is Useful

13.4.7.1 RMSE and MAE

For regression, two standard metrics:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \quad (13.5)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2} \quad (13.6)$$

Both are in the same units as your target (operating cycles). The key difference:

- **MAE** treats all errors equally. Being off by 10 cycles contributes 10× less than being off by 100.
- **RMSE** penalizes large errors disproportionately. Being off by 100 cycles contributes 100× more than being off by 10.

For maintenance applications, large errors are dangerous — predicting 200 cycles remaining when the real answer is 20 means the engine fails unexpectedly. **RMSE is therefore the primary metric** for predictive maintenance. MAE provides a complementary picture of typical, everyday error.

13.4.7.2 The Naive Baseline

The simplest possible predictor: always predict the mean RUL of the training set, regardless of any sensor inputs. This is the model that learns nothing.

If your sophisticated model cannot beat the naive baseline by at least 2× on RMSE, your model is not doing useful work. For C-MAPSS FD001:

Model	Approximate RMSE
Naive baseline (predict mean)	~50 cycles
Linear regression on raw sensors	~35 cycles
Random Forest on rolling features	~20 cycles
Tuned deep learning (LSTM)	~12–15 cycles

Your capstone target is approximately 20 cycles RMSE with Random Forest — comparable to published results on this benchmark using the same approach.

13.4.7.3 Asymmetric Errors in Maintenance Scheduling

A real plant does not care equally about errors in both directions. If you predict 50 cycles remaining and the true answer is 30, the engine fails before maintenance — a catastrophic, costly miss. If you predict 50 and the true answer is 70, you replaced a component with life left in it — wasteful but safe.

Specialized loss functions can encode this asymmetry (the original C-MAPSS competition used a custom scoring function that penalized late predictions far more than early ones). Your capstone uses symmetric RMSE for simplicity, but this is a known simplification of the real industrial problem.

13.4.8 Self-Check Questions — Week 3

1. A colleague's Random Forest achieves 5-cycle RMSE on C-MAPSS FD001 with default settings. You are suspicious. What is the most likely cause?
2. Why does clipping RUL at 125 improve model performance? Explain in terms of where the signal in the data actually is.
3. Why is a single decision tree usually worse than a Random Forest, even though they both start from the same idea?
4. What is the P-F interval? Give a concrete example from a manufacturing plant, naming the component, the potential failure indicator, and a realistic interval length.
5. A maintenance manager says “we replace the spindle bearings every 2,000 hours regardless.” Which of the four maintenance regimes is this? When does it make economic sense, and when does it not?

13.4.9 Further Reading — Week 3

- A. Saxena, K. Goebel, D. Simon, and N. Eklund, “Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation,” *Proc. PHM08*, Denver CO, October 2008. (The original C-MAPSS paper.)
- NASA Prognostics Data Repository. <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>
- scikit-learn. *Ensemble methods: Random Forests*. <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>
- Kaggle C-MAPSS dataset mirror. <https://www.kaggle.com/datasets/behrad3d/nasa-cmaps>
- Wikipedia. *Reliability-centered maintenance*. https://en.wikipedia.org/wiki/Reliability-centered_maintenance
- Andriy Burkov, *The Hundred-Page Machine Learning Book* (2019). Compact, rigorous, covers exactly what you need.
- PHM Society annual conference proceedings (free online). <https://www.phmsociety.org/>

13.5 Week 4 — Feature Engineering and Model Building

13.5.1 Learning Objectives

	By the end of this week, you will be able to...	Assessment
4.1	Direct an LLM to implement per-engine rolling-window features in Python (<code>groupby().transform()</code>), and verify the result is correct	Lab Week 4 Task 1
4.2	Justify the choice of window size and the effect of different rolling statistics on model performance	Self-check Q2
4.3	Direct an LLM to build a complete Random Forest pipeline (features → split → train → evaluate → compare to baseline), checking each stage as it goes	Lab Week 4 Tasks 3–5
4.4	Interpret a feature importance chart and explain what it says about which sensors matter	Lab Week 4 Task 6
4.5	Direct an LLM to save a trained scikit-learn model to disk for later deployment	Lab Week 4 Task 7

Before You Read: In Week 3 you identified which C-MAPSS sensors are most correlated with RUL. Does the *current value* of a sensor fully describe where an engine is in its degradation trajectory — or do you need to know the *history* of that value as well? Why?

13.5.2 Feature Engineering for Time-Series Data

A single sensor value at a single moment tells you the state of the engine *right now*, but not how it got there. Consider two engines, both showing `sensor_11 = 47.3` this cycle. One has been slowly climbing from 44.0 over the past 20 cycles; the other has been stable at 47.3 for 100 cycles. They are in very different situations — the first is degrading, the second is not. A model that sees only the current value cannot distinguish them.

Feature engineering extends the model’s view backwards in time, giving it the **trend information** it needs.

13.5.2.1 Rolling Statistics

The most powerful features for time-series prediction are computed over a rolling window — a moving neighbourhood of the most recent k cycles:

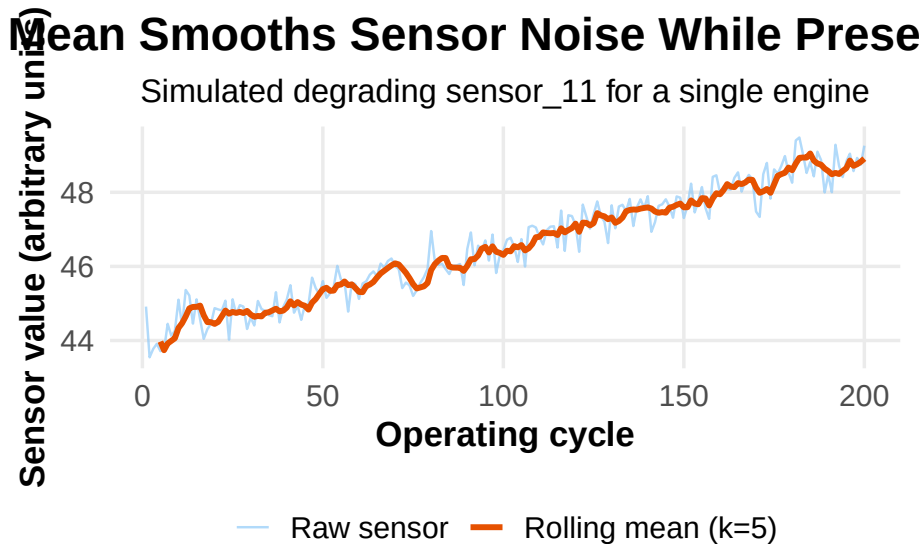


Figure 13.8: Effect of a rolling mean (window = 5) on a simulated degrading sensor signal. The rolling mean (orange) smooths cycle-to-cycle measurement noise while preserving the underlying degradation trend — exactly what the model needs to see.

Feature	How to compute	What it adds
Rolling mean ($k = 5$)	Average of the last 5 readings	Smooth estimate of current sensor level; reduces noise

Feature	How to compute	What it adds
Rolling std ($k = 5$)	Std deviation of the last 5 readings	Quantifies local variability — often increases as machine degrades
Rolling min / max	Min or max of the last 5 readings	Captures worst-case readings in the recent window
Delta (difference)	Current reading minus previous reading	Rate of change — useful for detecting acceleration of degradation
Lagged values	Reading from k cycles ago	Explicit history for the model

Window size matters: a window that is too narrow ($k = 2$) adds noise rather than smoothing it. A window that is too wide ($k = 50$) lags so far behind the current state that it misses fast-moving events. For C-MAPSS FD001, $k = 5$ is a well-validated choice. If you have time after the core capstone, try $k = 10$ or $k = 20$ and observe the effect on RMSE.

13.5.2.2 The `groupby().transform()` Pattern

When you ask an LLM to write your rolling-feature code, this is the single most important thing to check in what it hands back — and exactly the kind of subtle, easy-to-miss error you are training yourself to catch (Learning Objective 1.5):

```
CORRECT: window resets at each new engine boundary
df["sensor_11_rolling_mean"] = (
    df.groupby("unit_number")["sensor_11"]
      .transform(lambda x: x.rolling(5, min_periods=1).mean())
)

WRONG: window bleeds across engine boundaries
df["sensor_11_rolling_mean"] = df["sensor_11"].rolling(5).mean()
```

The wrong version treats the entire dataset as one continuous time series. At the boundary between engine #1 and engine #2 in the file, the last 4 rows of engine #1's history bleed into engine #2's first few rolling features. This is a subtle bug — your model will still train and produce predictions, but the features are contaminated, and your RMSE will be slightly worse than it should be. Worse, in a production deployment on a new engine, the contamination disappears, and the model behaves differently than it did during validation.

If you are not sure which version an LLM just gave you, ask it directly: “does this rolling window reset at each engine boundary — walk me through what

happens at the boundary between unit_number 1 and unit_number 2.” A good answer explains the mechanism; a hallucinated one will just reassure you it’s fine.

13.5.3 Random Forest Internals

Now that you are about to direct an LLM to train one, you need to know what is happening inside the `RandomForestRegressor` call — otherwise you cannot judge whether its code, or its explanation of the results, is right.

13.5.3.1 Growing a Single Tree

A decision tree is grown by the following process:

1. Start at the root node with the entire training dataset
2. At each node, consider every feature and every possible split point
3. Choose the split that minimizes the impurity of the two resulting subsets (for regression: usually variance reduction)
4. Recurse on each subset until a stopping criterion is met (maximum depth, minimum samples per leaf, etc.)

This is a greedy algorithm — it picks the best split at each node without backtracking. The result is a tree that perfectly fits training data if allowed to grow deep enough, but overfits badly: it memorizes noise alongside signal.

13.5.3.2 The Ensemble Solution

Random Forest addresses overfitting with two sources of randomness:

1. **Bootstrap aggregating (bagging):** each tree is trained on a bootstrap sample — a random sample of the training data *with replacement*, same size as the original. Roughly 37% of rows are left out of each tree’s training set (the “out-of-bag” samples), which can be used for internal validation.
2. **Feature subsampling:** at each split, the algorithm considers only a random subset of features (typically $\sqrt{p}$ features for classification, $p/3$ for regression, where p is the total number of features). This forces trees to differ from each other even when one feature is dominant.

Because each tree sees a different random view of the data, the trees make different mistakes. When you average 100 different-but-correlated predictions, the random errors cancel out while the shared signal remains. The bias is roughly the same as any single tree; the variance drops roughly by a factor of n_{trees} . This is the core statistical argument for ensemble methods.

Group Discussion: Feature importance in a Random Forest is computed as the average reduction in node impurity contributed by each feature across all trees. If two features are highly correlated (say, `sensor_11` and `sensor_4`

both drift together as the engine degrades), how might this affect the feature importance values for each? Would removing one of them hurt or help model performance?

13.5.3.3 Hyperparameters You Need to Know

Hyperparameter	Meaning	Typical value for C-MAPSS	Effect of increasing
<code>n_estimators</code>	Number of trees	100	More trees = lower variance, slower training
<code>max_depth</code>	Maximum depth per tree	10	Deeper = more expressive but higher overfitting risk
<code>min_samples_leaf</code>	Minimum samples per leaf node	1–5	Higher = simpler trees, less overfitting
<code>max_features</code>	Features considered per split	“sqrt” or 1/3 of total	Fewer = more diverse trees
<code>n_jobs</code>	Parallel threads	-1 (use all CPU cores)	More = faster training (no accuracy effect)

For C-MAPSS FD001, `n_estimators=100`, `max_depth=10`, `n_jobs=-1` with default feature selection gives you approximately 90% of the achievable performance with zero tuning effort. This is the value of Random Forest for industrial first-pass analysis.

13.5.4 Evaluation Done Right

13.5.4.1 Reading Residual Plots

A single RMSE number does not tell you where your model is failing. Always produce a **predictions vs. actuals scatter plot** (often called a residual plot):

- Points along the diagonal ($\hat{y} = y$) = perfect predictions
- Points above the diagonal = overestimates (predicted too much life remaining — dangerous in maintenance)
- Points below the diagonal = underestimates (predicted too little life — results in premature replacement, wastes money but is safe)
- Systematic patterns (e.g., all high-RUL points overestimated) = model bias at specific RUL ranges

For the C-MAPSS model you build in Week 4, a common pattern is that predictions are pulled toward the centre of the RUL distribution — the model overestimates RUL for engines close to failure, and underestimates it for very healthy engines. RUL clipping at 125 reduces the second problem significantly.

13.5.4.2 Comparing to the Naive Baseline

Ask your LLM to add this comparison to your pipeline — this is the kind of code you should be able to read and explain back, even if you did not type it yourself:

```
Naive baseline: predict the training set mean for every engine
y_naive = np.full_like(y_val, fill_value=y_train.mean(), dtype=float)
naive_rmse = np.sqrt(mean_squared_error(y_val, y_naive))

rf_rmse = np.sqrt(mean_squared_error(y_val, y_pred))

print(f"Random Forest - RMSE: {rf_rmse:.2f}")
print(f"Naive baseline - RMSE: {naive_rmse:.2f}")
print(f"Improvement: {naive_rmse / rf_rmse:.1f}× better than naive")
```

Target: **at least 2× better than the naive baseline.** If you are not hitting 2×, the most likely causes are: row-wise instead of engine-wise split (recheck the assert), missing rolling features, or constant sensors not removed.

13.5.5 Self-Check Questions — Week 4

1. Your validation RMSE is 6.8 cycles. The naive baseline RMSE on the same validation set is 48 cycles. Is this a good result or should you be suspicious? Why?
2. You are choosing between rolling windows of $k = 3$, $k = 5$, and $k = 20$ for your C-MAPSS features. What would you try first, and what would you monitor to decide whether a different value is better?
3. You have 42 features after feature engineering (raw sensors + rolling mean + rolling std, with constants removed). At each split in a Random Forest tree, how many features does the algorithm consider? Why not all 42?
4. A classmate's feature importance chart shows `sensor_14_rolling_mean` at 0.12 and `sensor_14` at 0.03. Why might the rolling feature dominate the raw feature despite measuring the same physical quantity?
5. You save `rul_model.pkl` but forget to save `feature_cols.pkl`. What happens when you try to use the model in the Streamlit app next week?

13.5.6 Further Reading — Week 4

- scikit-learn. *RandomForestRegressor API documentation*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor>.

- html
- scikit-learn. *Feature importances with a forest of trees*. https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html
 - Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed., 2022). Chapter 7 covers ensemble methods with worked examples.
 - Kaggle. *Feature Engineering* micro-course (free, online). <https://www.kaggle.com/learn/feature-engineering>
 - Wikipedia. *Bootstrap aggregating (bagging)*. https://en.wikipedia.org/wiki/Bootstrap_aggregating

13.6 Week 5 — Deploying AI in a Manufacturing Context

13.6.1 Learning Objectives

	By the end of this week, you will be able to...	Assessment
5.1	Explain the ISA-95 OT/IT boundary and identify where AI inference fits in a plant network	Self-check Q1
5.2	Describe why a Jupyter notebook is not a production deployment	Self-check Q2
5.3	Design a user-facing dashboard appropriate for an operator, a maintenance planner, and a plant manager	Self-check Q3
5.4	Explain three mechanisms by which a deployed ML model degrades over time	Self-check Q4
5.5	Direct an LLM to deploy a machine learning model as a public web application using Streamlit and GitHub	Lab Week 5

Before You Read: Your Random Forest model works perfectly in your notebook. A plant manager asks you to “put it on the shop floor.” What does that actually mean? Who will use it, on what device, and what happens when the model is wrong?

13.6.2 The OT/IT Boundary

Recall the ISA-95 hierarchy from Week 2. The boundary between Operational Technology (OT, Levels 0–2) and Information Technology (IT, Levels 3–4) is one of the most consequential lines in manufacturing.

Why AI models do not go on PLCs (Level 1):

- PLCs run deterministic, real-time control loops with cycle times in the millisecond range. A Python inference call that takes 50 ms would break the control loop and create safety hazards.
- PLCs are certified for functional safety under IEC 61508/61511. Adding a Python runtime would invalidate certification and create liability.
- PLCs run on embedded operating systems with no Python interpreter. Deploying there requires rewriting in IEC 61131 structured text or C.
- PLCs are maintained by OT teams with different update cadences and change management processes than IT systems.

Where AI models actually live in production:

Location	Typical role	Latency requirement
Edge server (on-premises, Level 3)	Real-time inference on process data; anomaly detection with millisecond or sub-second response	< 100 ms
Plant server / MES (on-premises, Level 3)	Batch inference (hourly or daily); maintenance scheduling recommendations	Seconds to minutes
Cloud / analytics platform (Level 4 or above)	Training, retraining, dashboards, historical analysis	Minutes to hours
OT-isolated analytics zone	Air-gapped from both OT and enterprise IT; common in defense and pharma	Varies

Edge vs. cloud tradeoffs:

Consideration	Edge (on-premises)	Cloud
Latency	Milliseconds (local)	Hundreds of milliseconds to seconds
Bandwidth	No external bandwidth consumed	Requires sending sensor data out of plant
IP/security	Data never leaves facility	Data leaves facility; security controls required
Reliability	Works during internet outage	Depends on connectivity
Cost	Hardware investment; IT overhead	Pay-per-use; lower up-front cost
Updates	Manual deployment; careful change management	Continuous deployment possible

Your capstone’s Streamlit app will be deployed to the public cloud (Streamlit Community Cloud), which is appropriate for a student project. A real plant deployment for sensitive industrial data would use an on-premises edge server or a private cloud with contractual data-protection guarantees.

13.6.3 Dashboards and Operators

A model that lives in a notebook and only an engineer can interpret is not delivering business value. The value comes when the right person gets the right information at the right time in a form they can act on.

The critical principle: the user of a predictive maintenance dashboard is rarely the engineer who built the model. Three very different users need three very different views of the same predictions:

User	What they need	Information format
Line operator	“Is this machine OK right now? What should I watch for?”	Single colour-coded status: green / yellow / red. Maximum 5 words of text.
Maintenance planner	“Which machines need attention in the next 2 weeks? In what order?”	Sortable list of assets with predicted RUL, confidence bands, recommended maintenance window

User	What they need	Information format
Plant manager	“What is the overall fleet health? Are we at risk of missing our production schedule?”	Aggregate fleet health index, downtime risk forecast, trend over the past 30 days

Good operator-facing dashboards:

- Never show a number without a unit and a context (“47 cycles” means nothing; “47 cycles remaining 3 weeks at current production rate” is actionable)
- Never show model confidence intervals to operators — they create uncertainty without context
- Always include a “what to do” recommendation, not just a prediction
- Always have a visible timestamp (“Last updated: 06:47”)
- Never fail silently — if the data feed is down, say so explicitly rather than showing stale values

Industry Application: Toyota’s production system principle of *andon* — giving any worker the authority to stop the line by pulling a cord when they detect a problem — translates directly to AI dashboards. A well-designed predictive maintenance dashboard makes it as easy as pulling an andon cord to flag a concern for a maintenance technician. The goal is not to have the AI make the decision; it is to surface information so the human can make a better decision faster.

13.6.4 Model Drift and the Production AI Lifecycle

13.6.4.1 Three Mechanisms of Model Drift

1. **Input drift (covariate shift):** The distribution of input features changes over time. Sensors are recalibrated, replaced with different-accuracy units, or repositioned. The model was trained on the old distribution and now receives systematically different values. The model is not wrong — the world changed. Detect this by monitoring the mean and variance of key features over time and comparing to training-set statistics.
2. **Concept drift:** The relationship between inputs and the target variable changes. A major spindle rebuild changes the wear pattern of the engine; the degradation trajectory looks different from the training data. The model learned the old relationship. Detect this by monitoring ground-truth RUL when you eventually get it (after planned or unplanned maintenance) and comparing to predictions.

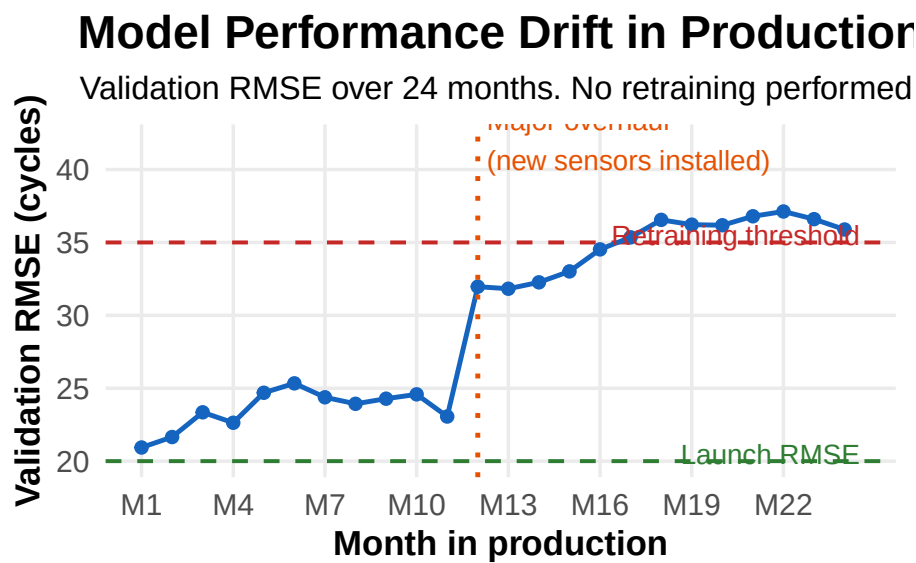


Figure 13.9: Simulated model performance degradation over time in production. Three mechanisms contribute: input drift (sensors recalibrated or replaced), concept drift (machine wear patterns changed after a major repair), and data quality drift (sensor failures increasing over time). Without monitoring and periodic retraining, a model that worked well at launch quietly degrades.

3. **Data quality drift:** The fraction of missing or corrupted sensor readings increases over time as sensors age. The model was trained on clean data; it now receives increasingly dirty inputs. Detect this by monitoring the rate of missing values and out-of-range sensor readings in the incoming data stream.

13.6.4.2 The Production ML Lifecycle

Phase	Actions	Responsible
Training	Collect and clean historical data; engineer features; train and validate model; document performance and assumptions	Data/ML engineer
Deployment	Package model; create inference endpoint; connect to data pipeline; set up monitoring	MLOps / IT
Monitoring	Track input distributions, prediction distributions, ground-truth error when available, data quality metrics	Data engineer + domain expert
Retraining	Triggered by drift detection or scheduled interval; retrain on updated data; validate against holdout; A/B test against production model	Data/ML engineer
Retirement	When the process changes fundamentally enough that the model assumptions no longer hold; decommission cleanly; notify users	Engineering management

Retraining cadence: there is no universal answer. Considerations: - How fast does the process change? A high-volume production line changes faster than a low-cycle heavy equipment fleet. - How expensive is labelled data? For C-MAPSS this is free (simulation). For a real plant, you only get the “true RUL” label after the equipment actually fails or is removed. - How expensive is

a wrong prediction? Higher stakes require more frequent validation and lower drift tolerance.

A practical starting point for most manufacturing predictive-maintenance applications: monthly validation against recent ground-truth events, with a scheduled quarterly retraining trigger and an event-triggered retraining trigger if RMSE exceeds $2\times$ the launch baseline.

13.6.5 Self-Check Questions — Week 5

1. A colleague proposes running the Random Forest inference directly on the PLC to minimize latency. What are three specific reasons this is not feasible?
2. What is the difference between “the model works in my notebook” and “the model is deployed in production”? Name three things that change when you move from notebook to production.
3. An operator sees this on their screen: “Remaining Useful Life: 47.3 ± 12.8 cycles.” Is this a good interface design? What would you change and why?
4. Your model was trained in June. It is now the following February. The plant ran through a major scheduled overhaul in October where 30% of the sensors were recalibrated. What type of drift are you most concerned about, and how would you detect it?
5. Describe three monitoring metrics you would track in a production deployment of your C-MAPSS model to catch problems before they cause bad maintenance decisions.

13.6.6 Further Reading — Week 5

- Streamlit documentation. <https://docs.streamlit.io>
 - Streamlit Community Cloud (free deployment). <https://share.streamlit.io>
 - NIST. *AI Risk Management Framework (AI RMF 1.0)*. <https://www.nist.gov/system/files/documents/2023/01/26/NIST.AI.100-1.pdf>
 - MLflow documentation (model versioning and experiment tracking, free and open-source). <https://mlflow.org/docs/latest/index.html>
 - Inductive Automation. *Ignition IIoT platform* — example of Level 3 analytics in manufacturing. <https://inductiveautomation.com>
 - Microsoft. *Machine Learning Operations (MLOps) overview*. <https://learn.microsoft.com/en-us/azure/machine-learning/concept-model-management-and-deployment>
-

13.7 Week 6 — Ethics, the Broader AI Landscape, and the Road Ahead

13.7.1 Learning Objectives

	By the end of this week, you will be able to...	Assessment
6.1	Identify at least three ethical risks in manufacturing AI systems and propose a mitigation for each	Self-check Q1–Q3
6.2	Explain what it means for a manufacturing AI decision to be “explainable” and when explainability is legally or contractually required	Self-check Q4
6.3	Describe the current state of AI adoption in manufacturing and identify two emerging areas likely to affect your career	Self-check Q5
6.4	Deliver a clear technical demonstration of your deployed Streamlit application to a non-specialist audience	Lab Week 6

Before You Read: Your predictive maintenance model predicts that engine #73 will fail in 15 cycles. Based on that prediction, the maintenance planner cancels a scheduled production run, incurs \$40,000 in rescheduling costs, and replaces the bearing — which, when removed, shows no sign of failure. Who is responsible for the \$40,000 cost?

13.7.2 AI Ethics in Manufacturing

Ethics in manufacturing AI is not abstract philosophy. It is a set of concrete questions about liability, bias, transparency, and human oversight that you will face in your first engineering role.

13.7.2.1 Bias in Training Data

Machine learning models learn patterns from historical data. If the historical data encodes biases, the model will replicate and often amplify them.

Manufacturing examples of training data bias:

- **Single-machine training:** a model trained on one lathe’s historical data may perform poorly on a different lathe of the same model, because the two machines have accumulated different wear histories and been maintained by different technicians. Deploying the model plant-wide without validating on each machine introduces risk.
- **Seasonal bias:** a model trained on summer data may perform poorly in winter because ambient temperature, coolant viscosity, and thermal expansion of machine frames all differ. This is a form of concept drift baked in at training time.
- **Operator exclusion:** if the training data comes entirely from one shift (because that shift’s data is in the historian), the model has not seen the setup patterns of other shifts. Predictions for equipment running under the excluded shift’s operators will be systematically less accurate.

Mitigation: before training, audit your dataset for coverage. Do all machines, operators, shifts, seasons, and operating conditions appear in roughly proportional representation? If not, deliberately stratify your sampling or weight your training examples accordingly. Document what your model does and does not cover.

13.7.2.2 Liability: Who Is Responsible When AI Is Wrong?

The Air Canada case (2024) established that companies are liable for what their AI-powered customer-service bots say. The principle extends to manufacturing: if your predictive maintenance model causes a consequential decision — stopping production, removing a part prematurely, or failing to flag an imminent failure — and that decision results in financial loss or injury, the question of liability is not trivial.

Current state of Canadian and US law: liability typically falls on the company deploying the system, not the model vendor. The engineer who deployed the system may be asked to demonstrate that the model was: - Validated appropriately before deployment - Used within its intended scope (not applied to equipment or conditions outside the training distribution) - Subject to human review for consequential decisions - Monitored for drift after deployment

This is why the “About this model” section in your Streamlit app is not optional decoration. It is a professional statement of scope and limitations. Get in the habit of writing it for every model you deploy.

13.7.2.3 Explainability

A Random Forest with 100 trees and 42 features produces predictions from millions of internal calculations. No engineer can explain any specific prediction by reading the model. This is the **black-box problem**.

For some decisions this is acceptable — if the stakes are low and the model is well-validated. For others, explainability is a hard requirement:

Context	Explainability requirement	Why
Stopping a production line	High — operator needs to understand why	\$10k+/hour downtime; operator may override
Recommending part replacement	Medium — maintenance planner needs enough context to validate the recommendation	Cost of unnecessary replacement vs. cost of failure
Flagging a safety-critical component (aircraft engine, vehicle brake system)	Very high — regulator may require auditable rationale	Liability and certification requirements
Optimizing a non-critical scheduling parameter	Low	Errors are cheap to reverse

Tools for explainability with Random Forest: - **Feature importance** (built in to scikit-learn): which features contributed most, on average - **SHAP values** (library: `shap`): which features pushed a *specific* prediction up or down, and by how much — the gold standard for local explanation - **Partial dependence plots**: how the prediction changes as one feature varies while others are held constant

For your capstone, feature importance is sufficient. In a real deployment for safety-critical equipment, SHAP values would be the minimum acceptable standard.

13.7.2.4 Human Oversight and Automation Bias

Warning

Automation Bias: Research consistently shows that humans become overreliant on automated recommendations over time — deferring to the machine even when their own judgment suggests the machine is wrong. For predictive maintenance systems, this can cause operators to override

legitimate sensor alarms (“the model says we’re fine”) or schedule unnecessary maintenance (“the model said replace it”).

Good AI deployment design includes: - Clear statements of what the model does and does not know - Confidence or uncertainty estimates displayed appropriately (to the right audience) - Easy mechanisms for human feedback and override - Regular review of cases where the human and the model disagreed — especially cases where the human was right

13.7.3 The Broader AI Landscape — Where This Is Going

13.7.3.1 Current State of AI in Manufacturing (2024–2026)

What is actually in production at scale (not pilot or demo) in North American manufacturing:

- **Statistical process control software with ML-augmented anomaly detection:** AVEVA, OSIsoft (now AVEVA PI), AspenTech, GE Digital — all ship SPC with ML alarm filtering
- **Computer vision quality inspection:** Cognex, Keyence, and custom solutions using YOLO/Detectron2 — in high-volume electronics, automotive body panels, food sorting
- **Predictive maintenance on rotating equipment:** vibration FFT analysis + ML on PLCs or edge servers — most common in paper, cement, oil refining, and heavy automotive
- **LLM-assisted documentation:** work order generation, SOP drafting, supplier emails — on the rise since 2023; most common at the engineering and supervision level, not on the shop floor

What is in rapid development / early deployment: - **Digital twins with ML feedback:** real-time simulation of production processes updated by sensor data — very capital-intensive but accelerating - **Autonomous quality control with closed-loop feedback:** the system detects a defect, diagnoses the probable cause, and sends a corrective offset directly to the CNC controller — no human in the loop for the adjustment - **Multimodal LLMs for maintenance technicians:** technician describes a symptom in natural language, uploads a photo of the machine, and receives a structured troubleshooting checklist from a model fine-tuned on OEM maintenance manuals

13.7.3.2 Technologies Likely to Affect Your Career in the Next 5–10 Years

1. **Foundation models for industrial time-series:** equivalent to GPT-4 but trained on sensor data rather than text — a single pre-trained model fine-tuned on your specific plant data rather than trained from scratch.

Early examples (Amazon Chronos, Google TimesFM) are already available.

2. **Simulation-accelerated training:** ML models trained primarily on physics-based simulation rather than historical operational data — solving the “I don’t have enough run-to-failure data” problem for new equipment. NASA C-MAPSS is an early example of this approach.
3. **Agentic AI for engineering workflows:** LLM-based systems that can take multi-step actions (search a manual, query a database, generate a work order, route it to the right technician) rather than just responding to a single question. These are beginning to appear in enterprise ERP and MES platforms.
4. **AI governance and certification requirements:** as AI systems make more consequential decisions, regulatory frameworks are evolving (EU AI Act, NIST AI RMF, ISO 42001). Engineers with both the technical skills and the regulatory literacy to certify AI systems for safety-critical applications will be in high demand.

13.7.3.3 Keeping Your Skills Current

The field moves fast. Three habits that will keep you current without burning out:

1. **One paper per month:** the PHM Society proceedings, arXiv (cs.LG for ML, eess.SP for signal processing), and the AAAI workshop on AI for manufacturing are free and publish the frontline work. You do not need to understand every paper — scan abstracts, read one or two per month deeply.
2. **One project per year:** applying a new technique to a real problem (even a toy project) converts abstract knowledge into practical skill faster than any course. The capstone you just completed is the template.
3. **Stay close to the domain:** the engineers who will have the most impact with AI are those who deeply understand both the technology and the manufacturing process. The pure ML specialists increasingly come from software companies; the competitive advantage of a manufacturing engineer is the physical intuition about what the data actually means.

13.7.4 Self-Check Questions — Week 6

1. Your C-MAPSS model was trained exclusively on FD001 (one operating condition, one fault mode). A plant manager asks you to deploy it on the hydraulic press line to predict cylinder seal failure. List three specific reasons why this deployment would be inappropriate, and what you would need to do before it could be considered.

2. The Air Canada chatbot case established that companies are liable for what their AI tools say. What does this imply for the “About this model” section of your Streamlit app — specifically what information should it contain to protect both you and your future employer?
3. A junior technician has been using your predictive maintenance dashboard for six months. You notice they never override the recommendations, even when they personally notice an unusual sound from a machine the model shows as “healthy.” What phenomenon does this illustrate, and how would you redesign the dashboard to mitigate it?
4. Your plant uses a Random Forest for bearing failure prediction on the stamping press. A failure occurs that the model did not predict (a false negative). The quality team asks you to explain why the model missed it. What tools would you use to investigate, and what information would you present?
5. Name two AI technologies not yet widely deployed in manufacturing (as of this course) that you think are most likely to change your specific engineering role within 10 years. Justify your answer.

13.7.5 Further Reading — Week 6

- NIST. *AI Risk Management Framework (AI RMF 1.0)*. <https://www.nist.gov/system/files/documents/2023/01/26/NIST.AI.100-1.pdf>
- European Commission. *EU AI Act — Summary for practitioners*. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689>
- ISO/IEC 42001:2023. *Artificial intelligence — Management system*. (Standard, not free, but your employer may have access.)
- Lundberg, S. and Lee, S.-I. “A Unified Approach to Interpreting Model Predictions,” *NeurIPS 2017*. <https://arxiv.org/abs/1705.07874> (the SHAP paper — foundational for model explainability)
- Crawford, K. *Atlas of AI* (2021). A readable, critical examination of AI’s social and political dimensions — highly recommended for its grounding perspective.
- Amazon Chronos (foundation model for time-series). <https://github.com/amazon-science/chronos-forecasting>
- PHM Society. Annual conference proceedings (free). <https://www.phmsociety.org/publications/proceedings>

13.8 Key Takeaways

Across six weeks, three themes have recurred:

1. **Use the simplest tool that solves your problem.** An LLM that hallucinates ASTM standards is the wrong tool for verifying a specification. A Random Forest that you can interrogate with feature importance is

often better than a neural network you cannot explain, even if the neural network is slightly more accurate.

2. **Data is the bottleneck, not the algorithm.** You spent more time in Week 2 cleaning 90 days of CNC data than you spent in Week 4 training the model. This ratio is typical of real industrial work. The engineers who understand the data — its provenance, its failure modes, its physical meaning — are the engineers who build systems that work in production.
3. **AI tools augment engineer judgment; they do not replace it.** An LLM that drafts your SOP still needs you to check it. A predictive maintenance model that says “15 cycles remaining” still needs a maintenance planner who understands what 15 cycles means in terms of the production schedule, the available technicians, and the cost of being wrong. You were never expected to become a Python programmer in six weeks — you were expected to learn to direct an LLM to write, explain, and debug code on your behalf, so you could automate the routine parts of this project and produce a working predictive-maintenance system that would otherwise have been well above your current level. The value of this course is not in the Python code — it is in the engineering judgment about when to trust the AI-generated output and when to push back.

Chapter 14

Total Productive Maintenance (TPM)

14.1 Learning Objectives

By the end of this chapter, you will be able to:

- Define Total Productive Maintenance and explain its objectives
- Describe the 8 pillars of TPM and their interconnections
- Calculate Overall Equipment Effectiveness (OEE) and its components
- Identify and categorize the Six Big Losses
- Implement autonomous maintenance activities
- Develop preventive and predictive maintenance strategies
- Apply TPM principles to improve equipment reliability and productivity

“Take care of your equipment, and it will take care of your production.” — Seiichi Nakajima, Father of TPM

14.2 Introduction to TPM

14.2.1 What is Total Productive Maintenance?

Total Productive Maintenance (TPM) is a holistic approach to equipment maintenance that strives to achieve perfect production with no breakdowns, no defects, and no accidents. It involves everyone in the organization, from operators to top management.

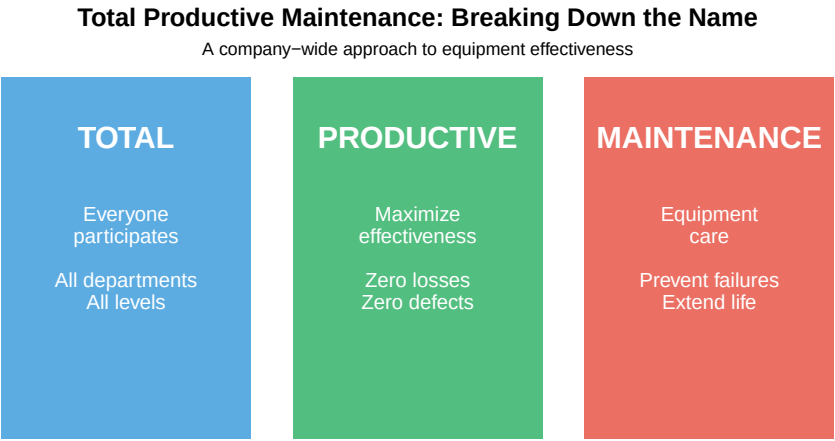


Figure 14.1: The TPM Philosophy

14.2.2 History of TPM

Table 14.1: Evolution of Maintenance Philosophy

Era	Maintenance Approach	Description
Pre-1950s	Breakdown Maintenance	Fix it when it breaks; reactive only
1950s	Preventive Maintenance	Scheduled maintenance to prevent failures
1960s	Productive Maintenance	Maintenance focused on reliability and maintainability
1970s	TPM Developed	Nakajima develops TPM at Nippondenso (Toyota supplier)
1980s	TPM Spreads Globally	Adopted by automotive, electronics, process industries
1990s-Present	TPM + Industry 4.0	Integration with predictive analytics, IoT sensors, AI

14.2.3 The Goals of TPM

TPM aims to achieve **zero** targets:

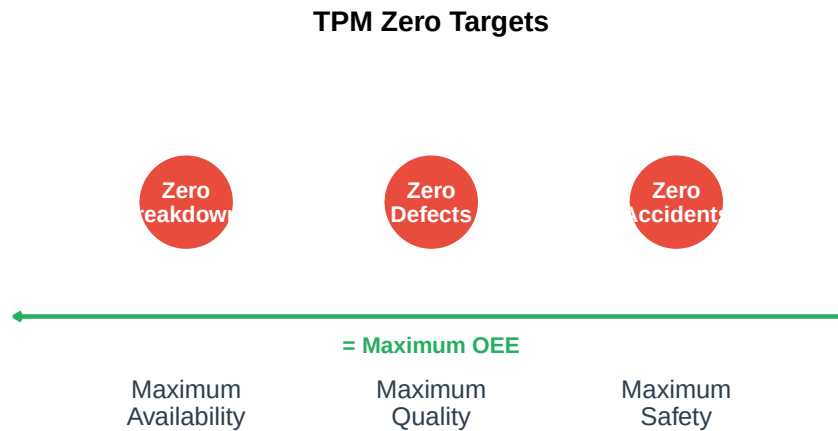


Figure 14.2: TPM Zero Targets

14.2.4 Traditional vs. TPM Approach

Table 14.2: Traditional Maintenance vs. TPM

Aspect	Traditional Maintenance	TPM Approach
Equipment ownership	'Maintenance owns the machines'	'We all own the machines'
Maintenance responsibility	Maintenance department only	Everyone (operators + maintenance)
Operator role	Operate; call maintenance when broken	Operate, clean, inspect, maintain
Maintenance focus	Fix breakdowns quickly	Prevent breakdowns; extend life
Problem solving	Reactive; fire-fighting	Proactive; root cause elimination
Goal	Minimize maintenance cost	Maximize equipment effectiveness

Question: Why involve operators in maintenance?

Operators are the first line of defense because:

1. **They know the equipment:** Operators work with machines every day and notice subtle changes
2. **Early detection:** Small problems are detected before becoming major failures
3. **Ownership mindset:** People take better care of things they “own”
4. **Reduced response time:** Operators can address minor issues immediately
5. **Freed maintenance resources:** Technicians can focus on complex tasks

Example: An operator notices a slight vibration change in a motor. Under traditional maintenance, they might ignore it. Under TPM, they report it, potentially preventing a \$50,000 breakdown.

“No one knows a machine better than the person who operates it every day.”

14.3 The Eight Pillars of TPM

TPM is built on **eight pillars**, each addressing a specific aspect of equipment management and organizational culture.

The House of TPM: Eight Pillars Built on 5S Foundation

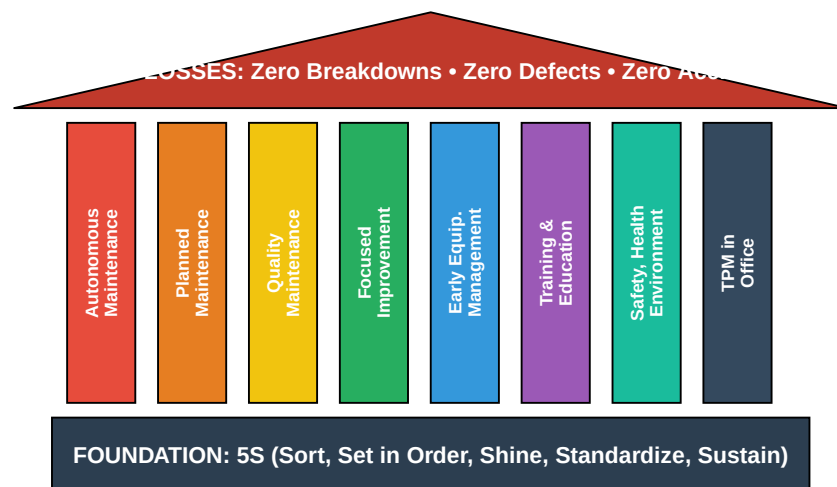


Figure 14.3: The Eight Pillars of TPM

Table 14.3: The Eight Pillars of TPM

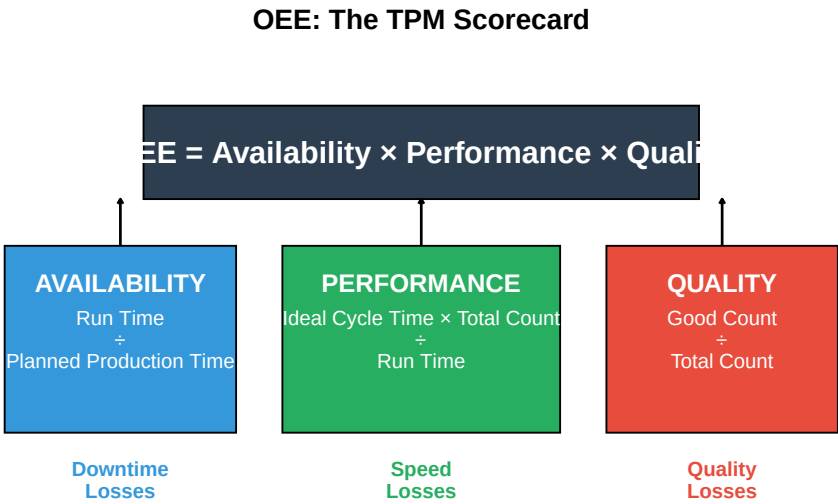
#	Pillar	Purpose
1	**Autonomous Maintenance**	Operators maintain their own equipment
2	**Planned Maintenance**	Scheduled maintenance by specialists
3	**Quality Maintenance**	Zero defects through equipment care

4	**Focused Improvement**	Eliminate losses through kaizen activities
5	**Early Equipment Management**	Design equipment for maintainability
6	**Training & Education**	Develop skilled operators and technicians
7	**Safety, Health, Environment**	Zero accidents, safe workplace
8	**TPM in Office**	Apply TPM principles to administrative pr

14.4 Overall Equipment Effectiveness (OEE)

14.4.1 What is OEE?

Overall Equipment Effectiveness (OEE) is the primary metric of TPM. It measures how effectively equipment is being used by combining three factors: Availability, Performance, and Quality.



Performance	$\frac{\text{Ideal Cycle Time}}{\text{Total Count}} \times \text{Run Time}$
Quality	$\frac{\text{Good Count}}{\text{Total Count}} \times 100\%$
OEE	$\text{Availability} \times \text{Performance} \times \text{Quality}$

14.4.3 OEE Calculation Example

Live cell — try different downtime, production, or defect numbers.

```
#| label: oee-calculation
# OEE Calculation Example: Injection Molding Machine

cat("=== SHIFT DATA ===\n")
# Time data
shift_length <- 480 # minutes (8 hours)
planned_downtime <- 30 # breaks, planned maintenance
planned_production_time <- shift_length - planned_downtime
cat("Planned Production Time:", planned_production_time, "minutes\n")

# Downtime losses
breakdown_time <- 25 # equipment failure
changeover_time <- 35 # mold changes
adjustment_time <- 10 # setup adjustments
total_downtime <- breakdown_time + changeover_time + adjustment_time
cat("Unplanned Downtime:", total_downtime, "minutes\n")

run_time <- planned_production_time - total_downtime
cat("Run Time:", run_time, "minutes\n\n")

# Production data
ideal_cycle_time <- 0.5 # minutes per part (design spec)
total_parts <- 700 # parts produced
good_parts <- 665 # parts passing inspection
defects <- total_parts - good_parts

cat("=== PRODUCTION DATA ===\n")
cat("Total Parts Produced:", total_parts, "\n")
cat("Good Parts:", good_parts, "\n")
cat("Defects:", defects, "\n\n")

# Calculate OEE Components
cat("=== OEE CALCULATION ===\n\n")

# Availability
availability <- (run_time / planned_production_time) * 100
cat("AVAILABILITY:\n")
cat(" = Run Time / Planned Production Time\n")
```

```

cat("  =", run_time, "/", planned_production_time, "\n")
cat("  =", round(availability, 1), "%\n\n")

# Performance
# Actual output vs. theoretical output during run time
theoretical_output <- run_time / ideal_cycle_time
performance <- (total_parts / theoretical_output) * 100
cat("PERFORMANCE:\n")
cat("  Theoretical Output =", theoretical_output, "parts\n")
cat("  Actual Output =", total_parts, "parts\n")
cat("  = Actual / Theoretical\n")
cat("  =", round(performance, 1), "%\n\n")

# Quality
quality <- (good_parts / total_parts) * 100
cat("QUALITY:\n")
cat("  = Good Parts / Total Parts\n")
cat("  =", good_parts, "/", total_parts, "\n")
cat("  =", round(quality, 1), "%\n\n")

# Overall OEE
oeo <- (availability/100) * (performance/100) * (quality/100) * 100
cat("=== OVERALL OEE ===\n")
cat("OEE =", round(availability, 1), "% ×", round(performance, 1), "% ×", round(quality, 1), "%\n")
cat("OEE =", round(oeo, 1), "%\n")

```

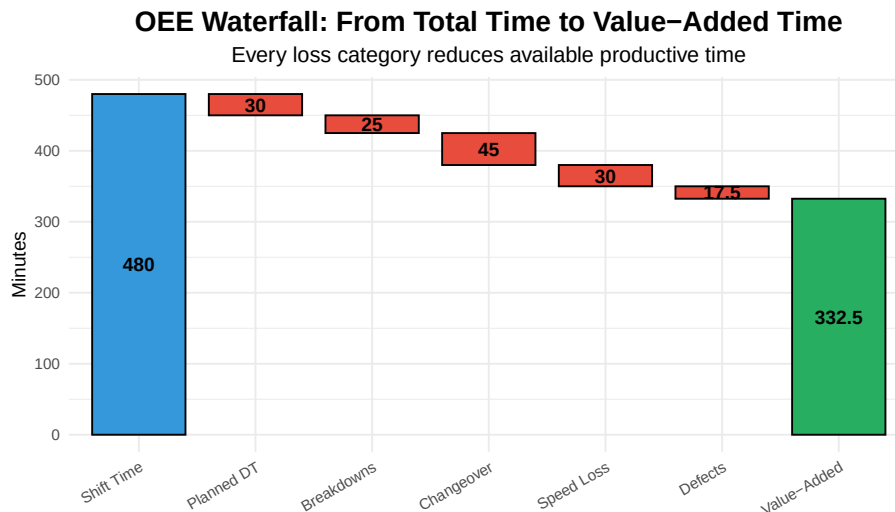


Figure 14.5: OEE Waterfall Chart: Where Time Goes

14.4.4 OEE Benchmarks

Table 14.5: OEE Benchmarks

OEE Level	Classification	Interpretation	Component Ta
< 65%	Unacceptable	Significant improvement opportunities exist	—
65% - 75%	Typical	Average for discrete manufacturing	A: 90%, P: 85%,
75% - 85%	Good	Well-managed operations	A: 92%, P: 90%,
> 85%	World Class	Best-in-class performance	A: 95%, P: 95%

Interactive Exercise: Calculate OEE

Given the following shift data, calculate OEE:

- Shift length: 480 minutes
- Planned breaks: 40 minutes
- Equipment breakdown: 30 minutes
- Setup/changeover: 20 minutes
- Ideal cycle time: 1 minute/part
- Total parts produced: 350
- Rejected parts: 14

Work through the calculation:

Solution:

Planned Production Time = 480 - 40 = 440 minutes

Run Time = 440 - 30 - 20 = 390 minutes

Availability = 390 / 440 = 88.6%

Theoretical Output = 390 / 1 = 390 parts

Performance = 350 / 390 = 89.7%

Quality = (350 - 14) / 350 = 336 / 350 = 96.0%

OEE = 88.6% × 89.7% × 96.0% = 76.3%

Interpretation: This is “Good” performance, above the typical range. The biggest opportunity is in Performance (speed losses).

14.5 The Six Big Losses

TPM categorizes all equipment losses into **six categories**, which map directly to the OEE components.

The Six Big Losses: Targets for TPM Improvement

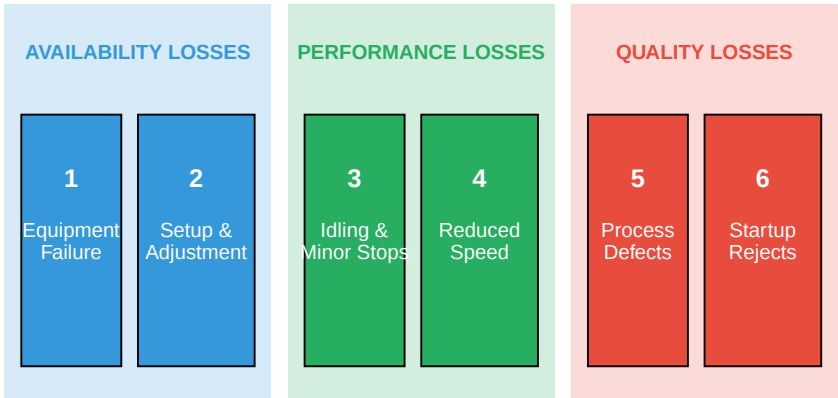


Figure 14.6: The Six Big Losses

Table 14.6: The Six Big Losses

#	Loss Category	OEE Component	Description
1	**Equipment Failure**	Availability	Unplanned equipment breakdown
2	**Setup & Adjustment**	Availability	Time for changeover, tool change
3	**Idling & Minor Stops**	Performance	Brief stoppages (jams, blockage
4	**Reduced Speed**	Performance	Running slower than design spe
5	**Process Defects**	Quality	Defects produced during norma
6	**Startup Rejects**	Quality	Defects during warmup, startu

14.5.1 Typical Loss Distribution

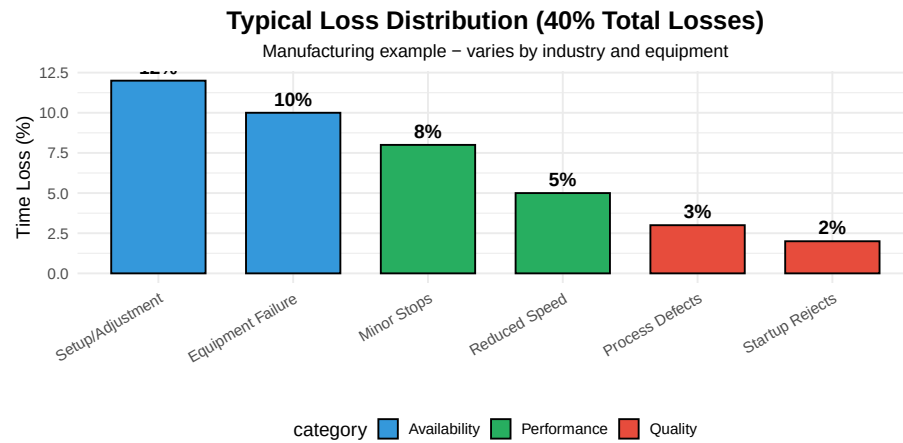


Figure 14.7: Typical Distribution of the Six Big Losses

14.6 Autonomous Maintenance

Autonomous Maintenance (AM) is the first and most visible pillar of TPM. It transfers basic maintenance tasks from maintenance specialists to operators.

14.6.1 The Seven Steps of Autonomous Maintenance



Figure 14.8: Seven Steps of Autonomous Maintenance

Table 14.7: Autonomous Maintenance Steps in Detail

Step	Name	Activities
1	**Initial Cleaning**	Deep clean equipment; identify abnormalities
2	**Eliminate Contamination Sources**	Find and fix sources of dirt, leaks, contamination
3	**Cleaning & Lubrication Standards**	Create standards for cleaning, lubricating, inspection
4	**General Inspection**	Train operators on equipment function; expand skills
5	**Autonomous Inspection**	Operators perform scheduled inspections independently
6	**Organization & Tidiness**	Standardize workplace; visual management
7	**Full Autonomous Maintenance**	Continuous improvement; operators propose improvements

14.6.2 The AM Tag System

During autonomous maintenance activities, operators identify abnormalities using **tags**:

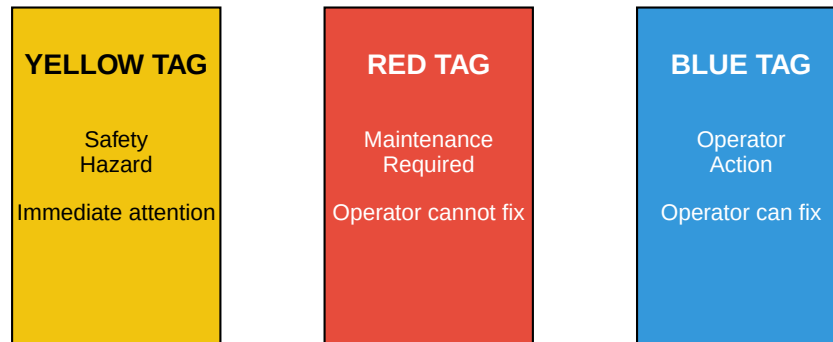
AM Tag System: Identifying and Tracking Abnormalities

Figure 14.9: AM Tag System

14.6.3 Example: AM Cleaning and Inspection Checklist

Table 14.8: Sample AM Inspection Checklist

#	Check Item	Method	Standard	Frequency
1	General cleanliness	Visual	No dust, debris, contamination	Daily
2	Oil/lubricant levels	Visual/gauge	Between min-max marks	Daily
3	Loose bolts/fasteners	Touch/wrench	Hand tight + 1/4 turn	Weekly
4	Unusual sounds	Listen	Normal operating sound	Each start
5	Vibration	Touch	Minimal, no unusual patterns	Each start
6	Temperature	Touch/thermometer	< 60°C (or per spec)	Hourly
7	Leaks (oil, air, coolant)	Visual	No visible leaks	Daily
8	Safety guards in place	Visual	All guards secure	Each shift

14.7 Planned Maintenance

Planned Maintenance is the second pillar, focusing on scheduled maintenance activities performed by maintenance specialists.

14.7.1 Types of Maintenance

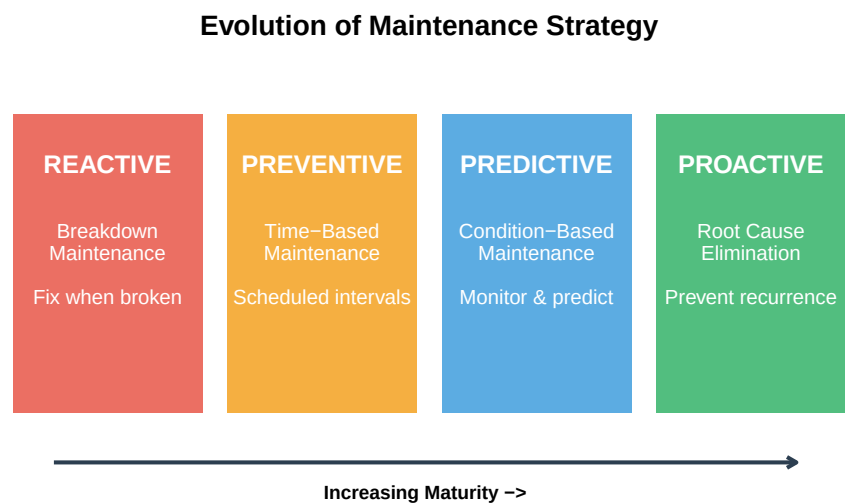


Figure 14.10: Types of Maintenance Activities

Table 14.9: Maintenance Strategy Comparison

Type	Approach	Advantages	Disadvantages
Breakdown	Run to failure	No upfront cost	Unplanned downtime
Preventive	Time/cycle-based replacement	Planned downtime, parts availability	May replace good components
Predictive	Condition monitoring	Optimal component life, targeted actions	Monitoring equipment costs
Proactive	Root cause elimination	Prevents future failures	Requires advanced analysis

14.7.2 Predictive Maintenance Technologies

Table 14.10: Predictive Maintenance Technologies

Technology	What It Detects	Equipment Applications
Vibration Analysis	Imbalance, misalignment, bearing wear, looseness	Rotating equipment (motors, pumps, fans)
Thermography	Hot spots, electrical issues, friction, insulation problems	Electrical panels, motors, bearings
Oil Analysis	Contamination, wear particles, lubricant degradation	Gearboxes, hydraulics, engines
Ultrasound	Leaks, bearing defects, electrical discharge	Steam traps, valves, bearings
Motor Current Analysis	Motor winding issues, rotor bar problems, power quality	Electric motors, drives, generators

14.7.3 The P-F Curve

The **P-F Curve** illustrates the concept of detecting potential failure (P) before functional failure (F).

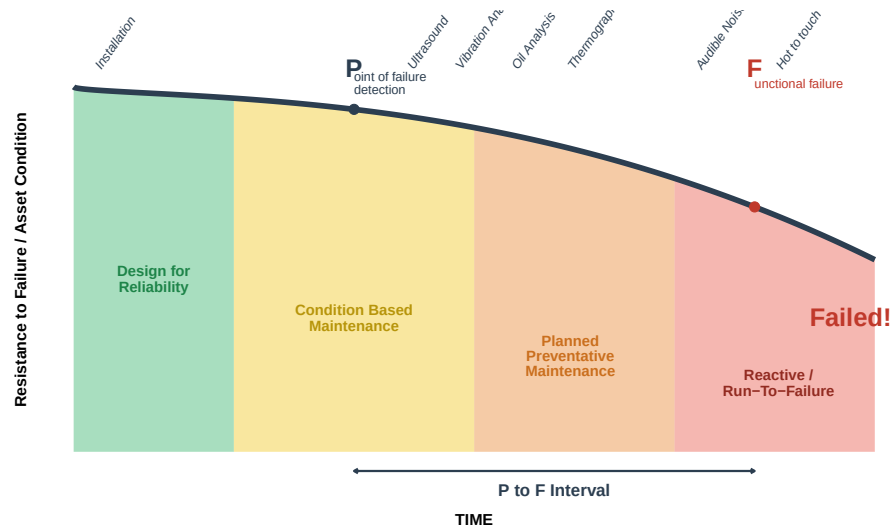


Figure 14.11: The P-F Curve: Detecting Failure Early

Question: Why is predictive maintenance better than preventive maintenance?

Preventive Maintenance (Time-Based): - Replaces parts at fixed intervals regardless of condition - May replace parts still in good condition (waste) - May miss failures that occur before scheduled replacement - Based on average life, not individual component condition

Predictive Maintenance (Condition-Based): - Monitors actual equipment condition - Replaces parts when condition indicates need - Maximizes component life - Provides warning time to plan repairs

Example: - Preventive: Replace bearing every 6 months (may last 9 months or fail at 4 months) - Predictive: Monitor vibration; replace when trending indicates 2-3 weeks to failure - Result: Optimal component life, planned downtime, no unexpected failures

Cost Comparison: - Reactive repair: \$10,000 (including secondary damage, lost production) - Preventive replacement: \$3,000 (may waste \$1,000 in parts life) - Predictive replacement: \$2,500 (optimal timing, minimal waste)

14.8 Focused Improvement (Kobetsu Kaizen)

Focused Improvement uses cross-functional teams to eliminate specific losses identified through OEE analysis.

14.8.1 The Kaizen Process

Focused Improvement (Kobetsu Kaizen) Process

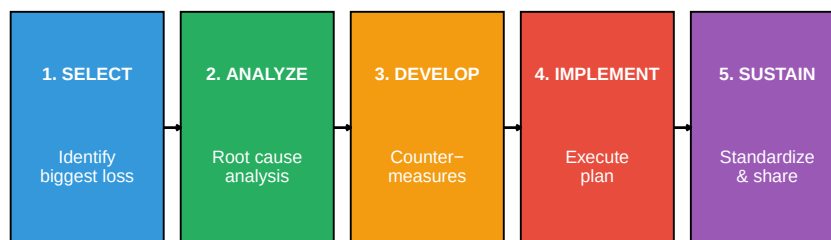


Figure 14.12: Focused Improvement Process

14.8.1.1 Understanding Kaizen

Kaizen () is a Japanese term meaning “change for better” or **continuous improvement**. Unlike large-scale transformation projects, Kaizen focuses on small, incremental improvements made consistently over time. The philosophy is built on the belief that every process can be improved, and everyone—from operators to managers—should participate in identifying and implementing improvements.

Core Principles of Kaizen:

- **Good processes bring good results** — Focus on improving the process, not blaming people
- **Go see for yourself (Gemba)** — Understand the actual situation at the workplace
- **Speak with data** — Base decisions on facts and measurements
- **Take action to contain and correct root causes** — Don’t just treat symptoms
- **Work as a team** — Cross-functional collaboration produces better solutions
- **Kaizen is everybody’s business** — Every employee can contribute improvements

14.8.1.2 The Five Steps in Detail

Step 1: SELECT — Identify the Biggest Loss

The first step uses data to identify which loss has the greatest impact on OEE. This typically involves:

- Reviewing OEE data and breakdown logs
- Creating Pareto charts to visualize loss magnitude
- Selecting a loss that is significant, measurable, and within the team's ability to address
- Forming a cross-functional team with members who understand the equipment and process

Key question: “Which single problem, if solved, would give us the biggest improvement?”

Step 2: ANALYZE — Root Cause Analysis

Once a loss is selected, the team investigates its root cause using structured problem-solving tools:

- **Why-Why Analysis (5 Whys):** Repeatedly asking “why” to drill down from symptoms to root causes
- **Fishbone Diagram (Ishikawa):** Categorizing potential causes by Machine, Method, Material, Man, Measurement, Environment
- **PM Analysis:** A systematic method for chronic losses that examines physical mechanisms
- **Data collection:** Gathering facts at the Gemba (actual workplace)

Key principle: Never assume you know the cause—verify with evidence.

Step 3: DEVELOP — Create Countermeasures

Based on root cause analysis, the team develops countermeasures—actions designed to eliminate or reduce the root cause:

- Brainstorm multiple potential solutions
- Evaluate options based on effectiveness, cost, and implementation time
- Create an action plan with clear responsibilities and deadlines
- Consider both immediate fixes and permanent solutions
- Apply error-proofing (Poka-Yoke) where possible

Important: Countermeasures address root causes, not just symptoms.

Step 4: IMPLEMENT — Execute the Plan

The team puts countermeasures into action:

- Follow the action plan systematically
- Make changes during planned downtime when possible
- Document all modifications made
- Train affected personnel on new procedures

- Monitor results immediately after implementation
- Be prepared to adjust if initial results are not as expected

Tip: Start with small-scale trials before full implementation when possible.

Step 5: SUSTAIN — Standardize and Share

The final step ensures improvements are maintained and shared across the organization:

- **Standardize:** Update procedures, checklists, and work instructions to reflect the new method
- **Visualize:** Create visual controls that make the standard easy to follow
- **Train:** Ensure all shifts and new employees learn the improved method
- **Verify:** Include checks in AM routines or audits to confirm the standard is being followed
- **Share:** Communicate successful improvements to other areas that might benefit (Yokoten—horizontal deployment)

Kaizen is never finished—today’s standard is tomorrow’s baseline for further improvement.

14.8.1.3 Kaizen Event vs. Daily Kaizen

Aspect	Kaizen Event (Blitz)	Daily Kaizen
Duration	3-5 days intensive	Ongoing, small improvements
Scope	Focused on one specific problem	Any improvement opportunity
Team	Dedicated cross-functional team	Individual or small group
Impact	Significant, measurable improvement	Cumulative effect over time
Example	Reduce changeover time by 50%	Reorganize tool storage for easier access

Both approaches are valuable—Kaizen events tackle significant losses while daily Kaizen builds a culture of continuous improvement.

14.8.2 Loss Elimination Tools

Effective loss elimination requires systematic tools that help teams identify problems, find root causes, and implement lasting solutions. The following tools are essential for Focused Improvement activities.

14.8.2.1 Pareto Analysis

Pareto Analysis applies the 80/20 rule: roughly 80% of losses come from 20% of causes. By creating a Pareto chart, teams can visualize which losses have the greatest impact and prioritize their improvement efforts accordingly.

How to use:

1. Collect data on all losses (downtime events, defects, speed reductions)
2. Categorize losses by type or cause
3. Calculate the frequency or duration of each category
4. Rank categories from highest to lowest
5. Create a bar chart with a cumulative percentage line
6. Focus improvement efforts on the “vital few” categories that account for most losses

14.8.2.2 Why-Why Analysis (5 Whys)

Why-Why Analysis is a simple but powerful technique for drilling down from a symptom to its root cause by repeatedly asking “Why?” The goal is to go beyond surface-level explanations to find the fundamental cause that, if addressed, will prevent recurrence.

Example:

Level	Question	Answer
Problem	Machine stopped	Overload tripped
Why 1?	Why did the overload trip?	Motor drew excessive current
Why 2?	Why excessive current?	Bearing was seized
Why 3?	Why was bearing seized?	Insufficient lubrication
Why 4?	Why insufficient lubrication?	Lubrication schedule not followed
Why 5?	Why not followed?	No visual indicator when lubrication is due
Root Cause		Missing visual management for lubrication schedule

Tips for effective Why-Why Analysis:

- Stay focused on one causal chain at a time
- Base answers on facts, not assumptions—verify at the Gemba
- Don’t stop at “human error”—ask why the error was possible
- The number 5 is a guideline, not a rule—continue until you reach a controllable root cause

14.8.2.3 PM Analysis

PM Analysis (Physical Mechanism Analysis or Phenomenon-Mechanism Analysis) is a systematic method for analyzing **chronic losses**—problems that occur repeatedly despite multiple repair attempts. It examines the physical principles behind equipment failures.

The PM Analysis Process:

1. **Clarify the phenomenon:** Describe exactly what happens, when, and under what conditions
2. **Conduct physical analysis:** Understand the physical principles involved (friction, heat, vibration, etc.)
3. **Identify conditions that produce the phenomenon:** List all conditions required for the problem to occur
4. **Investigate equipment, materials, and methods:** Examine each condition against the 4Ms (Machine, Material, Method, Man)
5. **Plan and implement countermeasures:** Address the conditions that allow the problem to occur
6. **Verify results:** Confirm the chronic loss has been eliminated

PM Analysis is particularly valuable when Why-Why Analysis doesn't resolve a recurring problem.

14.8.2.4 SMED (Single-Minute Exchange of Die)

SMED is a systematic approach to reducing changeover or setup time—the time between the last good part of one product and the first good part of the next. The goal is to complete changeovers in under 10 minutes (single-digit minutes).

The SMED Process:

SMED: Reducing Changeover Time

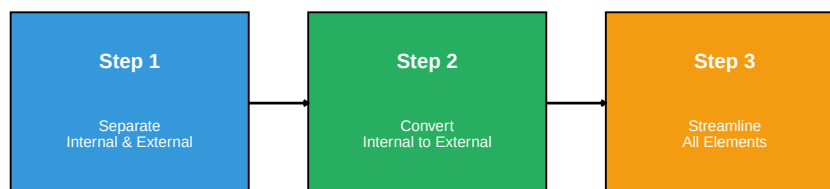


Figure 14.13: SMED Methodology

Key Concepts:

- **Internal setup:** Tasks that can only be done when the machine is stopped (changing the die itself)
- **External setup:** Tasks that can be done while the machine is running (preparing tools, preheating)

Typical SMED Improvements:

Technique	Example
Prepare in advance	Stage tools and materials before stopping the machine
Use quick-release clamps	Replace bolts with cam locks or hydraulic clamps
Eliminate adjustments	Use fixtures, stops, and standardized settings
Parallel operations	Two people working simultaneously on different tasks
Standardize components	Use common bolt sizes, connection types

14.8.2.5 Poka-Yoke (Error-Proofing)

Poka-Yoke () means “mistake-proofing” or “error-proofing.” These are devices or methods that either prevent errors from occurring or detect them immediately when they do occur.

Types of Poka-Yoke:

Type	Function	Example
Prevention	Makes errors impossible	Asymmetric connector that only fits one way
Detection	Identifies errors immediately	Sensor that stops the line if a part is missing
Warning	Alerts operator to potential error	Light or buzzer when a step is skipped

Poka-Yoke Design Principles:

1. **Design out the possibility of error** — The best Poka-Yoke makes the error impossible
2. **Detect at the source** — Catch errors where they occur, not downstream
3. **Stop and fix immediately** — Don’t pass defects to the next process
4. **Keep it simple** — Low-cost, reliable solutions are better than complex ones

Examples in Manufacturing:

- **Guide pins** that prevent a part from being loaded incorrectly
- **Counters** that verify the correct number of components were installed
- **Checklists with physical verification** (e.g., removing a flag for each completed step)
- **Limit switches** that confirm proper positioning before the cycle starts
- **Color coding** to match parts with their correct locations

Question: How do you decide which tool to use?

Tool Selection Guide:

- **Pareto Analysis:** Use first to identify which loss to focus on
- **Why-Why Analysis:** Use for sporadic failures with clear cause-effect relationships
- **PM Analysis:** Use for chronic losses that keep recurring despite repairs
- **SMED:** Use specifically for setup/changeover losses (Loss #2)
- **Poka-Yoke:** Use to prevent defects and human errors (Losses #5, #6)

Often, multiple tools are used together—Pareto to select the problem, Why-Why or PM Analysis to find the root cause, and Poka-Yoke to prevent recurrence.

14.8.2.6 Summary of Loss Elimination Tools

Table 14.11: Tools for Loss Elimination

Tool	Purpose	Application
Pareto Analysis	Identify biggest losses to target	Prioritize improvement efforts
Why-Why Analysis	Find root cause of failures	Breakdown analysis, defect investigation
PM Analysis	Analyze chronic losses systematically	Complex, recurring problems
SMED	Reduce setup/changeover time	Availability improvement (Loss #2)
Poka-Yoke	Error-proof the process	Quality improvement (Losses #5, #6)

14.9 TPM Implementation

14.9.1 The 12 Steps of TPM Implementation

Table 14.12: 12 Steps of TPM Implementation

Phase	Step	Activity	Duration
Preparation	1	Announce TPM introduction by top management	1 month
Preparation	2	Education and campaign to introduce TPM	3-6 months
Preparation	3	Create TPM promotion organization	1-2 months

Preparation	4	Establish basic TPM policies and goals	1-2 months
Preparation	5	Formulate master plan for TPM development	1-2 months
Implementation	6	Hold TPM kick-off	1 day
Implementation	7	Improve effectiveness of each piece of equipment	6-12 months
Implementation	8	Develop autonomous maintenance program	12-24 months
Implementation	9	Develop planned maintenance program	12-24 months
Implementation	10	Conduct training to improve skills	Ongoing
Stabilization	11	Develop early equipment management program	12-24 months
Stabilization	12	Perfect TPM implementation and raise levels	Ongoing

14.9.2 Success Factors

Table 14.13: TPM Success Factors

Factor	Why Critical
Top Management Commitment	Resources, culture change, sustained focus
Training and Education	Skills to identify problems, perform maintenance
Small Group Activities	Operator involvement, team problem-solving
Visual Management	Make abnormalities obvious, track progress
Patience and Persistence	TPM takes 3-5 years to fully implement

14.10 Case Study: TPM Implementation

14.10.1 Background

Company: Automotive component manufacturer

Equipment: CNC machining center

Initial OEE: 52%

14.10.2 Problem Analysis

```
# Initial State
cat("=== INITIAL OEE BREAKDOWN ===\n\n")

availability_initial <- 78 # %
performance_initial <- 82 # %
quality_initial <- 81 # %

oee_initial <- (availability_initial/100) * (performance_initial/100) * (quality_initial/100)
```

```
cat("Availability:", availability_initial, "%\n")
cat("Performance:", performance_initial, "%\n")
cat("Quality:", quality_initial, "%\n")
cat("OEE:", round(oe_initial, 1), "%\n\n")
```

```
# Loss breakdown
cat("=== TOP LOSSES IDENTIFIED ===\n")
cat("1. Equipment breakdowns: 8% of time\n")
cat("2. Setup/changeover: 7% of time\n")
cat("3. Minor stops: 12% of time\n")
cat("4. Defects: 4% of production\n")
```

```
=== INITIAL OEE BREAKDOWN ===
```

```
Availability: 78 %
Performance: 82 %
Quality: 81 %
OEE: 51.8 %
```

```
=== TOP LOSSES IDENTIFIED ===
1. Equipment breakdowns: 8% of time
2. Setup/changeover: 7% of time
3. Minor stops: 12% of time
4. Defects: 4% of production
```

14.10.3 Improvements Implemented

Loss	Root Cause	Countermeasure	Result
Breakdowns	Lack of basic maintenance	Autonomous maintenance program	8% → 3%
Setup time	No standard procedure	SMED implementation	35 min → 12 min
Minor stops	Chip buildup in sensors	Improved chip management	12% → 4%
Defects	Tool wear not detected	In-process gauging	4% → 1.5%

14.10.4 Results After 18 Months

```
# After TPM Implementation
cat("=== FINAL OEE BREAKDOWN ===\n\n")

availability_final <- 91 # %
```

```

performance_final <- 94      # %
quality_final <- 98.5      # %

oeo_final <- (availability_final/100) * (performance_final/100) * (quality_final/100)
cat("Availability:", availability_final, "% (was", availability_initial, "%)\n")
cat("Performance:", performance_final, "% (was", performance_initial, "%)\n")
cat("Quality:", quality_final, "% (was", quality_initial, "%)\n")
cat("OEE:", round(oeo_final, 1), "% (was", round(oeo_initial, 1), "%)\n\n")

improvement <- oeo_final - oeo_initial
cat("=== IMPROVEMENT ===\n")
cat("OEE Improvement:", round(improvement, 1), "percentage points\n")
cat("Relative Improvement:", round((improvement/oeo_initial)*100, 0), "%\n")

```

=== FINAL OEE BREAKDOWN ===

Availability: 91 % (was 78 %)
 Performance: 94 % (was 82 %)
 Quality: 98.5 % (was 81 %)
 OEE: 84.3 % (was 51.8 %)

=== IMPROVEMENT ===

OEE Improvement: 32.4 percentage points
 Relative Improvement: 63 %

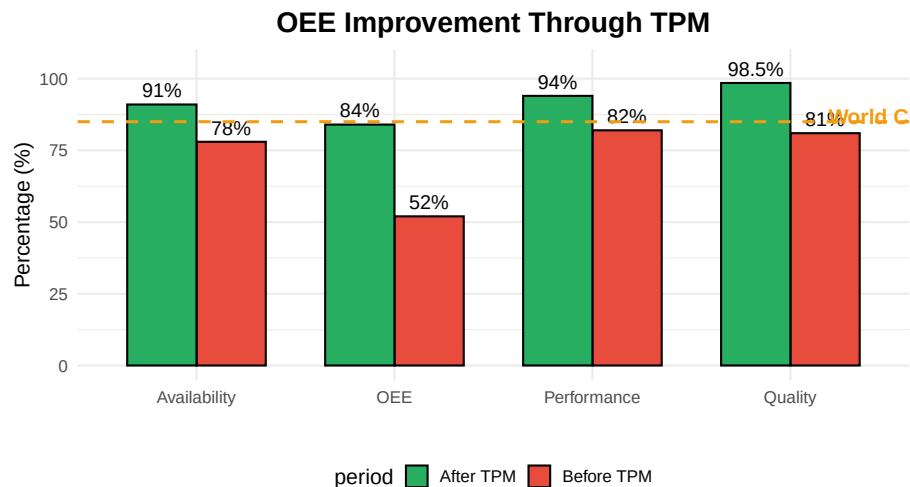


Figure 14.14: OEE Improvement: Before and After TPM

14.11 Summary

Table 14.14: TPM Key Concepts Summary

Topic	Key Points
TPM Definition	Total involvement, productive focus, maintenance excellence; zero breakdowns/defects
Eight Pillars	AM, PM, Quality, Focused Improvement, Early Equipment, Training, Safety, Office
OEE	OEE = Availability $\times$ Performance $\times$ Quality; World class > 85%
Six Big Losses	Equipment failure, setup, minor stops, speed loss, defects, startup rejects
Autonomous Maintenance	Operators maintain own equipment; 7 steps from cleaning to full ownership
Planned Maintenance	Preventive $\rightarrow$ Predictive $\rightarrow$ Proactive; use P-F curve for optimal timing

14.12 Review Questions

14.12.1 Conceptual Questions

Question 1: What does “Total” mean in Total Productive Maintenance?

“Total” has three dimensions:

1. **Total participation:** Everyone participates, from operators to top management
2. **Total equipment effectiveness:** Aim for maximum OEE (zero losses)
3. **Total system:** Covers the entire equipment lifecycle, from design to disposal

Key insight: TPM is not just a maintenance program — it’s a company-wide philosophy where everyone takes responsibility for equipment care.

Question 2: Why is OEE calculated as a product rather than a sum?

OEE uses multiplication because losses are sequential and compound:

1. **Availability** determines how much time is available
2. **Performance** determines how much of that time is productive
3. **Quality** determines how much of that production is good

Example: - If each factor is 90%: $OEE = 0.9 \times 0.9 \times 0.9 = 72.9\%$ (not 270% or 90%) - This reflects reality: you can only achieve quality on parts that were produced (performance) during time the machine ran (availability)

Why this matters: - Small improvements in all three factors compound - 5% improvement in each: $0.85 \times 0.85 \times 0.85 = 61\% \rightarrow 0.90 \times 0.90 \times 0.90 = 73\%$
- Focus on the weakest factor first for biggest impact

Question 3: What is the difference between autonomous maintenance and planned maintenance?

Aspect	Autonomous Maintenance	Planned Maintenance
Who	Operators	Maintenance technicians
What	Basic care (clean, inspect, lubricate)	Complex repairs, overhauls
When	Daily, during operation	Scheduled intervals
Skills	Basic equipment knowledge	Technical expertise
Goal	Prevent deterioration	Restore/improve equipment

They complement each other: - AM catches problems early (first line of defense) - PM provides specialized skills when needed - Together they achieve zero breakdowns

14.12.2 Calculation Questions

Question 4: Calculate OEE given the following data.

Given: - Shift length: 480 minutes - Breaks: 30 minutes - Changeover: 45 minutes - Breakdown: 20 minutes - Ideal cycle time: 30 seconds (0.5 min) - Total parts produced: 580 - Rejected parts: 23

Solution:

Planned Production Time = 480 - 30 = 450 minutes

Run Time = 450 - 45 - 20 = 385 minutes

Availability = 385 / 450 = 85.6%

Theoretical Output = 385 / 0.5 = 770 parts

Performance = 580 / 770 = 75.3%

Quality = (580 - 23) / 580 = 557 / 580 = 96.0%

OEE = 85.6% × 75.3% × 96.0% = 61.9%

Analysis: Performance is the biggest opportunity for improvement (75.3% vs. targets of 95%). Investigate minor stops and speed losses.

Question 5: A machine has OEE of 65% with Availability 90%, Performance 80%, Quality 90%. Which factor should be prioritized?

Analysis:

Current state: - Availability: 90% (Gap to world class: 5%) - Performance: 80% (Gap to world class: 15%) - Quality: 90% (Gap to world class: 9%)

Performance should be prioritized because:

1. **Biggest gap:** 15 percentage points below world class (95%)
2. **Biggest OEE impact:** Improving performance 10% $\rightarrow$ $OEE = 90\% \times 90\% \times 90\% = 72.9\%$
3. **Often overlooked:** Speed losses and minor stops are frequently accepted as normal

Investigation areas for performance: - Minor stops (jams, blockages, sensor trips) - Reduced speed operation - Idling and waiting - Operator pace

Question 6: If current OEE is 60% and target is 85%, what improvement is needed in each component assuming equal improvement?

Solution:

Current: $60\% = A \times P \times Q$

If we improve each equally, we need: - Target: $85\% = A' \times P' \times Q'$ - If $A' = P' = Q' = x$, then $x^3 = 0.85$ - $x = \sqrt[3]{0.85} = 0.947 = 94.7\%$

Each component needs to reach approximately 95%

Check: $0.947 \times 0.947 \times 0.947 = 0.849 = 84.9\%$

If current factors are A=80%, P=85%, Q=88%: - Availability needs: +15 points (80% $\rightarrow$ 95%) - Performance needs: +10 points (85% $\rightarrow$ 95%) - Quality needs: +7 points (88% $\rightarrow$ 95%)

Prioritize by gap size: Availability first, then Performance, then Quality.

14.12.3 Application Questions

Question 7: An operator notices a small oil leak on a machine. Under traditional maintenance vs. TPM, what happens?

Traditional Maintenance Approach: 1. Operator continues running machine 2. May or may not report to maintenance 3. If reported, added to backlog 4. Maintenance addresses when convenient (days/weeks) 5. Leak worsens, potential bearing damage 6. Eventually leads to breakdown

TPM Approach: 1. Operator notices during routine inspection 2. Tags the abnormality immediately 3. Determines if operator can fix (blue tag) or needs maintenance (red tag) 4. If simple: operator tightens fitting or adds sealant 5. If complex: maintenance prioritizes based on severity 6. Root cause investigated to prevent recurrence 7. Finding shared in team meeting

Key Differences: - **Ownership:** Operator takes responsibility vs. "not my job" - **Speed:** Immediate action vs. delayed response - **Prevention:** Fixed before failure vs. reactive repair - **Cost:** \$10 seal vs. \$5,000 bearing replacement + downtime

Question 8: List five items an operator should check during daily autonomous maintenance.

Five Essential Daily AM Checks:

1. **Cleanliness:** Remove chips, debris, dust; clean safety windows and sensors
 - *Why:* Contamination causes wear, sensor malfunctions, quality issues
2. **Lubrication:** Check oil levels, grease points, coolant level
 - *Why:* Inadequate lubrication causes friction, heat, accelerated wear
3. **Fasteners:** Check for loose bolts, guards, covers
 - *Why:* Loose fasteners cause vibration, safety hazards, misalignment
4. **Leaks:** Look for oil, coolant, air, hydraulic leaks
 - *Why:* Leaks indicate seal failure, waste resources, create hazards
5. **Abnormal sounds/vibration:** Listen and feel during startup
 - *Why:* Changes indicate developing problems (bearings, alignment)

Bonus items: - Safety guards in place - Emergency stops functional - Pressure/temperature gauges in normal range - No unusual smells (burning, chemical)

Question 9: Explain how SMED relates to OEE improvement.

SMED (Single Minute Exchange of Die) directly improves Availability:

Loss #2: Setup and Adjustment is an availability loss captured in OEE.

How SMED helps:

1. **Reduces changeover time:**
 - Before SMED: 45-minute changeover
 - After SMED: 10-minute changeover
 - Gain: 35 minutes per changeover
2. **Impact on OEE:**
 - If 2 changeovers per shift: 70 minutes saved
 - On 450-minute shift: Availability improves 15%+
3. **Secondary benefits:**
 - More changeovers become practical
 - Smaller batch sizes possible
 - Flexibility increased
 - Less WIP inventory

SMED Process: 1. Separate internal (machine stopped) from external (while running) setup 2. Convert internal to external where possible 3. Streamline remaining internal operations 4. Practice and standardize

Example: Die change on press - External: Stage next die, prepare tools, pre-heat - Internal: Remove old die, install new die, adjust - Target: All possible work done while machine runs

14.13 References

- Nakajima, S. (1988). *Introduction to TPM: Total Productive Maintenance*. Productivity Press.
 - Nakajima, S. (1989). *TPM Development Program: Implementing Total Productive Maintenance*. Productivity Press.
 - Shirose, K. (1992). *TPM for Operators*. Productivity Press.
 - Willmott, P., & McCarthy, D. (2001). *TPM: A Route to World-Class Performance*. Butterworth-Heinemann.
 - JIPM (Japan Institute of Plant Maintenance). *TPM Awards Criteria*.
 - Hansen, R.C. (2001). *Overall Equipment Effectiveness*. Industrial Press.
 - Borris, S. (2006). *Total Productive Maintenance*. McGraw-Hill.
-

Chapter 15

Troubleshooting and Root Cause Analysis

15.1 Learning Objectives

After completing this chapter, you will be able to:

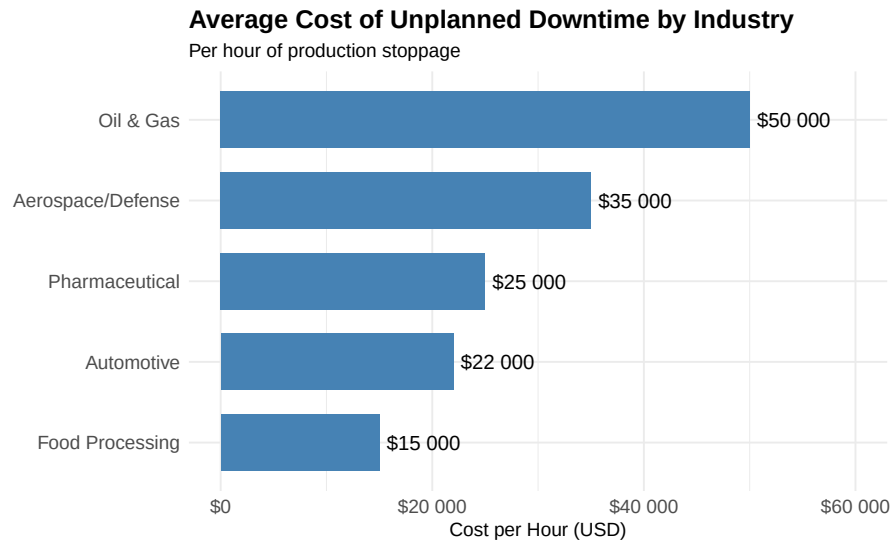
1. Apply systematic troubleshooting methodologies to diagnose equipment and process failures
 2. Distinguish between symptoms, immediate causes, and root causes
 3. Use the 5 Whys technique to drill down to fundamental causes
 4. Construct and analyze Ishikawa (fishbone) diagrams
 5. Apply Fault Tree Analysis (FTA) to complex failure scenarios
 6. Use Pareto analysis to prioritize problem-solving efforts
 7. Implement the 8D problem-solving methodology
 8. Document findings and implement effective corrective actions
 9. Prevent problem recurrence through systemic improvements
-

15.2 Introduction to Troubleshooting

Troubleshooting is the systematic process of identifying, diagnosing, and resolving problems in equipment, processes, or systems. For electromechanical technicians, effective troubleshooting is perhaps the most valuable skill you can develop.

15.2.1 The Cost of Downtime

Equipment failures and process problems have significant financial impacts:



15.2.2 Reactive vs. Proactive Problem Solving

Table 15.1: Comparison of Problem-Solving Approaches

Aspect	Reactive Approach	Proactive Approach
Timing	After failure occurs	Before or early in failure
Focus	Quick fix to restore operation	Root cause elimination
Stress Level	High - production pressure	Lower - planned approach
Cost	High - emergency repairs	Lower - scheduled work
Learning	Limited - rush to fix	High - thorough analysis
Documentation	Often skipped	Systematic and complete

Think About It: Why do organizations often default to reactive troubleshooting?

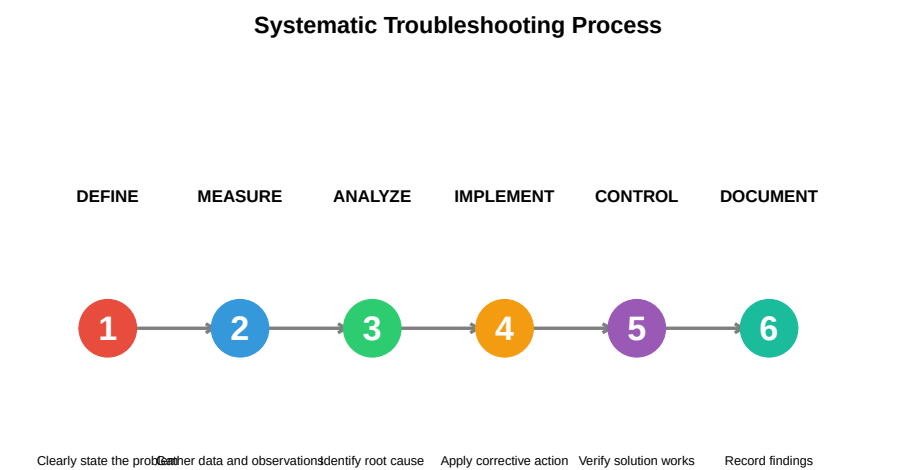
Common reasons include: - **Time pressure:** Production demands make it tempting to apply quick fixes - **Lack of training:** Technicians may not know RCA methodologies - **Reward systems:** Organizations may reward “firefighters” who quickly restore production - **Missing data:** Without good records, patterns are hard to identify - **Culture:** “We’ve always done it this way” mentality

The irony is that reactive troubleshooting actually costs more time in the long run due to recurring problems.

15.3 Systematic Troubleshooting Methodology

Effective troubleshooting follows a structured approach rather than random trial-and-error.

15.3.1 The DMAIC Troubleshooting Framework



15.3.2 Step 1: Define the Problem

A well-defined problem is half-solved. Use the **5W2H** method:

Table 15.2: 5W2H Problem Definition Framework

Question	What to Ask	Example Response
WHAT	What is the problem? What is happening vs. what should happen?	Motor overheating on conveyor
WHERE	Where does the problem occur? Location, machine, station?	Discharge end of conveyor
WHEN	When did it start? When does it occur? Continuous or intermittent?	Started Monday, occurs daily
WHO	Who discovered it? Who is affected? Who was operating?	Operator noticed, maintenance
WHY	Why is this a problem? What is the impact?	Risk of motor failure, line stop
HOW	How was the problem detected? How is it manifesting?	Detected by high-temp sensor
HOW MUCH	How often? How many units affected? What is the magnitude?	Occurs every day, 100% of units

15.3.3 Problem Statement Template

Good Problem Statement Format: “[WHAT] is occurring at [WHERE] since [WHEN], affecting [WHO/WHAT], with a magnitude of [HOW MUCH].”

Example: > “Motor overheating (85°C vs. normal 70°C) is occurring at Conveyor Line 3 discharge motor since Monday morning, affecting packaging department production, with the condition occurring every shift after approximately 2 hours of operation.”

Practice: Write a problem statement for this scenario

Scenario: An injection molding machine is producing parts with short shots (incomplete fill). The problem was noticed by the quality inspector during the night shift. About 15% of parts are affected. The issue seems worse when running the larger 500g parts compared to the smaller 200g parts.

Sample Problem Statement: “Short shots (incomplete cavity fill) are occurring on Injection Molding Machine #4 since night shift on Tuesday, affecting 15% of parts produced, with the defect rate increasing significantly when running 500g parts compared to 200g parts.”

15.3.4 Step 2: Gather Data and Observations

Before making any changes, collect information systematically:

Table 15.3: Systematic Data Gathering Checklist

Data Source	What to Check	Tools/Methods
Visual Inspection	Physical condition, wear, contamination, alignment, damage	Flashlight, mirror
Operator Interview	What changed? When did it start? Any unusual events?	Open-ended questions
Machine Parameters	Temperatures, pressures, speeds, currents, positions	HMI screens, gauges
Historical Records	Maintenance logs, previous repairs, similar issues	CMMS, logbooks
Product/Output	Defect patterns, measurements, reject rates	Calipers, gauges
Environmental	Temperature, humidity, vibration, power quality	Thermometer, hygrometer

15.3.5 The “Change Analysis” Approach

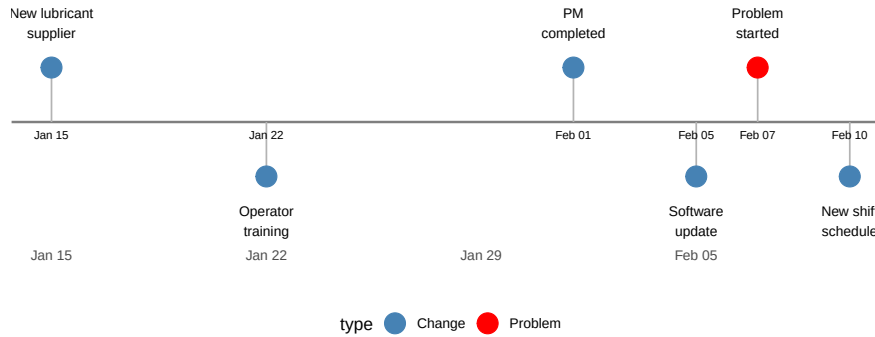
Many problems are caused by changes. Ask:

- What changed recently?
- New materials, suppliers, or batches?
- New operators or different shifts?
- Maintenance performed?
- Environmental changes (weather, season)?
- Software updates or parameter changes?

- New products or recipes running?

Change Timeline Analysis

Map changes against problem occurrence to identify correlations



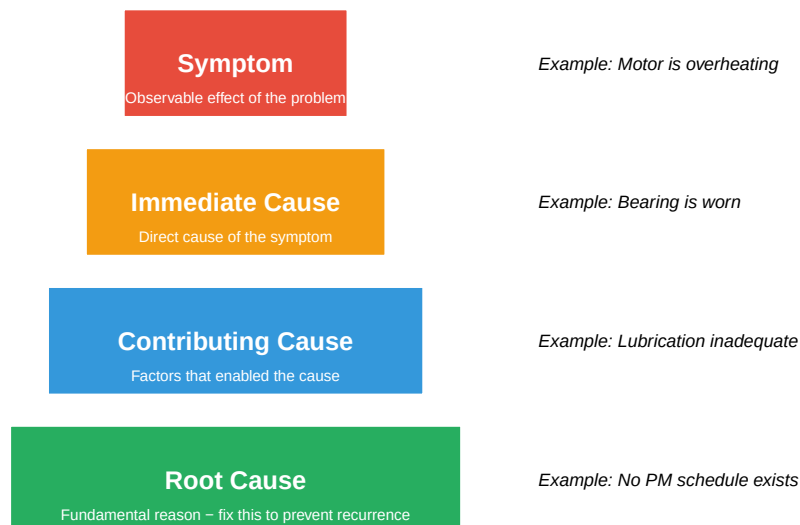
15.4 Understanding Root Cause vs. Symptoms

A critical concept in troubleshooting is distinguishing between different levels of causation.

15.4.1 The Causation Hierarchy

Causation Hierarchy

Dig deeper to find the root cause – don't stop at symptoms



15.4.2 Why Finding Root Cause Matters

Table 15.4: Comparison of Fix

Fix Level	Action Taken	Immediate Result
Symptom	Add cooling fan to motor	Motor runs cooler tem
Immediate Cause	Replace worn bearing	Motor works for a whi
Root Cause	Implement PM schedule with lubrication tasks	Problem eliminated

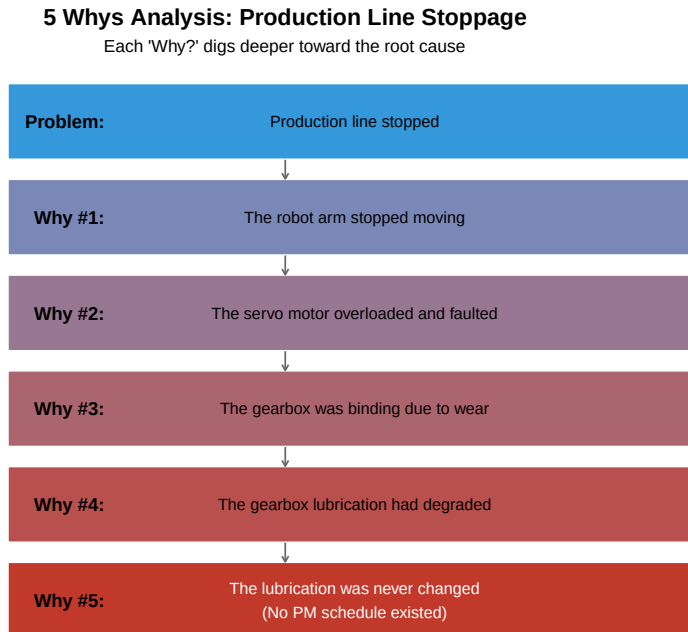
15.5 The 5 Whys Technique

The **5 Whys** is a simple but powerful technique developed by Sakichi Toyoda and used within Toyota’s manufacturing methods.

15.5.1 How It Works

1. State the problem
2. Ask “Why?” to identify the cause
3. Ask “Why?” again about that cause
4. Continue asking “Why?” until you reach the root cause
5. Typically requires 5 iterations (sometimes more, sometimes less)

15.5.2 5 Whys Example: Production Line Stoppage



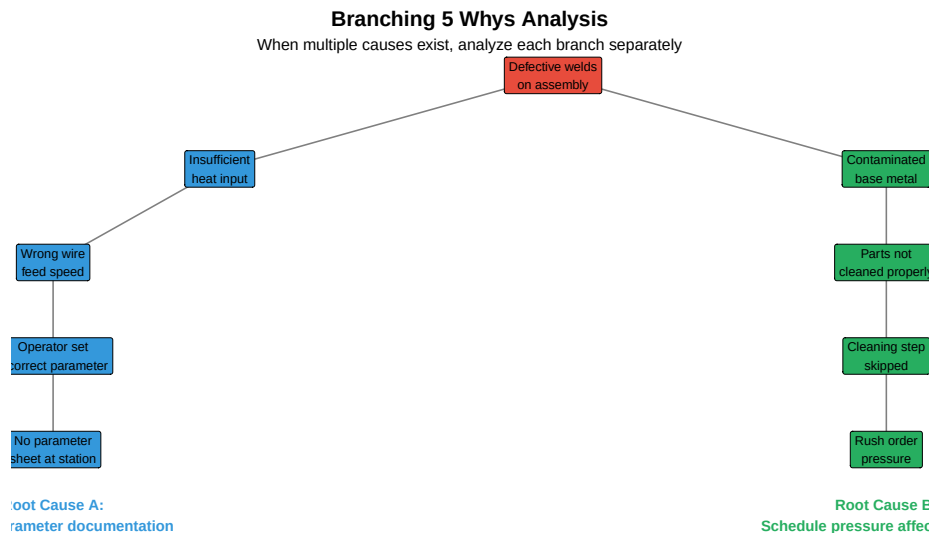
15.5.3 5 Whys Template

Table 15.5: 5 Whys Analysis Template

Step	Description	Typical Cause Type
Problem Statement	Clear, specific description of what happened	Symptom
Why #1	First-level cause (usually technical/physical)	Physical
Why #2	Why did that first cause occur?	Physical/Human
Why #3	Why did the second cause occur?	Human/Process
Why #4	Why did the third cause occur?	Process/System
Why #5	Why did the fourth cause occur?	System/Culture
Root Cause	The fundamental cause that, if fixed, prevents recurrence	Root

15.5.4 Multiple Branches in 5 Whys

Sometimes a problem has multiple causes. When you encounter this, branch your analysis:



Practice Exercise: Perform a 5 Whys Analysis

Scenario: A CNC machine produced 50 parts that were all 0.5mm undersize on a critical diameter.

Try to complete the 5 Whys analysis before looking at the sample answer below.

Sample Analysis:

1. **Problem:** CNC produced 50 undersize parts
2. **Why #1:** The tool offset was set incorrectly
3. **Why #2:** The operator entered the wrong offset value
4. **Why #3:** The operator misread the setup sheet
5. **Why #4:** The setup sheet had poor formatting and small print
6. **Why #5:** No standard format exists for setup documentation

Root Cause: Lack of standardized, clear setup documentation

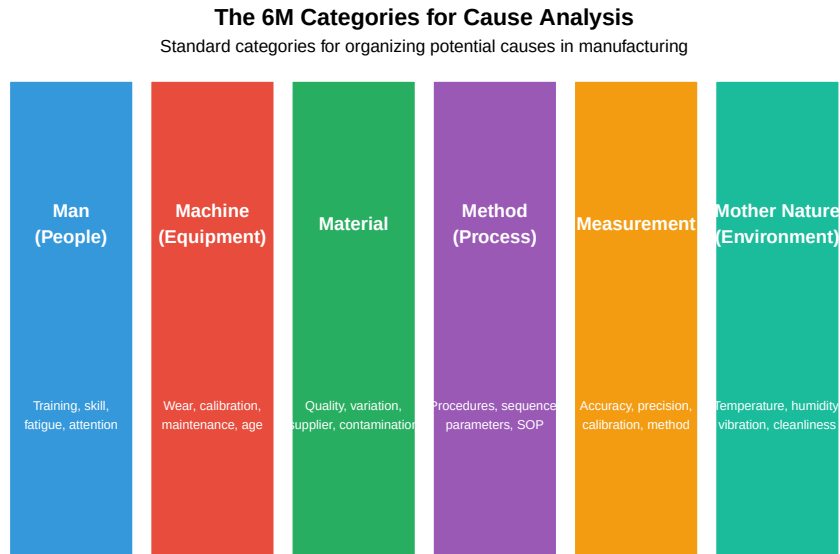
Corrective Actions: - Create standardized setup sheet template with large, clear fonts - Add visual verification step (photo of correct setup) - Implement first-piece inspection requirement before production run

15.6 Ishikawa (Fishbone) Diagram

The **Ishikawa diagram**, also called a **fishbone diagram** or **cause-and-effect diagram**, was developed by Kaoru Ishikawa in the 1960s. It provides a structured way to brainstorm and categorize potential causes.

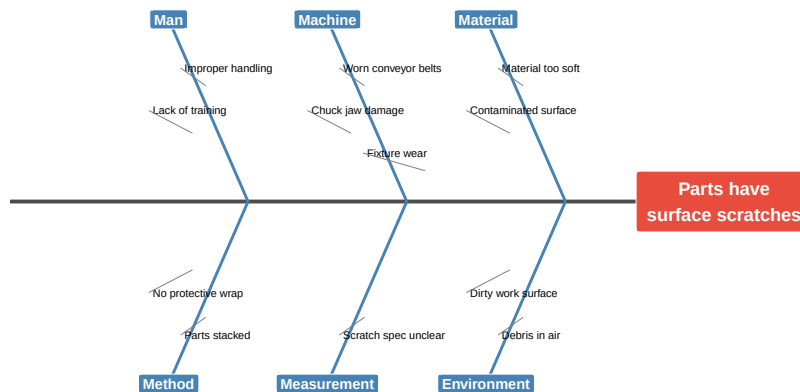
15.6.1 The 6M Categories

Manufacturing problems are typically organized using the **6M** categories:



15.6.2 Constructing a Fishbone Diagram

Fishbone Diagram: Parts Have Surface Scratches
Brainstorming potential causes organized by 6M categories



15.6.3 Fishbone Diagram Best Practices

Table 15.6: Best Practices for Fishbone Diagram Development

Guideline	Description
Team Approach	Include operators, engineers, maintenance, quality - diverse perspectives
No Evaluation During Brainstorming	Capture all ideas first; evaluate feasibility later
Use Specific Language	Not 'machine problem' but 'bearing wear on spindle motor'.
Aim for 3-5 Causes per Category	Some categories may have more; don't force causes into categories
Circle the Most Likely	After brainstorming, identify 2-3 most probable causes to investigate
Verify with Data	Don't assume - test hypotheses with data before implementing fixes

15.6.4 Video: How to Create a Fishbone Diagram

15.7 Fault Tree Analysis (FTA)

Fault Tree Analysis is a top-down, deductive analysis method that uses Boolean logic to analyze undesired events. Originally developed for the aerospace industry, FTA is particularly useful for complex systems with multiple potential failure paths.

15.7.1 FTA Symbols

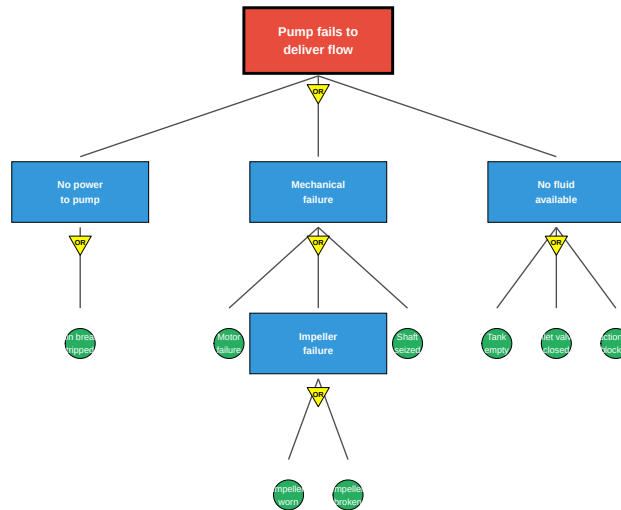
Table 15.7: Fault Tree Analysis Symbols

Symbol	Name	Meaning
Rectangle	Intermediate Event	A fault event that results from combination of other events
Circle	Basic Event	A basic failure that cannot be broken down further
Diamond	Undeveloped Event	Event not analyzed further (outside scope or insufficient data)
House	Normal Event	An event expected to occur during normal operation
AND Gate	AND Gate	Output occurs only if ALL inputs occur
OR Gate	OR Gate	Output occurs if ANY input occurs

15.7.2 FTA Example: Pump Fails to Deliver Flow

Fault Tree Analysis: Pump Fails to Deliver Flow

OR gates mean any single path can cause the top event



15.7.3 Calculating Probability with FTA

FTA allows quantitative risk analysis by calculating the probability of the top event:

OR Gate: $P(\text{output}) = 1 - [(1 - P_A) \times (1 - P_B) \times \dots]$ *Simplified:* $P(\text{output}) = P_A + P_B$ (when probabilities are small)

AND Gate: $P(\text{output}) = P_A \times P_B \times \dots$

Live cell — try different basic-event probabilities.

```

#| label: fta-calculation
# Example: Calculate top event probability
# Basic event probabilities (annual failure rates)
P_breaker_trip <- 0.02
P_motor_fail <- 0.01
P_impeller_worn <- 0.03
P_impeller_break <- 0.005
P_shaft_seize <- 0.008
P_tank_empty <- 0.05
P_valve_closed <- 0.01
P_line_blocked <- 0.02
  
```

```
# Impeller failure (OR gate)
P_impeller_fail <- 1 - (1 - P_impeller_worn) * (1 - P_impeller_break)
cat("P(Impeller failure):", round(P_impeller_fail, 4), "\n")

# Mechanical failure (OR gate)
P_mechanical <- 1 - (1 - P_motor_fail) * (1 - P_impeller_fail) * (1 - P_shaft_seize)
cat("P(Mechanical failure):", round(P_mechanical, 4), "\n")

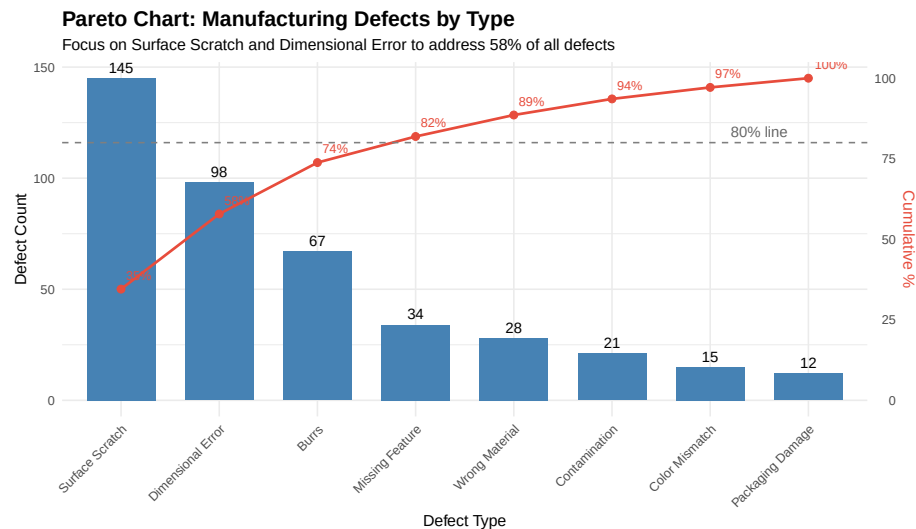
# No fluid available (OR gate)
P_no_fluid <- 1 - (1 - P_tank_empty) * (1 - P_valve_closed) * (1 - P_line_blocked)
cat("P(No fluid available):", round(P_no_fluid, 4), "\n")

# Top event: Pump fails (OR gate)
P_top <- 1 - (1 - P_breaker_trip) * (1 - P_mechanical) * (1 - P_no_fluid)
cat("P(Pump fails to deliver flow):", round(P_top, 4), "\n")
cat("This represents a", round(P_top * 100, 1), "% annual probability of failure")
```

15.8 Pareto Analysis

Pareto Analysis is based on the Pareto Principle (80/20 rule): roughly 80% of effects come from 20% of causes. This helps prioritize problem-solving efforts.

15.8.1 Creating a Pareto Chart



15.8.2 Interpreting Pareto Analysis

Live cell — try different defect counts.

```
#| label: pareto-interpretation
# Pareto analysis interpretation
defects <- data.frame(
  Defect = c("Surface Scratch", "Dimensional Error", "Burrs",
             "Missing Feature", "Wrong Material", "Contamination",
             "Color Mismatch", "Packaging Damage"),
  Count = c(145, 98, 67, 34, 28, 21, 15, 12)
)

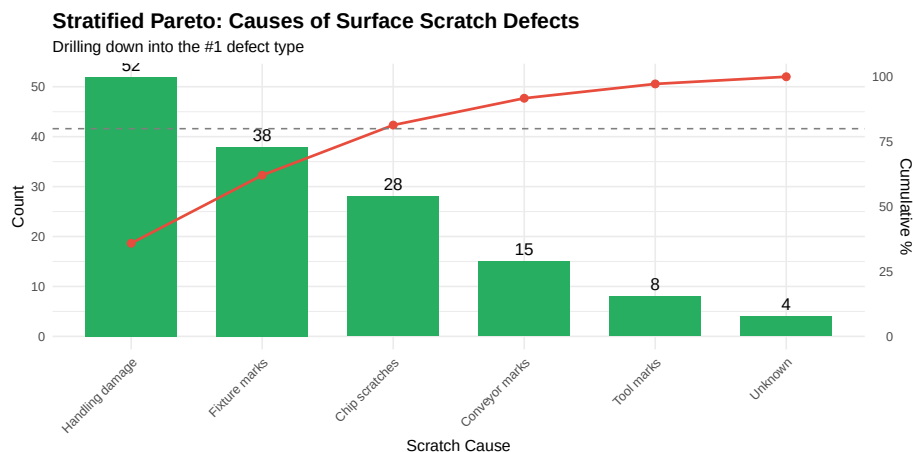
defects <- defects[order(-defects$Count),]
defects$Cumulative_Pct <- cumsum(defects$Count) / sum(defects$Count) * 100

# Identify the "vital few" (those that contribute to 80%)
vital_few <- defects[defects$Cumulative_Pct <= 80 |
  c(TRUE, defects$Cumulative_Pct[-nrow(defects)] < 80),]

cat("Total defects:", sum(defects$Count), "\n")
cat("\nVital Few (contributing to ~80% of defects):\n")
print(vital_few[, c("Defect", "Count", "Cumulative_Pct")])
cat("\nFocusing on these", nrow(vital_few), "defect types addresses",
    round(max(vital_few$Cumulative_Pct), 1), "% of all defects")
```

15.8.3 Stratified Pareto Analysis

Sometimes you need to break down categories further:



15.9 Is/Is Not Analysis

Is/Is Not Analysis is a structured comparison technique that helps narrow down the scope and identify distinguishing factors of a problem.

15.9.1 Is/Is Not Matrix

Table 15.8: Is/Is Not Analysis: Motor Overheating Problem

Dimension	Question	IS	IS NOT
WHAT	What object has the problem?	Motor on Line 3	Motors on Lines 1, 2
WHAT	What is the defect/symptom?	Overheating (85°C)	Noise, vibration, or f
WHERE	Where is the problem observed?	Packaging area, Station 4	Any other station
WHERE	Where on the object is the defect?	Motor housing, bearing end	Drive end, junction b
WHEN	When was it first observed?	Monday morning shift	Before weekend (Fric
WHEN	When in the process/cycle?	After 2 hours of operation	Cold start or end of
EXTENT	How many units affected?	100% - every shift	Intermittent
EXTENT	Is it trending up/down/stable?	Stable - same every day	Getting worse over t

15.9.2 Key Insights from Is/Is Not Analysis

From the example above, we can identify:

1. **Line 3 specific** - Not a general problem across all lines
2. **Thermal, not mechanical** - Overheating but no noise/vibration yet
3. **After warm-up** - Heat accumulation issue
4. **Started after weekend** - Something changed

Investigation focus: What changed over the weekend? Check maintenance logs, any work done on Line 3, any environmental changes.

15.10 The 8D Problem-Solving Methodology

The **8D (Eight Disciplines)** methodology is a comprehensive problem-solving approach developed by Ford Motor Company. It's widely used in automotive and other industries.

The 8D Problem–Solving Methodology		
A structured approach for complex problems requiring cross–functional teams		
D0	Plan	Plan the 8D approach; determine if 8D is appropriate
D1	Team	Establish a cross–functional team with required skills
D2	Problem	Define and describe the problem in measurable terms
D3	Containment	Implement temporary actions to protect customer
D4	Root Cause	Identify and verify root cause(s) using RCA tools
D5	Corrective Actions	Develop and verify permanent corrective actions
D6	Implementation	Implement and validate permanent corrective actions
D7	Prevention	Prevent recurrence; update systems and procedures
D8	Closure	Recognize team; document learnings; close out

15.10.1 Detailed 8D Steps

Table 15.9: 8D Methodology Detailed Breakdown

Discipline	Key Activities	Deliverable
D0: Plan	Emergency response actions, initial assessment, determine if 8D needed	8D initiation
D1: Team	Select team leader, identify required expertise, define roles	Team charter
D2: Problem	5W2H, Is/Is Not, quantify impact, identify affected parts/lots	Problem statement
D3: Containment	Sort suspect material, increase inspection, alert downstream	Containment actions
D4: Root Cause	5 Whys, Fishbone, FTA, verify cause by testing	Verified root cause
D5: Corrective Actions	Brainstorm solutions, verify effectiveness before full rollout	Verified corrective actions
D6: Implementation	Execute action plan, validate results, monitor KPIs	Implementation
D7: Prevention	Update FMEAs, procedures, training, share across organization	Updated procedures
D8: Closure	Final documentation, lessons learned, team recognition	8D report, team recognition

15.10.2 D3: Containment Actions

Containment is crucial to protect the customer while you investigate:

Table 15.10: Types of Containment Actions

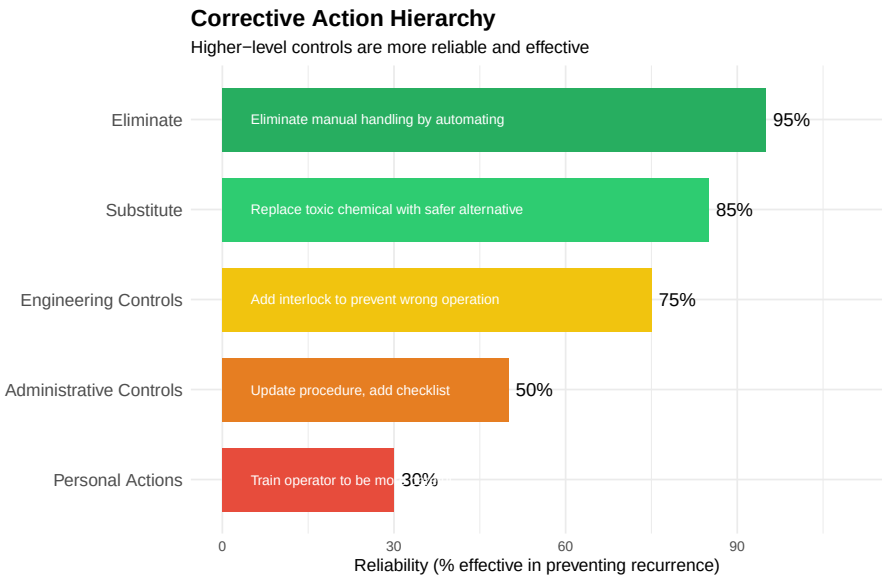
Type	Description	Example
Sort & Inspect	100% inspection of suspect inventory to separate good from bad	Inspect all parts from

Rework/Repair	Repair nonconforming units if possible and economical	Re-machine
Scrap	Dispose of units that cannot be repaired	Dispose
Hold/Quarantine	Prevent movement of suspect material until disposition	Tag and
Increase Monitoring	Add inspections, increase sampling, tighten control limits	Add 100
Supplier Notification	Notify suppliers if incoming material is suspected	Request
Customer Alert	Alert customers to check their inventory if product shipped	Contact

15.11 Corrective Action Implementation

Effective corrective actions address the root cause and prevent recurrence.

15.11.1 Types of Corrective Actions



15.11.2 SMART Corrective Actions

Corrective actions should be **SMART**:

Table 15.11: SMART Corrective Action Criteria

	Meaning	Description	Poor Example	Good Example
S	Specific	Clearly defined action, not vague	Improve quality	Install process
M	Measurable	Can verify completion and effectiveness	Train operators	Train all operators
A	Achievable	Realistic with available resources	Redesign entire product line	Add inspection

R	Relevant	Directly addresses the root cause	Repaint the floor	Add check fixture to
T	Time-bound	Has a target completion date	Do this sometime soon	Complete by March 1

15.12 Verification and Validation

Before closing out a problem, verify that your actions worked.

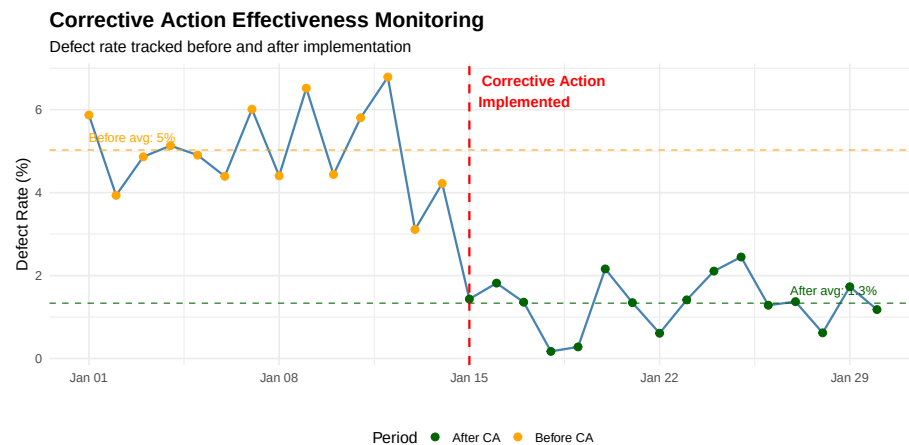
15.12.1 Verification vs. Validation

Table 15.12: Verification vs. Validation of Corrective Actions

Aspect	Verification	Validation
Definition	Did we implement the action correctly?	Did the action solve the problem?
Focus	Process compliance	Results and effectiveness
Question	Did we build the fix right?	Did we build the right fix?
Timing	During/after implementation	After implementation, during monitoring
Methods	Audits, inspections, checklists	Data analysis, trend charts, capability studies

15.12.2 Effectiveness Monitoring

Warning in `scale_x_date()`: A <numeric> value was passed to a Date scale.
i The value was converted to a <Date> object.



```
# Calculate improvement
before_avg <- mean(defect_rate[1:14])
after_avg <- mean(defect_rate[15:30])
improvement <- (before_avg - after_avg) / before_avg * 100
```

```
cat("Before corrective action: ", round(before_avg, 2), "% defect rate\n")
cat("After corrective action:   ", round(after_avg, 2), "% defect rate\n")
cat("Improvement:              ", round(improvement, 1), "% reduction\n")
```

```
Before corrective action: 5.03 % defect rate
After corrective action:  1.33 % defect rate
Improvement:             73.5 % reduction
```

15.13 Case Study: Automotive Welding Defects

Let's work through a comprehensive troubleshooting case using multiple RCA tools.

15.13.1 The Problem

A Tier-1 automotive supplier is experiencing weld porosity defects on a critical structural component. Customer complaints have increased, and the plant has received a quality alert.

15.13.2 Initial Problem Definition (5W2H)

Table 15.13: Case Study Problem Definition

Question	Details
WHAT	Porosity (gas pockets) in MIG welds on crossmember subassembly
WHERE	Robot welding Cell 4, occurring at weld joint W-23 (corner weld)
WHEN	Started approximately 2 weeks ago, occurs on all shifts
WHO	Detected by customer during their incoming inspection
WHY	Customer rejecting parts; risk of production line stop
HOW	Visible voids on X-ray inspection; some visible to naked eye
HOW MUCH	Estimated 8% of parts affected (up from baseline of 0.5%)

15.13.3 5 Whys Analysis

Table 15.14: 5 Whys Analysis: Weld Porosity

Level	Finding
Problem	Porosity in welds at joint W-23
Why #1	Gas entrapment during solidification
Why #2	Inadequate shielding gas coverage

Why #3 Gas flow rate dropped below specification

Why #4 Flow meter was not calibrated; showing higher than actual

Why #5 No calibration schedule existed for welding gas flow meters

15.13.4 Fishbone Diagram Findings

After brainstorming with the cross-functional team:

Table 15.15: Fishbone Analysis Results with Verification

Category	Potential Cause	Likelihood	Verification Result
Machine	Gas flow meter out of calibration	HIGH	YES - Root cause
Machine	Worn contact tip affecting arc stability	Medium	No - tips OK
Material	Different wire lot from new supplier	Medium	No - tested OK
Material	Base metal surface contamination	Low	No - clean parts
Method	Robot path too fast for corner weld	Medium	Partial contributor
Method	Gas pre-flow time insufficient	HIGH	YES - Contributing
Man	Recent operator turnover, less experience	Low	No - all shifts affected
Environment	Seasonal humidity changes	Low	No - controlled

15.13.5 8D Report Summary

Table 15.16: 8D Report: Weld Porosity Case Study

D#	Discipline	Summary
D1	Team	Team: Weld Engineer (lead), Quality, Maintenance, Production Supervisor, Supplier
D2	Problem	8% porosity rate at W-23 joint since Jan 15; customer quality alert received Jan 28
D3	Containment	100% X-ray inspection of all crossmembers; sort and quarantine 3 days production
D4	Root Cause	Primary: Flow meter out of calibration (reading 25 CFH, actual 18 CFH). Contributor
D5	Corrective Actions	1. Recalibrate flow meter immediately; 2. Increase pre-flow time to 0.5s; 3. Add flow
D6	Implementation	Actions completed Feb 5; validation testing shows 0.1% porosity rate (below 0.5% t
D7	Prevention	Updated PM schedule to include all gas flow meters quarterly; added to PFMEA w
D8	Closure	Customer satisfied with response; 8D closed Feb 15; cost avoidance: \$45,000 (avoided

15.14 Documentation and Knowledge Management

Proper documentation ensures learnings are captured and problems don't recur.

15.14.1 Key Documentation Elements

Table 15.17: RCA Documentation Checklist

Element	Contents	Benefit
Problem Description	5W2H, problem statement, impact/cost	Clear re
Timeline	When discovered, key investigation dates, implementation dates	Underst
Data Collected	Measurements, photos, test results, operator interviews	Evidenc
Root Cause Analysis	5 Whys, fishbone diagrams, FTA as applicable	Docume
Corrective Actions	Actions taken, responsible parties, target/actual dates	Account
Verification Results	Before/after data, statistical comparison, effectiveness %	Proof o
Lessons Learned	What we'd do differently, recommendations for similar situations	Continu
System Updates	PFMEA updates, procedure changes, training records	Prevent

15.14.2 Creating a Troubleshooting Knowledge Base

Build organizational memory by:

1. **Standardized templates** - Use consistent formats for all RCA documents
2. **Searchable database** - Store in CMMS, SharePoint, or dedicated quality system
3. **Categories and tags** - Equipment type, failure mode, root cause category
4. **Cross-references** - Link related issues and solutions
5. **Regular reviews** - Periodic analysis of trends and patterns

15.15 Troubleshooting Tools Quick Reference

Table 15.18: RCA Tool Selection Guide

Tool	Best For	When to Use
5 Whys	Simple problems with linear cause chains	Quick investigations, sta
Fishbone Diagram	Brainstorming multiple potential causes	Team problem-solving s
Fault Tree Analysis	Complex systems with multiple failure paths	Safety-critical systems,
Pareto Chart	Prioritizing which problems to tackle first	When facing multiple p
Is/Is Not	Narrowing down scope and identifying distinctions	When problem is hard t
8D	Customer complaints requiring formal response	Formal quality requirem
Change Analysis	Problems that started suddenly	When timing of problem

15.16 Video Resources

15.16.1 Understanding 5 Whys

15.16.2 8D Problem Solving

15.17 Summary

Effective troubleshooting and root cause analysis require:

1. **Systematic approach** - Don't guess; follow a structured methodology
2. **Clear problem definition** - Use 5W2H to clearly define what you're solving
3. **Data-driven analysis** - Collect facts before jumping to conclusions
4. **Root cause focus** - Don't stop at symptoms; dig to fundamental causes
5. **Multiple tools** - Use 5 Whys, Fishbone, FTA, Pareto as appropriate
6. **Effective corrective actions** - Aim for elimination, not just mitigation
7. **Verification** - Confirm your fix actually works with data
8. **Documentation** - Capture learnings to prevent recurrence

Remember: The goal isn't just to fix today's problem - it's to prevent tomorrow's.

15.18 Review Questions

Question 1: What is the difference between a symptom, immediate cause, and root cause? Provide an example of each for a machine that is producing parts with excessive surface roughness.

Answer:

- **Symptom:** The observable effect of the problem
 - *Example: Parts have Ra surface roughness of 3.2 μ m when specification is 1.6 μ m*
- **Immediate cause:** The direct, technical cause of the symptom
 - *Example: Cutting tool is worn beyond its effective life*
- **Root cause:** The fundamental reason that, if eliminated, prevents recurrence
 - *Example: No tool life monitoring system exists; operators judge tool changes by intuition rather than data*

Fixing the root cause (implementing tool life tracking) prevents the problem from recurring, whereas just replacing the tool (addressing immediate cause) means it will happen again.

Question 2: A food processing plant is experiencing frequent jams on their packaging line. Perform a 5 Whys analysis given the following information: The jam occurs at the carton loading station, started last week, happens 4-5 times per shift, and the maintenance team found the timing of the pusher mechanism is off.

Answer:

1. **Problem:** Carton loading station jams 4-5 times per shift
2. **Why #1:** Cartons are not in position when pusher activates → Timing mismatch
3. **Why #2:** Pusher timing is off by approximately 0.3 seconds
4. **Why #3:** Timing adjustment was changed during last week's maintenance
5. **Why #4:** Technician didn't have correct specification for timing setting
6. **Why #5:** Maintenance procedure doesn't include timing specifications

Root Cause: Maintenance procedures lack critical parameter specifications

Corrective Actions: - Update maintenance procedure with correct timing specification ($180\text{ms} \pm 10\text{ms}$) - Restore timing to correct setting immediately - Review other procedures for missing specifications - Add parameter verification checklist to PM activities

Question 3: Create a fishbone diagram outline (list the potential causes under each 6M category) for the problem "CNC machine producing parts 0.05mm oversize."

Answer:

MAN (People) - Operator entered wrong offset - Incorrect setup procedure followed - Inadequate training on new program - Fatigue/distraction during setup

MACHINE (Equipment) - Spindle thermal growth - Ballscrew wear/backlash - Tool holder runout - Axis positioning error

MATERIAL - Material hardness variation - Different lot from supplier - Material springback - Incorrect material grade

METHOD (Process) - Wrong tool offset entered - Program error in G-code - Incorrect cutting parameters - Missing roughing pass

MEASUREMENT - Gauge out of calibration - Wrong measurement technique - Temperature affecting measurement - Incorrect datum reference

MOTHER NATURE (Environment) - Shop temperature variation - Coolant temperature change - Vibration from nearby equipment - Humidity affecting gauges

Question 4: Given the following defect data from a sheet metal stamping operation, create a Pareto analysis and identify the vital few defects to focus

on.

Defect Type	Count
Scratches	89
Burrs	156
Wrinkles	45
Splits/Cracks	28
Dimensional	112
Surface Dents	67
Missing Features	18

Answer: (live — try your own defect counts)

```
#| label: pareto-practice
# Create and analyze the data
defects <- data.frame(
  Type = c("Burrs", "Dimensional", "Scratches", "Surface Dents",
           "Wrinkles", "Splits/Cracks", "Missing Features"),
  Count = c(156, 112, 89, 67, 45, 28, 18)
)

# Sort by count descending
defects <- defects[order(-defects$Count),]

# Calculate percentages
defects$Percent <- defects$Count / sum(defects$Count) * 100
defects$Cumulative <- cumsum(defects$Percent)

# Display results
print(defects)

# Identify vital few
vital_few <- defects$Type[defects$Cumulative <= 80 |
                          c(TRUE, defects$Cumulative[-nrow(defects)] < 80)]
cat("\nVital Few (contributing to ~80%):", paste(vital_few, collapse = ", "))
```

The **vital few** are Burrs, Dimensional errors, and Scratches - addressing these three defect types would address approximately 69% of all defects. Adding Surface Dents would bring the total to 82%.

Question 5: What are the key differences between verification and validation in the context of corrective actions?

Answer:

Aspect	Verification	Validation
Question asked	Did we implement the action correctly?	Did the action solve the problem?
Focus	Process compliance	Results/outcomes
Timing	During/immediately after implementation	Ongoing monitoring after implementation
Methods	Audits, checklists, inspections	Data analysis, trend charts, SPC
Example	Confirm new sensor was installed per specification	Measure defect rate over 30 days to confirm reduction

Verification confirms the action was done right. **Validation** confirms we did the right action.

Both are required - an action can be implemented perfectly (verified) but still not solve the problem (not validated).

Question 6: An aerospace manufacturer has a requirement to use the 8D methodology for customer complaints. What happens in D3 (Containment) and why is it critical?

Answer:

D3 Containment Actions protect the customer while the root cause investigation continues. Key activities include:

1. **Immediate actions:**

- Quarantine suspect inventory (tagged, segregated)
- 100% inspection of suspect lots
- Increase monitoring at subsequent operations
- Sort parts at customer location if already shipped

2. **Documentation:**

- Identify affected lot numbers, date codes, serial numbers
- Quantify the scope of potential exposure
- Document all containment actions taken

3. **Communication:**

- Notify customer of actions being taken
- Alert downstream operations
- Inform suppliers if incoming material involved

Why critical: - Prevents additional defective product from reaching the customer - Limits the scope of the problem (cost, liability) - Demonstrates responsiveness to customer - Buys time for proper root cause investigation - In aerospace, safety-of-flight concerns make containment essential

Without effective containment, the problem continues while you investigate, increasing customer impact and costs exponentially.

Question 7: Why might a team's 5 Whys analysis stop too early? What are signs that you haven't reached the true root cause?

Answer:

Reasons teams stop too early: 1. **Blame arrives** - Analysis ends at "operator error" without asking why the error occurred 2. **Comfort zone** - Team stops when they reach a cause they can easily fix 3. **Time pressure** - Rush to implement a fix, skipping deeper analysis 4. **Technical limit** - Team lacks expertise to go deeper into certain areas 5. **Defensiveness** - Reaching causes that implicate management or systems creates resistance

Signs you haven't reached root cause: 1. **Recurrence** - If the fix is "more training" or "be more careful," problem will return 2. **Human-based answer** - Root causes should typically be systems/processes, not individuals 3. **No prevention mechanism** - If fixing this cause doesn't prevent future occurrences, keep asking why 4. **Can still ask "why?"** - If there's a logical next level to investigate 5. **Administrative fix only** - If the solution is just a procedure or checklist with no engineering controls

Example of stopping too early: - Stopping at: "Operator set wrong parameter" - Should continue: Why? → No parameter sheet at workstation → Why? → No standard work documentation exists → Root cause: Missing standardized work process

The true root cause typically relates to systems, processes, or management decisions rather than individual actions.

Question 8: Describe how you would use Pareto analysis in a stratified manner to drill down into a quality problem.

Answer:

Stratified Pareto Analysis involves creating multiple levels of Pareto charts, progressively drilling down into the vital few categories.

Example: High scrap rate at a machining shop

Level 1: Scrap by defect type 1. Create Pareto of all defect types 2. Result: "Dimensional errors" is 45% of all scrap - investigate this first

Level 2: Dimensional errors by machine 1. Create Pareto of dimensional errors broken down by machine 2. Result: Machine #4 accounts for 60% of dimensional errors

Level 3: Machine #4 dimensional errors by feature 1. Create Pareto of which dimensions are failing on Machine #4 2. Result: Bore diameter is 70% of Machine #4 dimensional issues

Level 4: Bore diameter issues by shift/operator 1. Create Pareto by shift 2. Result: Evenly distributed - not operator-dependent (machine issue confirmed)

Final Analysis: Investigate boring operation on Machine #4 specifically - likely tool wear, spindle issue, or program error.

The stratified approach efficiently narrows focus from “we have a scrap problem” to “boring operation on Machine #4 has a systematic error.”

Question 9: A defense contractor needs to analyze a safety-critical failure. Why might Fault Tree Analysis (FTA) be preferred over simpler methods like 5 Whys?

Answer:

FTA advantages for safety-critical analysis:

1. **Handles complexity:**
 - Safety-critical systems often have multiple redundancies and complex failure paths
 - FTA can model combinations of events that must occur together (AND gates) or alternatively (OR gates)
 - 5 Whys follows single linear path; FTA captures parallel paths
2. **Quantitative capability:**
 - FTA allows probability calculations for top event
 - Can identify minimum cut sets (smallest combinations that cause failure)
 - Supports reliability and safety integrity level (SIL) calculations
 - Defense/aerospace require quantitative risk assessment
3. **Documentation requirements:**
 - Regulatory bodies (FAA, DoD) often require FTA for critical systems
 - Provides auditable, traceable analysis
 - Standard symbols and methodology
4. **Identifies hidden failures:**
 - Can reveal single points of failure
 - Shows where redundancy is (or isn't) effective
 - Identifies common cause failures that defeat redundancy
5. **Design improvement:**
 - Visual representation helps identify where to add safeguards
 - Sensitivity analysis shows which basic events most affect risk
 - Supports design reviews and safety cases

Example: A missile guidance system failure - 5 Whys might identify one cause path - FTA would show all possible failure paths, calculate overall failure probability, and identify that two specific sensor failures AND a software error all occurring together could cause the top event, even though each alone is unlikely.

15.19 References

1. Latino, R.J., Latino, K.C., & Latino, M.A. (2019). *Root Cause Analysis: Improving Performance for Bottom-Line Results* (5th ed.). CRC Press.
2. Okes, D. (2019). *Root Cause Analysis: The Core of Problem Solving and Corrective Action* (2nd ed.). ASQ Quality Press.
3. Andersen, B., & Fagerhaug, T. (2006). *Root Cause Analysis: Simplified Tools and Techniques* (2nd ed.). ASQ Quality Press.
4. AIAG. (2018). *CQI-20: Effective Problem Solving Guide*. Automotive Industry Action Group.
5. IEC 61025:2006. *Fault Tree Analysis (FTA)*. International Electrotechnical Commission.
6. Ishikawa, K. (1990). *Introduction to Quality Control*. 3A Corporation.
7. Juran, J.M. (1988). *Juran on Planning for Quality*. Free Press.
8. Ford Motor Company. (2019). *Global 8D Problem Solving Manual*. Ford Motor Company.
9. NASA. (2002). *Fault Tree Handbook with Aerospace Applications*. NASA Office of Safety and Mission Assurance.
10. Wilson, P.F., Dell, L.D., & Anderson, G.F. (1993). *Root Cause Analysis: A Tool for Total Quality Management*. ASQ Quality Press.

Chapter 16

Industrial Automation Fundamentals

16.1 Learning Objectives

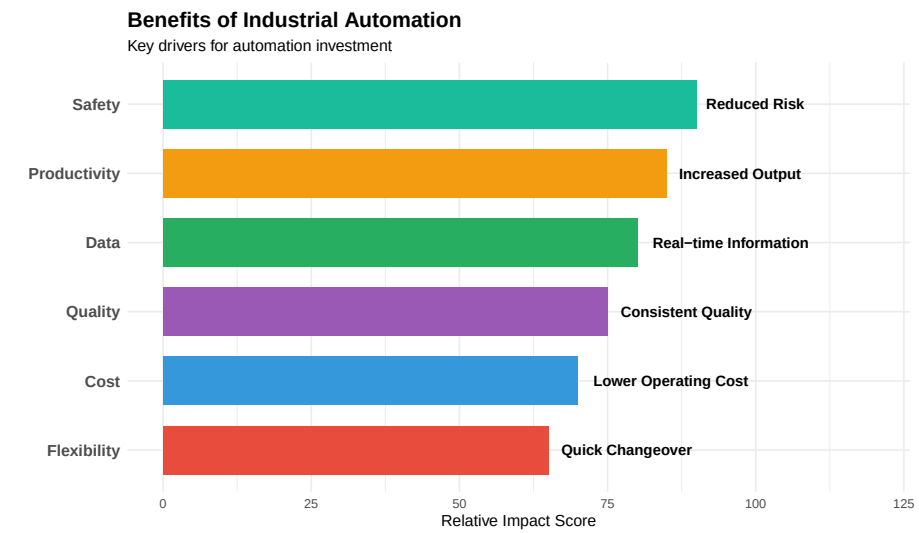
After completing this chapter, you will be able to:

1. Define industrial automation and explain its role in modern manufacturing
 2. Distinguish between fixed, programmable, and flexible automation
 3. Describe the architecture and function of Programmable Logic Controllers (PLCs)
 4. Identify common industrial sensors and their applications
 5. Explain motor control fundamentals including VFDs and servo systems
 6. Understand industrial communication protocols and networking
 7. Describe the role of HMI and SCADA in process monitoring
 8. Apply safety principles to automated systems
 9. Explain Industry 4.0 concepts and the Industrial Internet of Things (IIoT)
-

16.2 Introduction to Industrial Automation

Industrial automation is the use of control systems, machinery, and information technologies to handle processes and machinery in an industry, replacing human intervention where possible to increase efficiency, quality, and safety.

16.2.1 Why Automate?



16.2.2 Automation in Key Industries

Table 16.1: Automation Applications by Industry

Industry	Application	Technology	Benefit
Automotive			
Automotive	Robotic welding and assembly	6-axis robots, PLCs	Consistent w
Automotive	Automated paint systems	Conveyors, spray robots	Uniform coat
Automotive	Vision inspection	Machine vision, AI	100% inspect
Food & Beverage			
Food & Beverage	Filling and packaging lines	Servo drives, sensors	High speed,
Food & Beverage	Pasteurization control	PLCs, temperature control	Food safety,
Food & Beverage	Sorting and grading	Vision systems, conveyors	Quality sorti
Aerospace/Defense			
Aerospace/Defense	CNC precision machining	CNC, CMM integration	Tight toleran
Aerospace/Defense	Automated testing systems	Automated test equipment	Comprehensi
Aerospace/Defense	Clean room automation	Robotics, HEPA systems	Contaminati

16.3 Types of Automation

Industrial automation systems can be classified based on their flexibility and programming capability.

16.3.1 Automation Classification

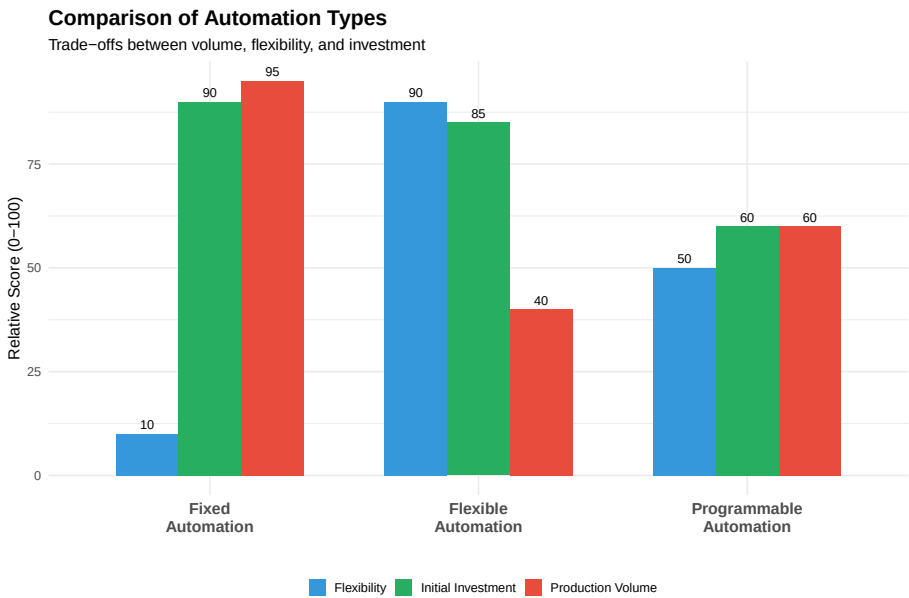
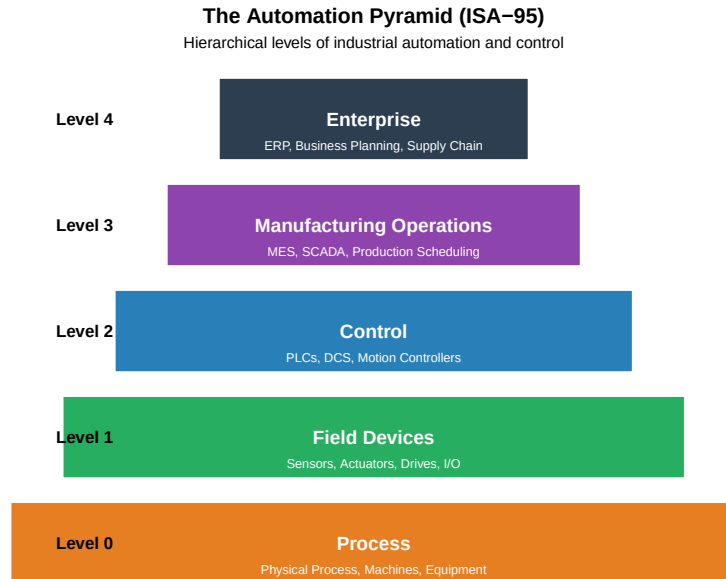


Table 16.2: Comparison of Automation Types

	Fixed Automation	Programmable Automation
Definition	Hard-wired, dedicated equipment for single product	Equipment can be reprogrammed for different products
Product Variety	Single product or very similar variants	Batches of different products
Production Volume	Very high (millions of units)	Medium to high
Changeover	Difficult, expensive, time-consuming	Requires reprogramming and setup
Initial Investment	Very high	High
Unit Cost	Very low per unit	Medium
Typical Equipment	Transfer lines, dedicated assembly machines	CNC machines, PLCs, industrial robots
Best For	Automotive components, fasteners, bottles	Batch manufacturing, job shops

16.3.2 The Automation Pyramid



Understanding the Pyramid Levels

Level 0 - Process: The actual physical equipment, machines, and processes being controlled. This includes conveyors, motors, valves, tanks, and the product being manufactured.

Level 1 - Field Devices: Sensors that measure process variables (temperature, pressure, flow, position) and actuators that affect the process (motors, valves, solenoids, drives).

Level 2 - Control: The “brain” of automation - PLCs, DCS, and motion controllers that execute control logic, process sensor inputs, and command actuators. Also includes HMI for operator interaction.

Level 3 - Manufacturing Operations: MES (Manufacturing Execution Systems), SCADA, production scheduling, quality management, and maintenance management. Bridges plant floor and business systems.

Level 4 - Enterprise: ERP systems, business intelligence, supply chain management, financial systems. Makes business decisions based on plant floor data.

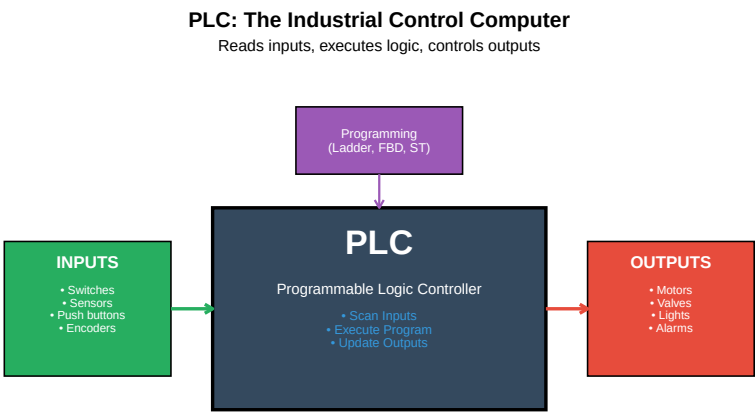
Key Principle: Data flows up (process information), commands flow down (control directives).

16.4 Programmable Logic Controllers (PLCs)

The **PLC (Programmable Logic Controller)** is the workhorse of industrial automation, providing reliable, real-time control of machines and processes.

16.4.1 What is a PLC?

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



16.4.2 PLC Hardware Components

Table 16.3: PLC Hardware Components

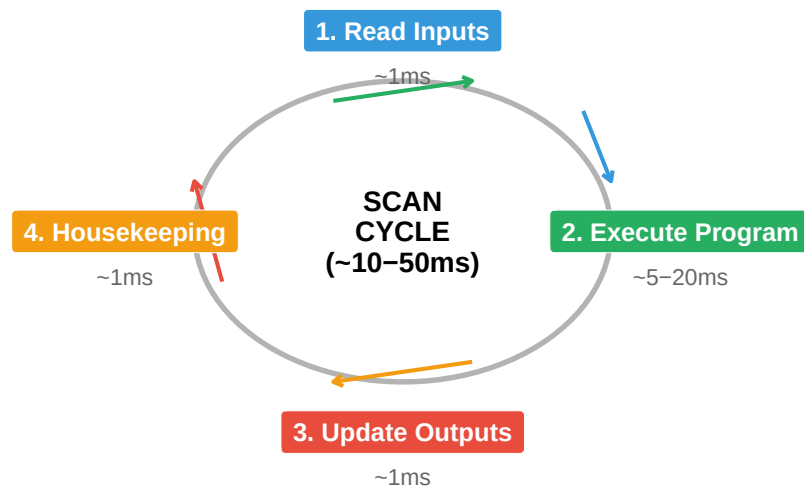
Component	Function
CPU (Processor)	Executes the control program, manages memory, coordinates all modules
Power Supply	Converts AC power to DC voltages required by PLC components
Input Modules	Interface field devices (sensors, switches) to CPU; converts signals to digital
Output Modules	Interface CPU to field devices (motors, valves); converts digital to power signals
Communication Modules	Enable networking: Ethernet/IP, Profinet, Modbus, DeviceNet
Programming Device	Laptop/PC with programming software for creating and downloading programs

16.4.3 The PLC Scan Cycle

PLCs operate in a continuous scan cycle:

PLC Scan Cycle

Continuous loop executing at predictable intervals



Live cell — try different scan time components.

```
#| label: scan-time-calc
# Scan Time Calculation Example
input_scan_time <- 1.2      # ms
program_execution <- 15.5   # ms (depends on program size)
output_update <- 1.0        # ms
housekeeping <- 2.3         # ms

total_scan_time <- input_scan_time + program_execution + output_update + housekeeping

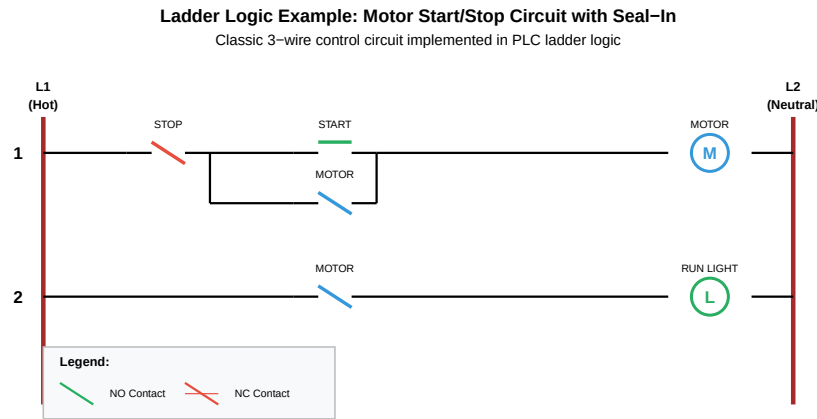
cat("PLC Scan Time Calculation:\n")
cat("      \n")
cat("Input Scan:      ", input_scan_time, "ms\n")
cat("Program Execution:", program_execution, "ms\n")
cat("Output Update:    ", output_update, "ms\n")
cat("Housekeeping:     ", housekeeping, "ms\n")
cat("      \n")
cat("Total Scan Time:  ", total_scan_time, "ms\n")
cat("\nScans per second: ", round(1000/total_scan_time, 0), "\n")
```

16.4.4 PLC Programming Languages (IEC 61131-3)

Table 16.4: IEC 61131-3 PLC Programming Language

Language	Type	Description
Ladder Diagram (LD)	Graphical	Resembles electrical relay circuits; most common in discrete
Function Block Diagram (FBD)	Graphical	Uses function blocks connected by lines; good for analog/pro
Structured Text (ST)	Textual	High-level language similar to Pascal; powerful for complex c
Instruction List (IL)	Textual	Low-level assembly-like language; rarely used today
Sequential Function Chart (SFC)	Graphical	Sequence/state-based programming; excellent for batch proce

16.4.5 Ladder Logic Example



How the Start/Stop Circuit Works

Rung 1 - Motor Control: 1. Power flows from L1 through the NC (Normally Closed) STOP button 2. If STOP is pressed, the circuit breaks and motor stops 3. START is NO (Normally Open) - pressing it allows power to flow to the MOTOR coil 4. When MOTOR energizes, its NO contact in the parallel branch closes 5. This “seals in” the circuit - motor stays running even after START is released 6. To stop, press STOP which breaks the seal-in circuit

Rung 2 - Run Indicator: 1. When MOTOR coil is energized, the MOTOR contact closes 2. This allows power to flow to the RUN LIGHT output 3. Light is ON whenever motor is running

This is called a “3-wire control” circuit - it provides: - Low voltage release protection (motor won’t restart after power failure) - Maintained contact operation (don’t need to hold START button)

16.5 Industrial Sensors

Sensors are the “eyes and ears” of automation systems, providing feedback about the process to the control system.

16.5.1 Sensor Classification

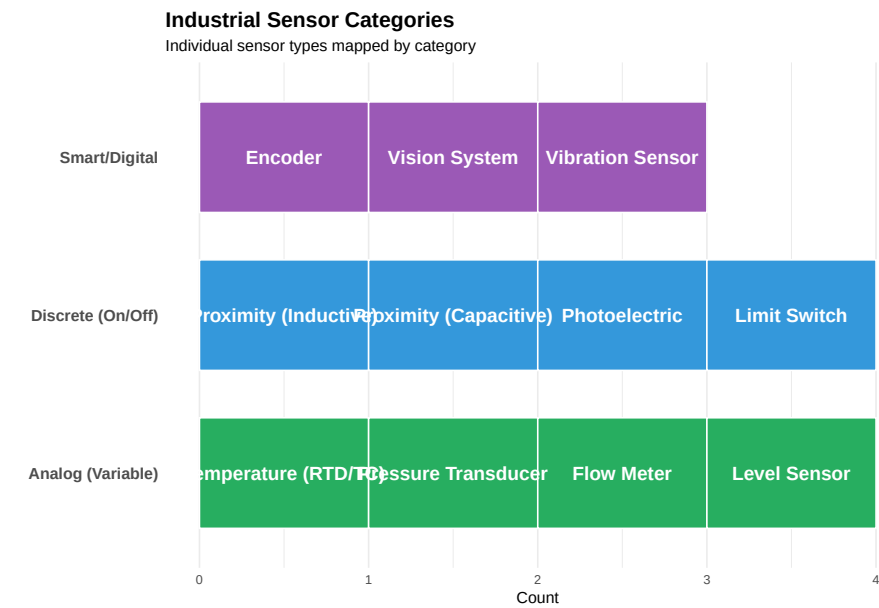
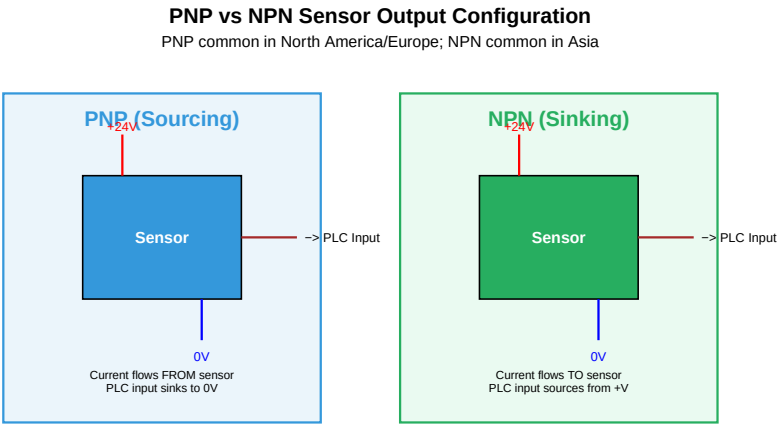


Table 16.5: Common Industrial Sensor Specifications

Sensor	Principle	Range	Adv
Inductive Proximity	Eddy current change in oscillating field	2-40mm typical	Dur
Capacitive Proximity	Capacitance change with target approach	2-25mm typical	Det
Photoelectric	Light beam interrupted or reflected	0.1-30m	Lon
RTD (PT100)	Resistance changes with temperature	-200 to 850°C	Acc
Thermocouple	Voltage generated at junction of dissimilar metals	-200 to 2300°C	Wid
Pressure Transducer	Diaphragm deflection converted to electrical signal	0-10000 psi	Acc
Incremental Encoder	Optical/magnetic pulses per revolution	100-10000 PPR	Sim
Absolute Encoder	Unique code for each position	12-25 bit	Pos

16.5.2 Sensor Wiring: PNP vs NPN



16.6 Actuators and Motor Control

Actuators convert control signals into physical motion or action.

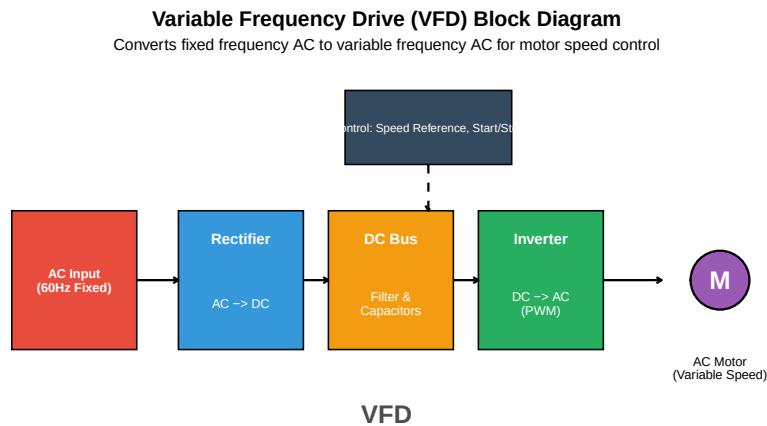
16.6.1 Types of Industrial Actuators

Table 16.6: Industrial Actuator Types and Applications

Category	Type	Application	Control
Electric			
Electric Motors	AC Induction Motor	Conveyors, pumps, fans, compressors	VFD for variable speed
Electric Motors	Servo Motor	Precise positioning, robotics, CNC	Servo drive with feedback
Electric Motors	Stepper Motor	Indexing, low-speed positioning	Stepper drive (open or closed loop)
Electric Motors	DC Motor	Battery vehicles, legacy systems	PWM, SCR drive
Pneumatic			
Pneumatic	Cylinder	Clamping, pushing, lifting	Solenoid valves, proportional valves
Pneumatic	Rotary Actuator	Rotating grippers, indexing	Solenoid valves
Hydraulic			
Hydraulic	Cylinder	Heavy lifting, presses	Proportional/servo valves
Hydraulic	Motor	Heavy machinery, mobile equipment	Proportional/servo valves

16.6.2 Variable Frequency Drives (VFDs)

A **Variable Frequency Drive (VFD)** controls AC motor speed by varying the frequency and voltage of the power supplied to the motor.



Live cell — try different frequencies or motor parameters.

```

#| label: vfd-calculations
# VFD Speed and Torque Calculations

# Motor nameplate data
motor_hp <- 10
poles <- 4
rated_voltage <- 460 # V
rated_frequency <- 60 # Hz
rated_rpm <- 1750
slip_rpm <- (120 * rated_frequency / poles) - rated_rpm

# Synchronous speed at rated frequency
sync_speed_60 <- 120 * rated_frequency / poles
cat("Motor Data:\n")
cat("Synchronous speed at 60Hz:", sync_speed_60, "RPM\n")
cat("Rated speed:", rated_rpm, "RPM\n")
cat("Slip:", slip_rpm, "RPM (", round(slip_rpm/sync_speed_60*100, 1), "%)\n\n")

# Calculate speed at different frequencies
frequencies <- c(15, 30, 45, 60, 75)
speeds <- sapply(frequencies, function(f) {
  sync <- 120 * f / poles
  sync - slip_rpm # Assuming constant slip (approximation)
})

cat("Speed vs Frequency (V/Hz mode):\n")
cat("      \n")
for(i in seq_along(frequencies)) {
  cat(sprintf("%2d Hz: %4d RPM\n", frequencies[i], speeds[i]))
}

```

```
}

# V/Hz ratio
vhz_ratio <- rated_voltage / rated_frequency
cat("\nV/Hz Ratio:", round(vhz_ratio, 2), "V/Hz\n")
cat("At 30Hz, voltage should be:", 30 * vhz_ratio, "V\n")
```

16.6.3 VFD Benefits

Table 16.7: Variable Frequency Drive Benefits

Benefit	Description	Typical
Energy Savings	Match motor speed to load requirements; huge savings on fans/pumps	20-50%
Soft Start/Stop	Ramp up/down gradually; eliminates inrush current (6-8x normal)	Reduce
Speed Control	Precise speed control from 0-100%+ of base speed	N/A
Process Control	Maintain constant pressure, flow, or tension	Improv
Reduced Mechanical Stress	Reduced wear on belts, gears, couplings from smooth acceleration	Extend
Power Factor	VFD presents near-unity power factor to supply	Avoid F

16.6.4 Servo Systems

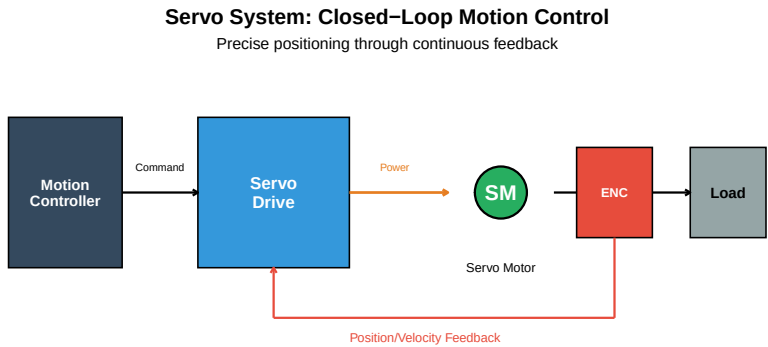


Table 16.8: VFD vs. Servo System Comparison

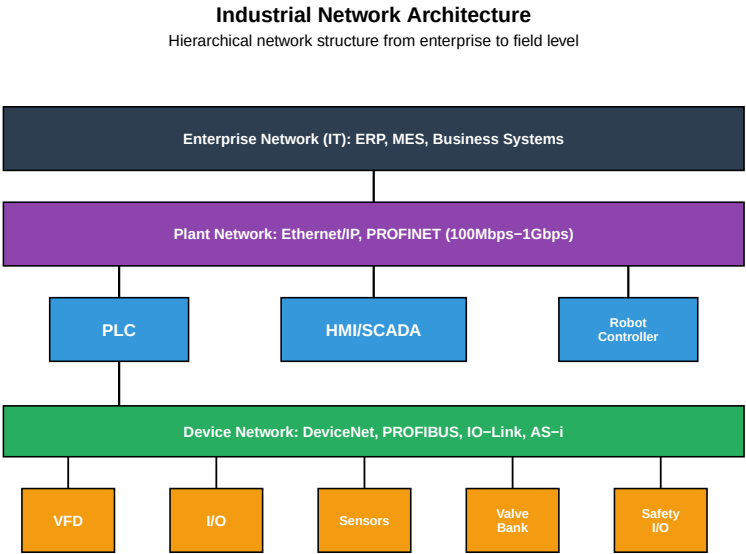
Characteristic	VFD + Induction Motor	Servo System
Control Type	Open loop (usually)	Closed loop (always)
Feedback	Optional encoder	High-resolution encoder required
Positioning Accuracy	±1-5% of speed	±0.01° or better
Dynamic Response	Moderate (100-500ms)	Fast (1-10ms)
Torque at Zero Speed	Limited (10-20%)	100% continuous

Cost	Lower (\$500-5000)	Higher (\$2000-20000)
Typical Application	Fans, pumps, conveyors	Robotics, CNC, packaging

16.7 Industrial Communication Networks

Modern automation systems rely on communication networks to connect devices, share data, and enable coordinated control.

16.7.1 Network Architecture



16.7.2 Common Industrial Protocols

Table 16.9: Common Industrial Communication Protocols

Protocol	Developer	Medium	Speed	Application
Ethernet/IP	ODVA (Rockwell)	Ethernet	100Mbps/1Gbps	General automation, I/O
PROFINET	PI (Siemens)	Ethernet	100Mbps/1Gbps	Siemens ecosystem, motion control
Modbus TCP	Modicon/Schneider	Ethernet	100Mbps	Simple, open, legacy systems
EtherCAT	Beckhoff	Ethernet	100Mbps	High-speed motion, precision positioning
DeviceNet	ODVA	CAN-based	500kbps	I/O, drives (legacy)
PROFIBUS	PI (Siemens)	RS-485	12Mbps	I/O, drives (legacy)

IO-Link	IO-Link Consortium	Point-to-point	230.4kbps	Smart sensors
OPC UA	OPC Foundation	Ethernet/Any	Varies	IT/OT integration, IIoT

Protocol Selection Guide

Choose Ethernet/IP if: - Using Allen-Bradley/Rockwell equipment - Need general-purpose industrial Ethernet - Require seamless integration with IT networks

Choose PROFINET if: - Using Siemens equipment - Need deterministic communication for motion control - Require IRT (Isochronous Real-Time) performance

Choose EtherCAT if: - Need highest speed/lowest latency - High-precision motion control - Many axes of coordinated motion

Choose Modbus TCP if: - Simple, low-cost solution needed - Connecting legacy equipment - Open protocol preference (no licensing)

Choose OPC UA if: - Need IT/OT convergence - IIoT/Industry 4.0 implementation - Secure, platform-independent communication

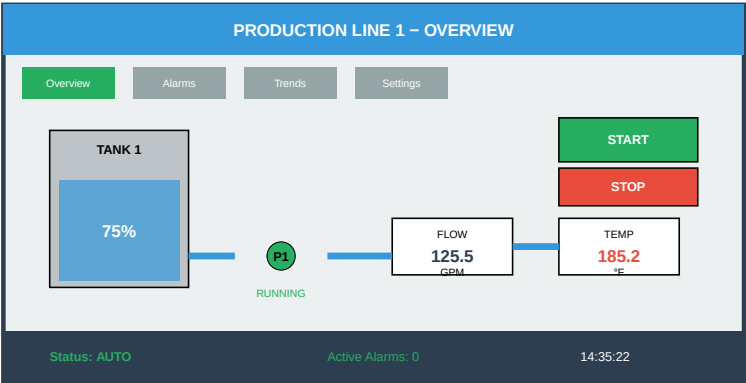
16.8 Human-Machine Interface (HMI) and SCADA

16.8.1 HMI Overview

An **HMI (Human-Machine Interface)** provides operators with a visual interface to monitor and control automated processes.

HMI Screen Example

Typical operator interface showing process status and controls



16.8.2 HMI Design Best Practices

Table 16.10: HMI Design Best Practices

Principle	Best Practice
Situational Awareness	Show abnormal conditions prominently; operator should know status at a
Alarm Management	Prioritize alarms (critical/warning/info); avoid alarm floods; require ackno
Navigation	Consistent layout; max 3 clicks to any screen; clear hierarchy
Color Usage	Use color sparingly for meaning; gray for normal; avoid red/green for criti
Data Display	Show trends, not just values; use appropriate precision; include units
Controls	Confirm destructive actions; use interlocks; provide feedback

16.8.3 SCADA Systems

SCADA (Supervisory Control and Data Acquisition) systems provide centralized monitoring and control across multiple locations or processes.

Table 16.11: SCADA System Components

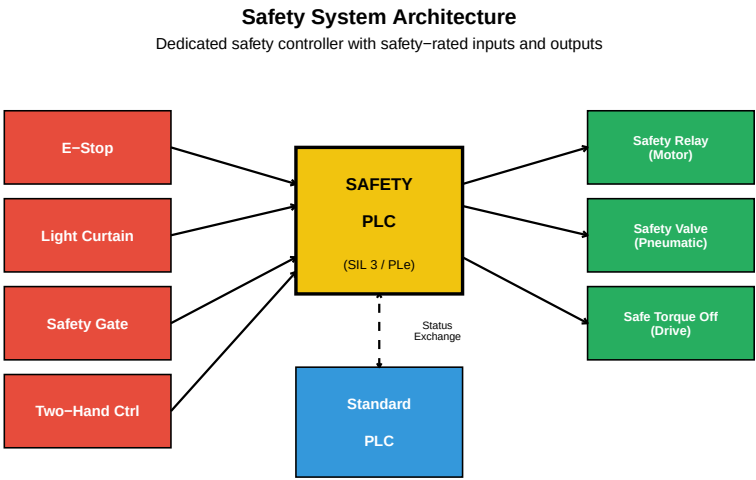
Component	Function
MTU (Master Terminal Unit)	Central server running SCADA software; processes data, execute
RTU (Remote Terminal Unit)	Field device that collects data from sensors and sends to MTU
Communication Network	Links MTU to RTUs; can be radio, cellular, satellite, fiber
HMI/Workstations	Operator interface for monitoring and control
Historian Database	Stores historical process data for trending and analysis

Alarm Server	Manages alarm generation, notification, and logging	Bui
--------------	---	-----

16.9 Safety in Automation

Safety is paramount in automated systems. Safety systems protect personnel and equipment from harm.

16.9.1 Safety System Architecture



16.9.2 Safety Integrity Levels (SIL)

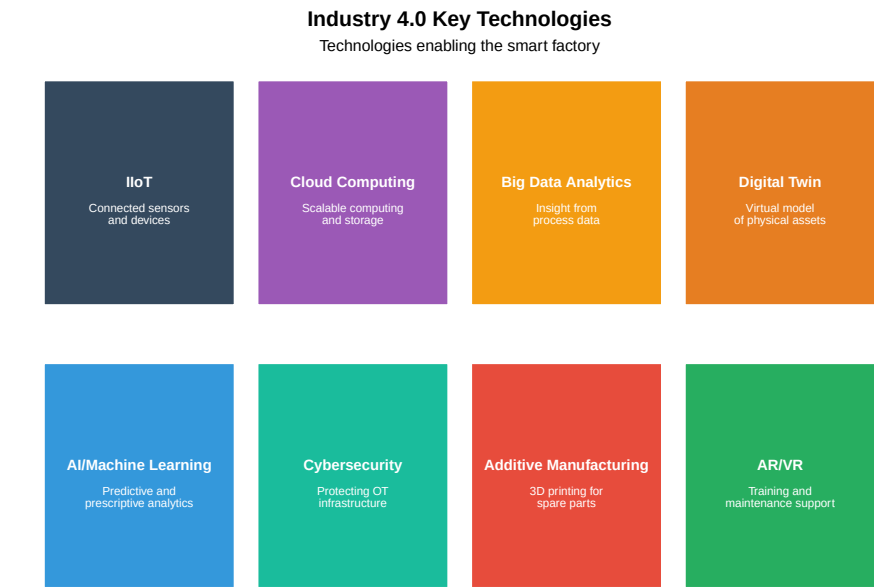
Table 16.12: Safety Integrity Levels (IEC 61508)

SIL Level	PFD (avg)	Risk Reduction Factor	Risk Level	Example
SIL 1	10^{-2} to $<10^{-1}$	10-100	Minor injury possible; first line of defense	Warning
SIL 2	10^{-3} to $<10^{-2}$	100-1,000	Serious injury possible; general industrial	Machine
SIL 3	10^{-4} to $<10^{-3}$	1,000-10,000	Death or severe injury; process industry standard	Emergen
SIL 4	10^{-5} to $<10^{-4}$	10,000-100,000	Catastrophic; nuclear, aviation	Nuclear

16.10 Industry 4.0 and IIoT

Industry 4.0 represents the fourth industrial revolution, characterized by the integration of digital technologies into manufacturing.

16.10.1 Industry 4.0 Technologies



16.10.2 IIoT Architecture

Table 16.13: IIoT Architecture

Layer	Components	Function
Edge/Device Layer	Sensors, actuators, PLCs, gateways, edge devices	Collect data, local processing
Network Layer	Industrial Ethernet, Wi-Fi, 5G, LoRaWAN, MQTT	Secure data transmission
Platform Layer	Cloud/on-premise servers, databases, data lakes	Store, process, and analyze data
Application Layer	Analytics dashboards, AI/ML models, business applications	Derive insights and optimize operations

16.10.3 Predictive Maintenance Example

```
# Simulating predictive maintenance with vibration data
set.seed(42)

# Generate 30 days of vibration readings
days <- 1:30
```

```

baseline_vibration <- 2.5 # mm/s RMS (normal)

# Simulate gradual bearing degradation
vibration <- baseline_vibration +
  0.05 * days + # Gradual increase
  0.5 * exp((days - 25)/5) * (days > 20) + # Accelerating failure
  rnorm(30, 0, 0.2) # Normal variation

# Alert thresholds
warning_threshold <- 4.0
alarm_threshold <- 6.0

# Find when thresholds crossed
warning_day <- min(which(vibration > warning_threshold))
alarm_day <- min(which(vibration > alarm_threshold))

```

Warning in min(which(vibration > alarm_threshold)): no non-missing arguments to min; returning Inf

```

cat("Predictive Maintenance Analysis:\n")
cat("      \n")
cat("Baseline vibration:", baseline_vibration, "mm/s RMS\n")
cat("Warning threshold:", warning_threshold, "mm/s RMS\n")
cat("Alarm threshold:", alarm_threshold, "mm/s RMS\n\n")
cat("Warning triggered on day:", warning_day, "\n")
cat("Alarm triggered on day:", alarm_day, "\n")
cat("Days of warning before alarm:", alarm_day - warning_day, "\n")
cat("\nRecommendation: Schedule bearing replacement before day", alarm_day, "\n")

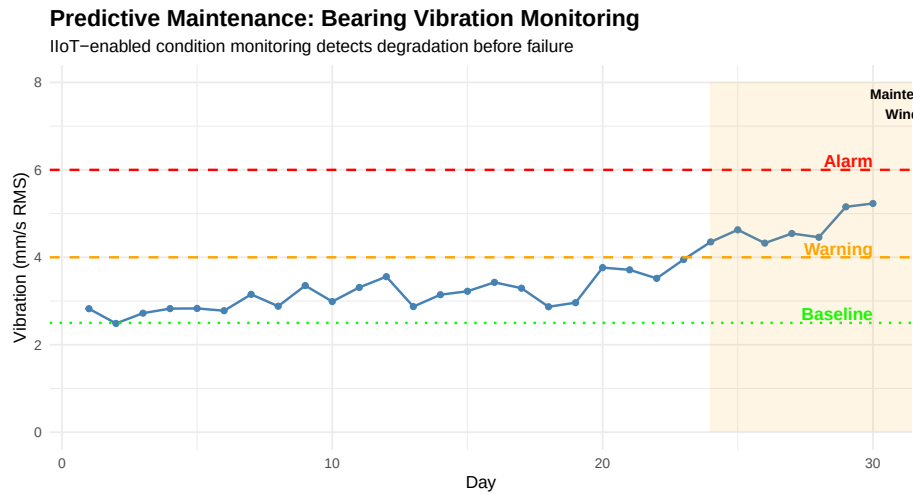
```

Predictive Maintenance Analysis:

Baseline vibration: 2.5 mm/s RMS
 Warning threshold: 4 mm/s RMS
 Alarm threshold: 6 mm/s RMS

Warning triggered on day: 24
 Alarm triggered on day: Inf
 Days of warning before alarm: Inf

Recommendation: Schedule bearing replacement before day Inf



16.11 Video Resources

16.11.1 Introduction to PLCs

16.11.2 Industrial Networks Explained

16.12 Summary

Industrial automation integrates multiple technologies to improve manufacturing efficiency, quality, and safety:

1. **Automation types** range from fixed (high-volume, single product) to flexible (low-volume, high variety)
2. **PLCs** are the workhorses of industrial control, using scan cycles to process inputs and control outputs
3. **Sensors** provide feedback on process conditions (discrete, analog, and smart)
4. **Motor control** through VFDs and servo systems enables precise speed and position control
5. **Industrial networks** connect devices using protocols like Ethernet/IP, PROFINET, and Modbus
6. **HMI and SCADA** provide operator interface for monitoring and control
7. **Safety systems** use dedicated controllers and devices to protect personnel
8. **Industry 4.0** brings IIoT, cloud computing, and AI to enable smart manufacturing

16.13 Review Questions

Question 1: Compare and contrast fixed, programmable, and flexible automation. Give an example application for each.

Answer:

Aspect	Fixed	Programmable	Flexible
Product variety	Single product	Batches of different products	High variety, mixed production
Volume	Very high	Medium-high	Low-medium
Changeover	Difficult, expensive	Hours to days	Minutes
Investment	Very high	High	Very high
Flexibility	None	Limited	High

Examples:

1. **Fixed Automation:** Automotive transfer line producing engine blocks
 - Same machining operations repeated millions of times
 - Dedicated stations optimized for specific operations
 - Changing product would require complete retooling
2. **Programmable Automation:** CNC machining job shop
 - Different parts programmed and run in batches
 - Significant setup time between batches
 - Equipment reprogrammable but not instantaneous
3. **Flexible Automation:** Robotic welding cell in aerospace
 - Different assemblies run with quick changeover
 - Robot program changes automatically based on part ID
 - Same cell handles multiple product variants

Question 2: Explain the PLC scan cycle and why scan time is important.

Answer:

The PLC Scan Cycle consists of four main phases executed continuously:

1. **Input Scan** (~1ms): Read all physical inputs and store values in the input image table
2. **Program Execution** (~5-20ms): Execute the user program (ladder logic, etc.) using input image data
3. **Output Update** (~1ms): Write output image table values to physical outputs
4. **Housekeeping** (~1-3ms): Communications, diagnostics, self-testing

Total scan time = Sum of all phases, typically 10-50ms

Why Scan Time Matters:

1. **Response Time:** An input change isn't acted upon until the next scan completes. If scan time is 50ms, worst-case response time is 50ms.
2. **Safety:** Fast-moving machinery requires short scan times for safety functions. A 100ms scan time with a motor at 1800 RPM means 3 shaft revolutions between scans.
3. **Application Limits:**
 - Motion control may need <1ms scan times (special motion PLCs)
 - High-speed counting needs special modules
 - Safety PLCs have guaranteed maximum scan times
4. **Program Size Impact:** Larger programs take longer to execute, increasing scan time. Optimization may be needed for time-critical applications.
5. **Communication Delays:** Network updates typically happen once per scan, affecting data freshness.

Question 3: What are the key differences between PNP and NPN sensor outputs? When would you use each?

Answer:

PNP (Sourcing): - Sensor output sources current TO the load - When activated, output connects to +V - Load is connected between sensor output and 0V - Current flows: +V → Sensor → Output → Load → 0V - Common in North America and Europe

NPN (Sinking): - Sensor output sinks current FROM the load - When activated, output connects to 0V (ground) - Load is connected between sensor output and +V - Current flows: +V → Load → Output → Sensor → 0V - Common in Asia (Japan)

Selection Criteria:

1. **PLC Input Type:**
 - Sinking PLC inputs work with PNP (sourcing) sensors
 - Sourcing PLC inputs work with NPN (sinking) sensors
2. **Regional Standards:**
 - North America: PNP is standard
 - Europe: PNP is standard
 - Japan/Asia: NPN is common
3. **Safety Considerations:**
 - PNP: A ground fault can cause false ON signal
 - NPN: A ground fault typically causes safe-off condition
 - For safety applications, consider sensor design carefully
4. **Existing Infrastructure:**

- Match new sensors to existing system type
- Converting between types requires different PLC input cards

Question 4: A VFD is controlling a 10 HP, 4-pole motor rated at 1750 RPM and 460V at 60Hz. Calculate the motor speed at 30Hz and 45Hz. What voltage should the VFD supply at these frequencies?

Answer: (live — change the numbers to check your own version)

```
#| label: q4-answer
# Motor parameters
poles <- 4
rated_voltage <- 460 # V
rated_frequency <- 60 # Hz
rated_rpm <- 1750

# Calculate synchronous speed and slip at rated conditions
sync_speed_60 <- 120 * rated_frequency / poles
slip_rpm <- sync_speed_60 - rated_rpm
slip_percent <- (slip_rpm / sync_speed_60) * 100

cat("Motor Analysis:\n")
cat("Synchronous speed at 60Hz:", sync_speed_60, "RPM\n")
cat("Rated speed:", rated_rpm, "RPM\n")
cat("Slip:", slip_rpm, "RPM (", round(slip_percent, 1), "%)\n\n")

# V/Hz ratio
vHz_ratio <- rated_voltage / rated_frequency
cat("V/Hz Ratio:", round(vHz_ratio, 2), "V/Hz\n\n")

# Calculate at 30Hz
freq_30 <- 30
sync_30 <- 120 * freq_30 / poles
speed_30 <- sync_30 - slip_rpm # Assuming constant slip in RPM
voltage_30 <- freq_30 * vHz_ratio

cat("At 30Hz:\n")
cat("  Synchronous speed:", sync_30, "RPM\n")
cat("  Motor speed (approx):", speed_30, "RPM\n")
cat("  VFD output voltage:", voltage_30, "V\n\n")

# Calculate at 45Hz
freq_45 <- 45
sync_45 <- 120 * freq_45 / poles
speed_45 <- sync_45 - slip_rpm
voltage_45 <- freq_45 * vHz_ratio
```

```
cat("At 45Hz:\n")
cat("  Synchronous speed:", sync_45, "RPM\n")
cat("  Motor speed (approx):", speed_45, "RPM\n")
cat("  VFD output voltage:", voltage_45, "V\n")
```

Note: The calculation assumes constant slip in RPM, which is an approximation. Actual slip varies somewhat with load and frequency. The V/Hz ratio is maintained constant to provide constant torque capability below base speed.

Question 5: Compare Ethernet/IP and PROFINET. When would you choose each?

Answer:

Feature	Ethernet/IP	PROFINET
Developer	ODVA (Rockwell Automation)	PROFIBUS International (Siemens)
Base Protocol	CIP over TCP/UDP	Based on standard Ethernet
Real-time	CIP Motion for motion control	IRT (Isochronous Real-Time)
Determinism	Standard: non-deterministic; CIP Sync: deterministic	RT: soft real-time; IRT: hard real-time
Typical Cycle	2-10ms (standard); <1ms (CIP Motion)	1-10ms (RT); <1ms (IRT)
Ecosystem	Allen-Bradley, DeviceNet-heritage	Siemens, PROFIBUS-heritage

Choose Ethernet/IP when: - Using Allen-Bradley/Rockwell PLCs - Integrating with DeviceNet legacy systems - Standard industrial Ethernet needs - North American installations (common) - Need CIP protocol compatibility

Choose PROFINET when: - Using Siemens PLCs (S7-1200, S7-1500) - Migrating from PROFIBUS installations - Need IRT for high-performance motion - European installations (common) - Require I-Device functionality

General Guidance: - Match the protocol to your PLC vendor's ecosystem - Both are capable industrial Ethernet solutions - Consider existing infrastructure and expertise - For motion, evaluate specific timing requirements

Question 6: What is the difference between HMI and SCADA? Where would each be used?

Answer:

HMI (Human-Machine Interface): - Local operator interface for a single machine or process - Typically a touchscreen panel mounted on or near

equipment - Communicates directly with one or few PLCs - Provides real-time control and monitoring - Limited data storage (trends, alarms) - Examples: Allen-Bradley PanelView, Siemens Comfort Panel

SCADA (Supervisory Control and Data Acquisition): - Centralized system monitoring multiple processes/locations - Server-based architecture with multiple client workstations - Communicates with many PLCs/RTUs across large distances - Supervisory control - high-level commands, not direct control - Extensive historical data collection and analysis - Examples: Wonderware, Ignition, FactoryTalk View SE

Where Each is Used:

Application	HMI	SCADA
Single CNC machine		
Packaging line		
Entire factory floor		
Water treatment plant	(local)	(central)
Oil pipeline network		
Building automation	(per zone)	(campus-wide)
Power grid		

Key Distinction: HMI is typically embedded/local for one process; SCADA is a system supervising many processes across the enterprise.

Question 7: Explain what SIL 3 means and give an example of a SIL 3 application.

Answer:

SIL 3 (Safety Integrity Level 3) is defined in IEC 61508 and represents:

- **PFD (Probability of Failure on Demand):** 10^{-5} to 10^{-3} (0.01% to 0.1%)
- **Risk Reduction Factor:** 1,000 to 10,000
- **Availability:** 99.9% to 99.99%

What SIL 3 Means: - The safety function will fail to operate when needed less than 1 in 1,000 demands - Requires redundant architecture (typically 2oo3 or 1oo2D) - Requires certified safety-rated components - Requires rigorous design, testing, and validation processes - Requires regular proof testing and maintenance

SIL 3 Requirements: - Hardware fault tolerance ≥ 1 (single fault won't cause dangerous failure) - Safe Failure Fraction $> 90\%$ (for Type A components) - Systematic capability rating of SC3 - Extensive documentation and lifecycle management

SIL 3 Application Examples:

1. **Emergency Shutdown System (ESD)** in oil refinery
 - Detects dangerous conditions (high pressure, fire)
 - Shuts down process and isolates fuel sources
 - Failure could result in explosion and fatalities
2. **Burner Management System (BMS)**
 - Controls furnace/boiler ignition sequence
 - Monitors flame presence and fuel/air ratio
 - Prevents furnace explosions
3. **High Integrity Pressure Protection System (HIPPS)**
 - Prevents overpressure in pipelines
 - Isolates high-pressure source before relief valve capacity exceeded
 - Protects against catastrophic pipeline rupture

Question 8: Describe three benefits of implementing IIoT in a manufacturing facility and give a specific example of each.

Answer:

1. Predictive Maintenance

Benefit: Detect equipment degradation before failure, enabling planned maintenance during scheduled downtime rather than emergency repairs.

Example: Vibration sensors on a conveyor gearbox transmit data to cloud analytics. Machine learning algorithms detect bearing wear signature 3 weeks before expected failure. Maintenance schedules replacement during next planned shutdown, avoiding \$50,000 in lost production from unplanned downtime.

2. Real-Time Production Visibility

Benefit: Instant access to production metrics, OEE, and quality data from anywhere, enabling faster decision-making.

Example: Dashboard shows real-time OEE for all production lines on plant manager's tablet. Sudden drop in Line 3 performance triggers alert. Investigation reveals material feed issue; corrected within 30 minutes instead of waiting for end-of-shift report.

3. Energy Optimization

Benefit: Monitor energy consumption at machine level, identify waste, and optimize based on production schedule.

Example: Power meters on each machine report to central system. Analysis reveals HVAC in unused areas runs at full capacity during weekends. Automated scheduling reduces weekend HVAC operation, saving 15% on energy bills (\$30,000/year).

Additional Benefits:

4. **Quality Traceability** - Every part traced through production with complete process data

5. **Remote Monitoring** - Engineers can diagnose issues without traveling to site
 6. **Supply Chain Integration** - Real-time inventory and production data shared with suppliers
-

16.14 References

1. Bolton, W. (2015). *Programmable Logic Controllers* (6th ed.). Newnes.
2. Petruzella, F.D. (2017). *Programmable Logic Controllers* (5th ed.). McGraw-Hill Education.
3. Rehg, J.A., & Sartori, G.J. (2013). *Industrial Electronics*. Pearson.
4. IEC 61131-3:2013. *Programmable Controllers - Part 3: Programming Languages*.
5. IEC 61508:2010. *Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems*.
6. ODVA. (2020). *The CIP Networks Library*. ODVA, Inc.
7. Siemens AG. (2019). *PROFINET System Description*. Siemens.
8. Gilchrist, A. (2016). *Industry 4.0: The Industrial Internet of Things*. Apress.
9. ISA-95/IEC 62264. *Enterprise-Control System Integration*.
10. Groover, M.P. (2016). *Automation, Production Systems, and Computer-Integrated Manufacturing* (4th ed.). Pearson.

Chapter 17

Quality Management Systems

17.1 Learning Objectives

After completing this chapter, you will be able to:

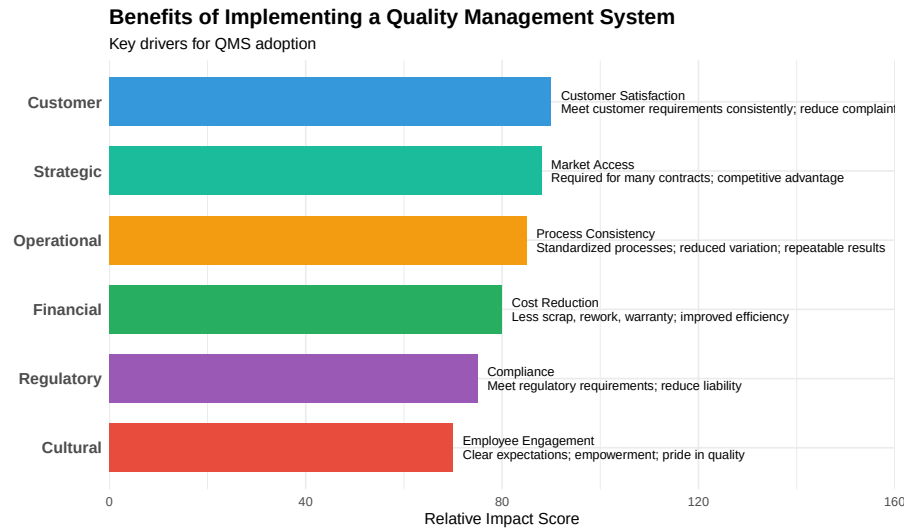
1. Define quality management systems and explain their purpose in manufacturing
 2. Describe the structure and requirements of ISO 9001:2015
 3. Identify industry-specific quality standards (IATF 16949, AS9100, ISO 22000)
 4. Explain the documentation hierarchy and control requirements
 5. Conduct internal audits using systematic methodologies
 6. Implement effective corrective and preventive action (CAPA) processes
 7. Understand management review and continuous improvement requirements
 8. Integrate quality tools (SPC, PFMEA, MSA) within a QMS framework
-

17.2 Introduction to Quality Management Systems

A **Quality Management System (QMS)** is a formalized system that documents processes, procedures, and responsibilities for achieving quality policies and objectives. It coordinates and directs an organization's activities to meet

customer and regulatory requirements while continuously improving effectiveness.

17.2.1 Why Implement a QMS?



17.2.2 The Evolution of Quality Management

Table 17.1: Evolution of Quality Management

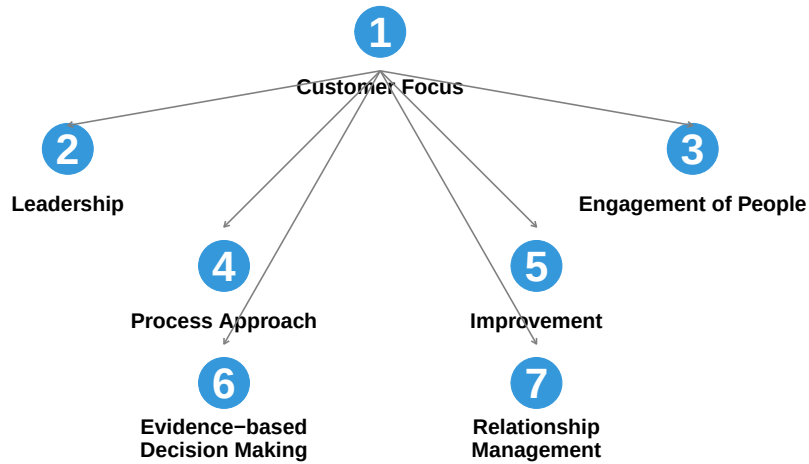
Era	Approach	Focus
Pre-1900s	Craftsmanship	Individual skill; master craftsmen
1900-1940s	Inspection	Sort good from bad; end-of-line inspection
1940-1960s	Statistical Quality Control	Control charts; sampling; Shewhart, Deming
1960-1980s	Quality Assurance	Prevention; systems approach; documented procedures
1980-2000s	Total Quality Management	Company-wide quality; customer focus; continuous improvement
2000s-Present	Integrated Management	Risk-based thinking; integration with business strategy

17.2.3 Quality Management Principles

ISO 9000 defines seven quality management principles that form the foundation of all QMS standards:

Seven Quality Management Principles (ISO 9000:2015)

Customer Focus is the primary principle; others support it



17.3 ISO 9001:2015 Structure

ISO 9001:2015 is the international standard for quality management systems. It uses the High-Level Structure (HLS) common to all ISO management system standards.

17.3.1 The Ten Clauses

ISO 9001:2015 Structure – The Ten Clauses

Clauses 4–10 contain the auditable requirements

1	Scope	Defines applicability of the standard
2	Normative References	References to ISO 9000:2015
3	Terms and Definitions	Terms used in the standard
4	Context of the Organization	Understanding the organization and its context
5	Leadership	Top management commitment and policy
6	Planning	Actions to address risks and opportunities
7	Support	Resources, competence, awareness, communication
8	Operation	Operational planning and control
9	Performance Evaluation	Monitoring, measurement, analysis, evaluation
10	Improvement	Nonconformity, corrective action, continual improvement

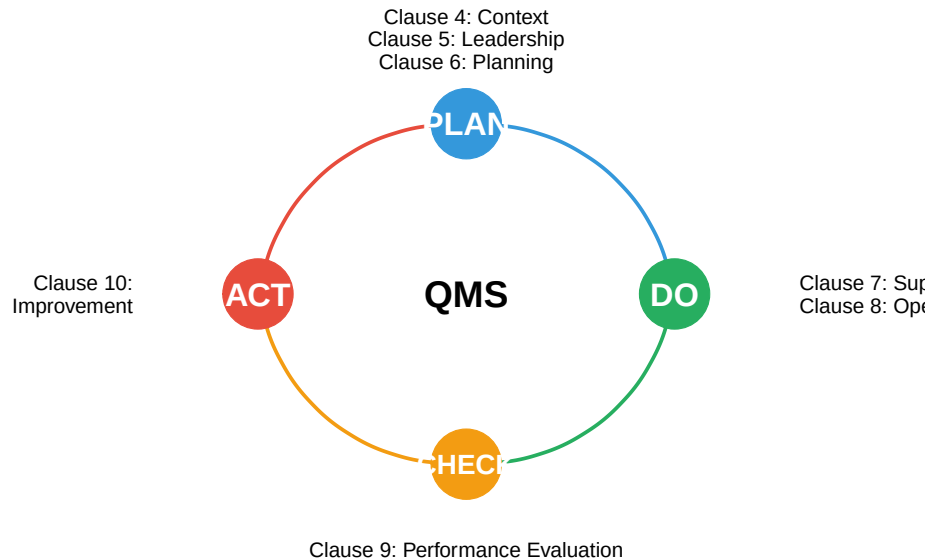
17.3.2 The Process Approach and PDCA

ISO 9001:2015 emphasizes the **process approach** combined with the **Plan-Do-Check-Act (PDCA)** cycle:

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

PDCA Cycle and ISO 9001:2015 Clauses

The continuous improvement cycle embedded in the standard



17.3.3 Risk-Based Thinking

ISO 9001:2015 introduced **risk-based thinking** throughout the QMS:

Table 17.2: Risk-Based Thinking in ISO 9001:2015

Clause	Requirement	Risk_Application
4.1	Context of the Organization	Identify internal/external issues that could affect QMS outcomes
4.2	Interested Parties	Understand needs/expectations that could impact quality
6.1	Actions to Address Risks and Opportunities	Plan actions to address risks and opportunities; integrate
8.1	Operational Planning and Control	Control processes considering risks identified in planning
9.1	Monitoring and Measurement	Monitor effectiveness of actions taken to address risks
10.2	Nonconformity and Corrective Action	Analyze nonconformities; update risks/opportunities as needed

Understanding Risk-Based Thinking

Risk-based thinking doesn't require formal risk management (like FMEA for every process), but rather:

1. **Consider risks** when designing and implementing the QMS
2. **Preventive action** is built into the system, not a separate activity

3. **Proportional response** - more rigorous controls for higher-risk processes
4. **Opportunities** are also considered (not just negative risks)

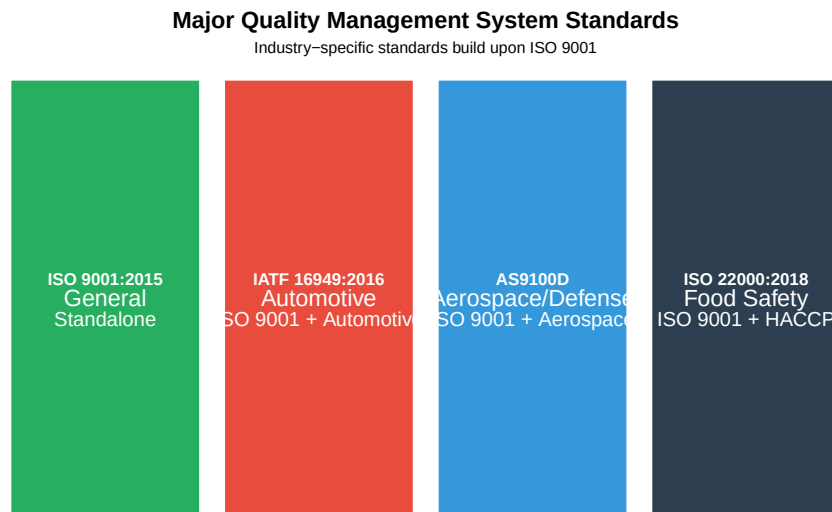
Example: A food manufacturer identifies “allergen cross-contamination” as a significant risk. Risk-based thinking leads them to: - Dedicated production lines for allergen-containing products - Enhanced cleaning verification procedures - Staff training on allergen awareness - Increased inspection frequency after changeovers

The extent of controls is proportional to the risk level.

17.4 Industry-Specific Quality Standards

Different industries have developed standards that build upon ISO 9001 with additional sector-specific requirements.

17.4.1 Comparison of Major Standards



17.4.2 IATF 16949:2016 (Automotive)

IATF 16949 is the automotive quality standard, required by most major OEMs (Ford, GM, Toyota, VW, etc.).

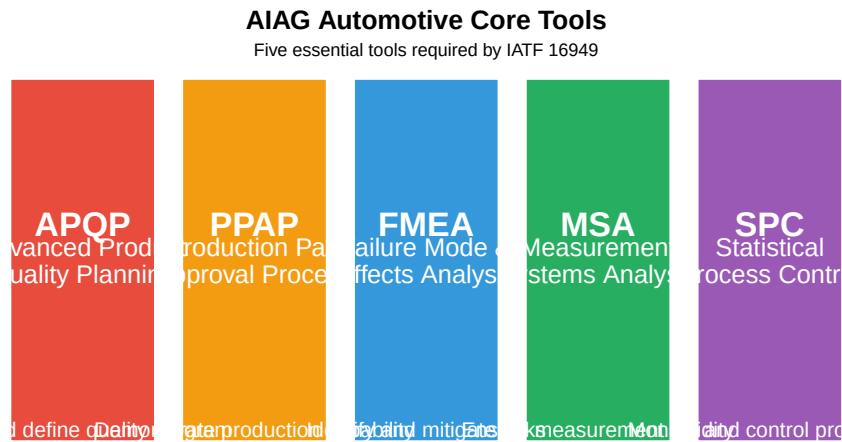
Table 17.3: IATF 16949 Additional Requirements Beyond ISO 9001

Requirement Area	ISO 9001	IATF 16949 Addition
------------------	----------	---------------------

Product Safety	Implied	Explicit requirements for product safety processes
APQP/PPAP	Not specified	APQP phases required; PPAP submission mandatory
Control Plans	Not specified	Control plans required for all parts; specific format
MSA	General requirement	MSA studies required per AIAG MSA manual
SPC	General requirement	SPC for all special characteristics
Supplier Quality	Basic purchasing	Supplier development; second-party audits; IATF certification
Warranty Management	Complaint handling	NTF analysis; warranty data analysis; field return analysis
Manufacturing Feasibility	Review of requirements	Documented manufacturing feasibility review

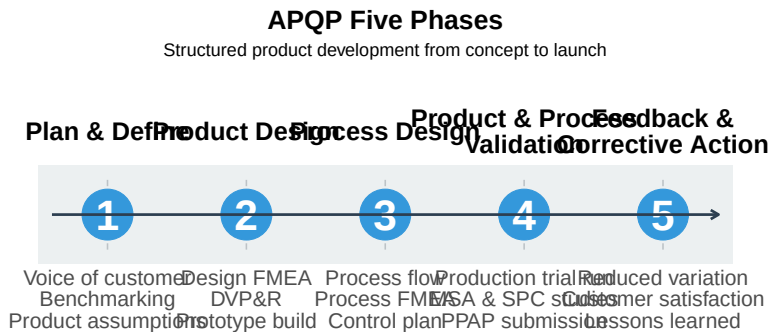
17.4.3 Automotive Core Tools

IATF 16949 requires the use of **five core tools**:



17.4.3.1 APQP: Advanced Product Quality Planning

APQP is a structured framework for developing products that ensures customer requirements are met through careful planning and defined milestones.



Example: New Brake Caliper Development

A Tier 1 supplier developing a new brake caliper for an OEM would follow APQP:

- **Phase 1:** Gather customer requirements (stopping distance, weight limits, environmental conditions), benchmark competitor products
- **Phase 2:** Design the caliper, create Design FMEA (what if piston seal fails?), build prototypes for testing
- **Phase 3:** Design manufacturing process, create Process FMEA, develop control plan for machining and assembly
- **Phase 4:** Run production trial, validate $C_{pk} > 1.67$ for critical dimensions, submit PPAP package
- **Phase 5:** Monitor warranty returns, implement kaizen improvements, document lessons for next program

17.4.3.2 PPAP: Production Part Approval Process

PPAP provides evidence that the supplier understands customer requirements and can consistently produce conforming parts during actual production runs.

Table 17.4: PPAP 18 Elements Required for Submission

Element	Description
1. Design Records	Part drawings, CAD files, specifications
2. Engineering Change Documents	Documentation of any design changes since original approval
3. Customer Engineering Approval	Evidence of customer sign-off on design
4. Design FMEA	Risk analysis of product design failure modes
5. Process Flow Diagram	Visual representation of manufacturing steps
6. Process FMEA	Risk analysis of process failure modes
7. Control Plan	Document describing controls for each process step
8. MSA Studies	Gage R&R studies proving measurement system adequacy
9. Dimensional Results	Actual measurements vs. specifications for all dimensions
10. Material/Performance Test Results	Test results for material properties and performance requirements
11. Initial Process Studies (SPC)	C_{pk}/P_{pk} studies demonstrating process capability
12. Qualified Lab Documentation	Scope and accreditation of testing laboratories
13. Appearance Approval Report	Required for parts with color, grain, or surface requirements
14. Sample Production Parts	Parts from production run for customer evaluation
15. Master Sample	Retained reference sample for future comparison
16. Checking Aids	Fixtures, gages, or templates used to inspect parts
17. Customer-Specific Requirements	OEM-specific requirements (Ford, GM, Toyota, etc.)
18. Part Submission Warrant (PSW)	Summary document with supplier certification

PPAP Submission Levels:

Level	Requirements	When Used
1	PSW only	Low-risk parts, trusted suppliers
2	PSW + samples + limited data	Standard submission
3	PSW + samples + complete data	Default level; new parts/suppliers
4	PSW + other per customer direction	Customer-specified requirements
5	PSW + samples + complete data at supplier site	Full documentation review on-site

Example: Fuel Injector PPAP Submission

A supplier submitting PPAP for a new fuel injector would include:

- **Dimensional results:** 100% inspection of 300 parts showing all 47 dimensions in spec
- **MSA study:** Gage R&R < 10% for critical spray angle measurement
- **Process capability:** Cpk = 1.89 for nozzle hole diameter (spec requires Cpk = 1.67)
- **Material certs:** Steel chemistry and hardness test results
- **Performance tests:** Flow rate, spray pattern, durability (1 million cycles)
- **PSW:** Signed warrant certifying all requirements met

17.4.3.3 FMEA: Failure Mode and Effects Analysis

FMEA is a systematic method for identifying potential failures, their causes, and effects, then prioritizing actions to reduce risk. Two types are used in automotive:

- **Design FMEA (DFMEA):** Analyzes potential failures in product design
- **Process FMEA (PFMEA):** Analyzes potential failures in manufacturing processes

Table 17.5: Process FMEA Structure and Example

Column	Description	Example
Process Step	The operation being analyzed	Torque fastener to 25 Nm
Potential Failure Mode	How the process could fail to meet requirements	Under-torque / Over-torque
Potential Effect(s)	Impact on customer if failure occurs	Joint loosens in service; w
Severity (S)	1-10 rating of effect seriousness (10 = safety/regulatory)	10 (safety critical)
Potential Cause(s)	Root cause of the failure mode	Worn tool, wrong program
Occurrence (O)	1-10 rating of cause likelihood (10 = very frequent)	4 (occasional occurrence)

Current Controls	Existing prevention or detection controls	Torque monit
Detection (D)	1-10 rating of detection ability (10 = cannot detect)	3 (high detect
RPN	$S \times O \times D$ = Risk Priority Number	$10 \times 4 \times 3 =$
Recommended Actions	Actions to reduce S, O, or D	Add DC torq

Action Priority Based on RPN:

RPN-Based Action Priority

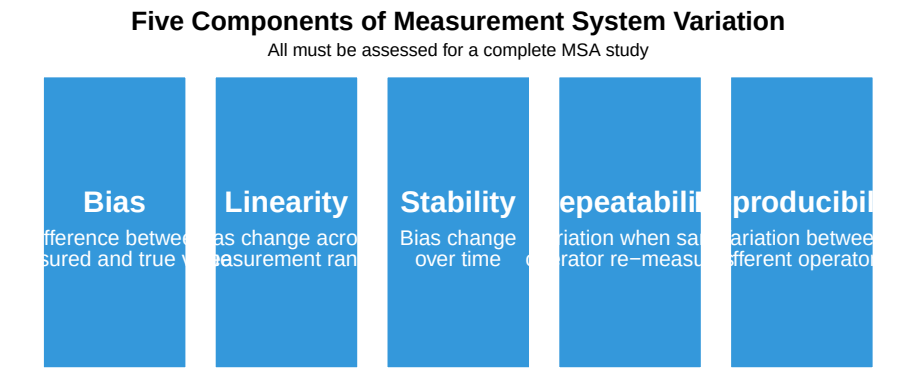


Example: Engine Block Machining PFMEA

Step	Failure Mode	Effect	S	Cause	O	Control	D	RPN	Action
Bore cylinder	Oversize bore	Piston slap, oil consumption	7	Tool wear	5	SPC chart, tool life monitor	4	140	Add in-process bore gage
Hone surface	Insufficient crosshatch	Poor ring seal, oil burning	8	Wrong hone stones	3	Visual inspection	6	144	Add surface roughness measurement
Wash block	Residual chips	Engine seizure	9	Blocked nozzle	4	Cleanliness spec check	5	180	Add particle counter, automated flush verification

17.4.3.4 MSA: Measurement Systems Analysis

MSA ensures that measurement systems provide reliable data for decision-making. Poor measurement systems can cause good parts to be rejected (false rejects) or bad parts to be accepted (false accepts).



Gage R&R Study Acceptance Criteria:

%GRR	Interpretation	Action
< 10%	Excellent	Acceptable for all applications
10-30%	Acceptable	May be acceptable depending on application, cost of gage, cost of repairs
> 30%	Unacceptable	Measurement system must be improved; cannot make reliable decisions

Number of Distinct Categories (ndc): Should be 5 to adequately discriminate between parts.

Example: Gage R&R Study for Shaft Diameter

A supplier conducts a Gage R&R study for a micrometer used to measure shaft diameter (spec: 25.00 ± 0.05 mm):

- Study Setup:** - 3 operators measure 10 parts, 3 times each = 90 measurements
- Parts selected to represent the range of production variation

Results: | Source | Variance | % Contribution | |:—|:—|:—| |

Process Capability Indices:

$$C_p = \frac{USL - LSL}{6\sigma}$$

$$C_{pk} = \min\left(\frac{USL - \bar{x}}{3\sigma}, \frac{\bar{x} - LSL}{3\sigma}\right)$$

Cpk Value	Interpretation	PPM Defective
< 1.00	Not capable	> 2,700
1.00 - 1.33	Marginally capable	64 - 2,700
1.33 - 1.67	Capable	0.6 - 64
1.67	Highly capable (IATF requirement)	< 0.6

Example: SPC for Injection Molded Connector

A supplier monitors critical dimensions on an injection molded electrical connector:

Process Setup: - Critical dimension: Pin spacing = 2.54 ± 0.08 mm - Sample 5 parts every 30 minutes - X-bar and R charts maintained at the press

Results over one shift: - Process mean (X-double-bar) = 2.539 mm - Average range (R-bar) = 0.024 mm - Estimated $\sigma = R\text{-bar} / d_2 = 0.024 / 2.326 = 0.0103$ mm

Capability: - $C_p = (2.62 - 2.46) / (6 \times 0.0103) = 2.59$ - $C_{pk} = \min[(2.62 - 2.539)/(3 \times 0.0103), (2.539 - 2.46)/(3 \times 0.0103)] = \min[2.62, 2.56] = \mathbf{2.56}$

Conclusion: $C_{pk} = 2.56$ exceeds IATF requirement of 1.67. Process is highly capable.

17.4.4 AS9100D (Aerospace/Defense)

AS9100D adds aerospace-specific requirements to ISO 9001:

Table 17.6: AS9100D Key Additional Requirements

Area	Requirement
Configuration Management	Control product configuration throughout lifecycle; change management
First Article Inspection	FAI per AS9102; documented verification of first production parts
Counterfeit Parts Prevention	Controls to prevent counterfeit parts entering supply chain
Special Processes	NADCAP accreditation for special processes (welding, heat treat, NDT)
Product Safety	Product safety requirements; reporting of unsafe conditions
Risk Management	Explicit risk management process (often using AS/NZS 4360 or similar)

Human Factors	Consideration of human factors in process design
---------------	--

17.4.5 ISO 22000:2018 (Food Safety)

ISO 22000 integrates quality management with HACCP (Hazard Analysis Critical Control Points):

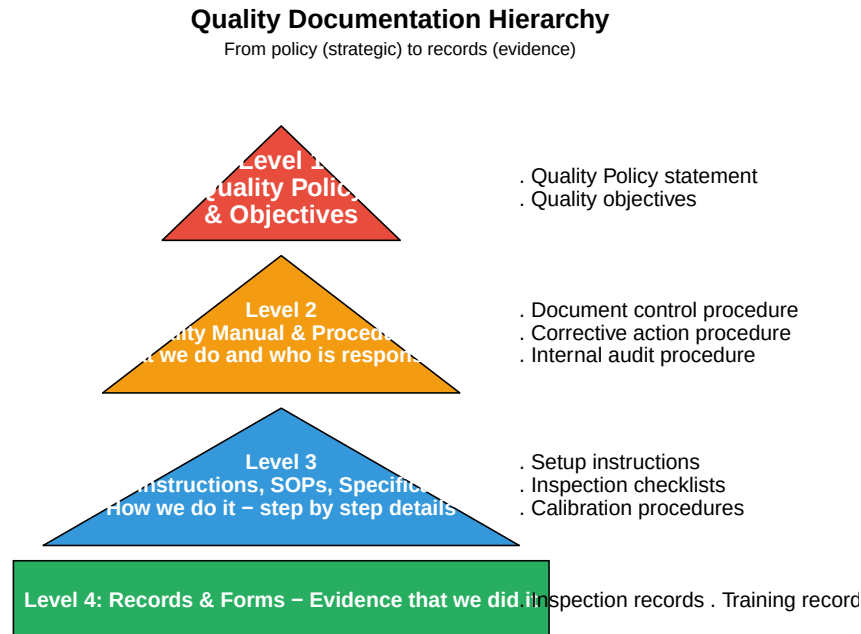
Table 17.7: ISO 22000 Food Safety M

Element	Description
Prerequisite Programs (PRPs)	Basic hygiene conditions: cleaning, pest control, personnel hygiene
Operational PRPs (OPRPs)	PRPs essential to control specific identified hazards
HACCP Plan	Critical Control Points (CCPs) for significant hazards; critical limits
Traceability	One-step-back, one-step-forward traceability; recall procedures
Emergency Preparedness	Procedures for food safety emergencies; recall/withdrawal
Validation and Verification	Validate control measures; verify HACCP plan effectiveness

17.5 Documentation Requirements

A QMS requires documented information to ensure consistency, provide evidence, and enable improvement.

17.5.1 Documentation Hierarchy



17.5.2 Document Control Requirements

Table 17.8: Document Control Requirements (ISO 9001 Clause 7.5)

Requirement	Description	Implementation
Approval	Documents reviewed and approved for adequacy before issue	Approval signatures
Review & Update	Reviewed, updated as necessary, and re-approved	Periodic review schedule
Identification	Identified with title, date, revision, author	Document numbering
Availability	Relevant versions available at points of use	Controlled copies available
Protection	Protected from loss, damage, unauthorized changes	Backup systems; access control
Obsolete Control	Obsolete documents identified and prevented from unintended use	Stamp 'OBSOLETE'
External Documents	External documents identified and distribution controlled	List of external documents
Record Retention	Records retained for specified periods; protected; retrievable	Retention matrix; storage

17.5.3 Documented Information Required by ISO 9001:2015

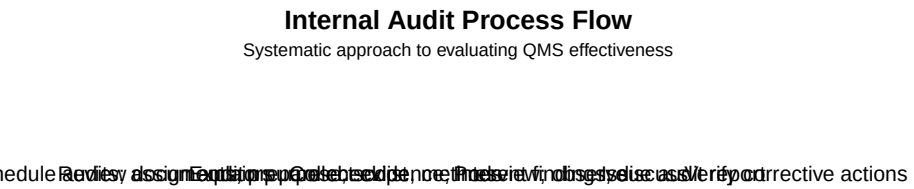
Table 17.9: Documented Information Required by ISO 9001:2015

Type	Clause	Requirement
Documents (shall be maintained)		
Documents (shall be maintained)	4.3	Scope of the QMS
Documents (shall be maintained)	5.2	Quality policy
Documents (shall be maintained)	6.2	Quality objectives
Documents (shall be maintained)	7.1.5	Monitoring and measuring resources (calibration)
Documents (shall be maintained)	7.2	Evidence of competence
Documents (shall be maintained)	8.1	Evidence of conformity to processes
Documents (shall be maintained)	8.5.1	Controlled conditions for production
Records (shall be retained)		
Records (shall be retained)	7.1.5.1	Calibration/verification results
Records (shall be retained)	7.2	Training records
Records (shall be retained)	8.2.3.2	Results of review of requirements
Records (shall be retained)	8.3.3	Design and development inputs
Records (shall be retained)	8.3.4	Design and development controls
Records (shall be retained)	8.3.5	Design and development outputs
Records (shall be retained)	8.3.6	Design and development changes
Records (shall be retained)	8.5.2	Traceability requirements
Records (shall be retained)	8.6	Release of products/services
Records (shall be retained)	9.1.1	Results of monitoring and measurement

17.6 Internal Auditing

Internal audits are systematic, independent evaluations of the QMS to determine if it conforms to requirements and is effectively implemented.

17.6.1 The Audit Process



17.6.2 Audit Checklist Example

Table 17.10: Sample Audit Checklist: Clause 7.1.5 Monitoring and Resources

Clause	Requirement	Question
7.1.5	Monitoring and measuring resources suitable for the type of activities	How do you determine what me
7.1.5	Resources maintained to ensure continued fitness for purpose	What maintenance is performed
7.1.5	Documented information on fitness for purpose retained	Can you show me calibration re
7.1.5	Measuring equipment identified to determine status	How can I tell if this gauge is c
7.1.5.2	Measurement traceability when required	What standards are these gauge

17.6.3 Types of Audit Findings

Table

Finding Type	Definition
Major Nonconformity	Absence or total breakdown of a system; significant risk to product/customer
Minor Nonconformity	Single lapse or isolated incident; system exists but not fully effective
Observation	Weakness that could lead to nonconformity if not addressed
Opportunity for Improvement	Suggestion for enhancement; not a nonconformity

17.6.4 Auditor Competence

Table 17.12: Internal Auditor Competency Requirements

Competency	Description	How to Develop
Knowledge of Standards	Understanding of ISO 9001 and applicable industry standards	Training, self-study
Audit Techniques	Planning, interviewing, evidence collection, reporting	Audit experience, training
Industry Knowledge	Understanding of processes, products, and industry practices	Work experience, training
Communication Skills	Active listening, clear questioning, professional writing	Practice, training
Objectivity	Independence from area being audited; impartial	Audit experience, training
Professional Judgment	Ability to evaluate significance of findings	Experience, training

17.7 Corrective and Preventive Action (CAPA)

Corrective action eliminates the cause of a detected nonconformity to prevent recurrence. **Preventive action** eliminates the cause of a potential nonconformity to prevent occurrence.

17.7.1 The CAPA Process

CAPA Process Flow

Eight-step process for effective corrective and preventive action



17.7.1.1 Step 1: Identification

The CAPA process begins when a nonconformity or potential problem is identified. Sources include:

Table 17.13: Common CAPA Initiation Sources

Source	Description	Priority
Customer Complaints	Product returns, warranty claims, customer feedback	High
Internal Audits	Nonconformities found during scheduled internal audits	Medium
Process Monitoring	SPC out-of-control conditions, trend analysis	Medium
Inspection/Testing	Failed inspections, test failures, rejected lots	High
Supplier Issues	Incoming inspection failures, supplier audit findings	Medium
Employee Reports	Near-misses, safety concerns, improvement suggestions	Low-Medium
Management Review	Issues identified during management review meetings	Medium
External Audits	Findings from certification body or customer audits	High

17.7.1.2 Step 2: Evaluation

Evaluate the significance of the issue to determine the appropriate response level:

Evaluation Criteria:

Factor	Questions to Consider
Safety	Could this harm people or the environment?
Customer Impact	How many customers affected? Severity of impact?
Regulatory	Does this violate regulations or standards?
Financial	What is the cost of the problem? Potential liability?
Frequency	Is this a one-time event or recurring pattern?
Detection	How was it found? Could others exist undetected?

Containment Actions - Immediate actions to limit the impact while the investigation proceeds:

- Quarantine suspect product
- Sort and inspect inventory
- Notify affected customers
- Implement temporary controls
- Stop shipments if necessary

Example: Evaluation of Dimensional Nonconformity

A batch of machined shafts is found with diameter 0.02 mm oversize:

- **Safety:** Low - not a safety-critical dimension
- **Customer Impact:** Medium - may cause assembly issues
- **Regulatory:** None
- **Financial:** 500 parts $\times$ \$15/part = \$7,500 potential scrap

- **Containment:** Quarantine batch, 100% inspection of last 3 lots, notify customer

Decision: Proceed with formal CAPA (cost exceeds \$5,000 threshold)

17.7.1.3 Step 3: Investigation

Gather all relevant data and evidence to understand what happened:

Investigation Activities:

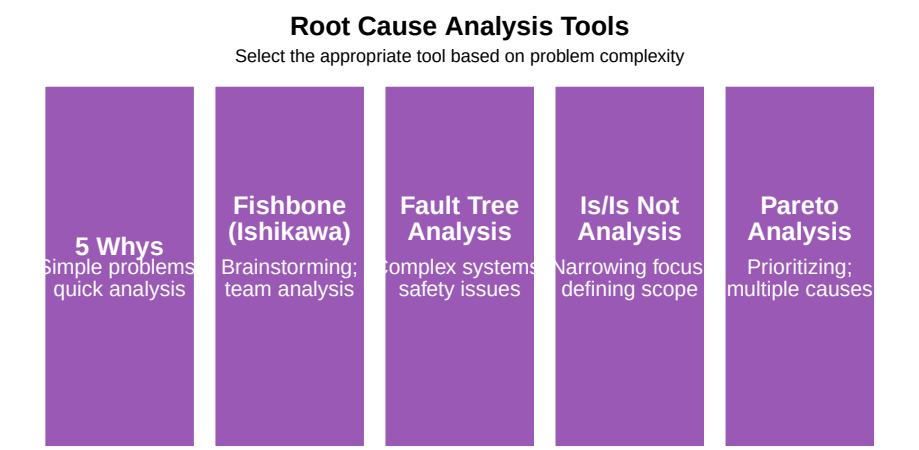
- **Review documentation:** Work instructions, control plans, inspection records
- **Examine physical evidence:** Failed parts, raw materials, tooling
- **Interview personnel:** Operators, inspectors, supervisors
- **Analyze data:** SPC charts, measurement data, process parameters
- **Timeline reconstruction:** When did the problem start? What changed?

Table 17.14: Data Collection for CAPA Investigation

Data Type	Examples	Collection Method
Process Data	Temperatures, pressures, speeds, feeds, cycle times	Data historians, SPC
Material Data	Lot numbers, certificates, incoming inspection results	Traceability system,
Equipment Data	Maintenance records, calibration status, tool wear	CMMS, calibration
Personnel Data	Training records, shift assignments, experience level	HR system, training
Environmental Data	Temperature, humidity, contamination levels	Environmental moni

17.7.1.4 Step 4: Root Cause Analysis

Determine the fundamental cause(s) of the problem using structured analysis tools:



The 5 Whys Example:

Level	Question	Answer
Problem	Shaft diameter is oversize	-
Why 1?	Why is the diameter oversize?	Tool is worn
Why 2?	Why is the tool worn?	Tool wasn't changed at scheduled interval
Why 3?	Why wasn't it changed?	Operator didn't know it was due
Why 4?	Why didn't operator know?	Tool life counter was reset incorrectly
Why 5?	Why was it reset incorrectly?	No verification step in setup procedure
Root Cause	Setup procedure lacks tool counter verification step	

The 6M Categories (Fishbone Diagram):

- **Man** (People): Training, skills, fatigue, attention
- **Machine**: Equipment condition, capability, maintenance
- **Material**: Raw material quality, specifications, storage
- **Method**: Procedures, work instructions, standards
- **Measurement**: Gages, calibration, measurement technique
- **Mother Nature** (Environment): Temperature, humidity, contamination

17.7.1.5 Step 5: Action Plan

Define specific corrective and/or preventive actions to address the root cause:

Table 17.15: Types of CAPA Actions

Action_Type	Definition	Example
Correction	Fix the immediate problem (symptom treatment)	Rework or scrap the nonconforming part
Corrective Action	Eliminate the root cause to prevent recurrence	Revise setup procedure to include tool counter verification
Preventive Action	Eliminate potential causes before problems occur	Add tool counter verification to ALL new setups

SMART Action Planning:

- **Specific**: Clearly defined action (not vague)
- **Measurable**: Can verify completion and effectiveness
- **Assignable**: Single owner responsible
- **Realistic**: Achievable with available resources

- **Time-bound:** Due date specified

Example: SMART Corrective Action

Poor: “Improve the setup procedure”

SMART: “Quality Engineer (J. Smith) will revise Work Instruction WI-2345 to add Step 4.7: ‘Verify tool life counter matches Tool Setup Sheet before starting production. Record counter reading on Setup Checklist Form F-102.’ Revision to be completed by April 15, training completed by April 22.”

17.7.1.6 Step 6: Implementation

Execute the action plan with clear ownership and tracking:

Implementation Requirements:

- Assign responsible person for each action
- Provide necessary resources (time, budget, authority)
- Communicate changes to affected personnel
- Update documentation (procedures, work instructions, forms)
- Train personnel on new requirements
- Implement changes in a controlled manner

Change Control Considerations:

Change Type	Approval Required	Documentation
Document revision	Document owner, Quality	Revision history, training records
Process change	Process owner, Engineering, Quality	Change request, validation if needed
Equipment change	Maintenance, Engineering, Quality	Equipment qualification, capability study
Product change	Engineering, Quality, Customer (if required)	ECN, customer approval, PPAP if applicable

17.7.1.7 Step 7: Verification

Verify that actions were completed AND effective:

Completion Verification: - Was the action actually implemented? - Is documentation updated and approved? - Are personnel trained? - Is the change being followed?

Effectiveness Verification: - Has the problem stopped recurring? - Do metrics show improvement? - Are there any unintended consequences?

Table 17.16: CAPA Effectiveness Verification Methods

Method	Description	When_to_U
Monitoring Period	Track for recurrence over defined period (e.g., 90 days, 3 production runs)	All CAPAs -
Statistical Analysis	Compare defect rates before/after; statistical significance testing	When baseli
Audit Verification	Include in next internal audit; verify implementation	Process/doc
Customer Feedback	Monitor complaints related to this issue	Customer-re
Process Capability	Cpk study to confirm process improvement	Manufacturi

17.7.1.8 Step 8: Closure

Complete documentation and share lessons learned:

Closure Requirements:

- All actions verified complete
- Effectiveness confirmed (no recurrence)
- Documentation updated and approved
- CAPA record completed with all evidence
- Management approval for closure

Lessons Learned:

- What was the root cause?
- Could this happen elsewhere? (Horizontal deployment)
- Should procedures/training be updated system-wide?
- Are there similar products/processes at risk?
- What can we do to prevent similar issues?

Example: Horizontal Deployment

The shaft diameter CAPA revealed that setup procedures lacked tool counter verification.

Horizontal deployment: Quality reviewed ALL 47 CNC machine setup procedures and found 12 others with the same gap. All procedures were revised, and the finding was shared at the monthly quality meeting as a lesson learned.

17.7.2 CAPA Effectiveness Criteria

This worked example is fixed as a reference — read through it once, then try the *editable* example further on in this chapter for yourself.

```
# CAPA Effectiveness Tracking
set.seed(42)

# Simulate CAPA data for a year
months <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
            "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
```

```

capa_data <- data.frame(
  Month = factor(months, levels = months),
  Total_CAPAs = c(12, 15, 18, 14, 11, 13, 10, 9, 8, 7, 9, 6),
  Closed_On_Time = c(8, 10, 12, 11, 9, 11, 9, 8, 7, 6, 8, 5),
  Effective = c(7, 9, 10, 10, 8, 10, 8, 8, 7, 6, 8, 5),
  Recurrence = c(2, 1, 3, 1, 0, 1, 0, 0, 1, 0, 0, 0)
)

# Calculate metrics
capa_data$On_Time_Rate <- round(capa_data$Closed_On_Time / capa_data$Total_CAPAs * 100
capa_data$Effectiveness_Rate <- round(capa_data$Effective / capa_data$Closed_On_Time *

cat("CAPA Performance Metrics Summary:\n")
cat("\n")
cat("Total CAPAs initiated:", sum(capa_data$Total_CAPAs), "\n")
cat("Closed on time:", sum(capa_data$Closed_On_Time), "(",
  round(sum(capa_data$Closed_On_Time)/sum(capa_data$Total_CAPAs)*100, 1), "%)\n")
cat("Verified effective:", sum(capa_data$Effective), "(",
  round(sum(capa_data$Effective)/sum(capa_data$Closed_On_Time)*100, 1), "%)\n")
cat("Recurrences:", sum(capa_data$Recurrence), "\n")

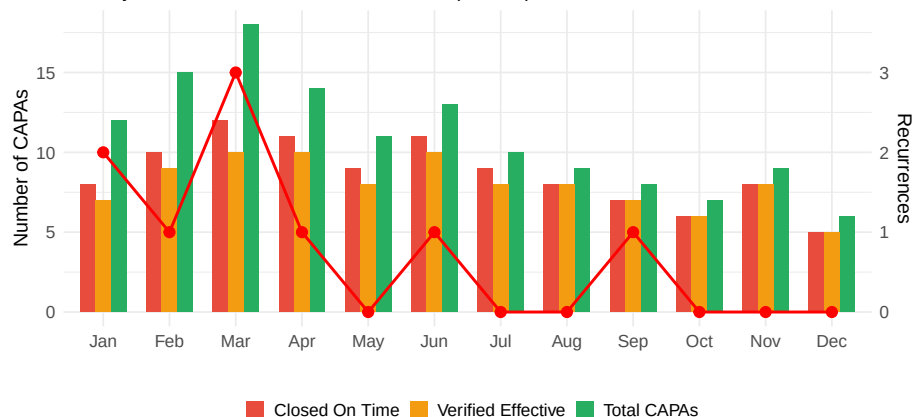
```

CAPA Performance Metrics Summary:

Total CAPAs initiated: 132
 Closed on time: 104 (78.8 %)
 Verified effective: 96 (92.3 %)
 Recurrences: 9

CAPA Performance Tracking

Monthly CAPA metrics with recurrence trend (red line)



17.8 Management Review

Management review is a formal evaluation of the QMS by top management to ensure its continuing suitability, adequacy, and effectiveness.

17.8.1 Management Review Inputs

Table 17.17: Management Review

Required Input	Details to Review
Previous Review Actions	Status of actions from previous management reviews
Changes in Issues/Context	Changes in external/internal issues; interested party requirements
Quality Performance	Customer satisfaction; quality objectives achievement; process performance; nonconformities
Resource Adequacy	Adequacy of resources for QMS maintenance and improvement
Risk/Opportunity Actions	Effectiveness of actions taken to address risks and opportunities
Improvement Opportunities	Opportunities for improvement identified from various sources

17.8.2 Management Review Outputs

Table 17.18: Management Review Outputs (ISO 9001:2015)

Required Output	ISO Requirement	Examples
Improvement Decisions	Decisions and actions related to improvement opportunities	Launch kaizen project
QMS Changes	Any need for changes to the QMS	Revise control plan based on new requirements
Resource Needs	Any resource needs	Hire additional quality engineers

17.8.3 Sample Management Review Dashboard



17.9 Continuous Improvement

Continuous improvement is a fundamental principle of quality management - the ongoing effort to improve products, services, and processes.

17.9.1 Improvement Methodologies

Table 17.19: Continuous Improvement

Methodology	Origin	Best_For	Step
PDCA	Deming/Shewhart	General improvement cycle; process changes	Plan
DMAIC	Six Sigma	Data-driven problem solving; reducing variation	Defin
Kaizen	Toyota/Lean	Small, incremental improvements; team-based	Ident
A3 Problem Solving	Toyota	Visual problem solving; root cause analysis	Back
8D	Ford	Customer complaints; complex problems	D0-I

17.9.2 Key Performance Indicators (KPIs)

Table 17.20: Common Quality Management KPIs

Category	KPI	Formula	Ta
Quality Metrics			
Quality	First Pass Yield (FPY)	$(\text{Good units first time} / \text{Total units}) \times 100\%$	98
Quality	Customer Complaints (PPM)	$(\text{Complaints} / \text{Units shipped}) \times 1,000,000$	<1
Quality	Internal Reject Rate	$(\text{Rejected units} / \text{Total inspected}) \times 100\%$	<1
Quality	Cost of Quality	Prevention + Appraisal + Internal Failure + External Failure	<3
Delivery Metrics			
Delivery	On-Time Delivery (OTD)	$(\text{Orders delivered on time} / \text{Total orders}) \times 100\%$	98
Delivery	Lead Time	Order date to delivery date	Inc
Delivery	Order Fulfillment Rate	$(\text{Complete orders shipped} / \text{Total orders}) \times 100\%$	99
Cost Metrics			
Cost	Scrap Rate	$(\text{Scrap value} / \text{Total production value}) \times 100\%$	<2
Cost	Rework Cost	Labor + Material for rework	Mi
Cost	Warranty Cost	Warranty claims cost / Revenue	<0
Safety Metrics			
Safety	Recordable Incident Rate	$(\text{Recordable incidents} \times 200,000) / \text{Hours worked}$	<1
Safety	Near Miss Reports	Count of near miss reports	En

17.9.3 Cost of Quality (COQ)

```
# Cost of Quality Analysis
coq_data <- data.frame(
  Category = c("Prevention", "Appraisal", "Internal Failure", "External Failure"),
  Description = c("Quality planning, training, process control",
                  "Inspection, testing, audits, calibration",
                  "Scrap, rework, reinspection, downtime",
                  "Warranty, returns, complaints, recalls"),
  Cost = c(50000, 75000, 120000, 180000)
)

total_coq <- sum(coq_data$Cost)
revenue <- 10000000 # $10M revenue

coq_data$Percent_of_COQ <- round(coq_data$Cost / total_coq * 100, 1)
coq_data$Percent_of_Revenue <- round(coq_data$Cost / revenue * 100, 2)

cat("Cost of Quality Analysis:\n")
cat("      \n")
print(coq_data[, c("Category", "Cost", "Percent_of_COQ")])
cat("      \n")
```

```

cat("Total Cost of Quality: $", format(total_coq, big.mark = " "), "\n")
cat("COQ as % of Revenue:", round(total_coq/revenue*100, 1), "%\n\n")

# Analysis
conformance <- coq_data$Cost[1] + coq_data$Cost[2]
nonconformance <- coq_data$Cost[3] + coq_data$Cost[4]

cat("Conformance Costs (Prevention + Appraisal): $", format(conformance, big.mark = " "),
    "(", round(conformance/total_coq*100, 1), "%)\n")
cat("Nonconformance Costs (Failures): $", format(nonconformance, big.mark = " "),
    "(", round(nonconformance/total_coq*100, 1), "%)\n")
cat("\nRecommendation: Increase prevention spending to reduce failure costs\n")

```

Cost of Quality Analysis:

	Category	Cost	Percent_of_COQ
1	Prevention	50000	11.8
2	Appraisal	75000	17.6
3	Internal Failure	120000	28.2
4	External Failure	180000	42.4

Total Cost of Quality: \$ 425 000

COQ as % of Revenue: 4.2 %

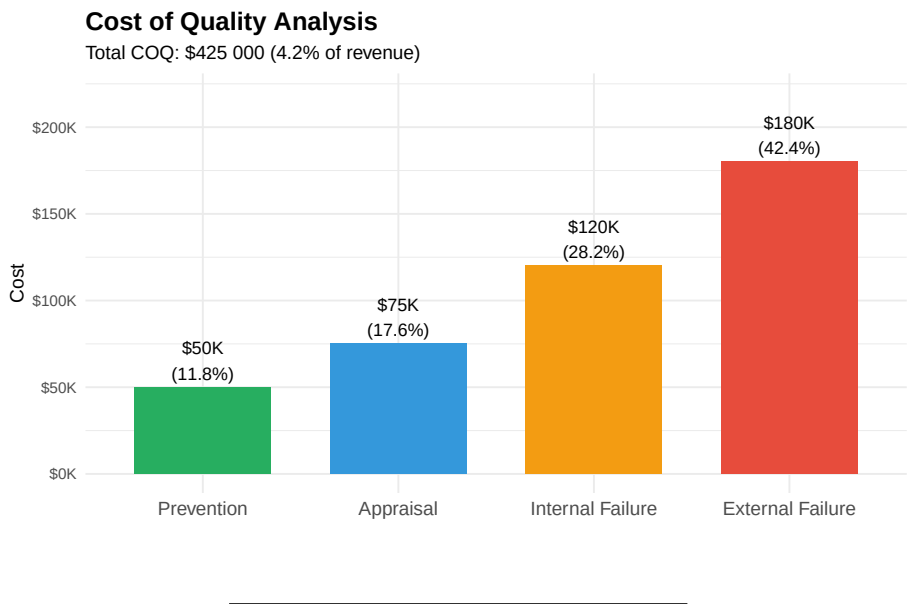
Conformance Costs (Prevention + Appraisal): \$ 125 000 (29.4 %)

Nonconformance Costs (Failures): \$ 3e+05 (70.6 %)

Recommendation: Increase prevention spending to reduce failure costs

Warning: Removed 1 row containing missing values or values outside the scale range
(`geom\_text()`).

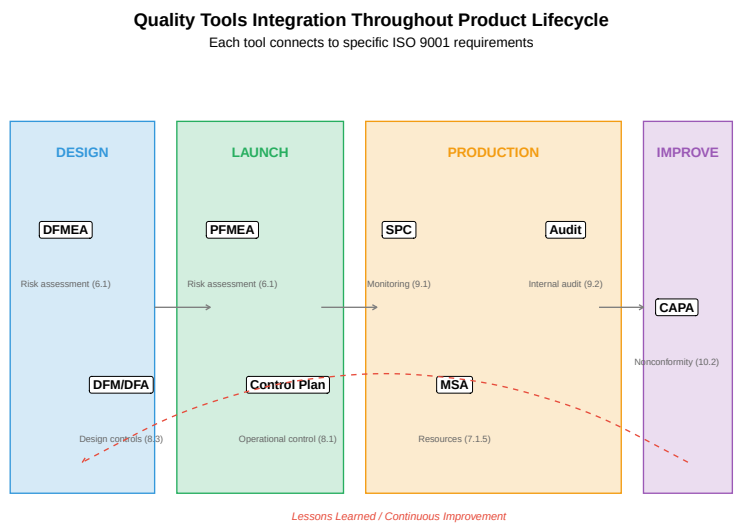
Removed 1 row containing missing values or values outside the scale range
(`geom\_text()`).



17.10 Integration with Quality Tools

A mature QMS integrates various quality tools throughout the organization.

17.10.1 Quality Tools Integration Map



17.10.2 Linking Tools to QMS Requirements

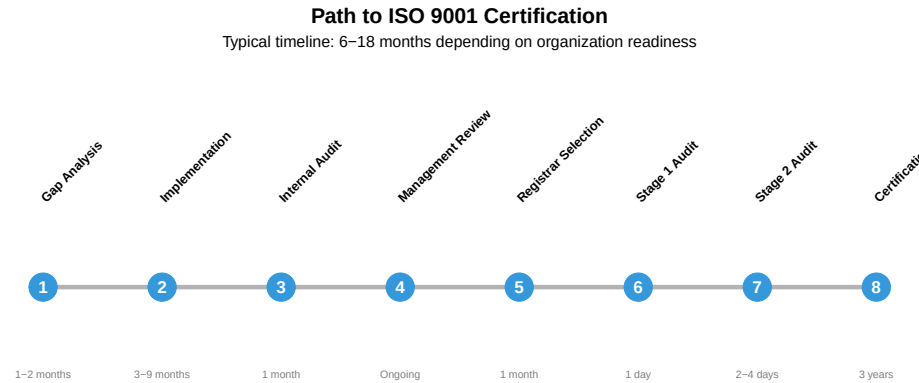
Table 17.21: Quality Tools and QMS Integration

Quality Tool	ISO 9001 Clause	Purpose
PFMEA	6.1, 8.1	Identify and mitigate process risks before production
Control Plan	8.1, 8.5.1	Define controls for special characteristics
SPC	9.1.1	Monitor process performance; detect special causes
MSA	7.1.5	Ensure measurement system is adequate
Internal Audit	9.2	Verify QMS conformance and effectiveness
CAPA	10.2	Eliminate causes of nonconformities
Management Review	9.3	Evaluate QMS suitability and effectiveness

17.11 Certification Process

Organizations can seek **third-party certification** to demonstrate QMS conformance to customers and stakeholders.

17.11.1 The Certification Journey



17.11.2 Certification Audit Types

Table 17.22: Third-Party Certification

Audit Type	Purpose	Focus Areas
------------	---------	-------------

Stage 1 (Documentation)	Review QMS documentation; verify readiness for Stage 2	Quality manual, procedures
Stage 2 (Certification)	Evaluate implementation and effectiveness of QMS	All clauses; process effectiveness
Surveillance	Verify continued conformance (annual)	Selected processes; preventive actions
Recertification	Full re-audit at end of 3-year cycle	Complete review similar to Stage 2

17.12 Video Resources

17.12.1 ISO 9001:2015 Overview

17.12.2 Internal Auditing Best Practices

17.13 Summary

A Quality Management System provides the framework for consistently meeting customer requirements and improving organizational performance:

1. **ISO 9001:2015** provides the foundational requirements using risk-based thinking and PDCA
2. **Industry standards** (IATF 16949, AS9100, ISO 22000) add sector-specific requirements
3. **Documentation** establishes the hierarchy from policy to procedures to records
4. **Internal audits** verify conformance and identify improvement opportunities
5. **CAPA** eliminates root causes of nonconformities and prevents recurrence
6. **Management review** ensures QMS suitability and drives strategic improvement
7. **Continuous improvement** is embedded through KPIs, COQ analysis, and improvement methodologies
8. **Quality tools** (PFMEA, SPC, MSA) integrate throughout the product lifecycle

“Quality is not an act, it is a habit.” — Aristotle

17.14 Review Questions

Question 1: Explain the seven quality management principles and how they relate to each other.

Answer:

The seven quality management principles form an interconnected foundation:

1. **Customer Focus** (Central principle)
 - Understanding and meeting customer needs is the primary purpose
 - All other principles support this goal
2. **Leadership**
 - Leaders establish unity of purpose and direction
 - Create conditions for people to achieve quality objectives
 - Enables all other principles
3. **Engagement of People**
 - Competent, empowered people at all levels
 - Essential for effective processes and customer focus
4. **Process Approach**
 - Activities managed as interrelated processes
 - Enables consistent, predictable results
5. **Improvement**
 - Ongoing focus on enhancing performance
 - Essential for maintaining customer satisfaction
6. **Evidence-based Decision Making**
 - Decisions based on analysis of data
 - Reduces uncertainty; improves decision quality
7. **Relationship Management**
 - Managing relationships with interested parties (suppliers, partners)
 - Optimizes their impact on performance

Relationships: - Customer Focus drives the need for all others - Leadership enables and supports all principles - Engagement of People is essential for Process Approach and Improvement - Evidence-based Decision Making supports Improvement - Relationship Management extends quality beyond the organization

Question 2: Compare ISO 9001, IATF 16949, and AS9100. When would each be required?

Answer:

Standard	Base	Industry	Additional Focus
ISO 9001:2015	Standalone	General	Quality fundamentals
IATF 16949:2016	ISO 9001 + auto-motive	Automotive	PPAP, APQP, Core Tools
AS9100D	ISO 9001 + aerospace	Aerospace/Defense	Configuration mgmt, FAI, counterfeit prevention

When Required:

ISO 9001: - Any organization wanting to demonstrate quality capability - Customer or contract requirement for QMS certification - Basis for all other standards - Suitable for service, manufacturing, any industry

IATF 16949: - Suppliers to automotive OEMs (Ford, GM, Toyota, VW, etc.) - Required for production parts/materials - Required throughout automotive supply chain - Includes Customer-Specific Requirements (CSRs)

AS9100: - Suppliers to aerospace/defense contractors - Boeing, Airbus, Lockheed Martin, military programs - Required for flight-critical components - OASIS database registration required

Key Differences: - IATF 16949: Emphasizes PPAP, APQP, manufacturing process control - AS9100: Emphasizes traceability, configuration control, first article inspection - Both build on ISO 9001 with sector-specific additions

Question 3: Describe the documentation hierarchy in a QMS. Give an example of each level.

Answer:

Level 1: Quality Policy and Objectives - Strategic direction for quality - Signed by top management - Communicated to all employees - *Example:* “ABC Company is committed to delivering defect-free products that meet customer requirements through continuous improvement of our processes and people.”

Level 2: Quality Manual and Procedures - Describes WHAT is done and WHO is responsible - System-level documents - *Example:* Document Control Procedure (QP-001) - Purpose: Control creation, approval, distribution of documents - Scope: All QMS documents - Responsibilities: Document Control Coordinator, Department Managers - Process flow for document changes

Level 3: Work Instructions and Specifications - Describes HOW tasks are performed - Detailed step-by-step instructions - *Example:* Work Instruction for CNC Lathe Setup (WI-MAC-015) - Step 1: Verify program number matches work order - Step 2: Install correct tooling per setup sheet - Step 3: Set tool offsets using touch-off procedure - Step 4: Run first piece and verify dimensions

Level 4: Records and Forms - Evidence that activities were performed - Completed forms, inspection results, logs - *Example:* First Article Inspection Report (FM-QC-008) - Part number, revision, date - Dimensional measurements vs. specifications - Inspector signature - Disposition (Accept/Reject)

Question 4: What are the key elements of an effective internal audit program?

Answer:

1. Audit Program Planning - Annual audit schedule covering all QMS processes - Risk-based frequency (higher risk = more frequent) - Documented audit program procedure - Resource allocation (trained auditors, time)

2. Auditor Competence - Trained in audit techniques - Knowledge of standards (ISO 9001, industry-specific) - Understanding of processes being audited - Independence from area being audited

3. Audit Preparation - Review of relevant documents (procedures, previous audits) - Prepared checklist linked to requirements - Notification to auditee - Defined scope and objectives

4. Audit Execution - Opening meeting (purpose, scope, schedule) - Evidence collection (documents, records, interviews, observation) - Objective evaluation against criteria - Note taking and evidence documentation

5. Reporting - Clear finding statements (nonconformities, observations) - Reference to requirement violated - Objective evidence cited - Closing meeting to present findings

6. Follow-up - Corrective action requests issued - Root cause analysis required - Verification of corrective action implementation - Verification of effectiveness

7. Continuous Improvement of Audit Program - Evaluate audit program effectiveness - Update based on organizational changes - Calibrate auditors periodically - Incorporate lessons learned

Question 5: Explain the difference between corrective action and preventive action. Provide an example of each.

Answer:

Corrective Action: - Eliminates the cause of a *detected* nonconformity - Reactive - responds to something that has already happened - Goal: Prevent recurrence of the specific problem

Example: - **Nonconformity:** Customer received shipment with wrong part numbers (3 incidents in past month) - **Root Cause:** Shipping labels printed from outdated part number list - **Corrective Action:** 1. Update master part number list 2. Link shipping system directly to ERP for current data 3. Add verification step comparing label to packing list - **Verification:** No recurrence after 3 months

Preventive Action: - Eliminates the cause of a *potential* nonconformity - Proactive - addresses something that hasn't happened yet - Goal: Prevent occurrence before it happens

Example: - **Potential Nonconformity:** New CNC machine may produce out-of-spec parts due to operator unfamiliarity - **Risk Identified Through:** PFMEA analysis during new equipment installation - **Preventive Action:** 1. Develop comprehensive training program before machine goes live 2. Create detailed setup procedures with photos 3. Implement first-article inspection for first 30 days 4. Assign experienced mentor to new operators - **Verification:** Track first-pass yield during initial production period

Key Difference: - Corrective = Fix what went wrong - Preventive = Stop what might go wrong

Note: ISO 9001:2015 doesn't explicitly require "preventive action" as a separate process. Instead, risk-based thinking throughout the QMS serves the preventive function.

Question 6: Calculate and analyze the Cost of Quality for the following data. What recommendations would you make?

Category	Cost
Quality planning	\$35,000
Training	\$25,000
Incoming inspection	\$40,000
In-process inspection	\$55,000
Final inspection	\$30,000
Scrap	\$85,000
Rework	\$65,000
Warranty claims	\$120,000
Customer returns	\$45,000

Annual revenue: \$8,000,000

Answer: (live — change the cost figures to check your own version)

```
#| label: q6-answer
# Cost of Quality Analysis
prevention <- 35000 + 25000 # Planning + Training
appraisal <- 40000 + 55000 + 30000 # Inspections
internal_failure <- 85000 + 65000 # Scrap + Rework
external_failure <- 120000 + 45000 # Warranty + Returns

total_coq <- prevention + appraisal + internal_failure + external_failure
revenue <- 8000000

cat("Cost of Quality Breakdown:\n")
cat("      \n")
cat("Prevention:      $", format(prevention, big.mark = " "),
    "(", round(prevention/total_coq*100, 1), "%)\n")
cat("Appraisal:      $", format(appraisal, big.mark = " "),
    "(", round(appraisal/total_coq*100, 1), "%)\n")
cat("Internal Failure: $", format(internal_failure, big.mark = " "),
    "(", round(internal_failure/total_coq*100, 1), "%)\n")
cat("External Failure: $", format(external_failure, big.mark = " "),
    "(", round(external_failure/total_coq*100, 1), "%)\n")
cat("      \n")
```

```

cat("Total COQ:      $", format(total_coq, big.mark = " "), "\n")
cat("COQ % of Revenue:", round(total_coq/revenue*100, 1), "%\n\n")

conformance <- prevention + appraisal
nonconformance <- internal_failure + external_failure

cat("Conformance Costs:  $", format(conformance, big.mark = " "),
    "(", round(conformance/total_coq*100, 1), "%)\n")
cat("Nonconformance Costs: $", format(nonconformance, big.mark = " "),
    "(", round(nonconformance/total_coq*100, 1), "%)\n")

```

Analysis: - COQ at 7.5% of revenue is HIGH (target <3%) - External failure costs are the largest category (27.5%) - Nonconformance costs (60.8%) greatly exceed conformance (39.2%) - Prevention is only 10% of COQ - very low

Recommendations: 1. **Increase Prevention Investment:** - Implement SPC on critical processes to reduce defects - Enhance operator training on quality standards - Apply PFMEA to identify and control failure modes

2. **Reduce External Failures:**

- Analyze warranty data for root causes
- Improve final inspection effectiveness
- Implement containment for known issues

3. **Optimize Appraisal:**

- Use SPC to reduce inspection where processes are capable
- Implement risk-based inspection (more for critical, less for proven)

4. **Target COQ Reduction:**

- Short-term: Reduce external failures by 50% (\$82,500 savings)
- Medium-term: Reduce internal failures by 30% (\$45,000 savings)
- Long-term: Achieve COQ <3% of revenue (\$240,000)

Question 7: What is risk-based thinking in ISO 9001:2015? How is it different from formal risk management?

Answer:

Risk-Based Thinking in ISO 9001:2015:

Risk-based thinking is a way of considering risks and opportunities when designing, implementing, and operating the QMS. It:

- Applies throughout the standard (not a separate clause)
- Is proportional to potential impact on conformity
- Makes preventive action part of routine planning
- Considers both risks (negative) and opportunities (positive)

Where Risk-Based Thinking Applies:

Clause	Application
4.1 Context	Identify external/internal issues affecting QMS
4.2 Interested Parties	Understand requirements that could impact quality
6.1 Planning	Plan actions to address risks and opportunities
8.1 Operations	Consider risks in operational planning
9.1 Performance	Monitor effectiveness of risk actions
10.2 Corrective Action	Update risks/opportunities based on nonconformities

How It Differs from Formal Risk Management:

Aspect	Risk-Based Thinking	Formal Risk Management
Requirement	Implicit throughout ISO 9001	Explicit (ISO 31000)
Documentation	No formal risk register required	Documented risk register
Methodology	No specific method required	Defined methodology (FMEA, etc.)
Quantification	Not required	Often quantitative
Scope	QMS processes	Enterprise-wide

Practical Implementation:

Organizations should: 1. Consider risks when designing processes 2. Build controls proportional to risk level 3. Monitor and review effectiveness 4. Update as context changes

ISO 9001 does NOT require: - Formal risk management process - Risk register - Specific risk assessment methodology - Documented risk assessments (unless organization decides to)

However, many organizations do use tools like PFMEA to fulfill risk-based thinking requirements, especially in automotive (IATF 16949 requires it).

Question 8: Describe what management review is and what should be included in the inputs and outputs.

Answer:

What is Management Review?

Management review is a formal, periodic evaluation of the QMS by top management to: - Ensure continuing suitability (appropriate for the organization) - Ensure adequacy (sufficient to meet requirements) - Ensure effectiveness (achieving

intended results) - Ensure alignment with strategic direction - Identify improvement opportunities

Frequency: At least annually; many organizations conduct quarterly reviews

Required Inputs (ISO 9001 Clause 9.3.2):

Input	What to Review
Status of previous actions	Actions from prior reviews - completed? effective?
Changes in external/internal issues	Market changes, regulations, technology, organizational changes
QMS performance and effectiveness	Customer satisfaction, quality objectives, process performance, nonconformities, audit results, supplier performance
Resource adequacy	People, infrastructure, environment, knowledge
Effectiveness of risk actions	Did actions address risks/opportunities?
Improvement opportunities	Suggestions, benchmarking, new technologies

Required Outputs (ISO 9001 Clause 9.3.3):

Output	Examples
Improvement decisions	Launch specific improvement projects, address recurring issues
QMS changes	Revise quality objectives, update processes, modify documentation
Resource needs	Budget approvals, hiring, equipment purchases, training

Best Practices: - Use a standard agenda aligned with required inputs - Prepare data in advance (dashboards, trend charts) - Document decisions and action items with owners and due dates - Follow up on action completion - Keep minutes as quality records

Evidence Required: - Meeting minutes or management review report - Attendance record (top management participation) - Action items with status tracking - Data/reports presented

17.15 References

1. ISO 9001:2015. *Quality Management Systems - Requirements*. International Organization for Standardization.
2. ISO 9000:2015. *Quality Management Systems - Fundamentals and Vocabulary*. International Organization for Standardization.
3. IATF 16949:2016. *Quality Management System Requirements for Automotive Production and Relevant Service Parts Organizations*. International Automotive Task Force.
4. AS9100D:2016. *Quality Management Systems - Requirements for Aviation, Space and Defense Organizations*. SAE International.
5. ISO 22000:2018. *Food Safety Management Systems - Requirements for Any Organization in the Food Chain*. International Organization for Standardization.
6. ISO 19011:2018. *Guidelines for Auditing Management Systems*. International Organization for Standardization.
7. Hoyle, D. (2017). *ISO 9000 Quality Systems Handbook* (7th ed.). Routledge.
8. AIAG. (2010). *Advanced Product Quality Planning and Control Plan (APQP)* (2nd ed.). Automotive Industry Action Group.
9. ASQ. (2019). *The Certified Quality Auditor Handbook* (4th ed.). ASQ Quality Press.
10. Juran, J.M., & De Feo, J.A. (2010). *Juran's Quality Handbook* (6th ed.). McGraw-Hill.

Chapter 18

Process Failure Mode and Effects Analysis (PFMEA)

18.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain the purpose and benefits of PFMEA in manufacturing
- Distinguish between DFMEA (Design) and PFMEA (Process)
- Apply the 10-step PFMEA methodology to a manufacturing process
- Correctly rate Severity, Occurrence, and Detection using standardized scales
- Calculate Risk Priority Numbers (RPN) and prioritize actions
- Develop recommended actions to reduce process risks
- Create and maintain PFMEA documentation as a living document

“An ounce of prevention is worth a pound of cure.” — Benjamin Franklin

18.2 Introduction to PFMEA

18.2.1 What is PFMEA?

Process Failure Mode and Effects Analysis (PFMEA) is a systematic, proactive method for evaluating a manufacturing or business process to identify where and how it might fail, and to assess the relative impact of different failures.

The goal is to identify and prioritize actions that will prevent defects before they occur.

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

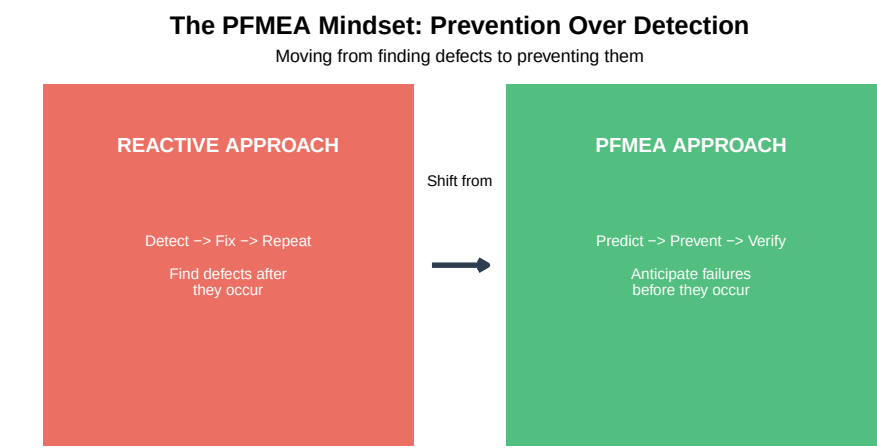


Figure 18.1: PFMEA: A Proactive Quality Tool

18.2.2 Brief History of FMEA

Table 18.1: Evolution of FMEA

Year	Development	Significance
1949	US Military develops FMEA (MIL-P-1629)	First formal FMEA method
1960s	NASA adopts FMEA for Apollo program	Proven reliability in life-critical systems
1970s	Automotive industry begins using FMEA	Ford, GM, Chrysler adopt FMEA
1980s	AIAG publishes first FMEA manual	Industry-wide consistency
1990s	QS-9000 requires FMEA for automotive suppliers	FMEA becomes mandatory
2000s	FMEA expands to healthcare, service industries	Methodology proves universal
2019	AIAG-VDA FMEA Handbook published (current standard)	Harmonized global standard

18.2.3 DFMEA vs. PFMEA

Two main types of FMEA are used in product development:

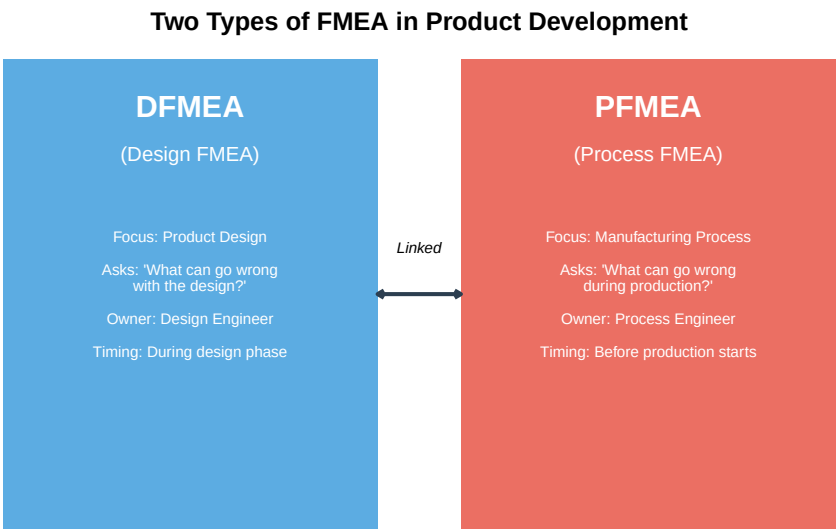


Figure 18.2: DFMEA vs PFMEA: Different Focus, Same Methodology

Table 18.2: DFMEA vs PFMEA Comparison

Aspect	DFMEA (Design)	PFMEA (Process)
Primary Question	Will the design meet requirements?	Will the process produce conforming parts?
Failure Mode Example	Shaft diameter too small	Shaft turned undersize
Effect Example	Premature bearing failure	Part fails incoming inspection
Cause Example	Inadequate stress analysis	Tool wear not monitored
Control Example	FEA simulation, prototype testing	In-process gauging, SPC
Responsible Team	Design engineering, R&D	Manufacturing, quality, production

Question: When should PFMEA be conducted?

Ideal Timing for PFMEA:

1. **New Process Development:** Before production begins, during process design
2. **New Product Introduction:** When a new product will use existing processes
3. **Process Changes:** Whenever significant changes are made to equipment, methods, or materials
4. **Quality Issues:** When field failures or customer complaints indicate process problems
5. **Regular Review:** Periodically (annually) to ensure continued relevance

Key Principle: The earlier PFMEA is conducted, the lower the cost of implementing improvements. Changes during design cost 10x less than changes during production, and 100x less than changes after customer delivery.

18.2.4 Why PFMEA Matters

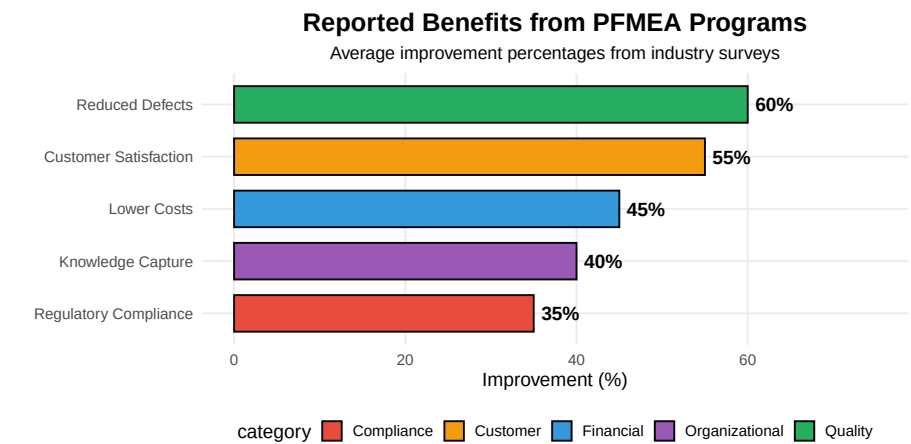


Figure 18.3: Benefits of PFMEA Implementation

18.3 PFMEA Fundamentals

18.3.1 Core Objectives

The fundamental objective of PFMEA is **prevention over detection** — identifying and eliminating potential failures before they occur, rather than finding defects after production.

Table 18.3: Core Objectives of PFMEA

Objective	Description	Outcom
Identify Failure Modes	Systematically identify all ways the process can fail	Comprel
Assess Risk	Evaluate severity, likelihood, and detectability of each failure	Quantifi
Prioritize Actions	Focus resources on highest-risk failure modes	Action p
Document Knowledge	Capture institutional knowledge in a structured format	Living d

Drive Improvement	Implement and verify effectiveness of countermeasures	Measurable reduction
----------------------------------	---	----------------------

18.3.2 The PFMEA Team

PFMEA is most effective when conducted by a **cross-functional team**. No single person has all the knowledge needed to identify every potential failure.

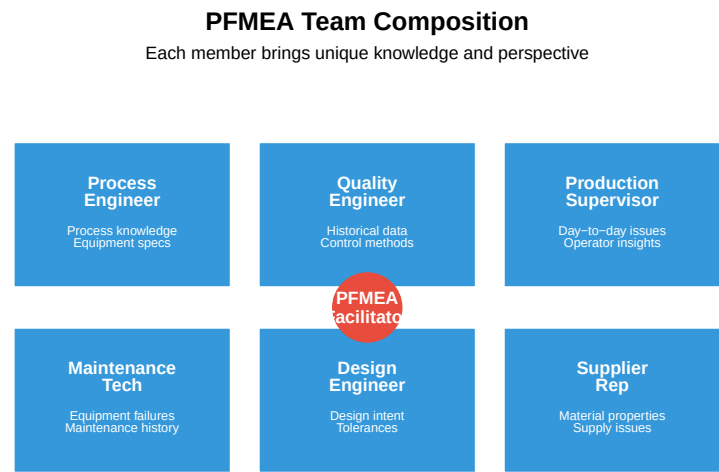


Figure 18.4: PFMEA Cross-Functional Team

Question: Why is a cross-functional team essential?

A cross-functional team is essential because:

1. **Diverse Knowledge:** No single person understands all aspects of the process
2. **Different Perspectives:** Quality sees defects, production sees constraints, maintenance sees failures
3. **Buy-in:** People support what they help create
4. **Historical Knowledge:** Operators often know about problems never formally documented
5. **System Thinking:** Process interactions are visible when multiple functions participate

Common Mistake: Having one engineer complete PFMEA alone as a “paperwork exercise.” This misses critical failure modes and produces an ineffective document.

Best Practice: Minimum 4-6 team members, including at least one person with hands-on process experience.

18.3.3 Key Terminology

Understanding PFMEA terminology is essential for effective analysis:

Table 18.4: PFMEA Key Terminology

Term	Definition
Process Function	What the process step is supposed to do
Failure Mode	How the process can fail to perform its function
Failure Effect	The consequence of the failure on the customer or next operation
Severity (S)	Rating (1-10) of how serious the failure effect is
Failure Cause	Why the failure mode might occur
Occurrence (O)	Rating (1-10) of how likely the cause is to happen
Current Controls	What currently prevents the cause or detects the failure
Detection (D)	Rating (1-10) of how likely current controls will detect the failure
RPN	Risk Priority Number = $S \times O \times D$
Action Priority (AP)	Alternative prioritization method (AIAG-VDA)

18.3.4 The PFMEA Form

The PFMEA is documented in a structured form. Here is a simplified template:

Table 18.5: PFMEA Form Template (Simplified)

Process Step	Function	Failure Mode	Effect	S	Cause	O	Current Controls
Example:	Drill hole	Hole undersize	Assembly interference	7	Drill wear	5	Visual inspection

18.4 The PFMEA Methodology: 10 Steps

PFMEA follows a systematic 10-step methodology. Each step builds on the previous one.

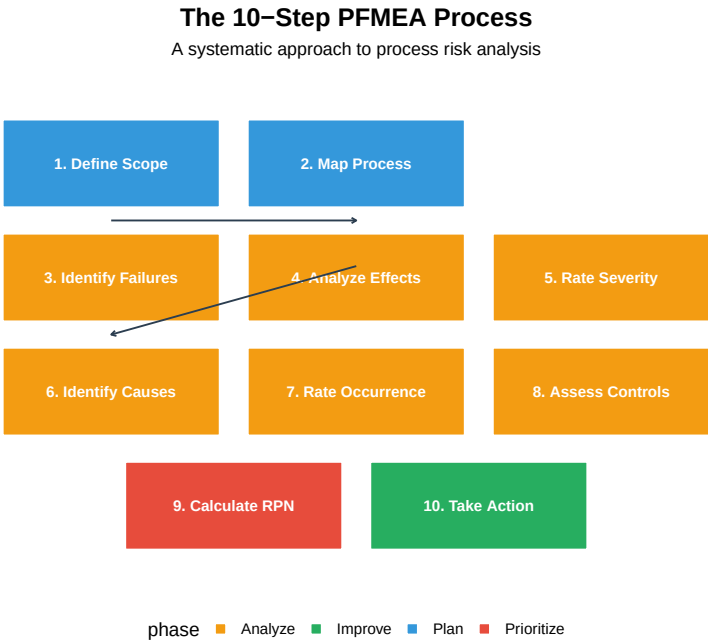


Figure 18.5: The 10-Step PFMEA Methodology

18.4.1 Step 1: Define Scope and Process Boundaries

Before starting the analysis, clearly define what is included and excluded.

Table 18.6: Step 1: Defining PFMEA Scope

Element	Description	Example: Injection Molding
Process Name	Clear name identifying the process	Plastic Part Injection Molding
Start Boundary	Where does our analysis begin?	Raw material loading into hopper
End Boundary	Where does our analysis end?	Part removed from mold, ready for packaging
Included Operations	All operations within scope	Material prep, injection, cooling, ejection
Excluded Operations	Operations analyzed separately or by others	Mold design (covered by DFMEA), material spec
Key Assumptions	Conditions assumed to be true	Material meets incoming spec, mold is maintained

18.4.2 Step 2: Process Mapping and Identifying Functions

Create a detailed process flow diagram and identify the function of each step.

Process Flow Diagram with Functions

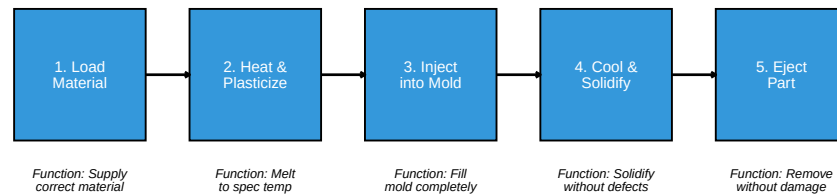


Figure 18.6: Example: Injection Molding Process Flow

18.4.3 Step 3: Identifying Potential Failure Modes

For each process function, ask: **“How can this step fail to perform its intended function?”**

Table 18.7: Step 3: Identifying Failure Modes

Process Function	Potential Failure Mode	Category
Load correct material	Wrong material loaded	Wrong item
Load correct material	Material contaminated	Degraded item
Load correct material	Insufficient material	Missing item
Melt to specification temperature	Temperature too low	Below specification
Melt to specification temperature	Temperature too high	Above specification
Fill mold completely	Short shot (incomplete fill)	Incomplete operation
Fill mold completely	Flash (overfill)	Excessive operation
Fill mold completely	Air entrapment	Unintended result

Common failure mode categories: - **No operation** (step doesn’t happen)
 - **Partial operation** (incomplete) - **Degraded operation** (below standard)
 - **Excessive operation** (too much) - **Wrong operation** (incorrect action) -
Intermittent operation (inconsistent)

Interactive Exercise: Identify Failure Modes

Process Step: Heat treat steel parts at 850°C for 2 hours

Function: Achieve required hardness (58-62 HRC)

Identify at least 5 potential failure modes:

Answers: 1. Temperature too low → Parts soft (below hardness spec) 2. Temperature too high → Parts brittle/cracked 3. Time too short → Incomplete

hardening 4. Time too long → Over-hardening, distortion 5. Uneven heating → Inconsistent hardness across part 6. Wrong atmosphere → Surface decarburization 7. Quench delay too long → Soft spots

18.4.4 Step 4: Analyzing Failure Effects

For each failure mode, determine: “**What is the consequence if this failure occurs?**”

Consider effects at multiple levels: 1. **Local effect:** Impact on immediate operation 2. **Next operation effect:** Impact on downstream process 3. **End customer effect:** Impact on final user

Table 18.8: Step 4: Analyzing Effects at Multiple Levels

Failure Mode	Effect Level	Effect Description
Short shot	Local	Part is incomplete/missing features
Short shot	Next Operation	Part rejected at inspection, line stoppage
Short shot	End Customer	If shipped: product malfunction, safety hazard
Flash	Local	Excess material on part edges
Flash	Next Operation	Secondary trimming required, increased cycle time
Flash	End Customer	If shipped: poor fit, appearance defect

18.4.5 Step 5: Determining Severity Ratings

Severity (S) rates the seriousness of the effect on a scale of 1-10.

Table 18.9: Severity Rating Scale (S)

Rating	Effect	Criteria
10	Hazardous - without warning	Safety hazard; noncompliance with regulations; n
9	Hazardous - with warning	Safety hazard; noncompliance with regulations; v
8	Very High	Product inoperable; loss of primary function
7	High	Product operable but at reduced performance
6	Moderate	Product operable; comfort/convenience affected; noticea
5	Low	Product operable; reduced comfort; noticed by average c
4	Very Low	Product operable; slightly reduced comfort; noticed by c
3	Minor	Fit/finish nonconforming; noticed by discriminating cus
2	Very Minor	Fit/finish nonconforming; noticed only by trained obser
1	None	No discernible effect

Important: Severity is determined by the **effect**, not the failure mode. Severity can only be reduced by changing the design — process changes typically cannot reduce severity.

18.4.6 Step 6: Identifying Potential Causes

For each failure mode, identify all possible causes: “**Why might this failure occur?**”

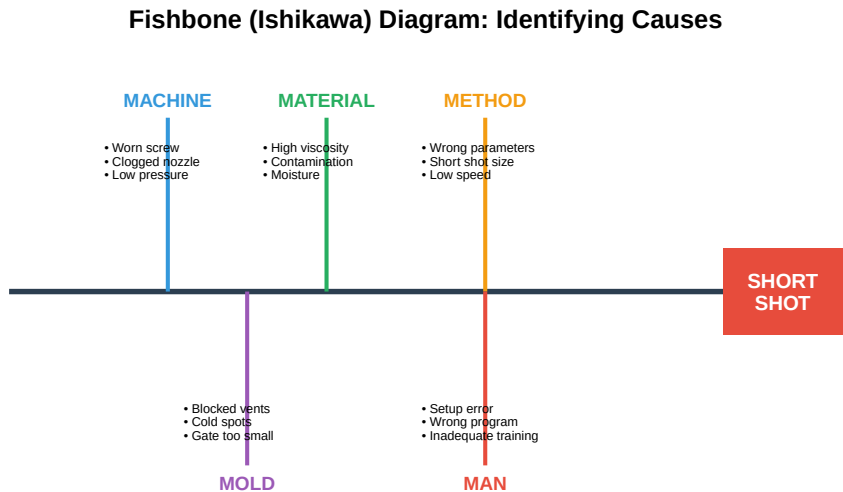


Figure 18.7: Fishbone Diagram: Causes of Short Shot

18.4.7 Step 7: Rating Occurrence

Occurrence (O) rates the likelihood that the cause will occur on a scale of 1-10.

Table 18.10: Occurrence Rating Scale (O)

Rating	Likelihood	Criteria	Approximate Rate	Cpk
10	Very High	Failure is almost inevitable	100 per 1000 (10%)	< 0.33
9	Very High	Failures occur very frequently	50 per 1000 (5%)	0.33
8	High	Failures occur frequently	20 per 1000 (2%)	0.51
7	High	Failures occur on regular basis	10 per 1000 (1%)	0.67
6	Moderate	Occasional failures	5 per 1000 (0.5%)	0.83
5	Moderate	Occasional failures expected	2 per 1000 (0.2%)	1.00
4	Low	Relatively few failures	1 per 1000 (0.1%)	1.17
3	Low	Few failures	0.5 per 1000 (500 ppm)	1.33
2	Remote	Failure unlikely	0.1 per 1000 (100 ppm)	1.50
1	Remote	Failure nearly impossible	0.01 per 1000 (10 ppm)	1.67

18.4.8 Step 8: Current Controls and Detection Ratings

Identify current controls and rate their ability to detect the failure or cause.

Types of Process Controls:

Table 18.11: Types of Process Controls

Control Type	Purpose	Examples
Prevention Controls	Prevent the cause from occurring	Error-proofing (poka-yoke), interlocks
Detection Controls	Detect the failure mode or cause after it occurs	Inspection, testing, gauging, SPC

Detection Rating Scale:

Table 18.12: Detection Rating Scale (D)

Rating	Detection	Criteria	Example Controls
10	Absolute Uncertainty	No current control; cannot detect or is not analyzed	No inspection
9	Very Remote	Control will probably not detect	Indirect measurement
8	Remote	Control has poor chance of detection	Visual inspection
7	Very Low	Control has low chance of detection	Manual 100% inspection
6	Low	Control may detect	Variable gauging
5	Moderate	Control has moderate chance of detection	SPC charting
4	Moderately High	Control has moderately high chance of detection	In-station gauging
3	High	Control has high chance of detection	Automated 100% inspection
2	Very High	Control almost certain to detect	Multiple independent inspections
1	Almost Certain	Control will detect; error-proofed	Error-proofing

Important: Detection ratings are often **harder to reduce** than occurrence ratings. Adding inspection doesn't prevent defects — it only catches them. Prevention is always better than detection.

Question: Why are detection ratings often harder to improve?

Detection ratings are harder to reduce because:

1. **Detection doesn't prevent:** The defect has already been made; you're just trying to find it
2. **Human inspection is unreliable:** Typical effectiveness is only 80-85% for visual inspection
3. **100% inspection is expensive:** Automated inspection requires significant investment
4. **Some defects are hard to detect:** Internal defects, intermittent problems, latent failures

5. **Sampling has limitations:** Even good sampling plans miss some defects

Better approach: Focus on reducing occurrence through: - Error-proofing (poka-yoke) - Process capability improvement - Preventive maintenance - Better training

18.4.9 Step 9: Calculating RPN and Prioritization

The **Risk Priority Number (RPN)** combines all three ratings:

$$\text{RPN} = \text{Severity (S)} \times \text{Occurrence (O)} \times \text{Detection (D)}$$

Live cell — try different Severity, Occurrence, and Detection ratings.

```
#| label: rpn-calculation
# Example RPN Calculations

# Failure Mode 1: Short shot
S1 <- 7    # Severity: Product inoperable (assembly)
O1 <- 5    # Occurrence: Occasional
D1 <- 6    # Detection: Moderate (visual inspection)

RPN1 <- S1 * O1 * D1
cat("Short shot RPN:", RPN1, "\n")

# Failure Mode 2: Flash
S2 <- 4    # Severity: Appearance issue
O2 <- 6    # Occurrence: Regular
D2 <- 3    # Detection: High (easy to see)

RPN2 <- S2 * O2 * D2
cat("Flash RPN:", RPN2, "\n")

# Failure Mode 3: Contamination
S3 <- 9    # Severity: Safety concern (medical device)
O3 <- 3    # Occurrence: Low
D3 <- 7    # Detection: Low (internal contamination)

RPN3 <- S3 * O3 * D3
cat("Contamination RPN:", RPN3, "\n")

# Comparison
cat("\n--- Priority Order by RPN ---\n")
cat("1. Short shot (RPN =", RPN1, ")\n")
cat("2. Contamination (RPN =", RPN3, ")\n")
cat("3. Flash (RPN =", RPN2, ")\n")
```


RPN Action Thresholds (Traditional):

Table 18.13: RPN Action Thresholds

RPN Range	Risk Level	Action Required
1-50	Low	Monitor; no immediate action
51-100	Moderate	Consider action if resources available
101-200	Significant	Action required within planning cycle
201-500	High	Immediate action required
501-1000	Critical	Stop production; immediate corrective action

Interactive Exercise: Calculate RPN

Calculate the RPN for each failure mode and rank them:

Failure Mode	Severity	Occurrence	Detection	RPN
A: Dimension out of spec	6	4	5	?
B: Surface scratch	3	7	2	?
C: Missing component	8	2	8	?
D: Wrong material	9	3	6	?

Answers: - A: $6 \times 4 \times 5 = 120$ - B: $3 \times 7 \times 2 = 42$ - C: $8 \times 2 \times 8 = 128$ - D: $9 \times 3 \times 6 = 162$

Priority Order: D (162) > C (128) > A (120) > B (42)

Question: Should we always address the highest RPN first? See limitations discussion below.

18.4.10 Limitations of RPN

RPN has known limitations that must be understood:

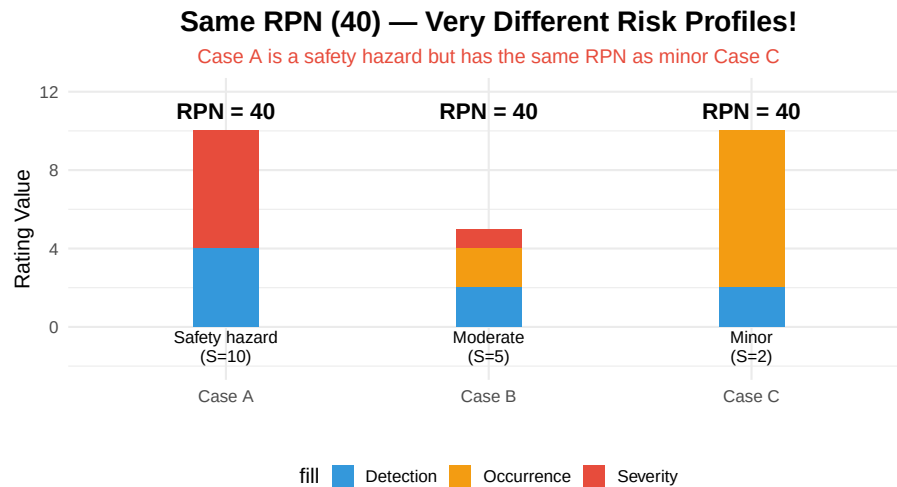


Figure 18.8: RPN Limitation Example

Key RPN Limitations:

1. **Same RPN, different risk:** $10 \times 1 \times 4 = 40$, but $2 \times 10 \times 2 = 40$
2. **Severity often overlooked:** High severity should always trigger action
3. **Arbitrary thresholds:** Why is 100 the magic number?
4. **Non-linear scale:** Difference between 2 and 3 is not same as 8 and 9

18.4.11 Action Priority (AP) Method — AIAG-VDA Alternative

The 2019 AIAG-VDA FMEA Handbook introduced the **Action Priority (AP)** method as an alternative to RPN.

Table 18.14: Action Priority (AP) Method

Priority	Criteria	Action Required
HIGH	Severity = 9 or 10 (any O, any D)	Must take action to improve
HIGH	Severity = 8 AND Occurrence 4	Must take action
MEDIUM	Severity = 7 AND Occurrence 5	Should take action
MEDIUM	Severity = 5-6 AND Occurrence 6 AND Detection 5	Should take action
LOW	All other combinations	May take action; document

18.4.12 Step 10: Recommended Actions and Follow-Up

For high-priority items, develop specific recommended actions.

Table 18.15: Step 10: Recommended Actions with T

Failure Mode	Current RPN	Recommended Action	Responsible	T
Short shot	210	Install shot size monitoring with automatic reject	Process Engineer	2
Short shot	210	Implement preventive maintenance for nozzle cleaning	Maintenance	2
Short shot	210	Add mold temperature monitoring with alarm	Quality Engineer	2

18.5 Complete PFMEA Example: Injection Molding

Let's work through a complete PFMEA for an injection molding process step by step.

Table 18.16: Complete PFMEA Example: Injection Molding Process

Step	Function	Failure Mode	Effect	S	Cause
3. Inject	Fill mold completely	Short shot	Part scrapped, line stoppage	7	Insufficient shot size
3. Inject	Fill mold completely	Flash	Secondary trim required	4	Excessive pressure
3. Inject	Fill mold completely	Air entrapment	Weak area, visual defect	6	Blocked vent
4. Cool	Solidify uniformly	Warpage	Dimensional rejection	7	Uneven cooling
4. Cool	Solidify uniformly	Sink marks	Visual defect	5	Thick section, short p
5. Eject	Remove without damage	Stuck in mold	Part damage, mold damage	8	Insufficient draft angle

Live cell — try different S/O/D ratings for each improvement option.

```
#| label: rpn-improvement
# Improvement Analysis for Short Shot (Highest RPN = 210)

cat("=== CURRENT STATE ===\n")
S_current <- 7 # Severity (cannot change - inherent to failure)
O_current <- 5 # Occurrence
D_current <- 6 # Detection
RPN_current <- S_current * O_current * D_current
cat("Current RPN:", RPN_current, "(S=7, O=5, D=6)\n\n")

cat("=== IMPROVEMENT OPTIONS ===\n\n")

# Option 1: Improve Detection
cat("Option 1: Add automatic shot weight monitoring\n")
D_option1 <- 2 # Automated detection
RPN_option1 <- S_current * O_current * D_option1
```

```
cat("New RPN:", RPN_option1, "(S=7, O=5, D=2)\n")
cat("Reduction:", round((1 - RPN_option1/RPN_current)*100), "%\n\n")

# Option 2: Improve Occurrence (Prevention)
cat("Option 2: Add screw position monitoring with feedback control\n")
O_option2 <- 2 # Much lower occurrence
RPN_option2 <- S_current * O_option2 * D_current
cat("New RPN:", RPN_option2, "(S=7, O=2, D=6)\n")
cat("Reduction:", round((1 - RPN_option2/RPN_current)*100), "%\n\n")

# Option 3: Both improvements
cat("Option 3: Both improvements combined\n")
RPN_option3 <- S_current * O_option2 * D_option1
cat("New RPN:", RPN_option3, "(S=7, O=2, D=2)\n")
cat("Reduction:", round((1 - RPN_option3/RPN_current)*100), "%\n\n")
```

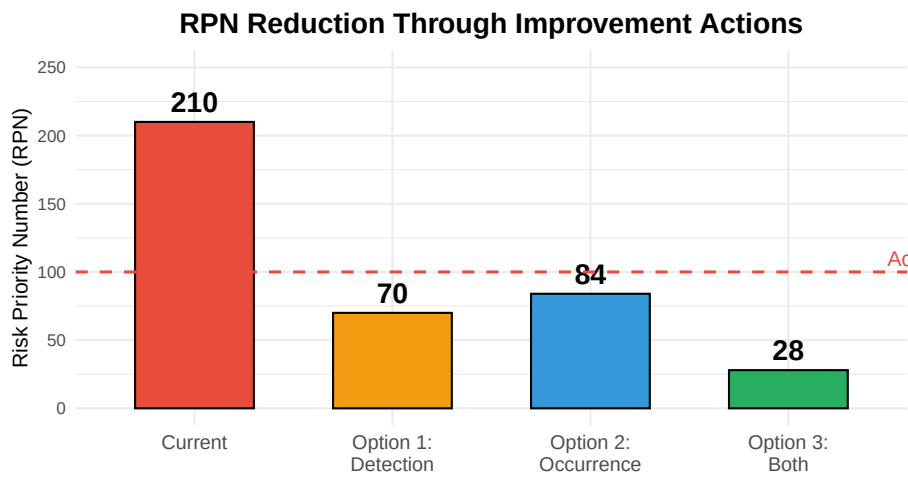


Figure 18.9: RPN Improvement Comparison

18.6 Industry-Specific PFMEA Examples

18.6.1 Example: Welding Process

Table 18.17: PFMEA Example: Welding Process

Failure Mode	Effect	S	Cause	O
Incomplete fusion	Weak joint, potential crack	9	Low heat input, contamination	4
Porosity	Leak path, strength reduction	8	Shielding gas issue, moisture	5
Undercut	Stress concentration, fatigue failure	7	Excessive current, wrong angle	4
Spatter	Appearance, cleanup required	3	Wrong parameters, contaminated wire	6
Distortion	Dimensional non-conformance	6	Excessive heat, improper sequence	4

18.6.2 Example: Heat Treatment Process

Table 18.18: PFMEA Example: Heat Treatment Process

Failure Mode	Effect	S	Cause	O	Current
Under-hardened	Premature wear, field failure	8	Low temp, short time	4	Hardne
Over-hardened/Brittle	Part cracks under load	9	High temp, long time	3	Hardne
Surface decarburization	Soft surface, wear	7	Wrong atmosphere	4	Surface
Distortion/Warpage	Dimensional rejection	6	Uneven heating, rapid quench	5	CMM s
Quench cracking	Scrap, safety hazard	10	Thermal shock, design	2	Visual

18.6.3 Example: Food Processing (Pasteurization)

Table 18.19: PFMEA Example: Pasteurization Process

Failure Mode	Effect	S	Cause	O	C
Under-processing	Pathogen survival, food safety	10	Equipment malfunction, calibration	2	C
Over-processing	Nutrient loss, off-flavor	5	Control system error	3	T
Temperature deviation	Inconsistent kill, shelf life	8	Sensor drift, flow variation	3	C
Time deviation	Process variation	6	Timer malfunction	3	T
Cross-contamination	Product recall, illness	10	Gasket leak, CIP failure	2	C

Interactive Exercise: Create a PFMEA

Create a simple PFMEA for a coffee brewing process in a commercial setting.

Process Steps: 1. Fill water reservoir 2. Load coffee grounds 3. Brew coffee 4. Dispense into cup

Identify at least 4 failure modes with S, O, D ratings and RPN.

Example Answer:

Step	Failure Mode	Effect	S	Cause	O	Control	D	RPN
1. Fill water	Wrong water level	Weak/stro- ng coffee	4	No level indica- tor	5	Visual check	5	100
2. Load grounds	Wrong amount	Weak/stro- ng coffee	4	No mea- suring	6	Visual check	6	144
3. Brew	Temperature too low	Under- extracted	6	Heating ele- ment worn	3	Temp display	4	72
4. Dis- pense	Overflow	Burn hazard, mess	8	Cup sensor failure	2	Cup sensor	4	64

Priority: Wrong amount (144) > Water level (100) > Temperature (72) > Overflow (64)

18.7 Integration with Other Quality Tools

PFMEA doesn't exist in isolation — it connects to other quality tools and systems.

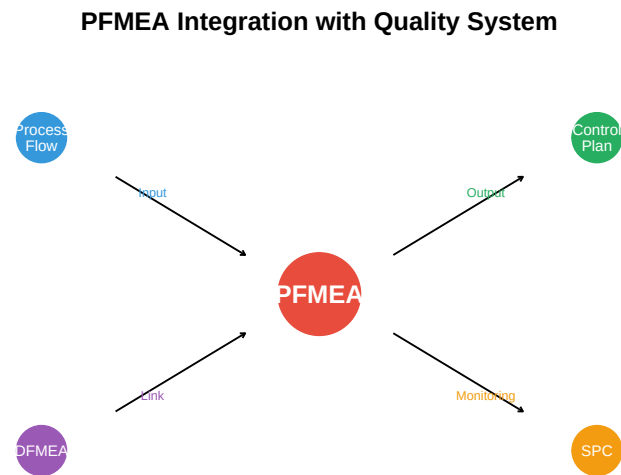


Figure 18.10: PFMEA Integration with Quality Tools

18.7.1 From PFMEA to Control Plan

The PFMEA directly feeds the **Control Plan** — a document that specifies what controls are needed at each process step.

Table 18.20: PFMEA to Control Plan Linkage

From PFMEA	To Control Plan
Process Step	Operation/Step
Failure Mode	Characteristic to Control
Severity	Classification (Critical/Significant)
Cause	Process Parameter
Current Control	Control Method
Detection	Reaction Plan

18.8 Common Pitfalls and Best Practices

18.8.1 Common Pitfalls to Avoid

Table 18.21: Common PFMEA Pitfalls and Solutions

Pitfall	Problem	Solution
Paperwork Exercise	PFMEA done only to satisfy customer requirement	Conduct
One-Person Show	Engineer completes alone without team input	Require c
Optimistic Detection	Overestimating effectiveness of current controls	Validate
Static Document	Never updated after process changes	Review a
RPN Obsession	Focusing only on RPN numbers, missing high-severity items	Also cons
Vague Failure Modes	Failure modes too general to be actionable	Be specif

18.8.2 Best Practices

Table 18.22: PFMEA Best Practices

Practice	Description
Start Early	Begin PFMEA during process design, not after problems occur
Include Operators	Operators know failures that never get documented
Use Historical Data	Use warranty data, scrap reports, customer complaints
Be Specific	Specific failure modes lead to specific actions
Follow Up	Verify that actions were implemented and effective
Living Document	Review and update regularly (minimum annually)

18.9 Case Study: Preventing a Major Quality Issue

18.9.1 Background

Company: Automotive brake component manufacturer

Product: Brake caliper mounting bracket

Situation: New manufacturing process being implemented for a high-volume contract

18.9.2 PFMEA Analysis

During the PFMEA session, the team identified a critical failure mode:

Table 18.23: Case Study: Critical Failure Mode Identified

Element	Analysis
Process Step	Drill mounting holes
Function	Create holes at 45.0 $\pm$ 0.1mm spacing
Failure Mode	Hole spacing out of tolerance
Effect	Caliper misalignment, brake performance affected
Severity	9 (safety-related)
Cause	Fixture wear, thermal expansion
Occurrence	4 (occasional)
Current Control	CMM check 5 per shift
Detection	5 (moderate sampling)
RPN	180

18.9.3 Actions Taken

Based on the PFMEA, the team implemented:

1. **In-process gauging:** Automated measurement after every part (D: 5 $\rightarrow$ 2)
2. **Fixture monitoring:** Temperature compensation and wear tracking (O: 4 $\rightarrow$ 2)
3. **Enhanced fixture design:** Carbide wear surfaces (O: 4 $\rightarrow$ 2)

Live cell — try different before/after ratings.

```

#| label: case-study-improvement
# Before improvements
S_before <- 9
O_before <- 4
D_before <- 5
RPN_before <- S_before * O_before * D_before
cat("Before RPN:", RPN_before, "\n")

# After improvements
S_after <- 9    # Severity unchanged (inherent)
O_after <- 2    # Reduced through fixture improvements
D_after <- 2    # Reduced through 100% gauging
RPN_after <- S_after * O_after * D_after
cat("After RPN:", RPN_after, "\n")
cat("Reduction:", round((1 - RPN_after/RPN_before)*100), "%\n")

```

18.9.4 Results

- **Zero field failures** related to hole spacing after 18 months of production
- **Estimated avoided cost:** \$2.4 million in potential recall and warranty

- **Investment:** \$85,000 for gauging and fixture upgrades
- **ROI:** 28:1

Key Lesson: The PFMEA investment of \$85,000 prevented a potential \$2.4 million problem. This is the power of proactive risk management.

18.10 Summary

Table 18.24: PFMEA Key Concepts Summary

Topic	Key Points
PFMEA Purpose	Prevent defects before they occur; proactive vs. reactive quality
Key Ratings	Severity (effect), Occurrence (likelihood), Detection (control effectiveness)
RPN Calculation	$RPN = S \times O \times D$; Range 1-1000; Higher = More risk
Prioritization	Address high RPN first, but always act on high severity (9-10)
Action Priority	AIAG-VDA method: High/Medium/Low based on S-O-D combination
Best Practice	Cross-functional team, living document, follow-up on actions

18.11 Review Questions

18.11.1 Conceptual Questions

Question 1: What is the difference between a failure mode and a failure effect?

Failure Mode: How the process fails to perform its intended function - Example: “Hole drilled undersize” - Example: “Temperature too low” - Example: “Missing component”

Failure Effect: The consequence of the failure mode - Example: “Part rejected at assembly” (effect of undersize hole) - Example: “Product not properly cured” (effect of low temperature) - Example: “Product doesn’t function” (effect of missing component)

Key Relationship: One failure mode can have multiple effects at different levels (local, downstream, customer).

Question 2: Why should PFMEA be conducted by a cross-functional team?

Reasons for Cross-Functional Team:

1. **Diverse expertise:** No single person understands all aspects
2. **Multiple perspectives:** Quality sees defects differently than production

3. **Historical knowledge:** Operators know undocumented problems
4. **Buy-in:** People support what they help create
5. **System thinking:** Process interactions become visible
6. **Better solutions:** Different functions propose different improvements

Typical team composition: - Process Engineer (owner) - Quality Engineer - Production Supervisor/Operator - Maintenance Technician - Design Engineer - Supplier Representative (when applicable)

Question 3: Explain why detection ratings are often harder to reduce than occurrence ratings.

Detection is harder to reduce because:

1. **Detection doesn't prevent:** Defect already exists; just trying to find it
2. **Human inspection is unreliable:** 80-85% effectiveness typical
3. **100% inspection is expensive:** Requires automation investment
4. **Some defects are hidden:** Internal defects, intermittent failures
5. **Sampling limitations:** Cannot catch all defects with sampling

Better approach: Focus on occurrence reduction: - Error-proofing (poka-yoke) - Process capability improvement - Preventive maintenance - Better training

Example: Reducing drill wear (occurrence) is more effective than adding more inspection (detection).

Question 4: At what point in process development should PFMEA ideally be conducted?

Ideal Timing: During process design, before production begins

Why early is better: - Changes cost 1x during design - Changes cost 10x during production - Changes cost 100x after customer delivery

PFMEA Timing Points: 1. **New process development:** Before equipment is ordered 2. **New product introduction:** During process planning 3. **Process changes:** Before implementing changes 4. **Quality issues:** When problems indicate process risk 5. **Regular review:** Annually at minimum

Key Principle: The earlier PFMEA is conducted, the more opportunity exists to design out failures rather than inspect them out.

18.11.2 Calculation Questions

Question 5: Given S=8, O=6, D=4, calculate RPN. If improved controls reduce detection to 2, what is the new RPN?

Initial Calculation:

$$\text{RPN} = S \times O \times D$$

$$\text{RPN} = 8 \times 6 \times 4 = 192$$

After Detection Improvement:

New RPN = $8 \times 6 \times 2 = 96$

Analysis: - Original RPN: 192 (Significant risk, action required) - New RPN: 96 (Moderate risk) - Reduction: 50%

Note: Severity (8) remains unchanged because it's determined by the effect, not by process controls. Only occurrence and detection can be reduced through process improvements.

Question 6: Two failure modes have RPNs of 120 and 150. Should the higher RPN always be addressed first?

Answer: Not necessarily!

Consider the following scenarios:

Scenario A: - Failure Mode 1: RPN = 150 (S=3, O=10, D=5) — Minor cosmetic issue - Failure Mode 2: RPN = 120 (S=10, O=3, D=4) — Safety hazard

Priority: Address Failure Mode 2 first despite lower RPN because: - Severity 10 indicates safety hazard - High-severity items should always be addressed regardless of RPN - The AP (Action Priority) method would classify this as HIGH priority

When to prioritize lower RPN: - Higher severity (especially 9-10) - Regulatory or safety implications - Customer-critical characteristic - Easier/faster to implement

Key Lesson: RPN is a prioritization tool, not the only decision criterion. Use engineering judgment.

18.11.3 Application Questions

Question 7: For a metal stamping operation, identify at least five potential failure modes and their likely effects.

Metal Stamping PFMEA:

#	Failure Mode	Effect
1	Part not fully formed	Dimensional rejection, functional failure
2	Burrs on edges	Injury hazard, assembly interference
3	Cracks/splits	Structural failure, scrap
4	Wrong material thickness	Dimensional/strength issues
5	Mislocated features	Assembly issues, rejection

#	Failure Mode	Effect
6	Surface scratches	Appearance rejection, corrosion initiation
7	Springback out of tolerance	Dimensional rejection
8	Die mark/impression	Appearance defect

Potential Causes for “Part not fully formed”: - Insufficient press tonnage
- Die wear - Material too hard - Lubrication insufficient - Press speed too fast

Question 8: A process has RPN of 240 (S=8, O=6, D=5). Propose three improvement strategies and calculate resulting RPN.

Current State: S=8, O=6, D=5, RPN=240

Strategy 1: Improve Detection - Add automated inspection (100% gauging)
- New D = 2 - New RPN = $8 \times 6 \times 2 = 96$ (60% reduction)

Strategy 2: Improve Occurrence - Add error-proofing (poka-yoke) to prevent cause - New O = 2 - New RPN = $8 \times 2 \times 5 = 80$ (67% reduction)

Strategy 3: Combined Approach - Error-proofing ($O \rightarrow 3$) + Better detection ($D \rightarrow 3$) - New RPN = $8 \times 3 \times 3 = 72$ (70% reduction)

Recommendation: Strategy 2 (error-proofing) is typically preferred because:
- Prevents defects from occurring - More sustainable long-term - Reduces inspection costs - Better for customer confidence

18.11.4 Critical Thinking Questions

Question 9: Discuss the limitations of using RPN as the sole prioritization method.

RPN Limitations:

- Equal weighting problem:**
 - $10 \times 1 \times 4 = 40$ (safety hazard, rare)
 - $2 \times 10 \times 2 = 40$ (minor, frequent)
 - Same RPN, very different risk!
- Severity often overlooked:**
 - High-severity items (9-10) should always be addressed
 - RPN can hide safety-critical issues
- Non-linear scales:**
 - Difference between S=2 and S=3 is not same as S=8 and S=9
 - Multiplication assumes linear relationship
- Arbitrary thresholds:**
 - Why is RPN=100 the “action” threshold?
 - No scientific basis for specific cutoffs
- Gaming the system:**

- Teams may manipulate ratings to avoid action
- Subjectivity in ratings

Better Approaches: - Action Priority (AP) method - Always act on S = 9 regardless of RPN - Consider multiple criteria (cost, feasibility, timing) - Use risk matrix visualization

Question 10: How would you handle a situation where team members disagree on severity ratings?

Approach to Rating Disagreements:

- 1. Clarify the failure effect:** - Ensure everyone understands the same effect
- Be specific about which customer sees which impact
- 2. Reference the rating criteria:** - Use the standardized scale consistently
- Point to specific criteria, not opinions
- 3. Consider worst-case scenario:** - PFMEA should consider worst reasonable case - “Could this effect happen?” vs “Will it happen?”
- 4. Gather data:** - Review historical data if available - What have similar failures caused in the past?
- 5. When in doubt, go higher:** - Conservative approach is appropriate for safety - Can always reduce later with evidence
- 6. Document the discussion:** - Record the rationale for the rating - Note if there was disagreement and why decision was made
- 7. Get external input:** - Bring in customer perspective if needed - Consult subject matter experts

Final Rule: If a safety or regulatory issue is possible, severity should be 9 or 10 regardless of probability.

18.12 References

- AIAG & VDA. (2019). *FMEA Handbook*. Automotive Industry Action Group.
- Stamatis, D.H. (2003). *Failure Mode and Effect Analysis: FMEA from Theory to Execution*. ASQ Quality Press.
- McDermott, R.E., Mikulak, R.J., & Beauregard, M.R. (2009). *The Basics of FMEA*. Productivity Press.
- Carlson, C.S. (2012). *Effective FMEAs: Achieving Safe, Reliable, and Economical Products and Processes*. Wiley.
- SAE J1739. (2009). *Potential Failure Mode and Effects Analysis in Design and Manufacturing*.
- ISO 31000:2018. *Risk management — Guidelines*.

- IATF 16949:2016. *Quality management system requirements for automotive production.*
-

Chapter 19

Labs

This chapter collects the lab instructions, prep work, rubrics, marking notes and supplementary information for every lab in the course, one sub-chapter per lab. The last section, Teacher preparation, is a term-planning aid for the instructor: it lists the room, equipment, software and a consolidated shopping list for every lab.

19.1 How to use this chapter

Every lab sub-chapter is broken into the **same fixed parts**, and each part has its own colour so you always know which part of the lab you are reading:

Colour	Part	What it is
Dark blue	Overview	What the lab is about and why you do it
Teal	Learning outcomes	What you should be able to do afterwards
Orange	Prep work	Readings, videos and set-up to finish <i>before</i> the lab period
Blue	Materials & equipment	What to bring / what is provided
Purple	Procedure	The steps you carry out during the lab
Green	Deliverables & submission	What you hand in, and when
Slate	Rubric	How the mark is calculated

Colour	Part	What it is
Red, dashed	Instructor notes	Marking guidance and prep — not required reading for students

! Lab structure

The lab set is built around a single course-long workplace narrative — **Précitech Components**, a Tier-1 automotive supplier — with a six-week machine-learning module in the middle of the term. **The term is 14 weeks and there are 14 labs: the lab number is the week number.**

#	Week	Lab	Prereq. chapters
1	1	Logic Thinking for Robot Control — LightBot & Rocksi	—
2	2	Engineering Design Process Documentation	2, 4
3	3	Manufacturing Facility Design & Simulation (AnyLogic)	4, 16
4	4	Lean: Value Stream Mapping, Takt and Pull	3, 5
5	5	Ergonomics in Automated Manufacturing (REBA & Inseer)	7
6	6	SPC and Gauge R&R — the Virtual Caliper	10, 11

7	7	Machine Learning Week 1 — AI literacy & prompt engineering	20
8	8	Machine Learning Week 2 — Manufacturing data & SPC	21
9	9	Machine Learning Week 3 — the C-MAPSS dataset	21
10	10	Machine Learning Week 4 — building the model	21
11	11	Machine Learning Week 5 — deploying the app	21
12	12	Machine Learning Week 6 — capstone presentations	21
13	13	TPM/OEE, Root Cause and a New Dataset	12, 15, 21
14	14	Robotic Work-Cell Design — RoboDK for Web (take-home, released wk 11)	3, 6, 8

Why the labs sit where they do:

- **Lab 6 (SPC and Gauge R&R by hand) precedes Lab 8 (SPC in Python) by two weeks.** That is deliberately the arc Chapter 13 is built on: *do it yourself first, then direct an LLM to do it.*
- **Lab 7 (ML Week 1) sits in midterm week on purpose.** The midterm is written during *lecture* time, so the week-7 lab period runs as normal. ML Week 1 is the right lab for that slot — individual, browser-only, no dataset, no prep beyond a login: the lightest lab in the set.
- **Lab 13 follows the capstone** because it reuses the whole ML pipeline on an unseen, differently-shaped dataset — which only works once students have the skill.
- **Lab 14 is the take-home**, released in week 11 and due at the end of week 14, reusing the course plan's existing take-home precedent. RoboDK for Web needs no install, so it genuinely works from home, and it bookends Lab 1: toy logic in week 1, a real industrial arm in week 14.

Not in the current set: the popsicle-stick bridge lab, the lean paper-cutting simulation, and the automation/functional-safety lab series. The old handouts still exist in the `LabInstructions/` folder and can be reinstated if the course plan changes; the older **instrumentation-trainer** stream (B1033 benches) is summarised in Appendix: instrumentation-trainer lab stream.

Labs are due at the end of the scheduled lab period; the late penalty is 25 % per lab session. **Lab 14 is the exception — it is a take-home, released in week 11 and due at the end of week 14, with a short screen recording.** Marks may be deducted for incomplete prelab work, missing safety training, not bringing required materials, not cleaning your station, or unprofessional conduct in the lab room. The schedule in `index.qmd` is the current plan; follow FOL if it diverges.

19.2 Lab 1 — Logic Thinking for Robot Control: LightBot & Rocksi

An introduction to programming logic — sequencing, loops and conditionals — and to robot teach-pendant operation, using two free browser simulators: **LightBot** (a puzzle game that teaches sequence/loop/conditional thinking) and **Rocksi** (a teach-pendant simulator for the Sawyer robot). The focus is on *optimising* the number of programming steps.

AI use: not permitted — the outcome *is* your own logic. Lab 14 returns to a

real 6-axis arm in RoboDK for Web.

19.2.1 Learning outcomes

After this lab you should be able to:

- Explain why programming logic matters for reliable, safe, repeatable automation.
- Use sequences, loops and conditionals to solve a control problem with the fewest steps.
- Describe the role of a teach pendant (jogging, teaching points, deadman/E-stop, reduced-speed teach mode, I/O testing).
- Program a simple pick-and-place-style sequence on a simulated teach pendant.

19.2.2 Prep work

- Read `Process Engineering Lab#2 Programming Logic.pdf` (importance of programming logic; loops; conditionals; teach pendants).
- Watch: *Rocksi robot simulator: Meet Sawyer*.

Rocksi robot simulator: Meet Sawyer: <https://www.youtube.com/watch?v=9oAFmTlMqzg>

- Check that **LightBot** and **Rocksi** load on your device.

19.2.3 Materials & equipment

- Computer or tablet with internet access — **one per student** (this is an individual lab)
- Access to **LightBot** (browser)
- Access to **Rocksi** teach-pendant simulator (browser)
- Lab worksheet (provided — the questions are reproduced in the Procedure below)

19.2.4 Procedure

Part 1 — Programming logic with LightBot

1. *Sequencing*. Complete the first levels. **Worksheet:** describe the sequence of commands you used to complete Level 1. *(1 mark)*
2. *Loops*. Complete levels that require loops. **Worksheet:** how did using loops let you complete the level more efficiently? *(2 marks)*
3. *Conditionals*. Complete levels that use conditional commands. **Worksheet:** give an example of a conditional you used and explain its purpose. *(2 marks)*

4. *Challenge — maximum level & optimisation.* Reach Level 15 with the minimum number of steps. **Worksheet:** describe your strategy and how you optimised. (2 marks)

Part 2 — Robot teach-pendant simulation with Rocksi

5. *Basic movements.* Jog the robot to several positions. **Worksheet:** describe the steps to move the robot to Position A. (1 mark)
6. *Programming a sequence.* Program a sequence of moves with the teach pendant. **Worksheet:** list the command sequence you used. (1 mark)
7. *Simulating inputs & outputs.* Simulate an I/O to troubleshoot a robot operation. **Worksheet:** explain how you simulated the I/O and its effect. (1 mark)

Finish each part with a short reflection on how the skills apply to engineering design and safety.

19.2.5 Deliverables & submission

Completed worksheet (all questions above plus the two reflections), submitted on FOL by the end of the lab period. Assessment is based on task completion in both simulators and the quality of the written answers.

19.2.6 Rubric

From the handout's marking table — **10 marks total**:

Task	Marks
Sequencing (LightBot)	1
Loops (LightBot)	2
Conditionals (LightBot)	2
Challenge task (LightBot) — reach Level 15, optimised	2
Basic movements (Rocksi)	1
Programming a sequence (Rocksi)	1
Simulating inputs & outputs (Rocksi)	1

(The handout body lists the LightBot challenge at 2 marks; an earlier summary line totals the same tasks to 10. Use the table above.)

19.2.7 Instructor notes

- **Prep:** confirm both simulators are reachable through the lab network/firewall *the day before* — LightBot and Rocksi are third-party sites and are the single point of failure for this lab. Have an offline fallback (screenshots / a slide walk-through) ready.
- Individual lab — one machine per student; no group submission.

- **Marking:** award loop/conditional marks for *explanation*, not just level completion. For the challenge task, ask for the step count as evidence of optimisation.

19.3 Lab 2 — Engineering Design Process Documentation

You produce the engineering-documentation package that a real project needs to move a design into production — specification sheets, a bill of materials, a task list / process flow, a project schedule and Gantt chart, a production-facility layout drawing, and a health-safety-environment (HSE) risk assessment — for the **Précitech Components shaft-kit assembly cell** described in the case study below. The same cell is simulated in Lab 3, so the documents you build here feed directly into the next lab. Tools: Microsoft Visio/Word, Project and Excel.

AI use: not permitted for any part of the report.

i Case study — Précitech Components: new shaft-kit assembly & packing cell

Précitech Components is a Tier-1 automotive supplier in Granby, Québec. Its CNC turning cells produce transmission **shaft blanks** (AISI 4140 steel, $\varnothing 25.000 \text{ mm} \pm 0.050 \text{ mm}$) for a major OEM customer. The OEM now wants the shafts delivered as a **ready-to-install kit** rather than as loose blanks, so Précitech is adding a **manual kitting, sub-assembly and packing cell** immediately downstream of the CNC turning cells.

One shaft kit contains: 1 machined shaft blank, 2 tapered roller bearings, 1 retaining ring, 1 lip seal, 1 dowel pin, 1 desiccant sachet — placed in a moulded tray inside a labelled carton.

Target output: 250 kits/day on one 8-hour shift.

Process outline: finished blanks arrive from the CNC cells on a trolley; bought-in components (bearings, rings, seals, pins) are held in a small storage rack that is replenished from the main warehouse when stock falls to a reorder level; an operator picks the components for one kit, presses the two bearings onto the shaft, fits the retaining ring and lip seal, inspects the assembly against a checklist, places it in a tray, closes and labels the carton, and stages it for the shipping dock.

Plant manager **Sophie Tran** has asked your team to produce the engineering documentation package for this cell **before it is built**.

This is the same plant used in the machine-learning labs (Labs 7–12), which build a predictive-maintenance proof of concept for Précitech's CNC

turning cells.

19.3.1 Learning outcomes

After this lab you should be able to:

- Explain terms such as HSE, Basic Engineering Design Data (BEDD), Front-End Engineering Design (FEED), fast-tracking, project schedule and General Arrangement (GA) drawing.
- Produce a specification sheet and a bill of materials for a manufactured product.
- Build a task list / process-flow chart with durations and turn it into a project schedule and Gantt chart in Microsoft Project.
- Draft a production-facility layout drawing showing product in/out, stations, storage, and material and people flow, and mark starvation / blockage risk points.
- Complete an HSE risk assessment for a manual assembly cell.

19.3.2 Prep work

- Read the **case study** above and Unit 2 content on the basics of process engineering and design.
- Skim **Lab 3 - Supplemental.pdf** for the document templates and the detail-design / engineering process-flow diagram (ignore the popsicle-bridge framing — the document types are the same).
- Confirm you can open Microsoft Visio/Word, Project and Excel on a lab PC.
- Form teams of 2–3.

19.3.3 Materials & equipment

- Lab PC with **Microsoft Word/Visio, Microsoft Project, Microsoft Excel**
- Templates from FOL:
 - **Bill_of_Materials_Template_for_Excel.xlsx**
 - **Gantt_chart.mpp** (Microsoft Project example file)
 - HSE risk-assessment Word template (**Risk assessment template**)
- The Précitech shaft-kit case study (above) — the single source of requirements

19.3.4 Procedure

Phase 1 — Specification sheet & bill of materials. Write a specification sheet for the shaft kit (customer, part number, contents, key dimensions and tolerances, packaging, marking, applicable standards) and a specification sheet for at least one bought-in component (e.g. the tapered roller bearing). Fill in

the Excel BOM template for one complete kit — every item, quantity, unit and source (machined in-house vs. purchased).

Phase 2 — Task list / process flow. Build a chronological task list for producing one kit: *receive blanks* → *store components* → *pick components for one kit* → *press bearings* → *fit retaining ring and seal* → *inspect* → *place in tray* → *close and label carton* → *stage for dock*. Add a column for the time each task needs. Use the task-numbering scheme (Task 1 → Sub-task 11 → Sub-sub-task 121...) from the supplement. Draw the flow as a chart in Visio or Word.

Phase 3 — Project schedule & Gantt chart. Open the example .mpp file, enter the tasks and durations for **standing up the cell** (procure racking and fixtures, install bearing press, set up inspection station, write the SOP, train operators, run a pilot batch), and generate a schedule and Gantt chart.

Phase 4 — Production-facility layout drawing. From the task list, allocate space for each activity and draw a General Arrangement of the kitting cell. Show: product in (blanks) and product out (kits), component storage rack, assembly station(s), bearing press, inspection station, packing bench, staging area, and the raw-material, finished-goods, waste and people-movement flows. Mark where **material starvation** (components not replenished in time) and **material blockage** (finished kits not removed) could occur. Discuss with your team and the instructor and get signed off.

Phase 5 — HSE risk assessment. Identify at least **five** health-and-safety risks in the cell — for example manual handling of the blank trolley, pinch points at the bearing press, sharp edges on retaining rings, repetitive assembly motion, and the carton knife — and document each in the risk-assessment template (hazard, who is harmed and how, current controls, further action, owner, deadline, done).

19.3.5 Deliverables & submission

One report containing all five deliverables: specification sheets + BOM, task list / process-flow chart, Microsoft Project schedule + Gantt chart, facility layout drawing, and the completed HSE risk assessment. Keep the electronic layout drawing — you refine and reuse it in Lab 3. Submit on FOL by the end of the lab period.

AI tools may not be used for any part of the report.

19.3.6 Rubric

Adapted from Lab 3 - Supplemental.pdf — **5 marks total:**

Deliverable	Marks
Specification sheet(s)	1
Bill of materials for one kit	1

Deliverable	Marks
Task list / process-flow chart	1
Project schedule + Gantt chart	1
Facility layout drawing + HSE risk assessment	1

(The source handout weighted these as five 1-mark items for a bridge; the split above keeps five items for the kitting cell. Confirm the exact weighting before the term.)

19.3.7 Instructor notes

- **Prep:** verify Microsoft **Project** and **Visio** licences are live on every lab PC (these are the parts most likely to be missing from a standard Office install). Re-upload the three FOL templates each term and check the `.mpp` opens in the installed Project version. Post the case study on FOL as its own one-page handout.
- **Continuity:** this lab and Lab 3 share the Précitech kitting cell. Tell students up front that the layout drawing they produce here becomes the model they build in Lab 3, so it is worth doing well.
- **Marking:** look for traceability — the spec sheet, BOM, task list, schedule and layout should all describe the *same* cell and the *same* kit, not generic examples. The starvation / blockage annotations on the layout are the link to the simulation lab; check they are present and sensible.

19.4 Lab 3 — Manufacturing Facility Design & Simulation (AnyLogic)

You turn the Précitech kitting-cell layout from Lab 2 into an electronic drawing, then use **AnyLogic Personal Learning Edition** to run a discrete-event simulation of the cell's order-fulfilment behaviour. By changing order and replenishment parameters and reading the statistics screens, you see how component-stock policy drives waiting time, picking (kitting) time and throughput.

AI use: not permitted for any part of the report.

19.4.1 Learning outcomes

After this lab you should be able to:

- Produce an electronic facility layout showing stations, storage, flows, manual vs. automated stations, and starvation / blockage risk points.
- Run an AnyLogic model, change input parameters, and capture the statistics output.

- Interpret order statistics (preorder waiting time, picking time, full processing time, order states) and goods statistics (unavailable goods, stored reserved / unreserved goods, storing time) and explain how they are linked.
- Explain how simulation supports facility-design decisions such as where to hold buffer stock and how often to replenish.

19.4.2 Prep work

- Re-read the Précitech case study and review Unit 3 content on integrated manufacturing systems.
- Read ENGR-3027- Lab-4-Supplemental Material.pdf (order statistics, goods statistics, and the relationships between them).
- Watch the AnyLogic walk-through video (link on FOL).
- **Install AnyLogic Personal Learning Edition** on your machine *before* the lab and confirm it launches.
- Bring your Lab 2 layout drawing, spec sheets, schedule, BOM and flow documents.

19.4.3 Materials & equipment

- Lab PC with **AnyLogic Personal Learning Edition** and **Microsoft Visio/Word/Project/Excel**
- The AnyLogic order-fulfilment / warehouse demo model referenced in the video — *confirm the exact model and stage it on FOL / the lab PCs*
- Your Lab 2 documents for the Précitech kitting cell

19.4.4 Procedure

Phase 1 — Facility drawing. Recreate your Lab 2 kitting-cell layout electronically (Visio or equivalent). Work through the checklist and record your findings: product in/out shown? work-sequence task list with timings? tools and component bins shown? component (raw-material) flow? finished-kit flow? waste-material flow? people-movement flow? how many stations, and are any automated (e.g. the bearing press)? where could component **starvation** occur? where could finished-kit **blockage** occur? are storage / assembly / inspection / packing / staging / social areas identified? Discuss with your team and the instructor and **get signed off** before Phase 2.

Phase 2 — AnyLogic simulation. Start AnyLogic and follow the video. Map the model onto the case study: *orders* = daily shipment demand for kits; *goods* = bought-in components held in the storage rack; *picking* = kitting one order; *reorder level* = the component stock at which the warehouse replenishes the rack. Run the model for **3 minutes** at each of the four parameter sets below; screenshot the Statistics screen and write observations & analysis for each:

Run	Min goods/order	Max goods/order	Order interarrival (s)	Reorder level
1	3	5	25	45
2	3	5	25	20
3	1	5	25	45
4	3	5	45	45

For each run, relate what you see back to the cell: does a lower reorder level starve the kitting station? does a faster order arrival rate create a backlog?

19.4.5 Deliverables & submission

Two deliverables in one report: **(1)** the electronic facility layout drawing of the Précitech kitting cell, and **(2)** the AnyLogic activity report — four labelled statistics screenshots with observations and analysis tied to the case study. Submit on FOL by the end of the lab period.

AI tools may not be used for any part of the report.

19.4.6 Rubric

From ENGR-3027- Lab-4-Supplemental Material.pdf — **10 marks total:**

Deliverable	Marks
Manufacturing-facility drawing	4
AnyLogic Run 1 — observation & analysis	2
AnyLogic Run 2 — observation & analysis	2
AnyLogic Run 3 — observation & analysis	2
AnyLogic Run 4 — observation & analysis	2

(Handout lists drawing + three analysis blocks = 10 marks. If four runs are required, confirm the split — likely drawing 4, analysis 1.5×4 .)

19.4.7 Instructor notes

- **Prep:** AnyLogic PLE install is the biggest failure point — push it to lab PCs ahead of the term and test a full model run. PLE has a model-size limit; confirm the demo model stays under it. Have the exact model file on FOL.
- Give the Phase 1 sign-off some rigour — students who skip it produce screenshots with no layout to tie them to. Check the layout matches their Lab 2 submission.

- **Marking:** the drawing carries 40 % of the mark; analysis should reference the *specific* statistic that moved *and* the case-study consequence (e.g. “lower reorder level → more unavailable goods → longer preorder waiting time → kitting station idle, shipments late”).

19.5 Lab 4 — Lean: Value Stream Mapping, Takt and Pull

You map the Précitech shaft line as it runs today, size it against customer demand, and redesign the flow. Working in **draw.io** (browser) you build a current-state **value stream map**; in **Google Sheets** you compute **takt time**, an operator balance chart and **process cycle efficiency**; and on **AnyLogic Cloud** (browser, free public tier) you compare push, pull and a kanban-capped supermarket. You finish with a future-state map carrying one SMED and one pull improvement. Everything runs in a browser — nothing to install.

AI use: permitted with disclosure, but **not** for the map itself or the takt / PCE calculations — those must be your own work. Name any tool you used and what for.

19.5.1 Learning outcomes

After this lab you should be able to:

- Draw a current-state value stream map from a station data table, with a material-flow line, an information-flow line and a lead-time ladder.
- Compute takt time from customer demand and compare it against station cycle times on an operator balance chart to find the bottleneck.
- Classify observed waste against the seven wastes and compute process cycle efficiency (value-added time ÷ lead time).
- Explain how push, pull and a kanban-capped supermarket differ in WIP and lead time, using simulation output as evidence.
- Propose a future state and justify the projected lead-time reduction.

19.5.2 Prep work

- Read Chapter 5 (Lean) on takt time, the seven wastes, pull systems, kanban and SMED, and the Chapter 3 material on line balancing.
- Create a free **draw.io** document (app.diagrams.net — “Device” storage is fine, no account needed) and skim the built-in *Value Stream* shape library.
- Confirm **AnyLogic Cloud** (cloud.anylogic.com) loads and that you can run a public model and edit its input parameters in the browser.
- Download `vsm_station_data.csv` from FOL — the station table for the Précitech shaft line (cycle time, changeover, uptime, WIP, batch size per

station).

- Form teams of 2–3.

19.5.3 Materials & equipment

- Computer with internet access; one per team is enough, one per student is better
- **draw.io** (browser) — current- and future-state maps
- **Google Sheets** (browser) — takt, balance chart, PCE
- **AnyLogic Cloud** (browser, free public tier) — push/pull model
- `vsm_station_data.csv` from FOL
- The Précitech case study — demand is 250 kits/day on one 8-hour shift

19.5.4 Procedure

Run the setup step first: open `vsm_station_data.csv` in Google Sheets and read the column headings so you know what each station contributes.

Task 1 — Current-state value stream map (30 min). In draw.io, map the shaft line end to end: CNC turn → deburr → wash → inspect → the Lab 2 kitting cell → shipping. For each process box record cycle time (C/T), changeover time (C/O), uptime and batch size from the supplied table; put an inventory triangle with the WIP count between each pair of boxes; add the supplier and customer, the production-control box, and the information-flow arrows (forecast, daily order, schedule). Draw the lead-time ladder along the bottom — processing time on the lower step, inventory wait time ($\text{WIP} \div \text{daily demand}$) on the upper step — and total it to a **production lead time** and a **value-added time**. (3 marks)

Task 2 — Takt and the balance chart (20 min). In Sheets, compute takt time = available shift time $\div$ customer demand for the 250 kits/day target (state your planned/unplanned downtime assumption). Build an operator balance chart — one bar per station, station C/T on the y-axis, a horizontal takt line across all bars. Name the bottleneck station and say by how much it exceeds takt. (2 marks)

Task 3 — Waste and process cycle efficiency (25 min). Walk the map and classify every wait, move and rework against the **seven wastes** (transport, inventory, motion, waiting, overproduction, overprocessing, defects) — at least one concrete example per waste that is present. Compute process cycle efficiency = value-added time $\div$ production lead time from your Task 1 ladder, and state whether the line is “lean” by the usual 25 % rule of thumb. (2 marks)

Task 4 — Push vs pull on AnyLogic Cloud (25 min). Open the supplied push/pull model on AnyLogic Cloud. Run it three ways, changing only the control policy: (a) **push** — every station runs whenever it has parts; (b) **pull** — downstream withdrawal triggers upstream; (c) **kanban-capped supermarket** — a fixed WIP cap between the bottleneck and its predecessor. For each run

19.5. LAB 4 — LEAN: VALUE STREAM MAPPING, TAKT AND PULL527

record average WIP and average lead time from the model output, in a small Sheets table. (2 marks)

Task 5 — Future-state map (20 min). In draw.io, redraw the line with **one SMED change** (halve the changeover at one station and justify the halving) and **one pull change** (a supermarket or FIFO lane where you had push). Recompute the lead-time ladder and state the projected lead-time reduction and which of the seven wastes each change attacks. (1 mark)

19.5.5 Deliverables & submission

One team report (PDF) containing: the current-state VSM (draw.io export), the takt calculation and operator balance chart, the seven-wastes table and the PCE calculation, the push/pull/supermarket results table with a two-sentence interpretation, and the future-state VSM with its justification. If you used an AI tool for anything permitted, add a one-line disclosure. Submit on FOL by the end of the lab period.

19.5.6 Rubric

10 marks total.

Deliverable	Marks
Current-state VSM — process boxes with C/T, C/O, uptime, batch; WIP triangles; information flow; lead-time ladder totalled	3
Takt calculation correct (demand, shift time, downtime stated); operator balance chart vs takt; bottleneck named with the gap quantified	2
Seven-wastes classification with a concrete example each; PCE computed correctly from the ladder and interpreted	2
Push / pull / supermarket runs completed; WIP and lead time recorded for all three; interpretation consistent with the numbers	2
Future-state VSM with one SMED and one pull change; projected lead-time reduction justified against the ladder	1

19.5.7 Instructor notes

- **Prep:** generate `vsm_station_data.csv` with `Rscript R/make_lab_data.R` and stage it on FOL. Publish the answer key from the same script — expected takt, bottleneck station, PCE and the push-vs-pull WIP/lead-time ordering.
 - Build or fork the AnyLogic Cloud push/pull model once and confirm the input parameters students need (control policy, WIP cap) are exposed on the public run screen. Test it through the college firewall the day before.
 - Confirm `draw.io` and `cloud.anylogic.com` are reachable on the lab network; both are on the Lab 4 / 13 firewall-check list in Teacher preparation.
 - **Marking:** the map and the takt/PCE arithmetic must be the students' own (AI not permitted for those). Check the lead-time ladder actually totals to the numbers they quote — a ladder that does not add up is the most common error.
 - This is the takt reference for **Lab 14** — students compare the RoboDK cycle time against the takt they compute here, so make sure the takt value is right.
-

19.6 Lab 5 — Ergonomics in Automated Manufacturing (REBA & Inseer)

You assess three workplace tasks for musculoskeletal risk two ways: manually with the **REBA (Rapid Entire Body Assessment)** worksheet, and automatically with **Inseer**, an AI video-based ergonomic-assessment tool. You then compare the two methods.

AI use: permitted with disclosure. Note that Inseer is itself an AI tool and is **required** for this lab — that is a lab instrument, not a generative AI writing your report.

19.6.1 Learning outcomes

After this lab you should be able to:

- Explain why ergonomics matters in an automated manufacturing facility and name common ergonomic risks and their effects on workers.
- Score a task with the REBA worksheet and interpret the result.
- Generate an ergonomic risk-assessment report with Inseer.
- Compare manual vs. automated assessment — advantages and limitations of each.

19.6.2 Prep work

- Review the introduction-to-ergonomics and REBA material (video links in the handout, Procedure topics 1–2).
- Skim **Cheatsheet - Ergonomics -REBA.pdf** and the **REBA-worksheet.pdf**.
- Form groups of 3–4.

19.6.3 Materials & equipment

- **REBA Employee Assessment Worksheet** (printed, one per student/group)
- **Inseer** software tool — web-based; account/login required
- Computer with internet access
- Sample tasks from an automated manufacturing facility (videos / photo sets):
 - Task 1 — lifting boxes repeatedly
 - Task 2 — computer-workstation setup
 - Task 3 — floor cleaning with a buffer

19.6.4 Procedure

1. **Intro to ergonomics** — watch the overview videos (importance; common risks and impacts).
2. **Overview of REBA** — watch how to use the worksheet.
3. **Task evaluation** — each group is assigned one of the three tasks. Observe and analyse it, note ergonomic risks (repetitive motion, awkward posture, heavy lifting, vibration, glare, chair height ...), score it with the REBA worksheet, and interpret the score.
4. **Intro to Inseer** — overview of the tool; input the task data and generate a risk-assessment report.
5. **Data & observations** — record REBA scores and Inseer results for the task; note insights.
6. **Analysis** — compare the manual REBA assessment with the Inseer report; discuss advantages and limitations of each.
7. **Conclusion** — reflect on the importance of ergonomics in automated manufacturing and the benefits of AI tools like Inseer.

19.6.5 Deliverables & submission

One group report containing: the procedure, your group's REBA analysis, your group's Inseer report, and your group's observations and conclusions. Submit on FOL by the end of the lab period.

19.6.6 Rubric

10 marks total.

Criterion	Marks
Procedure followed; all three assigned tasks assessed	1
REBA scoring — correct Table A/B/C lookups, coupling and activity adjustments, final score and risk band for each of the three tasks	4
Inseer report — assessment run, output correctly read and quoted	2
Manual vs automated comparison — quantified, references the Chapter 7 risk factors	2
Conclusion and recommended controls, ordered by the hierarchy of controls	1

19.6.7 Instructor notes

- **Prep:** set up **Inseer** access for the class *well ahead* — confirm the account type, seat count and whether students upload their own clips or use supplied ones. This is the lab’s dependency risk.
- **Print:** REBA worksheets + the REBA cheat sheet, one per student.
- Stage the three task video/photo sets on FOL (**Lifting Boxes.pdf**, **Workstation.pdf**, **Cleaning.pdf** and the matching clips).
- **Marking:** REBA scoring is where students lose marks — check the trunk/neck/leg and arm/wrist sub-scores and the coupling/activity adjustments, not just the final number.

19.7 Lab 6 — SPC and Gauge R&R: the Virtual Caliper

Before you ask an LLM to do statistical process control in Python (that is Lab 8), you do it yourself, by hand, as the measurement system. This lab runs entirely inside this page: **webR** cells render a **virtual vernier caliper** at a hidden, seeded true value, you read the scale and type the value, and the readings you produce carry *genuine human reading error against known ground truth* — so bias, linearity, repeatability and reproducibility are all computable, which a paper Gauge R&R cannot do. You then build an X/R chart and compute Cpk against the Précitech shaft spec of $\varnothing 25.000 \pm 0.050$ mm.

AI use: not permitted for the readings or the calculations — *you* are the measurement system being studied. You may use an LLM to check your written reflection only, with disclosure.

19.7.1 Learning outcomes

After this lab you should be able to:

- Read a vernier scale to its resolution and record a blinded, randomised measurement set.
- Run a range-method Gauge R&R: compute EV, AV, GRR, PV, TV, %GRR and the number of distinct categories (ndc), and judge gauge acceptability.
- Estimate gauge bias and linearity against certified reference values.
- Build an X/R control chart, apply the Western Electric rules, and compute Cpk against a two-sided specification.
- Explain how an inadequate measurement system distorts a capability index.

19.7.2 Prep work

- Read Chapter 10 (SPC — X/R charts, the Western Electric rules, Cpk) and Chapter 11 (MSA — bias, linearity, repeatability, reproducibility, the range method, ndc).
- Skim the worked webR cells in Chapter 11: **bias-calculation** and **range-method** — you reuse that method here.
- Nothing to install and no account to create. This lab needs **no external service** — if the book page loads, the lab works.
- Form groups of **exactly 3**. Each group member is one of the three operators in the Gauge R&R study.

19.7.3 Materials & equipment

- One computer per group with this book page open (a second screen helps — one person reads the caliper, one records)
- The webR cells in this section (the caliper, the Gauge R&R calculator, the X/R + Cpk cell)
- A spreadsheet or paper grid to hold 90 readings (10 parts $\times$ 3 operators $\times$ 3 trials) before you type them into the calculator cell
- The Précitech case study: shaft blanks, AISI 4140, $\varnothing 25.000 \pm 0.050$ mm

19.7.4 Procedure

webR runs top to bottom on one shared page environment.

Run the **setup cell first** and leave it run; every later cell depends on the objects it creates. If a later cell errors with “object not found”, re-run the setup cell. Base R graphics only — do not add `library(ggplot2)`; it is slow and fragile in webR and not needed here.

Task 1 — Calibrate your eye (10 min). Run the setup cell. Then run the demo cell three times with `demo = 1, 2, 3`; each prints the correct reading after

you have written your own. The caliper resolution is **0.05 mm**: read the last main-scale mark the vernier zero has passed, then find the vernier mark that best lines up with a main-scale mark and add that many hundredths. Zoom the browser if the coincidence is hard to see.

Task 2 — The measurement run (35 min). (*2 marks*) Each operator reads **all 10 parts, 3 trials each**, using `draw_caliper(part, trial)`. Read blind — do not look at another operator’s numbers, and do not look at your own previous trial. Work through the parts in the randomised order the setup cell prints for you, a different order per operator. Record all 90 readings in your grid.

Task 3 — Range-method Gauge R&R (20 min). (*3 marks*) Type your 90 readings into the `lab6-grr` cell (10 parts $\times$ 3 operators $\times$ 3 trials) and run it. It reports EV, AV, GRR, PV, TV, %GRR and `ndc` by the AIAG average-and-range method — the same method as Chapter 11’s `range-method` cell, extended to 3 operators and 3 trials. Record every output.

Task 4 — Bias and linearity (15 min). (*1 mark*) Run the `lab6-bias` cell. It compares each operator’s mean reading of each part against that part’s certified reference value (printed by the setup cell), reports overall bias, and fits bias vs. size to get linearity. State the bias in mm and whether it changes across the size range.

Task 5 — X/R chart and Cpk (25 min). (*3 marks*) Run the `lab6-spc` cell. It generates a **seeded 25-subgroup** production dataset (5 shafts per subgroup) against $\varnothing 25.000 \pm 0.050$ mm — the same for every group — draws the $\bar{X}$ and R charts, and computes Cpk. On the printed chart: list every subgroup that breaks a **Western Electric rule** (one point beyond 3 ; 2 of 3 beyond 2 ; 4 of 5 beyond 1 ; 8 in a row on one side) and name the rule. Report Cpk and say whether the process clears the automotive Tier-1 bar of 1.33.

Task 6 — Reflection (15 min). (*1 mark*) Answer in a few sentences: (1) Is your gauge acceptable? Use the AIAG bands — %GRR < 10 % acceptable, 10–30 % marginal, > 30 % unacceptable; `ndc` 5 required. (2) Your %GRR came from a study on parts spanning the process range. If the measurement system contributes the %GRR you found, what does that do to the Cpk you computed in Task 5 — is the true capability better or worse than the number? (3) Would you release shafts to the customer on this gauge?

```
#| label: lab6-setup
#   Lab 6 setup - RUN THIS FIRST
# Defines the virtual caliper and the hidden true values. No third-party
# packages: base R graphics only.

set.seed(3027)

# Certified reference value of each of the 10 study parts (what a CMM reports),
```

```

# chosen to span the process range around the Ø 25.000 mm nominal.
caliper_ref <- round(sort(runif(10, 24.90, 25.10)), 3)

# The caliper under study has a small, slightly size-dependent bias (mm).
caliper_bias <- 0.03 + 0.05 * (caliper_ref - 25.00)

# What the caliper *shows* for a given part and trial: reference + gauge bias +
# a small amount of mechanism/technique slop. Seeded per (part, trial) so the
# same call always draws the same scene.
caliper_shown <- function(part, trial) {
  set.seed(1000L * part + trial)
  caliper_ref[part] + caliper_bias[part] + rnorm(1, 0, 0.015)
}

# Draw a zoomed vernier caliper (resolution 0.05 mm) at the shown value.
draw_caliper <- function(part, trial) {
  stopifnot(part >= 1, part <= 10, trial >= 1, trial <= 3)
  shown <- caliper_shown(part, trial)
  lo <- floor(shown) - 4
  hi <- lo + 24
  op <- par(mar = c(1, 1, 3, 1)); on.exit(par(op))
  plot(NA, xlim = c(lo, hi), ylim = c(3.5, 9), axes = FALSE, xlab = "", ylab = "",
       main = sprintf("draw_caliper(part = %d, trial = %d)", part, trial))
  # fixed main scale (mm), ticks every 1 mm, labels every 5 mm
  for (m in lo:hi) {
    big <- (m %% 5 == 0)
    lines(c(m, m), c(7, if (big) 6.1 else 6.6), lwd = if (big) 2 else 1)
    if (big) text(m, 7.5, labels = m, cex = 0.9)
  }
  lines(c(lo, hi), c(7, 7), lwd = 2)
  text(lo, 8.6, "main scale - millimetres", pos = 4, cex = 0.9, font = 3)
  # sliding vernier: 20 divisions spanning 19 mm -> each 0.95 mm, resolution 0.05
  v0 <- shown
  for (k in 0:20) {
    xk <- v0 + k * 0.95
    big <- (k %% 5 == 0)
    lines(c(xk, xk), c(5.9, if (big) 5.0 else 5.5), lwd = if (big) 2 else 1)
    if (big) text(xk, 4.7, labels = sprintf("%.2f", k * 0.05), cex = 0.8)
  }
  lines(c(v0, v0 + 19), c(5.9, 5.9), lwd = 2)
  polygon(c(v0 - 0.22, v0 + 0.22, v0), c(6.0, 6.0, 6.5), col = "black")
  text(v0, 4.2, "vernier 0", cex = 0.8, font = 3)
  invisible(NULL)
}

```

```

# Randomised part order, one per operator (A, B, C).
operator_order <- lapply(1:3, function(i) { set.seed(70 + i); sample(1:10) })
names(operator_order) <- c("A", "B", "C")

cat("Setup complete.\n\n")
cat("Randomised part order:\n")
for (nm in names(operator_order))
  cat("  Operator", nm, ":", paste(operator_order[[nm]], collapse = " "), "\n")
cat("\nUse draw_caliper(part, trial) with part 1..10 and trial 1..3.\n")

#| label: lab6-demo
# Practice reads - set demo to 1, 2 or 3, run, write your reading, then check.
demo <- 1

practice_parts <- c(2, 6, 9)
p <- practice_parts[demo]
draw_caliper(part = p, trial = 1)
cat("Write your reading (mm) before running the next line.\n")
cat("Correct reading to 0.05 mm:",
     format(round(caliper_shown(p, 1) / 0.05) * 0.05, nsmall = 2), "mm\n")

#| label: lab6-grr
# Range-method Gauge R&R - 10 parts, 3 operators, 3 trials
# Replace the numbers below with YOUR 90 readings. Each operator matrix is
# 10 rows (parts 1..10) x 3 columns (trials 1..3).

op_A <- matrix(c(
# trial1 trial2 trial3
  25.00, 25.05, 25.00, # part 1
  24.95, 24.95, 25.00, # part 2
  24.90, 24.95, 24.90, # part 3
  25.10, 25.05, 25.10, # part 4
  24.85, 24.90, 24.85, # part 5
  25.20, 25.15, 25.20, # part 6
  24.80, 24.85, 24.80, # part 7
  25.15, 25.15, 25.10, # part 8
  24.75, 24.80, 24.80, # part 9
  25.25, 25.20, 25.25 # part 10
), ncol = 3, byrow = TRUE)

op_B <- op_A + 0.01 # <-- replace with Operator B's real readings
op_C <- op_A - 0.01 # <-- replace with Operator C's real readings

trials <- 3
operators <- 3

```

```

parts      <- 10

# AIAG average-and-range constants
K1 <- 0.5908          # trials = 3    (repeatability)
K2 <- 0.5231          # operators = 3 (reproducibility)
K3 <- 0.3146          # parts = 10   (part variation)

ops <- list(A = op_A, B = op_B, C = op_C)

# Repeatability (EV): mean within-operator, within-part range
part_op_range <- sapply(ops, function(M) apply(M, 1, function(x) max(x) - min(x)))
Rbar <- mean(part_op_range)
EV   <- Rbar * K1

# Reproducibility (AV): range of the operator grand means, corrected for EV
op_means <- sapply(ops, mean)
Xdifff   <- max(op_means) - min(op_means)
AV        <- sqrt(max(0, (Xdifff * K2)^2 - (EV^2 / (parts * trials))))

# Gauge R&R
GRR <- sqrt(EV^2 + AV^2)

# Part variation (PV): range of the part averages across all operators & trials
part_means <- rowMeans(sapply(ops, function(M) rowMeans(M)))
Rp <- max(part_means) - min(part_means)
PV <- Rp * K3

TV   <- sqrt(GRR^2 + PV^2)
pGRR <- 100 * GRR / TV
ndc  <- floor(1.41 * PV / GRR)

cat(sprintf("EV   (repeatability)   = %.4f mm\n", EV))
cat(sprintf("AV   (reproducibility) = %.4f mm\n", AV))
cat(sprintf("GRR                      = %.4f mm\n", GRR))
cat(sprintf("PV   (part variation)   = %.4f mm\n", PV))
cat(sprintf("TV   (total variation)  = %.4f mm\n", TV))
cat(sprintf("%GRR                      = %.1f %%\n", pGRR))
cat(sprintf("ndc                      = %d\n", ndc))
cat("\nAIAG: %GRR < 10 acceptable | 10-30 marginal | > 30 unacceptable; ndc >= 5.\n")

#| label: lab6-bias
#   Bias and linearity vs certified reference values
# Uses op_A / op_B / op_C from the lab6-grr cell and caliper_ref from the setup cell.

ops <- list(A = op_A, B = op_B, C = op_C)

```

```

obs_part_mean <- rowMeans(sapply(ops, function(M) rowMeans(M))) # 10 part means
bias_by_part <- obs_part_mean - caliper_ref

cat("Part | reference | observed mean | bias (mm)\n")
for (i in 1:10)
  cat(sprintf("%4d | %9.3f | %13.3f | %7.3f\n",
              i, caliper_ref[i], obs_part_mean[i], bias_by_part[i]))

cat(sprintf("\nOverall bias = %+.4f mm\n", mean(bias_by_part)))

lin <- lm(bias_by_part ~ caliper_ref)
slope <- coef(lin)[2]
cat(sprintf("Linearity slope = %+.4f mm of bias per mm of size\n", slope))
cat(sprintf("Linearity over the %.2f mm study range = %.4f mm\n",
            diff(range(caliper_ref)), abs(slope) * diff(range(caliper_ref))))

#| label: lab6-spc
# X-bar / R chart and Cpk - seeded 25-subgroup production dataset
# Ø 25.000 +/- 0.050 mm. Same data for every group.

set.seed(250)
n_sub <- 25; n <- 5
USL <- 25.050; LSL <- 24.950; nominal <- 25.000

mu_series <- rep(25.002, n_sub)
mu_series[12:16] <- mu_series[12:16] + 0.030 # a real special-cause shift
x <- t(sapply(mu_series, function(m) rnorm(n, m, 0.011)))

xbar <- rowMeans(x)
R <- apply(x, 1, function(v) max(v) - min(v))
xbarbar <- mean(xbar); Rbar <- mean(R)

A2 <- 0.577; D3 <- 0; D4 <- 2.114; d2 <- 2.326 # n = 5
UCL_x <- xbarbar + A2 * Rbar; LCL_x <- xbarbar - A2 * Rbar
UCL_R <- D4 * Rbar; LCL_R <- D3 * Rbar

par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))
plot(xbar, type = "b", pch = 19, ylim = range(xbar, UCL_x, LCL_x),
     xlab = "Subgroup", ylab = "X-bar (mm)", main = "X-bar chart")
abline(h = c(UCL_x, xbarbar, LCL_x), lty = c(2, 1, 2), col = c("red", "black", "red"))
plot(R, type = "b", pch = 19, ylim = range(0, R, UCL_R),
     xlab = "Subgroup", ylab = "R (mm)", main = "R chart")
abline(h = c(UCL_R, Rbar, LCL_R), lty = c(2, 1, 2), col = c("red", "black", "red"))
par(mfrow = c(1, 1))

```



```

sigma_hat <- Rbar / d2
Cp <- (USL - LSL) / (6 * sigma_hat)
Cpk <- min(USL - xbarbar, xbarbar - LSL) / (3 * sigma_hat)

cat(sprintf("X-bar-bar = %.4f mm   R-bar = %.4f mm   sigma_hat = %.4f mm\n",
           xbarbar, Rbar, sigma_hat))
cat(sprintf("X-bar control limits: [%.4f, %.4f]\n", LCL_x, UCL_x))
cat(sprintf("Cp = %.2f   Cpk = %.2f   (automotive Tier-1 target: Cpk >= 1.33)\n", Cp, Cpk))
cat("\nBeyond-3-sigma subgroups:",
    paste(which(xbar > UCL_x | xbar < LCL_x), collapse = " "), "\n")
cat("Apply the remaining Western Electric rules by eye on the chart above.\n")

```

19.7.5 Deliverables & submission

One group submission (PDF or notebook export) containing: the 90-reading grid; the Gauge R&R cell output with EV, AV, GRR, PV, TV, %GRR and ndc; the bias and linearity output; the X/R chart with the Western Electric rule breaks listed and named, and the Cpk; and the Task 6 reflection. Submit on FOL by the end of the lab period.

19.7.6 Rubric

10 marks total.

Deliverable	Marks
Task 2 — 90 readings, blinded and in the randomised order, all 10 parts × 3 trials per operator	2
Task 3 — Gauge R&R computed: EV, AV, GRR, PV, TV, %GRR and ndc all reported and internally consistent	3
Task 4 — bias in mm and a linearity statement against the reference values	1
Task 5 — X/R chart read correctly, every Western Electric rule break listed and named, Cpk reported and compared to 1.33	3
Task 6 — acceptability judged against the %GRR and ndc bands; effect of gauge error on Cpk reasoned through; release decision stated	1

19.7.7 Instructor notes

- **Prep:** none beyond opening the rendered page and checking the webR cells load. This lab has **no external dependency** — no site, no account, no dataset file — which is deliberate: it removes the single-point-of-failure risk the browser labs otherwise carry (see Lab 1’s notes).
- Run `Rscript R/make_lab_data.R` to print the Lab 6 answer key: the `caliper_ref` values, the expected overall bias (-0.03 mm), and the expected %GRR / ndc / Cpk ranges. The caliper and SPC data are generated inside the webR cells, so there is no file to stage.
- Groups must be **exactly 3** — the study needs three operators. An absent member means the group runs 2 operators and notes it; do not let a group of 4 double up.
- **Marking:** the readings must show real spread between trials and operators — a grid where every operator recorded identical values was not read blind. Check that %GRR and ndc are consistent with the EV/AV/PV they reported.
- This lab is the “do it yourself first” half of the arc into Lab 8 (SPC in Python, directed by an LLM). Point that forward explicitly.

19.8 Machine-learning module: Labs 7 to 12

Labs 7–12 are the laboratory component of Chapter Chapter 13, running in course weeks 7 to 12. Each week has two hours of classroom theory (Chapter Chapter 13) followed by a two-hour lab. The six sessions build on each other: by the end of Lab 11 you will have a working predictive-maintenance web application at a public URL, and Lab 12 is the live demo of it.

Lab	Course week	Module week	Focus	What you submit
7	7	1	AI literacy: prompt engineering, hallucination hunting, Python code generation	<code>prompts.md</code> + <code>spc_starter.py</code>
8	8	2	Manufacturing data: cleaning, SPC, Cpk on real plant data	Completed <code>week2_lab.ipynb</code>

Lab	Course week	Module week	Focus	What you submit
9	9	3	Capstone kickoff: understanding the C-MAPSS dataset	Notebook with sensor plots and reflection
10	10	4	Capstone build: feature engineering, model training, evaluation	Notebook + <code>rul_model.pkl</code> + <code>feature_cols.pkl</code>
11	11	5	Capstone deploy: Streamlit app on the public internet	Public URL submitted via FOL
12	12	6	Capstone presentations: live demo to the class	Presentation (no file submission)

Tools used across Labs 7–12:

- **Google Colab** (colab.research.google.com) — free, no local installation required
- **Claude.ai or ChatGPT** (free tier) — for the Week 1–2 exercises
- **GitHub** (free account) — required from Week 5; optional earlier
- **Streamlit Community Cloud** (free) — required for the Week 5 deployment

A note on using AI assistants in these labs: you are explicitly encouraged to use LLMs to help you write, debug, and understand code throughout this module. However, you must be able to explain every line you submit. If the instructor asks “why does this `groupby().transform()` reset the rolling window at each engine boundary?” and you cannot answer, that line does not count as your work — regardless of whether it runs correctly.

i The course narrative — Précitech Components

You are a new process engineer at Précitech Components, a Tier-1 automotive supplier in Granby, Québec (the same plant documented in Labs 2–3). The plant runs CNC turning cells producing transmission shaft blanks for a major OEM customer; your cell produces roughly 250 parts per day.

Your plant manager, **Sophie Tran**, has seen what GE and Toyota are doing with predictive maintenance and wants to know whether Précitech can benefit too — without leaking IP to public tools, without trusting unverified outputs, and without a million-dollar infrastructure investment. She has assigned you to:

1. Learn enough about AI tools to use them safely and productively for daily engineering work (Weeks 1–2).
2. Build a proof-of-concept predictive-maintenance system using publicly available data and free tools (Weeks 3–5).
3. Present your findings and the deployed application to plant leadership (Week 6).

By the end of Week 5 you will have a working web application at a public URL. It will not yet work on Précitech’s actual machines — you used a NASA simulation dataset, not plant data — but it will demonstrate the methodology, code quality and user interface well enough for the plant manager to decide on next steps.

19.9 Lab 7 — Machine Learning, Week 1: AI Literacy and Prompt Engineering

The first lab of the machine-learning module, in course week 7 (the midterm is written in lecture time that week, so this lab period runs as normal). You compare how an LLM responds to poorly vs. well-structured prompts, deliberately trigger and document a hallucination, and use an LLM to generate the Python code you will build on in Lab 8. Set in the Précitech narrative above.

AI use: required — you are assessed on directing an LLM well, not on writing Python.

19.9.1 Learning outcomes

- How an LLM responds to poorly structured vs. well-structured prompts.
- How to detect and document an LLM hallucination.
- How to generate Python code using an LLM and evaluate whether the output is trustworthy.

19.9.2 Prep work

- Read Chapter Chapter 13, **Week 1** (the AI landscape; how LLMs work; why they fail; IP risk; the R-C-T-C-F prompt pattern).
- Create a free account on Claude.ai or chat.openai.com (free tier is fine).
- Create a text file `prompts.md` in your course folder — you add to it throughout the session and submit it at the end.

19.9.3 Materials & equipment

- Computer with internet access — one per student (individual lab)
- An LLM chat account (Claude.ai or ChatGPT, free tier)
- `prompts.md` (you create it)
- No dataset or local install needed this week

19.9.4 Procedure

Exercise 1 — Bad prompt → good prompt (30 min). Five real prompts from manufacturing engineers, all of which produced poor results:

#	Original prompt	Why it fails
1	“explain SPC”	No audience, no purpose, no format
2	“what’s wrong with my process”	No context — the LLM will guess
3	“write code to plot data”	Which language? what data? what plot?
4	“is 1.33 a good Cpk”	The answer depends on the industry — the LLM needs that context
5	“summarize this FMEA” (then paste 30 pages)	No audience, no focus, no length cap

Rewrite each using the **R-C-T-C-F** pattern (Role, Context, Task, Constraints, Format), run the original and your rewrite, and compare. In `prompts.md` record: the original; your R-C-T-C-F rewrite (label R=, C=, T=, C=, F=); a two-sentence reflection on what changed.

Exercise 2 — FMEA summarisation at three audiences (25 min). Using the CNC Cell 3 FMEA in Chapter Chapter 13 (Week 1), craft and run three prompts that summarise it for (a) plant manager Sophie Tran — top business risks in one paragraph; (b) a quality engineer — top 5 risks by RPN with mitigation status; (c) a new operator — a plain-language “what to watch for” checklist. Compare the three outputs with a partner. Add the three prompts and a two-sentence reflection to `prompts.md`.

Exercise 3 — The hallucination hunt (20 min). Deliberately get the LLM to fabricate something and then verify it is fabricated — e.g. ask about a made-up ASTM standard number, made-up paper authors, or a made-up alloy. When it answers confidently, record in `prompts.md`: the exact prompt; the exact response (copy-paste); how you verified it was false; one sentence on how you would prevent this in your own workflow.

 Warning

After you have personally caught an LLM confidently fabricating a citation, you will be far more careful about verifying LLM outputs in professional work. This one behaviour change is worth the entire week.

Exercise 4 — LLM-assisted Python starter code (30 min). Use the LLM to generate Python that, on a file `spc_data.csv` with columns `timestamp`, `operator`, `part_id`, `diameter_mm`, `surface_roughness_um`, `cycle_time_sec`, `tool_age_cycles`: loads the CSV and parses `timestamp` as datetime; plots `diameter_mm` over time with title and axis labels; prints the mean and standard deviation of `diameter_mm`; flags rows more than 3 from the mean; and does not crash on missing values. Constrain the prompt: pandas + matplotlib only; a comment above each logical block; runnable in Google Colab with no local paths; standard Colab libraries only. **Read every line and make sure you can explain it** — the instructor will ask in Lab 8. Save it as `spc_starter.py`.

19.9.5 Deliverables & submission

Submit via FOL by end of session:

- `prompts.md` — before/after prompt pairs (Exercises 1–2) and the hallucination documentation (Exercise 3)
- `spc_starter.py` — the Python code from Exercise 4

19.9.6 Rubric

10 marks total.

Criterion	Marks
<code>prompts.md</code> — five prompts rewritten with all five R-C-T-C-F elements present	3
FMEA summarised at three audiences — audience-appropriate and technically accurate	2

Criterion	Marks
Hallucination hunt — a fabrication documented, with evidence of how it was verified	3
<code>spc_starter.py</code> runs, is commented, and the student can explain it on request	2

19.9.7 Instructor notes

- **Prep:** confirm Claude.ai / ChatGPT and colab.research.google.com are reachable on the lab network. Post the FMEA table (Chapter Chapter 13, Week 1) and a blank `prompts.md` on FOL.
- No dataset needed this week; `spc_data.csv` is staged for Lab 8.
- **Marking:** Exercise 3 is the point of the week — require genuine verification (a screenshot of the empty ASTM search, a Google Scholar “no results”), not just “I think this is made up”.
- The capstone (Labs 9–12) is assessed together against the capstone rubric in Lab 12; weights live in FOL and the instructor handbook, not here.

19.10 Lab 8 — Machine Learning, Week 2: Manufacturing Data and SPC

You run your Lab 7 Python on real (messy) plant data — 90 days of production from Précitech CNC Cell 3, with intentional data-quality problems embedded — find and document each problem without silently deleting data, then build an X-bar control chart and compute Cpk. This is the SPC you did by hand in Lab 6, now directed through an LLM.

AI use: required — you direct an LLM to write the Python and must be able to explain every line.

19.10.1 Learning outcomes

- How LLM-generated code behaves on real, messy industrial data.
- How to find and document data-quality problems without silent data deletion.
- How to build an X-bar control chart in Python and identify out-of-control events.

19.10.2 Prep work

- Read Chapter Chapter 13, **Week 2** (where plant data comes from; why industrial data is messy; statistical process control; where SPC fails).
- Have your `spc_starter.py` from Lab 7 ready.
- Open Google Colab and check you can upload files.

19.10.3 Materials & equipment

- Computer with internet access; Google Colab
- `spc_data.csv` (90 days of CNC Cell 3 data, with embedded problems) — from FOL
- `week2_lab.ipynb` — pre-structured lab notebook, from FOL
- Your `spc_starter.py` from Lab 7

19.10.4 Procedure

Task 1 — Run your Week 1 code on real data (25 min). Paste `spc_starter.py` into the first cell of `week2_lab.ipynb` and run it. Expect: a spike so large it compresses the chart; your 3rd detector flagging the wrong rows; possibly a warning. In the first markdown cell, describe the spike (approximate value), whether outlier detection worked, and any error encountered.

Task 2 — Find and fix data-quality issues (35 min). Every change must be recorded in a quality-log DataFrame or markdown cell — **never silently delete data** (ISO 9001 traceability). Log pattern:

```
# Pattern for quality logging - add one row per correction
quality_log = []
quality_log.append({
    "row_index": 42,
    "column":    "diameter_mm",
    "original":  250.123,
    "corrected": 25.0123,
    "reason":    "Missing decimal point - impossible on Mazak QT-200 (max bar stock 51
    "action":    "Divided by 10; rechecked against neighbouring rows",
})
```

Problems to find: two missing days (timestamp gap — document as a network outage, do **not** interpolate); one impossible diameter (> 30 mm — decimal correction); a block of rows with roughness in inches not μm (convert: $\times 25400$); one negative cycle time (flag and remove); and a real special-cause cluster around days 43–47 (**keep it** — document that it is a genuine event). After cleaning, verify no diameters > 30 mm remain, no negative cycle times remain, and the quality log is populated.

Task 3 — Build an X-bar control chart (35 min). Using the cleaned data, build an X-bar chart for `diameter_mm` (subgroup = 5 parts/day). Control

limits use `sigma / sqrt(n)`, **not** the standard deviation of the daily means. Spec limits USL = 25.050, LSL = 24.950. Identify which days are out of control, cross-reference them with the `operator` and `tool_age_cycles` columns, and compute Cpk using Equation 13.4 from Chapter Chapter 13:

```
mu    = df_clean["diameter_mm"].mean()
sigma = df_clean["diameter_mm"].std()
cpk    = min((usl - mu) / (3 * sigma), (mu - lsl) / (3 * sigma))
print(f"Cpk = {cpk:.3f}")
```

Expected: Cpk 1.1–1.2 (marginal — below the automotive Tier-1 standard of 1.33), with the out-of-control excursion around days 43–47.

Task 4 — Reflection (15 min). In a markdown cell, answer briefly: (1) how many parts had already been made during the days 43–47 event before the first out-of-control point appeared? (2) would a model predicting tomorrow’s mean diameter from today’s tool age, operator and the last 5 days of data have given earlier warning than SPC? (3) what data, not in this CSV, would you want to build a *predictive* rather than *reactive* model?

19.10.5 Deliverables & submission

Submit your completed `week2_lab.ipynb` (all cells executed, quality log populated, chart generated, Cpk computed, reflection answered) via FOL by end of session.

19.10.6 Rubric

10 marks total.

Criterion	Marks
Week 1 code runs against the real dataset	1
All five data-quality issues found; <code>quality_log</code> complete (row_index, column, original, corrected, reason, action); the days 43–47 special cause correctly kept, not deleted	4
X-bar chart with correct control limits — <code>sigma / sqrt(n)</code> , not the std of daily means	3
Cpk against USL 25.050 / LSL 24.950, in the expected 1.1–1.2 range and interpreted	1
Reflection	1

19.10.7 Instructor notes

- **Prep:** stage `spc_data.csv` and `week2_lab.ipynb` on FOL. Keep a key of the embedded problems (row indices, correct fixes) for marking.
 - The most common error is using the standard deviation of the daily means for the control limits instead of `sigma_individual / sqrt(n)` — check this explicitly.
 - Follow up on a `spc_starter.py` line from Lab 7: ask a student to explain it.
 - **Marking:** deduct for silent row deletion even if the final chart looks right — the traceable quality log is the learning objective.
-

19.11 Lab 9 — Machine Learning, Week 3: The C-MAPSS Dataset

The capstone kickoff. You load NASA’s C-MAPSS turbofan run-to-failure data, compute the supervised-learning label (Remaining Useful Life, RUL) from the raw records, and identify which of the 21 sensors actually carry predictive information.

AI use: required — you direct an LLM to write the Python and must be able to explain every line.

19.11.1 Learning outcomes

- How to load and explore industrial time-series run-to-failure data.
- How to compute a supervised-learning label (RUL) from raw run-to-failure records.
- How to identify which sensors carry predictive information.

19.11.2 Prep work

- Read Chapter Chapter 13, **Week 3** (the economics of maintenance; the P-F curve; the C-MAPSS dataset; the ML problem; the fatal mistake of row-wise splitting; Random Forest intuition; evaluation).
- Get `train_FD001.txt` from the instructor’s FOL folder (or the Kaggle mirror, free account).
- Open Google Colab.

19.11.3 Materials & equipment

- Computer with internet access; Google Colab
- `train_FD001.txt` — NASA C-MAPSS FD001 training set, from FOL

- `predictive_maintenance.ipynb` — capstone notebook, pre-structured with starter code for Weeks 3–5, from FOL

19.11.4 Procedure

Task 1 — Load and label the data (20 min). The file has no headers and is space-separated: 26 columns = unit number, time in cycles, 3 operating settings, 21 sensors, and 2 trailing empty columns to drop. Expected: 20 631 rows, 100 engines. Then create the RUL label — there is no RUL column, you compute it:

```
# For each engine, the max cycle is the failure cycle
max_cycles = train.groupby("unit_number")["time_in_cycles"].max()
# RUL at any row = that engine's failure cycle minus the current cycle
train["RUL"] = train["unit_number"].map(max_cycles) - train["time_in_cycles"]
```

Task 2 — Visualise one engine's life (30 min). Plot all 21 sensors for engine #1 over its whole life (a 7×3 grid). Classify sensors as flat lines (never change — useless), noise without trend, or clear trends (drift systematically toward failure — these are the useful ones). Write your lists in a markdown cell and compare with a classmate.

Task 3 — Identify constant sensors (20 min). Compute each sensor's standard deviation across the whole dataset; sensors with `std < 0.001` are effectively constant and carry no information. Expected: about 6 constant sensors in FD001 (`sensor_1`, 5, 10, 16, 18, 19).

Before deleting a near-constant sensor in a real plant: what would you investigate first, and what are three possible reasons a sensor might read constant? Are all three equally acceptable to engineering leadership?

Task 4 — Correlation with RUL (20 min). For the non-constant sensors, compute the Pearson correlation between each sensor and RUL; $|r| > 0.5$ marks the most predictive raw features. Expected top sensors: `sensor_11` ($r = -0.71$), `sensor_4` ($r = -0.69$), `sensor_12` ($r = +0.67$), `sensor_7` ($r = +0.66$).

Task 5 — Reflection (15 min). In a markdown cell: (1) if you plot RUL for all 20 631 rows in order, what shape appears and why — and what does that shape say about the danger of splitting rows randomly between train and validation? (2) which 3 sensors would you most want in a model, citing evidence from Task 2 or 4? (3) what would you do before dropping the 6 dead sensors in a real plant?

19.11.5 Deliverables & submission

Submit `predictive_maintenance.ipynb` with all four code tasks executed, the 7×3 sensor plot included, and the reflection answers written. Export as `.ipynb` and submit via FOL by end of session.

19.11.6 Rubric

Weeks 3–6 (Labs 9–12) are assessed together against the **capstone rubric in Lab 12**. For this session the instructor confirms progress: data loaded and labelled, sensor plot produced, constant sensors identified, correlations computed, reflection answered.

19.11.7 Instructor notes

- **Prep:** stage `train_FD001.txt` and `predictive_maintenance.ipynb` on FOL (host the dataset yourself so it does not depend on Kaggle sign-in during the lab). Confirm Colab is reachable.
 - The “sawtooth” reflection question (Task 5.1) sets up the engine-wise split in Lab 10 — make sure students actually answer it.
 - **Marking:** rolled into the capstone rubric; here just verify the notebook runs end to end and the reflection is attempted.
-

19.12 Lab 10 — Machine Learning, Week 4: Building the Predictive-Maintenance Model

The capstone build. You engineer rolling-window features per engine, perform a correct engine-wise train/validation split, train a Random Forest to predict RUL, evaluate it against a naive baseline, and save the model for deployment.

AI use: required — you direct an LLM to write the Python and must be able to explain every line.

19.12.1 Learning outcomes

- How to engineer rolling-window features per engine without data leakage.
- How to perform a correct engine-wise train/validation split and verify it is clean.
- How to train, evaluate and compare a Random Forest against a naive baseline.
- How to save a trained model for deployment.

19.12.2 Prep work

- Read Chapter Chapter 13, **Week 4** (feature engineering for time-series; Random Forest internals; evaluation done right).
- Have your `predictive_maintenance.ipynb` from Lab 9 with the data loaded, RUL computed, and the useful-sensor list established.

19.12.3 Materials & equipment

- Computer with internet access; Google Colab
- Your `predictive_maintenance.ipynb` from Lab 9 (continue in the same notebook)
- Python packages available in a standard Colab environment (`pandas`, `numpy`, `scikit-learn`, `matplotlib`, `joblib`)

19.12.4 Procedure

Task 1 — Rolling features (25 min). For each useful sensor, compute a rolling mean and rolling standard deviation over a 5-cycle window, **grouped by engine** so the window resets at each engine boundary. Verify the boundary: engine #2's first-cycle rolling mean must equal its raw value and must not include any of engine #1's data.

Task 2 — Clip RUL at 125 (10 min). `train_fe["RUL_clipped"] = train_fe["RUL"].clip(upper=125)` — anything above 125 cycles is “basically healthy”. Plot the before/after histogram.

Task 3 — Engine-wise train/validation split (15 min). Hold out 20 engines for validation. **Do not split rows randomly.** End with an assert that must pass before you train:

```
overlap = set(train_data["unit_number"]) & set(val_data["unit_number"])
assert len(overlap) == 0, f"DATA LEAK: engines in both splits: {overlap}"
```

Build `X_train` / `y_train` / `X_val` / `y_val`, excluding non-feature columns (`unit_number`, `time_in_cycles`, `RUL`, `RUL_clipped`, the constant sensors, and the mostly-constant `op_setting_*`).

Task 4 — Train the Random Forest (20 min). `RandomForestRegressor(n_estimators=100, max_depth=10, n_jobs=-1, random_state=42); model.fit(X_train, y_train)`. Training takes 30–90 s in Colab.

Task 5 — Evaluate against the naive baseline (20 min). Compute validation RMSE and MAE. Compare with a naive baseline that predicts `y_train.mean()` for every row. **Target: RMSE 20–25 cycles, at least 2× better than naive.** If RMSE is suspiciously low (< 10 cycles) you almost certainly have a data leak — go back to Task 3. Produce a predicted-vs-actual scatter plot and describe the pattern (are predictions pulled toward the centre? more error at high or low RUL?).

Task 6 — Feature importance (15 min). Plot the top 15 features by mean-decrease-in-impurity. In a markdown cell compare with your Week 3 correlation analysis: are the top features raw or rolling? do they match the top-correlation sensors? what might explain a feature you expected but do not see?

Task 7 — Save the model (15 min). Save **both** files — the app needs the feature names and order:

```
import joblib
joblib.dump(model, "rul_model.pkl")
joblib.dump(feature_cols, "feature_cols.pkl")
```

Download both from the Colab file browser; you need them in your GitHub repo for Lab 11.

19.12.5 Deliverables & submission

Submit via FOL: your completed `predictive_maintenance.ipynb` with all cells executed, plus `rul_model.pkl` and `feature_cols.pkl` (uploaded as files, or confirmed already in your GitHub repo).

19.12.6 Rubric

Assessed against the **capstone rubric in Lab 12** (Working Predictive Model, 30 points; Code Quality, 20 points). The single biggest scoring risk is using scikit-learn's `train_test_split` instead of a manual engine-wise split — the symptom is an RMSE of 5–8 cycles. If that happened, fix it: understanding and correcting the mistake matters more than a misleading score.

19.12.7 Instructor notes

- **Prep:** none beyond Lab 9 — same notebook, same data. Have a reference solution notebook with the expected RMSE (~21 cycles) and feature-importance chart.
- **The teaching moment:** circulate during Task 5 and look for RMSE < 10 — that is the row-wise-split data leak. Make students diagnose it themselves via the Task 3 assert.
- **Marking:** rolled into the capstone rubric.

19.13 Lab 11 — Machine Learning, Week 5: Deploying the App

You wrap the trained model in a Streamlit web application, push it to GitHub, and deploy it to Streamlit Community Cloud so it is live at a public URL — the proof-of-concept deliverable for Sophie Tran. **Lab 14 (the take-home RoboDK work cell) is released this week** — read its brief now so you can plan the build.

AI use: required — you direct an LLM to write the Python and must be able to explain every line.

19.13.1 Learning outcomes

- How to wrap a trained ML model in a user-facing web application.
- How to deploy that application to the public internet for free.
- The difference between “works on my machine” and “deployed and accessible to others”.

19.13.2 Prep work

- Read Chapter Chapter 13, **Week 5** (the OT/IT boundary; dashboards and operators; model drift and the production AI lifecycle).
- Create a free **GitHub** account and a new **public** repo named `rul-predictor`.
- Create a **Streamlit Community Cloud** account at `share.streamlit.io` using your GitHub login.
- Have `rul_model.pkl` and `feature_cols.pkl` from Lab 10 on your machine.

19.13.3 Materials & equipment

- Computer with internet access
- GitHub account + `rul-predictor` public repo
- Streamlit Community Cloud account
- `streamlit_app.py` starter template (with TODO markers), from FOL
- `rul_model.pkl`, `feature_cols.pkl` from Lab 10
- A sample one-engine CSV (`sample_engine.csv`) — last 30 cycles of engine #1

19.13.4 Procedure

Task 1 — Run the Streamlit app locally (40 min). The `streamlit_app.py` template already has the page config, cached model loading, single-engine rolling features, CSV upload, colour-coded RUL display (red < 30, orange < 60, green > 60 cycles), a sensor-trend plot, and an “About this model” expander. Complete the TODOs: add your name and course to the caption; build `sample_engine.csv` and wire up the “Use sample data” button; fill in the “About this model” expander (your name, your actual Lab 10 RMSE, one Précitech-specific limitation). Run with `streamlit run streamlit_app.py` (or in Colab via `pyngrok` for a temporary public URL) and test a CSV upload.

Task 2 — Push to GitHub (15 min). Repo contents: `streamlit_app.py`, `rul_model.pkl`, `feature_cols.pkl`, `requirements.txt`, `sample_engine.csv`. Use tested pinned versions in `requirements.txt` (`streamlit==1.39.0`, `scikit-learn==1.5.2`, `pandas==2.2.3`, `numpy==1.26.4`, `matplotlib==3.9.2`, `joblib==1.4.2`). Push via the GitHub web UI (drag-and-drop) or the command line.

Task 3 — Deploy to Streamlit Community Cloud (20 min). At `share.streamlit.io` → **New app** → select your repo → main file `streamlit_app.py` → **Deploy**. Deployment takes 2–3 min. Common failures: `rul_model.pkl` > 50 MB (retrain with `n_estimators=50`); version conflict in `requirements.txt`.

Task 4 — Polish the app (30 min). Plain-language title and description a plant manager would understand; working sample-data button; complete “About this model” expander (training data = NASA C-MAPSS FD001 simulation, model = RF 100 trees / max depth 10, your validation RMSE, an explicit statement of what the model is **not** for); graceful handling of a wrong-column CSV.

Task 5 — Test with a classmate (15 min). Swap URLs. Try a valid CSV, an invalid CSV (wrong/extra columns, missing values), and edge cases (1-row, 1000-row). Record one bug you find in their app and one they find in yours (you need not fix them before submission).

19.13.5 Deliverables & submission

Submit via FOL: your public Streamlit URL; a screenshot of the app showing a prediction; and the public `rul-predictor` GitHub repository URL.

19.13.6 Rubric

Assessed against the **capstone rubric in Lab 12** (Streamlit Application, 25 points; Documentation & Honest Disclosure, 15 points).

19.13.7 Instructor notes

- **Prep:** stage `streamlit_app.py` and a known-good `requirements.txt` on FOL. Walk through one full deploy yourself the week before — Streamlit Cloud and GitHub UIs change. Confirm `github.com` and `streamlit.io` are reachable on the lab network.
- Have students do the GitHub / Streamlit account creation as prep, not in the lab — it eats 20 min otherwise.
- **Marking:** rolled into the capstone rubric; the deliverable is a URL that loads and predicts.

19.14 Lab 12 — Machine Learning, Week 6: Capstone Presentations

Each student demos their deployed predictive-maintenance app live to the class and defends one technical decision — the “present to plant leadership” step

of the Précitech assignment. This session carries the capstone rubric for Labs 9–12.

AI use: required across the module — but the presentation and defence must be your own.

19.14.1 Learning outcomes

- Present a working technical deliverable to a non-specialist audience.
- Explain and defend a modelling decision under questioning.
- State the limitations of your own model honestly and specifically.

19.14.2 Prep work

- Read Chapter Chapter 13, **Week 6** (AI ethics in manufacturing; the broader AI landscape; where this is going).
- Confirm your app loads from its public URL and the sample CSV produces a sensible prediction.
- Prepare an 8–10 minute walk-through (no slides required — the app is the presentation) and be ready for 3–5 minutes of questions.

19.14.3 Materials & equipment

- Computer + projector/display; internet access
- Your deployed Streamlit app (public URL) and `sample_engine.csv`
- Your Lab 10 notebook (for the RMSE / feature-importance figures if asked)

19.14.4 Procedure

Format: 8–10 minutes presenting, 3–5 minutes of questions from the class and instructor.

What to demo:

1. Open your app live from the public URL; show it loads and works.
2. Upload the sample CSV; show a prediction; explain the colour coding and how a maintenance planner would act on it.
3. Explain **one** key technical decision — engine-wise vs. row-wise split (and the RMSE you would have got the wrong way); why RUL was clipped at 125; the rolling-window size; or what the feature-importance chart shows.
4. State **one** limitation honestly and specifically (e.g. “trained on a simulation of one fault mode in one operating condition; deploying on a real engine would need retraining on actual failure data and validation against known events”).

Be ready for questions such as: what does RMSE 22 cycles mean operationally? walk through exactly what happens when a user uploads a CSV;

why can't Random Forest feature importance tell the maintenance team which physical sensor to buy? name three things needed before recommending this for deployment on the real CNC cells. "I don't know, but I would check..." is a professional answer; guessing is not.

19.14.5 Deliverables & submission

The live presentation. No file submission — but your app URL and GitHub repo (submitted in Lab 11) must still be live and working at presentation time.

19.14.6 Rubric

Capstone evaluation rubric — Labs 9–12. The 100 points below are the marking scheme for the four capstone sessions together; how they map onto the course grade is maintained in FOL and the instructor handbook, not here.

Working predictive model (30 points)

Score	Criteria
27–30	Validation RMSE ≤ 22 cycles; beats naive baseline $\geq 2\times$; engine-wise split, verified with assert; RUL clipped at 125
21–26	RMSE 22–30 cycles; beats naive; split is engine-wise
15–20	Model trains and predicts; RMSE > 30 , or no naive comparison
0–14	Does not train; or row-wise split (data leak); or RMSE near naive

Code quality (20 points)

Score	Criteria
18–20	Clean, commented, organised; rolling features grouped by engine; random seed set; <code>feature_cols</code> saved with the model
14–17	Mostly clean; minor issues (missing comments, some hardcoded values)
10–13	Works but hard to follow; copy-pasted from an LLM without adaptation; not explainable in the presentation
0–9	Major structural bugs; no comments; student cannot explain their own code

 Warning

LLM policy: using LLMs to help write, debug and understand code is permitted and encouraged. But if in the Lab 12 presentation you cannot explain a non-trivial code block you submitted, up to 20 points may be deducted across all categories. You may use any tool to help you write code; you are responsible for understanding every line you submit.

Streamlit application (25 points)

Score	Criteria
22–25	Public URL loads; handles CSV upload; RUL colour-coded; sensor-trend plot appears; bad input handled gracefully
17–21	Loads and predicts; minor rough edges; bad input may crash unhelpfully
12–16	Loads but predictions look wrong or UI confusing; sample button broken
0–11	Does not deploy; or model fails to load; or predictions nonsensical

Documentation and honest disclosure (15 points)

Score	Criteria
14–15	“About this model” states training-data source and limits; actual validation RMSE; 2 explicit “not suitable for” statements; sample input provided
10–13	Some documentation; limitations mentioned
6–9	Title/description only; no honest limitations
0–5	No documentation; or overclaims accuracy / applicability

Week 6 presentation (10 points)

Score	Criteria
9–10	Live demo works; one technical decision explained clearly; handles 2–3 questions; one specific limitation stated
7–8	Demo works; basic explanation; answers some questions
5–6	Demo works but cannot explain technical decisions
0–4	Demo fails during the presentation; or cannot present own work

Category	Points
Working predictive model	30
Code quality	20
Streamlit application	25
Documentation & honesty	15
Presentation	10
Total	100

19.14.7 Instructor notes

- **Prep:** build the presentation schedule; test the projector with a couple of student URLs beforehand; have your own reference app ready to show if a student's deployment is down.
- **Stretch goals** continue in Lab 13, which reuses this pipeline on the AI4I 2020 dataset. Genuinely optional extras a strong student can take on for bonus or to replace a weaker component: XGBoost comparison, SHAP values, FD002 generalisation, and prediction intervals.
- Weighting for the module and the 100-point capstone is maintained in FOL and the instructor handbook; publish it before the module starts.

19.15 Lab 13 — TPM/OEE, Root Cause and a New Dataset

You bring the whole term together on one problem: a Précitech CNC turning cell that is not making its numbers. You compute **OEE** from a shift log and split the losses into the **Six Big Losses**; you Pareto the failure modes; you run a **5-Why and Ishikawa** root-cause analysis; and then you **direct an LLM** to build a failure classifier on a *new* dataset — the **AI4I 2020 Predictive**

Maintenance Dataset from the UCI repository (CC BY 4.0, 10 000 rows, 5 labelled failure modes). This is *classification* on a differently-shaped dataset, so the C-MAPSS recipe from the capstone does not carry over — the point is to show you can transfer the skill, not copy the code. Tools: **Google Sheets**, **draw.io**, **Google Colab**.

AI use: required for the classifier task; permitted with disclosure elsewhere. As in the ML module, you must be able to explain every line.

19.15.1 Learning outcomes

After this lab you should be able to:

- Compute OEE (availability $\times$ performance $\times$ quality) from a shift log and attribute the losses to the Six Big Losses.
- Build a Pareto chart of failure modes and identify the vital few.
- Run a 5-Why and an Ishikawa (fishbone) analysis using measured process data as evidence.
- Direct an LLM to build and evaluate a classifier on an unfamiliar, class-imbalanced dataset, and choose metrics that survive the imbalance.
- Write the D4 (root cause) and D5 (corrective action) sections of an 8D report, tying model evidence back to the fishbone.

19.15.2 Prep work

- Read Chapter 12 (TPM, OEE, the Six Big Losses) and Chapter 15 (troubleshooting, 5-Why, Ishikawa, the 8D report).
- Skim your capstone notebook from Labs 9–12 — you reuse the *workflow* (load, explore, split, train, evaluate) but not the code.
- Open the AI4I 2020 dataset page and confirm you can download the CSV in the browser. Columns: air temperature, process temperature, rotational speed, torque, tool wear, a machine-failure flag, and five failure-mode flags — **TWF** (tool wear), **HDF** (heat dissipation), **PWF** (power), **OSF** (overstrain), **RNF** (random).
- Download `precitech_shift_log.csv` from FOL (one week of shift records for the CNC cell: planned time, downtime events, ideal cycle time, count, scrap).
- Form teams of 2–3.

19.15.3 Materials & equipment

- Computer with internet access
- **Google Sheets** — OEE calculation and the Pareto chart
- **draw.io** — the Ishikawa diagram
- **Google Colab** — the classifier
- `precitech_shift_log.csv` from FOL
- **AI4I 2020** CSV, downloaded in-browser from UCI

- Chapter 12 OEE definitions and Chapter 15 8D template

19.15.4 Procedure

Task 1 — OEE and the Six Big Losses (25 min). (*2 marks*) From `precitech_shift_log.csv`, compute availability (run time $\div$ planned production time), performance (ideal cycle time $\times$ total count $\div$ run time) and quality (good count $\div$ total count), then OEE = the product. Take each downtime and loss entry in the log and assign it to one of the Six Big Losses (breakdowns, setup & adjustment, small stops, reduced speed, startup rejects, production rejects). State which Big Loss is costing the most OEE points.

Task 2 — Pareto the failure modes (20 min). (*1 mark*) In the AI4I data, count how many rows carry each of TWF, HDF, PWF, OSF and RNF. Build a Pareto chart in Sheets (bars descending, cumulative line). Identify the vital few failure modes that account for most of the failures, and note where RNF sits and why that matters.

Task 3 — 5-Why and Ishikawa (25 min). (*2 marks*) Take the top failure mode from Task 2. Run a 5-Why chain and draw an Ishikawa diagram in draw.io with the six standard bones (machine, method, material, measurement, man, environment). Use the **measured feature columns as evidence** on the bones — for example, compare tool wear, torque, rotational speed and the air-vs-process temperature gap for the failing rows against the healthy rows, and put the numbers on the diagram.

Task 4 — Direct an LLM to build a failure classifier (35 min). (*4 marks*) In Colab, prompt an LLM to build a classifier that predicts the machine-failure flag from the five process features. You are transferring the capstone workflow, not its code — this is tabular classification, not time-series RUL regression. Report a **confusion matrix** and **precision and recall** for the failure class. The failure rate in AI4I 2020 is about **3.4 %**, so **accuracy is a useless metric here** — a model that predicts “no failure” every time scores ~ 96.6 % accuracy and catches nothing. State explicitly which class-imbalance trap you hit and how you detected it. **That is the lesson of this task.**

Task 5 — 8D D4/D5 (15 min). (*1 mark*) Write the “D4 — root cause” and “D5 — permanent corrective action” sections of an 8D report for the top failure mode. Tie the model’s feature importances back to specific bones on your Task 3 fishbone, and propose a corrective action that a TPM programme could actually carry out (an autonomous-maintenance check, a setup standard, a condition limit).

19.15.5 Deliverables & submission

One team report (PDF) with: the OEE calculation and Six-Big-Losses attribution; the failure-mode Pareto chart; the 5-Why chain and the Ishikawa diagram (draw.io export) with the feature evidence on it; the Colab notebook link with

the confusion matrix, precision/recall and the class-imbalance write-up; and the D4/D5 sections. Add an AI-use disclosure line. Submit on FOL by the end of the lab period.

19.15.6 Rubric

10 marks total.

Deliverable	Marks
OEE computed correctly (availability, performance, quality shown separately); every log entry attributed to a Big Loss; largest loss named	2
Failure-mode Pareto chart correct (descending bars, cumulative line); vital few identified; RNF discussed	1
5-Why chain plus a six-bone Ishikawa diagram with measured feature evidence on the bones	2
Classifier built via LLM direction; confusion matrix and precision/recall reported; class-imbalance trap identified and explained; accuracy correctly rejected as the metric	4
D4/D5 sections tie feature importances to the fishbone and propose a TPM-executable corrective action	1

19.15.7 Instructor notes

- **Prep:** generate `precitech_shift_log.csv` with `Rscript R/make_lab_data.R` and stage it on FOL; publish the expected OEE and Pareto order from the answer key the script prints. Host a copy of the AI4I 2020 CSV on FOL as a fallback in case UCI is unreachable from the lab.
- Confirm `archive.ics.uci.edu`, `draw.io` and `Colab` are reachable on the lab network the day before (firewall-check list in Teacher preparation).
- **Marking:** Task 4 is the point of the lab. A team that reports high accuracy and stops has missed it — full marks require them to have *noticed* the imbalance and switched to precision/recall (or a resampling / class-weight fix). Reused C-MAPSS code that does not fit tabular data is a red flag that the transfer did not happen.
- This lab is the destination for the capstone stretch goals — see Lab 12 instructor notes.

19.16 Lab 14 — Robotic Work-Cell Design in RoboDK for Web

A **take-home** lab: released in week 11, due at the end of week 14, done individually, with a short screen recording as part of the submission. You design and program the Précitech kitting cell's pick-and-place robot in **RoboDK for Web** — a free, no-install, in-browser robot simulator with 600+ real arms. It bookends Lab 1: toy logic in week 1, a real 6-axis industrial arm in week 14. You close the loop on the course by checking the robot's cycle time against the **takt time you computed in Lab 4**.

AI use: permitted with disclosure. The station layout, the program and the cycle times must be your own work.

19.16.1 Learning outcomes

After this lab you should be able to:

- Jog a 6-axis robot in joint, linear and tool coordinates and recognise singularities and joint-limit stops.
- Lay out a work cell and verify that every pick and place point lies inside the robot's reach envelope.
- Program a pick-and-place cycle with targets and gripper I/O and read the simulated cycle time.
- Compare a simulated cycle time against a required takt time and optimise the program to close the gap.
- Compute a minimum safeguarding distance from robot stopping performance and select an appropriate safeguard.

19.16.2 Prep work

- Read Chapter 8 (industrial robotics — coordinate systems, the work envelope, singularities) and Chapter 6 (machine guarding — the minimum-distance / safeguarding-distance formula and the safeguard hierarchy).
- Have your **Lab 4** takt calculation to hand — you need the takt time for 250 kits/day.
- Open RoboDK for Web and confirm it loads and runs a sample station in your browser. No account or licence key is required.
- Confirm your screen-recording tool works (the same tool the course plan's take-home lab uses).

19.16.3 Materials & equipment

- Personal computer with internet access and a browser (this is a take-home lab — no lab PC required)
- RoboDK for Web — no install, no licence key
- A screen-recording tool

- Your Lab 4 takt time; the Précitech case study (250 kits/day, one 8-hour shift)

19.16.4 Procedure

Task 1 — Jog and observe (20 min). (1 mark) Load a 6-axis arm from the RoboDK library (any mid-size industrial arm; state which). Jog it in **joint**, **linear** and **tool** coordinates. Drive it toward a wrist singularity and toward a joint limit and record what the simulator does in each case. Relate this back to the teach-pendant work in Lab 1 and to the Chapter 8 coordinate systems.

Task 2 — Build the kitting cell (30 min). (2 marks) Assemble the Précitech kitting cell around the robot: an infeed conveyor delivering shaft blanks, a bearing-press station, and a tray to be filled. Place a pick point and every place point. Check that **every pick and place point is inside the reach envelope**; record the one reach limit that forces you to move a station, and show the before/after positions.

Task 3 — Program the pick-and-place (30 min). (3 marks) Create targets and a program that picks a blank from the conveyor, moves it to the press, then places the finished part in the tray, with gripper open/close on digital output. Run it. Record the **simulated cycle time** for one part.

Task 4 — Cycle time vs takt (25 min). (3 marks) Take the **takt time from Lab 4** for 250 kits/day. Does one robot at your Task 3 cycle time meet takt? Then optimise — reorder targets, raise joint speeds within limits, shorten approach and retract moves — and record the improved cycle time. State by how much you beat or missed takt, and what a real cell would do if one robot could not keep up.

Task 5 — Safeguarding distance (15 min). (1 mark) Using the Chapter 6 minimum-distance formula, compute the minimum guarding distance from an assumed robot-plus-sensor stopping time (state your assumption). State which Chapter 6 safeguard you would specify for this cell (fixed guard, interlocked gate, light curtain, area scanner, two-hand control) and why.

19.16.5 Deliverables & submission

Submit on FOL by the end of week 14: a report (PDF) covering all five tasks with the reach-envelope evidence, the cycle-time and takt comparison, and the safeguarding calculation; **the RoboDK station file**; and a **screen recording (3–5 min)** showing the program running and you talking through one design decision. Add an AI-use disclosure line.

19.16.6 Rubric

10 marks total.

Deliverable	Marks
Jogging in joint / linear / tool coordinates demonstrated; singularity and joint-limit behaviour recorded	1
Kitting cell built; every pick/place point verified inside the reach envelope; the binding reach limit and the layout change recorded	2
Working pick-and-place program with targets and gripper I/O; runs in the simulator; cycle time reported	3
Cycle time compared against the Lab 4 takt; one round of optimisation with the improved time and the gap quantified	3
Minimum safeguarding distance computed from a stated stopping time; a specific Chapter 6 safeguard chosen and justified	1

19.16.7 Instructor notes

- **Release in week 11** alongside Lab 11, so students have the full three weeks; it is due at the end of week 14. This mirrors the course plan’s existing take-home lab pattern (released three weeks out, video submission).
- **Prep:** the day before release, run through Tasks 1–3 in RoboDK for Web yourself on the college network — confirm `web.robodk.com` is reachable, the library loads, and a station can be saved and re-loaded. Have a reference station file and an expected cycle-time range (from the answer key in `R/make_lab_data.R` output) ready for marking.
- No lab room is needed — this is done from home — but reserve 15 minutes in the week-11 lab to demo the tool and walk through the brief.
- **Marking:** the takt comparison in Task 4 only works if the student’s Lab 4 takt was correct; if it was not, mark Task 4 on the *method* against a takt you supply. Watch for cycle times that are physically implausible (joint speeds cranked past limits) — that is where the “your own work” line is tested.

19.17 Teacher preparation

Everything the instructor needs to prep the term: the room, equipment and software per lab, then a consolidated shopping list, a software/accounts checklist,

and a room-booking checklist. Items marked **[confirm]** are not stated in the source handouts.

The full term-prep material — quickstart, term grid, grade-item checklist, paste-ready Course Plan notes, the AI-use policy, answer keys, all 14 rubrics and the per-lab failure modes — lives in the **instructor handbook** (`instructor/instructor-guide.qmd`, renders to `instructor/instructor-guide.pdf` for FOL). The matrix below is the quick per-lab view; the handbook is the authoritative version.

19.17.1 Per-lab prep matrix

Lab (week)	Room	Equipment (reusable)	Software / online	Consumables & printing
1 (wk 1) — LightBot & Rocks	Computer lab, 1 PC/student [confirm]	PCs/tablets with internet	LightBot (browser), Rocks (browser); prelab YouTube 9oAFmT1Mqzg	Print worksheet (optional)
2 (wk 2) — Design docu- mentation	Computer lab with full Office [confirm]	PCs	Microsoft Word/Visio, Microsoft Project, Microsoft Excel	None; post the Précitech case study + re-upload FOL templates: BOM .xlsx, Gantt .mpp, HSE .docx
3 (wk 3) — Facility design & AnyLogic	Computer lab [confirm]	PCs (check AnyLogic PLE model-size limit)	AnyLogic Personal Learning Edition, Microsoft Vi- sio/Project/Excel; AnyLogic order- fulfilment demo model [confirm which] + prelab video	None

Lab (week)	Room	Equipment (reusable)	Software / online	Consumables & printing
4 (wk 4) — Lean VSM	Computer lab with internet	PCs	draw.io (browser), Google Sheets , AnyLogic Cloud (free public tier) — all browser, no install	None; generate + stage <code>vsm_station_data.csv</code> (run <code>Rscript R/make_lab_data.R</code>); build/fork the AnyLogic Cloud push-pull model and expose its input parameters Print REBA worksheets + REBA cheat sheet (per student); stage task clips/photos (Lifting Boxes, Workstation, Cleaning) on FOL
5 (wk 5) — Ergonomics	Computer lab with internet [confirm]	PCs	Inseer web tool (class accounts/login — arrange early); browser; ergonomics + REBA videos	
6 (wk 6) — SPC & Gauge R&R	Computer lab, 1 PC/group of 3	PCs with internet (to load the book page)	This book's webR cells only — no external site, no account, no dataset. Just confirm the rendered <code>30-Labs.html</code> page and its webR cells load	None; print the Lab 6 answer key from <code>Rscript R/make_lab_data.R</code> for marking

Lab (week)	Room	Equipment (reusable)	Software / online	Consumables & printing
7 (wk 7) — ML Wk1: AI literacy	Computer lab, 1 PC/student	PCs with internet	Claude.ai or ChatGPT (free tier); Google Colab	None; post FMEA table + blank <code>prompts.md</code> on FOL. Midterm is in lecture time this week — the lab runs as normal
8 (wk 8) — ML Wk2: data & SPC	Computer lab	PCs with internet	Google Colab ; Python (pandas, numpy, matplotlib)	None; stage <code>spc_data.csv</code> + <code>week2_lab.ipynb</code> on FOL
9 (wk 9) — ML Wk3: C-MAPSS	Computer lab	PCs with internet	Google Colab ; scikit-learn stack	None; stage <code>train_FD001.txt</code> + <code>predictive_maintenance.ipynb</code> on FOL
10 (wk 10) — ML Wk4: model build	Computer lab	PCs with internet	Google Colab ; scikit-learn, joblib	None; have a reference solution notebook (expected RMSE 21)
11 (wk 11) — ML Wk5: deploy	Computer lab	PCs with internet	GitHub + Streamlit Community Cloud accounts; Google Colab	None; stage <code>streamlit_app.py</code> + tested <code>requirements.txt</code> on FOL. Release Lab 14 this week
12 (wk 12) — ML Wk6: presentations	Room with projector/display + internet [confirm]	Projector; PCs	Student public Streamlit URLs	None; presentation schedule

Lab (week)	Room	Equipment (reusable)	Software / online	Consumables & printing
13 (wk 13) — TPM/OEE & RCA	Computer lab with internet	PCs	Google Sheets, draw.io, Google Colab; AI4I 2020 CSV from archive.ics.uci.edu/ml/machine-learning-databases/ai4i/	None; generate + stage <code>precitech_shift_log.csv</code> (run <code>Rscript R/make_lab_data.R</code>); host an AI4I copy on FOL as a fallback
14 (wk 14) — RoboDK work cell (<i>take-home</i>)	None — done from home. Reserve 15 min in the wk-11 lab to demo the tool	Student's own computer	RoboDK for Web (web.robodk.com/simulation/) — no install, no licence key; a screen-recording tool	None; provide a station file and an expected cycle-time range

19.17.2 Consolidated shopping list (buy / restock each term)

The current lab set uses **no physical consumables** — the removed bridge and lean-simulation labs were the only ones that did. If either is reinstated from `LabInstructions/`, restore its shopping list from the previous revision of this chapter (git history) or the original handouts.

Printing (per term)

- ☐ Lab 5 REBA worksheets + REBA cheat sheet (one per student)
- ☐ Lab 2 Précitech case-study handout + document templates
- ☐ Lab 1 worksheet (optional)
- ☐ Lab 7 FMEA table (from Chapter Chapter 13, Week 1)
- ☐ Lab 6 answer key (caliper reference values, expected %GRR / ndc / Cpk) — for marking only

Reusable — check stock

- ☐ Projector / display for Lab 12 presentations

19.17.3 Software, licences & accounts — set up before the term

- ☐ **AnyLogic Personal Learning Edition** installed on every Lab 3 PC; run a full test of the demo model (watch the PLE size limit)
- ☐ **Microsoft Project** and **Microsoft Visio** licences live on Lab 2 / Lab 3 PCs (most likely gap in a standard Office image)
- ☐ **Microsoft Excel / Word** available on Lab 2 / Lab 3 PCs
- ☐ **Inseer** class accounts / login created and tested (Lab 5) — confirm seat count and upload workflow
- ☐ **LightBot** and **Rocksi** confirmed reachable through the lab network/firewall (Lab 1); offline fallback prepared
- ☐ **draw.io** (app.diagrams.net) reachable on the lab network (Labs 4, 13)
- ☐ **AnyLogic Cloud** (cloud.anylogic.com) reachable, and the push-pull model published with editable input parameters (Lab 4)
- ☐ **Google Sheets** reachable (Labs 4, 13)
- ☐ **UCI ML Repository** (archive.ics.uci.edu) reachable for the AI4I 2020 download, with a FOL fallback copy staged (Lab 13)
- ☐ **RoboDK for Web** (web.robodk.com/simulation) reachable; run Tasks 1–3 through the college firewall before releasing Lab 14
- ☐ **Lab 6 needs no external service** — only confirm the rendered book page and its webR cells load in the lab browsers
- ☐ **Google Colab** (colab.research.google.com) reachable on the lab network (Labs 8–11, 13)
- ☐ **Claude.ai / ChatGPT** reachable on the lab network (Labs 7–8, 13)
- ☐ **GitHub** (github.com) and **Streamlit Community Cloud** (share.streamlit.io) reachable (Lab 11); students create accounts as prep
- ☐ FOL templates re-uploaded: BOM .xlsx, Gantt .mpp, HSE .docx (Lab 2)
- ☐ Lab data generated and staged on FOL — run `Rscript R/make_lab_data.R: vsm_station_data.csv` (Lab 4), `precitech_shift_log.csv` (Lab 13); notebooks and datasets `spc_data.csv` + `week2_lab.ipynb` (Lab 8), `train_FD001.txt` + `predictive_maintenance.ipynb` (Lab 9), `streamlit_app.py` + `requirements.txt` (Lab 11)
- ☐ Task clips / photo sets staged on FOL for Lab 5 (Lifting Boxes, Workstation, Cleaning)
- ☐ Prelab video links checked for Labs 1, 3, 5
- ☐ Grade items verified in FOL against the instructor handbook's grade-item checklist (34 items, 6 categories; Lab-13 and Lab-14 added; take-home availability window on Lab-14)

19.17.4 Room bookings

- ☐ **Computer lab** with internet + full Microsoft Office + Project + Visio + AnyLogic PLE — Labs 1–13 (Labs 2–3 need Project/Visio; Labs 4, 6–13 need only a browser and internet)

- **Room with a projector/display** for Lab 12 capstone presentations
[confirm — the computer lab may suffice]
- **No room for Lab 14** — it is a take-home lab; only the 15-minute demo slot in the week-11 lab is needed

19.17.5 Appendix: instrumentation-trainer lab stream

An older set of labs for this course code was written for the **B1033** trainer benches (author “MZhang”). They are **not** part of the current lab set, but the handouts are in the **LabInstructions/** folder and are summarised here in case the course runs them:

Alt lab	Title	Room	Equipment	Software
Alt — Pres- sure/level	Pressure Measurement for Level	B1033	Trainer: power-supply module, pump/tank unit, RTD temperature transmitter, pressure transmitter, float/reed switch, 2 × DMM, banana leads	—
Alt — Motor control	Motor Control & Intro to VFDs	B1033	3-phase motor, FVNR (full-voltage non- reversing) starters, control- relay/MCR wiring, E-stop, pilot lights, pushbuttons; VFD	—

Alt lab	Title	Room	Equipment	Software
Alt — PLC	PLC Introduction with Siemens	B1033	Siemens PLC trainer, field devices (pushbuttons, selector switches, pilot lights, cylinders with proxy sensors), potentiometer 0–10 VDC, DMM	Siemens TIA Portal ; reference video Y0vQhvjaeqY
Alt — Flow rate	Flow Rate Measurement	B1033	Trainer: pump/tank unit, paddle-wheel flow transmitter, rotameter (LPM/GPM), RTD temperature transmitter, 2 × DMM, hoses/banana leads	—

Prep for the alt stream: book **B1033**; confirm every trainer bench has a working power-supply module, pump/tank unit and the two DMMs; confirm **Siemens TIA Portal** licences for the PLC lab; print each handout's instructor sign-off sheet (4 sign-offs per lab).

Bibliography